



GeoLibre 레퍼런스

아키텍처, 렌더러, 배포, API, 프로젝트 문서

2026-10-08 GeoLibre 공식 문서를 바탕으로 생성됨

<https://geolibre.app> · github.com/opengeos/GeoLibre

목차

아키텍처	10
개요	10
패키지	10
상태 흐름	11
렌더링 엔진 모델	12
3D 지구본 뷰(CesiumJS)	12
DuckDB-WASM	16
고급 Add Data 워크플로	18
Python 사이드카	18
오프라인 지원(PWA)	19
컨테이너 이미지	21
보안	22
성능: Linux(WebKitGTK)에서의 지도 렌더링	24
MapLibre 렌더러	26
지원되는 경로	26
맵, 지구본, 지형	26
베이스맵과 플러그인	26
아키텍처와 라이선스	27
Mapbox 렌더러	28
지원되는 경로	28
플러그인	29
라이선스 및 약관	32
로딩 및 크기	33
Add Data 호환성	33
Cesium 렌더러	37
자격 증명	37
지원되는 경로	37
장면과 카메라	38
로딩과 아키텍처	38
ArcGIS 렌더러	39
아무것도 번들되지 않습니다	39
API 키	39
지원되는 경로	40
2D 및 3D	42
데이터 추가	43

deck.gl 레이어.....	44
타일 아카이브.....	44
Zarr 및 NetCDF 그리드.....	45
플러그인 컨트롤.....	45
적응된 플러그인 패널.....	46
아직 지원되지 않는 항목	46
테스트.....	46
라이선스 및 약관	47
셀프 호스팅	48
동일 출처가 중요한 이유	48
아키텍처	49
1. GeoLens로 데이터 호스팅하기.....	49
2. GeoLibre 웹 빌드를 그 옆에 호스팅하기	50
3. GeoLibre를 GeoLens에 연결하기.....	56
두 가지 배포 패턴 비교.....	57
GeoLens 없이 비공개 데이터 다루기	59
외부 요청 줄이기	59
배포 체크리스트	60
관련 페이지.....	61
서버 API.....	62
규약.....	62
참조 서버가 운영자에게 맡기는 부분.....	62
제한.....	63
가시성.....	64
신원.....	64
조직.....	67
그룹.....	68
엔터프라이즈 로그인	70
프로젝트	78
개인 토큰 스코프	85
OAuth 2.0 로그인(Authorization Code + S256 PKCE)	85
호환성.....	87
Android	88
설치.....	88
Android와 데스크톱에서 동작하는 기능 비교	88
툴체인 설정(최초 한 번).....	89
빌드.....	90
서명.....	90
지속적 통합.....	91
설치 / 테스트	92

Google Play에 게시	92
알려진 제한 사항 / 후속 작업.....	93
다른 앱에서 위치 열기	93
iOS	95
설치.....	95
iOS와 데스크톱에서 동작하는 기능 비교	95
위치 권한(필수)	96
최소 iOS 버전.....	97
틀체인 설정(최초 한 번).....	98
빌드.....	98
서명.....	99
지속적 통합.....	100
로컬 빌드 실행 및 테스트.....	102
App Store에 게시	102
알려진 제한 사항 / 후속 작업.....	103
Mac App Store	104
MAS 빌드가 제거하는 것	104
로컬에서 빌드.....	106
CI: mas-store.yml 워크플로	107
제출.....	108
프로젝트 형식	109
데스크톱에서 파일 열기.....	109
스키마.....	109
플러그인 상태	110
범례.....	111
인쇄 레이아웃	111
스토리 맵	112
대시보드 위젯.....	113
북마크.....	113
상호작용	114
레이어 객체.....	114
레이어 타입.....	120
API	121
자격 증명 삭제	121
플러그인 API	123
인터페이스	123
플러그인 등록.....	129
내장 플러그인 추가 (이 저장소에서)	129
내장 플러그인	131

예제 플러그인	132
읽기 전용 레이어 및 피쳐 조회	134
활성 지도 클릭 도구	135
레이어 그룹	135
레이어 스타일	136
래스터 및 타일 레이어	137
Zarr 레이어	140
Time Slider에서 레이어 자체의 시간 차원 구동하기	143
다른 플러그인 활성화 및 비활성화	144
커스텀(WebGL) 레이어와 paint 소유권	144
오른쪽 사이드바 패널	145
툴바 메뉴	147
내장 메뉴에 항목 추가	148
앱 언어 따르기	149
자격 증명 저장	150
세션 쿠키로 로그인	151
플로팅 패널	152
외부 플러그인	152
어시스턴트 도구	157
지도 렌더러	159
협업	164
동기화되는 항목	164
아키텍처	164
동기화 프로토콜	165
Durable Object(workers/collab)	167
프런트엔드	168
신원 및 권한(MVP)	168
채팅(#754)	170
기능 플래그	171
릴레이 배포(collab.geolibre.app)	171
제한 사항 / v2	172
테스트	172
UI 프로필	175
사용자를 위한 안내	175
관리자를 위한 안내	176
배포 기능	178
UI 프로필과는 다릅니다	178
기능 목록	179
구성 방법	179
임베드 API 동작	180

관련 페이지.....	181
배포 정책	182
로딩.....	182
예시.....	183
필드 참조	184
생략과 빈 값의 차이	186
플러그인 우선순위	186
소스 간 우선순위	187
버전 관리	187
검증.....	187
데스크톱	187
Docker.....	188
국제화	190
동작 방식	190
컴포넌트에서 번역 사용하기	191
새 언어 추가하기	192
카탈로그를 완전하게 유지하기	192
오른쪽에서 왼쪽으로 쓰는 언어.....	193
로케일을 인식하는 서식 유지하기	193
처리 도구 메타데이터.....	194
플러그인 관련 표면.....	195
알려진 제한 사항	196
Python 패키지	197
설치.....	197
빠른 시작	197
양방향 동기화.....	199
지도 옵션	200
인터랙티브 스크립팅	200
API 참조	201
marimo에서 사용하기	212
동작 원리	213
MCP 서버	214
소스에서 빌드하기	215
렌더링 엔진과 혼합 창(pane) 레이아웃	215
R 패키지	216
설치.....	216
빠른 시작	216
sf를 이용한 공간 데이터	217
원격 래스터.....	217

프로젝트 파일	217
Shiny	217
자체 호스팅	218
MCP 서버	219
설치	219
실행하기	219
워크스페이스	220
도구	220
참고 사항과 제한	224
에이전트에서 서버 제어하기	224
내부 구조	225
렌더러와 패널 작성	225
에이전트 스킴	226
용도	226
가르치는 내용	226
설치	227
다른 에이전트 런타임	227
노트북 패널	228
셀에서 지도 스크립팅하기	228
외부 클라이언트(VS Code 등)에서 지도 제어하기	229
테마	230
웹 JupyterLite 사이트 빌드하기	230
데스크톱 서버	231
로드맵	232
v0.1: 지도 뷰어와 GeoJSON	232
v0.2: 프로젝트 영속성	232
v0.3: 클라우드 네이티브 포맷	232
v0.4: DuckDB Spatial	232
v0.5: 고급 Add Data 및 플러그인 기반 레이어	232
v0.6: 프로젝트 접근, 웹 임베드, 확장된 통합	233
v0.7: Add Data 확장, 식별, 설정, 처리	233
v0.8: 뷰어, 데스크톱 패키징, 플러그인, 동적 레이어	233
v0.9: 데이터 통합, 처리, 메뉴 재구성	234
v1.0: 처리 파이프라인, 외부 플러그인 시스템, 안정적인 프로토타입	235
v1.1: 벡터 스타일링, 속성 테이블 관리, 대기 효과	235
v1.2: 새 데이터 소스, 속성 분석, 라우팅, 플랫폼 다듬기	236
v1.3: 분석 심화, 실시간 협업, 스토리 맵, 스크립팅, AI 어시스턴트	236
v1.4: 지도 옆의 Jupyter, 분광 지수, 지리참조, 현장 수집	238
v1.5: 대시보드, 인브라우저 Whitebox, 지도 탐색 기록, 서명된 macOS 설치 프로그램	238

v1.6: 다중 지도 레이아웃, 고급 심볼로지와 라벨링, 플러그인 zip 설치	239
v1.7: 플러그인 UI 호스트 API, 색상 램프 미리보기, 범주별 탐색 Whitebox 도구	240
v1.8: 카메라 투어, 라이브 스토리 맵, 독립 실행형 HTML 내보내기, 지도 주석	240
v1.9: CAD 가져오기, 더 똑똑한 서비스 검색, 도킹된 SQL 워크스페이스	241
v1.10: I3S 씬 레이어, OGC API 벡터 타일, 로컬 NetCDF/HDF 불러오기	242
v2.0: 3D 지구본, 행성 지도 제작, 심볼로지 교환, 편집 가능한 소스 레이어	242
v2.1: 데이터 소스 Browser 패널, 경로 애니메이션, 인브라우저 객체 탐지, 지도 녹화	243
v2.2: 스타일링 개편, 표현식 기반 필드와 라벨, 인쇄 아틀라스, 브라우저 네이티브 변환	244
v2.3: 지도 위 Legend 패널, iOS와 Google Play, GeoLens 카탈로그, 시공간 핫스팟	245
v2.4: STAC 및 Earthdata 카탈로그, 자유 비행 카메라, 타일과 데이터 큐브용 타임라인, 라이브 임베드 API	246
v2.5: QGIS 및 ArcGIS Pro 프로젝트 가져오기, 이산 전역 격자, 초분광 큐브, 자체 호스팅 가능한 서버	247
v2.6: 데이터 및 스타일 딥 링크, AI 클라이언트용 MCP 서버, COG 기반 지형, 자체 디자인 시스템	249
v2.7: 비주얼 Model Builder, 모든 종류의 STAC 자산, Chrome 데이터 열기 확장, 그리기로 선택	251
v2.8: 팝업 디자이너, 빠른 필터, 레이어별 권한, npm에 게시된 패키지	253
v2.9: 토큰 없는 3D 지구본, 키오스크 및 교실 배포, 도킹된 서비스 패널, 키 없는 전역 고도	254
v3.0: 일급 렌더링 엔진으로서의 Cesium, 지구본 위의 동적 3D 씬, ArcGIS 쓰기 저장, 편집 가능한 SVG 인쇄 레이아웃	256
v3.1: 렌더링 엔진에 합류한 Mapbox와 ArcGIS, 라이브 God's Eye View 지구본, 공유를 위한 OAuth 로그인, 음성 구동 AI Assistant	257
v3.2: 포인트 클라우드 어노테이션, 비공개 S3 버킷, OS 키체인에 저장되는 자격 증명, 미국 모든 주의 GIS 포털	259
v3.3: 가시선, MGRS 격자와 좌표, SQL Server, 비교 및 복원이 가능한 프로젝트 이력, 그리고 도킹하고 검증하며 메뉴를 확장하는 플러그인 생태계 (현재)	260
플러그인 마켓플레이스와 레지스트리 (설계)	261
기여하기	264
기여 방법	264
사전 요구 사항	264
설정	264
로컬에서 실행하기	265
저장소 구조	265
개발 워크플로	265
품질 검사	266
문서	271
플러그인과 확장	272
백엔드 사이드카	272
후원	272
라이선스	272
유지 관리 노트	273
수동 확인이 필요한 의존성 범프	273
공유 의존성 범위	289
블렌드 모드 추가하기	289
커버리지 하한	290

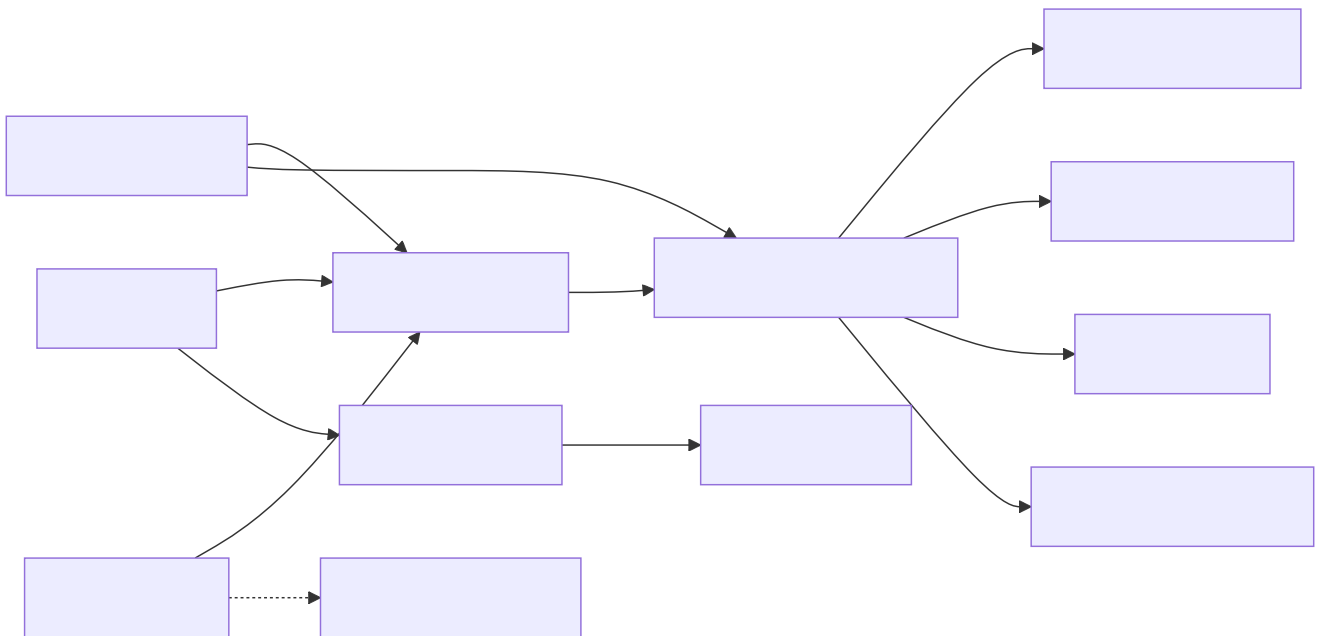
린트 경고 래치	291
테스트 타입 검사 래치	292
부트 번들 예산	293
의존성 업데이트와 감사 허용 목록	294
@geolibre/core 와 @geolibre/map 게시	294
번들된 사이드카 잠금 파일	295
자격 증명은 재배포 가능한 빌드에 절대 들어가서는 안 됩니다	295
생성 파일과 파일 간 동기화	297
인용 방법	299
권장 인용	299
BibTeX	299
APA	299
인용 메타데이터 파일	300
의존성 인용	300
감사의 글	301
오픈 소스 프로젝트	301
커뮤니티 기여자	302
참여하기	302
개인정보 처리방침	303
요약	303
로컬에서 처리되는 데이터	303
제3자 서비스로 전송되는 데이터(선택적 기능)	303
플러그인 레지스트리의 플러그인	304
웹사이트 분석	304
개인 정보	305
아동	305
사용자의 선택	305
이 방침의 변경	305
문의	306
서비스 약관	307
서비스	307
라이선스	307
허용되는 사용	307
Google Earth Engine 및 기타 제3자 서비스	308
귀하의 콘텐츠	308
보증의 부인	308
책임의 제한	308
앱 및 본 약관의 변경	309
문의	309

<https://geolibre.app/architecture/>

GeoLibre 아키텍처

개요

GeoLibre는 무료 오픈 소스의 경량 클라우드 네이티브 GIS 플랫폼으로, 웹 브라우저, 데스크톱, 모바일, Jupyter 노트북 안에서 모두 실행되며, 이 모든 것이 하나의 npm workspaces 모노레포에서 나옵니다. UI는 React 앱으로, Tauri v2가 호스팅하는 네이티브 데스크톱 앱과 브라우저 기반 웹 앱으로 배포되며, 모바일과 작은 화면에 반응형으로 적응합니다. 네 가지 엔진이 지도를 렌더링할 수 있습니다: MapLibre GL JS(기본값), Mapbox GL JS, CesiumJS, ArcGIS Maps SDK for JavaScript입니다. deck.gl은 엔진이 지원하는 곳에서 고급 래스터, 포인트 클라우드, 3D 오버레이를 제공합니다. 애플리케이션 상태는 Zustand 스토어([@geolibre/core](#))에 있습니다. 사용자용 [렌더링 엔진 가이드](#)와 상세한 [MapLibre](#), [Mapbox](#), [Cesium](#), [ArcGIS](#) 레퍼런스를 참고하세요.



패키지

패키지	책임
@geolibre/core	도메인 타입, 프로젝트 JSON 스키마, 전역 스토어
@geolibre/map	렌더링 엔진 생명주기와 어댑터, 레이어 동기화, GeoJSON, 래스터, 타일, MBTiles, 컨트롤, 선택 스타일링
@geolibre/ui	공유 UI 프리미티브(shadcn 스타일)
@geolibre/processing	클라이언트 측 알고리즘 레지스트리
@geolibre/plugins	플러그인 인터페이스와 내장 플러그인

패키지	책임
@geolibre/embed	iframe 임베드 API용 타입이 지정된 의존성 없는 클라이언트, npm에 게시됨
@geolibre/collab-core	workers/collab 와 workers/collab-node 릴레이가 공유하는 전송 중립적 협업 릴레이 코어(와이어 프로토콜, 권한, 페이로드 검증기)
geolibre-desktop	셀 레이아웃, Tauri I/O, 컴포지션

상태 흐름

1. 사용자가 Add Data(데이터 추가) 메뉴, Tauri 대화 상자, 브라우저 파일 선택기, 드래그 앤 드롭, 또는 내장 플러그인 컨트롤을 통해 데이터를 추가합니다.
2. 로컬 벡터 데이터는 직접 파싱되거나 DuckDB-WASM Spatial로 GeoJSON으로 변환된 뒤 스토어의 `addGeoJsonLayer`에 전달됩니다.
3. 타일, 서비스, 래스터, ArcGIS, MBTiles, 플러그인 기반 레이어는 소스 메타데이터와 (해당하는 경우) 네이티브 MapLibre 레이어 id를 가진 `GeoLibreLayer` 레코드를 생성합니다.
4. 선택된 `MapEngine`이 `layers`를 구독하고 자신의 SDK와 조정(reconcile)합니다. MapLibre는 `MapController.syncLayers`를 사용하고, Mapbox, Cesium, ArcGIS는 각 엔진 전용 어댑터를 사용합니다. 두 Style-Spec 엔진이 공유하는 레이어 컨트롤 연결 코드는 `packages/map/src/layer-control-host.ts`에 있습니다.
5. 스타일 패널과 레이어 패널의 업데이트가 레이어 상태를 변경하면, 지도 동기화가 페인트, 가시성, 불투명도, 순서, 제거를 반영합니다.
6. 속성 테이블 선택은 강조 표시된 피쳐 소스를 업데이트하고 선택된 피쳐로 지도를 줌할 수 있습니다.
7. 데스크톱 저장은 `projectFromStore`를 사용하여 `.geolibre`를 디스크에 기록합니다. 이전의 `.geolibre.json` 이름도 계속 읽을 수 있습니다.

모든 레이어 작업은 `layers` 배열을 교체하므로, `useAppStore((s) => s.layers)`는 어떤 레이어의 어떤 편집에도 컴포넌트를 다시 렌더링합니다. 모든 레이어의 모든 필드가 정말로 필요한 구독자(지도 캔버스의 동기화, Layers 패널)만 그렇게 해야 합니다. 그 외에는 `packages/core/src/layer-selectors.ts`의 좁은 훅을 사용합니다: 레코드 하나에는 `useLayer(id)`, id 목록에는 `useLayerIds()`, `{ id, name, type, visible, groupId }` 행에는 `useLayerSummaries()`, 그리고 항상 마운트되어 있지만 열려 있는 동안에만 전체 배열이 필요한 대화 상자와 패널에는 `useLayersWhen(open)`을 사용합니다.

`useAppStore`는 하나의 Zustand 스토어이지만, 코드는 `packages/core/src/store/` 아래의 슬라이스(project, map view, layers, layer groups, project content, processing, libraries, session, collaboration, capabilities, 그리고 ui 대화 상자 플래그)로 분할되어 `packages/core/src/store.ts`에서 합성됩니다. 새 필드나 액션은 그 데이터를 소유하는 슬라이스에 추가하세요. 다른 슬라이스의 필드도 함께 업데이트해야 하는 액션(`removeLayer`가 모든 곳의 참조를 정리하는 것처럼)은 `set/get`을 통해 전체 상태를 읽고 씁니다. 실행 취소 기록(`store/undo-history.ts`)은 `partializeHistory`가 나열하는 필드(레이어, 레이어 그룹, 베이스맵, 스토리맵, 댓글, 대시보드 위젯, 범례)만 추적합니다. 그 외의 모든 것, 즉 전체 ui 하위 상태, 선택, 카메라는 실행 취소 단계를 기록하지 않으며 실행 취소나 다시 실행으로 되돌려지지 않습니다. 보조 패인과 Print Layout도 레이어를 참조

하지만 추적되지 않습니다(패인은 자체 카메라를 가지며 추적되지 않는 그리드 레이아웃과 일치해야 하고, 컴포저는 로컬 사본을 다시 기록합니다). 따라서 각 실행 취소/다시 실행 단계는 그 단계가 제거한 레이어에 대한 참조를 이들에 정리합니다.

렌더링 엔진 모델

Zustand 스토어는 렌더러 중립적입니다. 매핑 SDK가 소유하는 객체가 아니라 일반 `GeoLibreLayer` 레코드, 환경 설정, `MapViewState` 값을 담습니다. `MapLibre`, `Mapbox`, `Cesium`, `ArcGIS`는 각각 카메라 작업, 기능 확인, 피킹, 컨트롤, 배치, 범위 그리기, 캡처를 위한 공유 `MapEngine` 인터페이스를 구현합니다. `primaryRenderer` 는 주 캔버스를 선택하고, 각 `SecondaryMapView.viewKind` 는 분할 패인의 엔진을 선택합니다. 어느 값을 바꾸든 이전 엔진이 언마운트되고 교체 엔진이 같은 프로젝트 상태에 연결됩니다.

기능(capability)은 가정되지 않고 명시적으로 선언됩니다. 셀은 활성 `MapEngine` 에 대해 명령을 게이트하고, 레이어는 엔진 호환성을 보고하며, 플러그인은 `engines` 목록을 선언합니다. 덕분에 모든 SDK에 모든 기능을 에뮬레이션하도록 요구하지 않고도, 프로젝트는 전환 중에 지원되지 않는 레이어나 일시 중단된 플러그인을 유지할 수 있습니다.

`packages/map/src` 안에서 엔진 이름이 붙은 파일(`arcgis-*`, `cesium-*`, `mapbox-*`, 그리고 `MapLibre`의 `layer-sync`, `map-controller`, `MapCanvas`)은 다른 엔진의 파일을 절대 import하지 않습니다. 두 엔진이 공유하는 코드는 하는 일을 이름으로 삼은 중립 모듈(`pmtiles-archive.ts`, `protocol-tiles.ts`, `gl-style-compiler.ts`, `feature-style.ts` 등)에 있습니다. `local/no-cross-engine-imports` ESLint 규칙 (`eslint-rules/no-cross-engine-imports.mjs`)이 이 경계를 강제하고 분류를 나열합니다.

3D 지구본 뷰(CesiumJS)

기본 `MapLibre` 지도는 `CesiumJS`로 렌더링되는 3D 지구본과 나란히 놓이거나 그것으로 대체될 수 있습니다. `Cesium`은 주 캔버스를 소유하든 혼합 엔진 그리드의 패인 하나를 소유하든, 위에서 설명한 동일한 엔진 중립 스토어를 구독합니다. `MapLibre`는 여전히 기본값이며, `MapLibre` 스타일 API를 통해 직접 그리는 통합 기능은 계속 `MapLibre`를 필요로 합니다.

- **렌더링 위치.** 두 곳이며, 둘 다 같은 스토어 상태를 그림니다:
- **보조 패인으로.** 지도 그리드의 각 패인에는 2D/3D 토글(`SecondaryMapView.viewKind`)이 있어, 지구본을 2D 지도 옆에 두고 비교할 수 있습니다.
- **주 지도로(이슈 #2217).** **View → Rendering engine → Cesium**(보기 → 렌더링 엔진 → Cesium)은 스토어의 `primaryRenderer` 를 통해 지구본을 선택합니다. `DesktopShell` 은 이전 엔진의 캔버스 자리에 `PrimaryCesiumCanvas` 를 마운트하므로, 그 렌더러는 단순히 숨겨지는 것이 아니라 언마운트되어 GPU 자원 소비를 멈춥니다. 전환은 패인을 추가하거나 제거하지 않으며, 다중 패인 그리드는 여전히 네 엔진을 모두 섞을 수 있습니다. `CesiumCanvas` 는 `viewId` 가 주어졌는지 여부로 두 역할을 구분합니다. `viewId` 가 있으면 `secondaryMapViews` 레코드가 뒷받침하는 패인(자체 카메라, 자체 가시성 재정의)이고, 없으면 그것이

곧 주 지도로서 공유 `mapView` 를 직접 읽고 쓰며 `syncView` 토글(패인이 주 카메라를 따르게 하기 위해 존재)을 무시합니다. 모든 엔진이 같은 스토어를 읽으므로, 지원되는 범위에서 카메라, 베이스맵, 레이어, 그룹, 가시성, 불투명도는 전환 후에도 그대로 유지됩니다.

- **지구본이 (아직) 할 수 없는 것.** `MapController` 를 구동하는 모든 도구 — 지도 컨텍스트 메뉴, 범례, 댓글, 스토리 맵, 지형 설정, ML 패널, 그리고 `MapLibre` 타입의 플러그인 API — 는 `MapLibre` 전용입니다. 이들을 고장난 채로 두는 대신, `DesktopShell` 은 지구본이 주 영역을 소유하는 동안 이들을 마운트하지 않으며, View 메뉴는 컨트롤러를 구동하는 항목(줌, 뷰포트 기록, Reset Orientation, Set View, Google Maps/Earth 핸드오프)을 아무 동작 없이 조용히 넘어가게 두지 않고 회색으로 비활성화합니다. 렌더러 중립적이고 스토어 기반인 화면(Layers 패널, Style Manager, 속성 테이블, 프로세싱, Dashboard)은 계속 사용할 수 있습니다. 지구본이 그릴 수 없는 레이어 종류는 프로젝트에 남아 Layers 패널에서 "2D only"로 태그되며, `MapLibre`로 돌아오면 다시 나타납니다. 렌더러 중립 컨트롤러 인터페이스는 #2217의 후속 마일스톤입니다.
- **활성화.** 2D/3D 패인 토글과 Rendering engine 메뉴는 Cesium Ion 토큰 구성 여부와 관계없이 모두 제공됩니다. 지구본은 토큰이 필요 없는 프로젝트 베이스맵을 그리기 때문입니다. 토큰이 있으면 Cesium World Terrain이 추가되고, 래스터 형태가 없는 베이스맵에 대한 폴백으로 Ion World Imagery가 제공됩니다. 토큰이 없으면 뷰에 그 사실을 알리는 한 줄 힌트가 표시됩니다. 토큰은 `getCesiumIonToken()` (`@geolibre/core`)을 통해 해석되며, 이 함수는 빌드의 `VITE_CESIUM_TOKEN / CESIUM_TOKEN` 또는 런타임 재정의에서 값을 읽으므로, 빌드 시(`CESIUM_TOKEN`; 선택적 3D 지구본 자격 증명 참고) 또는 런타임에 재빌드 없이 설정할 수 있습니다. Settings → Environment Variables(설정 → 환경 변수)에는 장치 로컬 `DesktopSettings` (데스크톱에서는 OS 자격 증명 저장소, 웹에서는 `localStorage`; 공유 프로젝트 파일에는 절대 저장되지 않음)가 뒷받침하는 전용 마스킹 **Cesium Ion token** 필드가 있습니다.
`useRuntimeEnvironmentVariables` 는 이를 `window.__GEOLIBRE_RUNTIME_ENV__` 전역의 `VITE_CESIUM_TOKEN` 으로 투영합니다(빈 값은 투영되지 않으므로 빌드 시 토큰을 지울 수 없습니다). 공유 `useCesiumIonToken()` 혹은 `geolibre:runtime-env-change` 이벤트에서 토큰을 다시 해석하고, 두 마운트 지점 모두 지구본을 이 토큰으로 키잉하므로, 새로 입력한 토큰은 다시 로드하지 않아도 적용됩니다.
- **베이스맵.** `basemapToCesiumImagery()` (`@geolibre/core`)는 프로젝트의 `basemapStyleUrl` 을 지구본이 그려야 할 것에 대한 단순한 서술자로 변환하고, `applyBasemapImagery()` (`packages/map/src/cesium-basemap.ts`)는 이를 이미지 레이어로 바꿉니다. 이 분리는 결정을 엔진에서 독립시키고 그것이 읽는 베이스맵 카탈로그 옆에 두므로, 2D와 3D 렌더러가 하나의 단일 진실 공급원을 공유합니다. 행성 및 지역 베이스맵은 이미 래스터 타일 세트이므로 그대로 넘어갑니다(TMS 소스는 `{y}` 가 Cesium의 `{reverseY}` 로 교체되고, 하이브리드 베이스맵의 라벨 오버레이는 그 이미지 바로 위에 올라갑니다). GeoLibre 자체의 벡터 스타일은 비슷한 톤의 키 없는 래스터 베이스맵으로 매핑되므로, 어두운 프로젝트 베이스맵은 어두운 지구본을 만듭니다. 빈 베이스맵은 맨 타원체를 남깁니다. 그 외의 모든 것 — 제공자 스타일, 사용자 지정 URL, 오프라인 아카이브 — 은 래스터 형태가 없으므로, 렌더러는 토큰이 구성되어 있으면 Ion World Imagery로, 아니면 OpenStreetMap으로 폴백합니다. 베이스맵 레이어는 이미지 스택의 맨 아래(인덱스 0, 오버레이는 1)에 삽입되며, 이것이 `CesiumLayerSync` 가 추가하고 맨 위로 올리는 데이터 레이어 아래에 베이스맵을 유지하는 방법입니다.
- **지연 로딩.** 전체 Cesium 엔진(~4.8 MB)은 지구본이 처음 마운트될 때만 `import()` 되며, 자체 빌드 청크(`vite.config.ts` 의 `CODE_SPLITTING_GROUPS`)에 담겨 2D 부팅 경로 밖에 있습니다. Cesium이나 3 MB 넘는 JS가 앱 엔트리의 정적 import 그래프에 들어가면 빌드가 실패합니다(`bootBundleBudgetPlugin`).

Vite 플러그인(`vite-plugins/copy-cesium-assets.ts`)이 Cesium의 런타임 Workers/Assets/Widgets 를 `public/cesium/` 에 스테이징하고, 캔버스는 엔진이 이를 찾을 수 있도록 `window.CESIUM_BASE_URL` 을 설정합니다.

- **그리기.** 두 엔진 모두 공유 포인터 생명주기와 지리적 범위로 `drawExtent` 를 구현합니다. Cesium은 수동 배치에 드래그 가능한 엔티티를, 추출 미리보기에 지면에 고정된 사각형을 사용합니다. Escape, 포인터 취소, 포커스 상실, 렌더러 해제는 탐색을 복원합니다. 래스터 서브셋 패널은 두 렌더러 모두에서 이 작업을 사용하며, 날짜변경선을 넘는 범위는 `west > east`를 유지하여 추출기가 분할해야 함을 설명할 수 있게 합니다. 베이 스맵 추출 패널은 그리기 헬퍼를 공유하지만, 그 메뉴는 여전히 벡터 스타일 지원 여부에 따라 게이트됩니다.

- **카메라 동기화.** `packages/map/src/cesium-camera.ts` 는 MapLibre의 Web-Mercator `MapViewState` (줌, 천저 기준 피치, 베어링)와 Cesium의 카메라(미터 단위 거리, 수평선 기준 피치, 헤딩) 사이를 변환하며, **지상 해상도**(픽셀당 미터)로 맞추므로 패인의 높이가 달라도 화면상의 축척이 일치합니다. `CesiumCanvas` 는 공유 `mapView` 에서 카메라를 초기화하고, 스토어 변경을 지구본에 적용하며, 지구본 자체의 움직임을 다시 기록합니다 — 2D 패인처럼 양방향이며, `apply→moveEnd` 에코를 억제하는 허용 오차 검사를 두어 지터 루프가 없습니다.

- **레이어 동기화.** `CesiumLayerSync` (`packages/map/src/cesium-layer-sync.ts`)는 `MapController.syncLayers` 가 MapLibre에 대해 하는 것과 같은 방식으로 스토어의 `GeoLibreLayer[]` 를 지구본에 조정하며, `SecondaryMapCanvas` 와 동일한 패인별 가시성 재정의와 그룹 효과를 재사용합니다. Cesium이 자연스럽게 맞는 종류를 렌더링합니다 —

`GeoJSON`(`createFeatureStyleResolver` (`packages/map/src/feature-style.ts` , 이슈 #2278)로 피처별 스타일이 지정되는 엔티티를 가진 드레이프된 `GeoJsonDataSource` : 이는 `@geolibre/core` 가 2D 지도용으로 만드는 것과 같은 MapLibre 표현식 — 단일, 범주형, 등급형, 규칙별 심볼 재정의가 있는 규칙 기반, 표현식 모드, `simplestyle` 속성, 비례 크기, 미터 단위 스트로크 — 을 `style-spec` 엔진으로 평가하므로, 폴리곤 채우기와 외곽선, 선 너비와 화살표 장식, 원 반지름, 분류된 마커 스프라이트, 채우기 패턴, 레이어의 줌 범위(공유 `DistanceDisplayCondition`)가 2D 지도와 일치합니다. 포인트는 레이어가 마커를 렌더링하지 않는 한 원으로 그려지며, `pointRenderer: "cluster"` 인 포인트 전용 레이어는 데이터 소스의

`EntityCluster` 를 통해 2D 클러스터 레이어처럼 스타일이 지정된 버블과 축약된 개수로 클러스터링되고 `clusterMaxZoom` 을 넘으면 꺼집니다. 50,000개 피처를 넘는 포인트 전용 레이어는 엔티티를 우회하여 하나의 `PointPrimitiveCollection` 을 사용하며, 그 프리미티브는 `id` 에 피처 참조를 담아 피킹, 강조 표시, 필터가 계속 동작합니다(`packages/map/src/cesium-points.ts` , 이슈 #2282; 히트맵 렌더러는 지구본 형태가 없어 일반 원으로 그려집니다). 필드 또는 표현식 라벨, 헤일로, 라벨 줌 제한도 포함됩니다), XYZ/래스터/WMTS, WMS, ArcGIS MapServer(`ArcGisMapServerImageryProvider`), 지리참조 이미지 오버레이(`SingleTileImageryProvider`)(모두 `ImageryLayer` 로), 그리고 3D Tiles(레이어의 타일셋 URL, 요청 헤더, 고도 오프셋을 직접 사용하는 `Cesium3DTiles` 프리미티브로, 그 피처는

`cesium-tiles` (이슈 #2290)로 분류됩니다: 레이어의 색상 표현식과 합성된 피처 필터가 MapLibre 표현식에서 3D Tiles 스타일링 언어로 변환되어 `Cesium3DTileStyle` 로 적용되므로, 타일셋이 돌출 벡터 레이어처럼 범주화, 등급화되거나 규칙 트리를 따르며, 레이어 불투명도는 텍스처 타일을 색조 변화 없이 페이드하는 흰색 곱셈으로 전달됩니다 — 변환할 수 없는 색상이나 필터가 있으면 추측하는 대신 타일셋이 자체 색상을 그리고 모든 피처를 표시하도록 돕니다. `Style` 패널이 나열하는 속성 이름은 처음 렌더링된 타일에서 가져와 `metadata.fields` 에 게시되는데, 타일셋은 스키마를 레이어 레코드가 아니라 타일 안에 담고 있기 때문입니다. ArcGIS I3S 씬 레이어는 `loaders.gl` 대신 Cesium 자체의 `I3SDataProvider` 를 통해

처리되고, 에셋이 3D Tiles 타일셋인 Gaussian splat 레이어는 Cesium이 `KHR_gaussian_splatting` 타일을 네이티브로 그리므로 같은 분기로 렌더링되며, 3D Tiles 형태의 포인트 클라우드는 `pointCloudShading` 아이돔 라이팅을 얻습니다), 더불어 `copc` 패키지로 브라우저에서 디코딩되어 아카이브의 WKT를 통해 재투영된 경계 있는 `PointPrimitiveCollection` 미리보기로 표시되는 COPC 포인트 클라우드(`packages/map/src/cesium-point-cloud.ts`, 이슈 #2285) — 실시간 가시성/불투명도, 소스 변경 시 재빌드, 제거를 지원합니다. MapLibre 커스텀 프로토콜을 통해 2D 지도에 도달하는 타일 소스 — WASM 타일러의 COG 타일, 래스터 PMTiles, 로컬 MBTiles, 데스크톱의 네이티브 XYZ/WMS 페처, KML 슈퍼 오버레이 — 는 `ProtocolImageryProvider` (`packages/map/src/cesium-protocol-imagery.ts`, 이슈 #2283)를 통해 렌더링됩니다. 이는 MapLibre와 같은 방식으로 `{z}/{x}/{y}` 템플릿을 확장하고, 각 타일 URL을 `maplibregl.config.REGISTERD_PROTOCOLS`의 핸들러(프로세스 전역이므로 MapLibre 지도가 마운트되어 있지 않아도 동작)에 넘기며, 바이트를 Cesium이 기대하는 방향으로 뒤집은 `ImageBitmap`으로 디코딩하는 Cesium `ImageryProvider`입니다. 등록되지 않은 스킴은 빈 채로 그려지는 대신 레이어 오류로 보고됩니다. COG 레이어는 래스터 컨트롤이 사용하는 것과 같은 `cog-tiler-wasm` 소스를 열고 영속화된 `metadata.rasterState`에서 렌더링하므로(`cesium-cog-imagery.ts`), COG는 두 렌더러에서 같은 모습입니다. 데스크톱 파일 경로만 가진 프로젝트에서 복원된 COG는 래스터 컨트롤이 다시 열기 전까지 2D 전용으로 남습니다. Zarr 레이어는 `zarr-cesium`의 `ZarrLayerProvider` (`cesium-zarr-imagery.ts`, 이슈 #2261)를 통해 그려지며, 2D Zarr 컨트롤과 ArcGIS 렌더러가 읽는 것과 같은 레코드(URL, 등록된 스토어 또는 `kerchunk` 매니페스트, 변수, 선택터, 색상 한계, 램프)에서 공급받습니다. Time Slider 단계나 재스타일링은 스토어를 다시 열지 않고 라이브 프로바이더를 업데이트하고 그 위에 새 `ImageryLayer`를 교체해 넣으며, EPSG:4326/3857 이외의 CRS를 가진 그리드는 레이어 오류로 보고됩니다. `zarr-cesium`은 `cesium` 래퍼를 import하는데, Vite가 이를 `@cesium/engine`으로 별칭 처리하여 지구본이 하나의 엔진 인스턴스를 유지합니다. 래스터 심볼로지(밝기, 대비, 채도, 색조)는 모든 이미지 종류에 대해 `ImageryLayer` 자체의 조정으로 매핑됩니다. 타일 기반 벡터 레이어(`vector-tiles`, 벡터 PMTiles, 벡터 MBTiles, 해석된 스타일이 있는 ArcGIS 벡터 타일)는 드레이프됩니다(`cesium-drape.ts`, 이슈 #2284): 지구본은 헤드리스 `createLayerSync` 경로를 통해 같은 스토어 레이어를 받는 숨겨진 단일 타일 MapLibre 지도 하나를 실행하고, 한 번에 Web Mercator 타일 하나씩 렌더링하여(타일 중심으로 `jumpTo`, `idle` 대기, 캔버스 읽기) `ProtocolImageryProvider`를 통해 Cesium에 공급하므로, Style Spec을 다시 구현하지 않으며 모든 스타일, 필터, 순서, 가시성 편집이 드레이프의 이미지 레이어를 재빌드합니다. ArcGIS 벡터 타일 가져오기는 해석된 소스와 접두사가 붙은 스타일 레이어를 영속화하여, 헤드리스 동기화가 렌더러 전환이나 프로젝트 다시 로드 후에 이를 다시 만들고 서비스 색상을 보존하면서 가시성, 필터, 불투명도를 적용할 수 있게 합니다. Style 패널은 그런 레이어에 대해 도달하지 못할 색상 편집기 대신 불투명도, 순서, 줌 범위, 필터만 제공합니다. 해석된 스타일이 없는 이전 ArcGIS 레코드는 2D 전용으로 남으며 다시 추가해야 합니다. 드레이프된 콘텐츠는 평면이고, 라벨은 타일별로 배치되며, 글리프는 원격 폰트 호스트에서 가져옵니다(따라서 오프라인에서 보는 로컬 아카이브는 텍스트 없이 드레이프됩니다). 그 위에서의 피킹, 해석된 스타일이 없는 레거시 `arcgis` 레코드, EPT 및 일반 LAS/LAZ 포인트 클라우드, 원시 `.ply` / `.splat` 스피릿 파일, `deck.gl` 시각화는 여전히 지구본에서 건너뛰어지고 2D 패인에서 렌더링됩니다. 내보낸 `isCesiumSupportedLayerType` 프레디케이트를 통해 패인의 레이어 메뉴 — 그리고 지구본이 주 렌더러일 때 Layers 패널 — 가 이들을 "2D only"로 태그합니다. 네이티브 KML/KMZ 문서는 `KmlDataSource`를 사용하여 문서 스타일링, 네트워크 링크, 오버레이

이를 보존하며, 가시성과 불투명도는 원래 색상 속성을 교체하지 않고 적용됩니다. 로컬 KMZ 아카이브는 데이터 URL로 영속화됩니다. Elevation Profile 플러그인은 네이티브 엔티티를 그리고 활성 지형 프로바이더를 샘플링하며, 그 프로바이더가 바뀌면 결과를 거부합니다.

- **지구본 상호작용 동등성.** 주 지구본은 지면 커서 좌표와 선택적 지형 고도를 공유 상태 표시줄에 게시하고, 포인터 이탈과 해제 시 이를 지웁니다. 투영 씬 모드는 Mercator 투영을 보고합니다. Home, 씬 모드, 전체 화면 위젯은 DOM을 다시 마운트하지 않고 공유 컨트롤 위치 지정에 참여합니다. Controls 메뉴에서 이들은 기존 항목에 응답합니다: **Compass**(나침반)는 Home(뷰 재설정) 버튼을, **Globe**(지구본)는 씬 모드 선택기를 관장하며, 이는 2D의 베어링 재설정 및 투영 컨트롤에 대응하는 지구본 측 상대입니다. 툴바는 새로 마운트된 지구본에 이 선택을 다시 적용합니다.
- **환경 플러그인.** Sun simulation, Atmospheric Effects, Clouds, Precipitation, Flight Simulator 플러그인은 2D 엔진과 함께 Cesium을 선언하고(Sun, Effects, Flight Simulator는 MapLibre, Mapbox, Cesium을 나열하고, Clouds와 Precipitation은 ArcGIS도 나열합니다) `app.getCesiumScene()` (이슈 #2287)에서 분기합니다. 이는 주 지구본의 네임스페이스, 위젯, 씬, 카메라, 클록에 대한 타입이 지정된 핸들로, `CesiumEngine.getCesiumScene()` 이 `getMap()` 의 지구본 측 상대로 노출합니다. 지구본에서 Sun은 네이티브 `SunLight`, `globe.enableLighting`, 씬 클록을 구동하고(밤 쪽 깊이는 `globe.vertexShadowDarkness` 에 매핑), Effects는 스카이 박스와 대기를 토글하고 헤일로 설정의 색조/채도/밝기 변화로 `SkyAtmosphere` 를 다시 틸트하며, Flight Simulator는 카메라 어댑터 뒤에서 두 2D 지도와 키보드, 물리, HUD를 공유합니다 — 지구본에서는 `screenSpaceCameraController.enableInputs` 를 중단한 채 매 프레임 `camera.setView`, Mapbox에서는 `setFreeCameraOptions`, MapLibre에서는 `calculateCameraOptionsFromCameraLngLatAltRotation` 을 사용합니다. 날씨 오버레이는 스토어 타일 레이어이며 `CesiumLayerSync` 를 통해 변경 없이 렌더링됩니다. Spinning Globe 컨트롤은 분기가 필요 없습니다. 컨트롤 호스트의 MapLibre 파사드를 통해 스토어를 구동하기 때문입니다. `DesktopShell` 은 어느 엔진이 마운트되든 그 후 이 플러그인들을 다시 연결하므로, 렌더러 교체는 MapLibre 재초기화와 같은 방식으로 이들을 다시 바인딩합니다.
- **영속성.** 패인의 `viewKind` 는 `SecondaryMapView` 의 일부이며 `.geolibre.json` 프로젝트 형식을 통해 왕복합니다(`normalizeSecondaryMapViews`). 따라서 지구본 패인과 함께 저장된 프로젝트는 토큰 유무와 관계없이 지구본으로 다시 열립니다. 워크스페이스 자체의 선택은 최상위 `primaryRenderer` (`normalizePrimaryRenderer`)로 영속화되며, 기본값이 아닐 때만 기록됩니다 — 그래서 MapLibre 프로젝트는 이 설정이 존재하기 전에 저장된 것과 바이트 단위로 동일하고, 3D 우선 프로젝트는 지구본에서 바로 다시 열립니다.

DuckDB-WASM

벡터 파일 가져오기는 MapLibre가 렌더링하기 전에 변환이 필요한 형식에 대해 DuckDB-WASM을 사용합니다:

```
INSTALL spatial;
LOAD spatial;
```

GeoParquet는 Spatial을 로드한 뒤 DuckDB의 Parquet 리더로 읽습니다. 다른 로컬 벡터 형식은 WebAssembly 확장이 GDAL 기반 리더를 로드할 수 있을 때 Spatial `ST_Read` 에 전달됩니다. 압축된 Shapefile은 먼저 `shpjs` 로 파싱하고, 그 파서가 파일을 읽지 못하면 DuckDB Spatial을 시도합니다. KML 파일(그리고 압축 해제된 KMZ 아카

이브 안의 KML)은 내장 심볼로지를 보존하는 자체 파서로 읽으며, `simplestyle-spec` 속성(`fill`, `stroke`, `stroke-width` 등)을 생성하므로 스타일이 지정된 KML이 Google Earth에서와 같은 모습으로 렌더링됩니다. 파서가 처리할 수 없는 KML은 DuckDB Spatial 리더로 폴백하며, 이 경우 스타일 없이 지오메트리만 로드됩니다. 피처가 `simplestyle` 속성을 가진 벡터 레이어는 피처별로 렌더링됩니다. 페인트 표현식이 피처별 색상/너비를 읽고, 없으면 평면 레이어 스타일로 폴백합니다.

Apache Iceberg

Iceberg 테이블은 같은 엔진에서 읽으며, DuckDB의 코어 `iceberg` 확장을 Spatial과 함께 로드합니다(`INSTALL iceberg; LOAD iceberg;`). 메타데이터 위치로 지정된 테이블은 `iceberg_scan()` 으로 스캔하고, Iceberg REST 카탈로그는 고정 별칭으로 `ATTACH` 한 뒤 `SHOW ALL TABLES` 로 열거하고 선택한 테이블을 정규화된 이름으로 읽습니다. 생성된 두 문장 모두 기본값일 뿐입니다. 대화 상자가 이를 SQL 박스에 미리 채우고, 그 박스에 있는 내용이 스캔의 소스가 되므로, `WHERE`, 조인, 프로젝션으로 어떤 지오메트리를 렌더링할지 결정할 수 있습니다. 사용자 지정 문장은 서브셀렉트로 감싸기 전에 `cleanStatement / containsMultipleStatements ( sql-workspace.ts` 의 리터럴 인식 헬퍼)로 검사하므로, 붙여넣은 스크립트는 생성된 래퍼를 가리키는 파싱 오류 대신 명확한 메시지와 함께 실패합니다.

지오메트리 처리는 벡터 파일 리더보다 의도적으로 좁습니다. DuckDB가 네이티브 `GEOMETRY` 로 보고하는 열만 보입니다. Iceberg v3에는 실제 지오메트리 타입이 있고 — `iceberg_column_definition.cpp` 가 이를 `LogicalType::GEOMETRY` 로 매핑합니다 — 따라서 여기서 BLOB이나 VARCHAR는 일반 속성이며, 일반 Parquet에서는 합리적인 blob/base64-WKB 폴백은 여기서는 `ST_AsGeoJSON` 안에서 실패하는 로드만 만들어 낼 것입니다. **CRS는 사용자가 아니라 스키마에서 옵니다.** DuckDB는 CRS 주석이 달린 열을 `GEOMETRY(<crs>)` 로 표현하므로, `icebergTransformCrs` 가 이를 읽어 `ST_Transform` 을 통해 WGS84로 재투영합니다. CRS 매개변수가 없는 열은 사양에 따라 `OGC:CRS84` 이며(DuckDB 확장이 `IcebergConstants::DefaultGeometryCRS` 로 하드코딩한 것과 같은 상수), 이는 이미 GeoJSON의 경도/위도 관례이므로 변환 없이 통과합니다. 좌표계에 관한 어떤 것도 레이어에 영속화되지 않으므로, 업스트림에서 재투영된 테이블은 다음 다시 로드 시에도 올바르게 유지됩니다.

작업은 DuckDB 의존성 없이 영속화된 레이어 구성, SQL 빌더, 테이블 선택 규칙을 담은 `lib/iceberg.ts` 와, 엔진을 소유하는 `lib/iceberg-loader.ts` 로 분할됩니다. 리더는 공유 인스턴스가 아니라 **SQL Workspace**의 전용 DuckDB 인스턴스에서 실행됩니다. Iceberg 스캔은 테이블의 Parquet 데이터 파일을 HTTP로 읽으므로, 그 인스턴스의 spatial 이전 워밍업과 오염된 인스턴스 복구(`resetSqlDatabase` 참고)가 필요하기 때문입니다.

Iceberg 테이블이 크다는 점에서 비롯되는 두 가지 결과는 사용자에게 말하지 않고 앱에 내장되어 있습니다. 모든 로드는 행 제한으로 상한이 정해지고, 총 행 수 — 구성된 소스에 대한 `count(*)` 로, 필터 없는 테이블에 대해서는 DuckDB가 매니페스트 메타데이터에서 답할 수 있지만 사용자 지정 문장의 필터, 조인, 프로젝션은 평가해야 합니다 — 는 어떤 것도 구체화되기 전에 표시됩니다. 또한 결과 레이어는 필요에 따라 새로 고칠 수는 있지만, `lib/layer-refresh.ts` 의 `supportsAutoRefresh` 에 의해 타이머 기반 새로 고침에서 제외됩니다 — 그런 크기의 테이블을 주기적으로 다시 스캔하는 것은 사용자가 의도한 바가 아니므로, 자동 새로 고침 간격은 단순히 기본 꺼짐이 아니라 아예 사용할 수 없습니다.

고급 Add Data 워크플로

Add Data(데이터 추가) 화면에는 XYZ, WMS, WFS, 벡터 파일(`maplibre-gl-vector` 가 뒷받침하는 Add Vector 대화 상자를 통해), GeoJSON URL, 벡터 타일 소스, 구분 텍스트, 래스터 타일 템플릿, COG 및 GeoTIFF 래스터(`maplibre-gl-raster` 가 뒷받침하는 Add Raster 대화 상자를 통해), MBTiles, ArcGIS FeatureServer 또는 VectorTileServer 레이어, Apache Iceberg 테이블, GPX 웨이포인트·트랙·루트를 위한 네이티브 대화 상자가 포함되어 있습니다. 또한 3D Tiles 레이어, id로 지정하는 Cesium Ion 에셋(`metadata.sourceKind: "cesium-ion"` 을 가진 타일셋 또는 이미지 레이어로, 지구본이 주 렌더러일 때 제공되고 2D 지도에서는 "3D only" 배지가 붙습니다; `packages/core/src/cesium-ion.ts`, 이슈 #2290), WFS 및 GeoJSON URL 새로 고침, 텍스트 마커 라벨, 그리고 식별·선택·속성 테이블을 지원하는 여러 DuckDB SQL 쿼리 결과 레이어를 지원합니다. Components 플러그인은 FlatGeobuf, PMTiles, Zarr, LiDAR, Gaussian splat용 `maplibre-gl-components` 패널을 감싸고, 추가된 레이어를 GeoLibre 스토어에 미러링하여 Layer 패널, 프로젝트 형식, 레이어 컨트롤이 이를 다룰 수 있게 합니다. 추가 데이터 소스는 Planetary Computer 및 Earth Engine 패널, Overture Maps 플러그인, 연방 Web Services 플러그인을 통해 사용할 수 있습니다. `maplibre-gl-time-slider` 가 뒷받침하는 Time Slider 플러그인은 도킹된 타임라인을 통해 시계열 래스터 및 벡터 데이터(COG, XYZ/WMTS, WMS-Time, 시간 필터링된 GeoJSON)를 애니메이션하며, 추가하는 각 소스를 외부 네이티브 레이어로 GeoLibre 스토어에 미러링합니다.

로컬 MBTiles 타일은 Tauri 명령이 뒷받침하는 커스텀 MapLibre 프로토콜을 통해 읽습니다. 원격 래스터는 필요할 때 데스크톱 백엔드를 통해 가져오며, 로컬 개발 서버에는 CORS 처리가 필요한 선택된 릴리스 에셋을 위한 래스터 프록시가 포함되어 있습니다.

Python 사이드카

`backend/geolibre_server` 의 FastAPI 앱은 관리되는 로컬 프로세싱 사이드카를 통해 형식 Conversion 도구, 래스터 도구, 그리고 Whitebox 툴박스의 선택적 서버 엔진을 뒷받침합니다. 데스크톱 앱은 사이드카를 필요에 따라 시작하고, `127.0.0.1` 로 통신하며, 무거운 Python 프로세싱 스택을 브라우저 번들 밖에 둡니다.

Whitebox 툴박스는 사이드카에 의존하지 **않습니다**. 1,000개가 넘는 도구가 기본적으로 `geolibre-wasm` WebAssembly 런타임에서 브라우저 안에서 실행되며, 사이드카는 디렉터리에 대한 일괄 실행을 위해 네이티브 파일 경로를 읽을 수 있는 옵트인 대체 엔진입니다.

Vector 도구(Processing → GeoLibre Toolbox → Vector)는 Turf.js로 클라이언트 측에서 실행되며 사이드카가 필요 없습니다. 모든 도구는 투영을 고려한 결과를 위해 GeoPandas와 Shapely가 뒷받침하는 사이드카의 `/vector` 엔드포인트에서 선택적으로 실행할 수 있습니다. 사이드카는 `/vector/status` 를 통해 가용성을 보고하고, 선택적 `vector extra`가 설치되어 있지 않으면 대화 상자가 클라이언트 엔진으로 폴백합니다.

세 번째 Vector 엔진인 **Python (Pyodide)**는 같은 GeoPandas/Shapely 코드를 `Pyodide`(WebAssembly로 컴파일된 CPython)를 통해 **브라우저 안에서** 실행하므로, GeoPandas 경로를 서버 없이 웹 빌드에서 사용할 수 있습니다. 지오메트리 로직은 프레임워크에 독립적인 모듈 `backend/geolibre_server/geolibre_server/vector_ops.py` 로, 사이드카와 브라우저 모두가 실행합니다 — Vite 플러그인(`vite-plugins/copy-vector-ops.ts`)이 이를 앱 번들로 복사하고, 클래식 Web Worker(`public/pyodide/pyodide-worker.js`)가 CDN에서

Pyodide를 로드하고 `geopandas` 를 설치한 뒤 JSON 문자열 경계를 통해 `run_vector_tool` 을 호출합니다. 단일 진실 공급원이므로 Sidecar와 Pyodide 엔진은 동일한 결과를 반환합니다. Pyodide 런타임은 처음 사용할 때 지연 다운로드되며, `VITE_PYODIDE_INDEX_URL` 환경 변수는 오프라인/프로덕션 배포를 위해 자체 호스팅 미러를 가리키게 합니다.

SQL Workspace는 DuckDB와 PostGIS와 함께 **Apache Sedona** 엔진을 제공합니다. 이는 단일 노드 Rust(DataFusion + Arrow) 엔진인 **SedonaDB**에서 Sedona 공간 SQL을 실행하며, 선택적 `sedona-extra`(`apache-sedona[db]`)가 뒷받침하는 사이드카의 `/sql` 엔드포인트(`/sql/status`, `/sql/run`)를 통합합니다. 브라우저에서 — 그리고 `extra`가 설치되지 않은 데스크톱에서 — 같은 엔진이 SedonaDB의 WebAssembly 빌드인 **CereusDB**에서 전적으로 클라이언트 측에서 실행됩니다. 워크스페이스는 도달 가능하면 사이드카를 우선하고 자동으로 CereusDB로 폴백합니다. CereusDB 번들은 크므로 자체 체크로 동적 import되며, Sedona 쿼리가 처음 실행될 때만 다운로드됩니다.

사이드카는 그 이후 변환을 넘어 성장했습니다. 라우터는 `backend/geolibre_server/geolibre_server/app/`에 있습니다. `raster.py`는 `rasterio/numpy/contourpy`에서 Raster 툴박스를 실행하고, `vector.py`는 선택적 GeoPandas 벡터 엔진을 뒷받침하며, `sql.py`는 위의 Sedona 엔진을 뒷받침하고, `postgis.py`와 `mssql.py`는 PostGIS와 SQL Server 테이블을 편집 가능한 레이어로 로드하고 편집 내용을 다시 기록하며, `pointcloud.py`는 포인트 클라우드 어노테이터 라벨을 전체 LAS/LAZ/COPC 파일에 기록하고, `m1.py`는 AI Segmentation을 `samgeo-api` 서버(SamGeo/SAM 3)로 리버스 프록시하는데, SAM 3은 수 기가바이트의 GPU Torch 빌드를 필요로 하므로 프로세스 밖에 둡니다. 모든 엔드포인트는 **Server API 레퍼런스**를 참고하세요. Leafmap과 GeoAI 워크플로도 같은 패턴을 따를 것으로 예상됩니다.

오프라인 지원(PWA)

독립 실행형 웹 빌드는 설치 가능한 Progressive Web App입니다. `vite-plugin-pwa` (`apps/geolibre-desktop/vite.config.ts`에서 구성)가 웹 매니페스트와 Workbox 서비스 워커를 생성하고, `src/main.tsx`가 `installStaleChunkReload` 옆에서 이를 등록하여 둘이 조율되도록 합니다. 서비스 워커는 웹 빌드에서만 빌드됩니다. Tauri 데스크톱 빌드(번들된 에셋으로 이미 오프라인)와 임베디드 Jupyter 휠(`GEOLIBRE_EMBED=1`)에서는 비활성화되며, 그곳에서 `registerSW`는 no-op으로 해석됩니다.

첫 방문을 가볍게 유지하기 위해 캐싱은 분할되어 있습니다:

- **프리캐시(앱 셀).** HTML과 지도를 부팅하는 JS/CSS 청크는 프리캐시되므로, 첫 방문 후에는 네트워크 없이 셀이 로드됩니다. 아래의 무거운 청크들은 첫 로드 다운로드가 커지지 않도록 프리캐시에서 제외됩니다.
- **런타임 캐시, CacheFirst.** 프리캐시가 건너뛰는 콘텐츠 해시가 붙은 빌드 에셋(`/assets/` 아래의 모든 것)은 처음 사용할 때 캐시됩니다: **MapLibre** 번들, **DuckDB-WASM**과 그 **spatial 확장**, 그리고 MapLibre 기능 플러그인 청크입니다. 해시된 파일명 덕분에 CacheFirst가 안전합니다 — 재배포는 새 URL을 발급하므로, 오래된 항목이 최신으로 제공되는 일이 없습니다. 이것이 로컬 파일 워크플로(DuckDB Spatial 변환)가 온라인에서 한 번 실행된 뒤 오프라인에서 동작하게 하는 요소입니다.

`VITE_DUCKDB_SPATIAL_EXTENSION_PATH`를 통해 spatial 확장을 자체 호스팅하면 동일 출처(same-origin)로 유지되어 역시 캐시됩니다. **PGLite/PostGIS**는 이 동일 출처 목록에 없습니다. 빌드에서 제외하기

위해 jsDelivr CDN에서 가져오기 때문입니다 — 원본 ~25 MB, Tauri 바이너리에서는 압축되지 않는 ~22 MB의 무게 — 따라서 PostGIS SQL 엔진은 처음 사용할 때 네트워크가 필요합니다. 별도의 규칙이 이후 이를 캐시합니다(아래 참고).

- **런타임 캐시, CDN 엔진.** 두 번째 CacheFirst 규칙(`geolibre-cdn-engines`)은 jsDelivr에서 제공되는 무거운 엔진들을 다룹니다: **Pyodide**와 그 유틸, **PGLite/PostGIS**, **CereusDB**(Apache Sedona) wasm, 그리고 `gda13.js` 입니다. 이들의 URL은 정확한 패키지 버전을 포함하므로 재배포는 새 URL을 발급하고 CacheFirst는 오래된 엔진을 절대 제공하지 않으며, jsDelivr의 관대한 CORS 헤더 덕분에 이들은 다른 항목처럼 재검증하고 제거할 수 있는 일반(비불투명) 200 응답으로 돌아옵니다. 이것이 브라우저 SQL(PostGIS), Sedona SQL, Python 기능이 **첫 온라인 사용 이후** 오프라인에서 동작하게 하는 요소입니다.

`GEOLIBRE_PGLITE_CDN=0` / `GEOLIBRE_CEREUS_CDN=0` 이면 그 엔진들은 대신 `/assets/` 아래에 번들되며 이 규칙은 단순히 그들과 일치하지 않습니다. Pyodide는 어떤 경우에도 항상 CDN에서 로드됩니다. 이 규칙은 `/npm/@duckdb/` 도 다루는데, 이는 `GEOLIBRE_DUCKDB_WASM_CDN=1` 이 DuckDB를 출처 밖으로 옮길 때(아래)와 `maplibre-gl-duckdb` 가 자체 엔진을 가져올 때(모든 빌드에서 jsDelivr에서 가져옴) 일치합니다.

- **베이스맵.** CORS 친화적인 기본 호스트(OpenFreeMap, CARTO)의 타일과 스타일은 런타임 캐시됩니다. 다른 원격 타일, 서비스, ArcGIS/WMS/WFS 소스는 설계상 네트워크 전용으로 남으며 오프라인에서는 사용할 수 없습니다.

따라서 이 CDN 엔진들 각각은 **첫** 사용 시 네트워크가 필요하며, 그 이후에는 오프라인에서 동작합니다. 첫 사용 의존성까지 제거하려면 `GEOLIBRE_PGLITE_CDN=0` 과 `GEOLIBRE_CEREUS_CDN=0` 으로 빌드하세요. 이는 PGLite/PostGIS와 CereusDB wasm을 `/assets/` 아래의 빌드에 다시 번들링하여 동일 출처 규칙이 이를 다루게 합니다(PGLite만으로도 Tauri 바이너리에 ~22 MB가 다시 추가됩니다).

빌드 출력에서 **GeoLibre가 제어하는 모든** 외부 CDN 참조를 제거하려면 — 신뢰할 수 없는 CDN에서 어떤 리소스도 로드할 수 없는 배포를 위해 — `GEOLIBRE_NO_EXTERNAL_CDN=1` 을 사용하세요. 이는

`GEOLIBRE_PGLITE_CDN=0` , `GEOLIBRE_CEREUS_CDN=0` , `GEOLIBRE_GDAL_CDN=0` ,
`GEOLIBRE_DUCKDB_WASM_CDN=0` 을 함축하며, 추가로 코드가 외부 CDN을 참조하는 기능을 비활성화합니다: 스토리맵 HTML 내보내기(`unpkg.com` 의 `<script>` 태그를 주입), 내장 객체 탐지 모델(`cdn.jsdelivr.net` 의 YOLO ONNX 가중치), Segment Everything, 3D Tiles Draco/KTX2 디코더, GDAL 내보내기입니다.

`VITE_PYODIDE_INDEX_URL` 이 승인된 미러를 가리키면 Pyodide는 계속 사용할 수 있습니다. 런타임 `deployment.json` 은 클라이언트 정책을 구성하며, 이 빌드 시점 CDN 패키징을 바꾸지 않습니다.

DuckDB-WASM은 반대 방향으로 갑니다. 로컬 벡터 파일을 여는 핵심 경로에 있으므로 기본적으로 번들되기 때문입니다. `GEOLIBRE_DUCKDB_WASM_CDN=1` 은 이를 jsDelivr로 옮깁니다 — 또는 `npm run lite:build` 를 사용하면 그 플래그를 설정한 뒤 생성된 어떤 파일도 상한을 넘지 않음을 _단언_하므로, 회귀가 발생하면 업로드가 아니라 빌드가 실패합니다. `duckdb-wasm` 자체의 `getJsDelivrBundles()` 에 의해 설치된 버전에 고정되므로, 가져온 엔진이 번들에 컴파일된 로더와 어긋날 수 없습니다. 이 플래그가 존재하는 이유는 하나입니다: `duckdb-mvp.wasm` (~40 MB)과 `duckdb-eh.wasm` (~35 MB) 모두 **Cloudflare Pages와 Workers 정적 에셋의 에셋당 25 MiB 제한**을 초과하여 업로드가 즉시 거부되는데, 빌드의 다른 어떤 것도 이에 근접하지 않습니다(다음으로 큰 것이 ~22 MB). 따라서 이 플래그 하나가 웹 빌드를 그곳에 호스팅할 수 있는지 여부를 결정합니다 — 출력을 ~251 MB에서 ~176 MB로 줄이고 상한을 넘는 파일이 없게 합니다. GitHub Pages는 파일당 100 MB를 허용하므로 이 중 어느 것도 필요 없습니다. 이 플래그는 서비스 워커를 포함하지 않는 두 타깃 모두에서 무시됩니다: 오프라인 가능 상태를 유지해야 하는 Tauri 빌드와, 런타임 캐시 없이 매 노트북 세션마다 엔진을 다시 가져오게 될 임베드 빌드(`GEOLIBRE_EMBED=1`)입

니다. 바이너리와 월은 Cloudflare에 업로드되지 않으므로 어느 쪽도 이 플래그가 존재하는 이유인 크기 상한이 없습니다. CDN 호스팅 워커 스크립트는 `new Worker` 에 전달할 수 없으므로(동일 출처여야 함), 그 변형은 실제 스크립트를 `importScripts` 하는 동일 출처 blob으로 감쌉니다. 이를 위해 `worker-src blob:` 과 `script-src` 의 CDN이 필요하며, 둘 다 이미 `docker/nginx.conf` 에 있습니다. 중첩된 `importScripts` 는 `worker-src` 가 아니라 `script-src` 에 대해 매칭됩니다 — Firefox가 역사적으로 워커 하위 리소스를 `worker-src / child-src` 에 대해 검사해 왔기 때문에, Chromium과 Firefox에서 해당 정책에 대해 검증했습니다.

두 엔진에는 그런 탈출구가 없습니다. **Pyodide**는 항상 가져옵니다. `VITE_PYODIDE_INDEX_URL` 은 이를 직접 호스팅하는 미러로 다시 가리키게 합니다 — jsDelivr에 접근할 수 없거나 허용되지 않을 때 할 만한 일입니다 — 하지만 어느 CacheFirst 규칙도 그런 미러와 일치하지 않는다는 점에 유의하세요. 하나는 `cdn.jsdelivr.net` 으로, 다른 하나는 동일 출처 `/assets/` 로 범위가 제한되어 있으므로, 미러가 `/assets/` 아래에서 제공되지 않는 한 서비스 워커가 아니라 일반 HTTP 캐싱으로 폴백합니다. `gdal3.js` — Georeferencer의 클라이언트 측 GeoTIFF/COG 내보내기를 뒷받침 — 는 전혀 벤더링되지 않습니다. 그 `wasm(~28 MB)`과 데이터(~12 MB)는 오직 CDN에서만 오며, `GEOLIBRE_GDAL_CDN=0` 은 이를 번들하는 대신 옴트아웃하여 로더에 경로가 없고 그 내보내기를 사용할 수 없게 됩니다.

데스크톱 빌드는 서비스 워커를 포함하지 않으므로, 이 런타임 캐싱 중 어느 것도 거기에는 적용되지 않습니다. PGLite/PostGIS와 CereusDB를 위의 플래그로 번들하지 않는 한, 설치마다 처음 사용할 때 CDN 로드 엔진을 모두 다시 가져옵니다.

새 배포는 `registerType: "autoUpdate"` 를 통해 반영됩니다. 새 서비스 워커가 백그라운드에서 설치되어 제어권을 가져가므로(`skipWaiting + clientsClaim`), 그 새 프리캐시가 이후 요청을 제공합니다. Workbox의 기본 활성화 시 강제 다시 로드는 `src/main.tsx` 의 `onNeedReload` 를 통해 의도적으로 억제됩니다 — 상대 `base /demo/` 하위 경로에서 그 다시 로드가 잘못 발동되어 진행 중인 지도 상태를 버렸기 때문입니다. 페이지 복구는 `installStaleChunkReload` 에 위임되며, 이는 고아가 된 지연 청크가 404를 반환할 때만 필요에 따라 다시 로드합니다(쿨다운으로 보호; `sessionStorage` 가 차단되면 다시 로드를 건너뛰고 프리로드 오류가 표면화되게 됩니다). 프리캐시된 청크는 캐시에서 제공되며 절대 404가 나지 않으므로, 페이지가 사용자 모르게 다시 로드되는 일이 더 이상 없습니다.

컨테이너 이미지

루트 Dockerfile은 브라우저 앱을 패키징합니다. Node 빌드 스테이지가 `geolibre-desktop` 워크스페이스 빌드를 실행한 뒤, `apps/geolibre-desktop/dist` 를 nginx로 복사합니다. nginx 구성은 정적 에셋을 제공하고 브라우저 진입 URL에 대해 `index.html` 로 폴백합니다.

시작 시 엔트리포인트는 선택된 소스 정책을 검증하고, 비어 있지 않은 환경 변수 재정의의 적용하고, nginx 가드를 생성하고, 조건부로 Python 사이드카를 시작하며, 생성된 공개 `/deployment.json` 을 제공합니다. 브라우저는 첫 렌더링 전에 그 정책을 가져온 뒤, 정책, 런타임 환경, 빌드 환경 순서로 설정을 해석합니다. [Deployment Policy](#)와 [Self-Hosting: 컨테이너 정책 적용](#)을 참고하세요.

Publish Container Image 워크플로는 pull request에 대해 이미지를 빌드하고, main 으로의 push, 버전 태그, 수동 실행에 대해 GitHub Container Registry에 게시합니다. 업스트림 이미지 이름은 ghcr.io/opengeos/geolibre 입니다.

이미지는 선택적 Python 사이드카(unicorn)를 번들하고 /sidecar 에서 리버스 프록시합니다. 사이드카는 최종 정책이 processing:run 또는 data:add 를 부여하고 GEOLIBRE_DISABLE_SIDE CAR 가 1 이 아닐 때만 시작됩니다. 컨테이너는 Tauri 데스크톱 셸을 실행하지 않으므로, 데스크톱 파일시스템 접근에 의존하는 워크플로에는 여전히 설치된 데스크톱 앱이 필요합니다. 클라이언트 정책과 nginx 가드는 데스크톱 프로세싱, 브라우저 WASM, 별도로 노출된 사이드카, 플러그인 실행을 강제하지 않습니다.

보안

- Tauri CSP는 script-src 를 버전별 jsDelivr 경로에 고정하고, 사용자가 제공하는 데이터 URL을 위해 connect-src 를 https: / http: 로 열어 둡니다. 데스크톱 CSP script-src 허용 목록을 참고하세요.
- 파일 접근은 대화 상자에서 선택한 경로만 사용합니다.

자격 증명 저장소

Settings에 저장되는 토큰과 API 키(공유 토큰, Cesium Ion 토큰, Mapbox 토큰, ArcGIS API 키, AI Assistant 프로필의 비밀 필드), 저장된 PostGIS 연결 문자열, 그리고 Share.GeoLibre OAuth 리프레시 토큰은 빌드마다 다른 위치에 저장됩니다:

- **데스크톱(Tauri, macOS/Windows/Linux):** OS 자격 증명 저장소(macOS Keychain, Windows Credential Manager, Linux Secret Service)에 서비스 org.geolibre.desktop 아래 자격 증명당 하나의 항목으로, src-tauri/src/secure_store.rs 의 secure_store_* 명령을 통해 저장됩니다. geolibre.desktopSettings localStorage blob은 그 필드들을 비워 두고, geolibre.postgres.connectionIds 는 postgres.connection.<uuid> 항목의 비밀이 아닌 순서만 보관합니다. 인덱스는 새 항목보다 먼저 기록되므로, 충돌이 발생해도 참조되지 않는 자격 증명이 남는 일이 없습니다. 시작 시 자격 증명 없이 인덱싱된 id를 발견하면(기록이 실패한 경우) 그 id를 삭제하고 경고를 표시합니다. lib/credential-hydration.ts 는 앱이 렌더링되기 전에 이를 메모리에 로드하고, 이전 빌드가 localStorage에 남긴 평문 값을 마이그레이션하며, 각 값은 키체인 기록이 성공한 뒤에만 제거합니다. 그 로드가 유일한 읽기입니다. secure_store_get_many 는 웹뷰의 페이지 로드당 한 번만 응답하고(lib.rs 의 다시 로드 혹은 이를 다시 엽니다), 외부 플러그인 로더가 플러그인을 import하기 전에 호출하는 secure_store_seal 은 읽지 않고 닫으므로, 플러그인 코드는 저장소에서 저장된 토큰을 꺼낼 수 없습니다 (이슈 #2858). 쓰기와 삭제는 계속 열려 있습니다. OAuth 리프레시 토큰은 발급자별로 share.oauth.refreshToken.<issuer> 로 저장되며 리프레시마다 교체됩니다. 액세스 토큰과 PKCE verifier는 메모리에 유지됩니다. 서버는 이미 소비된 리프레시 토큰이 제시되면 세션을 취소하므로, 발급자는 각 키체인 기록 전에 비밀이 아닌 geolibre.share.oauth.unsavedIssuers 목록에 기록되고, 그 발급자에 대해 마지막으로 큐에 넣은 기록이 완료되면 제거됩니다. 시작 시 자신의 발급자가 목록에 있으면 저장된 토큰을 삭제하고, 이미 소비되었을 수 있는 토큰을 신뢰하는 대신 로그아웃 상태로 시작합니다. 목록 자체를 기록할 수 없으면 새 토큰은 저장되지 않습니다. 저장된 사본의 삭제가 큐에 들어가고 로그인은 GeoLibre가 닫

힐 때까지만 유지됩니다. 리프레시 후 그 삭제는 최선의 노력(best effort)입니다. 완료되기 전에 종료하면 소비된 토큰이 저장된 채 남으므로, 다음 실행은 시작 시가 아니라 첫 리프레시에서 로그아웃됩니다. 로그아웃은 로그아웃됨을 보고하기 전에 삭제를 기다리며, 삭제 성공 여부와 관계없이 취소 요청을 보냅니다. 삭제가 실패하면 Settings는 로그아웃이 불완전하다고 보고합니다. 삭제는 큐에 남아 앱이 실행되는 동안 이후의 자격 증명 기록과 함께만 재시도됩니다. 재시작 후 발급자가 unsaved 목록에 있으면 토큰은 폐기됩니다. 그 목록도 기록할 수 없었다면 토큰은 복원되며, 취소 요청도 실패했을 때만 여전히 동작합니다. 프로젝트 파일이 달리 담고 다닐 자격 증명(지오코딩 API 키 preferences.geocoding.apiKeys 와 비밀 Environment Variables)은 이름별로 장치 전체에 project.geocoding.apiKey.<providerId> 와 project.env.<VARIABLE> 로 저장됩니다. 비밀이 아닌 geolibre.projectCredentials.accounts 목록이 같은 인덱스 우선 기록 순서로 이를 인덱싱합니다. 프로젝트를 저장하면 지오코딩 키와 고유한 이름의 비밀 환경 값이 키체인으로 옮겨지고 파일에는 빈 값이 남습니다. 중복되거나 이름 없는 비밀 환경 행은 고유한 계정을 할당할 수 없어 로컬 유지/제거 저장 프롬프트의 대상으로 남습니다. Settings는 사용자가 값을 변경할 때만 기록합니다. 필드를 비우면 그 필드가 해당 값을 표시하고 있었을 때만 저장된 값을 삭제합니다. 그것과 달랐던 프로젝트 파일 재정의의 비우면 재정의가 삭제되고 공유 값이 다시 적용됩니다. 여전히 평문 값을 담고 있는 파일을 열면 그 세션에 그 값을 사용하고 다음 저장까지 아무것도 기록하지 않습니다. @geolibre/core 의 project-credential-refs.ts 는 사용되는 곳(런타임 환경, Settings)에서 빈 값을 저장된 값으로 채웁니다. 파일의 값이 우선합니다. 변수를 제거하거나 이름을 바꿔도 저장된 값은 절대 삭제되지 않는데, 장치의 다른 프로젝트가 같은 이름을 사용할 수 있기 때문입니다. 키체인 기록이 실패하면 마찬가지로 유지/제거 저장 프롬프트로 폴백합니다. 레이어 요청 헤더는 레이어별로 저장되지 않습니다. 헤더 값은 \${NAME} 으로 변수를 참조하며(예: Authorization: Bearer \${TILES_TOKEN}), 입력한 그대로 저장되고 요청이 이루어질 때 프로젝트의 활성화된 Environment Variables에서 해석됩니다 (resolveProjectHeaderReferences). 변수가 설정되지 않은 헤더는 전송되지 않습니다. 마스킹 처리는 선택적 스킴 단어와 참조 하나만으로 이루어진 헤더 값은 유지하고, 그 외의 헤더 값은 이전과 같이 제거합니다. 플러그인이 app.credentials 를 통해 스스로 저장하는 토큰은 plugin.<pluginId>.<name> 으로 저장되며(호스트가 플러그인 id를 추가), 비밀이 아닌 geolibre.pluginCredentials.accounts 목록이 같은 인덱스 우선 기록 순서로 이를 인덱싱하고, lib/plugin-credentials.ts 가 플러그인이 동기적으로 읽을 수 있도록 메모리에 보관합니다. 내장 플러그인은 API로 옮겨 가기 전까지 여전히 자체 localStorage 키를 사용합니다.

• **웹, Jupyter 임베드, 모바일:** 이전과 같이 localStorage입니다. 플러그인 자격 증명은

geolibre.pluginCredential.<pluginId>.<name> 입니다. 웹 OAuth 리프레시 토큰은 탭 범위의 sessionStorage에 유지됩니다. 프로젝트 파일 자격 증명은 유지/제거 저장 프롬프트 뒤에서 프로젝트 파일에 남습니다.

자격 증명 저장소를 사용할 수 없으면(예: Linux 세션에 Secret Service가 없는 경우) 자격 증명은 세션 동안 메모리에 유지되며 Settings는 키체인이 프로젝트 자격 증명을 저장할 수 없다고 보고합니다. 로컬 프로젝트 저장은 남은 자격 증명을 평문으로 유지할지 제거할지 묻습니다. 명시적인 선택 없이는 절대 기록하지 않습니다.

네이티브 HTTP 신뢰 저장소와 상호 TLS

데스크톱 앱은 일부 원격 요청(타일/URL 해석, OGC GetCapabilities 등)을 WebView가 아니라 네이티브 Rust 프로세스에서 보냅니다. 그 경로(`src-tauri/src/lib.rs`의 `guarded_http_client`)는 번들된 Mozilla 루트 인증서에 더해 **OS/시스템 인증서 저장소**를 신뢰하므로, 기기에 설치된 기업 CA가 서명한 서버는 추가 구성 없이 수락됩니다.

상호 TLS(mTLS) 뒤에 있는 엔드포인트의 경우, WebView `fetch` 경로는 대화형 OS 인증서 프롬프트를 표시하지만 네이티브 Rust 클라이언트는 그럴 수 없으므로, 환경 변수로 클라이언트 인증서를 지정하세요:

변수	용도
<code>GEOLIBRE_HTTP_CA_CERT</code>	OS 저장소에 더해 신뢰할 추가 CA 인증서의 PEM 번들 경로입니다(시스템 전체에 설치되지 않은 사설 CA용).
<code>GEOLIBRE_HTTP_CLIENT_CERT</code>	제시할 클라이언트 인증서의 경로입니다. <code>.pem</code> 파일(인증서 체인과 암호화되지 않은 PKCS#8 개인키) 또는 PKCS#12 번들(<code>.p12</code> / <code>.pfx</code>)입니다.
<code>GEOLIBRE_HTTP_CLIENT_CERT_PASSWORD</code>	PKCS#12 클라이언트 인증서의 암호입니다. 이 변수가 존재하면 PKCS#12 코드 경로가 강제됩니다.

`.p12` / `.pfx` 확장자(또는 제공된 암호)는 PKCS#12를 선택하며, 그 외의 경로는 PEM으로 읽습니다. PEM 신원은 기본 `rustls` 백엔드를 사용하고, PKCS#12 신원은 플랫폼 `native-tls` 백엔드(Windows에서는 `SChannel`, macOS에서는 `Secure Transport`, Linux에서는 `OpenSSL`)를 사용하며, 이 백엔드도 OS 신뢰 저장소를 읽습니다. `rustls` 경로를 선호한다면 `openssl pkcs12 -in cert.p12 -out cert.pem -nodes`로 PKCS#12 내보내기를 PEM으로 변환하세요. 이 변수들 중 어느 것이든 설정되었지만 비어 있는 값은 설정되지 않은 것으로 취급됩니다.

구성된 클라이언트 인증서는 공유 네이티브 HTTP 클라이언트에 보관되므로, 인증서가 발급된 엔드포인트뿐 아니라 네이티브 요청 중 **인증서를 요구하는 모든 HTTPS 서버에 제시됩니다**(프로젝트가 가리키는 타일, 스타일, OGC 호스트). TLS는 서버가 요구할 때만 클라이언트 인증서를 보내므로 무조건적인 노출은 아니지만, 앱이 통신하는 호스트가 신뢰할 수 있을 때만 클라이언트 인증서를 구성하세요.

성능: Linux(WebKitGTK)에서의 지도 렌더링

데스크톱 앱은 시스템 WebView를 사용합니다. Linux에서는 **WebKitGTK**이며, 그 WebGL/JavaScript 파이프라인은 브라우저 빌드가 실행되는 Chromium 엔진보다 상당히 느립니다. 가장 눈에 띄는 영향은 **낮은 줌에서의 지도 이동(패닝)**입니다:

- 빈 지도(타일 레이어 없음)는 어떤 줌에서도 안정적인 60 FPS로 이동합니다.
- 타일 레이어(벡터 또는 래스터 XYZ)가 하나라도 있으면 **타일이 로드되는 동안** FPS가 한 자릿수로 급락했다가, 같은 줌에서 로드가 멈추면 60으로 바로 돌아옵니다. 낮은 줌에서 이 현상이 지속되는 이유는 전 세계를 가로질러 이동하면 타일이 계속 로드되어 캐시가 결코 안정되지 않기 때문입니다.

비용은 WebKitGTK가 새로 로드된 각 타일을 메인 스레드에서 처리하는 데서 옵니다. 동기 WebGL 호출을 통한 텍스처(래스터) 또는 정점 버퍼(벡터)의 GPU 업로드와 그 이후의 페이드인 다시 그리기 프레임입니다. 벡터 타일 파싱과 버킷 빌드는 MapLibre의 Web Worker에서 실행되므로 여기서 병목이 아닙니다. 각 타일 통합 렌더 사이클은 WebKitGTK에서 벽시계 기준 ~125 ms로 측정된 반면 Chromium에서는 몇 ms였습니다. 차이는 WebKitGTK의 WebGL 구현과 컴포지터 파이프라인(드라이버 명령 스트림 플러시, TextureMapper GL 서피스 합성)에 있으며, 그 JavaScriptCore JS 엔진만의 문제는 아닙니다. 이는 WebView 엔진의 한계이지 GeoLibre의 버그가 아니며, 브라우저 빌드나 (테스트되지 않은) macOS/Windows WebView에는 영향을 주지 않습니다.

진단 중에 배제된 항목(향후 조사가 반복하지 않도록): 소프트웨어 렌더링(GPU가 사용됨, Intel i915 확인), GPU 포화(렌더 엔진이 ~20% 유틸 상태 유지), Tauri IPC 파일 읽기(22 MB GeoJSON에 ~126 ms), `JSON.parse` (~36 ms), KWin 컴포지터 지연, `renderWorldCopies`, 지구본 대 메르카토르 투영, 그리고 `preserveDrawingBuffer`.

측정을 재현하려면: MapLibre `render` 이벤트, `dataloading` (타일) 이벤트, 그리고 초당 한 번의 `requestAnimationFrame` 카운터를 임시로 로깅하세요. FPS는 타일 로드 수와 반비례합니다. 가장 빠른 확인 방법은 이 프레임레이트 미터를 WebView 개발자 도구 콘솔에 붙여넣은 뒤, 로깅되는 FPS를 지켜보며 지도를 이동하는 것입니다:

```
let f = 0,
    t = performance.now();
(function loop(n) {
  f++;
  if (n - t >= 1000) {
    console.log("FPS", f);
    f = 0;
    t = n;
  }
  requestAnimationFrame(loop);
})(t);
```

완화책(타일당 비용은 엔진에 의해 고정되므로, 이동 중 로드되는 타일 _수_를 줄이는 것)은 아직 구현되지 않았습니다. 더 큰 `maxTileCacheSize`, 256px 대신 512px 래스터 타일, 그리고 `fadeDuration: 0`이며, Chromium 기반 빌드가 완전한 충실도를 유지하도록 WebKitGTK에만 적용하는 것이 이상적입니다.

<https://geolibre.app/maplibre-renderer/>

MapLibre 렌더러

GeoLibre의 기본 렌더러를 사용하려면 **View → Rendering engine → MapLibre**(보기 → 렌더링 엔진 → MapLibre)를 선택하세요. MapLibre GL JS는 모든 GeoLibre 배포판에 번들로 포함되어 있고, 계정이나 액세스 토큰이 필요 없으며, 기본 맵과 모든 분할 패널에서 사용할 수 있습니다.

MapLibre는 GeoLibre에서 가장 폭넓은 기능 및 플러그인 호환성을 제공합니다. 새 프로젝트는 `primaryRenderer`에 다른 값이 지정되지 않는 한 MapLibre를 사용하며, 보조 패널에서 `viewKind`가 생략된 경우에도 MapLibre를 의미합니다.

지원되는 경로

- 단일, 범주형, 단계 구분, 규칙 기반, 표현식, 비례, 히트맵, 클러스터, 패턴, 레이블, 돌출(extrusion) 스타일링을 지원하는 GeoJSON 및 가져온 벡터 데이터.
- XYZ, WMS, WMTS, COG/GeoTIFF, 이미지 및 비디오 오버레이, 래스터 PMTiles, 데스크톱 MBTiles를 포함한 래스터 및 이미지 소스.
- 가져온 Mapbox 스타일과 ArcGIS 벡터 타일 서비스를 포함한 벡터 타일 및 벡터 PMTiles 또는 MBTiles.
- GeoLibre의 전용 렌더러와 오버레이를 통한 3D Tiles, I3S, LiDAR 및 COPC 포인트 클라우드, Gaussian splat, 3D 모델, Zarr, NetCDF/HDF, deck.gl 시각화 레이어.
- 피쳐 선택(picking), 선택, 편집, 측정, 범위 그리기, 드래그 가능한 배치, 인쇄 및 스크린샷 캡처, 그리고 MapLibre 지향 플러그인 표면 전체.

사용자 대상 전체 목록은 [지원되는 데이터 형식과 기능](#)을 참조하세요. MapLibre 엔진 자체에는 요구 사항이 없더라도 개별 데이터 소스에는 플랫폼, 자격 증명 또는 네트워크 요구 사항이 있을 수 있습니다.

맵, 지구본, 지형

MapLibre는 메르카토르 투영과 지구본 투영을 지원합니다. **Controls → Globe**(컨트롤 → 지구본)는 투영을 전환하고, **Controls → Terrain**(컨트롤 → 지형)은 두 뷰 중 어느 쪽에든 고도를 추가합니다. 지형 소스는 GeoLibre의 전역 소스이거나 호환되는 로컬 또는 원격 COG DEM일 수 있습니다. 이 지구본 투영은 여전히 MapLibre 스타일 맵입니다. 네이티브 3D 지구본과 Cesium 네이티브 장면 형식이 필요하다면 [Cesium 렌더러](#)를 선택하세요.

베이스맵과 플러그인

공유 Basemaps(베이스맵) 패널은 키가 필요 없는 스타일과 함께 **Settings → Environment Variables**(설정 → 환경 변수)에서 구성하는 선택적 공급자 스타일을 제공합니다. 공급자 키는 해당 서비스에 속한 것이며 MapLibre 자체의 요구 사항은 아닙니다.

플러그인은 매니페스트에서 `engines` 를 생략하면 기본적으로 MapLibre를 지원하는 것으로 간주됩니다.

MapLibre 전용 플러그인은 `app.getMap()` 을 통해 네이티브 맵에 접근할 수 있으며, 이식 가능한 플러그인은 대신 [플러그인 API](#)에 설명된 렌더러 중립 API 또는 공유 Style-Spec API를 사용해야 합니다.

아키텍처와 라이선스

`MapController` 는 `MapEngine` 의 MapLibre 구현입니다. `packages/map/src/layer-sync.ts` 는 프로젝트 레이어 레코드를 네이티브 소스 및 스타일 레이어와 동기화하며, 전용 컨트롤은 MapLibre가 직접 그리지 않는 형식을 위해 프로토콜이나 공유 오버레이를 등록합니다. 재사용 가능한 헤드리스 표면은 `@geolibre/map` 으로 게시됩니다.

MapLibre GL JS와 GeoLibre는 관대한 라이선스의 오픈 소스입니다. 공유 엔진 모델은 [아키텍처](#)를, 프로젝트 및 라이선스 링크는 [감사의 글](#)을 참조하세요.

<https://geolibre.app/mapbox-renderer/>

Mapbox 렌더러

View → Rendering engine → Mapbox(보기 → 렌더링 엔진 → Mapbox)를 선택하면 Mapbox GL JS로 프로젝트를 렌더링합니다. 이 엔진은 각 분할 창의 렌더링 엔진 메뉴에서도 사용할 수 있습니다. MapLibre가 기본값으로 유지됩니다. 프로젝트는 기본 및 보조 렌더러 선택을 저장하며, Python 및 iframe API는 렌더러 이름으로 `mapbox`를 받습니다.

토큰을 **Settings → Environment Variables → Mapbox token**(설정 → 환경 변수 → Mapbox 토큰)에 붙여 넣고 **Save Settings**(설정 저장)를 클릭하세요. Cesium 토큰과 마찬가지로 이 토큰은 프로젝트 파일 밖, 이 기기에 저장됩니다. 기존에 활성화된 Mapbox 환경 변수 행은 설정을 저장할 때 이 필드로 옮겨지며, Cancel을 누르면 변경되지 않습니다. 또는 개발 서버를 환경에 `MAPBOX_TOKEN`을 넣어 실행할 수도 있습니다. 토큰이 변경되면 Mapbox 지도가 다시 생성됩니다. 토큰이 없으면 설정 안내가 표시되고, 지도 로드 오류에서는 액세스 토큰이 가려집니다. 애플리케이션에 적합한 공개 Mapbox 토큰을 사용하세요. Mapbox 사용은 해당 토큰의 계정에 연결되며 Mapbox의 약관과 사용량 기반 요금을 따릅니다.

새 프로젝트는 Mapbox 렌더러에 기본적으로 Mapbox Standard를 사용합니다. Layers 패널이나 Add Data에서 공유 **Basemaps**(베이스맵) 패널을 열어 Mapbox 스타일(Streets 포함), 공개 스타일, 또는 중첩된 래스터 베이스맵을 선택하세요. 별도의 플로팅 Mapbox 선택기는 제거되었습니다.

Basemaps 패널에서 선택한 스타일은 Mapbox가 활성화된 동안 `preferences.map.mapboxStyleUrl`로 저장되며, 공유 MapLibre/Cesium 배경은 변경하지 않습니다. 모든 Mapbox 창이 그 선택을 공유합니다. 이전에 저장한 프로젝트는 선택했던 스타일을 유지합니다. 제공자 자격 증명은 환경 변수 또는 베이스맵 컨트롤의 API 키 패널에서 가져옵니다.

지원되는 경로

- 네이티브 GeoJSON. 벡터 가져오기 도구가 구체화한 데이터를 포함하며, 점, 선, 폴리곤, 돌출, 라벨, 데이터 기반 색상, 불투명도, 필터 스타일을 지원합니다.
- Mapbox GL JS의 네이티브 아카이브 리더를 통한 원격 벡터 PMTiles 아카이브.
- deck.gl 오버레이를 통한 LiDAR(LAS/LAZ, COPC) 및 표준 3D Tiles.
- **Add Data → Deck.gl Layer**(데이터 추가 → Deck.gl 레이어)에서 만든 Deck.gl 레이어(빌더의 모든 종류, **Add Data → 3D Model**(데이터 추가 → 3D 모델) 뒤의 3D 모델 scenegraph 종류 포함)와, 일반 벡터 레이어의 3D Z 값 및 피쳐 다이어그램 렌더링. 같은 공유 인터리브 deck.gl 오버레이를 통해 그려집니다.
- **Add Data → DuckDB**(데이터 추가 → DuckDB)의 DuckDB 쿼리 레이어. 패널 자체의 deck.gl 오버레이로 그려집니다.
- **Add Data → Zarr Layer**(데이터 추가 → Zarr 레이어)의 Zarr 레이어, STAC Zarr 자산, 그리고 시간 축이 있는 NetCDF/HDF 또는 Kerchunk 큐브. `@carbonplan/zarr-layer` — MapLibre(지구본 및 Mercator) 뿐 아니라 Mapbox GL도 대상으로 하는 `CustomLayerInterface` 구현 — 로 그려지며, 같은 Zarr 컨트롤이

추가합니다. Zarr Layer와 STAC 추가는 현재 투영을 유지하고, 로컬 NetCDF/HDF 큐브와 Kerchunk 참조는 Web Mercator에서 타일 없이 그려지므로 하나를 추가하면 MapLibre에서처럼 지도가 Mercator로 전환됩니다. 원격 NetCDF/HDF URL은 대신 선택한 슬라이스를 이미지 오버레이로 렌더링합니다.

- HTTP(S) 래스터 타일(XYZ, WMS, WMTS), 이름이 지정된 소스 레이어가 있는 벡터 타일, 지오레퍼런스된 이미지/비디오 소스.
- 공유 레이어/그룹 가시성, 불투명도, 순서. 동기화 또는 독립 분할 뷰 카메라. Mercator/지구본 투영과 Mapbox 지형.
- 피쳐 선택, 선택 하이라이트, 범위 그리기, 드래그 가능한 배치, 엔진 수준 이미지 캡처.
- MapLibre와 같은 기본 지도 위 컨트롤이 같은 **Controls**(컨트롤) 메뉴로 관리됩니다. 전체 화면, 그 아래의 피쳐 및 방위 재설정 나침반, 지구본/Mercator 토글, 축척 막대(프로젝트의 축척 단위를 따름), 저작자 표시, 그리고 사용할 수 있지만 기본적으로 꺼져 있는 내비게이션과 지오로케이트. Mapbox GL JS에는 자체 지구본 컨트롤이 없으므로 엔진은 MapLibre의 버튼 마크업을 그대로 따르는 대체 컨트롤 (`packages/map/src/mapbox-globe-control.ts`)을 마운트합니다. 클릭하면 MapLibre에서처럼 투영이 프로젝트 환경 설정에 저장됩니다. 여기서 지형은 썸 설정입니다(Controls 메뉴가 Cesium에서처럼 버튼 없이 Mapbox 지형을 직접 토글). 저작자 표시 컨트롤은 숨길 수 없고, Mapbox는 자체 로고를 그리며, Maptoolkit 로고는 MapLibre 전용입니다.
- 지도 위 레이어 컨트롤(`maplibre-gl-layer-control`). MapLibre에서와 같은 레이어별 가시성, 불투명도, 줌 이동, 스타일 편집기와 스토어 간 왕복을 제공합니다. 이 컨트롤은 공유 스타일 API만 필요로 하므로 두 2D 엔진 모두 하나의 호스트 (`packages/map/src/layer-control-host.ts`)를 통해 구동합니다.
`mapbox://` 스타일은 컨트롤이 가져올 수 없으므로, 엔진은 베이스맵과 프로젝트 레이어를 구분할 수 있도록 로드된 스타일 자체의 레이어로 컨트롤을 시드합니다. MapLibre에서처럼 **Controls → Layer Control**(컨트롤 → 레이어 컨트롤)에서 토글하세요. 분할 창은 두 번째 컨트롤을 마운트하지 않습니다.

플러그인

플러그인은 지원하는 렌더러를 선언하며(`engines`), Plugins 메뉴와 명령 팔레트는 나머지를 회색으로 비활성화합니다. `app.getMap()`은 MapLibre 전용으로 유지되므로, Mapbox를 지원하는 플러그인은 `getStyleMap(app)`(`app.getMapboxMap()`으로 폴백)을 통해 지도를 읽고 두 엔진이 공유하는 Style Spec 표면에 머뭙니다. 규칙, 이를 강제하는 감사, 플러그인이 그린 레이어가 채택되는 방식은 [Mapbox 렌더러 지원](#)을 참고하세요. 벡터 가져오기 패널은 기존의 스토어 기반 지오메트리 브리지를 사용합니다.

Mapbox에서 사용 가능(2026년 9월 플러그인 감사, 각각 인증된 Mapbox 지도에 대해 브라우저에서 검증):

- **Layer Control, Basemaps, Deck.gl Layer, Components**(이전과 동일).
- **Web Services** — FEMA NFHL, NASA Earthdata (GIBS), US EPA EnviroAtlas, USGS National Map, USGS NLDI, Vantor, Planet Open Data, Earthdata GIS, OpenAerialMap, ArcGIS Hub, Socrata, CKAN, STAC Catalogs, Portolan, Source Cooperative, Natural Earth, Hugging Face, Esri Wayback, GeoLens. 도킹된 패널은 Mapbox 지도에 마운트되며, 컨트롤이 만드는 래스터 레이어는 컨트롤 자체의 id로 엔진에 채택됩니다(하나의 복사본, 스토어 가시성/불투명도/제거 적용, 베이스맵 교체 후 재구성). GeoLens 비공개 래스터는 MapLibre의 `setTransformRequest`만 주입할 수 있는 요청별 API 키가 필요합니다. 패널은 MapLibre 렌더러가 필요하다고 알려 주며, 공개 래스터와 벡터 데이터는 동작합니다.

- **Gridlines**와 **DGGS** 그리드(H3, S2, A5, DGGRID, DGGAL, OLC, Geohash, Tilecode). 셀 라벨과 클릭 식별을 포함합니다. Mapbox Standard의 루트 스타일에는 글꼴을 빌려 올 심볼 레이어가 없으므로, 라벨은 Mapbox의 `Open Sans Regular / Arial Unicode MS Regular` 글리프를 사용합니다.
- XYZ, WMS, GeoJSON, TiTiler 제공 COG 소스용 **Time Slider**. `gpu / wasm` 엔진(MapLibre 전용 타일 프로토콜)에 바인딩된 COG는 TiTiler를 통해 다시 추가되고, 모자이크 매니페스트는 콘솔 경고와 함께 제외됩니다. 픽셀 시계열 도구는 Mapbox에서도 점을 선택하고 표시하며(엔진의 클릭 이벤트와 투영된 DOM 마커), `Identify`는 MapLibre에서처럼 픽셀의 밴드 값을 읽습니다.
- **NetCDF** 샘플 마커, 3D 큐브의 "current view" 및 "draw" 범위, COG 스펙트럼 프로파일 클릭은 두 2D 엔진 모두에서 동작합니다.
- **Timelapse**. Mapbox 캔버스를 비디오로 녹화하는 기능을 포함합니다.
- **Elevation Profile**, **USGS LiDAR**(3DEP 인덱스 래스터는 네이티브로 채택되며, 포인트 클라우드는 이미 `deck.gl`로 그려졌음), **Mapillary**(커버리지 벡터 타일은 스토어에서 미러링된 플러그인 소유 네이티브 레이어).
- **Elements**(주석)와 **Dimensions**. MapLibre의 `Marker`는 `mapbox-gl` 지도에 추가할 수 없으므로, 핀, 스틱커 메모, 이미지 카드는 엔진 중립 마커(`annotation-marker.ts`)로 배치됩니다. MapLibre에서는 MapLibre 자체 마커, Mapbox에서는 `project()`로 위치를 다시 잡는 DOM 요소입니다.
- **Route Animation**(공유 스타일 API를 통한 마커, 궤적, 화살표 이미지. 렌더러 교체 시 다시 바인딩). 패널의 레이어 선택기는 Mapbox에서 인라인 GeoJSON 선 레이어를 나열합니다. 지오메트리가 지도 소스에만 있는 레이어(Add Vector Layer의 GeoJSON 모드)는 MapLibre의 `getData()`가 필요합니다.
- **Clouds**와 **Precipitation**(스토어 타일 레이어. 프레임 스크랩은 Mapbox에서 즉시 `setTiles` 단축 경로 대신 스토어를 거칩).
- **Geo Editor**. `Sketches`, 스토어 레이어의 제자리 지오메트리 편집, 지도 뷰 피처를 편집기로 로드하기를 포함합니다. `Geoman`은 엔진 호출을 하나의 지도 어댑터 뒤에 두며, MapLibre 객체를 만드는 세 멤버(정점 핸들러와 커서 마커 뒤의 `Marker`, 자르기 도구의 쿼리 상자 뒤의 `LngLatBounds`, 기본 마커 아이콘용 프로미스 방식 `loadImage`)는 Mapbox 지도에서 `mapbox-gl`의 것으로 교체됩니다(`geo-editor-mapbox.ts`). 툴바의 회전 및 피처 속성 팝업은 `maplibre-gl-geo-editor`의 `createPopup` 옵션에서 오며, 여기에 `mapbox-gl`의 `Popup`를 전달합니다. 텍스트 마커는 스타일에 빌려 올 글꼴이 없을 때 Mapbox의 `Open Sans Regular` 글리프를 사용합니다.
- **Atmospheric Effects**(오버레이 캔버스가 Mapbox 캔버스 컨테이너에 마운트되며, 컨트롤 컨테이너는 MapLibre에서처럼 그 위로 올려짐)와 **Sun**(캔버스 야간 마스크, 래스터 레이어, `setLight` 모두 `mapbox-gl`에 적용).
- **Flight Simulator**. `mapbox-gl`은 MapLibre가 제거한 자유 카메라를 유지했으므로, Mapbox 어댑터는 시점을 직접 배치합니다 — 고도를 담은 `MercatorCoordinate`와 `setPitchBearing` — 반면 MapLibre는 `calculateCameraOptionsFromCameraLngLatAltRotation`을 통해 변환합니다. 지형, 일시 중단된 상호작용 핸들러, 넓어진 피치 상한, 종료 뷰는 모두 MapLibre에서처럼 동작합니다.
`setCenterClampedToGround`에는 대응물이 없고 필요하지도 않습니다. 자유 카메라는 지도 중심이 아니라 시점을 배치하기 때문입니다. 한 가지 설정은 여기서 아무 효과가 없습니다. `mapbox-gl 3`에는 카메라 롤 축이 전혀 없으므로(자유 카메라 방향은 피치와 방위만으로 표현 가능하다고 문서화됨), **Bank the horizon in turns**(선회 시 수평선 기울이기)는 수평선을 수평으로 둡니다. 항공기는 여전히 기울어지고, 기울임은 여전히 선회를 만듭니다. Mapbox 렌더러가 기본인 동안 패널이 이를 알려 줍니다.

- **Street View**(Google 및 Mapillary). 업스트림 컨트롤이 건드리는 모든 것은 위치 마커를 제외하고 공유 표면에 있습니다. MapLibre의 Marker 는 위치가 갱신될 때마다 `map._camera.transform` 을 읽으므로 첫 지도 클릭에서 예외를 던졌습니다. `maplibre-gl-streetview` 0.8.0은 `createMarker` 팩토리를 받으며, 플러그인은 Mapbox 호스트에서 `mapbox-gl` 자체의 Marker 를 전달합니다. 컨트롤은 여전히 마커 요소와 방향 화살표를 소유하며, 위치 지정만 엔진이 바뀝니다. 렌더러 교체는 컨트롤을 다시 만들므로, 마커는 기본이 된 엔진을 따라갑니다.
- 네이티브 스타일 레이어용 **Layer Swipe**. 이 컨트롤은 두 엔진이 공유하는 표면만을 통해 두 지도를 구동하므로, 스스로 만든 하나의 지도 — 클립된 비교 창, 지금까지는 항상 MapLibre였던 — 는 `maplibre-gl-swipe` 의 `createMap` 옵션(0.13.0에서 추가)에서 오며, 여기에 `mapbox-gl`의 Map 을 전달합니다. 두 가지 Mapbox 특이 사항이 따라옵니다. 창에 액세스 토큰을 명시적으로 전달하고(`mapbox-gl`은 앱이 설정하지 않는 전역에서 토큰을 읽으므로, 토큰 없이 만든 두 번째 지도는 아무것도 렌더링하지 않음), 베이스맵 그룹화를 `basemapLayerIds` 로 시드합니다. `mapbox://` 스타일 URL은 가져올 수 없기 때문이며, 레이어 컨트롤이 자체 시드를 하는 것과 같은 이유입니다. 패널은 각 행을 구동하는 스타일 레이어 id가 아니라 Layers 패널이 보여 주는 이름으로 나열합니다. 엔진이 MapController와 같은 스타일 레이어 id → 이름 브리지 (`packages/map/src/layer-labels.ts`)를 공개하기 때문이며, 이것이 없으면 행이 `geolibre-mapbox-<id>-geojson-fill` 로 표시되었습니다. **deck.gl 래스터 제공자는 MapLibre 전용으로 남습니다.** COG 및 `maplibre-gl-raster` 레이어는 기본 Mapbox 지도에서 deck.gl 어댑터를 통해 그릴 수 있지만, 스와이프 제공자는 이를 비교 창으로 미러링할 수 없습니다. 따라서 스와이프 패널은 해당 레이어를 생략합니다. 알려진 결함 하나: 스와이프가 활성화된 동안 베이스맵을 바꾸면 이전 비교 창 — 그리고 그 안의 지도, 살아 있는 WebGL 컨텍스트 — 이 Mapbox 캔버스에 고아로 남습니다. 스와이프 자체는 새 베이스맵에 대해 계속 동작합니다. MapLibre에서 같은 순서를 실행하면 창이 하나만 남으므로, 이는 플러그인이 아니라 Mapbox 컨트롤 생명주기에 있는 문제입니다. #2430에서 추적 중입니다.
- **GeoAgent**. 거의 모든 도구가 이미 공유 Style Spec 표면에 있었으나 네 개는 그렇지 않았고, 각각 다른 방식으로 고장 났습니다. `add_marker` 는 MapLibre의 Marker / Popup 을 만들었고, `set_projection` 은 `{ type }` 을 썼으며(Mapbox는 이름 문자열을 받음), `get_map_state` 는 `projection.type` 을 읽었고, `run_maplibre_script` 는 사용자가 작성한 코드에 잘못된 네임스페이스를 건넸습니다. 단순히 마운트에 실패하는 컨트롤보다 여기서 더 중요한 이유는, 에이전트 실행이 이미 지도를 바꾼 뒤 중간에 깨지기 때문입니다. `maplibre-gl-geoagent` 0.6.1은 엔진을 하나의 옵션(`mapEngine`)으로 받고 네 도구 모두가 이를 따르므로, 플러그인은 호스트의 엔진을 한 번 지정하고(`geoagent-map-engine.ts`) 전체 `mapbox-gl` 네임스페이스를 넘겨줍니다. `run_maplibre_script` 는 이를 실행하는 스크립트에 그대로 전달합니다. 에이전트 오버레이는 MapLibre에서처럼 엔진이 채택하는 플러그인 소유 행으로 Layers 패널에 도달합니다. `set_sky / clear_sky` 는 MapLibre 전용으로 남습니다. `mapbox-gl`에는 `setSky` 가 없으며(하늘을 스타일 레이어로 그림), 도구는 조용히 실패하는 대신 이를 보고합니다.
- **Overture Maps**. `mapbox-gl` 3.30+는 `api.mapbox.com` (데스크톱 및 웹 CSP에서 허용 목록에 포함)에서 가져오는 타일 제공자를 통해 `.pmtiles` 아카이브를 직접 읽으므로, 플러그인은 `maplibre-gl-overture-maps` 에 `nativePmtiles` 옵션을 전달하고 컨트롤은 MapLibre의 `pmtiles://` 프로토콜을 등록하는 대신 일반 https 아카이브 URL을 추가합니다. 검사 팝업은 컨트롤의 `createPopup` 옵션에서 오며, 여기에 `mapbox-gl`의 Popup 을 전달합니다. Layers 패널 미러는 플러그인 소유 행이므로(엔진이 컴파일하지도 다시 그리지도 않음), 해제, 가시성, 불투명도, 색상, 내보내기가 모두 MapLibre에서처럼 동작합니다. Style 패널의 buildings 테마 3D 돌출은 MapLibre 레이어 동기화 기능이며 MapLibre 전용으로 남습니다.

MapLibre 내부 의존성 때문에 발목이 잡힌 플러그인은 없습니다. MapLibre 전용으로 남아 있는 것은 플러그인이 아니라 지도 기능입니다.

포팅과 함께 두 가지 엔진 변경이 있었으며 모든 플러그인에 적용됩니다. Mapbox 소스가 아직 로드 중일 때 추가된 스토어 레이어는 이제 그 소스가 완료되면 동기화되고(이전에는 `idle` 을 기다렸는데, 애니메이션 캔버스 소스가 있는 지도는 결코 그 상태에 도달하지 않음), 공유 페인트 빌더는 플러그인 소유 네이티브 레이어의 불투명도를 스토어 불투명도로 대체하는 대신 곱해 조정합니다.

오프라인(로컬 PMTiles) 베이스맵도 MapLibre 전용입니다. `pmtiles://` 소스 프로토콜이 Mapbox에 등록되어 있지 않으므로, 프로젝트 베이스맵이 오프라인 아카이브인 Mapbox 창은 기본 베이스맵으로 폴백합니다(콘솔 경고와 함께). 대신 공유 Basemaps 패널에서 Mapbox 스타일을 선택하세요. MapLibre 사용자 지정 프로토콜, 브리지의 구체화 한계를 넘는 타일/스트리밍 벡터 가져오기, 사용자 지정 COG 지형, 기타 플러그인 소유 레이어는 추가 어댑터가 필요합니다. **사용자 지정 지형 소스**는 그중 자체 UI가 있는 것입니다. `mapbox-gl`에는 COG가 뒷받침할 수 있는 `raster-dem` 소스가 없으므로, Terrain 설정 대화 상자의 "Terrain source" 섹션(COG URL, 로컬 파일, 또는 이미 지도에 있는 래스터 레이어)은 이 렌더러에서 숨겨지며 그 이유를 설명합니다. Mapbox 자체의 전역 지형과 수직 과장 슬라이더는 계속 동작합니다.

`deck.gl`로 그려지는 레이어에는 어댑터가 필요 없습니다. `@deck.gl/mapbox` 는 Mapbox GL JS를 네이티브로 대상으로 하므로, 엔진은 `capabilities.deckOverlay` 를 보고하고 공유 인터리브 오버레이는 `app.getMapboxMap()` 을 통해 Mapbox 지도에 바인딩됩니다. 표시된 미지원 레이어는 조용히 생략되는 대신 지도에 오류를 보고합니다. 히트맵과 클러스터 점 렌더러는 네이티브 Mapbox 레이어로 컴파일되며, 클러스터 레이어의 작성된 필터는 MapLibre에서처럼 클러스터링 전에 데이터에 적용됩니다. Style 패널의 심볼로지는 MapLibre에서처럼 네이티브 Mapbox 레이어로 컴파일됩니다. 마커 및 KML 아이콘, Geo Editor 텍스트 마커, 채우기 패턴과 선 장식(생성된 스프라이트는 `styleimagemissing` 을 통해 제공), 반전 채우기와 지오메트리 생성기(보조 GeoJSON 소스), 줌 단계 돌출 아래의 평면 채우기, 속성 라벨(중복 제거된 라벨은 집계된 보조 소스를 읽으며, 데이터 정의 크기, 색상, 불투명도, 가시성, 우선순위 표현식이 적용됨). 레이어 혼합 모드는 재현되지 않으며, Style 패널은 혼합 모드를 설정한 레이어에서 이를 알려 줍니다. 대용량 GeoJSON(50,000 피처 초과)은 `mapbox-gl` 자체의 GeoJSON 소스를 유지합니다. MapLibre는 이런 레이어를 `geojson-vt` 프로토콜을 통해 벡터 타일로 제공하고 `mapbox-gl`에는 `addProtocol` 훅이 없지만, 그 소스는 이미 워커에서 `geojson-vt`로 데이터를 타일링합니다. 200,000개의 점으로 레이어 추가에 Mapbox에서 약 2.1초, MapLibre에서 1.5초가 걸렸고, 스타일 변경은 0.9초 대 0.8초, 데이터 편집은 2.1초 대 1.2초였으며, 가장 긴 메인 스레드 정지는 Mapbox 쪽이 더 짧았습니다(0.8초 대 0.85초).

Mapbox Standard는 공유 불투명도 설정이 있는 로컬 스타일 임포트로 로드됩니다. Background 카드는 프로젝트 레이어와 Standard의 구성을 보존하면서 육지와 수역 색상, 라벨(해양 라벨 포함), 3D 객체, 대기를 페이드합니다.

라이선스 및 약관

GeoLibre 자체는 MIT 라이선스이지만, Mapbox 렌더러는 오픈 소스가 아닌 **Mapbox GL JS v3**(작성 시점 기준 `mapbox-gl 3.32.0`)에 의존합니다. Mapbox GL JS v3는 **Mapbox 서비스 약관**과 **라이선스**에 따라 배포되며, 활성 Mapbox 계정이 필요하고, 해당 계정의 액세스 토큰 및 관련 Mapbox 제품과 함께만 사용할 수 있으며, 약관은 SDK

의 과금, 회계, 데이터 수집 코드 변경을 제한합니다. 사용량 기반 과금과 Mapbox의 저작자 표시 요구 사항은 사용자의 계정에서 Mapbox 서비스를 어떻게 사용하는지와 적용 약관에 따라 달라지므로, 제품에서 렌더러를 활성화하기 전에 해당 약관을 확인하세요. SDK는 @geolibre/map의 런타임 의존성이므로, 이 패키지의 npm 사용자와 데스크톱 및 웹 배포판은 MapLibre가 활성 렌더러로 유지되더라도 이를 받게 되지만, Mapbox 창이 열리기 전까지는 Mapbox 코드가 실행되지 않습니다(그리고 어떤 Mapbox 서비스에도 접속하지 않습니다).

로딩 및 크기

Mapbox의 JavaScript와 CSS는 Mapbox 창이 마운트될 때만 가져옵니다. 프로덕션 빌드는 이들을 별도 청크로 분리하고 PWA의 초기 프리캐시에서 제외합니다. 따라서 MapLibre 시작 시 Mapbox 엔진을 다운로드하지 않습니다. 데스크톱/웹 배포 아티팩트에는 여전히 포함됩니다. Mapbox GL JS 3.30.0 기준으로 엔진과 스타일시트는 원본 약 1.91 MB, gzip 시 532 KB를 추가하며, 여기에는 지도 타일과 기타 서비스 응답은 포함되지 않습니다.

Add Data 호환성

Add Data 메뉴는 Mapbox가 로드를 마칠 때까지 기다린 뒤 가져오기를 받으므로, 일찍 클릭해도 패널 열기 요청이 사라지지 않습니다.

Add Data 메뉴와 명령 팔레트는 구현되지 않은 MapLibre 프로토콜이나 사용자 지정 렌더 패스가 필요한 로더를 분류합니다. 이러한 항목은 메뉴에 표시되지만 Mapbox 호환성 안내와 함께 비활성화됩니다. MBTiles와 Gaussian Splatting이 해당합니다. Cesium Ion 및 CZML 씬 로더는 Cesium 전용으로 남으며, KML / KMZ는 모든 렌더러에서 열리고, 지구본 밖에서는 호스트 KML 가져오기 도구(끌어다 놓기 경로)를 거칩니다.

Deck.gl Layer, 3D Model, DuckDB는 활성화되어 있습니다. 앞의 둘은 공유 인터리브 deck.gl 오버레이를 통해 (3D Tiles가 이미 그랬듯이), 세 번째는 DuckDB 패널 자체의 deck.gl 오버레이를 통해 렌더링됩니다. 모든 deck.gl 오버레이와 마찬가지로 레이어가 표시되는 동안 지도를 Mercator 투영으로 유지하므로, 하나를 추가하면 지구본 뷰가 Mercator로 전환됩니다. Deck.gl Layer 데이터는 MapLibre에서처럼 프로젝트에 인라인으로 저장됩니다. DuckDB 패널은 렌더러 교체 후 라이브 지도에 다시 마운트되며, 패널에만 캐시되어 있던 결과는 패널을 다시 열면 다시 그려지고, 프로젝트에서 복원된 쿼리 레이어는 MapLibre에서와 똑같이 쿼리를 다시 실행해야 합니다.

FlatGeobuf는 Mapbox에서 공유 벡터 가져오기 도구를 사용합니다. ArcGIS 벡터 타일 서비스는 해석된 타일 소스, 서비스 스타일, 분류 필터, 가시성, 불투명도를 유지합니다. STAC은 MapLibre와 Mapbox 모두에서 카탈로그 탐색, 범위 검색, bbox 그리기, 풋프린트, 선택을 지원하며, 원격 벡터 PMTiles와 Zarr 자산은 어느 쪽에서든 추가할 수 있습니다.

NetCDF/HDF 파일과 직접 읽을 수 있는 원격 파일은 두 엔진 모두에서 선택한 평면을 이미지로 렌더링합니다. 시간 축이 있는 큐브와 Kerchunk 참조는 Mapbox에서도 MapLibre와 똑같이 Zarr 렌더러를 거칩니다.

브라우저 검증

2026년 9월 감사는 활성화된 37개의 Add Data 항목을 모두 열어 보고, 기존의 비활성화된 항목을 확인했으며, 아래의 공개 데이터 경로를 인증된 Mapbox 지도로 실행해 보았습니다. 패널이 마운트된 것만으로는 가져오기 성공이 아니며, 표는 검증 수준을 기록합니다. 백엔드 및 서비스 제한은 명시적으로 포함되어 있습니다.

패널	결과
Vector	미국 주 GeoJSON: 52개 피쳐 가져오기 및 렌더링
Raster	공개 DEM GeoTIFF: GPU 래스터 표시
Delimited Text	미국 도시 CSV: 109개 점 가져오기 및 렌더링
CAD	미국 주 DXF: 58개 엔티티 발견, EPSG:5070 가져오기 실행
File Geodatabase	패널은 열림. 로컬 GDAL/사이드카 워크플로는 Desktop 필요
Geotagged Photos	EXIF 샘플 JPEG: 위치가 있는 사진 1장 가져오기
GPX	Fells Loop: 86개 웨이포인트와 경로 1개. 두 네이티브 소스 모두 생성됨
Encoded Polyline	정밀도 5 샘플: 선 1개 가져오기 및 렌더링
MBTiles	비활성화: 로컬 사용자 지정 프로토콜에 Mapbox 어댑터 없음
OSM PBF	모나코 추출본: 4,249개 점, 4,002개 선, 2,341개 폴리곤 가져오기 및 표시
XYZ	USGS 영상 샘플: 네이티브 래스터 소스 마운트
WMS	USGS NAIP 샘플: 네이티브 래스터 소스 마운트
CSW Catalog	Open Canada 카탈로그 검색. Manitoba Economic Regions를 8개 GeoJSON 피쳐로 가져옴
WFS	MapServer 대륙 서비스 가져오기. GeoServer 샘플은 원격 서비스에 의해 차단됨
WMTS	EOX Sentinel-2 cloudless 샘플: 네이티브 래스터 소스 마운트
OGC API - Features	pygeoapi 호수 샘플: 25개 피쳐 가져오기 및 렌더링
OGC Vector Tiles	PDOK BGT 샘플: 네이티브 벡터 타일 소스 마운트
ArcGIS	4,186개 도시 피쳐 렌더링. Santa Monica 필지는 7개 서비스 스타일 레이어를 모두 사용해 렌더링
GeoRSS	USGS 일일 지진 피드: 34개 피쳐 가져오기(실시간 개수는 변동)
STAC	Earth Search 연결. Sentinel-2 검색 풋프린트 20개 추가
Video	Mapbox 해안 비디오 샘플: 네이티브 비디오 소스 마운트

패널	결과
Deck.gl	Scatterplot 샘플(맨해튼 점)을 공유 deck.gl 오버레이로 렌더링. 가시성과 불투명도는 레이어 스토어를 따름
GeoParquet	미국 주: 52개 피쳐 가져오기 및 렌더링
FlatGeobuf	국가: 179개 피쳐를 공유 벡터 브리지로 가져옴
PMTiles	원격 벡터 아카이브는 네이티브 Mapbox 소스 사용. Tilezen의 9개 소스 레이어와 Mapbox의 지진 아카이브 렌더링
Zarr	CarbonPlan 기후 샘플을 패널로 추가: 사용자 지정 레이어가 Mapbox 지도에 마운트되고 피라미드(6개 레벨, band/month 축)를 로드. 페인트 가능한 토큰 없이 확인했으므로 픽셀 출력은 확인되지 않음. 렌더러의 Mapbox 지원은 업스트림의 것
NetCDF / HDF	기온 파일: 선택한 시간 슬라이스를 네이티브 이미지로 추가
LiDAR	Autzen COPC 렌더링(아카이브 점 10,653,336개). 소형 PDAL COPC 픽처는 1,065개 점 로드
Gaussian Splatting	패널은 열림. 사용자 지정 렌더링 미지원이며 항목 비활성화
3D Tiles	AGI 본사 타일셋을 deck.gl로 렌더링. 고도 배치, 가시성, 복원에 회귀 테스트 커버리지 있음
Cesium Ion	비활성화: Cesium 전용
CZML	비활성화: Cesium 전용
KML / KMZ	호스트 KML 가져오기 도구를 통해 GeoJSON, 지면 오버레이, 모델 레이어로 가져옴. 끌어다 놓은 파일과 같은 경로
3D Model	상하이 샘플 모델을 scenegraph 빌더로 배치
DuckDB	NYC 샘플 데이터베이스 쿼리. 결과 레이어가 렌더링되었고 MapLibre → Mapbox 렌더러 교체에도 유지됨
PostgreSQL	패널이 Desktop/Martin 요구 사항을 설명. 데이터베이스 연결은 테스트하지 않음
Apache Iceberg	패널은 열림. 가져오기를 위한 테이블/카탈로그 연결은 제공하지 않음

오픈트인 회귀 스위트가 FlatGeobuf, ArcGIS 벡터 타일, STAC, 메뉴 경계 검사를 밝은 테마와 어두운 테마에서 반복합니다:

```
# Supply a public token in the environment before running.
npm exec -- playwright test e2e/mapbox-add-data.spec.ts
```

MAPBOX_TOKEN 이 없으면 테스트는 건너뛰니다. 라이브 공개 서비스를 사용하므로, 원격 서비스 가용성도 이 통합 검사의 일부입니다.

PMTiles 및 3D 어댑터

PMTiles 패널은 아카이브 탐색과 소스 레이어 선택 UI를 유지하지만, Mapbox 가져오기는 레이어 스토어를 거칩니다. 선택한 각 소스 레이어는 독립적인 Mapbox 소스와 편집 가능한 스타일을 가집니다. 이 경로는 URL 경로가 `.pmtiles`로 끝나는 원격 HTTP(S) 벡터 아카이브를 지원하며, 래스터 아카이브, 로컬 파일, 오프라인 PMTiles 베이스맵은 여전히 지원되지 않고 로드 오류를 받습니다.

LiDAR는 기존 포인트 클라우드 로더와 별도의 `deck.gl` 캔버스를 사용합니다. 표준 3D Tiles는 공유 인터리브 오버레이를 사용하며 MapLibre와 같은 프로젝트 소스 레코드를 유지합니다. 둘 다 Mercator가 필요합니다. 타일셋 고도 오프셋은 지오메트리와 순회 경계를 함께 이동시키므로, 낮춰진 타일도 확대하거나 저장된 프로젝트를 다시 열 때 계속 보입니다. 3D Tiles 패널은 로딩 및 가져오기 오류를 보고하며, 가시성, 불투명도, 제거는 레이어 스토어를 따릅니다.

유틸리티 `e2e/mapbox-archives-3d.spec.ts`는 공개 PMTiles, COPC, AGI 타일셋 URL을 사용하고 밝은 테마와 어두운 테마에서 저장/다시 열기를 확인합니다. 런타임에 `MAPBOX_TOKEN`을 설정한 다음 `npx playwright test e2e/mapbox-archives-3d.spec.ts`로 실행하세요. Google Photorealistic 및 인증된 I3S 서비스는 자체 자격 증명에 필요하며, 이러한 서비스와 모든 3D Tiles 확장은 이 테스트에서 다루지 않습니다.

<https://geolibre.app/cesium-renderer/>

Cesium 렌더러

기본 워크스페이스를 네이티브 CesiumJS 지구본으로 렌더링하려면 **View → Rendering engine → Cesium**(보기 → 렌더링 엔진 → Cesium)을 선택하세요. Cesium은 모든 분할 패널의 렌더링 엔진 메뉴에서도 선택할 수 있으므로, 프로젝트에서 지구본을 다른 세 엔진 중 어느 것과도 나란히 배치할 수 있습니다. 기본 맵과 패널의 선택은 프로젝트와 함께 저장됩니다.

자격 증명

지구본은 Cesium ion 토큰 없이도 동작합니다. 프로젝트 베이스맵을 기본 영상으로 입히고, 키가 필요 없는 Terrarium 고도(또는 구성된 COG)를 지형에 사용하며, 토큰 설정으로 연결되는 힌트를 표시합니다. 토큰을 추가하면 Cesium World Terrain, 래스터 형태가 없는 베이스맵의 대체 영상인 Ion World Imagery, 그리고 ion에 호스팅된 에셋에 접근할 수 있습니다. 호스팅된 웹 버전에는 데모 토큰이 번들로 포함되어 있으며, 데스크톱 및 모바일 앱에서 해당 기능을 사용하려면 자신의 토큰이 필요합니다.

토큰은 **Settings → Environment Variables → Cesium Ion token**(설정 → 환경 변수 → Cesium Ion 토큰)에 입력하세요. 토큰은 프로젝트 파일이 아니라 현재 기기에 저장됩니다. 개발자는 빌드 시점에 `CESIUM_TOKEN` 또는 `VITE_CESIUM_TOKEN` 을 설정할 수도 있습니다. [선택적 3D 지구본 자격 증명](#)을 참조하세요.

지원되는 경로

- 범주형, 단계 구분, 규칙 기반, 표현식, 비례, 마커, 클러스터, 레이블, 패턴, 돌출(extrusion), Z값 인식 렌더링을 포함한 스타일이 적용된 GeoJSON.
- XYZ, WMS, WMTS, COG, 래스터 PMTiles, 로컬 MBTiles, 이미지 오버레이, 그리고 GeoLibre의 프로토콜 브리지를 통해 연결되는 기타 타일 소스.
- Style Specification 출력이 보존되도록 숨겨진 MapLibre 렌더러를 통해 입혀지는 벡터 타일, 벡터 PMTiles, 벡터 MBTiles.
- 네이티브 3D Tiles, Cesium ion 타일셋 및 영상, CZML, KML/KMZ, I3S 장면 레이어, 지원되는 포인트 클라우드, Gaussian splat 타일셋.
- 지구본 네이티브 선택(picking) 및 강조 표시, 범위 그리기, 드래그 가능한 배치, 스크린샷, 인쇄 레이아웃, 비디오 및 카메라 투어, 지형을 고려한 고도 프로필, 환경 효과.

Layers(레이어) 패널은 지원되지 않는 콘텐츠를 삭제하지 않고 유지합니다. 그러한 레이어는 활성 엔진에서 사용할 수 없는 것으로 표시되며, 프로젝트를 호환되는 렌더러로 전환하면 다시 나타납니다. 플러그인도 같은 모델을 따릅니다. `engines` 목록에 `cesium` 이 포함된 플러그인만 활성 상태로 유지됩니다.

장면과 카메라

지구본은 3D, 2D, Columbus 장면 모드, 지형, 네이티브 지구본 조명, 그리고 GeoLibre의 공유 경도, 위도, 줌, 방위 (bearing), 피치(pitch) 상태와 상호 변환되는 카메라를 제공합니다. 혼합 패널은 원시 줌 값이 아니라 지상 해상도를 기준으로 동기화되므로, 패널 크기가 서로 다르더라도 겹보기 축척이 일치하게 유지됩니다.

로딩과 아키텍처

Cesium의 JavaScript는 첫 번째 Cesium 캔버스가 마운트될 때만 로드됩니다. Workers, Assets, Widgets는 애플리케이션과 함께 배치되며 `CESIUM_BASE_URL` 에서 확인됩니다. 다른 엔진을 선택하면 Cesium 캔버스가 언마운트되고 렌더링 리소스가 해제됩니다.

CesiumEngine 은 공유 MapEngine 인터페이스를 구현하고, CesiumLayerSync 는 프로젝트 레이어를 Cesium 데이터 소스, 영상 레이어, 프리미티브로 변환합니다. 자세한 구현, 호환성 경계, 카메라 변환, 영속성 모델은 [3D 지구본 뷰\(CesiumJS\)](#)를 참조하세요.

CesiumJS는 Apache 2.0 라이선스의 오픈 소스입니다. Cesium ion은 자체 계정 약관과 사용 한도가 있는 별도의 호스팅 서비스입니다.

<https://geolibre.app/arcgis-renderer/>

ArcGIS 렌더러

View → Rendering engine → ArcGIS(보기 → 렌더링 엔진 → ArcGIS)를 선택하면 **ArcGIS Maps SDK for JavaScript**로 프로젝트를 렌더링합니다. 이 엔진은 각 분할 창의 렌더링 엔진 메뉴에서도 사용할 수 있습니다. MapLibre가 기본값으로 유지됩니다. 프로젝트는 기본 및 보조 렌더러 선택을 저장하며, Python 및 iframe API는 렌더러 이름으로 `arcgis`를 받습니다.

아무것도 변들되지 않습니다

SDK는 GeoLibre 빌드의 일부가 **아닙니다**. `@arcgis/core`는 18,810개 파일에 걸쳐 압축 해제 시 84 MB이며, 이를 번들하면 웹 빌드, Python 휠, 데스크톱 설치 프로그램이 수십 MB 커집니다. 대신 엔진은 ArcGIS 창이 처음 마운트될 때 Esri의 버전별 ES 모듈 CDN (<https://js.arcgis.com/<version>/@arcgis/core/...>)에서 필요한 모듈을 가져옵니다. PostGIS 및 Sedona SQL 엔진을 jsDelivr에서 가져오는 것과 같은 방식입니다. GeoLibre와 함께 배포되는 것은 `packages/map/src/arcgis-*.ts`의 어댑터 코드뿐입니다.

이에 따른 결과:

- 첫 ArcGIS 창은 데스크톱에서도 네트워크 접근이 필요합니다. 서비스 워커가 처음 사용 후 SDK 모듈과 스타일시트를 캐시하므로(`vite.config.ts`의 `geolibre-arcgis-sdk`), 이후에는 다른 CDN 로드 엔진과 마찬가지로 SDK 자체는 오프라인으로 부팅됩니다. Esri 베이스맵, 타일 서비스 및 기타 원격 레이어는 다른 모든 렌더러와 마찬가지로 여전히 네트워크가 필요합니다.
- SDK 버전은 `packages/map/src/arcgis-sdk.ts` (`ARCgis_SDK_VERSION`)에 고정되어 있습니다. 이를 올리는 것은 의도적인 변경이며, [유지 관리](#)를 참고하세요.
- Esri는 ES 모듈 CDN을 프로토타이핑 경로로 문서화하고 있으며 세션당 한 번 "Only use ES modules from ArcGIS CDN for testing"을 로그에 남깁니다. Esri가 지원하는 프로덕션 경로는 npm 빌드인데, 이 통합은 크기 때문에 바로 그것을 피합니다. 첫 로드는 수백 개의 작은 모듈을 가져오므로, 세션의 첫 ArcGIS 창은 Mapbox 창보다 나타나는 데 오래 걸립니다.
- 데스크톱 앱(`tauri.conf.json`)과 Docker 이미지(`docker/nginx.conf`)의 콘텐츠 보안 정책은 `script-src`와 `font-src`(SDK의 아이콘 및 텍스트 글꼴)에 <https://js.arcgis.com/>을 허용 목록에 넣습니다. 두 정책 모두 외부 스타일시트를 허용하지 않으므로 SDK의 스타일시트는 텍스트로 가져와 인라인됩니다.

API 키

렌더러는 키 없이도 동작합니다. 그 경우 새 프로젝트는 ArcGIS 베이스맵으로 키가 필요 없는 **Esri World Imagery**를 표시합니다. 공유 **Basemaps**(베이스맵) 패널에서 선택한 베이스맵은 SDK가 그릴 수 있는 타일로 변환되며(3D 지구본이 사용하는 것과 같은 변환), 타일 형태가 없는 스타일에는 World Imagery가 폴백으로 사용됩니다. 사용자의 레이어는 SDK 자체의 레이어 클래스를 통해 그려집니다.

ArcGIS API key(ArcGIS API 키)를 추가하면 Esri의 베이스맵 스타일을 사용할 수 있습니다. ArcGIS 계정(ArcGIS Online 또는 ArcGIS Location Platform)의 **Content → New item → Developer credentials → API key credentials**(콘텐츠 → 새 항목 → 개발자 자격 증명 → API 키 자격 증명)에서 **Basemaps**(베이스맵) 권한을 부여해 키를 만든 다음, **Settings → Environment Variables → ArcGIS API key**(설정 → 환경 변수 → ArcGIS API 키)에 붙여 넣고 **Save Settings**(설정 저장)를 클릭하세요. Mapbox 및 Cesium 토큰과 마찬가지로 이 키는 프로젝트 파일 밖, 이 기기에 저장됩니다. 또는 개발 서버를 환경에 `ARCGIS_API_KEY`를 넣어 실행할 수도 있습니다. 키가 변경되면 ArcGIS 지도가 다시 생성됩니다. 키로 보낸 요청은 해당 계정에 계량되며 Esri의 약관을 따릅니다. 베이스맵 요청에는 넉넉한 무료 할당량이 있습니다.

키가 있으면 새 프로젝트는 ArcGIS 렌더러에 **ArcGIS Streets**를 사용합니다. ArcGIS가 활성화되어 있고 키가 구성된 경우, **Change background**(배경 변경) 선택기는 Esri의 Streets, Navigation, Topographic, Light Gray, Dark Gray, Imagery, Imagery (no labels), Oceans, Outdoor, OpenStreetMap 스타일을 제공합니다. 하나를 선택해도 카메라는 유지됩니다. 선택은 `preferences.map.arcgisBasemap ( arcgis/streets , arcgis/imagery , osm/standard` 같은 Esri 베이스맵 스타일 id)으로 저장됩니다. ArcGIS가 활성화된 상태에서 공유 **Basemaps** 패널에서 베이스맵을 선택하면 이 값이 지워지고, 창은 다시 공유 MapLibre/Cesium 베이스맵을 따릅니다. 키가 없으면 이 재정의는 대신 Esri World Imagery를 그림니다.

지원되는 경로

- 네이티브 GeoJSON. 벡터 가져오기 도구가 구체화한 데이터, FlatGeobuf 및 GeoParquet 가져오기, 프로젝트에 이미 로드된 ArcGIS 피쳐 레이어를 포함합니다. 점, 선, 폴리곤 심볼로지, 채우기 및 외곽선 불투명도, 원 반경, 라벨(필드 또는 표현식, 크기, 색상, 헤일로, 배치, 오프셋, 회전, 대소문자 변환), 그리고 데이터 기반 색상 모드 — 범주형, 단계 구분, 규칙 기반, 표현식, `simplestyle` — 가 모두 렌더링됩니다. SDK에는 MapLibre Style Spec이 없으므로, 엔진은 `@geolibre/core`가 2D 지도용으로 만드는 것과 같은 MapLibre 표현식을 `style-spec` 엔진으로 피쳐별로 평가해 그 결과를 피쳐에 구워 넣습니다. 레이어의 렌더러는 그 결과 심볼 키에 대한 `unique-value` 렌더러입니다. 레이어 필터, 빠른 필터, 타임 슬라이더의 필터, 임베드 필터는 피쳐가 SDK에 도달하기 전에 같은 방식으로 적용됩니다. 줌 의존 표현식(미터 단위 외곽선, 규칙별 줌 범위)은 정수 줌이 바뀔 때 다시 평가됩니다.
- 심볼로지 팩은 필터링된 피쳐로 만든 보조 레이어를 통해 그려집니다. **반전 채우기**는 폴리곤을 잘라낸 전 세계 마스크로(Mapbox에서처럼 Web Mercator 위도 한계로 클램프), **지오메트리 생성기**(중심점, 경계 상자, 볼록 껍질, 버퍼)는 자체 채우기, 외곽선, 비례 원 크기로, **선 장식**은 스타일의 간격으로 도형을 반복하고 화살표를 선 방향에 맞춰 돌리는 CIM 마커 라인으로 그려집니다(평면 `MapView`에서만 가능하며, SDK는 씬에서 CIM 선 심볼을 그리지 않습니다). **라벨 중복 제거**는 라벨 전용 레이어의 집계된 점에 라벨을 붙입니다. 라벨 크기, 색상, 불투명도, 가시성 표현식은 피쳐별로 평가되어 해석된 크기와 색상별로 하나의 라벨 클래스로 묶입니다. SDK의 라벨 심볼이 데이터 기반이 아니기 때문입니다(연속 램프는 수십 개의 클래스로 구간화됩니다). 레이어의 불투명도는 라벨에도 적용되므로, 불투명도 표현식으로 라벨을 레이어보다 더 불투명하게 만들 수는 없습니다. 보조 레이어는 식별이나 선택의 대상이 아닙니다.
- HTTP(S) 래스터 타일(XYZ, WMTS 타일 템플릿), WMS(GetMap 템플릿을 SDK의 `WMSLayer` 설명으로 분해), 그리고 이름이 지정된 소스 레이어가 있는 벡터 타일(Mapbox 엔진이 컴파일하는 것과 같은 스타일 레이어에서 SDK의 `VectorTileLayer`로 그림). 마커, 선 장식, 채우기 패턴은 GeoLibre가 생성한 이미지이며,

SDK가 요청 인터셉터를 통해 읽는 스프라이트 시트에 구워 넣어집니다. 라벨은 Esri의 공개 글리프 서비스에서 가져온 Arial을 사용하므로 네트워크가 필요합니다. 소스의 최대 zoom을 넘어서면 SDK는 마지막 타일을 확대하며 패턴과 장식 간격도 함께 확대되는 반면, MapLibre는 이를 선명하게 유지합니다. `WebTileLayer`가 표현할 수 없는 래스터 템플릿 — `{bbox-epsg-3857}` 요청, TMS 스킴, `{-y}` 또는 `{quadkey}`, 512 px 타일 크기, 소스 `minzoom / maxzoom`, 여러 템플릿 — 은 MapLibre가 하는 것처럼 각 타일을 요청하는 사용자 지정 타일 레이어를 통해 그려지며, 512 px 타일은 잘라 내고 `maxzoom`을 넘어서는 오버zoom합니다. 등록된 MapLibre 프로토콜상의 템플릿(예: 데스크톱 앱의 CORS 면제 `geolibre-wms:// WMS` 페치)은 지구본과 마찬가지로 해당 프로토콜의 핸들러에 요청합니다.

- **Add Data → ArcGIS Layer**(데이터 추가 → ArcGIS 레이어)는 캐시된 MapServer 및 ImageServer 서비스를 타일 템플릿으로, 동적 서비스(하위 레이어, 렌더링 규칙, 사용 가능한 캐시 없음)를 `/export / / exportImage` 경계 상자 템플릿으로 저장하며, 둘 다 위와 같이 그려집니다. FeatureServer 레이어와 `type: "arcgis"`로 기록된 모든 서비스(수작업 작성 또는 Python 레코드)는 SDK 자체의 `FeatureLayer`, `TileLayer`, `MapImageLayer`, `ImageryLayer`를 통해 그려집니다. 단일 MapServer 하위 레이어 (`MapServer/2`)도 `FeatureLayer`입니다. 서비스 레이어는 2D 지도에서처럼 레이어 스타일로 그려지며, 식별된 피쳐는 하이라이트할 수 있고, 피쳐는 GeoJSON으로 다시 읽어 올 수 있습니다(서비스의 요청당 레코드 제한을 넘어 최대 50개 요청까지 페이지 처리). FeatureServer 레이어의 필터(빠른 필터, 표현식 필터, 시간 및 임베드 필터)는 서비스의 SQL `definitionExpression`이 됩니다. SQL 형태가 없는 필터는 지도의 배너에 보고되며 서비스는 필터 없이 그려집니다.
- 지오레퍼런스된 이미지. 제어점 지오레퍼런스를 통해 네 모서리로 배치되므로, 회전되거나 기울어진 맞춤도 MapLibre에서와 같은 위치에 놓입니다.
- 공유 레이어/그룹 가시성, 불투명도, 순서. 동기화 또는 독립 분할 뷰 카메라. 프로젝트의 줌 및 경계 제약.
- Style 패널의 래스터 밝기, 대비, 채도, 색조 슬라이더는 래스터 레이어에 대한 SDK의 CSS 필터 `effect`로, 모든 레이어의 혼합 모드는 SDK의 `blendMode`로 적용됩니다(`add`는 SDK의 `plus`). `SceneView`는 래스터 효과를 무시하며, 타일 및 영상 레이어만 혼합합니다.
- Geo Editor 텍스트 마커와 주석 텍스트는 각자의 색상으로 해당 지점에 텍스트로 그려집니다.
- 피쳐 선택(레이어의 팝업 템플릿 및 필드 가시성을 사용하는 클릭 식별. **Identify visible layers**(표시된 레이어 식별)는 WMS `GetFeatureInfo`, Time Slider 픽셀, COG/NetCDF 픽셀, DuckDB 레이어를 포함해 모든 레이어의 결과를 묶어 보여 줌), 호버 맵 팁(레이어의 호버 필드를 Mapbox에서처럼 스토어의 피쳐에서 가져옴), Identify 도구 없이 클릭 시 나타나는 지오태그 사진 팝업, 선택 하이라이트, 범위 그리기, 드래그 가능한 배치, 엔진 수준 이미지 캡처. **Layers → Select features**(레이어 → 피쳐 선택, 단일, 사각형, 폴리곤, 자유형, 반경)는 기본 지도에서 MapLibre와 같은 제스처로 동작합니다. 상태 표시줄의 포인터 고도는 씬의 고도 표면에서 커서 아래 지면을 읽거나, (동의 시) 다른 지도가 사용하는 것과 같은 원격 조화를 사용합니다.
- **GPS Tracking**(GPS 추적, 위치 마커, 정확도 원, 트랙, 위치 따라가기), **Add comment**(댓글 추가), Georeferencer의 **Link on map**(지도에서 연결), Pixel Time Series 및 NetCDF 샘플 마커는 엔진의 렌더 표면과 클릭 이벤트를 통해 동작합니다.
- 지도에 호스팅된 플러그인 컨트롤은 MapLibre의 `move`, `zoom`, `rotate`, `click`, `mousemove` 이벤트를 받으므로, View State나 미니맵 같은 판독 표시가 카메라를 따라갑니다. 스토리 첩터의 레이어 페이드는 첩터 전환 동안 실행됩니다.
- **Search places**(장소 검색)는 임시 마커와 함께 장소 및 좌표로 날아가고, H3 셀을 채워진 외곽선으로 표시합니다. 검색을 지우면 다른 곳에서 한 선택은 제거하지 않고 검색 하이라이트만 제거합니다.

- 점 히트맵은 2D와 3D 모두에서 공유 색상 램프, 반경, 강도, 선택적 가중치 필드를 사용합니다. Esri의 밀도 커널은 MapLibre와 다르므로 시각적 밀도가 다를 수 있습니다. SceneView는 반경을 112포인트로 제한합니다. 클러스터링은 2D에서 네이티브 개수 라벨과 구성된 반경 및 최대 줌을 사용합니다. 내장 및 사용자 지정 SVG 채우기 패턴도 2D에서 피쳐별 채우기 불투명도와 독립적인 외곽선으로 렌더링됩니다.
- **Controls**(컨트롤) 메뉴가 관리하는 내장 컨트롤은 SDK 자체 위젯으로 제공됩니다. 전체 화면, 나침반(회전 재설정), 줌(내비게이션), 위치 찾기(지오로케이션), 그리고 지구본/Mercator 토글과 지형([2D 및 3D](#) 참고). 축척 막대는 2D 지도 자체의 것으로, 활성 천체의 반지름을 사용해 뷰 중심 주변을 미터법, 야드파운드법 또는 해리 단위로 평면 지도와 썸 모두에서 측정합니다. 저작자 표시는 뷰 자체가 그리며(`attributionVisible`), Esri가 요구하는 사항이므로 숨길 수 없습니다.
- **Print Layout**(인쇄 레이아웃) 아틀라스는 MapLibre의 `fitBounds`가 계산한 맞춤으로 뷰의 카메라를 북쪽이 위로 가도록 조정해 각 페이지를 구성하고, 캡처 전에 뷰가 그리기를 마칠 때까지 기다립니다. 현재 피쳐 바깥의 마스크는 라이브 지도에 그리는 대신 캡처 위에 칠해지며, SDK가 주어진 줌을 반올림하기 때문에 고정 축척은 요청한 값의 0.5 %를 목표로 최대 세 번 보정됩니다.
- **Controls → Layer Control**(컨트롤 → 레이어 컨트롤)은 네이티브 ArcGIS 레이어 목록을 토글하며, 공유 플러그인과 마찬가지로 기본 지도에서 기본적으로 활성화됩니다. 그 가시성 토글은 프로젝트와 사이드바를 갱신하고, 사이드바 변경은 목록을 갱신합니다. 혼합 지오메트리 GeoJSON 레코드는 모든 부분에 대해 하나의 항목을 가집니다. 임시 검색 및 선택 하이라이트는 생략됩니다. 분할 창은 기본적으로 컨트롤을 숨긴 채로 두며, 독립적인 레이어 가시성을 유지합니다.
- 스토어 레이어만 추가하는 플러그인: **ArcGIS Hub**, **Socrata** 및 **CKAN** 검색, **OSM Downloader**, **Source Cooperative**, **Hugging Face**, **Earthdata GIS**, 그리고 **Weather** 오버레이(Clouds 및 Precipitation).
- 호스트가 컨트롤에 건네주는 지도를 통한 MapLibre 컨트롤 기반 플러그인([플러그인 컨트롤](#) 참고): **Web Services** 카탈로그(FEMA NFHL, NASA Earthdata, US EPA EnviroAtlas, USGS National Map), **Historical Imagery**(Esri Wayback), **DGGS** 그리드(H3, S2, A5, DGGRID, DGGAL, OLC, Geohash, Tilecode)와 **Gaticule**, 그리고 풋프린트가 있는 영상 카탈로그(STAC, OpenAerialMap, Vantor, Fields of the World, Satellite Embeddings, IGN LiDAR HD, GeoLens). 풋프린트 클릭과 호버, 선택 외곽선, 상자 그리기가 동작합니다. API 키가 필요한 GeoLens 래스터는 여전히 MapLibre가 필요합니다.
- MapLibre, Mapbox 또는 Cesium 지도를 필요로 하는 플러그인의 **Controls** 메뉴 항목과 플러그인 (Atmospheric Effects, Sun, Route Animation, Flight Simulator, Directions, Reverse Geocode, Geo Editor, Annotations, Swipe, Street View, Timelapse 등)은 ArcGIS가 기본 렌더러인 동안 그 이유와 함께 회색으로 비활성화됩니다.
- CDN에서 SDK를 로드할 수 없으면 지도의 배너에 **Retry**(다시 시도)가 표시되며, 이는 페이지를 다시 로드합니다. 브라우저는 실패한 모듈 가져오기를 페이지가 살아 있는 동안 유지하므로, 다시 로드해야만 다시 가져올 수 있습니다.

2D 및 3D

SDK는 평면 지도와 3D 썸을 서로 다른 두 뷰 클래스로 그리므로, ArcGIS 창은 프로젝트의 지도 환경 설정에서 하나를 고르고 선택이 바뀌면 뷰를 다시 만듭니다. 카메라(중심, 줌, 방위, 피치)는 그대로 이어집니다.

투영	지형	뷰
지구본	곰 또는 켄	SceneView, 전역(3D 지구본)
Mercator	켄	SceneView, 로컬(평면 3D 지도)
Mercator	곰	MapView (평면 2D 지도)

새 프로젝트는 지구본 투영을 사용하므로 ArcGIS 창은 지구본으로 열립니다. 나침반 아래의 지구본 버튼이 투영을 전환하고(MapLibre에서처럼 분할 창의 버튼은 해당 창만 전환), **Controls → Terrain**(컨트롤 → 지형)이 지형을 켜거나 끕니다. 3D 모듈(`views/SceneView` 와 고도 레이어, 1메가바이트에 가까움)은 첫 로드에는 포함되지 않습니다. 평면 ArcGIS 지도는 페이지가 유휴 상태가 되면 백그라운드에서 이를 가져오므로, 지구본으로 처음 전환할 때 네트워크를 기다리지 않습니다. 새 뷰가 로드되는 동안 이전 뷰는 화면에 남아 있다가 새 뷰의 베이스맵이 그려지면 교체되며, 데이터 레이어와 지형은 새 뷰에서 로드를 마칩니다.

썸네일은:

- 카메라가 기울어지며(오른쪽 드래그 또는 프로젝트에 저장된 피치), 프로젝트의 최대 피치로 제한됩니다. 상태 표시줄에 카메라의 고도가 표시됩니다.
- 조명은 카메라를 따르므로(SDK의 가상 조명) 보이는 지도 전체가 밝게 비춰집니다. SDK의 기본 시뮬레이션 태양은 지구본의 일부를 밤쪽에 남겨 둡니다.
- 지형은 Esri의 [World Elevation](#) 서비스 위에 지도를 드레이프합니다. API 키가 필요 없습니다. **Controls → Terrain exaggeration**(컨트롤 → 지형 과장)이 높이를 조정합니다.
- **Controls → Terrain exaggeration**은 다른 엔진과 같은 리더를 사용해 EPSG:3857 또는 EPSG:4326의 로컬 또는 원격 COG DEM도 받습니다. 로컬 파일은 기기에 남아 있습니다. 소스와 과장 값은 세션 동안 평면 지도, 로컬 썸, 지구본 간 전환에도 유지되지만 프로젝트에는 저장되지 않습니다. 누락된 DEM 픽셀과 COG 바깥 영역은 0미터를 사용합니다. **Use global terrain**(전역 지형 사용)은 Esri의 World Elevation을 복원합니다.
- 스타일이 돌출되는 폴리곤 레이어(Style 패널의 **3D extrusion**(3D 돌출))는 MapLibre의 fill-extrusion과 같은 높이와 색상으로 돌출된 3D 도형으로 그려집니다. 높이 속성 공급기 높이 배율(또는 고급 높이 표현식)이 윗면, 기준 높이가 아랫면이 되며, 색상은 레이어의 범주형, 단계 구분 또는 규칙 기반 심볼로지, 고급 색상 표현식, 또는 돌출 색상을 따릅니다. 2D MapView에서는 평면 채우기로 남습니다.
- 식별, 선택 하이라이트, 범위 그리기, 캡처는 2D에서와 같이 동작합니다. 프로젝트의 줌 범위는 지구본에서 썸의 고도 범위가 됩니다.
- **3D (Z values)**(3D (Z 값))는 벡터 좌표를 구성된 수직 배율과 오프셋을 적용해 절대 고도에 배치합니다. 선택 하이라이트는 같은 변환된 좌표를 사용합니다. 원본 데이터는 변경되지 않습니다.

데이터 추가

지도에 끌어다 놓은 파일, **Add Data → FlatGeobuf Layer / GeoParquet Layer / KML / KMZ / Delimited Text**(데이터 추가 → FlatGeobuf 레이어 / GeoParquet 레이어 / KML / KMZ / 구분 텍스트) 뒤의 호스트 가져오기 도구, 그리고 **XYZ, WMS, WMTS, ArcGIS Layer** 대화 상자는 모두 ArcGIS 지도에서 동작합니다. **Vector**

Layer(벡터 레이어) 패널은 크기가 제한된 벡터 가져오기에 공유 스토어 브리지를 사용하며, 대용량 스트리밍 GeoParquet은 여전히 MapLibre가 필요합니다. **Raster Layer**(래스터 레이어)는 로컬 GeoTIFF 또는 HTTP(S) URL용 호스트 대화 상자를 엽니다.

GeoTIFF/COG 파일과 URL은 기존 WebAssembly COG 타일러와 네이티브 ArcGIS 타일 레이어를 통해 2D와 3D에서 렌더링됩니다. 저장된 RGB 밴드, 연속 색상 램프, 스트레치, 감마, nodata, 불투명도가 반영되며, Style 패널에서 편집할 수 있습니다. 브라우저 파일은 세션 동안 유지되고, 데스크톱 로컬 경로는 같은 기기에서 다시 열 수 있습니다. GPU 전용 분류/사용자 지정 색상 램프는 여전히 MapLibre 래스터 컨트롤이 필요합니다.

deck.gl 레이어

Add Data → Deck.gl Layer / 3D Model(데이터 추가 → Deck.gl 레이어 / 3D 모델)은 기본 평면 지도와 로컬 3D 씬에서 동작합니다. 공유 오버레이는 저장된 시각화와 모델을 렌더링하고, deck.gl 레이어 간의 가시성, 불투명도, 순서를 따르며, 렌더러 전환 시 GPU 리소스를 해제합니다. 공유 오버레이는 하나의 네이티브 SDK 레이어이므로, deck.gl 레이어를 네이티브 ArcGIS 레이어와 개별적으로 끼워 넣을 수는 없습니다. 네이티브 피쳐 Z 렌더링은 ArcGIS가 계속 담당합니다.

어댑터는 `@deck.gl/arcgis 9.4.0`의 MIT 라이선스 컴포지터를 CDN에서 로드한 SDK 클래스와 함께 사용하여, 번들된 `@arcgis/core` 의존성을 피합니다. 로컬 3D 렌더링은 업스트림의 실험적 카메라 근사를 사용하므로, 극단적인 카메라 각도에서 어긋날 수 있고 지형 깊이 버퍼를 공유하지 않습니다. 전역 씬과 보조 창은 오버레이를 호스팅하지 않으며, 메뉴와 레이어 배지가 이 제한을 반영합니다. 레이어를 복원하려면 평면 지도나 로컬 씬으로 돌아가세요.

타일 아카이브

Add Data → PMTiles(데이터 추가 → PMTiles)는 공유 PMTiles 리더를 통해 원격 벡터(MVT) 및 래스터 아카이브를 직접 로드합니다. 네이티브 벡터 타일은 폴리곤, 선, 점 스타일을 유지하고, 래스터 아카이브는 네이티브 타일 영상을 사용하며 확대 시 마지막 네이티브 레벨을 리샘플링합니다. 공유 레지스트리에 있는 기존 인메모리 아카이브도 동작합니다. **Add Data → MBTiles**(데이터 추가 → MBTiles)는 벡터와 래스터 타일 모두에 데스크톱 파일 리더를 사용합니다. MBTiles는 여전히 데스크톱 앱이 필요합니다.

벡터 어댑터는 레이어마다 요청 인터셉터를 소유하며 레이어가 교체되거나 제거되면 이를 제거합니다. 타일은 취소를 포함해 필요할 때 읽히며, 합성 타일 주소는 절대 네트워크로 나가지 않습니다. MLT 인코딩과 아카이브 텍스트 레벨은 지원되지 않습니다. 어댑터는 아카이브 헤더에서 PMTiles 줌 한계를 읽으며, 이러한 한계를 생략한 이전 프로젝트에도 적용됩니다.

각 벡터 소스 레이어는 스타일과 가시성을 독립적으로 제어할 수 있도록 별도의 네이티브 `VectorTileLayer`를 사용합니다. PMTiles 리더는 공유되지만 SDK는 네이티브 레이어마다 타일을 별도로 디코딩합니다. 따라서 소스 레이어가 많은 아카이브는 공유 MapLibre 소스보다 더 많은 디코딩 작업과 메모리를 사용하므로, 현재 뷰에 필요한 레이어만 활성화하세요.

Zarr 및 NetCDF 그리드

Add Data → Zarr(데이터 추가 → Zarr)는 네이티브 타일 영상을 통해 숫자형 Zarr v2/v3 변수를 로드합니다. 리더는 규칙적인 간격의 1차원 공간 축, 오름차순 또는 내림차순 위도, 0-360° 경도, CF scale/offset 및 fill 값, 그리고 다른 차원에 대한 정수 선택자를 지원합니다. Time Slider는 같은 선택자 경로를 사용합니다. 투영된 그리드는 가져오기 API를 통한 CRS 또는 proj4 정의가 필요합니다. 곡선 좌표와 자동 다중 축척 선택은 지원되지 않으며, 피라미드 레벨은 변수 경로로 선택할 수 있습니다. 플러그인 `queryZarrLayer` API(점 값 및 영역 통계)는 네이티브 ArcGIS 그리드에서는 아직 지원되지 않습니다.

Add Data → NetCDF(데이터 추가 → NetCDF)는 기존 파일 대화 상자를 사용합니다. 이미지 슬라이스는 네이티브 이미지 오버레이로 렌더링되며, `kerchunk` 기반 그리드는 타일 Zarr 리더를 공유합니다. 참조 매니페스트는 프로젝트 복원을 위해 레이어 소스에 보존됩니다. 등록된 로컬 Zarr 스토어는 세션 로컬로 유지됩니다. 읽기는 제한된 윈도우를 반환하고 레이어당 최대 32 MiB의 압축 데이터를 유지하며, 타일링되지 않은 대형 배열의 거친 뷰는 여전히 많은 청크 요청이 필요할 수 있습니다.

플러그인 컨트롤

ArcGIS 지도에 마운트된 플러그인 컨트롤은 MapLibre 형태의 지도를 받습니다. 카메라, 투영, 포인터 이벤트, DOM은 뷰를 통해 전달됩니다. 그 스타일 호출(`addSource`, `addLayer`, `setPaintProperty`, `getStyle` 등)은 SDK가 직접 그리지 않는 스타일에 기록되며, 대신:

- 플러그인이 GeoLibre 스토어로 미러링하는 레이어(Web Services WMS 레이어, `registerExternalNativeLayer` 로 등록된 풋프린트)는 다른 레이어와 마찬가지로 엔진이 그 스토어 레코드에서 그립니다.
- 그 밖의 GeoJSON 레이어(그리드, 선택 외곽선, 그리기 미리 보기)는 프로젝트 레이어 위의 하나의 오버레이에 그래픽으로 그려집니다. 채우기, 선, 원, 텍스트 레이어가 그려지며, 필터, 줌 범위, 데이터 기반 또는 줌 표현식은 MapLibre가 평가하는 것처럼 피쳐별로 평가됩니다. 아이콘, 채우기 패턴, 돌출, 선을 따르는 텍스트는 그려지지 않으며, 라벨에는 충돌 처리가 없습니다.
- 레이어 범위 이벤트(`map.on("click", layerId, ...)`, `mouseenter`, `mouseleave`)와 점 `queryRenderedFeatures` 는 위 둘 중 해당 레이어를 그리는 쪽이 응답합니다. 상자 쿼리는 빈 결과를 반환합니다.
- `dragPan.disable()` 은 뷰의 이동을 멈추므로, 컨트롤 자체의 상자 또는 선 그리기가 드래그를 받습니다. 피쳐 상태, 이미지, 기타 상호 작용 핸들러는 동작하지 않습니다.

사용자 지정 레이어, 캔버스 소스, `Marker / Popup`, 미러링하지 않는 래스터 또는 벡터 타일 소스, 또는 MapLibre 전용 API를 통해 그리는 플러그인은 여전히 ArcGIS 지원을 선언하지 않습니다. 플러그인은 `packages/plugins/src/plugins/style-map.ts` 의 `getControlMap` 을 통해 이 지도를 읽습니다.

적응된 플러그인 패널

LiDAR, DuckDB, 일반 3D Tiles는 기본 평면 지도 또는 로컬 씬에서 deck.gl을 통해 렌더링됩니다. Google Photorealistic 및 I3S 타일은 여전히 다른 렌더러가 필요합니다. 전역 지구본과 보조 창은 이러한 오버레이를 호스팅하지 않습니다. LiDAR는 기존 COPC/EPT 스트리밍 및 스타일링 컨트롤을 유지하며, 지형 토클은 호스트 지형 설정에 위임합니다. 저장된 URL LiDAR 및 3D Tiles 레이어는 뷰가 다시 만들어질 때 복원됩니다. 브라우저 로컬 포인트 클라우드 파일과 캐시된 DuckDB 쿼리 결과는 기존 세션 수명을 유지하므로, 필요하면 소스/쿼리를 다시 여세요. 씬 오버레이에는 위에서 설명한 것과 같은 실험적 정렬 및 깊이 제한이 있습니다.

아직 지원되지 않는 항목

- 가우시안 스피클렛과 Cesium 전용 소스, STAC 검색 결과, 비디오 오버레이. ArcGIS가 기본 렌더러인 동안 **Add Data**(데이터 추가)는 이들을 회색으로 비활성화하고, 레이어 패널은 이러한 레이어에 **No ArcGIS** 배지를 붙입니다. 플러그인이 MapLibre에서만 그리는 레이어(예: Planetary Computer)도 지도의 배너에 이름이 표시됩니다.
- Esri 벡터 타일 서비스 자체의 아이콘 및 라벨 레이어(저장된 스타일에 스프라이트나 클리프가 없음), 모든 뷰에서.
- 내비게이션 한계가 MapLibre와 약간 다릅니다. 줌 범위나 제한된 경계를 벗어나 멈춘 뷰는 그 안으로 부드럽게 되돌아가고(평면 지도에서는 휠도 줌 한계에서 멈춤), 경계는 전체 뷰포트가 아닌 뷰의 중심을 붙잡으며, SDK는 항상 날짜 변경선을 넘어 감싸므로 세계 복사본을 끄면 앱이 적용하는 뷰만 클램프됩니다.
- Measure와 지오메트리 편집기는 MapLibre/Mapbox 스타일 API를 통해 그림니다. Controls 메뉴는 Measure를 회색으로 비활성화하고, 레이어의 지오메트리를 편집하려 하면 렌더러가 지원하지 않는다고 알립니다.
- 다이어그램과 라벨 우선순위 표현식(SDK에는 피처별 라벨 우선순위가 없음). SceneView에서의 클러스터링, 채우기 패턴, 혼합 모드, 선 장식. 평면 지도에서의 돌출. Style 패널은 레이어가 켜 있을 중 해당하는 항목을 알려 줍니다.
- 임의의 MapLibre 사용자 지정 레이어와 렌더링 API는 여전히 어댑터가 필요합니다. 기본 뷰는 내비게이션 메서드가 있는 DOM 컨트롤을 호스팅하며, Vector, LiDAR, DuckDB, 3D Tiles에는 명시적인 렌더링 브리지가 있습니다. Layer Control은 ArcGIS의 네이티브 레이어 목록에 위임합니다.
- 비디오 오버레이. SDK는 SceneView에서 클러스터링이나 그림 채우기 패턴을 지원하지 않으므로, 씬은 개별 점 심볼과 단색 폴리곤 채우기를 유지합니다. 히트맵 라벨도 씬에서는 지원되지 않습니다. 마커(내장 도형, 사용자 지정 SVG, KML 아이콘)는 MapLibre가 사용하는 것과 같은 스프라이트에서 구운 그림 심볼로 그려집니다.

테스트

`tests/arcgis-layers.test.ts` 는 레이어 컴파일러와 베이스맵 플래너를, `tests/arcgis-engine.test.ts` 는 가짜 SDK에 대해 엔진을 구동하며, `tests/arcgis-renderer.test.ts` 는 프로젝트 형식, 설정, 로더 경계를 다룹니다. 어느 것도 네트워크에 접근하지 않습니다. `e2e/arcgis-renderer.spec.ts` 는 Esri의 실제 CDN을 대상으로

하는 옵트인 브라우저 검사입니다. 전체 스위트를 위해서는 `ARCGIS_API_KEY` 를, 키 없는 좌표 및 H3 검색 커버리지를 위해서는 `ARCGIS_E2E=1` 을 설정하세요. `ARCGIS_E2E=1` 이 설정되면 `e2e/arcgis-offline.spec.ts` 도 프로젝트 Tauri CSP 하에서의 새 키 없는 부팅, 브라우저가 오프라인일 때 캐시된 SDK/인라인 데이터 시작, 그리고 첫 사용 시 CDN을 사용할 수 없을 때 표시되는 오류를 검증합니다. CSP 테스트는 정확한 정책 헤더로 Chromium에서 실행되며, 각 데스크톱 플랫폼의 네이티브 웹뷰 테스트를 대체하지는 않습니다.

라이선스 및 약관

ArcGIS Maps SDK for JavaScript는 오픈 소스 라이선스가 아닌 Esri 자체의 [이용 약관](#)에 따라 배포됩니다.

GeoLibre는 이를 재배포하지 않습니다. SDK는 런타임에 사용자의 브라우저가 Esri의 CDN에서 가져오며, Esri의 베이스맵과 위치 서비스 사용은 API 키가 속한 계정의 규정을 따릅니다.

<https://geolibre.app/self-hosting/>

자체 호스팅과 비공개 데이터

GeoLibre는 클라이언트 측 애플리케이션입니다. 지도, 레이어, 분석은 모두 브라우저나 데스크톱 셀 안에서 실행되며, 선택적 온라인 기능을 명시적으로 사용하지 않는 한 어떤 것도 GeoLibre 서비스에 업로드되지 않습니다([개인정보 처리방침](#) 참고). 따라서 데이터를 공개 호스트에 전혀 올릴 수 없는 조직에 잘 맞습니다. 자체 모니터링 데이터를 관리하는 원주민 및 지역 공동체, 보건 및 인도주의 프로그램, 보호종 기록, 또는 데이터가 조직이 통제하는 인프라에 머물러야 하는 모든 배포가 여기에 해당합니다.

GeoLibre가 하지 **않는** 것은 데이터를 호스팅하는 일입니다. GeoLibre는 사용자가 가리키는 곳 어디에서든 데이터를 읽습니다. 따라서 비공개 배포는 사실 두 가지 질문으로 귀결됩니다:

1. 데이터셋과 프로젝트는 어디에 있으며, 누가 읽을 수 있는가?
2. GeoLibre는 브라우저에서 어떻게 그곳에 도달하는가?

이 페이지는 두 질문 모두에 답합니다. 짧게 요약하면:

권장 구성

데이터는 계정, 데이터셋별 권한, OGC/STAC API를 갖춘 자체 호스팅 공간 카탈로그 (FastAPI + PostGIS)인 **GeoLens**로 호스팅하세요. **GeoLibre 웹 빌드**는 그 옆에 같은 출처(origin)에서 같은 인증 계층 뒤에 호스팅하세요. 그런 다음 GeoLibre에 내장된 **GeoLens 플러그인**으로 카탈로그를 검색하고 데이터셋을 지도에 추가하세요. 모든 바이트는 자신의 출처에서 인증된 브라우저로만 제공되고 다른 어디로도 가지 않으며, 자격 증명은 그 출처에만 전송되고, 구성할 CORS가 없습니다.

동일 출처가 중요한 이유

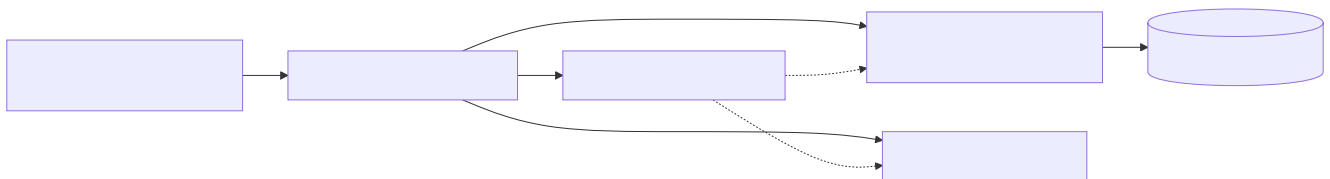
GeoLibre가 가져오는 모든 것은 브라우저가 가져오므로, 브라우저의 규칙이 적용됩니다:

- **쿠키.** GeoLibre의 요청은 브라우저 기본 자격 증명 모드(`same-origin`)를 사용합니다. SSO 계층이 설정한 세션 쿠키는 데이터 URL이 앱과 **같은 출처**에 있을 때 자동으로 전송되고, 다른 출처에 있을 때는 전송되지 **않습니다**. 따라서 쿠키로 보호된 데이터셋은 하나의 출처에서는 바로 동작하지만, 교차 출처에서는 서버가 무엇을 하든 실패합니다. 요청이 서버가 확인할 쿠키 없이 도착하기 때문입니다. 동일 출처는 필요조건이지 충분 조건은 아닙니다. 쿠키 자체의 `path`가 요청되는 URL을 포함해야 하므로, 앱, API, 프로젝트 파일을 형제 경로에 분산하는 레이아웃에서는 `Path=/'`로 설정하세요.
- **CORS.** 동일 출처 요청에는 CORS 헤더가 전혀 필요 없습니다. 교차 출처 요청은 데이터 호스트가 GeoLibre 출처를 명시적으로 허용해야 합니다.
- **타일 요청.** MapLibre는 래스터 및 벡터 타일 요청을 직접 보냅니다. 이 요청들도 같은 쿠키 및 CORS 규칙을 따르며, 그래서 세션 쿠키로 보호된 타일 엔드포인트는 동일 출처일 때만 동작합니다.

- `?url=` **프로젝트**. `?url=` **딥 링크**로 로드되는 `.geolibre.json` 도 같은 방식으로 가져오므로, SSO 계층 뒤의 프로젝트 파일은 앱이 같은 출처에서 제공될 때 로드되고, 그렇지 않으면 네트워크/CORS 오류로 실패합니다.
- **Content Security Policy.** Docker 이미지는 `connect-src` 에 `https:` 와 로컬 개발용 루프백을 허용하므로, 이를 통해 접근하는 자체 호스팅 데이터 서버는 **HTTPS**여야 합니다(일반 `http://` 는 `localhost` / `127.0.0.1` 에서만 동작). 데스크톱 앱은 추가로 모든 호스트에 대해 일반 `http:` 를 허용하므로, HTTPS 없이 로컬 네트워크의 자체 호스팅 Ollama, SamGeo 또는 기타 서비스에 도달할 수 있습니다(이슈 #2620).

GeoLibre와 데이터를 하나의 출처에 두면 이 다섯 가지 모두가 구성 문제에서 문제 아닌 것으로 바뀝니다.

아키텍처



하나의 호스트명, 하나의 로그인, 세 개의 경로입니다. `/gis` 의 GeoLibre는 `/api` 의 카탈로그, `/api/tiles/...` 의 타일, `/projects/...` 의 저장된 프로젝트를 모두 방문자 자신의 세션으로 읽습니다.

1. GeoLens로 데이터 호스팅하기

GeoLens는 FastAPI, PostGIS, React, MapLibre 위에 구축된 오픈 소스 자체 호스팅 지리공간 데이터 카탈로그이자 지도 빌더입니다. GeoLibre가 의도적으로 되려 하지 않는 부분을 담당하며, 둘은 잘 맞물립니다:

GeoLens가 제공하는 것	GeoLibre가 소비하는 방식
데이터셋 검색(퍼지 검색과 pgvector 기반 선택적 시맨틱 랭킹)	GeoLens 플러그인 패널의 카탈로그 목록
서명된 XYZ 벡터 타일(MVT)	벡터 타일 레이어, 토큰은 자동 갱신
OGC API Features(<code>/api/collections/{id}/items</code>)	전체 속성을 가진 GeoJSON 레이어
STAC 1.0과 서버 렌더링 래스터 타일	래스터 레이어
데이터셋별 권한, 계정, OAuth 2.0 / OIDC SSO, 사용자별 API 키	플러그인이 볼 수 있는 것을 결정하는 신원
Shapefile, GeoPackage, GeoJSON, GeoParquet, CSV, XLSX, GeoTIFF/COG 업로드	직접 변환할 것이 없음

데이터는 표준 PostGIS와 개방형 형식에 머무르므로, 이 선택에서 되돌릴 수 없는 것은 없습니다.

같은 VM에 Docker Compose로 설치하세요:

```
curl -fsSL https://getgeolens.com/install.sh | sh
```

그런 다음 노출하기 전에 `.env`에 최소한 다음을 설정하세요([GeoLens 구성 레퍼런스](#) 참고):

변수	여기서 중요한 이유
<code>PUBLIC_APP_URL</code> / <code>PUBLIC_API_URL</code>	브라우저에 노출되는 URL입니다. GeoLibre 플러그인에 입력하는 값이므로, Compose 서비스 이름이 아니라 공개 HTTPS URL이어야 합니다.
<code>CORS_ALLOWED_ORIGINS</code>	GeoLibre가 다른 출처에서 제공될 때만 필요합니다. 그 출처를 정확히 나열하세요.
<code>ENVIRONMENT=production</code>	API 문서 엔드포인트를 숨기고 OAuth 쿠키를 강화합니다.
<code>JWT_SECRET_KEY</code>	세션 서명 비밀 키입니다.

데이터셋을 업로드하고 비공개로 유지하세요. GeoLens는 데이터셋별 권한을 가진 역할 기반 접근 제어를 사용합니다. 그러면 GeoLibre 플러그인은 두 가지 방식으로 이를 읽을 수 있으며, 어느 쪽이 적용되는지는 배포가 API 요청을 어떻게 인가하는지에 달려 있습니다. 동일 출처 배포에서는 플러그인의 호출이 방문자의 GeoLens 세션을 자동으로 전달합니다. 그렇지 않으면 사용자별 API 키를 만들어 플러그인 패널에 붙여넣으세요.

2. GeoLibre 웹 빌드를 그 옆에 호스팅하기

게시된 이미지는 nginx로 프로덕션 웹 빌드를 제공합니다:

```
docker run -d --name geolibre -p 8081:80 \
  -e GEOLIBRE_SHARE_URL=off \
  ghcr.io/opengeos/geolibre:latest
```

`/gis`와 같은 경로 아래에 배포하려면, 빌드 시 `base` 경로를 넣고 프록시에서 접두사를 제거하세요:

```
docker build --build-arg GEOLIBRE_APP_BASE=/gis/ -t geolibre-private .
```

권장 배포 계약은 런타임 `deployment.json` 정책입니다. Docker에서는 `GEOLIBRE_DEPLOYMENT_FILE`이 읽기 전용 소스 파일(예: `/etc/geolibre/deployment.json`)을 선택합니다. 시작 시 이를 엄격하게 검증한 뒤, 비어 있지 않은 환경 변수 재정의의 필드별로 적용하고 생성된 정책을 `/usr/share/nginx/html/deployment.json`에 기록하며, 이는 `/deployment.json`에서 제공됩니다. 소스를 그 생성된 출력 위에 마운트하지 마세요. 소스나 재정의가 없으면 시작 시 `{"version":1}`이 생성됩니다. 클라이언트는 정책, 런타임 환경, 빌드 환경 순으로 해석합니다. Docker nginx는 생성된 최종 정책만 강제합니다. 빌드 전용 권한 부여는 그 가드를 구성하지 않습니다.

변수	권장 값	효과
<code>GEOLIBRE_DEPLOYMENT_FILE</code>	설정 안 함, 또는 마운트된 소스 정책	부팅 시 검증됩니다. 생성된 공개 파일은 <code>/usr/share/nginx/html/deployment.json</code> 이며 <code>/deployment.json</code> 에서 제공됩니다.
<code>GEOLIBRE_CAPABILITIES</code>	설정 안 함, 또는 부여할 기능	런타임 필드 재정의입니다. <code>none</code> 은 아무것도 부여하지 않습니다. 알 수 없는 이름은 부팅을 중단시킵니다. 생성된 최종 정책이 nginx 사이드카 가드와 사이드카 시작을 제어합니다.

변수	권장 값	효과
GEOLIBRE_AI_URL / GEOLIBRE_AI_PROXY_URL / GEOLIBRE_AI_PROXY_TOKEN	설정 안 함, 또는 승인된 프록시 3종	GEOLIBRE_AI_URL=/ai 는 정책을 재정의하여 AI를 활성화합니다. 프록시 URL은 자격 증명/경로/쿼리/프래그먼트가 없는 HTTPS 출처여야 하며, 토큰은 [A-Za-z0-9._-] 만 허용됩니다. 구성이 누락되거나 잘못되면 부팅이 중단됩니다. 업스트림 URL과 토큰은 비공개로 유지하세요.
VITE_GEO_LIBRE_CAPABILITIES (레거시 빌드 인수; 여전히 지원됨)	설정 안 함, 또는 부여할 기능	클라이언트 폴백 전용이며 nginx 가드를 구성하지 않습니다.
GEOLIBRE_AI_MODEL	선택 사항, GEOLIBRE_AI_URL 과 함께	GEOLIBRE_AI_URL 이 설정된 경우에만 정책 모델을 재정의합니다.
GEOLIBRE_COLLAB_URL	설정 안 함, 또는 직접 운영하는 릴레이	설정하지 않으면 실시간 협업이 꺼진 상태로 유지됩니다. 호스팅 릴레이 없이 멀티플레이어 편집을 원하면 직접 운영하는 wss:// 릴레이로 설정하세요.
GEOLIBRE_GEOLENS_URL	이미지 기본값은 설정 안 함, same-origin, 자신의 GeoLens URL, 또는 off	GeoLens 플러그인을 미리 채우고 자동으로 연결합니다. 기본 이미지는 브라우저 출처를 기본값으로 하며, 이는 같은 리버스 프록시 뒤에 함께 배치된 GeoLens API에 적합합니다. URL은 쿼리나 프래그먼트가 없는 서버 루트여야 합니다. off 는 사용자가 서버를 선택할 때까지 패널을 유휴 상태로 둡니다. 마지막으로 성공한 서버는 브라우저별로 기억됩니다.
GEOLIBRE_AUTH_USER / GEOLIBRE_AUTH_PASSWORD	빠른 단일 자격 증명이 필요하면 설정	앱과 /sidecar API에 대한 nginx Basic Auth입니다. 계정이 아니라 하나의 공유 자격 증명입니다. 다중 사용자나 SSO에는 실제 인증 프록시를 사용하세요.
GEOLIBRE_CLERK_PUBLISHABLE_KEY	설정 안 함, 또는 Clerk publishable key	설정하지 않으면 앱은 공개 상태이며 Clerk를 로드하지 않습니다. 키가 있으면 웹 인터페이스가 렌더링되기 전에 개별 사용자가 로그인해야 합니다. 서버 API는 별도로 보호하세요.
GEOLIBRE_CLERK_WHITELIST	설정 안 함, 또는 Clerk 키와 함께 1	로그인 화면에 Clerk의 대기자 명단 양식을 추가하여, 방문자가 접근을 요청하고 관리자가 Clerk Dashboard에서 각각 승인할 수 있게 합니다. 요청에 아무도 응답하지 않을 초대 전용 ("restricted") 인스턴스에서는 설정하지 마세요.

변수	권장 값	효과
GEOLIBRE_AUTH0_DOMAIN / GEOLIBRE_AUTH0_CLIENT_ID	설정 안 함, 또는 Clerk 대신 Auth0를 사용하는 경우 둘 다	Auth0 Universal Login이 뒷받침하는 동일한 로그인 게이트입니다. 둘 다 함께 필요하며, Clerk도 구성되어 있으면 컨테이너가 시작을 거부합니다 — 제공자 하나를 선택하세요.
GEOLIBRE_CONVERSION_ROOTS	/data (이미지 기본값)	모든 사이드카 읽기와 쓰기를 마운트된 디렉터리로 제한합니다.
GEOLIBRE_POSTGIS_HOSTS	필요하지 않으면 설정 안 함	사이드카의 PostGIS 엔드포인트는 이 변수가 허용된 데이터베이스를 지정할 때까지 모든 대상을 거부하므로, 호출자가 컨테이너만 접근할 수 있는 호스트를 겨냥할 수 없습니다.
GEOLIBRE_MSSQL_HOSTS	필요하지 않으면 설정 안 함	나열된 호스트/IP와 선택적 포트에 대해서만 SQL Server 엔드포인트를 활성화합니다. mssql extra와 별도로 설치된 ODBC Driver 18이 포함된 파생 이미지가 필요합니다. 명명된 인스턴스에는 호스트만 지정한 항목이 필요합니다.
GEOLIBRE_MSSQL_ALLOW_MANAGED_IDENTITY	필요하지 않으면 설정 안 함	관리 ID 인증은 비활성화되어 있습니다. 사이드카를 공유하는 모든 호출자에 대해 컨테이너 자체의 Azure ID로 로그인하기 때문입니다. 사이드카가 단일 테넌트이고 그 ID가 데이터베이스 접근을 뒷받침해야 할 때만 1로 설정하세요. Windows 및 대화형 Entra 로그인도 활성화하는 GEOLIBRE_MSSQL_DESKTOP_AUTH=1은 이 옵션을 함축하므로, 공유 사이드카에서는 절대 설정하지 마세요.
GEOLIBRE_DISABLE_SIDECAR	필요 없으면 1	기능 부여와 관계없이 uvicorn을 건너뛵니다. 모든 /sidecar/ 요청은 application/json detail 응답과 함께 HTTP 403을 반환합니다.
GEOLIBRE_EMBED_ORIGINS	설정 안 함, 또는 정확한 호스트 페이지 출처	기본적으로 꺼져 있으므로, 프레임에 넣은 배포를 프레임을 만든 누구나 조종할 수는 없습니다.
GEOLIBRE_NO_EXTERNAL_CDN (빌드 인수)	제한된 배포에는 1	빌드 출력에서 GeoLibre 자체의 외부 CDN(unpkg.com , cdn.jsdelivr.net) 참조를 제거합니다. 에셋을 CDN에서만 구할 수 있는 기능은 비활성화되거나 저하됩니다: 스토리맵 HTML 내보내기, 내장 객체 탐지 모델, ONNX WASM, 3D Tiles Draco/KTX2 디코더, gdal3.js 내보내기. Pyodide는 완전히 비활성화되지 않습니다 — 이 플래그는 기본 인덱스 URL만 제거하므로, VITE_PYODIDE_INDEX_URL 을 승인된 미러로 설정하면 계속 동작합니다. 또한 GEOLIBRE_PGLITE_CDN=0 , GEOLIBRE_CEREUS_CDN=0 , GEOLIBRE_GDAL_CDN=0 , GEOLIBRE_DUCKDB_WASM_CDN=0 을 강제하므로 — PGLite/PostGIS, CereusDB, DuckDB-WASM 은 가져오는 대신 /assets/ 아래의 빌드에 번들링되어(빌드 크기는 커짐) 계속 사용 가능 합니다. 일부 서드파티 패키지(DuckDB-WASM, loaders.gl, maplibre-gl-3d-tiles)는 이 플래그가 제거할 수 없는 자체 내부 CDN URL을 담고 있다는 점에 유의하세요. 자세한 내용은 architecture.md 를 참고하세요. 신뢰할 수 없는 외부 CDN을 참조할 수 없는 배포(예: 엄격한 CSP 요구 사항이 있는 기업 환경)를 위한 것입니다.

변수	권장 값	효과
GEOLIBRE_APP_NAME	선택 사항, 예: Acme Maps	툴바 시작 부분과 브라우저 탭 제목의 "GeoLibre"를 대체합니다. 연속 공백은 하나로 축약되고 이름은 60자로 제한됩니다. VITE_GEOLIBRE_APP_NAME 은 동등한 빌드 인수입니다.
VITE_WELCOME_DISABLED=1 (빌드 인수)	선택 사항	모든 방문자에 대해 첫 실행 마법사를 건너뛰니다.

전체 목록은 [시작하기](#)를 참고하세요.

이미지를 공개적으로 노출하기 전에

`docker/nginx.conf` 는 로컬 개발에서 다른 포트의 개발 서버로부터 데이터를 로드할 수 있도록 `connect-src` 에 `http://localhost:*` 와 `http://127.0.0.1:*` 를 허용합니다. 공개 호스트에서는 이로 인해 제공되는 JavaScript가 각 방문자의 루프백 인터페이스를 탐색할 수 있습니다. 공개 배포에서는 CSP에서 이 허용 항목을 제거하세요.

컨테이너 정책 적용

컨테이너는 환경 변수 재정의 이후의 최종 배포 정책을 사용하여 nginx에서 다음 라우트 계열을 보호합니다. 각 행은 **any-of** 검사입니다. 나열된 기능 중 최소 하나가 부여되어야 합니다.

사이드카 라우트 접두사	필요한 기능(다음 중 하나)
<code>/sidecar/whitebox</code>	<code>processing:run</code>
<code>/sidecar/raster</code>	<code>processing:run</code>
<code>/sidecar/vector</code>	<code>processing:run</code>
<code>/sidecar/pointcloud</code>	<code>processing:run</code>
<code>/sidecar/ml</code>	<code>processing:run</code>
<code>/sidecar/sql</code>	<code>processing:run</code>
<code>/sidecar/postgis</code>	<code>data:add</code>
<code>/sidecar/mssql</code>	<code>data:add</code>
<code>/sidecar/conversion</code>	<code>processing:run</code> 또는 <code>data:add</code>

거부된 라우트는 사이드카에 도달하는 대신 `detail` 을 담은 `application/json` 본문과 함께 HTTP 403을 반환합니다. `capabilities` 를 생략하면 모든 기능이 부여됩니다. 최종 권한 부여에 `processing:run` 과 `data:add` 가 모두 없으면 `uvicorn`은 시작되지 않고 모든 `/sidecar/` 요청은 그 JSON 403 응답을 반환합니다.

`GEOLIBRE_DISABLE_SIDE CAR=1` 은 권한 부여와 관계없이 같은 효과를 냅니다.

uvicorn이 실행 중일 때 `/health`, `/algorithms`, `/shutdown`, `/run`, `/docs`, `/redoc`, `/openapi.json`은 기능 부여로 보호되지 않습니다. 이는 라우트 계열 정책이지 메서드별 인가가 아닙니다. 가드는 중복 슬래시 병합과 퍼센트 디코딩된 접두사를 포함하여 nginx의 정규화된 URI를 사용합니다.

최종 정책은 `/ai`도 제어합니다. `ai.enabled: false` 또는 `ai.enabled`의 부재는 프록시를 비활성화합니다. 활성화하려면 세 값이 모두 필요합니다: `GEOLIBRE_AI_URL=/ai`, `GEOLIBRE_AI_PROXY_URL=https://<approved-origin>`, `GEOLIBRE_AI_PROXY_TOKEN=<instance-token>`. URL은 자격 증명, 경로, 쿼리, 프래그먼트가 없는 HTTPS 출처여야 하며, 토큰은 `[A-Za-z0-9._-]`만 포함할 수 있습니다. 구성이 누락되거나 잘못되면 부팅이 중단됩니다. `GEOLIBRE_AI_URL=/ai`는 마운트된 `false` 값을 재정의합니다. 업스트림 URL/토큰만으로는 AI가 활성화되지 않습니다. `GEOLIBRE_AI_MODEL`은 `GEOLIBRE_AI_URL`이 설정된 경우에만 정책을 재정의합니다. 프록시 자격 증명, 사이드카 토큰, 신뢰할 수 있는 프록시, Basic Auth, CSP는 비공개로 유지하세요. 생성된 파일에는 공개 정책만 들어가야 합니다.

컨테이너 가드는 일반적인 인가가 아닙니다

이 제한은 번들된 컨테이너의 nginx 라우트에만 적용됩니다. 브라우저 WASM 엔진, 데스크톱 프로세싱, 별도로 노출된 서비스에는 기능을 강제하지 않습니다. 사이드카와 AI 업스트림은 nginx 뒤에 비공개로 두고, 파일시스템 제한을 유지하며, 사용자 인증에는 Basic Auth나 실제 인증 프록시를 사용하세요. 클라이언트 측 숨기기만으로는 인가 경계가 되지 않습니다.

둘 다 하나의 인증 계층 뒤에 두기

Basic Auth는 단일 공유 자격 증명에는 충분합니다. 계정이나 SSO가 필요하다면 수정하지 않은 이미지 앞에 `oauth2-proxy`, `Authelia`, 또는 Cloudflare Access를 두고, 두 애플리케이션을 하나의 호스트명 아래로 라우팅하세요. 최소한의 Caddy 예시입니다:

```
maps.example.org {
  # Everything is behind the same forward-auth / SSO layer.
  forward_auth authelia:9091 {
    uri /api/verify?rd=https://auth.example.org
    copy_headers Remote-User Remote-Groups Remote-Email
  }

  handle_path /gis/* {
    reverse_proxy geolibre:80
  }

  handle_path /projects/* {
    root * /srv/projects
    file_server
  }

  # Catch-all: Geolens's own entry point, which serves its UI and routes
  # /api to its API service internally, so one upstream covers both. Check
  # your Geolens deployment: if it exposes the API on a separate address,
  # give it its own route above this block, and match on `handle` rather than
  # `handle_path` so the `/api` prefix survives (the API expects it).
  handle {
    reverse_proxy geolens:8080
  }
}
```

이렇게 구성하면:

```
https://maps.example.org/gis/?url=https://maps.example.org/projects/watershed.geolibre.json
```

이 URL은 인증된 사용자에게는 비공개 프로젝트를 열고, 그 외의 누구에게나 로그인 리디렉션을 반환합니다. CORS 헤더도, URL 안의 토큰도 없으며, SSO 계층이 허용하지 않은 누구에게도 아무것도 제공되지 않습니다.

인증 계층은 하나가 아니라 둘입니다

forward-auth 계층은 누가 출처에 도달하는지를 결정합니다. 그것만으로는 GeoLens에 방문자가 누구인지 알려주지 않습니다. GeoLens는 자체 계정과 데이터셋별 권한을 적용하고, GeoLibre 플러그인은 브라우저가 첨부하는 것(동일 출처 쿠키)과 키를 제공한 경우 `X-API-Key` 헤더만 보냅니다. 따라서 방문자에게는 GeoLens 신원도 필요합니다. GeoLens가 사용 중인 제공자를 사용하도록 구성하거나(OAuth 2.0 / OIDC를 지원하므로 같은 ID 제공자를 가리키게 하면, 로그인한 방문자는 이 출처에서 GeoLens 세션 쿠키를 받습니다), 각 사용자가 사용자별 API 키를 플러그인 패널에 붙여넣게 하세요. 둘 다 아니면 앱은 로드되고 공개 데이터셋은 나타나지만 비공개 데이터셋은 보이지 않는, 혼란스러운 실패가 발생하므로 먼저 배제할 가치가 있습니다. 프록시 뒤의 `/projects` 및 기타 정적 파일은 영향을 받지 않습니다. 이들은 forward-auth 계층만으로 보호됩니다.

`url=` 은 절대 경로여야 합니다

프로젝트 딥 링크는 절대 `http(s)` URL로 검증되므로, `?url=/projects/watershed.geolibre.json` 은 무시됩니다. 위와 같이 전체 URL을 쓰세요. 여전히 동일 출처이므로 세션 쿠키는 계속 전송됩니다.

로그인한 사용자를 프로젝트 서버에 전달하기

프로젝트 서버(`docker-compose.yml` 의 `geolibre-server`)도 운영한다면, 같은 프록시가 사용자를 거기에도 로그인시킬 수 있으므로 Share와 Project Gallery에 별도의 GeoLibre 비밀번호가 필요 없습니다. 서버는 `GEOLIBRE_PROXY_AUTH=true` 일 때 로그인(동의) 페이지에서 프록시의 `Remote-User` 와 `Remote-Email` 헤더를 신뢰하며, `GEOLIBRE_TRUSTED_PROXIES` 에 나열된 주소에서 오는 연결에 대해서만 그렇게 합니다:

1. `GEOLIBRE_PROXY_AUTH=true` 를 설정하고, `GEOLIBRE_TRUSTED_PROXIES` 를 프로젝트 서버가 프록시의 연결을 받는 주소로 설정하세요. 프록시 컨테이너의 IP나 프록시와 서버만 포함하는 Docker 네트워크 같은 IP 또는 CIDR의 심표 구분 목록입니다. 다른 모든 피어의 신원 헤더는 무시됩니다.
`GEOLIBRE_TRUSTED_PROXIES` 만 설정하면 프록시의 `X-Forwarded-For` 만 신뢰합니다.
2. 프록시만 접근할 수 있도록 프로젝트 서버를 바인딩하세요. Compose 파일은 이를 `127.0.0.1` 에만 게시합니다. 프록시가 같은 Compose 네트워크에서 실행되면 그 포트를 완전히 제거하세요.
3. 프록시는 **클라이언트가 보낸 신원 헤더를 반드시 제거해야 합니다**. 그렇지 않으면 프록시가 헤더를 덮어쓰지 않는 라우트에 도달하는 누구나 임의의 사용자라고 주장할 수 있습니다.

프로젝트 서버는 GeoLens처럼 `/api/*` 에서 응답하므로, 자체 호스트명을 주고 웹 이미지의 `GEOLIBRE_SHARE_URL` (그리고 서버의 `GEOLIBRE_PUBLIC_URL`)을 그곳으로 가리키게 하세요. `forward auth`를 거치는 것은 동의 페이지뿐입니다. 앱은 토큰 엔드포인트와 API를 bearer 토큰으로 교차 출처 호출하므로, 거기서 로그인 리디렉션이 발생하면 깨집니다.

```
projects.example.org {
  # Drop client-sent identity headers before anything else runs. Caddy orders
  # request_header ahead of handle, so this precedes the forward_auth below.
  request_header -Remote-User
  request_header -Remote-Email

  # The consent page: forward auth sets the headers for the signed-in user.
  handle /oauth/authorize {
    forward_auth authelia:9091 {
      uri /api/verify?rd=https://auth.example.org
      copy_headers Remote-User Remote-Email
    }
    reverse_proxy geolibre-server:8000
  }

  # `handle`, not `handle_path`: the server expects the /oauth and /api prefixes.
  handle /oauth/* {
    reverse_proxy geolibre-server:8000
  }

  handle /api/* {
    reverse_proxy geolibre-server:8000
  }
}
```

그러면 앱에서 로그인하면 프록시 사용자를 명시하는 동의 페이지가 열리고 (Signed in through your organization's proxy as ...) 장치 라벨만 묻습니다. 첫 로그인에서 계정이 생성됩니다. 프록시가 다른 헤더 이름을 사용한다면(oauth2-proxy는 `X-Forwarded-User` 와 `X-Forwarded-Email` 을 보냅니다) `GEOLIBRE_PROXY_USER_HEADER` 와 `GEOLIBRE_PROXY_EMAIL_HEADER` 를 설정하고 대신 그 이름들을 제거하세요. 전체 계약은 [신뢰 헤더 프록시 로그인](#)을 참고하세요.

웹 이미지에 내장된 nginx Basic Auth(`GEOLIBRE_AUTH_USER` / `GEOLIBRE_AUTH_PASSWORD`)는 대체재가 아닙니다. 단일 공유 자격 증명이므로 프로젝트 서버에 방문자가 누구인지 알려줄 수 없습니다.

3. GeoLibre를 GeoLens에 연결하기

GeoLens 플러그인은 내장되어 있습니다. 설치할 것도, 마켓플레이스 단계도, 배포할 번들 드롭인도 없습니다. **Plugins**(플러그인) 메뉴를 열고 **GeoLens**를 선택하세요.

패널에서:

1. 목록에서 서버를 선택하거나 직접 base URL을 입력하세요. 예: `https://maps.example.org` . `/api` 경로가 아니라 서버 루트를 입력하세요. 플러그인이 `/api/...` 를 직접 덧붙입니다. 후행 슬래시는 자동으로 제거되고, 호스트명만 입력하면 `https://` 로 해석됩니다.

2. 공개 데이터셋에는 **API key**(API 키)를 비워 두고, 비공개 데이터셋을 보고 읽으려면 GeoLens API 키를 붙여 넣으세요. 동일 출처 배포에서는 요청이 방문자의 GeoLens 세션 쿠키도 전달하므로, 배포가 세션으로 API 요청을 인가한다면 키 없이도 비공개 데이터셋이 보입니다.
3. 카탈로그를 검색하고 데이터셋을 추가하세요. 벡터 데이터셋은 **vector tiles**(벡터 타일)(빠르고 큰 테이블로 확장 가능)로 추가하거나, OGC API Features를 통해 **GeoJSON**(전체 속성, 편집 가능, 패널에서 설정할 수 있는 피쳐 제한 적용)으로 추가할 수 있습니다. 래스터 데이터셋은 서버 렌더링 래스터 타일로 추가됩니다.
4. 각 결과에는 GeoLens의 해당 페이지로 돌아가는 **Metadata**(메타데이터) 링크가 있습니다.

데이터셋이 지도에 올라오면 일반적인 GeoLibre 레이어입니다. 스타일을 지정하고, 분류하고, **속성 테이블**을 열고, **프로세싱 도구**와 **공간 SQL**을 실행하고, **Story Map**이나 **임베디드 지도**에 게시할 수 있습니다.

GeoLens로 편집 내용 되돌려 쓰기

GeoJSON으로 추가된 데이터셋은 GeoEditor로 다시 그리거나 속성 테이블에서 다시 입력할 수 있으며, 플러그인의 **Edits**(편집) 섹션이 변경 사항을 피쳐 단위로 GeoLens 데이터셋에 다시 기록합니다. 이를 위해서는 자격 증명 **그리고** 데이터셋 편집이 활성화된 서버(`enable_dataset_editing`, 연결 시 `/api/settings/feature-flags/` 에서 읽음)가 필요합니다. 꺼져 있으면 플러그인은 중간에 실패할 쓰기를 제공하는 대신 저장을 사용할 수 없다고 표시합니다.

자격 증명 처리 방식

데이터 주권에서 중요한 부분입니다:

- API 키는 **세션 동안 메모리에만** 보관됩니다. `localStorage` 에 기록되지 않으며, 프로젝트 파일에도 기록되지 않습니다.
- 저장된 `.geolibre.json` 은 각 레이어에 대해 GeoLens **서버 URL**과 **데이터셋 id**만 기록합니다. 그 프로젝트를 다른 사람에게 보내도 데이터나 키는 전달되지 않습니다.
- 프로젝트를 다시 열 때 **공개** 데이터셋의 레이어는 서명된 타일 토큰을 자동으로 다시 발급받아 렌더링됩니다. **비공개** 데이터셋의 레이어는 권한이 있는 키로 플러그인 패널에서 다시 연결할 때까지 비어 있습니다.
- 서명된 벡터 타일 토큰은 데이터셋별로 발급되는 수명이 짧은 HMAC 토큰이며, 플러그인이 만료 전에 갱신합니다.
- 비공개 래스터의 API 키는 **정확히** 그 래스터의 타일 URL 접두사에만 첨부되고 다른 어디에도 첨부되지 않으므로, 베이스맵, 다른 플러그인의 타일, 같은 출처의 두 번째 GeoLens 서버는 결코 이를 보지 못합니다.
- 플러그인을 비활성화하면 등록된 모든 키가 삭제되어 비공개 레이어의 렌더링이 멈춥니다. 자격 증명은 플러그인보다 오래 살아남지 않습니다.

두 가지 배포 패턴 비교

토론 #1807에서 제기된 두 가지 선택지이며, 둘은 상당히 다르게 동작합니다.

패턴 A: 커뮤니티 서버의 GeoLibre(권장)

위에서 설명한 대로, 데이터와 같은 VM, 같은 출처에서 같은 SSO 계층 뒤에 웹 빌드를 제공합니다.

속성	결과
세션 쿠키가 데이터에 도달	예
필요한 CORS 구성	없음
비공개 ?url= 프로젝트	동작
자체 호스트의 비공개 타일, COG, GeoParquet, FlatGeobuf	동작
서드파티에 도달하는 데이터 또는 토큰	없음
비용	컨테이너 하나를 더 운영하고 업데이트

데이터가 자체 인증 계층이 허용한 사용자 외에는 누구에게도, 그리고 중간의 어떤 서드파티 서비스에도 도달하면 안 된다는 요구 사항이 있을 때 선택할 패턴입니다.

패턴 B: 호스팅된 GeoLibre, 비공개 데이터는 다른 곳에

사용자가 `https://web.geolibre.app/?url=https://community.example.org/project.geolibre.json` 을 엽니다.

이제 프로젝트 가져오기와 모든 레이어 가져오기가 교차 출처이므로:

- **세션 쿠키가 전송되지 않습니다.** SSO 쿠키로 보호된 URL은 로그인 페이지(또는 리디렉션)를 반환하고, GeoLibre는 가져오기 또는 파싱 오류를 보고합니다. 이를 바꾸는 설정은 없습니다.
- 데이터 호스트에서 **CORS가** `https://web.geolibre.app` 를 허용해야 하며, 의존하는 사용자 지정 요청 헤더도 포함해야 합니다.
- **따라서 자격 증명은 URL이나 헤더에 담겨 이동해야 합니다.** 실제로는 서명된 만료 URL(S3/MinIO presigned URL, GeoLens 서명 타일 URL, 또는 자체 HMAC 방식)을 의미합니다. MapLibre 자체의 타일 요청은 GeoLibre에 알려지지 않은 사용자 지정 헤더를 실을 수 없기 때문입니다. GeoLens 플러그인은 헤더 경로에서 예외입니다. API 호출에 `X-API-Key` 를 보내고, MapLibre의 `transformRequest` 혹은 통해 그 데이터셋의 래스터 타일에 이를 첨부합니다.
- **URL 안의 토큰은** 브라우저 기록, 프록시 로그, referrer 헤더, 사용자가 복사해 붙여넣는 모든 것으로 **유출됩니다.** 만료를 짧게 유지하고 각 토큰의 범위를 데이터셋 하나로 제한하세요.
- 데이터 호스트는 여전히 사용자 브라우저의 요청을 보고, 서드파티 앱 출처는 여전히 그것을 읽는 코드를 제공합니다. 거버넌스 규칙이 데이터를 커뮤니티 서버 밖에서 제공되는 소프트웨어가 처리해서는 안 된다고 한다면, 바이트가 서버에서 사용자 브라우저로 직접 가더라도 이 패턴은 그 규칙을 충족하지 않습니다.

패턴 B는 **반공개** 데이터(기능 URL이 있는 게시된 지도)에는 합리적이지만 **민감한** 데이터에는 부적합합니다. 확신이 없으면 패턴 A를 사용하세요. 웹 빌드는 정적 파일 디렉터리이므로 호스팅 비용이 매우 적습니다.

GeoLens 없이 비공개 데이터 다루기

GeoLens는 카탈로그, 계정, 권한을 추가하기 때문에 권장 경로입니다. 이미 데이터 제공 인프라가 있다면, 다음은 모두 비공개, 인증된, 동일 출처 호스트에 대해 동작합니다:

- **클라우드 네이티브 파일.** COG, PMTiles, FlatGeobuf, GeoParquet, Zarr는 HTTP range 요청으로 읽으므로, 인증 계층 뒤의 어떤 정적 파일 서버든 동작합니다. 같은 출처에서 제공하면 모든 range 요청과 함께 세션 쿠키가 전송됩니다.
- **OGC 서비스.** 자체 서버의 WMS, WMTS, OGC API Features 엔드포인트는 Add Data(데이터 추가) 메뉴에서 추가합니다.
- **PostGIS.** 데스크톱 앱에서 **Add Data → PostgreSQL/PostGIS**(데이터 추가 → PostgreSQL/PostGIS)는 로컬 **Martin** 타일 서버를 통해 데이터베이스에서 바로 타일을 제공하며, 이는 QGIS를 데이터베이스에 연결하는 것에 가장 가까운 방식입니다. 데스크톱 전용이며, 샌드박스 처리된 Mac App Store 빌드에서는 사용할 수 없습니다([다운로드](#) 참고).
- **MBTiles와 로컬 파일.** 데스크톱 앱은 서버 없이 로컬 MBTiles, 래스터, 벡터 파일을 읽으며, 이것이 가장 단순한 비공개 워크플로입니다. 커뮤니티 서버에서 다운로드하고 오프라인에서 분석하세요.
- **사이드카.** 번들된 Python 사이드카를 활성화한다면 `GEOLIBRE_CONVERSION_ROOTS` 를 정확히 마운트한 디렉터리로 유지하고, 해당 엔드포인트가 필요하지 않은 한 `GEOLIBRE_POSTGIS_HOSTS` 와 `GEOLIBRE_MSSQL_HOSTS` 는 설정하지 마세요. SQL Server에는 pyodbc와 ODBC Driver 18이 설치된 파생 이미지도 필요합니다.

외부 요청 줄이기

비공개 배포는 보통 브라우저가 공개 인터넷에서 가져오는 것도 최소화하려 합니다:

기능	기본값	내부에 유지하는 방법
베이 스맵	OpenFreeMap / CARTO 타일	Basemaps 플러그인의 custom style URL (사용자 지정 스타일 URL)을 사용하여 자체 스타일과 PMTiles 베이스맵을 자신의 서버에서 제공하거나, 빈 배경을 사용하세요. 모든 HTTPS 호스트는 이미 CSP에서 허용됩니다. 일반 <code>http://</code> 호스트는 CSP 변경이 필요합니다.
지오 코딩	공개 Nominatim	자체 호스팅 Nominatim이나 Pelias를 가리키게 하세요(데이터 통합 참고).
라우팅과 등시 선	공개 FOSSGIS Valhalla(<code>valhalla1.openstreetmap.de</code>)	<code>VITE_ROUTING_ENDPOINT</code> 를 자체 Valhalla 서버로 설정하세요. Processing → GeoLibre Toolbox → Network와 Drive time (운전 시간) / Walk time (도보 시간) 빠른 작업 이 여기에 해당합니다. 빌드 시 또는 Settings → Environment variables에서 브라우저별로 읽습니다. Docker 이미지에는 이를 위한 빌드 인수나 런타임 변수가 없습니다. HTTPS 엔드포인트는 CSP 변경이 필요 없습니다.
포인 터 고 도 표시	3D 지형에 해당 지점의 샘플이 없을 때 마다 공개 Open-Meteo 고도 API	표시를 꺼 두거나(기본적으로 꺼져 있음), 첫 원격 조회 전에 GeoLibre가 표시하는 동의 프롬프트를 거부하세요 — 그것이 리졸버가 확인하는 게이트입니다. 3D 지형을 활성화하면 원격 호출이 드물어지지만 완전히 배제되지는 않습니다. 지형이 답할 수 없는 지점은 여전히 원격으로 넘어가기 때문입니다.

기능	기본값	내부에 유지하는 방법
Python (Pyodide) 백터 엔진	jsDelivr에서 Pyodide 로드	VITE_PYODIDE_INDEX_URL 을 Pyodide 배포본의 미리 사본으로 설정하세요(빌드 시 또는 Settings → Environment variables에서 브라우저별로). 미리를 자신의 출처에서 제공하거나, jsDelivr Pyodide 경로만 고정하는 CSP script-src 에 추가하세요.
AI 어시스턴트	구성하지 않으면 꺼짐	GEOLIBRE_AI_URL 을 설정하지 않거나, 자체 프록시를 통해 라우팅하세요.
프로젝트 공유	share.geolibre.app	GEOLIBRE_SHARE_URL=off , 또는 자체 프로젝트 서버.
협업	구성하지 않으면 꺼짐	GEOLIBRE_COLLAB_URL 을 설정하지 않거나, workers/collab-node 를 직접 운영하세요.
텔레메트리	없음	GeoLibre는 분석이나 사용 데이터를 수집하지 않습니다. 호스팅된 사이트(geolibre.app, web.geolibre.app)는 Google Analytics를 실행하지만, 직접 배포하는 것은 아무것도 실행하지 않습니다. Docker 이미지는 이를 결 수 없습니다 . Dockerfile 은 이를 위한 빌드 인수를 선언하지 않고, 컨테이너의 CSP는 googletagmanager.com 을 허용하지 않습니다. 소스에서 빌드하면 자체 측정 ID로 npm run build 에 VITE_GEOLIBRE_GA_MEASUREMENT_ID 를 설정하여 결 수 있습니다. 이를 활성화하는 런타임 변수는 없습니다. 개인정보 처리방침 을 참고하세요.

배포 체크리스트

- [] GeoLens(또는 자체 데이터 호스트)가 커뮤니티 VM에서 실행 중이며, 데이터셋은 기본적으로 비공개.
- [] GeoLibre 웹 빌드가 **같은 출처**에서 /gis 같은 경로 아래에 제공되며, 일치하는 GEOLIBRE_APP_BASE 로 빌드됨.
- [] 둘 앞에 SSO 계층을 갖춘 TLS 종단 리버스 프록시 하나.
- [] 실수로 외부에 아무것도 게시되지 않도록 GEOLIBRE_SHARE_URL=off (또는 자체 서버).
- [] 생성된 공개 /deployment.json 을 검토하고, 거부된 사이드카 라우트를 시험하여 403을 반환하는지 확인.
- [] GEOLIBRE_CONVERSION_ROOTS 제한, GEOLIBRE_POSTGIS_HOSTS 와 GEOLIBRE_MSSQL_HOSTS 는 필요하지 않으면 설정 안 함, 사용하지 않으면 사이드카 비활성화.
- [] CSP에서 루프백 connect-src 허용 항목 제거.
- [] 배포가 공개 인터넷에 도달하면 안 된다면 베이스맵, 지오코딩, Pyodide 소스를 내부 호스트로 지정.
- [] **로그아웃**한 브라우저로 테스트: 앱, 카탈로그, 타일, ?url= 프로젝트 모두 접근할 수 없어야 함.

관련 페이지

- 시작하기: Docker로 실행
- 프로젝트, 계정, 공유의 자체 호스팅을 위한 [Server API](#)
- 자체 호스팅 가능한 릴레이를 위한 [협업](#)
- 데이터 통합
- 임베딩과 공유
- 개인정보 처리방침

<https://geolibre.app/server-api/>

GeoLibre 프로젝트 및 신원 API

이 문서는 GeoLibre의 Project Gallery와 **Project → Share**(프로젝트 → 공유) 흐름이 사용하는 HTTP 계약의 버전 1을 정의합니다. 호환 서버는 어떤 구현이나 스토리지 엔진을 사용해도 됩니다. 참조 구현은 `backend/geolibre_server_api`에 있습니다.

규약

- 기본 URL은 컨테이너 런타임에 `GEOLIBRE_SHARE_URL` 로(또는 빌드 시점에 `VITE_GEOLIBRE_SHARE_URL` 로) 구성합니다.
- JSON 요청 및 응답 본문은 `application/json` 과 카멜 케이스 키를 사용합니다.
- 날짜는 UTC ISO 8601 문자열입니다.
- 인증이 필요한 엔드포인트는 `Authorization: Bearer <token>` 으로 개인 API 토큰 또는 OAuth 액세스 토큰을 받습니다.
- 오류 응답은 `error` 문자열을 가진 JSON 객체입니다. 401 은 토큰이 없거나, 유효하지 않거나, 만료되었음을 뜻합니다. 403 은 인증된 주체에게 권한이 없음을 뜻합니다. 404 는 존재하지 않는 프로젝트와 호출자가 발견해서는 안 되는 프로젝트를 의도적으로 함께 다룹니다. 409 는 고유성 충돌, 422 는 잘못된 형식의 입력, 429 는 속도 제한입니다.
- 공개 및 비공개 목록(unlisted) 원본 프로젝트 본문(최신 및 버전별)은 강한 ETag 와 함께 `Cache-Control: public, no-cache` 를 사용합니다. 일치하는 태그로 `If-None-Match` 를 보내면 304 를 반환합니다. 캐시는 본문을 저장할 수 있지만 사용할 때마다 재검증해야 하므로, 가시성 변경은 다음 가져오기에서 적용됩니다. 비공개, 조직, 또는 그룹 보호 콘텐츠를 포함하는 응답은 메타데이터 목록을 포함해 `Cache-Control: private, no-store` 를 사용해야 합니다.
- 교차 출처(cross-origin) 웹 배포는 GeoLibre 웹 오리진으로부터의 `Authorization` 과 `Content-Type` 을 허용해야 합니다. 브라우저 fetch를 사용하는 자체 호스팅 Tauri 설치에서는 `tauri://localhost` 및/또는 `http://tauri.localhost` 를 명시적으로 허용하세요. 기본 제공 데스크톱 HTTPS 공유 오리진은 인증된 요청에 네이티브 HTTP를 사용하며, 그 전송 방식은 CORS에 의존하지 않습니다.

참조 서버가 운영자에게 맡기는 부분

배포 보호 조치는 여전히 운영자의 책임입니다:

- **개인 토큰 수명 주기.** `expiresInDays` 를 생략하면 v1의 삭제 전용 수명 주기가 유지되어 만료되지 않는 토큰이 생성됩니다. 기한이 있는 자격 증명이 필요한 곳에서는 1-365일의 명시적 수명을 요구하고, 삭제 전용 및 레거시 토큰은 운영적으로 취소하거나 교체하세요.
- **속도 제한.** OAuth 동의 흐름은 브라우저 바인딩당 대기 중인 상호작용 수를 제한하지만, 새 쿠키를 쓰면 그 제한을 우회할 수 있습니다. 참조 서버에는 일반적인 요청 제한기가 없습니다. OAuth를 공개적으로 활성화하기 전에, 인그레스에서 **GET 및 POST** `/oauth/authorize`, `GET /oauth/sso/callback`, `POST /oauth/`

token, POST /api/auth/token, POST /api/accounts, POST /api/account/password에 대해 클라이언트 IP별 제한을 적용하세요. 마지막 네 개의 POST에는 비밀번호 또는 토큰 작업이 포함됩니다. 동의 로그인과 PAT/계정 라우트는 scrypt를 실행하고, 싱글 사인온 콜백은 조직의 신원 제공자를 호출합니다. 인그레스가 자격 증명을 안전하게 파싱할 수 있는 곳에는 사용자 이름별 제한도 추가하세요. API로 향하는 모든 공개 경로는 이 제한기를 거쳐야 합니다. Compose는 기본적으로 API 호스트 포트를 루프백에 바인딩합니다. 루트 발급자 nginx 배포라면 이 존을 http 컨텍스트에, 이 location을 TLS 발급자 server 컨텍스트에 둘 수 있습니다:

```
# http context
limit_req_zone $binary_remote_addr zone=geolibre_auth:10m rate=12r/m;

# TLS issuer server context; proxy other API routes separately.
location ~ ^/(oauth/(authorize|token|sso/callback)|api/(auth/token|accounts|account/password))
$ {
    limit_req zone=geolibre_auth burst=6 nodelay;
    limit_req_status 429;
    client_max_body_size 16k;
    access_log off;
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $http_host;
}
```

포트를 포함한 원래 Host 권한(authority)을 보존하세요. 그렇지 않으면 OAuth 호스트 바인딩이 요청을 거부합니다. 발급자의 정확한 디스커버리 URL과 다른 API 경로를 같은 백엔드로 라우팅하세요. 경로 접두사가 붙은 발급자의 경우, 외부에 보이는 접두사에 제한을 적용하고 프록시할 때 그 접두사를 제거하세요. 프록시, WAF, 로드 밸런서 로 그에서는 웹 인그레스의 인가 요청 쿼리 문자열, 콜백 Location 헤더, 콜백 요청 쿼리 문자열을 기록하지 않도록 하세요. - **요청 크기 제한**. 서버는 너무 큰 선언된 Content-Length 를 본문을 읽기 전에 거부하지만, 체크 또는 HTTP/2 요청은 길이를 선언하지 않으므로 라우트별 제한이 적용되기 전에 전체가 파싱됩니다. 프록시에서도 요청 크기를 제한하세요.

참조 구현은 정확성의 기준선이지, 강화된 배포가 아닙니다.

제한

필드	제한
프로젝트 제목(업로드된 프로젝트에서 파생)	유니코드 코드 포인트 100개
사용자 이름	소문자 ASCII 문자, 숫자, 하이픈 3-39자. 하이픈으로 시작하거나 끝날 수 없음
슬러그	소문자 ASCII 문자, 숫자, 하이픈 1-100자
설명	유니코드 코드 포인트 2,000개
태그	태그 20개, 각각 유니코드 코드 포인트 40개
프로젝트 문서	50 MiB UTF-8 JSON
썸네일	5 MiB. PNG, JPEG, 또는 WebP
limit	기본값 24, 최대 100

서버는 더 작은 업로드 제한을 구성할 수 있지만, 그 경우 413 과 그 제한을 설명하는 `error` 를 반환해야 합니다. 참조 서버는 문서와 썸네일 제한을 `GEOLIBRE_MAX_PROJECT_BYTES` 와 `GEOLIBRE_MAX_THUMBNAIL_BYTES` 에서 읽습니다.

가시성

- `public`: 공개 목록에서 발견 가능하며 인증 없이 읽을 수 있습니다.
- `unlisted`: 공개 목록에서는 제외되지만, URL을 가진 누구나 읽을 수 있습니다. 소유자의 인증된 목록에는 나타납니다.
- `private`: 개인 소유 프로젝트는 그룹에 명시적으로 공유되지 않는 한 소유자만 읽을 수 있습니다. 조직 소유 비공개 프로젝트는 조직 관리자와, 생성자가 관리자, 게시자, 또는 구성원으로 남아 있는 동안 그 생성자가 읽을 수 있습니다. 다른 조직 구성원과 뷰어는 명시적인 그룹 공유가 필요합니다. 원본 및 썸네일 URL은 메타데이터 엔드포인트와 같은 Bearer 토큰을 요구합니다.
- `organization`: 소유 조직의 로그인된 모든 구성원이 읽을 수 있습니다. 공개 목록에서는 제외되며, 원본/썸네일 응답은 항상 `Cache-Control: private, no-store` 이므로 구성원을 제거하면 그 클라이언트가 재검증하는 즉시 알려진 URL이 취소됩니다.

가시성 변경은 모든 버전에 즉시 영향을 줍니다. 따라서 비공개 프로젝트의 원본 URL은 능력(capability) URL이 아닙니다.

신원

POST /api/accounts

계정을 만들고 개인 API 토큰을 한 번 반환합니다. 설치 환경이 신원을 외부 제공자에게 위임하는 경우 이 엔드포인트는 비활성화될 수 있습니다. `name`, `scopes`, `expiresInDays` 는 선택 사항입니다. 새 토큰은 기본적으로 세 가지 프로젝트 스코프를 모두 가집니다. `expiresInDays` 를 생략하면 v1의 삭제 전용 토큰 수명 주기가 유지됩니다(토큰이 만료되지 않음). 허용되는 명시적 수명은 1-365일입니다. 알 수 없거나 빈 `scopes` 목록은 400 {"error": "invalid_scope"} 를 반환하고, 1-365 범위를 벗어난 `expiresInDays` 는 400 {"error": "invalid_request"} 를 반환합니다. `email` 은 선택 사항이며, 공백이 제거되고 소문자로 정규화되며, 검증되고, 있을 경우 고유해야 합니다. 8자보다 짧은 비밀번호나 잘못된 형식의 사용자 이름은 422 이고, 이미 사용 중인 사용자 이름이나 이메일은 409 입니다.

```
{
  "username": "ada",
  "password": "correct horse battery staple",
  "email": "ada@example.org",
  "name": "GeoLibre desktop",
  "scopes": ["read:projects", "write:projects"],
  "expiresInDays": 30
}
```

응답 201:

```
{
  "account": {"id": "uuid", "username": "ada", "email": "ada@example.org", "createdAt": "2026-08-03T12:00:00Z"},
  "token": "secret-token",
  "tokenId": "uuid",
  "scopes": ["read:projects", "write:projects"],
  "expiresAt": "2026-09-02T12:00:00Z"
}
```

POST /api/auth/token

계정 자격 증명을 개인 API 토큰으로 교환합니다. 계정 생성과 같은 선택적 정책 필드(단, email 제외)를 받고 같은 형태를 반환합니다. 토큰은 불투명하며 SHA-256 다이제스트로만 저장됩니다.

상태	error
401	invalid username or password
401	account temporarily locked (조직 잠금 정책)
403	password expired (POST /api/account/password 로 변경하세요)
403	single sign-on required (계정이 속한 조직이 내장 계정을 허용하지 않음)

GET /api/account 및 PATCH /api/account

GET /api/account 는 유효한 모든 자격 증명에 대해 {"account": <account>} 를 반환합니다.

PATCH 는 write:projects 가 필요합니다. {"email": "ada@example.org"} 는 로그인된 계정의 검증·정규화된 이메일을 설정하고, {"email": null} 은 이를 지웁니다. 중복된 이메일은 409 입니다. 응답은 {"account": <account>} 이며 Cache-Control: private, no-store 를 사용합니다.

DELETE /api/auth/token

제시된 Bearer 토큰을 취소합니다. 개인 토큰은 삭제되고, OAuth 액세스 토큰은 세션 패밀리 전체를 취소합니다. 응답: 204 .

GET /api/users/me

계정, 유효 자격 증명 스코프, OAuth 세션 ID를 반환합니다. 개인 토큰의 경우 sessionId 는 null 입니다. 관리 권한 부여(management grant)는 프로젝트 스코프가 아닌 ["manage:sessions"] 만 보고합니다.

```
{
  "user": {"id": "uuid", "username": "ada", "email": "ada@example.org", "createdAt": "2026-08-03T12:00:00Z"},
  "sessionId": "oauth-session-uuid",
  "scopes": ["read:projects", "write:projects", "share:public"]
}
```

신원 제공자는 사용자 이름 없이 계정을 만들 수 있습니다. 그런 계정의 프로젝트 생성은 안정적이고 대소문자를 구분하지 않는 센티널 텍스트 `username required` 를 포함하는 오류와 함께 `400` 을 반환해야 합니다. 기존 클라이언트는 이 문구를 인식하여 사용자를 계정 설정으로 안내합니다.

세션 및 개인 토큰 관리

이 라우트들은 **유일한** 스코프가 `manage:sessions` 인 별도의 OAuth Bearer 권한 부여가 필요합니다. 같은 계정의 프로젝트 OAuth 권한 부여라 하더라도, 그리고 모든 개인 API 토큰은 `403 insufficient_scope` 를 받습니다. 클라이언트는 먼저 프로젝트 자격 증명으로 계정 ID와 프로젝트 `sessionId` 를 확인한 다음, 새로운 관리 동의를 요청하고 목록을 조회하기 전에 관리 계정 ID를 비교합니다. 관리 자격 증명은 저장하거나 프로젝트/갤러리 호출에 사용해서는 안 됩니다.

GET `/api/auth/sessions?limit=50&offset=0¤tSessionId=<project-session-id>` 는 `{"sessions": [<session>], "limit": 50, "offset": 0, "total": 1}` 을 반환합니다. `limit` 은 1-100이고 `offset` 은 음수가 아닌 값입니다. `currentSessionId` 를 제공한 경우, 관리 계정이 소유한 활성 프로젝트 세션을 가리켜야 하며, 그렇지 않으면 응답은 `404` 입니다. 서버는 정확히 그 항목에 `current: true` 를 표시합니다. 활성 상태이고 만료되지 않은 프로젝트 OAuth 세션과 개인 API 토큰만 나타나며, 수명이 짧은 관리 권한 부여는 절대 나타나지 않습니다. 행은 생성 시간 최신순으로 정렬되고, 동물이면 ID로 정렬됩니다. 각 행은 토큰이 아닌 공개 UUID를 포함합니다:

```
{
  "id": "uuid",
  "kind": "oauth",
  "clientId": "geolibre-desktop",
  "label": "GeoLibre Desktop",
  "scopes": ["read:projects", "write:projects", "share:public"],
  "createdAt": "2026-08-03T12:00:00Z",
  "lastUsedAt": null,
  "expiresAt": "2026-09-02T12:00:00Z",
  "current": true,
  "legacy": false
}
```

개인 토큰은 `kind: "personal-token"`, `clientId: null`, `current: false` 를 사용하며 `expiresAt: null` 일 수 있습니다. 정책이 없는 오래된 토큰은 목록 조회 시 백필되어 `legacy: true` 로 표시되며, 취소될 때까지 유효합니다. 관리 응답은 `Cache-Control: private, no-store` 를 사용합니다.

DELETE `/api/auth/sessions/{id}` 는 소유한 프로젝트 OAuth 패밀리 또는 개인 토큰에 대해, 이미 취소된 것을 포함하여 `204` 를 반환합니다. 관리 권한 부여는 이 엔드포인트로 지정할 수 없습니다. 알 수 없는 ID, 다른 사람의 ID, 관리 전용 ID는 모두 같은 `404` 를 반환합니다. 취소는 액세스 토큰 하나가 아니라 패밀리를 무효화합니다. ID가 현재 프로젝트 세션이면, 클라이언트는 즉시 로컬 프로젝트 자격 증명과 보호된 Gallery/원격 편집 상태를 지워야 하며, 오래된 세션 UI를 유지해서는 안 됩니다. 붙여 넣은 개인 토큰은 별도의 자격 증명으로 남으며 OAuth 권한 부여로 조용히 대체되지 않습니다.

POST `/api/auth/sessions/revoke-others` 는 `{"currentSessionId": "<project-session-id>"}` 를 받고 `204` 를 반환합니다. 이 계정의 다른 모든 OAuth 패밀리와 **모든** 개인 토큰을 원자적으로 취소하되, 요청에 명시된 소유 중인 활성 프로젝트 세션과 호출 중인 관리 권한 부여는 둘 다 유지합니다. 없거나, 다른 사람의 것이거나, 만료되

있거나, 취소되었거나, 관리 전용인 ID는 부분 취소 없이 404 입니다. 클라이언트는 이 파괴적인 동작을 확인받아야 하며, 개인 토큰을 사용하는 스크립트와 CI가 동작을 멈출 것임을 경고해야 합니다. PostgreSQL에서는 리프레시 교체와 일괄 취소가 세션 행에 대해 직렬화됩니다.

조직

조직 및 그룹 라우트는 프로젝트 라우트와 같은 스코프를 사용합니다. 모든 GET (멤버십, 구성원, 초대, 갤러리, 제한된 그룹의 썸네일, GET /api/projects?shared_with_me=true)은 read:projects 가 필요하고, 모든 변경(조직 또는 그룹 생성, 설정 또는 멤버십 변경, 초대 발급·취소·수락, 참여, 참여 요청 결정, 모더레이션, 썸네일)은 write:projects 가 필요합니다. 비공개 프로젝트처럼 조직이나 그룹을 통해 접근하는 비공개 프로젝트를 읽는 것도 read:projects 가 필요합니다.

POST /api/organizations 는 조직을 만들고 호출자를 첫 번째 administrator 로 지정합니다. 본문에는 slug, name, publicSharingPolicy (yes, publishers, 또는 no), defaultVisibility, 선택적 categories 가 들어갑니다. 슬러그는 전역적으로 고유합니다. 조직 안에서 visibility 없이 프로젝트를 만들면 서버가 defaultVisibility 를 적용합니다. public 기본값은 publicSharingPolicy 가 yes 여야 합니다. 조직을 그 외의 조합으로 생성하거나 패치하면 422 를 반환합니다.

조직 역할은 다음과 같습니다:

- administrator: 설정과 멤버십을 관리하고, 조직 소유 프로젝트를 모두 변경할 수 있습니다.
- publisher: 조직 콘텐츠를 만들고, 정책이 publishers 또는 yes 일 때 공개적으로 게시할 수 있습니다.
- member: 조직 콘텐츠를 만들고 조직 안에서 공유할 수 있습니다. 정책이 yes 일 때만 게시할 수 있습니다.
- viewer: 조직에 보이는 콘텐츠만 읽을 수 있습니다.

조직 콘텐츠를 만든 게시자나 구성원은 그 조직 역할을 유지하는 동안 해당 콘텐츠를 관리할 수 있습니다. 관리자는 모든 조직 프로젝트를 관리할 수 있습니다. 뷰어로 강등되거나 조직에서 제거되면 생성자의 관리 권한이 즉시 제거됩니다. 프로젝트는 고아가 되지 않고 조직 소유로 남습니다.

같은 규칙이 비공개 읽기에도 적용됩니다. 관리자와 활성 생성자는 자신이 관리할 수 있는 비공개 조직 프로젝트를 읽을 수 있습니다. 멤버십만으로는 게시자, 구성원, 뷰어가 다른 사람의 비공개 프로젝트에 접근할 수 없습니다.

관리자는 POST /api/projects/{id}/transfers ({"organizationId": "..."})로 프로젝트를 조직 안으로 옮길 수도 있습니다. 호출자가 이미 수신자를 관리하므로 즉시 적용됩니다([이전 참조](#)).

라우트:

- GET /api/organizations/mine 은 멤버십과 각 호출자의 role 을 나열합니다.
- GET /api/organizations/{id} 는 구성원에게 설정을 반환합니다.
- PATCH /api/organizations/{id} 는 name, publicSharingPolicy, defaultVisibility, 또는 categories 를 변경합니다. 관리자 전용입니다. publicSharingPolicy 를 publishers 나 no 로 강화하면, 새 정책에서 생성자가(현재 역할 기준. 더 이상 조직에 없는 생성자 포함) 게시할 수 없는 모든 public 조직 프로젝트가 organization 가시성으로 바뀌고, 각각에 대해 visibility_change 활동이 기록됩니다.

- DELETE /api/organizations/{id} 는 조직과 조직 소유 프로젝트 및 저장된 객체, 그룹, 구성원, 초대를 삭제합니다. 관리자 전용입니다. 구성원의 개인 프로젝트는 유지됩니다. 조직 프로젝트 중 하나라도 삭제 보호가 걸려 있으면, 그 프로젝트의 보호가 해제될 때까지 409 로 전체 삭제를 거부합니다.
- GET /api/organizations/{id}/members 는 구성원을 나열합니다.
- PUT /api/organizations/{id}/members 는 {"username": "ada", "role": "member"} 를 추가하거나 갱신합니다. 관리자 전용입니다. 역할을 낮추면 공개 공유 정책이 다시 적용됩니다. 생성자가 더 이상 게시할 수 없는 public 프로젝트는 organization 이 됩니다.
- DELETE /api/organizations/{id}/members/{username} 은 구성원을 제거하고(관리자 전용), {username}=me 는 탈퇴합니다. 마지막 관리자는 제거·탈퇴·강등될 수 없습니다(409). 조직 신원 제공자의 비상(break-glass) 관리자도 마찬가지입니다 (422 account is the organization's break-glass administrator). 먼저 제공자에서 breakGlassUsername 을 지우세요. 탈퇴자가 만든 프로젝트는 조직 소유로 남으며, 정책이 yes 가 아닌 한 그중 public 인 것은 organization 이 됩니다. 조직 그룹에서 탈퇴자의 멤버십과 대기 중인 초대는 제거됩니다. 탈퇴자가 소유한 그룹은 그를 제거한 관리자에게 넘어갑니다. 그룹을 소유한 구성원은 탈퇴 전에 소유권을 이전해야 합니다(409).
- POST /api/organizations/{id}/invitations 는 정확히 하나의 username 또는 email 에 대한 대기 중인 초대를 만듭니다. 같은 경로의 GET 은 대기 중, 수락됨, 취소된 상태의 초대를 나열합니다. 발급과 목록 조회는 관리자 전용입니다. 생성 응답에만 불투명 token 이 포함됩니다.
- DELETE /api/organizations/{id}/invitations/{invitationId} 는 대기 중인 초대를 revoked 로 바꿉니다. 관리자 전용입니다.
- POST /api/organizations/invitations/{token}/accept 는 로그인에 필요하며, 계정의 사용자 이름 또는 이메일을 확인하고, 초대된 역할로 구성원을 추가한 뒤, 초대를 accepted 로 바꿉니다.
- GET /api/organizations/{id}/projects 는 조직 갤러리를 반환합니다. 관리자가 아닌 사용자는 공개 및 조직에 보이는 프로젝트와, 자신의 그룹 중 하나에 별도로 공유된 비공개 프로젝트를 봅니다.

프로젝트 생성 또는 패치에서 organizationId 를 제공하면 프로젝트가 조직 소유로 이전됩니다. 그러면 username 은 null 이 되고, 모든 조직 관리자가 관리할 수 있으며, 원본 라우트는 /org/{organizationSlug}/{projectSlug} [.geolibre.json] 을 사용합니다. 조직 소유 프로젝트의 organizationId 는 소유 조직의 관리자만 변경하거나 지울 수 있습니다(그 외에는 403). 지우면 프로젝트는 생성자의 개인 계정으로 돌아갑니다. organizationId 를 변경하는 패치는, groupIds 로 보내졌든 이전 값이 유지되든, 새 조직에서의 생성처럼 프로젝트의 그룹 대상을 다시 확인합니다. 더 이상 자격이 없는 대상을 교체하려면 groupIds 를 보내세요. 공개 공유 정책은 직접 API 요청을 포함해 생성과 패치에서 강제됩니다. 서버는 소유권과 별개로 null 허용 생성자 신원을 보관합니다. 새 프로젝트는 소유권이 개인이든 조직이든 생성 계정을 기록합니다. 조직 소유권이 여전히 권위를 가지며, 생성자 신원은 username 을 채우거나 개인 프로젝트 URL을 만들지 않습니다.

그룹

POST /api/groups 는 독립형 또는 조직 연결 그룹을 만듭니다. 본문에는 name , 선택적 description 과 organizationId , joinPolicy (invite , request , 또는 open), 그리고 sharedUpdate 가 들어갑니다. 조직 연결 그룹은 해당 조직의 구성원만 받습니다. 그 외의 누군가를 추가하거나, 사용자 이름으로 초대하거나, 참여시키

거나, 초대를 수락하거나, 요청을 승인하면 403 을 반환합니다. `sharedUpdate` 는 생성 시 고정되며 패치할 수 없습니다. `name`, `description`, `joinPolicy` 는 설정입니다. 선택적 PNG, JPEG, 또는 WebP 썸네일은 `PUT / GET / DELETE /api/groups/{id}/thumbnail` 을 사용합니다.

그룹 역할은 `owner`, `manager`, `member` 입니다. 수락된 구성원 중 정확히 한 명이 소유자입니다. 소유자는 `PUT /api/groups/{id}/members` 를 통해 `owner` 를 할당하여 소유권을 이전하며, 이전 소유자는 원자적으로 관리자(`manager`)가 됩니다. 관리자는 일반 구성원을 추가/제거하고, 초대하고, 참여 요청을 결정하고, 그룹에서 프로젝트를 제거할 수 있습니다. 관리자를 관리하거나 소유권을 이전하는 것은 소유자만 할 수 있으며, 소유자는 소유권을 이전하기 전까지 탈퇴할 수 없습니다.

라우트:

- `GET /api/groups/mine` 은 수락된 멤버십을 나열하고, `GET /api/groups/{id}` 는 로그인된 호출자에게 그룹 상세 정보를 반환합니다. `PATCH /api/groups/{id}` 는 `name`, `description`, 또는 `joinPolicy` 를 변경합니다. 소유자 또는 관리자 전용입니다.
- `DELETE /api/groups/{id}` 는 그룹을 멤버십, 초대, 썸네일과 함께 삭제합니다. 소유자 전용입니다. 그룹에 공유된 프로젝트는 유지되며 그 그룹 대상만 없습니다.
- `GET /api/groups/{id}/members` 는 수락된 구성원을 나열합니다. 소유자/관리자는 대기 중인 참여 요청도 봅니다.
- `PUT /api/groups/{id}/members` 는 `username` 과 `role` 로 구성원을 추가하거나 변경합니다. `DELETE /api/groups/{id}/members/{username}` 은 구성원 한 명을 제거하고, `{username}=me` 는 탈퇴합니다.
- `POST /api/groups/{id}/invitations` 는 정확히 하나의 `username` 또는 `email` 에 대한 대기 중인 초대를 만듭니다. 생성 응답에는 불투명 토큰이 포함됩니다. 관리자용 목록(같은 경로의 `GET`)은 토큰을 생략하고 대기 중, 수락됨, 취소됨 행을 유지합니다. `DELETE .../invitations/{invitationId}` 는 대기 중인 초대를 `revoked` 로 바꾸고, `POST /api/groups/invitations/{token}/accept` 는 로그인된 대상 계정을 추가하면서 초대를 `accepted` 로 바꿉니다.
- `POST /api/groups/{id}/join` 은 `open` 그룹에는 즉시 참여하고, `request` 그룹에는 대기 중인 요청을 만들며, 초대 전용 그룹은 거부합니다. `accept` 또는 `reject` 결정과 함께 `POST /api/groups/{id}/members/{username}/decide` 를 호출하면 대기 중인 요청을 모더레이션합니다.
- `GET /api/groups/{id}/projects` 는 대상으로 지정된 프로젝트를 나열합니다. `DELETE /api/groups/{id}/projects/{projectId}` 는 프로젝트를 삭제하지 않고 그 대상만 제거합니다.

프로젝트 생성 및 패치 요청은 `groupIds` 를 받습니다. 호출자는 모든 대상의 수락된 구성원이어야 합니다. 조직 소유 프로젝트의 경우, 관리자가 아닌 사용자는 해당 조직의 그룹만 대상으로 지정할 수 있습니다. 구성원은 자신의 그룹을 대상으로 한 비공개 프로젝트를 읽을 수 있으며, 그 그룹의 불변 `sharedUpdate` 값이 `true`인 경우에만 콘텐츠를 갱신할 수 있습니다. 멤버십이나 대상을 제거하면 다음 요청에서 접근이 취소됩니다. 보호된 원본 및 썸네일 응답은 절대 공유되거나 영속적으로 캐시되지 않습니다.

초대 토큰은 `bearer` 자격 증명입니다. 조직 초대와 그룹 초대 모두에서 서버는 SHA-256 다이제스트만 저장하고, 원시 토큰은 생성 호출에서만 반환하며, 수락 조회 전에 경로 토큰을 해시해야 합니다. 수락되거나 취소된 토큰은 재사용할 수 없습니다.

그룹 썸네일은 그룹의 참여 정책을 따릅니다. `open` 그룹의 썸네일은 공개이며 `Cache-Control: public, max-age=3600` 을 사용할 수 있습니다. `invite` 및 `request` 그룹의 경우 수락된 구성원만 썸네일을 가져올 수 있고, 성공한 모든 응답은 `Cache-Control: private, no-store` 를 사용합니다. 비구성원은 `404` 를 받습니다. 이렇게 하면 안정적인 공개 썸네일 URL이 멤버십이 제한된 그룹의 콘텐츠를 노출하는 것을 막습니다.

엔터프라이즈 로그인

조직 보안 정책

조직 관리자는 구성원에 대한 보안 정책을 설정할 수 있습니다.

- `GET /api/organizations/{id}/security-policy (read:projects)` 는 `idleTimeoutSeconds` , `absoluteSessionSeconds` , `adminReauthSeconds` , `adminIpAllowlist` (목록), `passwordMinLength` , `passwordMinClasses` , `passwordMaxAgeDays` , `lockoutThreshold` , `lockoutSeconds` 가 포함된 `{"securityPolicy": {...}}` 를 반환합니다. 설정되지 않은 값은 `null` 입니다. 정책이 없으면 모든 값이 `null` 이고 허용 목록은 비어 있습니다.
- `PUT /api/organizations/{id}/security-policy (write:projects)` 는 정책 전체를 교체하고(생략된 필드는 `null` 이 됨) `GET` 형태를 반환합니다.

둘 다 조직 관리자가 필요하며 `Cache-Control: private, no-store` 로 응답합니다. 범위:

필드	허용 범위
<code>idleTimeoutSeconds</code>	300–2592000
<code>absoluteSessionSeconds</code>	900–31536000
<code>adminReauthSeconds</code>	60–86400
<code>passwordMinLength</code>	8–128
<code>passwordMinClasses</code>	1–4(소문자, 대문자, 숫자, 기호)
<code>passwordMaxAgeDays</code>	1–3650
<code>lockoutThreshold</code>	3–100
<code>lockoutSeconds</code>	60–86400
<code>adminIpAllowlist</code>	최대 50개의 IP 주소 또는 CIDR 네트워크

범위를 벗어난 값은 `422` 입니다. 그 밖의 `422` 오류: `adminIpAllowlist entries must be IP addresses or networks` , `lockoutThreshold and lockoutSeconds must be set together` , `adminIpAllowlist must include your current address` (관리자가 스스로를 잠그지 못하도록).

계정이 여러 조직에 속할 때는 가장 엄격한 값이 적용됩니다. 유효 시간 제한, 절대 세션 수명, 비밀번호 사용 기간, 잠금 임계값은 가장 짧은 값이, 최소 길이, 문자 종류 수, 잠금 지속 시간은 가장 긴 값이 적용됩니다. 비밀번호 길이는 절대 8 미만이 되지 않습니다.

- **유효 및 절대 세션 제한**은 OAuth 세션과 개인 토큰에 적용됩니다. `idleTimeoutSeconds` 보다 오래 유효 상태였거나 로그인 후 `absoluteSessionSeconds` 보다 오래된 OAuth 세션은 취소됩니다. Bearer 사용은 `401 invalid or expired token` 을, 리프레시는 `400 invalid_grant` 를 반환합니다. 개인 토큰은 생성 시점과 마지막 사용 시점을 기준으로 측정되며 같은 방식으로 취소됩니다. 새 OAuth 패밀리는 절대 제한보다 오래 살아남지 못합니다.
- **잠금**: `lockoutThreshold` 번 연속으로 잘못된 비밀번호를 입력하면 계정이 `lockoutSeconds` 동안 잠깁니다. 로그인에 성공하면 횟수가 초기화됩니다.
- **비밀번호 교체**: `passwordMaxAgeDays` 보다 오래된 비밀번호는 `POST /api/account/password` 로 변경할 때까지 로그인 시 거부됩니다. 이 기능 이전에 만들어진 계정의 경우, 업그레이드 후 첫 로그인부터 기간을 셉니다.
- **관리자 재인증**: 조직이 `adminReauthSeconds` 를 설정하면, 로그인 시점이 그보다 오래된 자격 증명으로 수행하는 관리자 변경(조직 설정, 구성원, 초대, 보안 정책)은 `WWW-Authenticate: Bearer error="insufficient_user_authentication", max_age="<seconds>"` (RFC 9470)와 함께 `401 {"error":"reauthentication_required"}` 를 반환합니다. 계속하려면 다시 로그인하세요. 읽기는 영향을 받지 않습니다.
- **관리자 IP 허용 목록**: `adminIpAllowlist` 가 비어 있지 않으면, 그 밖의 주소에서 오는 해당 조직의 모든 관리자 라우트는 `403 administrative access is not allowed from this network` 를 반환합니다. 클라이언트 주소는 직접 피어이지만, 피어가 `GEOLIBRE_TRUSTED_PROXIES` (쉼표로 구분된 IP 또는 CIDR)에 나열되어 있으면 `X-Forwarded-For` 에서 가장 오른쪽의 신뢰되지 않은 항목을 사용합니다.

POST /api/account/password

인증 불필요. 본문: `{"username":"ada","currentPassword":"...","newPassword":"..."}` (새 비밀번호는 최대 1024자). 현재 비밀번호가 만료되었어도 동작합니다. 응답: `204` .

상태	error
401	invalid username or password
401	account temporarily locked
403	single sign-on required
422	new password must differ from the current password
422	password must be at least <n> characters
422	password must use at least <n> of: lowercase, uppercase, digits, symbols

조직 신원 제공자

조직 관리자는 하나의 OpenID Connect 제공자(Entra ID, Okta, Google, Keycloak, ADFS 2016+ 등)를 조직에 연결할 수 있습니다.

- GET `/api/organizations/{id}/identity-provider ( read:projects )`는 `{"identityProvider": {...}}` 또는 404 `identity provider not configured` 를 반환합니다.
- PUT `/api/organizations/{id}/identity-provider ( write:projects )`는 제공자를 만들거나 교체하고 GET 형태를 반환합니다.
- DELETE `/api/organizations/{id}/identity-provider ( write:projects )`는 제공자를 제거합니다.
응답: 204 . 구성된 것이 없을 때도 같습니다.

세 가지 모두 조직 관리자가 필요하며(조직의 IP 허용 목록이 적용되고, PUT 과 DELETE 에는 재인증이 적용됨) Cache-Control: private, no-store 로 응답합니다. PUT 본문:

```
{
  "issuer": "https://login.example.org/realms/acme",
  "clientId": "geolibre",
  "clientSecret": "...",
  "tokenEndpointAuthMethod": "client_secret_basic",
  "scopes": ["openid", "email", "profile"],
  "usernameClaim": "preferred_username",
  "emailClaim": "email",
  "groupsClaim": "groups",
  "defaultRole": "member",
  "roleMappings": [{ "value": "gis-admins", "role": "publisher" }],
  "groupMappings": [{ "value": "gis-admins", "groupId": "group-uuid" }],
  "requireMfa": false,
  "allowBuiltinAccounts": true,
  "breakGlassUsername": null,
  "enabled": true
}
```

필드	규칙
issuer	필수. 최대 512자, 쿼리나 프래그먼트가 없는 <code>https://</code> . ID 토큰의 iss 와 정확히 비교되며 절대 정규화되지 않습니다.
clientId	필수. 1-255자.
clientSecret	1-512자. 생성 시 필수. 갱신 시 생략하거나 null 이면 저장된 비밀을 유지합니다. 절대 반환되지 않습니다.
authorizationEndpoint, tokenEndpoint, jwksUri	셋 모두(각각 <code>https://</code>) 또는 모두 생략. 모두 생략하면 발급자의 디스커버리 문서에서 읽습니다.
tokenEndpointAuthMethod	client_secret_basic (기본값) 또는 client_secret_post .
scopes	openid 를 포함한 최대 20개의 단순 스코프 토큰. 기본값 openid , email , profile .
usernameClaim, emailClaim, groupsClaim	클레임 이름. 최대 64자. 기본값 preferred_username , email , groups . groupsClaim 은 null 일 수 있습니다.

필드	규칙
defaultRole	일치하는 역할 매핑이 없을 때의 조직 역할. 기본값 member.
roleMappings	최대 100개의 {"value", "role"} 항목.
groupMappings	최대 100개의 {"value", "groupId"} 항목. 각 그룹은 이 조직에 속해야 합니다.
requireMfa	ID 토큰의 amr 에 "mfa" 를 요구합니다. 기본값 false. 개별 인증 요소만 나열하는 제공자(예: ["pwd", "otp"])는 "mfa" 를 추가하는 클레임 매퍼가 필요하며, 그렇지 않으면 모든 로그인이 거부됩니다.
allowBuiltinAccounts	false 이면 조직 구성원의 비밀번호 로그인을 비활성화합니다. 기본값 true.
breakGlassUsername	비밀번호 로그인을 유지하는 이 조직의 현재 관리자. allowBuiltinAccounts 가 false 일 때 필수입니다.
enabled	비활성화된 제공자는 제공되지도 수락되지도 않습니다. 기본값 true.

GET 형태는 설정(scopes 는 목록으로)과 함께 protocol ("oidc"), 저장된 엔드포인트, 비밀 대신 clientSecretSet: true, redirectUri, updatedAt 을 돌려줍니다. S256 PKCE를 사용하는 인가 코드 흐름의 기밀 클라이언트를 위해 redirectUri (이 서버의 <issuer>/oauth/sso/callback. OAuth가 구성되지 않았으면 null)를 신원 제공자에 등록하세요.

422 오류: issuer must be an https URL without query or fragment, clientSecret is required, set all three endpoints or none, identity provider endpoints must be https URLs, scopes must include openid and use simple scope tokens, group mapping must name a group in this organization, break-glass account must be an organization administrator, a break-glass administrator is required when built-in accounts are disallowed, identity provider discovery failed. 범위를 벗어난 값은 일반 422 입니다.

- **디스커버리:** 명시적 엔드포인트가 없으면 PUT 은 <issuer>/well-known/openid-configuration 을 가져옵니다. 그 issuer 는 구성된 issuer 와 정확히 같아야 하며, https:// 인가, 토큰, JWKS 엔드포인트 를 명시해야 합니다. 네트워크 오류, 200 이 아닌 상태, 1 MiB를 넘는 본문, 잘못된 JSON도 디스커버리를 실패시킵니다. 엔드포인트는 저장되며, 디스커버리는 다음 PUT 에서만 다시 실행됩니다. 서명 키는 첫 로그인 시 jwksUri 에서 가져와 캐시됩니다. issuer 또는 jwksUri 를 변경하면 캐시가 지워집니다. 제공자가 연합 신원을 연결한 후에는 PUT 으로 issuer 나 jwksUri 를 변경할 수 없으며 409 issuer or JWKS endpoint cannot change while federated identities are linked 를 반환합니다. 이는 대체 권한 기관이 sub 값을 재사용하여 기존 계정을 확인하는 것을 막습니다. 계정 마이그레이션이나 재연결은 자동이 아닙니다. 삭제는 신원 연결을 제거하므로 우회 방법으로 제공자를 삭제하고 다시 만들지 마세요. 이전 설정에 대해 검증된 로그인 은 신원 연결 전에 그 설정이 바뀌었다면 거부됩니다. PostgreSQL에서는 PUT 과 로그인이 제공자 행에 대해 직렬화됩니다. 제공자 제거나 교체와 경쟁하는 PUT 은 identity provider was removed or replaced; try again 메시지와 함께 HTTP 409를 반환합니다.

- **내부 주소(SSRF 보호):** 로그인한 사용자는 누구나 조직을 만들고 제공자 URL을 고를 수 있으므로, 서버는 공인 주소에서만 신원 제공자에 연결합니다. 각 호스트는 한 번 해석되며, 루프백, 사설, 링크 로컬, CGNAT(100.64.0.0/10), 멀티캐스트, 예약, 미지정 주소(IPv4 매핑, NAT64, 6to4, Teredo 형태 포함)는

모두 제외됩니다. 남은 주소가 없는 호스트는 네트워크 오류처럼 실패하므로, 디스커버리는 `identity provider discovery failed` 로 응답하고 로그인은 거부됩니다. 연결은 확인된 주소로 향하되, TLS는 여전히 URL의 호스트 이름에 대해 인증서를 검증합니다. 내부 네트워크의 제공자를 사용하려면 그 네트워크를 `GEOLIBRE_OIDC_ALLOWED_NETWORKS` (쉼표로 구분된 IP 또는 CIDR. 잘못된 항목은 시작을 실패시킴)에 나열하세요.

- **계정:** 제공자 주체(`sub`)의 첫 로그인은 그에 연결된 계정을 만듭니다. 사용자 이름은 `usernameClaim` (없으면 `emailClaim`)에서 가져옵니다. 소문자로 바꾸고, `@` 에서 자르고, 그 밖의 문자는 `-` 로 바꾸며, 이미 사용 중이면 `-2 ... -99` 접미사를 붙입니다. 아무것도 맞지 않으면 계정에는 사용자 이름이 없습니다(`GET /api/users/me` 아래의 `username required` 센티널 참조). 이메일은 `email_verified` 가 `true` 이고 다른 계정이 그 주소를 사용하지 않을 때만 설정됩니다. 로그인은 절대 이메일로 기존 계정에 연결되지 않으므로, 싱글 사인온으로 옮기는 기존 구성원은 새 계정을 받게 됩니다. 연합 계정에는 비밀번호가 없습니다.
- **매핑**은 `groupsClaim` 값(문자열 또는 문자열 목록)에서 매 로그인마다 적용됩니다. 역할은 일치하는 `roleMappings` 중 가장 높은 순위의 `role` 이고, 없으면 `defaultRole` 입니다. 새 구성원은 이를 받습니다. 기존 구성원의 역할은 `roleMappings` 가 비어 있지 않을 때만 이를 따르며, 조직의 마지막 관리자와 비상 관리자는 절대 강등되지 않습니다. 낮아진 역할은 `PUT /api/organizations/{id}/members` 와 같은 방식으로 공개 공유 정책을 다시 적용합니다. 구성원이 더 이상 게시할 수 없는 `public` 프로젝트는 `organization` 이 됩니다. 매핑된 각 그룹에 대해, 일치하는 값이 있으면 계정은 `member` 가 되고 없으면 일반 `member` 행을 잃습니다. 소유자와 관리자 행은 절대 변경되지 않습니다.
- **내장 계정:** `allowBuiltinAccounts: false` 이면, 비상 관리자를 제외한 조직의 모든 구성원에 대해 올바른 비밀번호가 거부됩니다. `POST /api/auth/token` 과 `POST /api/account/password`에서는 `403 single sign-on required`, 동의 페이지에서는 `Your organization requires single sign-on. Use "Sign in with your organization"`. 입니다. 비상 계정은 조직의 관리자로 남아 있는 동안에만 비밀번호 로그인을 유지합니다. 잠금은 여전히 적용됩니다. 비상 계정인 동안에는 강등·제거·탈퇴할 수 없습니다(`422`). 먼저 `breakGlassUsername` 을 지우세요.
- **삭제**는 제공자, 그 계정 연결, 대기 중인 로그인 리디렉션을 제거합니다. 계정과 멤버십은 남지만, 싱글 사인온으로 만들어진 계정은 로그인할 방법이 없습니다. 나중에 구성된 제공자는 주체를 새로 연결하므로 다음 로그인에서 새 계정이 만들어집니다. 오래된 계정은 제거하세요.
- **비밀 저장:** `clientSecret` 은 데이터베이스의 다른 내용과 같은 신뢰 수준으로, 암호화되지 않은 채 데이터베이스에 저장됩니다.

동의 페이지의 싱글 사인온

어떤 조직이든 활성화된 신원 제공자가 있으면, 동의 페이지에 두 번째 폼이 추가됩니다. `Organization` 필드(조직 슬러그)와 `Sign in with your organization` 버튼 (`decision=sso`, 같은 `interaction`, `csrf`, `label` 필드 포함)입니다. 이 폼의 `POST` 는 `state`, `nonce`, `S256 PKCE` 챌린지, 그리고 조직의 보안 정책이 `adminReauthSeconds` 를 설정한 경우 `max_age` 와 함께 해당 조직의 인가 엔드포인트로 `303` 을 응답합니다. 알 수 없는 슬러그나 비활성화된 제공자는 `Single sign-on is not configured for that organization` 과 함께 동의 페이지를 다시 렌더링합니다. 브라우저는 그 리디렉션에 `form-action` 을 적용하므로, 싱글 사인온이 제공되는 동안 동의 페이지의 `Content-Security-Policy` 는 `form-action` 에 `https:` 를 추가합니다.

GET /oauth/sso/callback 은 제공자의 응답을 받습니다. state 는 살아 있고 사용되지 않은 것이어야 하고, 상호작용은 결정되지 않았고 만료되지 않은 것이어야 하며, 요청은 동의를 시작한 브라우저의 브라우저 바인딩 쿠키를 가지고 있어야 합니다.

- 제공자로부터의 `error: error=access_denied` 와 함께 클라이언트의 콜백으로 303 .
- 그 외의 경우 서버는 클라이언트 비밀과 PKCE 검증자로 토큰 엔드포인트에서 `code` 를 교환하고 ID 토큰을 검증합니다. 제공자의 JWKS로부터의 RS256, PS256, 또는 ES256 서명(알 수 없는 키 ID는 분당 최대 한 번 키 세트를 다시 가져옴), `iss`, `aud` (대상이 여러 개일 때는 `azp` 도), 60초의 여유를 둔 `exp` 와 `iat`, `nonce`, `sub`, `requireMfa` 가 설정된 경우 `amr`, `max_age` 를 보낸 경우 `auth_time` 을 검증합니다. 성공하면 비밀번호 로그인과 똑같이 상호작용을 승인합니다. `code`, `state`, `iss` 와 함께 클라이언트의 콜백으로 303 합니다. OAuth 세션의 로그인 시각은 ID 토큰에 `auth_time` 이 있으면 그 값입니다.
- 재사용된 `state` 를 포함한 그 밖의 모든 실패는 `invalid_request: single sign-on response rejected` 와 함께 400 페이지를 반환합니다. 이유는 로그에만 남습니다(`oidc sign-in rejected: <reason>`).

신원 제공자 호출은 연결 또는 읽기가 10초 동안 멈추거나, 전체 응답이 10초 이상 걸리면 포기합니다. 리디렉션을 절대 따라가지 않고, 프록시 환경 변수를 무시하며, 1 MiB에서 읽기를 멈춥니다. 사설 CA가 발급한 인증서를 쓰는 제공자도 신뢰하려면 `GEOLIBRE_OIDC_CA_BUNDLE` 을 설정하세요. 공인 CA는 계속 신뢰됩니다.

신뢰 헤더 프록시 로그인

`GEOLIBRE_PROXY_AUTH=true` (또는 `1 / yes`)이고 `GEOLIBRE_TRUSTED_PROXIES` 에 나열된 신원 인식 프록시 뒤에 있으면, 동의 페이지는 프록시의 사용자 헤더 (`GEOLIBRE_PROXY_USER_HEADER`, 기본값 `Remote-User`)와 선택적 이메일 헤더 (`GEOLIBRE_PROXY_EMAIL_HEADER`, 기본값 `Remote-Email`)를 신뢰합니다.

`GEOLIBRE_PROXY_AUTH` 가 없으면 `GEOLIBRE_TRUSTED_PROXIES` 는 `X-Forwarded-For` 만 신뢰하고 신원 헤더는 절대 읽지 않습니다. 직접 피어가 신뢰 프록시이고 사용자 헤더가 있으면, 페이지는 비밀번호 및 싱글 사인온 폼 대신 `Signed in through your organization's proxy as <user>` 를 표시하고, `Allow` 는 그 사용자에게 연결된 계정에 대해 상호작용을 승인합니다. 첫 로그인은 계정을 만듭니다. 사용자 이름은 싱글 사인온과 같은 방식으로 파생되고, 이메일은 유효하고 사용 중이지 않을 때 설정됩니다. 조직 매핑과 내장 계정 스위치는 프록시 신원에 적용되지 않습니다. 빈 사용자, 255자를 넘는 사용자, 제어 문자를 포함한 사용자는 `invalid_request: invalid proxy identity` 와 함께 400 페이지를 반환합니다. 만들 수 없는 계정은 `invalid_request: proxy sign-in failed` 를 반환합니다. 다른 피어의 신원 헤더는 절대 읽지 않으므로, 프록시는 클라이언트가 보낸 신원 헤더를 제거해야 하며 API로 가는 유일한 네트워크 경로여야 합니다.

SCIM 2.0

조직의 신원 제공자는 SCIM 2.0(RFC 7643/7644)으로 사용자와 그룹을 프로비저닝할 수 있습니다. 조직 관리자가 이를 위한 토큰을 발급합니다:

- POST /api/organizations/{id}/scim-tokens (write:projects), 본문 `{"label": "Entra ID"}` (1-100자)는 201 `{"token": "...", "scimToken": {...}, "baseUrl": "<public URL>/scim/v2/<organization id>"}` 를 반환합니다. 원시 토큰은 여기서만 표시되며, 서버는 다이제스트를 저장합니다.

- GET /api/organizations/{id}/scim-tokens (read:projects)는 {"scimTokens": [...]} 를 최신 순으로 반환합니다.
- DELETE /api/organizations/{id}/scim-tokens/{tokenId} (write:projects)는 토큰을 취소합니다. 응답: 204. 알 수 없는 토큰이나 다른 조직의 토큰은 404 SCIM token not found 입니다.

토큰 객체는 {"id", "label", "createdAt", "lastUsedAt", "revokedAt"} 입니다 (lastUsedAt 은 분당 최대 한 번 갱신됨). 이 라우트들은 조직 관리자가 필요하며(조직의 IP 허용 목록 적용, POST 와 DELETE 에는 재인증 적용) Cache-Control: private, no-store 로 응답합니다.

신원 제공자에서 baseUrl 을 테넌트/SCIM URL로, 토큰을 Bearer 비밀로 구성하세요. 모든 SCIM 요청에는 해당 조직의 Authorization: Bearer <token> 이 필요합니다. 없거나, 취소되었거나, 다른 조직의 토큰은 WWW-Authenticate: Bearer 와 함께 401 입니다. 토큰은 생성자가 비활성화되거나 활성 조직 관리자가 아니게 되어도 동작을 멈춥니다. SCIM 요청은 머신 간 통신이므로 조직의 관리자 IP 허용 목록과 재인증 기간이 적용되지 않습니다. 제공자를 차단하려면 토큰을 취소하세요.

요청 본문은 application/scim+json (또는 application/json)이고, 응답은 application/scim+json 입니다. 오류는 SCIM 오류 메시지를 사용합니다: {"schemas": ["urn:ietf:params:scim:api:messages:2.0:Error"], "status": "409", "detail": "userName already exists", "scimType": "uniqueness"}.

엔드포인트	비고
GET / ServiceProvider Config, / ResourceTypes, / Schemas	디스커버리. PATCH와 필터링을 지원합니다(최대 100개 결과). 벌크, 정렬, ETag, 비밀번호 변경은 지원하지 않습니다.
GET /Users	startIndex (기본값 1)와 count (0-100, 기본값 100), 생성 순서로 정렬. filter 는 userName eq "..." (대소문자 구분 없음) 또는 externalId eq "..." 만 가능. 그 외는 400 invalidFilter.
POST /Users	계정을 만들거나, 이 조직의 활성화된 신원 제공자에 소문자화된 사용자 이름 클레임 또는 검증된 이메일 클레임이 고유하게 일치하는 기존 SSO 계정이 있으면 그 계정을 채택합니다. 비밀번호 계정, 프록시 신원, 다른 조직이 관리하는 계정, 모호한 일치는 채택되지 않습니다. userName (1-255자, 소문자로 저장)은 필수이며 조직별로 고유합니다(409 uniqueness).
GET / PUT / PATCH / DELETE / Users/{id}	id 는 계정 ID입니다. PUT 은 userName, externalId, displayName, emails, 그리고 (있는 경우) active 를 교체합니다.
GET /Groups	filter 는 displayName eq "..." (대소문자 구분 없음) 또는 externalId eq "..." 만 가능.
POST /Groups	토큰 생성자가 소유하고 join_policy 가 invite 인 조직 그룹을 만듭니다. 모든 구성원은 이 조직에 프로비저닝된 사용자여야 합니다(400 invalidValue).
GET / PUT / PATCH / DELETE / Groups/{id}	PUT 은 displayName, externalId, 구성원 집합을 교체합니다. DELETE 는 그룹을 삭제합니다.

사용자 리소스는 `id`, `userName`, `externalId`, `displayName`, `active`, `emails` (기본 주소만), `meta` 를 가집니다. 그 밖의 속성은 받아들여지지만 무시됩니다. 프로비저닝된 계정에는 비밀번호도 자체 이메일 주소도 없습니다. `emails` 는 SCIM 표현을 위해서만 유지됩니다. 사용자 이름은 싱글 사인온이 파생하는 방식으로 `userName` 에서 파생되며, 신원 제공자의 `defaultRole` (제공자가 없으면 `member`)로 조직에 참여합니다. 그룹 리소스의 `members` 에는 일반 수락 구성원만 나열됩니다. 소유자와 관리자는 SCIM에 의해 절대 나열되거나 변경되지 않습니다. SCIM을 통해 만들어진 사용자와 그룹만 SCIM에 보입니다(그 외는 `404 resource not found`).

SCIM 신원 클레임은 모든 OIDC 로그인에서 새로 고쳐집니다. 이메일 클레임은 제공자가 `email_verified: true` 를 선언할 때만 일치 대상이 됩니다. 이전에 로그인한 계정에 SCIM 리소스가 없고 일치하는 SCIM 리소스가 사용되지 않은 SCIM 생성 계정에 속해 있으면, 다음 로그인에서 SCIM 리소스를 실제 계정으로 조정하고, 이 조직의 SCIM 그룹에서 `member` 역할 항목만(중복 없이) 이전한 뒤, 고아 계정을 비활성화하고 그 조직 멤버십과 그룹 구성원 행을 제거합니다. 고아 계정 레코드는 유지됩니다. 비활성화된 SCIM 리소스는 조정 후에도 비활성화 상태로 남습니다.

관리되는 SCIM 사용자를 삭제하면 계정이 비활성화되고 OAuth 및 개인 토큰 자격 증명이 취소되지만, 연합 신원은 유지됩니다. 같은 사용자 이름을 다시 프로비저닝하면 그 비활성화된 계정을 채택하고 이전 자격 증명을 복원하지 않은 채 다시 활성화합니다. 비활성화 전에 승인된 인가 코드는 이후에 교환할 수 없습니다.

PATCH 본문에는 `urn:ietf:params:scim:api:messages:2.0:PatchOp` 스키마와 `Operations` 목록이 필요합니다(없으면 `400 invalidSyntax`). `op` 는 대소문자를 구분하지 않습니다. 사용자의 경우 `add` 와 `replace` 는 경로 `active`, `userName`, `externalId`, `displayName`, `emails`, `emails[type eq "work"].value` 를 받거나, 경로 없이 속성 객체를 받습니다. 알 수 없는 경로는 무시됩니다. `remove` 는 `externalId` 와 `displayName` 만 받습니다. 그 밖의 작업은 `400 unsupported patch operation (invalidSyntax)`입니다. 그룹의 경우 `add` 는 `members ({"value": "<user id>"}`의 목록), `displayName`, 또는 `externalId` 를 받습니다. `remove` 는 `members[value eq "<user id>"]`, 값 목록이 있는 `members` (값이 없으면 모든 구성원 제거), 또는 `externalId` 를 받습니다. `replace` 는 `displayName`, `externalId`, `members`, 또는 경로 없이 그 속성들의 객체를 받습니다. 성공한 PATCH는 리소스와 함께 `200` 을 반환합니다.

Entra ID: `active` 는 문자열 `"True"` / `"False"` 일 수 있고, 작업 이름은 대문자로 시작할 수 있으며(Replace), Entra의 추가 속성 경로는 무시되므로 기본 속성 매핑이 그대로 동작합니다. Entra는 `active` 를 `false` 로 설정하여 소프트 삭제하고, 나중에 DELETE 를 보냅니다.

비활성화. `active` 를 `false` 로 설정하거나(PUT 또는 PATCH) DELETE 하면, 계정을 누가 관리하는지에 따라 달라집니다:

- 이 조직이 관리하는 계정(해당 조직의 SCIM 또는 싱글 사인온으로 생성)은 비활성화됩니다. 같은 트랜잭션에서 모든 OAuth 세션과 개인 토큰이 취소되고, Bearer 사용은 `401 invalid or expired token` 을, 리프레시는 `400 invalid_grant` 를 반환하며, 비밀번호 로그인은 잘못된 비밀번호로 실패하고, 싱글 사인온은 `This account has been deactivated.` 와 함께 `403` 페이지를 반환하며, 신뢰 프록시 로그인은 동의 페이지에 같은 메시지를 표시합니다.
- 그 밖의 계정은 이 조직의 멤버십과 조직 그룹의 멤버십(그룹 소유권 제외)만 잃습니다. 계정 자체와 다른 조직은 영향을 받지 않습니다. 인가가 멤버십을 실시간으로 읽기 때문에 다음 요청에서 접근이 끝납니다. 구성원이 아닌 동안 SCIM `active` 는 `false` 로 읽힙니다.

- `active` 를 다시 `true` 로 설정하면 관리되는 계정이 다시 활성화되고, 조직 멤버십이 없으면 다시 추가됩니다. 비활성화로 취소된 자격 증명은 취소된 채로 남으며, 사용자는 다시 로그인합니다.
- `DELETE /Users/{id}` 는 위와 같이 비활성화하고, 조직 및 그룹 멤버십을 제거하며, SCIM 사용자를 잊습니다. 조직이 관리하지 않는 계정의 비활성화나 `DELETE` 로 조직의 유일한 관리자를 제거하는 것은 `409 cannot remove the last organization administrator (mutability)` 이고, 신원 제공자의 비상 계정을 제거하는 것은 `409 cannot remove the organization's break-glass administrator (mutability)` 입니다. 먼저 제공자에서 `breakGlassUsername` 을 지우세요.
- 멤버십 제거는 구성원 API를 통한 탈퇴와 같은 방식으로 조직의 공개 공유 정책을 적용합니다. 정책이 `yes` 가 아닌 한, 제거된 계정의 공개 조직 프로젝트는 조직 전용이 됩니다.

싱글 사인온 연결: SCIM 사용자가 있는 조직에서 어떤 주체의 첫 싱글 사인온은 새 계정을 만드는 대신, `userName` 이 ID 토큰의 `usernameClaim` (소문자화)과 같거나, 없으면 `emailClaim` 과 같은 프로비저닝된 계정에 연결됩니다. 제공자가 그 클레임에 넣는 값으로 `userName` 을 프로비저닝하세요(Entra ID의 경우 `preferred_username` 의 UPN).

프로젝트

프로젝트 표현

```
{
  "id": "uuid",
  "username": "ada",
  "slug": "wetlands",
  "title": "Wetlands",
  "description": "",
  "visibility": "public",
  "canEdit": true,
  "organization": {"id": "uuid", "slug": "watershed-lab", "name": "Watershed Lab"},
  "groupIds": ["group-uuid"],
  "thumbnailUrl": "/api/projects/uuid/thumbnail",
  "views": 12,
  "forkCount": 0,
  "versionCount": 1,
  "featured": false,
  "deleteProtected": false,
  "role": "edit",
  "expiresAt": null,
  "hasPassword": false,
  "createdAt": "2026-08-03T12:00:00Z",
  "updatedAt": "2026-08-03T12:00:00Z",
  "tags": [],
  "rawJsonUrl": "https://example.org/ada/wetlands.geolibre.json",
  "projectUrl": "https://example.org/ada/wetlands",
  "viewerUrl": "https://example.org/?project=https%3A%2F%2Fexample.org%2Fada%2Fwetlands.geolibre.json"
}
```

`organization` 은 가시성과 관계없이 프로젝트가 조직 소유일 때마다 `null` 이 아닙니다. `groupIds` 는 프로젝트가 공유된 그룹 식별자의 배열입니다(없으면 빈 배열). 인증된 프로젝트, 목록, 생성, 갱신 응답에는 서버가 해당 호출자에 대해 계산한 `canEdit` 이 포함됩니다. 개인 소유자, 조직 관리자, 활성 조직 생성자, 또는 `sharedUpdate` 설정이

true인 대상 그룹의 구성원이면 true입니다. 클라이언트는 역할에서 인가를 재구성하는 대신 이 값을 사용해야 합니다. 익명 응답에서는 생략됩니다. 인증된 공개 응답은 호출자에 따라 달라지므로 `Cache-Control: private, no-store`를 사용합니다. 소비자는 알 수 없는 필드를 무시해야 합니다.

`deleteProtected`는 소유자의 프로젝트별 "삭제 방지" 스위치입니다. 기본값은 `false`이며 익명 응답을 포함한 모든 프로젝트 표현에 존재합니다. `true`인 동안 `DELETE /api/projects/{id}`는 거부됩니다(아래 참조).

POST /api/projects

인증 필요. 프로젝트와 그 첫 번째 불변 버전을 만듭니다.

```
{
  "filename": "Wetlands.geolibre.json",
  "content": "{\"version\":\"1.0\", ...}",
  "visibility": "public",
  "organizationId": "org-uuid",
  "groupIds": ["group-uuid-1", "group-uuid-2"]
}
```

`content`는 유효한 GeoLibre 프로젝트 JSON 문서를 담은 문자열입니다. `filename`은 대체 제목/슬러그를 제공하며, 프로젝트 문서의 비어 있지 않은 제목이 우선합니다. `visibility`는 선택 사항이며 `public`, `unlisted`, `private`, 또는 `organization`입니다. `visibility`가 `organization`이면 `organizationId`가 필수입니다. `groupIds`는 선택적 그룹 식별자 배열입니다. 호출자는 나열된 모든 그룹의 구성원이어야 하며, 조직 프로젝트의 경우 관리자가 아닌 사용자는 해당 조직의 그룹만 나열할 수 있습니다. `visibility`를 생략하면 조직의 `defaultVisibility`가 적용되고, 개인 프로젝트는 `private`이 됩니다.

선택적 공유 링크 설정(`public` 또는 `unlisted` 프로젝트에만 해당. 다른 가시성에서 하나라도 설정하면 422): `role` (`view`, `comment`, 또는 `edit`. 기본값 `edit`), `expiresIn` (`24h`, `7d`, `30d`, 또는 `never`), `password`. 이 값들은 모든 프로젝트 표현에서 `role`, `expiresAt` (ISO 타임스탬프 또는 `null`), `hasPassword`로 돌려줍니다. `role`은 뷰어를 위한 메타데이터이며, 서버는 만료와 비밀번호만 강제합니다. `expiresAt`이 지나면 프로젝트 관리자를 제외한 모든 사람이 모든 읽기 라우트에서 410 (`share link expired`)을 받습니다. 비밀번호가 설정된 동안, 그런 읽기 사용자들은 `{"password": "..."}` 와 함께 `POST /{username}/{slug}/access` (조직 프로젝트는 `POST /org/{organization}/{slug}/access`)로 링크를 잠금 해제할 때까지 401 (`share password required`)을 받습니다. 이 호출은 `Cache-Control: private, no-store`와 함께 `{"content": "<project JSON>", "role": "view"}`를 반환합니다. 한 클라이언트 주소에서 5분 안에 10번 잘못된 비밀번호를 입력하면, 올바른 비밀번호여도 해당 프로젝트에 대해 라우트가 429로 응답합니다. 횟수는 서버 프로세스별로 메모리에 보관됩니다.

버전 목록 라우트(`GET /api/projects/{id}/versions`)는 비밀번호를 받지 않으므로, 비밀번호가 설정된 동안에는 프로젝트 관리자를 제외한 모든 사람에게 401로 응답합니다.

GET /api/projects

최근 갱신순으로 한 페이지를 반환합니다:


```
{"projects": [], "limit": 24, "offset": 0, "total": 0}
```

쿼리 매개변수:

- `limit`: 정수 페이지 크기.
- `offset`: 건너뛴 일치 레코드 수(음수 아님).
- `featured=true`: 추천 프로젝트만 반환.
- `mine=true`: 비공개 목록 및 비공개 프로젝트를 포함한 호출자 자신의 프로젝트를 반환. 인증 필요. 유효한 토큰이 없으면 401.
- `shared_with_me=true`: 호출자가 속한 조직의 조직 가시성 프로젝트, 조직 공개 프로젝트, 관리 가능한 비공개/비공개 목록 조직 프로젝트를, 그리고 호출자의 그룹을 명시적으로 대상으로 한 프로젝트를 반환. 인증이 필요하며 `mine=true` 와 함께 쓸 수 없습니다.
- `shared_source=organizations|groups`: `shared_with_me=true` 와 함께 사용하여 페이지네이션과 집계 전에 쿼리를 제한합니다. `organizations` 는 호출자 조직의 공개 및 조직 가시성 프로젝트와, 관리자 또는 활성 생성자로서 관리 가능한 비공개/비공개 목록 프로젝트를 포함합니다. `groups` 는 수락된 그룹 멤버십을 명시적으로 대상으로 한 프로젝트를 포함합니다. `shared_with_me=true` 없이 이 매개변수를 사용하면 422 입니다.

`mine=true` 또는 `shared_with_me=true` 가 설정되지 않으면 공개 프로젝트만 반환됩니다. Authorization 헤더만으로는 공개 목록이 넓어지지 않습니다. 잘못된 페이지네이션이나 두 비공개 목록 모드의 조합은 422 입니다.

GET /api/users/{username}/projects

{username} 이 소유한 {"projects": [...]} 를 최근 갱신순으로 반환합니다. 인증은 선택 사항이며 결과의 범위를 결정합니다. 토큰이 {username} 을 식별하면 목록에 그 사용자의 비공개 목록 및 비공개 프로젝트가 포함됩니다. 인증 여부와 관계없이 그 밖의 모든 호출자는 그 사용자의 공개 프로젝트만 봅니다. 현재 클라이언트는 먼저 GET /api/users/me 로 사용자 이름을 확인한 다음 이 라우트를 호출합니다.

이 라우트는 `limit` (1-100, 기본값 24)과 `offset` (기본값 0)을 받습니다.

따라서 소유자가 아닌 사용자는 403 이 아니라 필터링된 200 을 받습니다. 목록은 거부하는 대신 좁아지므로, 상태 코드를 통해 사용자의 존재를 탐지할 수 없습니다.

GET /api/projects/{id}

호출자에게 보이는 경우 {"project": <project>} 를 반환합니다.

GET /api/projects/{id}/versions

인증과 프로젝트 읽기 권한이 필요합니다. 최신순으로 반환합니다:

```
{"versions": [{"number": 3, "createdAt": "2026-08-03T12:00:00Z", "url": "https://example.org/api/projects/uuid/versions/3"}]}
```


보호된 프로젝트 이력 응답은 `Cache-Control: private, no-store` 를 사용합니다. 기존 `GET /api/projects/{id}/versions/{version}` 라우트는 계속해서 불변 프로젝트 문서 자체를 반환합니다.

`PATCH /api/projects/{id}`

소유권, 또는 조직 소유 프로젝트의 경우 조직 관리자 / 활성 조직 생성자 권한이 필요합니다. 허용되는 필드는 `title`, `description`, `visibility`, `tags`, `organizationId`, `groupIds`, `deleteProtected` 입니다. 응답: `{"project": <project>}.null`을 허용하지 않는 `visibility`, `organizationId`, `deleteProtected` 에 명시적 `null` 을 보내면 커밋 시 실패하는 대신 `422` 로 거부됩니다.

`PUT /api/projects/{id}/content`

소유권, 조직 소유 프로젝트의 경우 조직 관리자 또는 활성 조직 생성자 권한, 또는 대상 공유 갱신 그룹의 멤버십이 필요합니다. 새로운 불변 버전을 만듭니다.

```
{"content": "{\"version\":\"1.0\", ...}", "expectedVersion": 3}
```

`expectedVersion` 은 선택 사항입니다. 제공되었지만 현재 최신 버전과 일치하지 않으면, 쓰기는 마지막 쓰기 우선으로 여전히 성공하고 `201` 응답에는 안정적인 문구 `version conflict` 를 포함하는 `warning` 문자열이 들어갑니다. 버전이 일치하거나 생략되면 `warning` 필드가 없습니다.

응답 `201`: `{"project": <project>, "version": <positive integer>}`.

`DELETE /api/projects/{id}`

소유권이 필요합니다. 메타데이터와 저장된 객체를 삭제합니다. 응답: `204`. 프로젝트의 `deleteProtected` 가 `true` 이면 요청은 `409` 와 `{"error": "project is delete-protected; turn off deleteProtected before deleting it"}` 로 거부됩니다. 문구 `delete-protected` 는 안정적인입니다. 클라이언트는 이 문구를 매칭하여 거부 사유를 설명합니다. `PATCH /api/projects/{id} {"deleteProtected": false}` 로 스위치를 끄면 삭제가 가능해집니다.

프로젝트를 삭제하면 대기 중인 이전과 리디렉션 행도 제거됩니다.

`GET /api/shares`

`read:projects` 가 필요합니다. 호출자가 관리하는 프로젝트 중 `visibility` 가 `public` 또는 `unlisted` 인 것들을 최근 갱신순으로 `{"shares": [<project>, ...]}` 로 반환합니다(조직 가시성 프로젝트는 링크 공유가 아님). 각 항목은 프로젝트 표현에 `projectSlug` 가 추가된 것입니다. 공유의 `id` 는 프로젝트 ID입니다. 만료된 링크도 취소할 수 있도록 계속 나열됩니다.

DELETE /api/shares/{id}

write:projects 와 프로젝트 관리 권한이 필요합니다. 공유를 취소합니다. 프로젝트는 private 이 되고 role , 만료, 비밀번호가 초기화됩니다. 프로젝트, 버전, 그룹 공유는 유지됩니다. 응답: 204 . 호출자가 프로젝트를 관리하지 않으면 403 . 프로젝트를 알 수 없거나, 이미 비공개이거나, 조직 가시성이면 404 .

GET /api/projects/{id}/activity

소유권이 필요합니다. 프로젝트의 활동 로그를 최신순으로, 최대 100개 항목까지 반환합니다:

```
{
  "activity": [
    {
      "id": "...",
      "action": "visibility_change",
      "actorId": "...",
      "details": {
        "before": "private",
        "after": "public",
        "createdAt": "..."
      },
      "id": "...",
      "action": "open",
      "actorId": null,
      "details": {
        "date": "2026-08-21",
        "count": 40,
        "createdAt": "..."
      }
    }
  ]
}
```

동작과 그 details : version_save (version), fork (forked_project_id), visibility_change (before , after), transfer (from , to — "org:<slug>" 또는 사용자 이름), 원본 JSON의 fetch (version), 프로젝트 페이지의 open . actorId 는 행위 계정이며, 익명 방문자는 null 입니다. 익명 open 및 fetch 이벤트는 **방문자별로 절대 저장되지 않습니다**. 동작과 UTC 일자별로 count 를 가진 하나의 행으로 집계되며, IP 주소나 다른 방문자 지문은 기록되지 않습니다. 행은 프로젝트가 다음에 이벤트를 기록할 때

GEOLIBRE_ACTIVITY_RETENTION_DAYS (기본값 90) 이후의 것이 정리됩니다. 로그는 소유자에게만 보이며 목록에는 절대 나타나지 않습니다.

DELETE /api/projects/{id}/activity

소유권이 필요합니다. 프로젝트의 모든 활동 행을 삭제합니다. 응답: 204 .

POST /api/projects/{id}/forks

인증 필요. 보이는 원본의 최신 콘텐츠로부터 호출자가 소유하는 새 프로젝트를 만듭니다. 요청 본문은 **선택 사항**입니다. {"visibility": ...} 는 포크의 가시성을 선택하며, 본문을 완전히 생략하면(흔한 "이 프로젝트 포크" 호출) 422 를 반환하는 대신 {"visibility": "private"} 처럼 동작해야 합니다. {"project": <project>} 와 함께 201 로 응답합니다. 원본의 forkCount 는 원자적으로 증가합니다. public 이나 unlisted 가 아닌 조직 프로젝트의 포크는 그 조직 소유로 남으므로, 호출자의 역할과 조직의 공개 공유 정책이 생성 시와 정확히 같이 적용됩니다.

원본 프로젝트 및 웹사이트 호환 라우트

- GET /{username}/{slug}.geolibre.json 은 최신 프로젝트 문서를 Content-Type: application/json 으로 반환합니다.
- GET /api/projects/{id}/versions/{version} 은 불변의 과거 문서를 반환합니다.
- GET /{username}/{slug} 는 HTML 프로젝트 페이지를 반환하거나 구성된 GeoLibre 뷰어로 리디렉션할 수 있습니다. API가 알리는 projectId 입니다.

- 조직 소유의 대응 라우트는 `GET /org/{organizationSlug}/{slug}.geolibre.json` 과 `GET /org/{organizationSlug}/{slug}` 입니다.

최신 원본 문서를 성공적으로 읽을 때마다 `views` 가 증가할 수 있습니다. 서버는 실패하거나 인가되지 않은 읽기를 세어서는 안 됩니다.

프로젝트가 이전으로 옮겨진 경우, 비워진 주소는 프로젝트의 새 원본 JSON (`.geolibre.json` 라우트) 또는 새 페이지 URL(페이지 라우트)로 `301 Moved Permanently` 를 응답합니다. 리디렉션은 호출자가 이미 대상 프로젝트를 볼 수 있을 때만 따라가므로, 비공개 또는 조직 전용 프로젝트의 새 주소는 다른 사람에게 노출되지 않습니다. 비워진 비공개 주소에 대한 익명 요청은 `301` 이 아니라 `404` 입니다. 비공개 또는 조직 전용 프로젝트로의 인가된 리디렉션은 `Cache-Control: private, no-store` 를 사용하므로, 옛 경로가 로그아웃 후에 이전에 인가된 목적지를 유지할 수 없습니다. 공개 및 비공개 목록 리디렉션은 `Cache-Control: public, no-cache` 를 사용하므로, 이후의 이전이나 가시성 변경이 목적지를 재검증합니다.

이전

프로젝트는 다른 사용자(수락이 필요함)나 호출자가 관리하는 조직(즉시 적용)에 넘길 수 있습니다. 프로젝트는 `id`, `views`, `forkCount`, 버전 이력, 활동을 유지하고, 네임스페이스와 슬러그만 바뀝니다. 모든 이전은 프로젝트의 그룹 공유를 지우고(이전 대상에게 준 권한이었으므로) 비워지는 주소에 대한 영구 리디렉션을 기록합니다.

`POST /api/projects/{id}/transfers` 는 소유권, 또는 조직 소유 프로젝트의 경우 관리자 멤버십과 함께 `write:projects` 가 필요합니다. 관리자가 아닌 활성 조직 생성자는 조직의 자산을 밖으로 이전할 수 없습니다. 본문은 `username` 또는 `organizationId` 중 정확히 하나와 선택적 `slug` 를 받습니다:

```
{"username": "bob", "slug": "wetlands"}
```

- **사용자** 대상은 `pending` 이전을 만듭니다. 그 사용자가 수락할 때까지 아무것도 움직이지 않으므로, 그동안 프로젝트는 현재 소유자와 주소를 유지합니다. `{"transfer": <transfer>, "project": <project>}` 와 함께 `201` 로 응답합니다.
- **조직** 대상은 호출자가 그 조직을 관리해야 하며 즉시 적용됩니다(이전은 `accepted` 로 저장됨). 프로젝트가 이미 옮겨진 상태로 `{"transfer": <transfer>, "project": <project>}` 와 함께 `201` 로 응답합니다.

거부는 클라이언트가 설명할 수 있도록 안정적인 문구를 사용합니다:

상태	error	원인
409	slug already exists for the new owner	대상 네임스페이스가 이미 요청한 슬러그를 사용 중입니다. 다른 slug 로 다시 시도하세요.
409	a transfer is already pending for this project	프로젝트당 대기 중인 이전은 하나입니다.
404	user not found	그 사용자 이름을 가진 계정이 없습니다.
422	provide exactly one of username or organizationId	대상이 둘 다 주어졌거나 하나도 주어지지 않았습니다.

상 태	error	원인
422	project already belongs to that owner	대상이 이미 소유자입니다.

수신 및 관리:

- GET /api/transfers/incoming 은 호출자에게 온 대기 중인 제안을 최신순으로 나열합니다: {"transfers": [...]}.
- GET /api/transfers/outgoing 은 호출자가 시작한 대기 중인 이전을 나열합니다: {"transfers": [...]}.
- POST /api/transfers/{id}/accept (수신자 전용)은 프로젝트를 옮깁니다. 본문은 선택 사항입니다. {"slug": "..."} 는 대상 슬러그를 재정의하며, 수신자가 슬러그 충돌 (409 slug already exists for the new owner)을 해결하는 방법입니다. {"project": <project>, "transfer": <transfer>} 와 함께 200 으로 응답합니다. 시작자가 더 이상 프로젝트를 관리하지 않거나 다른 요청이 제안을 먼저 해결했다면, 응답은 409 transfer is no longer valid 입니다.
- POST /api/transfers/{id}/decline (수신자 전용)은 204 로 응답합니다.
- DELETE /api/transfers/{id} 는 대기 중인 이전을 취소하고(시작자, 또는 여전히 프로젝트를 관리할 수 있는 누구나) 204 로 응답합니다.

이전은 가시성을 바꿀 수 있습니다. public 이었던 프로젝트가 publicSharingPolicy 가 yes 가 아닌 조직으로 옮겨지면 organization 이 되고, organization 이었던 프로젝트가 개인에게 옮겨지면 private 이 됩니다. 두 변경 모두 visibility_change 활동으로 기록됩니다.

비워진 <username>/<slug> (또는 /org/<slug>/<slug>) 주소는 리디렉션이 존재하는 동안 **예약된** 상태로 유지됩니다. 나중에 그 네임스페이스에 같은 제목을 업로드하면 옛 링크를 차지하는 대신 -2 접미사를 받습니다.

두 목록 라우트 모두 Cache-Control: private, no-store 를 사용합니다.

이전은 다음과 같이 반환됩니다:

```
{
  "id": "uuid",
  "projectId": "uuid",
  "projectTitle": "Wetlands",
  "projectSlug": "wetlands",
  "fromUsername": "ada",
  "toUsername": "bob",
  "toOrganization": null,
  "slug": "wetlands",
  "status": "pending",
  "createdAt": "2026-09-30T12:00:00Z",
  "resolvedAt": null
}
```

조직 이전에서는 toUsername 이 null 이고, 대신 toOrganization 이 {"id", "slug", "name"} 입니다.

status 는 pending, accepted, declined, 또는 cancelled 입니다. projectSlug 는 응답 시점의 프로젝트 슬러그이고, slug 는 목적지에서 갖게 될 슬러그입니다.

썸네일

PUT /api/projects/{id}/thumbnail 은 소유권이 필요하며 이미지 콘텐츠 타입과 함께 이미지 바이트를 받습니다. GET /api/projects/{id}/thumbnail 은 프로젝트 가시성을 따릅니다. DELETE 는 썸네일을 제거합니다. 업로드와 삭제 응답은 204 입니다.

개인 토큰 스코프

스코프	부여 권한
read:projects	비공개 목록/비공개 프로젝트를 포함한 호출자 자신의 프로젝트 나열 및 열기. 조직/그룹 멤버십, 갤러리, 호출자에게 공유된 프로젝트 읽기
write:projects	호출자가 관리하거나 공유 갱신할 수 있는 프로젝트의 생성, 갱신, 삭제, 포크. 조직, 그룹, 멤버십, 초대 생성 및 관리. 계정 이메일 변경
share:public	공개 프로젝트 생성 또는 프로젝트 가시성을 공개로 상향

새 개인 토큰에는 이 스코프들의 비어 있지 않은 부분 집합이 필요합니다. scopes 를 생략하면 기존 클라이언트를 위해 과거의 프로젝트 권한이 유지됩니다. expiresInDays 를 생략하면 토큰은 취소될 때까지 유효합니다(v1 삭제 전용 수명 주기). 만료되는 토큰을 발급하려면 expiresInDays 를 1-365로 설정하세요. 정책 테이블 이전의 토큰은 첫 사용 시 세 가지 프로젝트 스코프 모두, 만료 없음, 레거시 표시로 업그레이드됩니다.

필요한 스코프가 없는 유효한 자격 증명은 {"error": "insufficient_scope", "requiredScope": "<scope>"} 와 WWW-Authenticate: Bearer error="insufficient_scope" 와 함께 403 을 받습니다. 자격 증명 없이 Bearer 챌린지를 사용하고, 형식이 잘못되었거나, 알 수 없거나, 취소되었거나, 만료된 자격 증명은 Bearer error="invalid_token" 을 사용합니다.

OAuth 2.0 로그인(Authorization Code + S256 PKCE)

참조 서버는 공개 클라이언트를 위한 PKCE(S256 만) 기반 Authorization Code를 구현합니다. 클라이언트 비밀은 받지 않습니다. 등록은 정확해야 하며 GEOLIBRE_OAUTH_CLIENTS 를 통해 시작 시 검증됩니다. 지원되는 클라이언트 ID는 geolibre-web 과 geolibre-desktop 뿐입니다. 구성이 비어 있거나 설정되지 않으면 개인 토큰의 시작 동작은 바꾸지 않은 채 모든 OAuth 라우트가 비활성화됩니다.

발급자는 GEOLIBRE_PUBLIC_URL 입니다. OAuth가 활성화되면 절대 HTTPS URL이어야 합니다. 루프백 HTTP는 명시적 포트가 있는 localhost 또는 127.0.0.1 에 대해서만 허용됩니다. 포트를 포함한 요청 Host 헤더는 발급자 권한(authority)과 일치해야 합니다.

디스커버리

GET /.well-known/oauth-authorization-server 는 RFC 8414 메타데이터를 반환합니다. 경로가 /services/projects 인 발급자의 경우 라우트는 /.well-known/oauth-authorization-server/services/projects 입니다. 이 문서는 인가, 토큰, 취소 엔드포인트, 인가 코드 및 리프레시 권한 부여, S256, 세 가지 프로젝트 스코프, 그리고 manage:sessions (OAuth 전용)를 알립니다.

인가 및 동의

GET /oauth/authorize 는 response_type=code, client_id, 정확한 redirect_uri, 비어 있지 않은 scope, state, code_challenge, code_challenge_method=S256 을 각각 하나씩 받습니다. device_label 은 선택 사항입니다. State는 16-512자의 URL 안전 문자입니다. S256 챌린지는 패딩 없는 43자 base64url SHA-256 값입니다.

중복된 인가 매개변수, 알 수 없는 클라이언트, 등록되지 않은 리디렉션, 512자를 넘는 state 값은 Location 헤더 없이 로컬 오류 페이지를 반환합니다. 그 밖의 인가 오류는 이미 검증된 콜백으로 error, iss, 그리고 제공된 경우 정확한 state 값과 함께 리디렉션합니다.

POST /oauth/authorize 는 서버 소유 동의 폼을 제출합니다. 브라우저 바인딩 쿠키, CSRF 값, 동일 출처 Origin 또는 Referer, 계정 자격 증명이 필요합니다. 승인하면 일회용 코드, state, iss 와 함께 정확한 콜백으로 303 을 반환하고, 취소하면 access_denied 를 반환합니다. 인가 코드는 기본적으로 60초 후 만료됩니다 (GEOLIBRE_OAUTH_CODE_TTL_SECONDS). 프로덕션 HTTPS는 호스트 전용 Secure 브라우저 바인딩 쿠키를 사용하고, 허용된 루프백 HTTP 개발 환경은 Safari가 동의 폼을 제출할 수 있도록 호스트 전용 비 Secure 쿠키를 사용합니다.

웹 리디렉션은 /oauth-callback.html 로 끝나는 절대 HTTPS URL이어야 합니다. 개발용으로 명시적 포트의 루프백 HTTP가 허용됩니다. 데스크톱 리디렉션은 정확히 org.geolibre.desktop:/oauth/callback 이어야 합니다. 설치된 Tauri 데스크톱 앱은 시스템 브라우저에서 동의를 열고, 인바운드 HTTP 리스너가 아니라 OS 프로토콜 핸들러(macOS, Windows, Linux)를 통해 그 URI를 받습니다. 살아 있고 일치하는 state와 발급자에 대한 콜백만 받아들입니다. 대기 중인 검증자 없이 앱을 콜드 스타트하는 콜백은 로그인을 완료할 수 없습니다. 사용자는 동의를 다시 시작해야 합니다. 인가 코드나 토큰은 진단 로그나 저장된 프로젝트에 들어가서는 안 됩니다.

토큰 교환 및 교체

POST /oauth/token 은 최대 16 KiB의 form-urlencoded 본문을 받습니다:

- grant_type=authorization_code 는 client_id, code, redirect_uri, 그리고 43-128자의 code_verifier 가 필요합니다.
- grant_type=refresh_token 은 client_id 와 refresh_token 이 필요합니다. 선택적 scope 는 원래 권한 부여와 같은 스코프 집합이어야 하며, 순서는 상관없습니다.

성공 시 반환:

```
{
  "access_token": "opaque",
  "token_type": "Bearer",
  "expires_in": 600,
  "refresh_token": "opaque",
  "scope": "read:projects write:projects"
}
```

새로운 단계 상승(step-up) 동의를 위해서는 `manage:sessions` 를 **단독으로** 요청하세요. 프로젝트 스코프와 결합하면 `invalid_scope` 입니다. 성공 응답에는 `"scope": "manage:sessions"` 와 `"expires_in": 300` (서버가 더 짧은 액세스 수명을 강제하면 그 이하)이 있지만, `refresh_token` 은 **없습니다**. 서버는 이 권한 부여에 대해 리프레시 행을 절대 만들지 않고, 리프레시 시도를 거부하며, 액세스 토큰과 패밀리를 5분으로 제한합니다. 클라이언트는 관리 토큰을 메모리에만 보관해야 하며, 세션 관리가 닫히거나, 프로젝트 세션이 바뀌거나, 권한 부여가 만료되면 폐기해야 합니다. 관리 요청에 대한 `401` 은 새 동의를 요구하며, 프로젝트 세션을 로그아웃시켜서는 안 됩니다.

액세스 토큰은 기본적으로 600초 후 만료되며(`GEOLIBRE_OAUTH_ACCESS_TTL_SECONDS`) 패밀리보다 오래 살아남지 않습니다. 리프레시 토큰은 일회용이며 사용할 때마다 교체됩니다. 사용된 리프레시 토큰을 재사용하면 성공한 교체로 발급된 토큰을 포함해 패밀리 전체가 취소됩니다. 패밀리는 발급 시점에 구성된 리프레시 TTL (`GEOLIBRE_OAUTH_REFRESH_TTL_SECONDS` , 기본값 30일)을 더한 시점에 만료되며, 교체는 이를 절대 연장하지 않습니다.

활성화된 서버는 시작 시, OAuth 요청 중, 그리고 실행 중 5분마다 만료된 상호작용, 액세스 토큰, 패밀리를 제한된 배치로 삭제합니다. 사용된 리프레시 세대는 재생 감지가 계속 유효하도록 패밀리가 만료될 때까지 남습니다.

`POST /oauth/revoke` 는 `client_id` , `token` , 그리고 선택적 참고용 `token_type_hint` 를 받습니다. 일치하는 액세스 또는 리프레시 토큰은 패밀리 전체를 취소합니다. 알 수 없는 토큰, 이미 취소된 토큰, 잘못된 클라이언트의 토큰은 모두 같은 빈 `200` 을 반환합니다.

OAuth 실패는 `invalid_request` , `invalid_client` , `invalid_grant` , `invalid_scope` , `unsupported_grant_type` , 또는 `unsupported_token_type` 을 사용합니다. 토큰 및 취소 응답은 `no-store` 입니다. 원시 코드와 토큰은 한 번만 반환되며, 데이터베이스는 SHA-256 다이제스트만 저장합니다.

프로젝트 OAuth 액세스 토큰은 개인 토큰과 같은 프로젝트 스코프 매트릭스를 사용합니다. `manage:sessions` 는 위에 문서화된 세션 관리 라우트만 인가합니다. OAuth 동의 전용이며 개인 토큰에서는 절대 사용할 수 없고, 프로젝트 읽기 또는 쓰기 권한을 부여하지 않습니다. `admin:org` 는 예약된 상태로 남아 있으며 거부됩니다.

호환성

API는 버전 1 안에서 추가만 이루어집니다. 구현은 필드의 용도를 바꾸거나 가시성 규칙을 좁혀서는 안 됩니다. 새로운 선택적 필드와 엔드포인트는 추가될 수 있습니다. 호환성을 깨는 변경에는 새 `/api/v2` 네임스페이스가 필요합니다. 적합성 기준선은 `share-geolibre.ts` 와 `share-gallery.ts` 에 대한 프론트엔드 테스트와 참조 서버의 API 테스트입니다.

<https://geolibre.app/android/>

Android

GeoLibre는 **Tauri v2 mobile**을 통해 동일한 React 코드베이스에서 빌드된 네이티브 Android 앱으로 실행됩니다. 별도의 앱이 아닙니다. 웹뷰 UI가 APK에 번들로 포함되어 있으므로 앱 셀은 오프라인에서도 동작하며, 지도 타일과 무거운 엔진은 필요할 때 가져옵니다(데스크톱 빌드와 동일).

설치

GeoLibre는 [Google Play](#)에 게시되어 있습니다. Play 빌드는 서명되어 있으며 자동으로 업데이트됩니다.

Google Play에서 GeoLibre 받기

사이드로드를 선호하시나요? 모든 [릴리스](#)에는 서명된 ABI별 APK가 첨부되어 있습니다. [다운로드](#)를 참조하세요. 이 페이지의 나머지 부분은 앱을 직접 빌드하는 개발자를 위한 것입니다.

Android와 데스크톱에서 동작하는 기능 비교

Android 빌드에는 전체 지도 워크스페이스, Add Data(데이터 추가), Whitebox 지오프로세싱 툴박스(1,000개 이상의 도구, 브라우저에서 WebAssembly로 실행), Vector 도구(Turf.js / Pyodide를 통한 브라우저 내 GeoPandas), SQL Workspace (DuckDB-WASM 및 브라우저 내 PGlite/PostGIS 엔진), Python Console (Pyodide), 지오크딩, 통계, AI 어시스턴트, 스토리 맵, 플러그인이 포함됩니다.

로컬 데스크톱 프로세스에 의존하는 도구는 모바일에서 숨겨집니다. Android에는 Python 사이드카나 로컬 헬퍼 바이너리가 없기 때문입니다.

- Processing → GeoLibre Toolbox → **Conversion**(변환), **AI Segmentation**(AI 분할), 그리고 사이드카 기반 **Raster**(래스터) 도구(모두 Python 사이드카가 필요합니다. Raster 하위 메뉴에는 클라이언트 측 **Download Global DEM**(전역 DEM 다운로드) 도구만 남습니다)
- Add Data → **PostgreSQL**(로컬 Martin 타일 서버가 제공) 및 **SQL Server**

이들은 사용자 에이전트 `isMobile()` 검사로 게이트되므로, Processing 메뉴, Add Data 메뉴, Layer 패널의 데이터 추가 그룹에서는 해당 항목을 제공하지 않습니다. 그 밖의 모든 것은 클라이언트 측에서 실행됩니다.

데이터베이스 소스에는 이 검사가 다루지 않는 진입점이 하나 있습니다. Browser 패널은 탐색을 위해 모든 플랫폼에서 **Databases**(데이터베이스) 섹션을 유지하므로, 그 + 버튼은 여전히 연결 대화 상자를 엽니다. 대화 상자는 `isTauri()` 단독이 아니라 `isDesktopRuntime()` (`isTauri() && !isMobile()`)으로 게이트되므로, Android에서는 존재할 수 없는 헬퍼를 호출하는 대신 "requires GeoLibre Desktop" 안내를 표시하고 Connect를 비활성화합니다 (GeoLibre#2091).

툴체인 설정(최초 한 번)

Android SDK + NDK, JDK(17 또는 21. 더 새로운 JDK는 Android Gradle Plugin을 망가뜨릴 수 있습니다), 그리고 Rust Android 타킷이 필요합니다. 가장 깔끔하고 sudo가 필요 없는 구성은 모든 것을 `~/Android/Sdk` 의 사용자 쓰기 가능 SDK 아래에 두는 것입니다.

```
# 1. JDK 17/21 (or reuse Android Studio's bundled JBR at /opt/android-studio/jbr)
export JAVA_HOME=/path/to/jdk-21

# 2. Android SDK components (sdkmanager ships with Android Studio cmdline-tools)
export ANDROID_HOME="$HOME/Android/Sdk"
yes | sdkmanager --sdk_root="$ANDROID_HOME" --licenses
sdkmanager --sdk_root="$ANDROID_HOME" \
  "platform-tools" "platforms;android-36" \
  "build-tools;36.0.0" "ndk;27.3.13750724"
export NDK_HOME="$ANDROID_HOME/ndk/27.3.13750724" # Tauri needs NDK_HOME

# 3. Rust + the four Android targets (install rustup if you don't have it)
rustup target add aarch64-linux-android armv7-linux-androideabi \
  i686-linux-android x86_64-linux-android
```

NDK **r27(LTS)**이 Tauri v2에서 지원되는 라인입니다. 네 개의 `export` 를 셸 프로파일에 추가하여 모든 세션에서 사용할 수 있게 하세요.

API **36**(Android 16)이며, 34가 아닙니다. Tauri v2.11 Android 템플릿은 `compileSdk = 36 / targetSdk = 36` 을 생성하므로, Gradle에는 일치하는 플랫폼이 설치되어 있어야 합니다. 이는 Google Play의 하한선이기도 합니다. **2026-08-31**부터 새 앱과 업데이트는 API 36을 대상으로 해야 승인됩니다.

16 KB 페이지 크기

Google Play는 네이티브 라이브러리가 16 KB 메모리 페이지에 맞춰 정렬되지 않은 Android 15+ 대상 앱을 거부하며, 그런 라이브러리는 16 KB 기기에서 로드에서 실패합니다. NDK r28+는 기본적으로 이를 처리하지만 **r27은 그렇지 않으므로**, 플래그를 명시적으로 전달합니다. Android 빌드 전에 다음을 export하세요(CI는 `.github/workflows/android.yml` 의 워크플로 수준에서 설정합니다).

```
export RUSTFLAGS="-C link-arg=-Wl,-z,max-page-size=16384 -C link-arg=-Wl,-z,common-page-size=16384"
```

반드시 `RUSTFLAGS` **환경 변수**여야 합니다. 같은 플래그를 `.cargo/config.toml` 의 `target.<triple>.rustflags` 에 넣는 것은 동작하지 않습니다. Tauri CLI는 Android용으로 cargo를 호출할 때 `RUSTFLAGS` 를 직접 설정하며, 환경 변수 `RUSTFLAGS` 는 설정 파일을 완전히 덮어씁니다. 설정 파일 형식은 조용히 무시됩니다. 파싱되고, 빌드되고, Play가 거부하는 4 KB 정렬 라이브러리를 출시하게 됩니다. Tauri는 상속된 값에 덧붙이므로, export하면 동작합니다.

빌드된 APK, 즉 Play가 실제로 받는 바이트에서 결과를 확인하세요.

```
unzip -o -q app-arm64-release-unsigned.apk 'lib/**/*.so' -d /tmp/apkcheck
"$NDK_HOME/toolchains/llvm/prebuilt/linux-x86_64/bin/llvm-objdump" -p \
```

```
/tmp/apkcheck/lib/**/*.so | awk '$1 == "LOAD" { print $NF }' | sort -u
# every value must be 2**14 or greater; 2**12 means the flags did not apply
```

`readelf -l` 대신 `llvm-objdump -p` 를 사용하세요. `readelf`는 각 LOAD를 두 줄에 걸쳐 출력하므로, 잘못된 열을 파싱해서 주소를 정렬 값으로 읽기 쉽습니다. CI는 패키징된 모든 `.so`에 대해 이와 동일한 검사를 실행하며 회귀가 있으면 실패합니다.

빌드

```
cd apps/geolibre-desktop
npx tauri android init
npx tauri android build --apk --split-per-abi
npx tauri android build --aab
```

generate src-tauri/gen/android (once)
release APKs, one per ABI (~40 MB each)
universal AAB for Google Play

- `gen/android`는 생성되는 디렉터리(git에서 무시)이며 필요에 따라 재생성됩니다.
- `--debug`가 아니라 **릴리스**로 빌드하세요. 스트립되고 크기 최적화된 Cargo 프로파일은 각 APK를 약 40 MB로 만듭니다. 디버그 빌드는 약 200 MB입니다(debuginfo가 포함된 스트립되지 않은 `.so`).
- `--split-per-abi`는 약 150 MB의 단일 유니버설 APK 대신 아키텍처별로 APK를 하나씩 생성합니다. 실제 휴대폰에는 `arm64` APK를 설치하세요.
- 출력: `src-tauri/gen/android/app/build/outputs/apk/<abi>/release/app-<abi>-release-unsigned.apk`. 여기서 `<abi>`는 Tauri의 짧은 이름(`arm64`, `arm`, `x86`, `x86_64`)이며, APK 내부 `lib/` 아래에 나타나는 Android ABI 디렉터리 이름(`arm64-v8a`, `armeabi-v7a`)이 아닙니다.
- 사이드로드/GitHub 릴리스 경로: ABI별 **APK**.
- Google Play 경로: 유니버설 **AAB**(Play가 이로부터 기기별 분할을 생성합니다). AAB에는 `--split-per-abi`를 쓰지 마세요. Play는 하나의 번들을 원합니다.

Android에서 앱 이름은 **GeoLibre**이며(데스크톱 빌드는 "GeoLibre Desktop"), 패키지 ID는 `org.geolibre.app`입니다. 둘 다 `src-tauri/tauri.android.conf.json`에서 설정하며, 이 파일은 Android 번들에서 Python 백엔드도 제외합니다.

Android ID를 `tauri.conf.json`이 아니라 그곳에서 재정의하는 것은 의도적입니다. `identifier`는 모든 플랫폼이 공유하며 macOS 번들 ID, Linux AppStream ID, 웹뷰 데이터 디렉터리도 결정합니다. 이를 전역적으로 바꾸면 기존 데스크톱 사용자의 설정이 고아가 되고, 여전히 `org.geolibre.desktop`을 키로 사용하는 Linux/COPR/Homebrew 패키징이 망가집니다.

서명

릴리스 APK는 서명되어 있지 않습니다. 설치하려면 서명하세요(테스트에는 디버그 키로 충분하고, 배포에는 실제 키를 사용하세요).

```
BT="$ANDROID_HOME/build-tools/36.0.0"
KS="$HOME/.android/debug.keystore" # auto-created by Android tooling; or make your own
# -P 16, not -p: -p only guarantees 4 KB, and Play requires the .so to sit on a
# 16 KB boundary inside the zip. Same flag CI uses.
"$BT/zipalign" -P 16 -f 4 app-arm64-release-unsigned.apk aligned.apk
"$BT/apksigner" sign --ks "$KS" --ks-pass pass:android \
  --ks-key-alias androiddebugkey --key-pass pass:android \
  --out geolibre-arm64.apk aligned.apk
"$BT/apksigner" verify geolibre-arm64.apk
```

이 예제는 arm64 APK를 서명합니다. 필요한 ABI로 전체를 바꿔 쓰세요. `app-x86_64-release-unsigned.apk` → `geolibre-x86_64.apk`, 그리고 `armeabi-v7a` / `x86` 도 마찬가지입니다. 아래의 에뮬레이터 섹션은 `geolibre-x86_64.apk` 를 설치하는데, 이는 같은 절차에서 `arm64` 를 `x86_64` 로 바꾼 것입니다.

실제 업로드/릴리스 키의 경우:

```
keytool -genkeypair -v -keystore upload.jks -alias upload -keyalg RSA \
  -keysize 2048 -validity 10000
```

지속적 통합

`.github/workflows/android.yml` 은 GitHub 릴리스가 게시될 때마다(그리고 "Run workflow" 버튼으로 수동 실행 시) **서명된** ABI별 릴리스 APK를 빌드하여 `geolibre-android-release-apks` 아티팩트로 업로드합니다. 게시된 릴리스에서는 APK가 다운로드 가능한 자산으로 **해당 릴리스에도 첨부됩니다**. 단, 실제 릴리스 서명을 갖춘 경우에만 그렇습니다. 디버그 서명된 APK는 CI 아티팩트에만 남고 실행 로그에 그 사실을 알리는 경고가 기록됩니다. 다음 저장소 시크릿이 설정되어 있으면 릴리스 키스토어로 서명하고, 그렇지 않으면 임시 디버그 키로 대체하여 아티팩트를 여전히 테스트용으로 설치할 수 있게 합니다.

- `ANDROID_KEYSTORE_BASE64` — `base64 -w0 upload.jks`
- `ANDROID_KEYSTORE_PASSWORD`
- `ANDROID_KEY_ALIAS`
- `ANDROID_KEY_PASSWORD`

또한 유니버설 **AAB**를 빌드하여 별도의 `geolibre-android-play-aab` 아티팩트로 업로드합니다. 단, 실제 릴리스 키스토어가 있는 실행에서만 그렇습니다. Play는 디버그 서명된 번들을 거부하기 때문입니다. 키스토어가 없으면 AAB 빌드는 빌드 후 폐기되는 대신 완전히 건너뛴습니다. 게시된 릴리스에서는 `geolibre-android.aab` 가 APK와 함께 릴리스에 첨부되므로, 태그에 대해 제출된 정확한 번들이 CI 아티팩트 만료 후에도 복구 가능하게 남습니다.

사이드로드하시나요? `.apk` 를 받으세요. `.aab` 는 Google Play 업로드 형식이며 기기에 설치할 수 없습니다.

설치 / 테스트

휴대폰에서

1. 개발자 옵션(빌드 번호를 7번 탭)과 **USB 디버깅**을 활성화합니다.
2. 서명된 APK를 사이드로드합니다.

```
adb install -r geolibre-arm64.apk
```

또는 APK를 휴대폰에 복사한 뒤 탭하세요("알 수 없는 앱 설치" 허용).

핫 리로드를 사용하는 실시간 개발의 경우, 기기를 연결하고 `npm run tauri android dev` 를 실행하세요.

에뮬레이터에서

```
sdkmanager --sdk_root="$ANDROID_HOME" \
  "emulator" "system-images;android-36;google_apis_playstore;x86_64"
avdmanager create avd -n geolibre \
  -k "system-images;android-36;google_apis_playstore;x86_64" -d pixel_7
emulator -avd geolibre
# x86_64 APK to match the x86_64 system image - the emulator can translate the
# arm64 build, but far slower, and it would not exercise the x86_64 libraries.
adb install -r geolibre-x86_64.apk
```

다른 서명 키로 다시 빌드하는 경우, 먼저 이전 사본을 제거하세요 (`adb uninstall org.geolibre.app`). Android는 서명이 바뀐 업데이트를 거부합니다. 이는 사이드로드된 APK와 Play 빌드 사이를 오갈 때도 적용됩니다. Play App Signing은 Google의 키로 다시 서명하므로, 둘은 업그레이드 호환이 되지 않습니다.

Google Play에 게시

GeoLibre는 Play에 [org.geolibre.app](https://play.google.com/store/apps/details?id=org.geolibre.app) 으로 라이브 상태입니다. 이 섹션은 참고용으로, 그리고 포크를 게시하는 분들을 위해 온보딩 과정을 기록합니다. 릴리스마다 반복되는 것은 3단계뿐입니다. 업로드 키로 AAB를 빌드하고 `versionCode` 를 올립니다. 빌드 측면은 위의 CI 워크플로가 담당하고, 나머지는 일회성 Play Console 온보딩입니다.

1. **개발자 계정**(\$25, 일회성). 가능하면 개인 계정이 아니라 **조직**으로 등록하세요. 2023-11-13 이후에 생성된 개인 계정은 프로덕션 액세스를 신청하기 전에 **14일 연속으로 옵트인한 테스터 12명**과 비공개 테스트를 진행해야 합니다. 조직 계정은 면제됩니다.
2. **Play App Signing**. `upload.jks` 를 업로드 키로 업로드합니다. Google이 실제 앱 서명 키를 보유하고 각 번들을 다시 서명합니다. 저장소의 `ANDROID_KEYSTORE_*` 시크릿이 그 업로드 키입니다. 키스토어를 잃어버리면 Play 지원팀의 재설정이 필요하므로 백업을 유지하세요.
3. `geolibre-android-play-aab` CI 아티팩트에서, 또는 그 아티팩트가 완료된 후에는 릴리스의 `geolibre-android.aab` 자산에서 **AAB를 업로드**합니다. `versionCode` 는 `tauri.conf.json` 의 버전에서 파생되며 업로드할 때마다 증가해야 합니다.

4. **Store 목록 자산:** 512×512 아이콘, **1024×500 그래픽 이미지**, 그리고 최소 두 장의 휴대폰 스크린샷. 7인치와 10인치 태블릿 스크린샷도 추가하세요. Play는 이것이 없는 앱의 순위를 낮추며, GIS 워크스페이스는 태블릿에 정말 잘 어울립니다.
5. **개인정보 처리방침 URL** — 게시된 **개인정보 처리방침**을 가리키게 하세요.
6. **데이터 보안 양식.** 각 네트워크 목적지를 정직하게 선언하세요. 지오코딩, AI 어시스턴트, 베이스맵/타일 가져오기, Earth Engine용 Google OAuth입니다. 어떤 것이 전송되고 어떤 것이 수집되는지 구분하세요. GeoLibre는 사용자 데이터를 보관하는 백엔드를 운영하지 않지만, 양식은 목적별로 묻습니다.
7. **콘텐츠 등급** 설문과 대상 사용자.

업데이트마다 아래의 알려진 제한 사항을 염두에 두세요. 여러 Add Data 경로가 Android에서 여전히 동작하지 않습니다. 사용자가 탭했는데 아무 일도 일어나지 않으면 별 하나짜리 리뷰로 이어지므로, 사이드카 도구가 이미 `isMobile()` 로 게이트되어 있는 것처럼 모바일에서 게이트하세요.

알려진 제한 사항 / 후속 작업

- 로컬 파일 소스(MBTiles, 로컬 래스터, 프로젝트 파일)는 실제 파일시스템 경로를 가정합니다. Android 범위 지정 저장소는 content URI를 반환하므로, 해당 흐름이 네이티브로 동작하려면 조정이 필요합니다.
- **Download Offline Area**(오프라인 영역 다운로드) 도구는 서비스 워커에 의존하는데, Tauri 빌드(데스크톱 및 Android)는 이를 사용하지 않습니다. 이는 PWA 기능입니다. 네이티브 오프라인 베이스맵 캐싱(번들/다운로드된 MBTiles/PMTiles)은 향후 개선 사항입니다.
- Earth Engine OAuth는 데스크톱 루프백/다중 창 흐름을 사용합니다. 모바일 딥링크 리디렉션은 향후 작업입니다.

다른 앱에서 위치 열기

GeoLibre는 앱을 시작할 때와 이미 열려 있는 동안 모두 `geo:` 스킴의 Android `ACTION_VIEW` 인텐트를 처리합니다. 지원되는 형식은 다음과 같습니다.

- `geo:40.7128,-74.006`
- `geo:40.7128,-74.006?z=12`
- `geo:0,0?q=40.7128,-74.006(New%20York)&z=12`

좌표는 위도, 경도 순서를 사용합니다. 숫자 `q` 좌표는 URI의 자리 표시자 좌표를 덮어씁니다. 주소만 있는 쿼리는 지원되지 않습니다. 직접 URI의 선택적 세 번째 좌표는 미터 단위 고도이며, 검증은 되지만 지도 위치 지정 시에는 무시됩니다. 좁은 기본값 14이며 0에서 24 사이여야 합니다. 유효하지 않은 위치는 무시됩니다. 수신된 위치는 현재 프로젝트의 레이어를 교체하지 않고 지도만 이동합니다.

Tauri 딥링크 플러그인은 `tauri.android.conf.json` 에서 Android 인텐트 필터를 생성하므로, CI가 `gen/android` 를 재생성할 때도 적용됩니다. 연결된 기기에서 콜드 시작과 실행 중인 앱을 모두 확인하려면, 호출 사이에 지도를 이동하면서 이 명령을 두 번 실행하세요.

```
adb shell am start -a android.intent.action.VIEW -d 'geo:0,0?q=40.7128,-74.006&z=12'
org.geolibre.app
```

웹 링크는 <https://geolibre.app/?lat=40.7128&lon=-74.006&zoom=12> 또는 축약형 <https://geolibre.app/?12/40.7128/-74.006> 으로 같은 위치를 열 수 있습니다. 좌표만 있는 실행은 저장된 시작 설정보다 우선하며 온보딩을 건너뜁니다. 명시적인 프로젝트 또는 데이터 링크는 좌표 매개변수와 결합되어도 기존의 카메라/로딩 동작을 유지합니다.

<https://geolibre.app/ios/>

iOS

GeoLibre는 **Tauri v2 mobile**을 통해 동일한 React 코드베이스에서 빌드된 네이티브 iOS 앱으로 실행됩니다. 별도의 앱이 아닙니다. 웹뷰 UI(WKWebView)가 앱에 번들로 포함되어 있으므로 셀은 오프라인에서도 동작하며, 지도 타일과 무거운 엔진은 필요할 때 가져옵니다 (데스크톱 및 Android 빌드와 동일).

설치

GeoLibre는 iPhone과 iPad용으로 [App Store](#)에 게시되어 있습니다. Store 빌드는 서명되어 있으며 자동으로 업데이트됩니다.

App Store에서 GeoLibre 받기

GeoLibre는 최종 사용자를 위한 사이드로드용 `.ipa`를 게시하지 않습니다. App Store 목록과 베타 빌드용 TestFlight가 배포 경로입니다. 각 릴리스에는 `GeoLibre_<version>_ios_app-store.ipa` 자산이 포함되어 있지만, Android APK와 달리 이것은 App Store **제출** 패키지로, 제출을 재현 가능하게 유지하기 위해 태그 옆에 보관되는 것이며 기기에 설치할 수 없습니다. 이 페이지의 나머지 부분은 앱을 직접 빌드하는 개발자를 위한 것으로, 빌드는 반드시 Mac에서 수행해야 합니다(iOS는 Linux에서 크로스 컴파일할 수 없습니다).

해당 목록 뒤에 있는 서명 및 아카이브 파이프라인은 두 가지 방식으로 실행되며, 어느 쪽이든 제출 가능한 `.ipa`를 생성합니다. 번들 ID `org.geolibre.app`, Apple Distribution으로 서명, `get-task-allow=false`, 포함된 App Store 프로비저닝 프로파일, 병합된 위치 사용 설명 문자열, 그리고 바이너리에 포함된 웹 자산을 갖추고 있습니다.

- 로컬의 Mac에서(Xcode 26.6 / iOS SDK 26.5): `tauri ios init` → 서명되지 않은 아카이브 → `xcodebuild -exportArchive`.
- CI의 `macos-15` 러너에서(Xcode 26.3): `APPLE_IOS_*` 시크릿에서 인증서와 프로파일을 가져오고, 아카이브하고, 수동 서명으로 내보내고, 서명을 검증한 뒤 `geolibre-ios-ipa` 아티팩트를 게시합니다.

iOS와 데스크톱에서 동작하는 기능 비교

Android와 동일하게 나뉩니다. iOS 빌드에는 전체 지도 워크스페이스, Add Data(데이터 추가), Vector 도구 (Turf.js / Pyodide를 통한 브라우저 내 GeoPandas), SQL Workspace (DuckDB-WASM 및 PGlite/PostGIS), Python Console(Pyodide), 지오코딩, 통계, AI 어시스턴트, 스토리 맵, 플러그인이 포함됩니다.

로컬 데스크톱 프로세스에 의존하는 도구는 모바일에서 숨겨집니다. iOS에는 Python 사이드카나 로컬 헬퍼 바이너리가 없고, 샌드박스가 하위 프로세스 생성을 금지하기 때문입니다.

- Processing → GeoLibre Toolbox → **Conversion**(변환), **AI Segmentation**(AI 분할), 그리고 사이드카 기반 **Raster**(래스터) 도구(모두 Python 사이드카가 필요합니다. Raster 하위 메뉴에는 클라이언트 측 **Download Global DEM**(전역 DEM 다운로드) 도구만 남습니다). Whitebox 지오프로세싱 툴박스는 WebAssembly로 실행되므로 사이드카가 필요 없으며 계속 사용할 수 있습니다.
- Add Data → **PostgreSQL**(로컬 Martin 타일 서버가 제공) 및 **SQL Server**

이들은 사용자 에이전트 `isMobile()` 검사(이미 iPhone/iPad와 일치)로 게이트되므로, Processing 메뉴, Add Data 메뉴, Layer 패널의 데이터 추가 그룹에서는 해당 항목을 제공하지 않습니다. 그 밖의 모든 것은 클라이언트 측에서 실행됩니다.

데이터베이스 소스에는 이 검사가 다루지 않는 진입점이 하나 있습니다. Browser 패널은 탐색을 위해 모든 플랫폼에서 **Databases**(데이터베이스) 섹션을 유지하므로, 그 + 버튼은 여전히 연결 대화 상자를 엽니다. 대화 상자는 `isTauri()` 단독이 아니라 `isDesktopRuntime() ( isTauri() && !isMobile() )`으로 게이트되므로, iOS에서는 존재할 수 없는 헬퍼를 호출하는 대신 "requires GeoLibre Desktop" 안내를 표시하고 Connect를 비활성화합니다 (GeoLibre#2091).

Android와 달리, App Store 빌드는 **Earth Engine**도 숨깁니다. 패키징된 앱에서 로그인하려면 루프백 OAuth 리스너가 필요한데, Apple App Store 빌드는 앱이 수신 소켓을 바인딩하지 않도록 이를 컴파일에서 제외하기 때문입니다(`src-tauri/src/lib.rs`).

위치 권한(필수)

이 부분이 iOS가 Android와 뚜렷하게 다른 유일한 지점입니다. Android의 지오로케이션 플러그인은 런타임 권한을 선언하고 OS가 일반적인 대화 상자를 표시합니다. 반면 **iOS는 사용 설명 문자열이 없으면 앱이 위치를 요청하는 순간 앱을 종료합니다**. GeoLibre는 이를 `src-tauri/Info.ios.plist`에서 제공합니다.

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>GeoLibre uses your location to center the map, capture GPS points during Field Collection, and record GPS tracks.</string>
```

Tauri는 빌드 시 `Info.ios.plist`를 생성된 `gen/apple/geolibre-desktop_iOS/Info.plist`에 병합합니다. `gen/apple`은 git에서 무시되므로 이 파일이 해당 문자열의 영구적인 보관 위치입니다. Android의 매니페스트 권한을 수동 편집된 재생성 매니페스트가 아니라 플러그인에서 가져오는 것과 같은 이유입니다. 이 문자열은 세 가지 위치 사용처인 Field Collection, GPS Tracking, 그리고 Controls → GeoLocate 지도 컨트롤을 모두 포괄합니다. 빌드 후에는 키가 병합 과정에서 살아남았는지 확인하세요(아래 빌드 참조).

최소 iOS 버전

GeoLibre는 **iOS 15.0**을 대상으로 하며, 이는 `apps/geolibre-desktop/src-tauri/tauri.ios.conf.json` 에서 한 번만 설정합니다.

```
"bundle": { "iOS": { "minimumSystemVersion": "15.0" } }
```

Tauri는 이 값을 생성된 `gen/apple/project.yml` 에 XcodeGen의 `deploymentTarget.iOS` 로 템플릿화하고, Xcode는 이를 `IPHONEOS_DEPLOYMENT_TARGET` 으로, 최종적으로 앱의 `MinimumOSVersion` 으로 변환합니다. 생성된 프로젝트에서 수동으로 편집할 것은 없으며, `Info.ios.plist` 에도 아무것도 없습니다. Xcode가 빌드 설정에서 `plist` 키를 기록합니다.

이 키를 생략하면 **Tauri CLI 자체의 기본값이 적용되며**, 이는 `14.0` 입니다. 이는 Apple의 하한선보다 낮습니다. 2027년 봄부터 App Store Connect는 `MinimumOSVersion` 이 15.0 미만인 업로드를 거부합니다. 그 전까지는 빌드를 받아들인 뒤 경고 이메일을 보내는데(`ITMS-90068`), 버전 2.5.0 빌드 7이 14.0으로 출시되었다는 사실이 바로 이렇게 발견되었습니다. 그 빌드가 승인된 빌드이므로, 현재 App Store 목록에는 여전히 iOS 14.0 이상으로 표시됩니다. 목록은 다음 릴리스에서 15.0으로 갱신됩니다.

`gen/apple` 은 생성되는 것이므로, 여기서의 변경은 프로젝트가 재생성되어야 비로소 효력을 갖습니다. CI는 매 실행마다 재생성하므로 값을 자동으로 반영합니다. 로컬에서는 직접 재생성하세요(이 문서의 다른 모든 곳과 마찬가지로 경로는 `apps/geolibre-desktop` 기준 상대 경로입니다).

```
cd apps/geolibre-desktop
rm -rf src-tauri/gen/apple
npx tauri ios init

# What init generated:
grep -A2 'deploymentTarget:' src-tauri/gen/apple/project.yml

# What a build actually shipped. `MinimumOSVersion` exists only in the *built*
# app, since Xcode writes it from the build setting; it is not in the generated
# source Info.plist. So this needs an archive, which `init` alone does not
# produce.
npx tauri ios build --no-sign
/usr/libexec/PlistBuddy -c 'Print :MinimumOSVersion' \
  src-tauri/gen/apple/build/geolibre-desktop_iOS.xcarchive/Products/Applications/GeoLibre.app/
Info.plist
```

CI 작업은 이를 두 곳에서 검증합니다(지속적 통합 참조). 설정된 값을 Apple의 하한선 및 `ios init` 직후 생성된 `project.yml` 과 비교하고, 그다음 내보낸 `.ipa` 의 `MinimumOSVersion` 을 검사하는데, 이것이 App Store Connect가 실제로 읽는 사본입니다.

하한선을 더 올리는 것은 기술적 결정이 아니라 제품 결정입니다. iOS 15는 iPhone 6s/7 및 iPad Air 2 세대를 위한 마지막 릴리스이므로, 16.0으로 올리면 더 새로운 WebKit 기준선을 얻는 대신 해당 기기를 지원에서 제외하게 됩니다.

툴체인 설정(최초 한 번)

Xcode(App Store에서 설치. 한 번 열어 라이선스에 동의하고 iOS 플랫폼을 설치하세요)가 설치된 **Mac**, 명령줄 도구, CocoaPods, 그리고 Rust iOS 타겟이 필요합니다.

```
xcode-select --install          # command-line tools (if not already)
sudo xcodebuild -license accept
brew install cocoapods         # or `gem install cocoapods`

# Rust device + simulator targets (install rustup first if needed)
rustup target add aarch64-apple-ios aarch64-apple-ios-sim x86_64-apple-ios
```

aarch64-apple-ios 는 실제 기기와 App Store .ipa 용이고, aarch64-apple-ios-sim 은 Apple Silicon Mac 의 시뮬레이터용입니다.

빌드

```
cd apps/geolibre-desktop
npx tauri ios init          # generate src-tauri/gen/apple (once)
npx tauri ios dev           # run in the simulator / on a tethered device
npx tauri ios build --no-sign # unsigned .ipa + .xcarchive (no Apple account needed)
```

npx tauri ios build --export-method app-store-connect 는 그대로는 동작하지 않습니다. tauri ios init 이 생성하는 프로젝트는 디버그와 릴리스 구성 모두에 CODE_SIGN_IDENTITY = "iPhone Developer" 를 고정해 놓으므로, 내보내기 방식이 App Store이더라도 Xcode는 **development** 서명으로 해석합니다. 그러면 자동 development 서명이 등록된 기기를 요구하며 다음과 같이 실패합니다.

```
error: Your team has no devices from which to generate a provisioning profile.
error: No profiles for 'org.geolibre.app' were found: Xcode couldn't find any
      iOS App Development provisioning profiles matching 'org.geolibre.app'.
```

CODE_SIGN_IDENTITY="Apple Distribution" 으로 재정의해도 도움이 되지 않습니다. Xcode가 "automatically signed for development, but a conflicting code signing identity ... has been manually specified."라며 거부합니다.

대신 아카이브와 내보내기를 분리하세요. **서명 없이** 아카이브한 다음, 내보내기 단계에서 배포 서명을 수행하게 합니다. 이 방식은 등록된 기기가 필요 없고, Xcode가 Apple Distribution 인증서를 생성하고, App ID를 등록하고, App Store 프로파일을 즉석에서 생성합니다.

```
cd apps/geolibre-desktop
npx tauri ios build --no-sign          # → gen/apple/build/geolibre-desktop_iOS.xcarchive

G=src-tauri/gen/apple
cat > "$G/ExportOptions-appstore.plist" <<'PLIST'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>method</key><string>app-store-connect</string>
```

```
<key>teamID</key><string>YOUR_TEAM_ID</string>
<key>signingStyle</key><string>automatic</string>
<key>uploadSymbols</key><true/>
</dict></plist>
PLIST

xcodebuild -exportArchive -allowProvisioningUpdates \
  -archivePath "$G/build/geolibre-desktop_iOS.xcarchive" \
  -exportOptionsPlist "$G/ExportOptions-appstore.plist" \
  -exportPath "$G/build/appstore"
```

이렇게 하면 실제로 제출 가능한 `build/appstore/GeoLibre.ipa` 가 생성됩니다. Apple Distribution 으로 서명되고, `get-task-allow=false` 이며, 엔타이틀먼트는 `<TEAM>.org.geolibre.app` 이고, iOS Team Store Provisioning Profile이 포함됩니다. 업로드 전에 `codesign -dvvv` 로 검증하세요.

이 방식으로 Xcode가 생성하는 인증서는 **Cloud Managed**이므로 `security find-identity -v -p codesigning` 에 나타나지 않습니다. ID 목록이 비어 있다고 해서 서명이 실패했다는 증거는 아닙니다. `.ipa` 자체를 확인하세요.

- `gen/apple` 은 생성되는 디렉터리(git에서 무시)이며 필요에 따라 재생성됩니다. `init` 은 `tauri.ios.conf.json` (번들 ID, 배포 타겟, Python 백엔드 제외)을 적용하고, `Info.ios.plist` (위치 문자열)는 아래에 설명한 대로 나중에 `tauri ios build / dev` 가 병합합니다.
- iOS에서 앱 이름은 **GeoLibre**이며(데스크톱 빌드는 "GeoLibre Desktop"), 번들 ID는 `org.geolibre.app` 입니다. 둘 다 `src-tauri/tauri.ios.conf.json` 에서 설정하며, Android와 동일한 재정의 패턴과 이유를 따릅니다(`tauri.conf.json` 의 `identifier` 는 데스크톱 설정과 Linux/macOS 패키징의 키 역할을 계속하도록 `org.geolibre.desktop` 으로 유지됩니다).
- 빌드 후 위치 문자열이 제대로 들어갔는지 확인하세요.

```
/usr/libexec/PlistBuddy -c 'Print :NSLocationWhenInUseUsageDescription' \
  src-tauri/gen/apple/geolibre-desktop_iOS/Info.plist
```

병합은 `tauri ios build / dev` 중에 일어나며, `ios init` 중에는 일어나지 **않습니다**. `init` 직후에는 생성된 plist에 아직 키가 없으며, 이는 정상입니다. 또한 Xcode 프로젝트, 타겟, plist 디렉터리의 이름은 `productName` 이 아니라 **Cargo 패키지**(`src-tauri/Cargo.toml` 의 `geolibre-desktop`)에서 따온다는 점에 유의하세요. 사용자에게 보이는 앱 이름과 번들 ID는 여전히 `tauri.ios.conf.json` 에서 오며, 생성된 `project.yml` 에 `PRODUCT_NAME: GeoLibre` 와 `PRODUCT_BUNDLE_IDENTIFIER: org.geolibre.app` 으로 들어갑니다.

서명

실제 기기에서 실행되거나 App Store에 도달하는 모든 빌드는 서명되어야 합니다. Android의 디버그 키스토어 같은 지름길은 없습니다. 다음이 필요합니다.

1. **Apple Developer Program** 멤버십(연 \$99).
2. **서명 인증서**(App Store용 Apple Distribution, 기기 테스트용 Apple Development). Keychain에서 `.p12` 로 내보냅니다.

3. `org.geolibre.app` 앱 ID용 **프로비저닝 프로파일**.

4. **Team ID**(App Store Connect → Membership).

로컬에서는 `gen/apple/geolibre-desktop.xcodeproj` 를 Xcode에서 한 번 열고 팀을 지정해 "Automatically manage signing"을 활성화하는 것이 가장 간단한 방법입니다. CI의 경우 ID를 시크릿에서 가져옵니다(아래 참조).

로컬 ID와 CI ID는 서로 바꿔 쓸 수 없습니다. 위의 내보내기 방법은 Xcode가 **Cloud Managed Apple Distribution** 인증서와 Xcode 관리 프로파일을 생성하게 합니다. 둘 다 CI에 공급할 수 없습니다. 클라우드 관리 ID는 내보낼 수 있는 개인 키가 없으므로 base64로 인코딩할 `.p12` 가 없고, CI의 수동 서명 내보내기는 Xcode 관리 프로파일을 즉시 거부합니다 ("is Xcode managed, but signing settings require a manually managed profile"). CI를 위해서는 Developer 포털에서 **별도의 수동 관리** 쌍을 만들어야 합니다. Keychain Access에서 생성한 CSR로 만든 Apple Distribution 인증서(이렇게 하면 개인 키를 보유하므로 `.p12` 를 내보낼 수 있습니다)와, 그 인증서에 바인딩된 `org.geolibre.app` 용 App Store 프로비저닝 프로파일입니다. 클라우드 관리 배포 인증서와 수동 배포 인증서를 동시에 보유하는 것은 문제없으며 일반적인 일입니다.

지속적 통합

`.github/workflows/ios.yml` 은 GitHub 릴리스가 게시될 때마다 `macos-15` 에서 실행됩니다 (그리고 "Run workflow"로 수동 실행할 수도 있습니다). iOS는 Linux에서 크로스 컴파일할 수 없으므로, 이것이 macOS 러너가 필요한 유일한 모바일 워크플로입니다. 이미지는 자유롭게 선택할 수 없습니다. 아래의 Xcode 하한선을 참조하세요.

- **Apple 서명 시크릿이 설정되어 있으면**, ID를 임시 키체인으로 가져오고, **서명 없이** 아카이브하고, 수동 서명으로 서명된 `.ipa` 를 내보내고, 번들 ID, 서명, 빌드 번호 형식, `MinimumOSVersion` 을 검증한 뒤 `geolibre-ios-ipa` 아티팩트로 업로드합니다. 게시된 릴리스에서는 추가로 아티팩트의 14일 보존 기간보다 오래 남는 `GeoLibre-<version>-ios_app-store.ipa` 릴리스 자산으로도 업로드합니다.
- `APPLE_IOS_CERTIFICATE_BASE64` — `base64 -i dist.p12`
- `APPLE_IOS_CERTIFICATE_PASSWORD` — `.p12` 가 암호로 보호된 경우에만
- `APPLE_IOS_PROVISIONING_PROFILE_BASE64` — `base64 -i profile.mobileprovision`
- `APPLE_TEAM_ID` — 기존 macOS/Homebrew 서명 시크릿에서 **재사용**합니다. Team ID는 계정 전체에 공통입니다.
- **시크릿이 없으면**, 서명 없는 **컴파일 검사** (`cargo build --lib --release --target aarch64-apple-ios`)로 대체하여 CI가 여전히 iOS 빌드 손상을 잡아내도록 합니다. 다만 설치 가능한 `.ipa` 는 생성할 수 없습니다. 이 상태에서의 릴리스 실행은 릴리스를 실패시키는 대신 아무것도 첨부하지 않고 어떤 시크릿이 누락되었는지 경고를 기록합니다.

macOS/Homebrew 서명 시크릿을 재사용할 수 있나요?

Team ID만 재사용할 수 있습니다. 저장소의 릴리스 워크플로는 이미 Homebrew cask용 macOS 빌드를 서명하고 공증하지만, 그 `APPLE_CERTIFICATE` 는 **Developer ID Application** 인증서입니다. 이 유형은 App Store 외부에서 배포되는 macOS 앱을 서명하는 것이며 **iOS 앱은 서명할 수 없습니다**. iOS에는 자체 **Apple Distribution** 인증

서와 **프로비저닝 프로파일**이 필요하므로(Developer ID 배포에는 둘 다 없습니다), 위의 `APPLE_IOS_*` 시크릿이 그것이며, 잘못된 인증서/암호가 입력되는 것을 막기 위해 macOS 시크릿과 의도적으로 다르게 이름을 붙였습니다. 다행인 점은 이들이 추가 비용 없이 **동일한 유료 Apple Developer 계정**에서 생성된다는 것입니다. Developer 포털에서 `org.geolibre.app` 용 Apple Distribution 인증서와 App Store 프로비저닝 프로파일을 추가하고, 기존 `APPLE_TEAM_ID` 를 재사용하세요.

작업이 아카이브와 내보내기를 두 단계로 수행하는 이유. 작업은 `npx tauri ios build --no-sign` 으로 아카이브한 다음, 별도의 `xcodebuild -exportArchive` 단계에서 **수동** 서명으로 서명합니다. 그 이유는 빌드에서 설명한 대로입니다. `tauri ios build --export-method` 는 이 프로젝트를 서명할 수 없는데, 생성된 프로젝트에 `CODE_SIGN_IDENTITY = "iPhone Developer"` 가 고정되어 있어서 내보내기 방식이 무엇이든 자동 서명이 development 프로파일을 찾으려 하기 때문입니다.

이는 러너에서 검증되었습니다. `export_method: app-store-connect` 로 실행한 `workflow_dispatch` 실행이 녹색으로 완료되어 `IPA ... bundle id org.geolibre.app, signed by Apple Distribution` 을 보고하고 `geolibre-ios-ipa` 아티팩트를 게시했습니다. 이후 빌드(버전 2.5.0, 빌드 7)가 App Store 심사를 통과해 라이브 상태입니다. 아직 실행되지 않은 것은 **릴리스 트리거** 진입점입니다. 이 작업은 지금까지 수동으로만 시작되었습니다.

러너의 Xcode가 엄격한 하한선을 정합니다. `tauri ios init` 은 **object format 77**의 Xcode 프로젝트를 생성하는데, 이는 **Xcode 16+**만 열 수 있습니다. `macos-14`에서는 아무것도 컴파일하기 전에 아카이브가 즉시 실패했습니다.

```
xcodebuild: error: Unable to load workspace '.../geolibre-desktop.xcodeproj/project.xcworkspace/'.
Reason: The project 'geolibre-desktop' cannot be opened because it is in a future Xcode project file format (77).
```

따라서 작업은 `macos-15`에서 실행되며, 이미지의 기본값을 신뢰하는 대신 이미지에서 가장 최신 Xcode를 명시적으로 선택하고, 그것이 16보다 오래되었으면 명확한 메시지와 함께 조기에 실패합니다. 이 하한선은 이 장소의 어떤 것이 아니라 **Tauri CLI**가 생성하는 프로젝트에서 비롯된다는 점에 유의하세요. Tauri CLI를 올릴 때마다 다시 확인하세요.

별개로, App Store Connect는 현재 하한선 미만의 SDK로 빌드된 업로드를 거부합니다. 이 실패는 빌드 시점에는 조용히 지나갑니다. 오래되었지만 열 수 있는 Xcode는 여전히 `.ipa`를 생성하며 업로드 시점에만 실패합니다. 거부된 업로드를 추측이 아니라 실행 로그에서 진단할 수 있도록 Xcode 버전과 iOS SDK 목록이 실행 로그에 출력됩니다.

`workflow_dispatch`의 `export_method` 입력이 내보내기 경로를 선택합니다 (TestFlight/App Store용 `app-store-connect`, 애드혹 등록 기기용 `release-testing`, 개발용 `debugging`).

로컬 빌드 실행 및 테스트

- **시뮬레이터:** `npx tauri ios dev` 를 실행하고 시뮬레이터를 선택하거나, Xcode 프로젝트를 열고 Run을 누르세요. 시뮬레이터에는 유료 계정이 필요 없습니다.
- **본인 기기:** 기기를 연결하고 Xcode에서 `gen/apple/geolibre-desktop.xcodeproj` 를 연 뒤, 기기를 선택하고 Run을 누르세요(무료 Apple ID로 7일 기기 서명이 가능합니다).
- **테스터:** **TestFlight**를 통해 서명된 빌드를 배포하세요(Xcode Organizer 또는 Transporter로 `.ipa` 를 업로드한 뒤 App Store Connect에서 테스터를 초대합니다).

App Store에 게시

GeoLibre는 **App Store에 라이브 상태**이므로, 아래의 최초 온보딩은 완료되었습니다. 여기서 다루는 각 단계는 (완료.)로 표시되어 있으며, 해당 단계가 선언하는 내용이 바뀔 때만 다시 살펴보면 됩니다. 이후의 모든 릴리스는 2단계의 상시 제약 하에서 3단계(빌드 업로드 및 심사 제출)를 반복합니다. 빌드 측면은 CI 워크플로가 담당하고, 나머지는 App Store Connect에서 처리합니다.

1. **앱 레코드.** (완료.) App Store Connect에서 번들 ID `org.geolibre.app` 으로 새 앱을 만듭니다(먼저 Developer 포털에서 앱 ID를 등록하세요). GeoLibre의 레코드는 Apple ID 6796039674이며 Productivity 카테고리에 등록되어 있습니다.
2. **최소 기능(가이드라인 4.2).** Apple은 "재패키징된 웹사이트"인 앱을 거부합니다. GeoLibre는 원격 URL을 로드하는 래퍼가 아니라 GPS, 오프라인 가능한 지도 워크스페이스, 로컬 파일 처리 같은 실제 기기 통합을 갖춘 번들 네이티브 앱이므로 통과합니다. 이 상태를 유지하세요. 웹 자산을 바이너리에 포함해 출시하고(여기서는 기본값), 웹뷰가 `geolibre.app` 을 가리키게 하지 마세요.
3. `.ipa` 를 TestFlight 빌드로 **업로드**한 뒤(Transporter 또는 Xcode Organizer 사용), 그 빌드를 App Store 심사에 제출합니다. `geolibre-ios-ipa` CI 아티팩트에서 가져오거나, 아티팩트가 만료된 오래된 태그의 경우 해당 릴리스의 `GeoLibre_<version>_ios_app-store.ipa` 자산에서 가져오세요.

빌드 번호. App Store Connect는 마케팅 버전별로 `CFBundleVersion` 을 영구적으로 소비하며, 각 업로드는 마지막으로 소비된 값보다 높게 정렬되어야 하므로, 모든 업로드에는 새롭고 더 높은 값이 필요합니다. CI는 아카이브의 `CFBundleVersion` 에 `$GITHUB_RUN_NUMBER` (항상 증가하기만 하는 정수)를 찍고, `CFBundleShortVersionString` 은 계속 `tauri.conf.json` 을 따라가며 사용자가 보는 버전으로 유지됩니다.

기존 실행을 재실행하면 `$GITHUB_RUN_ATTEMPT` 가 덧붙습니다(42가 42.2가 됨). 실행 번호 자체는 실행의 수명 동안 고정되기 때문입니다. 접미사가 없으면, IPA가 이미 업로드된 실행을 재실행할 때 동일한 빌드 번호가 다시 생성됩니다. 수동으로 업로드된 유일한 2.4.0 빌드는 2.4.0을 사용했는데, Apple의 점으로 구분된 정수 비교에서는 3 이상의 단순 실행 번호가 이미 이를 넘어섭니다.

이는 의도적으로 `tauri ios build --build-number` 로 처리하지 **않습니다**. 그 플래그는 버전에 덧붙여서 예를 들어 2.4.0.42를 만듭니다. `CFBundleVersion` 은 마침표로 구분된 정수를 최대 **세 개**까지만 가질 수 있으므로, App Store Connect는 네 개 구성 요소의 값을 업로드 시 거부합니다 (ITMS-90060). 그것도 전체 빌드가 끝난 뒤예요. 검증 단계에서 형식을 검사하는 이유가 바로 이것입니다.

수동으로 업로드하는 경우(CI 없이), 마케팅 버전이 이전에 업로드된 적이 있다면 빌드 번호를 직접 설정해야 합니다.

4. **Store 목록.** (완료.) 아이콘(`src-tauri/icons/ios` 아래에 이미 생성됨), 필수 기기 크기용 스크린샷(6.7" 및 6.5" iPhone, 그리고 12.9" iPad. GIS 워크스페이스는 iPad에 정말 잘 어울립니다), 설명, 키워드. 5. **개인정보.** (완료.) **App Privacy** 설문을 정직하게 작성하세요. 각 네트워크 목적지(지오코딩, AI 어시스턴트, 베이스맵/타일 가져 오기, Earth Engine용 Google OAuth)를 선언하고, 위치는 사용 중일 때만 사용되며 백엔드에서 수집하지 않음을 선언합니다. 개인정보 처리방침 URL은 게시된 [개인정보 처리방침](#)을 가리키게 하세요. 6. **연령 등급 및 카테고리.** (완료.) GeoLibre는 **Productivity** 카테고리에 등록되어 있습니다. 7. **수출 규정 준수.** (완료.) 이미 `src-tauri/Info.ios.plist`에 선언되어 있습니다.

```
<key>ITSAppUsesNonExemptEncryption</key>
<false/>
```

App Store Connect는 모든 업로드마다 암호화에 대해 묻고, 질문에 답하지 않은 빌드는 제출하거나 테스터에게 보낼 수 없습니다("Missing Compliance"). plist에 선언해 두면 빌드 시 한 번에 답변이 됩니다.

`false`는 GeoLibre의 암호화 사용이 Apple이 면제로 취급하는 범위(타일, 지오코딩, AI 어시스턴트, 클라우드 카탈로그, 협업 릴레이를 위한 시스템 라이브러리 경유 HTTPS/TLS)에 한정되며, 앱이 자체 암호화를 포함하지 않는다는 것을 단언합니다. 이는 현재 사실이지만, 빌드 플래그가 아니라 **법적 선언**입니다. 이것이 바뀌면 다시 검토하세요.

이 키가 추가되기 전에 업로드된 빌드(첫 2.4.0 업로드)는 여전히 App Store Connect에서 질문에 수동으로 답해야 합니다.

알려진 제한 사항 / 후속 작업

대부분 Android와 동일합니다.

- 로컬 파일 소스(MBTiles, 로컬 래스터, 프로젝트 파일)는 실제 파일시스템 경로를 가정합니다. iOS는 문서 선택기를 통해 샌드박스 URL을 앱에 전달하므로, 해당 흐름이 네이티브로 동작하려면 조정이 필요합니다.
- **Download Offline Area**(오프라인 영역 다운로드) 도구는 서비스 워커에 의존하는데, Tauri 빌드는 이를 사용하지 않습니다. 이는 PWA 기능입니다. 네이티브 오프라인 베이스맵 캐싱은 향후 작업입니다.
- Earth Engine은 OAuth가 데스크톱 루프백 흐름을 사용하기 때문에 App Store 빌드에서 숨겨져 있습니다. 모바일 딥링크 리디렉션(iOS URL 스킴 / 유니버설 링크)은 향후 작업입니다.
- iPadOS 멀티태스킹(Split View / Stage Manager)은 아직 조정되지 않았습니다. 반응형 레이아웃이 적응해야 하지만, 릴리스 전에 실제 iPad에서 확인하세요.

<https://geolibre.app/mac-app-store/>

Mac App Store

GeoLibre Desktop은 [Mac App Store](#)에 게시되어 있습니다.

Mac App Store에서 GeoLibre Desktop 받기

두 macOS 빌드에 대한 사용자 대상 비교는 [다운로드](#)를 참조하세요. 이 페이지는 Store 빌드가 어떻게 생성되는지, 그리고 왜 다른지를 설명합니다.

GeoLibre Desktop은 macOS에서 두 가지 방식으로 제공됩니다.

- **Developer ID(기본값)**: 모든 GitHub 릴리스에서 `release.yml` 이 게시하고 웹사이트와 Homebrew를 통해 배포되는 공증된 `.dmg` 입니다. 전체 기능 세트, 강화된 런타임(hardened runtime), App Sandbox 없음.
- **Mac App Store(MAS)**: `mas-store.yml` 워크플로(또는 로컬에서 `npm run tauri:build:mas`)로 빌드되는 샌드박스 변형입니다. 이 페이지는 그 변형을 설명합니다.

App Store는 기본 빌드가 충족할 수 없는 두 가지 엄격한 규칙을 부과합니다.

1. **App Sandbox가 필수입니다.** 앱은 자신의 컨테이너와 사용자가 열기 대화 상자에서 명시적으로 선택한 파일로 제한됩니다. 다운로드된 서명되지 않은 실행 파일을 실행하는 것은 불가능합니다.
2. **가이드라인 2.5.2**: 앱은 기능을 변경하는 코드를 다운로드하거나 실행할 수 없습니다. 기본 빌드는 런타임에 `uv`, Python 환경, martin 타일 서버를 부트스트랩하는데, 이는 모두 실행 가능한 코드의 다운로드입니다.

따라서 MAS 빌드는 이러한 서비스를 고장 난 채로 출시하는 대신 컴파일에서 제외합니다.

MAS 빌드가 제거하는 것

`mas cargo` 기능(`apps/geolibre-desktop/src-tauri`)은 다음 Tauri 명령을 스텝으로 대체하고, `GEOLIBRE_MAS_BUILD=1` 은 해당 UI를 숨깁니다.

- **Python 사이드카**(Whitebox 서버 엔진, Conversion/Raster 사이드카 엔진, PostGIS 엔드포인트, SamGeo 분할, SedonaDB SQL).
- **Jupyter 서버**(Notebook 패널은 웹 빌드가 이미 사용하는 번들된 JupyterLite로 대체됩니다. Pyodide는 WebKit 내부에서 실행되며, App Review가 이를 허용합니다).
- **martin 타일 서버**와 그 GitHub 바이너리 다운로드. Add Data(데이터 추가)는 클라이언트 측 엔진이 없는 소스를 숨깁니다. **PostgreSQL(martin)**, 그리고 **SQL Server**와 **File Geodatabase**(둘 다 사이드카 기반).
- **외부 플러그인 설치**(zip 아카이브 및 레지스트리 설치). 내장 플러그인과 번들된 드롭인 플러그인은 계속 동작합니다.
- **앱 내 업데이트 확인**(`GEOLIBRE_STORE_BUILD=1` , Microsoft Store MSIX 빌드와 동일): Store 앱은 Store를 통해서만 업데이트됩니다.

- **Earth Engine 로그인**(Processing → Earth Engine, 그리고 GeoAgent 플러그인의 Earth Engine 오버레이). 위 항목들과 달리 이것은 샌드박스 제한이 아니라 엔타이틀먼트 제한입니다. 아래를 참조하세요.

Earth Engine과 `network.server` 엔타이틀먼트

Earth Engine 로그인은 네이티브 앱용 Google OAuth **루프백 리디렉션** 흐름을 사용합니다. `src-tauri/src/earth_engine_oauth.rs` 가 `127.0.0.1` 에 리스너를 바인딩하고, 시스템 브라우저에서 동의 페이지를 연 뒤, 토큰을 받기 위해 브라우저의 인바운드 리디렉션을 받아들입니다. 인바운드 연결을 받아들이려면 `com.apple.security.network.server` 가 필요합니다.

App Review는 정확히 이 문제로 **GeoLibre Desktop 2.4.0을 거부했습니다**(가이드라인 2.4.5, 제출 `76036eca-dc21-45d3-873e-39b015d37036`). 자동 분석은 일치하는 기능을 찾을 수 없을 때 서버 엔타이틀먼트를 플래그하는데, 바인딩이 Tauri 명령 내부에서 지연 실행되므로 정적으로 이를 가리키는 것이 없었습니다.

App Review Information에서 엔타이틀먼트를 변호하는 대신, 두 Apple 타깃 모두 이 기능을 제거합니다. 모듈은 `#[cfg(not(any(feature = "mas", target_os = "ios")))]` 로 게이트되고 아무것도 바인딩하지 않는 스텝 명령으로 대체되므로, **Mac App Store 빌드와 iOS 앱 모두 수신 소켓을 열지 않습니다**. 엔타이틀먼트는 `mas/Entitlements.mas.plist.template` 에서 제거되었으며, `mas-store.yml` 은 서명에 그것이 다시 나타나면 빌드를 실패시킵니다. (iOS에는 엔타이틀먼트 파일이 없습니다. 해당 샌드박스 키는 macOS 전용입니다. 하지만 같은 심사 파이프라인을 거치므로 같은 처리를 받습니다.)

UI 측면에서는 `isEarthEngineAvailable()` (`packages/plugins/src/plugins/earth-engine-auth.ts`) 이 Processing 메뉴 항목과 명령 팔레트 항목을 숨기고, 다른 무언가가 도달하면 `authenticateEarthEngine` 이 조기에 예외를 던집니다. macOS 빌드는 `__GEOLIBRE_MAS_BUILD__` 정의에서, iOS는 사용자 에이전트에서 감지하며, 패키징된 앱 내부에서만 적용됩니다. **웹 빌드는 모든 곳에서 Earth Engine을 유지하며**, iOS의 Safari도 포함됩니다. 브라우저는 Google의 팝업/리디렉션 흐름을 사용하고 아무것도 바인딩하지 않기 때문입니다.

향후 기능이 정말로 인바운드 연결을 받아들여야 한다면, 엔타이틀먼트를 다시 추가하는 것만으로는 충분하지 않습니다. 제출 전에 **App Review Information**에 기능을 설명하지 않으면 자동 검사가 다시 거부합니다.

Share.GeoLibre 로그인은 리스너가 아니라 사용자 지정 URI를 사용합니다

MAS 데스크톱 빌드는 Share.GeoLibre 프로젝트 로그인과 Settings → Environment → 세션 관리를 유지합니다. 그 PKCE 동의는 시스템 브라우저를 엽니다. 정확히 등록된 콜백 `org.geolibre.desktop:/oauth/callback` 은 설치된 앱의 OS URI 핸들러를 통해 돌아옵니다. 소켓을 바인딩하지 않으므로 이것은 `com.apple.security.network.server` 를 요구하지 **않습니다**. 앱은 로그인을 받아들이기 전에 콜백 리스너를 시작합니다. 콜드 프로세스를 시작하는 콜백에는 대기 중인 verifier가 없으므로 코드를 교환하는 대신 로그인 재시작 메시지를 표시합니다. 시간 초과되거나 취소된 동의에서 늦게 도착한 콜백은 무시되므로 즉각적인 재시도를 방해할 수 없습니다. 새 관리 동의는 최대 5분간 지속되며, 리프레시 토큰이 없고, 메모리에만 남습니다. 데스크톱 프로젝트의 리프레시 토큰은 OS 키체인에 보관되므로, 사용자가 로그아웃할 때까지 로그인이 재시작 후에도 유지됩니다. 붙여넣은 개인 API 토큰은 별도의 선택적 대체 수단으로 남아 있습니다.

제공되는 `https://share.geolibre.app` 오리진의 경우, 인증된 데스크톱 요청은 리디렉션된 Bearer 헤더가 호스트를 넘어가지 않도록 리디렉션을 비활성화한 네이티브 HTTP를 사용합니다. 자체 호스팅 공유 오리진은 브라우저 fetch를 사용하며 CORS에서 Tauri 웹뷰 오리진을 허용해야 합니다. MAS 샌드박스의 `network.client` 엔타이틀먼트가 그러한 아웃바운드 연결을 허용합니다. 어느 경로도 인바운드 리스너를 추가하거나 `network.server` 금지를 변경하지 않습니다.

클라이언트 측의 모든 것은 변경되지 않았으며 완전히 동작합니다. MapLibre/deck.gl 렌더링, 로컬 및 원격 파일용 Add Data, DuckDB-WASM 벡터 읽기, Whitebox WASM 도구, Turf/Pyodide 벡터 도구, 브라우저 엔진 변환, 클라이언트 래스터 도구, CereusDB/PGLite 기반 SQL Workspace, Python 콘솔, onnxruntime-web 탐지/segment-everything, 지오키오딩, 협업이 포함됩니다.

샌드박스 하의 알려진 제한 사항

- **날개 shapefile:** 샌드박스는 사용자가 선택한 파일에만 접근을 허용하므로, 선택한 `.shp` 옆의 `.dbf` / `.prj` / `.shx` 동반 파일을 자동으로 읽는 것이 거부됩니다. MAS 빌드는 같은 대화 상자 선택에서 동반 파일을 매칭하여 이를 보완하므로, **shapefile의 모든 구성 파일을 다중 선택**하거나 (`.shp`, `.dbf`, `.prj`, ...) **압축된 shapefile**을 로드하세요. `.shp` 만 단독으로 선택하면 속성이나 CRS 없이 로드됩니다. 이는 저장된 프로젝트가 날개 `.shp` 레이어를 복원할 때도 적용됩니다. 다시 추가하거나 zip을 사용하세요.
- **환경 변수:** "셸 환경에서 API 키 읽기" 편의 기능은 아무것도 반환하지 않습니다. 샌드박스 앱은 로그인 셸 환경을 상속하지 않기 때문입니다. 대신 Settings에서 키를 입력하세요.
- **로컬 파일을 참조하는 프로젝트 다시 열기:** 선택한 파일에 대한 샌드박스 권한은 앱이 보안 범위 북마크를 저장하지 않는 한 프로세스와 함께 끝나는데, GeoLibre는 아직 이를 하지 않습니다. 재시작 후, 컨테이너 외부의 로컬 경로를 기반으로 하는 프로젝트 레이어는 데이터 복원에 실패합니다. 파일을 다시 추가하여 다시 로드하세요. 보안 범위 북마크 도입(예: `tauri-plugin-persisted-scope`의 macOS 북마크 지원을 통해)이 계획된 해결책입니다.

로컬에서 빌드

다른 데스크톱 빌드와 달리, 이 빌드는 **JupyterLite 사이트를 포함하므로** 빌드 전에 `jupyter lite` CLI가 설치되어 있어야 합니다.

```
pip install -r apps/geolibre-desktop/jupyterlite/requirements.txt
```

이것이 없으면 빌드는 Notebook 패널이 잘못된 것을 로드하는 앱을 만드는 대신 명시적인 오류와 함께 실패합니다 (Tauri는 누락된 자산에 `index.html`로 응답하므로, JupyterLite가 없는 MAS 빌드는 패널 안에 GeoLibre의 두 번째 사본을 렌더링합니다).

```
# One-time: render the entitlements with your team id.
APPLE_TEAM_ID=XXXXXXXXXX scripts/render-mas-entitlements.sh

# Put a Mac App Store provisioning profile for org.geolibre.desktop at
# apps/geolibre-desktop/src-tauri/mas/embedded.provisionprofile
# (App Store Connect -> Profiles -> macOS App Development/Distribution).
```

```
APPLE_SIGNING_IDENTITY="Apple Distribution: Your Name (XXXXXXXXXX)" \
npm run tauri:build:mas -- --target universal-apple-darwin

# Wrap the .app into a Store-submittable installer package.
xcrun productbuild \
  --sign "3rd Party Mac Developer Installer: Your Name (XXXXXXXXXX)" \
  --component "apps/geolibre-desktop/src-tauri/target/universal-apple-darwin/release/bundle/
macos/GeoLibre Desktop.app" \
  /Applications GeoLibre-Desktop-mas.pkg
```

`npm run tauri:build:mas` 가 나머지 모든 것을 연결합니다. `mas cargo` 기능, `tauri.mas.conf.json` 오버레이(App Sandbox 엔타이틀먼트, 포함된 프로비저닝 프로파일, 번들된 사이드카 리소스 없음, 강화된 런타임 끄), 그리고 `GEOLIBRE_MAS_BUILD / GEOLIBRE_STORE_BUILD` 프론트엔드 플러그인입니다. `--native-duckdb` 와 함께 실행하는 것은 거부합니다(네이티브 DuckDB는 런타임에 spatial 확장을 서명되지 않은 네이티브 코드로 로드합니다).

샌드박스 앱은 빌드된 `GeoLibre Desktop.app` 을 직접 실행하여 스모크 테스트할 수 있습니다. 번들에 대한 `codesign -d --entitlements - --xml` 은 `com.apple.security.app-sandbox` 를 표시해야 하며, `com.apple.security.network.server` 는 표시하지 **않아야** 합니다.

OAuth 릴리스 스모크 테스트의 경우, URI 연결을 테스트하기 전에 서명된 `.app` 을 **설치**하세요. `geolibre-desktop` 클라이언트와 위의 정확한 콜백으로 발급자를 구성한 다음, Settings → Environment에서 시스템 브라우저를 통해 로그인하세요. 일치하는 콜백이 동의가 대기 중일 때만 완료되는지, 콜드 콜백이 재시작을 요청하는지, 두 번째 관리 동의가 세션을 나열하는지, 현재 프로젝트 세션을 철회하면 앱이 로그아웃되는지 확인하세요. 설치된 Windows/MSIX 및 Linux 패키지로 각 OS 러너에서 URI 왕복을 반복하세요. macOS에서 패키지를 컴파일하는 것만으로는 해당 프로토콜 연결이 검증되지 않습니다. 패키지된 로그에서 콜백 쿼리 매개변수가 마스킹되었는지 확인하고, `com.apple.security.network.server` 엔타이틀먼트가 없는지 앱 서명을 다시 확인하세요.

CI: mas-store.yml 워크플로

게시된 GitHub 릴리스마다(`release.yml` , `android.yml` , `ios.yml` 과 동일) 그리고 "Run workflow" 버튼으로 수동 실행 시 실행됩니다. 유니버설 샌드박스 앱을 빌드하고, 서명하고, 샌드박스 엔타이틀먼트를 검증하고, 서명된 `.pkg` 를 생성하여 `geolibre-mas-pkg` 아티팩트로 업로드합니다. 릴리스 실행에서는 그 `.pkg` 를 릴리스에도 첨부하므로, 태그에 대해 제출된 정확한 바이트가 CI 아티팩트 보존 기간 이후에도 사용 가능하게 남습니다. App Store Connect에 업로드하는 일은 결코 없습니다. 그 단계는 수동으로 남아 있습니다(아래 제출 참조).

MAS 서명된 `.pkg` 는 App Store를 통해서만 설치할 수 있습니다. 릴리스 페이지에서 다운로드하면 설치되지 않는 패키지를 받게 됩니다. 이것은 macOS 다운로드가 아니라 제출 아카이브로 첨부된 것입니다. 사용자에게는 App Store 목록, Homebrew cask, 또는 `release.yml` 의 Developer ID `.dmg` 를 안내하세요.

시크릿이 누락된 실행은 `secrets-gate` 작업이 처리합니다. `workflow_dispatch`에서는 하드 오류이고(`.pkg` 를 요청했는데 받을 수 없음), 릴리스에서는 경고를 기록하고 macOS 작업을 건너뛰어 그 외에는 정상인 릴리스가 빨간색이 되지 않게 합니다.

필요한 저장소 시크릿:

시크릿	내용
APPLE_MAS_CERTIFICATE	"Apple Distribution"과 "3rd Party Mac Developer Installer" 인증서 둘 다를 개인 키와 함께 담은 base64 .p12
APPLE_MAS_CERTIFICATE_PASSWORD	해당 .p12 의 암호
APPLE_MAS_SIGNING_IDENTITY	Apple Distribution: Name (TEAMID)
APPLE_MAS_INSTALLER_IDENTITY	3rd Party Mac Developer Installer: Name (TEAMID)
APPLE_MAS_PROVISIONING_PROFILE	org.geolibre.desktop 용 base64 Mac App Store .provisionprofile
APPLE_TEAM_ID	release.yml 용으로 이미 구성됨

입력 자료 만들기(Apple Developer 계정 필요):

1. 개발자 포털에서 org.geolibre.desktop 용 **App ID**를 만듭니다 (macOS, 추가 기능 없음).
2. Keychain Access CSR을 통해 두 인증서("Apple Distribution"과 "3rd Party Mac Developer Installer")를 만든 다음, Keychain Access에서 둘을 함께 하나의 .p12 로 내보냅니다: `base64 -i certs.p12 | pbcopy`.
3. 해당 App ID와 배포 인증서용 **Mac App Store Connect** 프로비저닝 프로파일을 만듭니다: `base64 -i profile.provisionprofile | pbcopy`.

제출

1. App Store Connect에서 org.geolibre.desktop 용 앱 레코드를 만듭니다 (iOS org.geolibre.app 레코드와 같은 팀. 둘은 별개의 앱입니다). GeoLibre Desktop의 경우 이미 완료되어 있으며, App Store ID는 6796848769 (App Store Connect가 App Information 페이지에서 "Apple ID"라고 표시하는 숫자 목록 식별자)이므로, 이후 릴리스는 2단계부터 시작합니다.
2. .pkg 를 다운로드합니다. 릴리스의 워크플로 실행에 있는 geolibre-mas-pkg 아티팩트에서, 또는 그 아티팩트가 완료된 후에는 릴리스 자체의 GeoLibre.Desktop_<version>_universal_mas.pkg 자산에서 받은 다음, **Transporter** 앱(또는 `xcrun altool --upload-app -f <pkg> -t macos`)으로 업로드합니다.
3. 스크린샷, 설명, 개인정보 설문을 작성합니다 (docs/privacy.md 참조. 앱은 추적 호출을 하지 않습니다).
4. 심사 메모에 처리 엔진이 웹뷰 내부에서 WebAssembly로 클라이언트 측에서 실행되며, 앱은 데이터(지도 타일, 사용자가 요청한 데이터셋)만 다운로드하고 코드는 결코 다운로드하지 않는다는 점을 언급합니다.

버전 번호는 tauri.conf.json (CFBundleShortVersionString)에서 가져옵니다. 모든 업로드에는 App Store Connect의 마지막 것보다 높은 버전 또는 빌드 번호가 필요합니다.

<https://geolibre.app/project-format/>

GeoLibre 프로젝트 형식

프로젝트는 `.geolibre` 파일로 저장됩니다. GeoLibre는 이전 이름인 `.geolibre.json` 도 계속 열 수 있지만, 단일 확장자를 사용하면 데스크톱 설치 프로그램이 시스템의 모든 JSON 파일을 차지하지 않고도 프로젝트를 GeoLibre에 연결할 수 있습니다.

데스크톱에서 파일 열기

데스크톱 설치 프로그램은 `.geolibre` 를 GeoLibre Project 문서로 등록합니다. 파일 관리자에서 파일을 열면 GeoLibre가 실행되거나 이미 열려 있는 GeoLibre 창이 앞으로 나오면서 프로젝트가 로드됩니다. 데스크톱 실행 파일은 명령줄에서 지원되는 두 이름을 모두 받아들입니다:

```
geolibre-desktop /path/to/watershed.geolibre
geolibre-desktop /path/to/legacy.geolibre.json
```

운영 체제는 `.geolibre.json` 이름의 끝에 있는 `.json` 만 인식합니다. 이러한 레거시 파일은 GeoLibre 내부와 명령줄에서는 계속 열리지만, 파일 관리자 연결을 위해서는 `.geolibre` 로 이름을 바꿔야 합니다. 파일 내용은 변하지 않습니다.

스키마

필드	타입	설명
<code>version</code>	string	형식 버전: 앱은 0.2.0 을 기록하고, 이전 파일과 Python 패키지는 0.1.0 을 기록합니다
<code>name</code>	string	프로젝트 표시 이름
<code>mapView</code>	object	<code>center</code> , <code>zoom</code> , <code>bearing</code> , <code>pitch</code> , 선택적 <code>bbox</code>
<code>basemapStyleUrl</code>	string	MapLibre 스타일 JSON URL, 또는 빈 배경을 위한 빈 문자열
<code>basemapVisible</code>	boolean	Background(배경) 레이어 표시 여부
<code>basemapOpacity</code>	number	0 부터 1 까지의 배경 레이어 불투명도
<code>blankBackgroundColor</code>	string null	Blank(빈) 배경의 선택적 사용자 지정 색상; null 이면 테마 기본값을 사용합니다
<code>layers</code>	array	레이어 정의(아래 참조)
<code>selectedLayerId</code>	string null	프로젝트 저장 시 Layers(레이어) 패널에서 선택되어 있던 선택적 레이어; null 이면 활성 레이어를 복원하지 않습니다

필드	타입	설명
layerGroups	array	선택적 Layers 패널 폴더(id , name , parentId , collapsed , visible , opacity); 비어 있으면 생략됩니다
styles	object	레이어 id → LayerStyle 매핑
preferences	object	프로젝트 지도 환경 설정, 환경 변수, 지오코딩 설정(자격 증명 삭제 참조)
plugins	object	선택적 외부 플러그인 매니페스트 URL, 활성 플러그인 ID, 플러그인 지도 컨트롤 위치, 플러그인 설정
legend	object	선택적 Print Layout(인쇄 레이아웃) 범례 사용자 지정(제목, 그룹화, 순서, 항목별 이름 변경/숨김)
printLayout	object	선택적 Print Layout 컴포저 설정(제목, 페이지 크기, 방향, 블록, 아틀라스); 기본값이면 생략됩니다
storymap	object	선택적 스크롤 기반 스토리 맵(챕터와 프레젠테이션 설정); 챕터가 없으면 생략됩니다
models	array	선택적 저장된 처리 모델(Model Builder 파이프라인)
processingHistory	array	선택적 처리 도구 실행 기록(Processing History)
widgets	array	선택적 Dashboard(대시보드) 패널 차트 위젯(아래 참조); 없으면 생략됩니다
dashboardColumns	number	선택적 Dashboard 위젯 그리드 열 수(1-6, 기본값 2); 기본값이면 생략됩니다
mapLayout	object	선택적 다중 지도 그리드(rows , cols , syncView); 단일 지도이면 생략됩니다
secondaryMapViews	array	선택적 기본 창 이외의 창들로, 각각 고유한 카메라와 레이어 표시 여부를 가집니다; 단일 지도이면 생략됩니다
primaryMapLabel	string	다중 지도 그리드에서 기본 창의 선택적 라벨
bookmarks	array	Bookmarks(북마크) 패널에서 저장한 선택적 지도 뷰(아래 참조); 없으면 생략됩니다
bookmarkGroups	array	북마크가 정리된 선택적 폴더; 없으면 생략됩니다
styleLibrary	array	선택적 프로젝트 범위 Style Manager(스타일 관리자) 항목(이름, 태그, 종류, LayerStyle 부분집합); 비어 있으면 생략됩니다
primaryRenderer	string	기본 지도 영역의 선택적 엔진: "maplibre" (2D, 기본값), "mapbox" (Mapbox GL JS), "arcgis" (ArcGIS Maps SDK for JavaScript) 또는 "cesium" (3D 지구본); 기본값이면 생략됩니다
comments	array	지도 지점이나 피처에 고정된 선택적 검토 댓글(답글 포함)
interaction	object	선택적 시작 시 Identify(식별) 대상과 컨트롤 표시 여부(아래 참조); 기본적으로 생략됩니다
metadata	object	자유 형식 프로젝트 메타데이터

플러그인 상태

```
{
  "manifestUrls": ["https://example.com/plugins/example-plugin/plugin.json"],
  "activePluginIds": ["maplibre-layer-control", "maplibre-gl-swipe"],
  "mapControlPositions": {
```

```

    "maplibre-layer-control": "top-right",
    "maplibre-gl-swipe": "top-left"
  },
  "settings": {
    "maplibre-gl-swipe": {
      "orientation": "vertical",
      "position": 50,
      "collapsed": false,
      "active": true,
      "leftLayers": ["layer-a"],
      "rightLayers": ["layer-b"]
    }
  }
}

```

`plugins` 섹션이 없는 프로젝트는 내장 기본 플러그인 상태로 열립니다.

범례

Print Layout(인쇄 레이아웃) 범례는 항상 표시된 레이어의 심볼로지에서 파생됩니다. `legend` 객체는 그 위에 덧입혀진 사용자의 편집 내용만 저장하므로, 레이어를 추가하거나 제거해도 사용자 지정 내용이 유지됩니다.

```

{
  "title": "Legend",
  "groupByLayer": true,
  "order": ["layer-b", "layer-a"],
  "overrides": {
    "layer-a": { "label": "Roads" },
    "layer-b::0": { "label": "Low" },
    "layer-b::1": { "hidden": true }
  }
}

```

- `title` — 범례 항목 위에 그려지는 제목입니다.
- `groupByLayer` — `true` 이면 등급/범주 클래스가 레이어별 제목 아래에 그룹화되고, `false` 이면 클래스가 평면적으로 나열됩니다.
- `order` — 레이어 id 기준 최상위 항목 순서(위에서부터)입니다. 나열되지 않은 레이어는 나열된 레이어 뒤에 기본 순서를 유지합니다.
- `overrides` — 안정적인 항목 키로 식별되는 항목별 `label` 및 `hidden` 편집 내용입니다. 항목 전체에는 레이어 id를, 등급/범주 항목 내의 개별 클래스에는 `${layerId}::${index}` 를 사용합니다.

`legend` 섹션이 없는 프로젝트는 기본 범례(레이어에서 자동 생성되며 제목은 "Legend")로 열립니다.

인쇄 레이아웃

Print Layout(인쇄 레이아웃) 컴포저(Project -> Print Layout)는 프로젝트에 속하므로, 컴포저가 구성한 페이지는 저장된 그대로 다시 열립니다. 이 섹션은 설정이 기본값과 달라질 때만 기록되므로, 컴포저를 한 번도 열지 않은 프로젝트에는 `printLayout` 키가 없습니다.

```
{
  "title": "Dentists by region",
  "subtitle": "2026",
  "paperSize": "a3",
  "orientation": "portrait",
  "pageMargin": "normal",
  "showLegend": true,
  "showScaleBar": true,
  "showNorthArrow": true,
  "showDataTable": true,
  "tableLayerId": "layer-a",
  "tableColumns": ["name", "count"],
  "atlasEnabled": false
}
```

- 제목 블록: `title`, `subtitle`, `titlePlacement`, `titleAlign`. 비어 있는 `title` 은 복사본을 고정하는 대신 프로젝트 이름을 따릅니다.
- 페이지: `paperSize` (`a4`, `a3`, `letter`, `legal`, `tabloid`, `fullhd`, `hd`, `uhd4k`, `square`, `custom`), `orientation`, `custom` 용 `customWidth` / `customHeight` / `customUnit`, `pageMargin`, 그리고 페이지/지도 프레임 색상.
- 요소: 범례, 축척 막대, 방위표, 바닥글, 날짜, 출처 표시, 컬러바, 사용자 지정 범례, 정보 블록에 대한 `show*` 토글과 각 요소별 설정.
- 데이터 블록: `tableLayerId` / `chartLayerId` 는 프로젝트 레이어를 지정합니다. 레이어가 프로젝트에 없는 블록은 빈 상태가 아니라 초기화된 상태로 열립니다.
- 아틀라스: `atlasEnabled` 와 지도 시리즈를 위한 커버리지 레이어, 범위 모드, 정렬, 필터, 파일 이름 패턴.

알 수 없거나 잘못된 값은 해당 필드의 기본값으로 대체되므로, 직접 편집한 파일이 컴포저를 사용할 수 없는 상태로 만드는 일은 없습니다. 세션별 상태(캡처된 지도 이미지, 현재 아틀라스 페이지, 대화 상자의 패널 너비)는 의도적으로 저장하지 않습니다.

스토리 맵

스토리 맵은 프로젝트를 스크롤 기반 내러티브로 바꿉니다. 각 챕터는 카메라 뷰와 텍스트를 담으며, 진입/이탈 시 프로젝트 레이어를 페이드 인/아웃할 수 있습니다. 프로젝트에 챕터가 없으면 이 섹션은 완전히 생략됩니다.

```
{
  "title": "A Tour of Three Cities",
  "subtitle": "Built with GeoLibre",
  "byline": "By the GeoLibre team",
  "footer": "Source: OpenStreetMap",
  "theme": "dark",
  "showMarkers": true,
  "markerColor": "#3fb1ce",
  "inset": false,
  "insetPosition": "bottom-right",
  "chapters": [
    {
      "id": "intro",
      "title": "San Francisco",
      "description": "A hilly city on the tip of a peninsula. <em>HTML allowed.</em>",
      "image": "https://example.com/sf.jpg",
      "alignment": "left",
      "hidden": false,
    }
  ]
}
```

```

    "location": { "center": [-122.4194, 37.7749], "zoom": 11, "pitch": 45, "bearing": 0 },
    "mapAnimation": "flyTo",
    "rotateAnimation": false,
    "onChapterEnter": [{ "layerId": "layer-a", "opacity": 1, "duration": 2000 }],
    "onChapterExit": [{ "layerId": "layer-a", "opacity": 0 }]
  }
}

```

alignment 는 left, center, right, full 중 하나이고, mapAnimation 은 flyTo, easeTo, jumpTo 중 하나입니다. 레이어 불투명도 변경은 프로젝트 레이어 id를 참조합니다. 스토리 맵은 **Project → Story Map**(프로젝트 → 스토리 맵)에서 만들고 발표하거나, 정적 호스팅을 위한 자체 완결형 HTML 페이지로 내보낼 수 있습니다.

대시보드 위젯

```

{
  "widgets": [
    { "id": "w1", "layerId": "layer-a", "type": "histogram", "field": "pop", "bins": 12 },
    { "id": "w2", "layerId": "layer-a", "type": "bar", "category": "kind", "aggregation": "sum", "valueField": "pop", "title": "Population by kind" }
  ]
}

```

각 위젯은 차트를 레이어의 속성에 바인딩합니다. type 은 histogram, scatter, bar, line, box, pie, indicator, selector, list 중 하나입니다. 어떤 다른 키가 적용되는지는 타입에 따라 다릅니다: field (histogram/line/box, 그리고 개수를 세지 않는 indicator), xField / yField (scatter), category + aggregation + valueField (bar/pie), bins (histogram), indicatorAggregation (count / sum / mean / min / max / median) + prefix + suffix (indicator), category + multiple (selector), listFields + sortBy + sortDir + limit (list). 막대의 aggregation 은 count / sum / mean 이고, 파이는 count / sum 만 지원됩니다. title 은 선택적 라벨이고 color 는 차트 마크를 위한 선택적 16진수 색상 (#rgb / #rrggbb)입니다 (단일 시리즈 차트에서는 시리즈 색상, bar/pie에서는 단색 램프의 기준 색상). 사용되지 않는 키는 무시됩니다. Dashboard(대시보드) 패널(Tools → Dashboard, 또는 속성 테이블의 **Dashboard** 버튼)은 위젯 그리드 열 수 (1-6, 기본값 2)인 dashboardColumns 도 프로젝트 최상위에 저장합니다. 차트는 GeoJSON 기반 벡터 레이어와 DuckDB 쿼리 레이어에서 데이터를 읽으며, 존재하지 않거나 속성이 없는 레이어에 바인딩된 위젯은 비어 있는 상태로 표시됩니다.

북마크

```

{
  "bookmarkGroups": [{ "id": "hoods", "name": "Neighborhoods", "collapsed": false }],
  "bookmarks": [
    {
      "id": "downtown",
      "name": "Downtown",
      "lng": -83.92,
      "lat": 35.96,
      "zoom": 14,
      "pitch": 30,
      "bearing": 340,
      "createdAt": 1700000000000,
    }
  ]
}

```

```

    "groupId": "hoods",
    "extra": { "visibleLayerIds": ["cities"] }
  }
]
}

```

Bookmarks(북마크) 패널(Controls → Bookmark)에서 저장한 뷰이며, 패널 순서대로 기록됩니다. lng / lat (-180..180 / -90..90), zoom (0-24), pitch (0-85), bearing (0-360으로 래핑)이 카메라이고, createdAt 은 에포크 이후 밀리초입니다. groupId 는 북마크를 bookmarkGroups 중 하나에 넣습니다(폴더는 중첩되지 않습니다). 존재하지 않는 폴더를 가리키는 groupId 는 로드 시 제거됩니다. extra 는 뷰와 함께 캡처된 상태입니다: visibleLayerIds 는 북마크를 열 때 표시할 레이어 목록이며, 나머지 레이어는 숨겨집니다. 카메라가 누락되었거나 범위를 벗어났거나 id 가 중복된 북마크는 로드 시 제거됩니다. 이 필드들이 생기기 전에는 북마크가 브라우저의 로컬 스토리지에만 저장되었습니다.

상호작용

```

{
  "interaction": {
    "identify": ["layer-a", "layer-b"],
    "controls": { "search": true, "globe": false }
  }
}

```

프로젝트를 열 때 적용되므로, 노트북에서 공유한 지도가 그곳에서 설정된 그대로 시작됩니다. Python 패키지는 save_project, to_project, to_html 에서 Map.set_identify 와 Map.show_control / Map.hide_control 로부터 이 블록을 기록합니다. identify 는 레이어 id, 표시된 모든 조회 가능 레이어를 뜻하는 "all", 해당 레이어만 식별하는 레이어 id 목록, 또는 끄기를 뜻하는 null 중 하나이며, 존재하지 않는 레이어를 가리키는 id는 건너뛴다. controls 는 툴바 패널(bookmark, search, measure, minimap, print) 또는 내장 지도 컨트롤(navigation, fullscreen, compass, geolocate, globe, scale, attribution, logo)을 시작 시 표시 여부에 매핑하며, 그 밖의 이름은 무시됩니다. 앱은 프로젝트를 저장할 때 이 블록을 변경 없이 그대로 다시 기록하며, 앱에서 Identify(식별)나 컨트롤을 켜거나 꺼도 이 블록은 편집되지 않습니다.

레이어 객체

```

{
  "id": "uuid",
  "name": "My Layer",
  "type": "geojson",
  "source": { "type": "geojson" },
  "visible": true,
  "opacity": 1,
  "style": {
    "minZoom": 0,
    "maxZoom": 24,
    "fillColor": "#3b82f6",
    "strokeColor": "#1e40af",
    "strokeWidth": 2,
    "strokeWidthUnit": "pixels",
    "fillOpacity": 0.6,
    "circleRadius": 6,
  }
}

```

```

    "rasterBrightnessMin": 0,
    "rasterBrightnessMax": 1,
    "rasterSaturation": 0,
    "rasterContrast": 0,
    "rasterHueRotate": 0
  },
  "metadata": {},
  "geojson": { "type": "FeatureCollection", "features": [] },
  "sourcePath": "/path/to/file.geojson",
  "editorTracking": {
    "enabled": false,
    "createdByField": "created_by",
    "createdAtField": "created_at",
    "editedByField": "edited_by",
    "editedAtField": "edited_at"
  },
  "capabilities": {
    "query": true,
    "create": true,
    "update": true,
    "delete": true,
    "export": true
  }
}

```

레이어는 허용되는 사용자 및 세션 작업을 명시적으로 선언하기 위해 선택적 `capabilities` 집합(`query`, `create`, `update`, `delete`, `export`)을 정의할 수 있습니다. 각 플래그는 자신이 지정하는 작업만 제어합니다: `query` 는 식별/속성 테이블과 피쳐 선택 경로를, `create` / `update` / `delete` 는 피쳐 쓰기를, `export` 는 레이어 데이터를 밖으로 복사하는 경로(Export, Save to Layer Library, Export selection)를 제어합니다. 레이어 이름 변경이나 프로젝트에서의 제거는 피쳐 편집이 아니므로 계속 사용할 수 있습니다. 생략된 `capabilities`는 레이어의 소스 종류에서 유추되는 동작을 기본값으로 사용합니다. `export` 가 `false` 이면 프로젝트 공유/게시 시 해당 레이어의 내장 GeoJSON이 제거됩니다.

`export: false` 는 프로젝트 파일 자체가 담고 있는 피쳐 데이터, 즉 레이어의 인라인 `geojson` 과 `metadata.embeddedGeoJSON` 스냅샷을 제거합니다. 레이어의 `source` 나 `connection` 은 제거하지 않으므로, 데이터를 실시간으로 가져오는 레이어(WFS, ArcGIS FeatureServer, 원격 GeoJSON 또는 타일 URL)는 여전히 접근 가능한 URL과 함께 공유되며 프로젝트를 열 때 같은 데이터를 다시 가져옵니다. 이러한 종류의 레이어에서 이 플래그는 데이터 자체가 아니라 파일 안의 복사본을 숨깁니다.

`capabilities` 는 레이어를 설명하는 것이고 앱은 이를 UI에서 적용합니다. 이는 접근 제어 메커니즘이 아닙니다. PostGIS 사이드카는 클라이언트가 저장 시 함께 보내는 `capabilities`와 모순되는 쓰기를 거부하므로 앱 내 편집이 레이어 자체의 구성과 모순되는 일은 막아 주지만, 사이드카는 테이블의 `capabilities`에 대한 독립적인 기록을 갖고 있지 않으며 호출자를 서로 구별할 수도 없습니다. 신뢰할 수 없는 클라이언트에 대해서도 유지되어야 하는 제한에는 데이터베이스 권한 부여를 사용하세요.

WFS GetFeature 및 GeoJSON URL 레이어의 경우 `metadata.refresh` 에 선택적 자동 새로 고침 간격을 저장할 수 있습니다. `intervalMs` 는 밀리초 단위의 임의의 양수 간격일 수 있습니다:

```

{
  "metadata": {
    "refresh": { "enabled": true, "intervalMs": 60000 }
  }
}

```


수동 새로 고침은 이 메타데이터 없이도 저장된 동일한 소스 URL을 사용합니다.

다시 로드할 수 있는 레이어는 최상위 `connection` 레코드도 저장합니다. 이 레코드는 새로 고침 주기의 기본 정보 출처이며, Layers(레이어) 패널에 표시되는 지속적인 동기화 상태를 담습니다. `metadata.refresh` 는 `connection` 이 생기기 전에 저장된 프로젝트를 위한 레거시 대체 수단으로 유지됩니다:

```
{
  "connection": {
    "layerId": "layer-1",
    "interval": 60,
    "lastSyncedAt": "2026-08-01T14:20:00.000Z",
    "lastError": null,
    "onFailure": "keep-last"
  }
}
```

`interval` 은 초 단위의 자동 새로 고침 주기이며, `null` 이면 수동 동기화만 수행합니다. `lastSyncedAt` 은 가장 최근의 성공한 동기화를, `lastError` 는 가장 최근의 실패를 기록합니다(다음 성공 시 지워집니다). `onFailure` 는 동기화 실패 시 마지막으로 성공한 데이터를 유지할지(`"keep-last"` , 기본값) 아니면 버릴지(`"clear"`)를 결정합니다.

SQL Server 레이어는 쓰기가 커밋되었으나 다시 읽기에 실패했거나 쓰기 결과를 확인할 수 없을 때 최상위 `mssqlWritebackPending: true` 를 추가로 가질 수 있습니다. 이 플래그는 프로젝트 저장/로드 후에도 유지되며, 수동 Refresh(새로 고침)가 성공하여 생성된 키와 읽기 기준선이 복원될 때까지 다른 Save Edits(편집 저장)를 차단합니다. 복구에 실패하더라도 `onFailure: "clear"` 인 경우에도 레이어의 피쳐는 보존됩니다. 복구에 성공하면 플래그가 제거되며, 이전 기준선 메타데이터는 해당 읽기가 성공할 때까지 변경되지 않습니다.

데스크톱 앱의 로컬 파일 벡터 레이어의 경우 `metadata.watch` 에 "이 파일의 변경 사항 감시" 토글을 저장할 수 있습니다. 활성화되면 데스크톱 앱은 파일 시스템 감시자를 등록하여 디스크의 파일이 변경될 때마다 `sourcePath` 에서 레이어의 피쳐를 다시 로드합니다:

```
{
  "metadata": {
    "watch": { "enabled": true }
  }
}
```

감시가 꺼져 있으면 이 키는 생략되며, 데스크톱 호스트가 아닌 곳에서는 효과가 없습니다(브라우저는 로컬 파일 시스템 경로를 감시할 수 없습니다).

레이어는 영구적인 속성 조인(Layer properties → Joins)을 가질 수 있습니다. 이는 다른 레이어의 속성 테이블(일반적으로 좌표 필드 없이 Delimited Text로 추가된 지오메트리 없는 테이블)의 열을 키 필드로 매칭하여 이 레이어의 피쳐 속성으로 구체화하는 실시간 왼쪽 조인입니다:

```
{
  "joins": [
    {
      "id": "uuid",
      "joinLayerId": "other-layer-uuid",
      "targetField": "name",
      "joinField": "state_name",
    }
  ]
}
```

```

    "fields": ["pop_2025", "median_income"],
    "prefix": "census_",
    "enabled": true,
    "addedFields": ["census_pop_2025", "census_median_income"],
    "stats": {
      "matchedCount": 50,
      "unmatchedTargetCount": 2,
      "unmatchedJoinCount": 2
    }
  }
]
}

```

`fields` (가져올 부분집합; 생략하면 키를 제외한 모든 필드), `prefix`, `enabled` 는 선택 사항입니다.

`addedFields` 와 `stats` 는 엔진의 내부 기록으로, 적용할 때마다 다시 기록됩니다: `addedFields` 는 조인이 추가한 출력 열을 나열하여(다시 적용할 때 먼저 제거할 수 있어 연산이 멍등성을 유지합니다), `stats` 는 Joins UI에 표시된 마지막 매칭 수를 기록합니다. 조인은 프로젝트를 열 때 로드된 레이어 집합에 대해 다시 해석되므로, 조인 테이블이 변경되었더라도 조인 출력의 저장된 복사본은 스스로 복구됩니다.

벡터 레이어는 **quick filters**(빠른 필터)(Layer properties → Quick filters)를 가질 수 있습니다. 이는 누구도 MapLibre 표현식을 작성하지 않고도 레이어가 그리는 내용을 좁히는 데이터 기반 필터 컨트롤입니다. 저장되는 것은 컴파일된 출력이 아니라 컨트롤 상태이므로, 저장된 필터는 언제나 다시 열어 변경할 수 있습니다:

```

{
  "quickFilters": [
    {
      "id": "uuid",
      "field": "state",
      "kind": "categorical",
      "values": ["OR", "WA"]
    },
    { "id": "uuid", "field": "pop", "kind": "range", "min": 200000, "max": null },
    {
      "id": "uuid",
      "field": "founded",
      "kind": "date",
      "dateKind": "iso",
      "start": "1840-01-01",
      "end": "1900-12-31"
    },
    { "id": "uuid", "field": "name", "kind": "text", "operator": "contains", "text": "port" }
  ]
}

```

`kind` 는 컨트롤과 비교 방식을 결정합니다: `categorical` (`values` 에 대한 체크박스), `range` (포함 범위의 숫자 `min / max`, 한쪽이 `null` 이면 열린 경계), `date` (포함 범위의 `YYYY-MM-DD` `start / end`), `text` (`text` 에 대한 대소문자 구분 없는 `contains / startsWith / equals`). `dateKind` 는 필드가 타임스탬프를 저장하는 방식을 나타내며 (`iso` (기본값, ISO 텍스트의 앞부분 `YYYY-MM-DD` 를 비교), `epochMs`, `epochS`), `enabled: false` 는 입력된 값을 버리지 않고 컨트롤을 음소거합니다. 아무것도 선택되지 않은 컨트롤은 제약을 두지 않으므로, 선택을 비우면 피처가 하나도 표시되지 않는 대신 모든 피처가 표시됩니다.

벡터 레이어는 영구적인 불리언 MapLibre 표현식 필터도 가질 수 있습니다. 이는 **Select by Expression → Filter layer**(표현식으로 선택 → 레이어 필터)에서 작성하며, 소스 데이터를 변경하거나 복사하지 않고 일치하지 않는 피처를 숨깁니다:

```
{
  "filterExpression": [ ">=", [ "get", "population" ], 100000 ]
}
```

이 표현식은 프로젝트와 함께 저장되며, Select by Expression 패널이나 레이어의 **Clear filters**(필터 지우기) 작업에서 지울 때까지 활성 상태로 유지됩니다. Quick Filters 및 아래에 설명하는 일시적 필터와 결합됩니다.

저장되는 것은 순수한 MapLibre 표현식이므로, Expression Builder의 @ 변수(@project_name, @layer_name, @feature_count, @map_zoom, @map_scale)는 필터가 적용되는 시점에 리터럴 값으로 해석되며 이후 지도를 따라가지 않습니다. 필터가 현재 줌을 따라야 한다면 대신 MapLibre의 ["zoom"] 연산자를 사용하세요.

클러스터 렌더러를 사용하는 포인트 레이어는 주목할 만합니다. MapLibre는 렌더러가 필터를 평가하기 전에 소스 단계에서 클러스터링하기 때문입니다. 따라서 GeoLibre는 클러스터링된 레이어의 소스 데이터를 영구 표현식 필터와 Quick Filters로 좁혀, 버블과 개수가 둘 다를 따르도록 합니다. Time Slider 윈도우, 규칙 필터, 임베드 `setFilter`는 여전히 피처별 렌더 필터이며 이미 만들어진 클러스터를 변경할 수 없으므로, 클러스터링이 켜져 있는 동안에는 이 필터들로 개수가 좁혀지지 않습니다.

컴파일된 필터는 일시적인 `timeFilter` 및 `embedFilter`, 그리고 규칙 기반 렌더러의 불일치 숨김 필터와 함께 하나의 `all` 아래에 결합되므로, 호스트 페이지의 `setFilter`, Time Slider 윈도우, 사용자의 빠른 필터가 서로를 대체하는 대신 함께 레이어를 좁힙니다. 빠른 필터는 읽기 전용 `layout=viewer` 크롬에서도 표시됩니다. 필터링은 공유된 지도를 편집하는 방법이 아니라 읽는 방법이기 때문입니다.

`geojson` 레이어가 `style.simpleStyleEnabled`를 활성화하면, 개별 피처가 `simplestyle-spec` 속성(`stroke`, `fill`, `stroke-width`, `fill-opacity` 등)으로 레이어 스타일을 재정의할 수 있습니다. GeoLibre는 텍스트 마커 포인트(Annotations 레이어에서 사용)의 피처별 `text-color`도 인식하며, 피처가 이를 설정하지 않으면 `style.textColor`로 대체합니다.

벡터 레이어는 `popup` 블록(Style panel → Popup)을 가질 수 있습니다. 이는 뷰어가 피처를 클릭하거나 마우스를 올렸을 때 보게 되는 내용의 디자인입니다. 어떤 필드가 어떤 순서로, 어떤 라벨과 값 형식으로 표시되는지, 팝업의 제목이 무엇인지, 호버 툴팁이 포인트를 따라다닐지를 결정합니다:

```
{
  "popup": {
    "click": true,
    "hover": true,
    "titleField": "name",
    "showFeatureId": false,
    "maxWidth": 640,
    "imageHeight": 420,
    "fields": [
      {
        "field": "pop_max",
        "label": "Population",
        "kind": "number",
        "format": { "thousands": true, "suffix": " people" },
        "hover": true
      },
      { "field": "region", "label": "Region" },
      { "field": "homepage", "kind": "link", "format": { "linkLabel": "City website" } }
    ]
  }
}
```

```
}
}
```

모든 키는 선택 사항이며, **popup 블록이 없는 레이어는 기존 동작을 유지합니다**: 레이어 이름을 제목으로, 그 다음 모든 속성을 행으로 표시합니다. `click`의 기본값은 `true` 이고(`false` 는 Identify 팝업을 억제합니다), `hover`의 기본값은 `false` 입니다. `fields`는 표시 내용을 좁히고 순서를 정합니다. 목록이 없거나 비어 있으면 데이터 자체의 순서대로 표시되는 모든 필드를 뜻하며, 피처가 갖고 있지 않은 나열된 필드는 빈 값으로 출력되는 대신 건너뛰니다. 각 필드의 `kind`는 `auto` (타입 없는 렌더링: 정제된 KML `description` 마크업과 인라인 `data:image/*;base64` 값을 썸네일로 표시), `text`, `number`, `date`, `link`, `image` 중 하나이며, `format`은 `decimals`, `thousands`, `dateFormat` (`date` | `datetime` | `time` | `iso` | `year`), `prefix`, `suffix`, `linkLabel`을 담습니다. 필드의 `hover: true`는 해당 필드를 툴팁의 짧은 부분집합에 넣습니다. 플래그가 지정된 필드가 없는 툴팁은 제목만 표시하거나, 제목이 단지 레이어 이름일 뿐이면 아무것도 표시하지 않습니다.

`maxWidth` (288-1200)는 클릭 팝업이 그려질 수 있는 최대 너비이고, `imageHeight` (40-1200)는 그 안에서 `image` 필드의 썸네일이 그려질 수 있는 최대 높이이며, 둘 다 CSS 픽셀 단위입니다. 각각 해당 범위로 제한되고 `maxWidth`는 추가로 뷰포트에 의해 제한되므로, 어느 쪽도 지도를 덮어 버리는 팝업을 만들 수 없습니다. 값이 없으면 팝업은 기본 너비와 이미지 높이를 유지합니다. 썸네일은 종횡비를 유지하므로, 가로 사진의 경우 `maxWidth`를 함께 맞추지 않으면 `imageHeight` 만으로는 효과가 거의 없습니다.

`titleField`는 레이어 이름 대신 피처 자체의 값으로 팝업을 시작합니다. `titleExpression` (MapLibre 표현식 소스)이 이보다 우선하며, 둘 다 아무 결과도 내지 않으면 레이어 이름으로 대체됩니다. `bodyExpression`은 본문 전체(필드 행과 `id` 행 포함)를 피처의 속성으로 구성된 문장으로 대체하므로, 이 값이 설정되어 있는 동안에는 `showFeatureId`가 효과가 없습니다.

`fieldVisibility`는 여전히 최종 권한을 가집니다: `"hidden"` 또는 `"excluded"`로 표시된 필드는 **popup** 블록이 지정하더라도 절대 팝업에 나타나지 않습니다. 팝업 디자인은 표시되는 항목 중에서 선택할 뿐, 작성자가 숨긴 것을 다시 노출할 수는 없습니다. 이는 표현식에도 마찬가지로 적용됩니다: `titleExpression`과 `bodyExpression`은 표시되는 속성에 대해서만 평가되므로, `["get", ...]`으로 숨겨진 열이나 GeoLibre 내부 열을 다시 끌어올 수 없습니다. 래스터 픽셀 식별은 다른 경로를 거치며 `popup`을 무시합니다.

설명 메타데이터

레이어는 `descriptiveMetadata` 블록을 가질 수 있습니다. 이는 레이어의 **Metadata**(메타데이터) 대화 상자(레이어 메뉴 → Metadata)에서 편집하는 카탈로그 설명입니다. 데이터를 문서화할 뿐 레이어 렌더링 방식에는 아무런 영향을 주지 않습니다. 렌더러와 플러그인이 참조하는 내부 상태를 담은 레이어의 `metadata` 레코드와는 별개입니다.

```
{
  "descriptiveMetadata": {
    "title": "Rivers of Tennessee",
    "abstract": "Major rivers digitized from 1:24k topographic maps.",
    "keywords": ["hydrology", "rivers"],
    "license": "CC-BY-4.0",
    "attribution": "© Tennessee GIS",
    "contact": { "name": "Ada Lovelace", "email": "ada@example.org", "organization": "TN GIS" },
    "lineage": "Digitized in 2019; generalized with Douglas-Peucker (10 m).",
  }
}
```

```

    "temporalExtent": { "start": "2019-01-01", "end": "2019-12-31" },
    "links": [{ "href": "https://example.org/rivers", "rel": "about", "title": "Project
page" }]
  }
}

```

모든 멤버는 선택 사항이며 모든 값은 문자열(또는 문자열 목록)입니다. `license` 는 [SPDX 식별자](#) 또는 자유 텍스트입니다. `temporalExtent` 의 경계는 ISO 8601 날짜(YYYY-MM-DD) 또는 날짜-시간이며, 열린 구간을 위해 어느 한 쪽을 생략할 수 있습니다. 링크에는 절대 `href` 가 필요합니다. `rel` 은 링크 관계(생략 시 `related`)이고 `title` 은 라벨입니다. 로드 시 비어 있는 멤버, 비어 있거나 중복된(대소문자 구분 없음) 키워드, `href` 가 없는 링크는 제거되며, 아무 내용도 남지 않은 블록은 삭제됩니다. 빈 `descriptiveMetadata` 는 절대 기록되지 않습니다. 잘못된 값(도메인이 없는 이메일, 불가능한 날짜)은 대화 상자에서 표시하고 수정할 수 있도록 유지되지만, 대화 상자는 이를 저장하지 않습니다.

대화 상자의 **Export as STAC Item**(STAC Item으로 내보내기)은 이 블록을 [STAC 1.0 Item](#)으로 기록합니다: 레이어 `id`를 `id` 로, 레이어의 WGS84 범위를 `geometry / bbox` 로, `title` (설정되지 않았으면 레이어 이름), `abstract` 를 `description` 으로, `keywords`, `license`, 연락처를 `producer` 제공자로, `lineage` 를 `processing:lineage` 로(`processing` 확장을 선언하여), 시간 범위를 `datetime` 또는 `start_datetime / end_datetime` 으로, 링크들, 그리고 원격 소스 URL에 대한 `data` 자산(`{z}/{x}/{y}` 타일 템플릿이면 `tiles` 자산)으로 기록합니다. [SPDX 형식 식별자](#)가 아닌 라이선스는 `proprietary` 로 내보내지며 그 텍스트는 `geolibre:license` 에 들어가고, 출처 표시는 `geolibre:attribution` 에 들어갑니다. STAC는 `datetime` 을 요구하므로, 시간 범위가 없는 레이어에는 내보내기 시각이 찍힙니다. 벡터 레이어를 GeoParquet로 내보내면 이 블록이 JSON으로 파일의 Parquet 키-값 메타데이터에 `geo` 키 옆의 `geolibre:metadata` 아래에 기록됩니다.

레이어 타입

타입	상태
<code>geojson</code>	가져온 파일과 GeoJSON URL에 대해 지원됩니다
<code>xyz</code>	래스터 타일 템플릿에 대해 지원됩니다
<code>wms</code>	타일 방식 WMS GetMap 레이어로 지원됩니다
<code>wmts</code>	WMTS 래스터 타일 템플릿에 대해 지원됩니다
<code>raster</code>	래스터 타일 템플릿에 대해 지원됩니다. <code>source.ionAssetId</code> 와 <code>metadata.sourceKind: "cesium-ion"</code> 이 있으면, 3D 지구본이 앱의 Ion 토큰으로 로드하는 Cesium Ion 영상 자산입니다(3D 전용)
<code>vector-tiles</code>	MapLibre 벡터 타일 소스에 대해 지원됩니다
<code>mbtiles</code>	데스크톱 앱에서 로컬 MapLibre 프로토콜을 통해 지원됩니다
<code>arcgis</code>	ArcGIS VectorTileServer 레이어에 대해 지원됩니다(FeatureServer 레이어는 <code>geojson</code> 으로, MapServer/ImageServer 레이어는 <code>raster</code> 로 저장됩니다)
<code>pmtiles</code>	Components 플러그인을 통해 지원됩니다
<code>cog</code>	COG 및 GeoTIFF 래스터 레이어에 대해 지원됩니다

타입	상태
flatgeobuf	Components 플러그인을 통해 지원되며, 로컬 벡터 파일로 로드하면 GeoJSON으로 가져옵니다
zarr	Components 플러그인을 통해 지원됩니다
lidar	Components 플러그인을 통해 지원됩니다
gaussian-splat	Components 플러그인을 통해 지원됩니다
geoparquet	DuckDB-WASM을 통해 GeoJSON으로 가져옵니다
duckdb-query	SQL 쿼리 결과 레이어에 대해 지원됩니다
deckgl-viz	Deck.gl Layer 플러그인을 통해 지원됩니다
video	지리참조된 비디오 오버레이에 대해 지원됩니다
image	지리참조된 이미지 오버레이에 대해 지원됩니다(KML 지상 오버레이, Georeferencer, NetCDF 그리드)
3d-tiles	maplibre-gl-3d-tiles 패키지를 통해 지원됩니다. source.ionAssetId와 metadata.sourceKind: "cesium-ion" 이 있으면, 3D 지구본이 앱의 Ion 토큰으로 로드하는 Cesium Ion 타일셋입니다(3D 전용)

API

```
import {
  createEmptyProject,
  parseProject,
  redactCredentials,
  serializeProject,
} from "@geolibre/core";
```

자격 증명 삭제

로컬 프로젝트에는 인증된 데이터를 복원하는 데 필요한 자격 증명(레이어 요청 헤더, 지오코딩 API 키, 환경 변수, 플러그인 설정 등)이 포함될 수 있습니다. 로컬 워크스페이스를 떠나는 모든 프로젝트는 먼저 `redactCredentials(project)` 를 거쳐야 합니다. GeoLibre는 이 불변 조건을 Share(공유), 독립 실행형 HTML 내보내기, 임베드 스냅샷, 협업 스냅샷에 적용합니다. 데스크톱 앱에서는 지오코딩 API 키와 고유한 이름을 가진 비밀 환경 변수가 파일 대신 OS 키체인으로 이동합니다. [자격 증명 저장소](#)를 참조하세요. 중복되거나 이름이 없는 비밀 환경 변수 행은 고유한 계정 아래에 저장할 수 없으므로, Local Save(로컬 저장)와 Save As(다른 이름으로 저장)는 키체인을 사용할 수 없거나 웹 빌드에 프로젝트 자격 증명이 포함된 경우에 사용하는 것과 동일한 유지/제거 프롬프트를 제공합니다.

레이어의 `source.requestHeaders` 값은 `${NAME}` 형태로 환경 변수를 참조할 수 있습니다(예: `"Authorization": "Bearer ${TILES_TOKEN}"`). 참조는 작성된 그대로 저장되며 요청 시점에 활성화된 `environmentVariables` 행에서 해석됩니다. 변수가 설정되지 않았거나 비어 있는 헤더는 전송되지 않습니다. 선택적 스킴 단어 하나와 참조 하나로만 이루어진 값은 삭제 과정에서 살아남으며, 그 밖의 모든 헤더 값은 제거됩니다.

각 `preferences.environmentVariables` 행은 `"secret": false` 를 가질 수 있습니다. 이 값이 없는 행은 비밀입니다: 삭제 과정에서 제거되며, 데스크톱 앱은 고유하고 비어 있지 않은 이름을 가진 경우 그 값을 키체인에 저장합니다. `"secret": false` 로 표시된 행은 일반 값입니다: 파일에 그대로 남으며 Share와 내보내기에서도 유지됩니다.

저장된 모델과 처리 기록의 매개변수 묶음은 현재 자격 증명을 받지 않으며 구조적 프로젝트 콘텐츠로 취급됩니다. 향후 처리 도구가 자격 증명을 받게 된다면, 해당 필드를 중앙 삭제 레지스트리와 순회 로직에 추가해야 합니다.

삭제 과정은 자격 증명에 담긴 필드와 URL 내 인증 매개변수를 제거하면서 비밀이 아닌 브로커 참조는 보존합니다. 외부 플러그인 설정은 임의의 내용이므로 기본적으로 송출 스냅샷에서 생략됩니다. 수신자는 플러그인 매니페스트와 활성화 메타데이터는 유지하지만, 자신의 설정은 직접 구성해야 합니다.

<https://geolibre.app/plugin-api/>

GeoLibre 플러그인 API

인터페이스

```
import type { Feature, FeatureCollection, Geometry } from "geojson";
import type { IControl } from "maplibre-gl";
import type * as Proj4 from "proj4";

export type GeoLibreMapControlPosition =
  | "top-left"
  | "top-right"
  | "bottom-left"
  | "bottom-right";

export type GeoLibreBuiltInMapControl =
  | "navigation"
  | "fullscreen"
  | "geolocate"
  | "globe"
  | "terrain"
  | "scale"
  | "attribution"
  | "logo"
  | "maptoolkit-logo"
  | "layer-control";

export interface GeoLibrePlugin {
  id: string;
  name: string;
  version: string;
  activeByDefault?: boolean;
  /**
   * Renderers this plugin supports. Defaults to `["maplibre"]`.
   * Engine-neutral plugins (e.g. catalog/service browsers that only write to
   * the GeoLibre store) or plugins with multi-engine adapters declare
   * `["maplibre", "cesium"]`; plugins that stay on the Style Spec surface the
   * two 2D engines share declare `["maplibre", "mapbox"]` (see "Supporting the
   * Mapbox renderer" below). The Plugins menu gates options against the active
   * renderer.
   */
  engines?: ("maplibre" | "mapbox" | "cesium" | "arcgis")[];
  /** Plugins in the same group cannot be active at the same time. */
  exclusiveGroup?: string;
  /** At least one name is required for handleUrlParameters to be called. */
  urlParameterNames?: string[];
  // Return (or resolve) false to refuse activation; the host then rolls back
  // the optimistic active state.
  activate: (app: GeoLibreAppAPI) => boolean | void | Promise<boolean | void>;
  deactivate: (app: GeoLibreAppAPI) => void;
  handleUrlParameters?: (
    app: GeoLibreAppAPI,
    params: URLSearchParams
  ) => void | Promise<void>;
  getMapControlPosition?: () => GeoLibreMapControlPosition;
  setMapControlPosition?: (
    app: GeoLibreAppAPI,
    position: GeoLibreMapControlPosition
  ) => boolean | void;
  getProjectState?: () => unknown;
  applyProjectState?: (app: GeoLibreAppAPI, state: unknown) => boolean | void;
}
```

```

/**
 * The plugin saves its own panel open/collapsed state through
 * getProjectState/applyProjectState, so the project-restore pass must not
 * collapse its control.
 */
restoresPanelCollapseState?: boolean;
/**
 * The plugin's project state is project data, not a preference: every
 * project load calls applyProjectState, with undefined when the file has no
 * state for it, so the previous project's data is not carried over.
 */
clearsStateOnProjectLoad?: boolean;
/**
 * A workspace tool, not part of a project (e.g. the S3 Browser panel):
 * project loads and map swaps leave it running, and it is never saved in
 * the project's active plugins. Only for plugins with no map controls.
 */
sessionScoped?: boolean;
}

// Resolved by app.getDeckGL(): GeoLibre's own deck.gl modules, so a plugin
// renders on the host's single instance instead of bundling its own copy.
export interface GeoLibreDeckGL {
  core: typeof import("@deck.gl/core");
  layers: typeof import("@deck.gl/layers");
  aggregationLayers: typeof import("@deck.gl/aggregation-layers");
  geoLayers: typeof import("@deck.gl/geo-layers");
  meshLayers: typeof import("@deck.gl/mesh-layers");
  mapbox: typeof import("@deck.gl/mapbox");
}

export interface GeoLibreLayerSummary {
  id: string;
  name: string;
  type: string;
  visible: boolean;
  opacity: number;
}

export interface GeoLibreLayerGroupSummary {
  id: string;
  name: string;
  parentId: string | null; // null for a group at the panel root
  visible: boolean;
  opacity: number;
  collapsed: boolean;
}

export interface GeoLibreSelection {
  layerId: string | null;
  features: Feature<Geometry | null>[];
}

export type GeoLibreActiveMapTool = "identify" | "feature-selection" | null;

export interface GeoLibreRasterWindowOptions {
  bounds: [number, number, number, number]; // WGS84 [west, south, east, north]
  width?: number; // sample grid, default 32
  height?: number;
  band?: number; // 1-based, default 1
  signal?: AbortSignal;
}

export interface GeoLibreRasterWindowReading {
  values: number[]; // row-major, width * height, nodata included
  width: number;
  height: number;
  band: number;
}

```

```

    noData: number | null;
    overviewLevel: number;
}

// An excerpt: the full interface, with every optional method, is in
// packages/plugins/src/types.ts.
export interface GeoLibreAppAPI {
  setBasemap: (styleUrl: string) => void;
  addGeoJsonLayer: (
    name: string,
    data: FeatureCollection,
    sourcePath?: string
  ) => string;
  listLayers?: () => GeoLibreLayerSummary[];
  getLayerFeatures?: (layerId: string) => Feature<Geometry | null>[];
  // Dress a layer with an SLD, QML or Mapbox GL style. See "Layer styles".
  importLayerStyle?: (
    layerId: string,
    text: string
  ) => GeoLibreImportLayerStyleResult;
  getSelectedFeatures?: () => Feature<Geometry | null>[];
  getSelectedLayerId?: () => string | null;
  // Sample a raster layer over a geographic window. See "Sampling raster
  // values" below.
  readRasterWindow?: (
    layerId: string,
    options: GeoLibreRasterWindowOptions
  ) => Promise<GeoLibreRasterWindowReading | null>;
  getDrawnFeatures?: () => Feature<Geometry | null>[];
  onSelectionChange?: (
    callback: (selection: GeoLibreSelection) => void
  ) => () => void;
  getActiveMapTool?: () => GeoLibreActiveMapTool;
  onActiveMapToolChange?: (
    callback: (tool: GeoLibreActiveMapTool) => void
  ) => () => void;
  // Native raster/tile layers (see "Raster and tile layers" below). Each
  // returns the new layer's id and the layer appears in the Layers panel and
  // persists with the project, like addGeoJsonLayer does for vector data.
  addTileLayer?: (
    name: string,
    url: string,
    options?: GeoLibreTileLayerOptions
  ) => string;
  addWmtsLayer?: (
    name: string,
    url: string,
    options?: GeoLibreTileLayerOptions
  ) => string;
  addWmsLayer?: (name: string, options: GeoLibreWmsLayerOptions) => string;
  addWfsLayer?: (name: string, options: GeoLibreWfsLayerOptions) => Promise<string>;
  // Native client-side COG (reads the GeoTIFF directly; band/rescale/colormap/
  // noData controls). Resolves with the new layer's id (see "Raster and tile
  // layers" below).
  addCogLayer?: (
    name: string,
    url: string,
    options?: GeoLibreCogLayerOptions
  ) => Promise<string>;
  // Zarr through the host's own @carbonplan/zarr-layer instance, with
  // crs/proj4 reprojection (see "Zarr layers" below).
  addZarrLayer?: (
    name: string,
    url: string,
    options: GeoLibreZarrLayerOptions
  ) => Promise<string>;
  setZarrLayerSelector?: (
    layerId: string,

```

```

    selector: Record<string, number | string>
  ) => Promise<boolean>;
  // Click-to-value / region statistics on a Zarr layer (see "Zarr layers").
  queryZarrLayer?: (
    layerId: string,
    geometry: GeoLibreZarrQueryGeometry,
    selector?: GeoLibreZarrQuerySelector,
    options?: GeoLibreZarrQueryOptions
  ) => Promise<GeoLibreZarrQueryResult | null>;
  // Register a layer the plugin added to the map itself, so it appears in the
  // Layers panel (see "Custom (WebGL) layers and paint ownership" below).
  registerExternalNativeLayer?: (
    layer: GeoLibreExternalNativeLayerRegistration
  ) => void;
  unregisterExternalNativeLayer?: (id: string) => void;
  // Layers-panel groups (folders). See "Layer groups" below.
  addLayerGroup?: (name?: string, layerIds?: string[]) => string;
  listLayerGroups?: () => GeoLibreLayerGroupSummary[];
  moveLayersToGroup?: (layerIds: string[], groupId: string | null) => void;
  moveLayerGroupToGroup?: (id: string, parentId: string | null) => void;
  removeLayerGroup?: (id: string) => void;
  // Time Slider adapters (see "Driving a layer's own time dimension" below).
  registerTemporalLayer?: (
    layerId: string,
    adapter: TemporalLayerAdapter,
    options?: { bind?: boolean }
  ) => () => void;
  unregisterTemporalLayer?: (layerId: string) => void;
  // See "Activating and deactivating other plugins" below.
  activatePlugin?: (pluginId: string, state?: unknown) => Promise<boolean>;
  deactivatePlugin?: (pluginId: string) => boolean;
  getActiveBasemap: () => string;
  onBasemapChange: (callback: (styleUrl: string) => void) => () => void;
  fetchArrayBuffer?: (url: string) => Promise<ArrayBuffer>;
  // Desktop only: fetch through the native client (see "Signing in with a session cookie").
  nativeFetch?: typeof globalThis.fetch;
  fitBounds?: (bounds: [number, number, number, number]) => void;
  // The extent the primary map currently shows, [west, south, east, north] in
  // degrees, on either renderer. The engine-neutral replacement for
  // getMap()?.getBounds() – see "Reading the viewport" below.
  getViewBounds?: () => [number, number, number, number] | null;
  getMap?: () => import("maplibre-gl").Map | null;
  // The active primary renderer, including while its canvas is being replaced.
  getMapRenderer?: () => "maplibre" | "mapbox" | "cesium" | "arcgis";
  // The native mapbox-gl map, only while Mapbox is the primary renderer; null
  // otherwise. Built-in plugins read the 2D map through getStyleMap(app), which
  // falls back to this when getMap() is null – see "Supporting the Mapbox
  // renderer" below.
  getMapboxMap?: () => import("mapbox-gl").Map | null;
  // The mapbox-gl namespace and the access token, only while Mapbox is the
  // primary renderer, for a plugin that must construct Mapbox classes or a
  // second Mapbox map.
  getMapboxGl?: () => typeof import("mapbox-gl").default | null;
  getMapboxAccessToken?: () => string | null;
  // The primary ArcGIS MapView or SceneView, or null on another renderer.
  // The shared deck overlay hosts flat maps and local scenes only.
  getArcgisView?: () => ReturnType<
    import("@geolibre/map").ArcgisEngine["getView"]
  >;
  // The MapLibre-shaped map controls receive on ArcGIS, or null on another
  // renderer. Its style calls are recorded, not drawn by the SDK: the host
  // draws the store layers they mirror and GeoJSON overlays itself (see
  // docs/arcgis-renderer.md, "Plugin controls"). Built-in plugins read it
  // through getControlMap(app) in style-map.ts.
  getArcgisControlMap?: () => import("maplibre-gl").Map | null;
  // The primary Cesium globe's scene (namespace, widget, scene, camera, clock,
  // canvas, readView), or null when the primary map is not a globe. The globe's
  // counterpart to getMap for plugins that declare engines: ["maplibre", "cesium"].

```

```

getCesiumScene?: () => import("@geolibre/map").CesiumSceneHandle | null;
addMapControl: (
  control: IControl,
  position?: GeoLibreMapControlPosition
) => boolean;
removeMapControl: (control: IControl) => void;
// Note: showing the "terrain" control (visible: true) also switches 3D
// terrain on, mirroring the Controls menu so the user doesn't have to click
// the control button as a second step. Hiding it turns terrain back off.
setBuiltInMapControlVisible: (
  control: GeoLibreBuiltInMapControl,
  visible: boolean
) => boolean;
getBuiltInMapControlPosition: (
  control: GeoLibreBuiltInMapControl
) => GeoLibreMapControlPosition;
setBuiltInMapControlPosition: (
  control: GeoLibreBuiltInMapControl,
  position: GeoLibreMapControlPosition
) => boolean;
getDeckGL?: () => Promise<GeoLibreDeckGL>;
getProj4?: () => Promise<typeof Proj4>;
// Right-sidebar panels (see "Right sidebar panels" below).
registerRightPanel?: (panel: GeoLibreRightPanelRegistration) => () => void;
unregisterRightPanel?: (id: string) => void;
openRightPanel?: (id: string) => boolean;
collapseRightPanel?: (id: string) => void;
closeRightPanel?: (id: string) => void;
getActiveRightPanel?: () => string | null;
setActiveRightPanelDock?: (dock: GeoLibreRightPanelDock) => void;
getActiveRightPanelDock?: () => GeoLibreRightPanelDock | null;
// Localization (see "Following the app language" below).
getLocale?: () => string;
onLocaleChange?: (listener: (locale: string) => void) => () => void;
translate?: (
  key: string,
  defaultValue: string,
  params?: Record<string, string | number>
) => string;
registerTranslations?: (
  resources: Record<string, Record<string, string>>
) => void;
// Credential storage (see "Saving credentials" below).
credentials?: GeoLibrePluginCredentials;
// Top toolbar menus (see "Toolbar menus" below).
registerToolbarMenu?: (menu: GeoLibreToolbarMenu) => () => void;
unregisterToolbarMenu?: (id: string) => void;
// Items in the built-in menus (see "Adding items to built-in menus" below).
registerMenuContribution?: (
  contribution: GeoLibreMenuContribution
) => () => void;
unregisterMenuContribution?: (id: string) => void;
// Floating panels (see "Floating panels" below).
registerFloatingPanel?: (
  panel: GeoLibreFloatingPanelRegistration
) => () => void;
unregisterFloatingPanel?: (id: string) => void;
openFloatingPanel?: (id: string) => boolean;
closeFloatingPanel?: (id: string) => void;
getOpenFloatingPanels?: () => string[];
// AI Assistant tools and guidance (see "Assistant tools" below).
registerAssistantTool?: (tool: Tool) => () => void;
registerAssistantToolSpec?: (spec: AssistantToolSpec) => () => void;
registerAssistantGuidance?: (text: string) => () => void;
}

// A getter re-reads the label on every render (see "Toolbar menus" below).
export type GeoLibreToolbarLabel = string | (() => string);

```

```

export interface GeolibreToolbarMenu {
  id: string;
  label: GeolibreToolbarLabel;
  icon?: string; // URL or data: URI
  items: GeolibreToolbarMenuItem[];
}

export type GeolibreToolbarMenuItem =
  | {
    type?: "action";
    id: string;
    label: GeolibreToolbarLabel;
    icon?: string;
    disabled?: boolean;
    onSelect: () => void;
  }
  | {
    type: "submenu";
    id: string;
    label: GeolibreToolbarLabel;
    icon?: string;
    items: GeolibreToolbarMenuItem[];
  }
  | { type: "separator"; id?: string };

export interface GeolibreMenuContribution {
  id: string;
  menu: "addData" | "processing" | "controls";
  items: GeolibreToolbarMenuItem[];
}

export interface GeolibreFloatingPanelRegistration {
  id: string;
  // A getter makes the title re-localize live on language changes: it is
  // re-evaluated on every getFloatingPanel() call, so it picks up the current
  // language without re-registering the panel. Caveat: the registry itself
  // does not subscribe to i18n events, so the getter is only re-run when a
  // consumer re-reads the panel. In practice every host component that renders
  // the title also calls useTranslation(), whose languageChanged re-render
  // re-reads the panel as a side effect; a host that reads the title without
  // that subscription would show a stale title after a language switch until
  // the next registry mutation, and must re-read the panel itself on language
  // change. A plain string is frozen at registration time.
  title: string | (() => string);
  icon?: string; // URL or data: URI
  defaultWidth?: number;
  defaultHeight?: number; // omitted: the card sizes to its content
  position?: GeolibreMapControlPosition; // corner it first opens at, default "top-left"
  render: (container: HTMLElement) => void | (() => void);
  onOpen?: () => void;
  onClose?: () => void;
}

export type GeolibreRightPanelDock =
  | "left-of-layers" // left of the Layers panel
  | "right-of-layers" // between the Layers panel and the map
  | "left-of-style" // between the map and the Style panel
  | "right-of-style" // right of the Style panel
  | "replace-style" // share the Style sidebar's single rail (shared-rail mode, default)
  | "replace-layers"; // share the Layers sidebar's single rail (shared-rail mode)

export interface GeolibreRightPanelRegistration {
  id: string;
  // A getter makes the title re-localize live on language changes: it is
  // re-evaluated on every getRightPanel() call, so it picks up the current
  // language without re-registering the panel. Caveat: the registry itself
  // does not subscribe to i18n events, so the getter is only re-run when a

```

```
// consumer re-reads the panel. In practice every host component that renders
// the title also calls useTranslation(), whose languageChanged re-render
// re-reads the panel as a side effect; a host that reads the title without
// that subscription would show a stale title after a language switch until
// the next registry mutation, and must re-read the panel itself on language
// change. A plain string is frozen at registration time.
title: string | (() => string);
/** Initial dock position; "replace-style" (default). */
dock?: GeolibreRightPanelDock;
/** Optional rail icon: a URL or data: URI rendered as an image. */
icon?: string;
/** Preferred expanded width in px (desktop only; host-clamped). */
defaultWidth?: number;
/** Deactivate the owning plugin when the user closes the panel. */
deactivatePluginOnClose?: boolean;
/** Fill the panel body with your own DOM. May return a cleanup function. */
render: (container: HTMLElement) => void | (() => void);
onOpen?: () => void;
onCollapse?: () => void;
onClose?: () => void;
/** Called only on an explicit close, not when another panel displaces it. */
onExplicitClose?: () => void;
}
```

플러그인 등록

```
import { PluginManager } from "@geolibre/plugins";

const manager = new PluginManager();
manager.register(myPlugin);
manager.activate("my-plugin", appApi);
```

내장 플러그인 추가 (이 저장소에서)

내장 플러그인은 GeoLibre 자체와 함께 배포됩니다. 대부분의 플러그인은 내장될 필요가 **없습니다**. GeoLibre를 포크하지 않고 플러그인을 배포하려면 **외부 플러그인**을 참고하세요. 이 저장소에 내장 플러그인을 추가하려면 다음과 같이 합니다.

1. `packages/plugins/src/plugins/` 에 플러그인 파일을 만듭니다.

```
import type { GeolibreAppAPI, GeolibrePlugin } from "../types";

export const myPlugin: GeolibrePlugin = {
  id: "my-plugin",
  name: "My Plugin",
  version: "0.1.0",
  activate: (app: GeolibreAppAPI) => {
    app.setBasemap("https://example.com/style.json");
  },
  deactivate: () => {
    /* basemap remains until the user changes it */
  },
};
```


베이스맵 플러그인은 비활성화 시 자신이 적용한 스타일을 의도적으로 그대로 둡니다. 내장 `osm-basemap` 및 `carto-light` 플러그인도 마찬가지인데, 사용자가 보고 있는 지도를 되돌리는 것이 마지막으로 선택한 것을 유지하는 것보다 더 당황스럽기 때문입니다. 컨트롤, 리스너 또는 레이어를 추가하는 플러그인은 `deactivate()` 에서 반드시 이를 되돌려야 합니다.

1. `packages/plugins/src/index.ts` 에서 내보냅니다.

```
export { myPlugin } from "../plugins/my-plugin";
```

1. `apps/geolibre-desktop/src/hooks/usePlugins.ts` 의 `BUILT_IN_PLUGINS` 배열에 추가하여 등록합니다. 이 배열은 `manager.registerAll` 에 전달됩니다.

```
import { myPlugin } from "@geolibre/plugins";

const BUILT_IN_PLUGINS: GeoLibrePlugin[] = [
  maplibreLayerControlPlugin,
  maplibreGeoEditorPlugin,
  // ...
  myPlugin,
];
manager.registerAll(BUILT_IN_PLUGINS);
```

MapLibre 컨트롤 플러그인의 경우 패키지 의존성을 추가한 다음 `activate()` 에서 `app.addMapControl(control, "top-left")` 를, `deactivate()` 에서 `app.removeMapControl(control)` 을 호출합니다. 컨트롤의 npm 패키지가 자체 스타일시트를 제공한다면, 기존 `maplibre-gl-*/style.css` 임포트와 나란히 `apps/geolibre-desktop/src/main.tsx` 에서 해당 스타일시트를 임포트하세요. 이는 의존성 자체의 CSS에만 해당하며, 그 위에 얹는 앱 전용 수정은 아래 [서드파티 컨트롤 스타일링](#)에서 설명하듯 `index.css` 에 넣어야 합니다.

Navigation, Fullscreen, Geolocate, Globe, Terrain, Scale, Attribution, Logo와 같은 내장 MapLibre 컨트롤은 데스크톱 앱의 **Controls**(컨트롤) 메뉴에서 켜고 끕니다. 이 메뉴는 Components 플러그인이 뒷받침하는 독립형 장소 검색 패널인 **Search**(검색)도 엽니다. Layer Control 및 Components처럼 플러그인 API를 사용하거나 플러그인 수명 주기 동작이 필요한 프로젝트 전용 컨트롤은 **Plugins**(플러그인) 메뉴에 두세요.

Components 플러그인은 `maplibre-gl-components` 컨트롤을 감싸고 그 레이어 이벤트를 GeoLibre 스토어에 연결합니다. FlatGeobuf, PMTiles, Zarr, LiDAR, Gaussian splat을 위한 **Add Data**(데이터 추가) 바로가기를 제공하며, 래스터 COG 및 GeoTIFF 레이어는 표준 **Add Raster Layer**(래스터 레이어 추가) 대화상자를 통해서도 추가할 수 있습니다.

서드파티 컨트롤 스타일링

서드파티 MapLibre 컨트롤에 앱 전용 스타일 수정이 필요하다면 `node_modules` 의 파일을 편집하는 대신 `apps/geolibre-desktop/src/index.css` 에 범위를 한정된 오버라이드를 추가하세요. 선택자는 해당 플러그인의 컨트롤 클래스로 제한해야 합니다. 예를 들어 GeoEditor 툴바 버튼은 MapLibre의 기본 컨트롤 버튼 CSS가 flex 가운데 정렬을 덮어쓸 수 있으므로 로컬 오버라이드가 필요합니다.

```
.geo-editor-control .geo-editor-tool-button {
  align-items: center;
  display: flex !important;
  justify-content: center;
  line-height: 0;
  padding: 0;
}

.geo-editor-control .geo-editor-tool-button svg {
  display: block;
  flex: 0 0 auto;
  margin: 0;
}
```

변경 사항을 제출하기 전에 `npm run build` 와 `pre-commit run --files <changed files>` 를 실행하세요. docs/ 아래의 페이지도 수정했다면 사이트를 빌드하세요. CI는 `zensical build --strict` 를 실행하므로 깨진 링크나 `mkdocs.yml` 의 `nav` 에 빠진 페이지가 있으면 빌드가 실패합니다. 두 게이트의 전체 내용은 [기여하기](#)를 참고하세요.

내장 플러그인

내장 플러그인 중 일부를 소개합니다.

ID	설명
osm-basemap	OpenFreeMap Liberty 스타일
carto-light	CARTO Positron GL 스타일
maplibre-gl-basemap-control	MapLibre 베이스맵 선택기를 추가합니다
maplibre-gl-components	MapLibre Components 컨트롤 그리드와 FlatGeobuf, COG, PMTiles, Zarr, LiDAR, Gaussian splat용 패널을 추가합니다
maplibre-gl-geo-editor	GeoEditor 그리기 컨트롤을 추가합니다
maplibre-gl-dimensions	Dimension 도구(선형/각도 CAD 스타일 치수선, 선택적 정점 스냅 지원)를 추가합니다
maplibre-gl-geoagent	GeoAgent 지도 어시스턴트 컨트롤을 추가합니다
maplibre-gl-usgs-lidar	USGS 3DEP LiDAR 포인트 클라우드를 검색하고 스트리밍합니다
geolibre-ign-lidar-hd	IGN LiDAR HD 타일 커버리지(WFS)를 검색하고 포인트 클라우드(COPC LAZ) 파일을 다운로드합니다. 실제 네트워크 테스트는 <code>RUN_LIVE_TESTS</code> 로 선택 실행합니다(<code>tests/ign-lidar-hd.test.ts</code> 참고)
maplibre-gl-streetview	스트리트 뷰 컨트롤을 추가합니다

ID	설명
maplibre-gl-swipe	지도 스와이프 컨트롤을 추가합니다

예제 플러그인

```
import type { GeoLibreAppAPI, GeoLibrePlugin } from "@geolibre/plugins";

export const myPlugin: GeoLibrePlugin = {
  id: "my-plugin",
  name: "My Plugin",
  version: "0.1.0",
  activate(app: GeoLibreAppAPI) {
    app.setBasemap("https://example.com/style.json");
  },
  deactivate() {
    // Clean up controls, listeners, and plugin state here.
  },
};
```

지도 컨트롤 플러그인은 선택적으로 `getMapControlPosition()` 과 `setMapControlPosition()` 을 노출하여 데스크톱 **Plugins**(플러그인) 메뉴에서 컨트롤을 지도 모서리 사이로 옮길 수 있게 할 수 있습니다. 위치를 인식하는 플러그인은 위치가 바뀌면 컨트롤을 제거한 뒤 다시 만들거나 다시 추가해야 합니다.

직렬화 가능한 런타임 설정을 가진 플러그인은 `getProjectState()` 와 `applyProjectState()` 를 노출하여 GeoLibre가 해당 설정을 프로젝트 파일에 저장하고 복원할 수 있게 할 수 있습니다. 래퍼는 이 혹은 사용해 `getState()` 와 같은 업스트림 컨트롤 API를 적응시켜야 하며, 모든 업스트림 패키지가 GeoLibre 전용 인터페이스를 구현할 필요가 없도록 해야 합니다.

deck.gl로 렌더링하는 플러그인은 `app.getDeckGL()` (프로미스를 반환)을 호출하여 GeoLibre 자체의 deck.gl 모듈인 `core`, `layers`, `aggregationLayers`, `geoLayers`, `meshLayers`, `mapbox` 를 얻어야 합니다(MapLibre와 인터리브 렌더링하려면 `mapbox.MapboxOverlay` 를 사용). 두 번째 사본을 번들링하지 말고 호스트의 단일 deck.gl 인스턴스 위에 렌더링하세요. deck.gl과 luma.gl은 버전이 맞지 않으면 예외를 던지고 전역 싱글톤을 공유하므로, 번들된 사본은 렌더링에 실패합니다. 호스트 변형에 따라 deck.gl이 포함되지 않을 수 있으므로 옵셔널 체이닝 (`app.getDeckGL?.()`)으로 호출하세요.

공유 투영 라이브러리

외부 플러그인은 `app.getProj4()` 를 사용해 호스트의 proj4 모듈 네임스페이스를 얻을 수 있습니다. 그 `default` 내보내기가 호출 가능한 라이브러리입니다.

```
import type * as Proj4 from "proj4";

const module: typeof Proj4 | undefined = await app.getProj4?.();
if (!module) throw new Error("This plugin requires a host with getProj4 support.");
const proj4 = module.default;
proj4.defs("my-plugin:utm32n", "+proj=utm +zone=32 +datum=WGS84 +units=m +no_defs");
const [longitude, latitude] = proj4("my-plugin:utm32n", "EPSG:4326", [500000, 0]);
```

이 코드는 경도 9, 위도 0을 반환합니다. 좌표 변환과 CRS 정의에 대해서는 [proj4 API](#)를 참고하세요. 변경 가능한 정의 레지스트리는 호스트(ArcGIS/Zarr 재투영 포함) 및 모든 플러그인과 공유됩니다. 위와 같이 플러그인 접두사가 붙은 이름만 등록하고, EPSG:* 이름이나 호스트 또는 다른 플러그인에 속한 이름은 등록하거나 재정의하지 마세요. 충돌하는 정의는 이후에 생성되는 변환을 조용히 손상시킬 수 있습니다. 이러한 명명 규칙은 플러그인 작성자의 책임이며, 이 API가 강제하는 격리 경계가 아닙니다. 알 수 없는 EPSG 정의는 자동으로 가져오지 않습니다.

일회성 변환에는 공유 레지스트리를 수정하는 대신 정의를 직접 전달하세요.

```
const [longitude, latitude] = proj4(
  "+proj=utm +zone=32 +datum=WGS84 +units=m +no_defs",
  "EPSG:4326",
  [500000, 0],
);
```

오래된 호스트나 변형 호스트에는 `getProj4` 가 없을 수 있으므로 플러그인은 그 부재를 처리해야 합니다. 공유 접근이 `addGeoJsonLayer` 의 WGS84 경도/위도 요구 사항을 바꾸지는 않습니다. 레이어를 추가하기 전에 원본 좌표를 재투영하세요. 이 API를 사용할 때는 플러그인 번들에서 `proj4`의 런타임 임포트와 의존성을 제외하세요. 위의 네임스페이스 임포트처럼 타입 전용 참조에는 런타임 사본이 필요 없습니다.

플러그인은 URL 쿼리 매개변수를 선언하고 GeoLibre가 열릴 때 이를 처리할 수도 있습니다. URL 매개변수 핸들러는 지도가 준비되고, 외부 플러그인이 로드되고, 프로젝트 플러그인 상태가 복원된 뒤에 실행됩니다. GeoLibre는 선언된 매개변수 이름이 URL에 있는 플러그인의 핸들러를 호출하며, 같은 플러그인에 대해 같은 URL 컨텍스트가 반복 처리되는 것을 억제합니다. 일치하는 플러그인이 등록(설치)되어 있지만 비활성 상태라면 GeoLibre는 먼저 `PluginManager.activate` 로 활성화를 시도하고, 활성화가 성공한 경우에만 핸들러를 실행합니다(`activate()`가 `false` 를 반환하거나 예외를 던지면 플러그인은 비활성 상태로 남고 디스패치는 건너뛵니다). URL 쿼리 매개변수가 그렇듯 매개변수 이름은 대소문자를 구분합니다. `exampleGeoJson` 을 선언하면 `?ExampleGeoJson=...` 과는 일치하지 않습니다.

```
import type { GeoLibreAppAPI, GeoLibrePlugin } from "@geolibre/plugins";

export const plugin: GeoLibrePlugin = {
  id: "example-url-loader",
  name: "Example URL Loader",
  version: "0.1.0",
  urlParameterNames: ["exampleGeoJson"],
  activate() {
    // Set up controls or plugin state here.
  },
  deactivate() {
    // Clean up controls, listeners, and plugin state here.
  },
  async handleUrlParameters(app: GeoLibreAppAPI, params: URLSearchParams) {
    for (const dataUrl of params.getAll("exampleGeoJson")) {
      // URL parameter values are attacker-controlled: only fetch HTTPS URLs
      // and verify the origin is one you trust before loading. Parsing the
      // value rejects malformed URLs, and the protocol check blocks
      // non-HTTPS schemes (file://, data:, http://); neither protects
      // against SSRF to loopback or private-network addresses.
      let parsedUrl: URL;
      try {
        parsedUrl = new URL(dataUrl);
      } catch {
        continue;
      }
    }
  }
}
```

```

    if (parsedUrl.protocol !== "https:") continue;
    const response = await fetch(parsedUrl.href);
    if (!response.ok) continue;
    app.addGeoJsonLayer("Example URL layer", await response.json(), dataUrl);
  },
},
};

```

URL 매개변수 값은 사용하기 전에 검증하세요. 누구나 GeoLibre로 향하는 링크를 만들 수 있으므로, 매개변수 값을 가져오는 핸들러는 예상치 못한 스킴(`file://` , `data:` , 일반 `http://`)을 거부하고 신뢰하는 출처에만 접속해야 합니다.

예를 들면 다음과 같습니다.

```

https://web.geolibre.app/?url=https://example.com/
project.geolibre.json&exampleGeoJson=https://example.com/data.geojson

```

URL 매개변수는 이를 소유한, 이미 등록(설치)된 플러그인만 활성화하며, URL에서 플러그인을 로드하지는 않습니다. 외부 플러그인의 경우 URL 핸들러에 의존하기 전에 플러그인 매니페스트 URL을 프로젝트의 `plugins` 상태에 포함하세요(그래야 플러그인이 등록됩니다). 그러면 일치하는 매개변수가 활성 집합에 없는 플러그인이라도 활성화하고 디스패치합니다.

읽기 전용 레이어 및 피처 조회

외부 플러그인은 현재 레이어 목록, 레이어의 GeoJSON 피처, 현재 피처 선택, GeoEditor의 Sketches 레이어에 있는 피처를 조회할 수 있습니다. 이 메서드들은 선택 사항이므로 같은 플러그인이 이전 호스트와도 호환됩니다.

```

const layers = app.listLayers?.() ?? [];
const selectedLayerId = app.getSelectedLayerId?.() ?? null;
const selectedFeatures = app.getSelectedFeatures?.() ?? [];

if (selectedLayerId && app.getLayerFeatures) {
  const layerFeatures = app.getLayerFeatures(selectedLayerId);
  console.log(layers, layerFeatures, selectedFeatures);
}

const drawnFeatures = app.getDrawnFeatures?.() ?? [];

```

`getSelectedFeatures` 는 가장 최근에 선택된 피처만이 아니라 선택된 레이어의 선택된 피처 전체를 반환합니다. GeoJSON `id` 가 없는 피처는 0부터 시작하는 배열 인덱스를 문자열로 변환한 값으로 매칭됩니다. 선택이 비어 있으면 빈 배열을 반환합니다. `getLayerFeatures` 는 레이어 `id` 를 알 수 없으면 예외를 던지고, GeoJSON 피처가 없는 레이어에 대해서는 빈 배열을 반환합니다.

선택 구독은 선택된 레이어 또는 선택된 피처 `id` 배열이 바뀐 뒤에 호출됩니다. 반환된 구독 해제 함수를 보관했다가 플러그인 비활성화 시 호출하세요.

```

const unsubscribe = app.onSelectionChange?.(({ layerId, features }) => {
  console.log(layerId, features);
});

```

```
// In deactivate or another cleanup path:
unsubscribe?();
```

이 메서드들은 읽기 전용 조회 표면입니다. 호출해도 GeoLibre 스토어는 변경되지 않습니다. 플러그인은 반환된 GeoJSON 피쳐도 읽기 전용으로 취급해야 하며, 데이터를 추가해야 할 때는 `addGeoJsonLayer` 와 같은 호스트 API를 사용해야 합니다.

활성 지도 클릭 도구

지도 클릭이나 호버를 처리하는 플러그인은 GeoLibre 지도 도구가 포인터를 소유하고 있는 동안 자신의 상호작용을 표시하지 않을 수 있습니다. 선택적인 실시간 게터와 구독은 여러 지도 렌더러에 걸쳐 Identify와 피쳐 선택을 다루며, 커서 스타일이나 저장된 프로젝트 스냅샷에 의존하지 않습니다.

```
const currentTool = app.getActiveMapTool?.() ?? null;
if (currentTool !== null) closePluginPopup();
const unsubscribe = app.onActiveMapToolChange?.((tool) => {
  if (tool !== null) closePluginPopup();
});

function handleMapClick(event: unknown) {
  if ((app.getActiveMapTool?.() ?? null) !== null) return;
  openPluginPopup(event);
}

function handleMapHover(event: unknown) {
  if ((app.getActiveMapTool?.() ?? null) !== null) return;
  showPluginTooltip(event);
}

// In deactivate or another cleanup path:
unsubscribe?();
```

`getActiveMapTool` 은 Identify가 활성화된 동안 "identify" 를, 선택 제스처가 지도 클릭을 소유하는 동안 "feature-selection" 을, 그 외에는 `null` 을 반환합니다. 두 상태가 잠시 겹치면 피쳐 선택이 우선합니다. Identify 대상을 바꾸는 것은 도구 변경 이벤트를 발생시키지 않습니다. 구독자는 등록 이후의 변경만 받고 초기 콜백은 받지 않으므로, 플러그인 상태를 초기화하려면 게터를 읽으세요. 반환된 구독 해제 함수를 보관했다가 비활성화 시 호출하세요. 활성 선택 제스처를 다른 제스처로 교체하면 중간에 `null` 을 거치지 않고 "feature-selection" 이 활성 상태로 유지됩니다.

두 메서드 모두 선택 사항이므로 플러그인은 이전 호스트와 호환됩니다.

레이어 그룹

관련 레이어를 여러 개 추가하는 플러그인은 레이어를 패널 루트에 홀어 두는 대신 Layers 패널 그룹(폴더)에 넣을 수 있으며, 사용자가 직접 하는 것과 같은 방식으로 그룹을 다른 그룹 안에 중첩할 수 있습니다.

```
// Create a folder, optionally moving existing layers into it. Returns its id.
const basins = app.addLayerGroup?.("Basins", [catchmentLayerId]) ?? null;

// Append to a folder you created earlier rather than creating a second one
```

```
// with the same name. A null group id lifts the layers back to the root.
app.moveLayersToGroup?([outletLayerId], basins);

// Nest a folder inside another one. A null parent lifts it back to the root.
const subBasins = app.addLayerGroup?("Sub-basins");
if (basins && subBasins) app.moveLayerGroupToGroup?(subBasins, basins);

// Remove the folder without removing the layers inside it.
if (subBasins) app.removeLayerGroup?(subBasins);
```

`moveLayerGroupToGroup`은 `moveLayersToGroup`의 그룹-대-그룹 버전으로, Layers 패널 자체의 **Move to group**(그룹으로 이동) 메뉴가 수행하는 것과 같은 부모 재지정을 합니다. 두 id 중 하나라도 알 수 없거나, 그룹이 이미 해당 부모 안에 있거나, 이동으로 인해 그룹이 자기 자신의 조상이 되는 경우에는 아무 동작도 하지 않습니다. 호스트가 트리를 손상시키는 대신 순환을 거부하므로, 플러그인이 직접 부모 체인을 따라갈 필요가 없습니다.

`addLayerGroup`은 플러그인이 만드는 폴더의 그룹 id를 얻는 유일한 방법입니다. 플러그인이 만들지 않은 폴더(사용자가 만들었거나 다른 플러그인이 만든 폴더)를 다루려면 먼저 트리를 읽으세요.

```
const groups = app.listLayerGroups?() ?? [];
const existing = groups.find((group) => group.name === "Basins");
const roots = groups.filter((group) => group.parentId === null);
```

`listLayerGroups`는 모든 그룹을 반환하며 `parentId`는 상위 그룹의 id, 루트에서는 `null`로 설정되므로, 이 평면 배열이 전체 폴더 트리를 설명합니다. 이는 스토어 자체의 그룹 순서이며 패널의 순서와는 다릅니다. 패널은 표시를 위해 그룹을 부모 다음으로 재정렬하지만, 부모 재지정은 배열을 그대로 둡니다. 다른 읽기 전용 조회와 마찬가지로 호출해도 스토어는 변경되지 않습니다.

그룹 가시성과 불투명도는 각 하위 레이어 자체의 값과 **결합**됩니다. 숨겨진 그룹은 하위 레이어의 개별 `visible` 플래그를 건드리지 않고 지도에서 숨기며, 그룹 불투명도는 각 하위 레이어의 불투명도에 곱해집니다. 이 API를 통한 `removeLayerGroup`은 폴더만 제거하고 내용물은 절대 제거하지 않습니다. 폴더에 있던 레이어는 패널 루트로 이동하고, 그 안에 중첩된 그룹은 제거된 폴더의 부모로 재지정됩니다.

이 메서드들은 호스트 변형과의 상위 호환성을 위해 선택적으로 타입이 지정되어 있으므로 옵셔널 체이닝으로 호출하세요.

레이어 스타일

`importLayerStyle`은 Layers 패널의 **Import style**(스타일 가져오기)처럼 다른 형식으로 작성된 스타일을 레이어에 적용합니다. OGC SLD, QGIS QML 또는 Mapbox GL 스타일 JSON을 내용으로부터 감지합니다. 스타일은 레이어의 현재 스타일 위에 병합되므로 스타일이 기술하지 않는 필드는 값을 유지하며, 프로젝트와 함께 저장됩니다. Layers 패널과 마찬가지로 GeoJSON 및 벡터 타일 레이어만 스타일링합니다. 플러그인 자신의 레이어뿐 아니라 그런 레이어의 어떤 id든 받아들이며, 알 수 없는 id에는 예외를 던집니다.

웹 서비스에서 피처를 추가하는 플러그인은 서비스가 하는 것과 같은 방식으로 피처를 꾸밀 수 있습니다. 예를 들어 GeoServer는 WMS `GetStyles`에 대해 레이어의 SLD를 반환합니다.


```
const layerId = app.addGeoJsonLayer("Land cover", features);
const sld = await(
  await fetch(
    `${wmsUrl}?service=WMS&version=1.1.1&request=GetStyles&layers=M5:L4`
  )
).text();
const result = app.importLayerStyle?.(layerId, sld);
if (result && !result.ok)
  console.warn(`Style not applied (${result.reason})`, result.warnings);
```

`result.warnings` 는 스타일이 요구했지만 GeoLibre가 표현할 수 없었던 항목을 나열합니다. 실패 시 `reason` 은 텍스트가 읽을 수 있는 어떤 형식의 스타일도 아닐 때 `invalid`, 파싱은 되었지만 레이어가 읽을 수 있는 심볼로지를 기술하지 않을 때 `no-match`, 레이어가 벡터 레이어가 아닐 때 `unsupported-layer` 이며, 레이어는 그대로 유지됩니다.

래스터 및 타일 레이어

`addGeoJsonLayer` 는 벡터 데이터를 네이티브 레이어로 등록합니다. 래스터와 타일 데이터에는 이에 대응하는 세 가지 헬퍼인 `addTileLayer` (XYZ), `addWmtsLayer` (WMTS), `addWmsLayer` (WMS)가 있습니다. 각각 새 레이어의 id를 반환하며, 레이어는 완전한 불투명도, 순서 변경, 스타일링 지원과 함께 Layers 패널에 나타나고 프로젝트와 함께 유지되므로, 플러그인이 더 이상 `getMap().addSource()/addLayer()` 를 직접 호출할 필요가 없습니다(직접 호출하면 레이어가 GeoLibre의 레이어 스토어에 보이지 않게 됩니다).

```
export interface GeolibreTileLayerOptions {
  tileSize?: number; // default 256
  attribution?: string;
  bounds?: [number, number, number, number]; // [west, south, east, north] in WGS84
  minzoom?: number;
  maxzoom?: number;
  scheme?: "xyz" | "tms"; // default "xyz"
  visible?: boolean; // default true
  opacity?: number; // default 1
  beforeLayerId?: string; // insert beneath this layer
  metadata?: Record<string, unknown>; // provenance merged into layer.metadata
}

export interface GeolibreWmsLayerOptions extends GeolibreTileLayerOptions {
  url: string; // WMS GetMap endpoint
  layers: string; // comma-separated layer name(s)
  styles?: string;
  format?: string; // default "image/png"
  transparent?: boolean; // default true
  version?: string; // "1.1.1" (default) or "1.3.0" (sends CRS instead of SRS)
  crs?: string; // "EPSG:3857" (default), "EPSG:4326", "CRS:84" (1.3.0 only), any "EPSG:<code>"
  queryable?: boolean; // false when the capabilities mark the layers queryable="0": identify skips it
}

export interface GeolibreWfsLayerOptions {
  url: string; // WFS GetFeature endpoint
  typeName: string; // advertised feature type
  version?: string; // defaults to "2.0.0"
  bbox?: [number, number, number, number]; // [west, south, east, north] in WGS84
  maxFeatures?: number; // positive integer, default 1000; sent as count (WFS 2.x) or maxFeatures
  metadata?: Record<string, unknown>; // provenance merged into layer.metadata
}
```

```

}

export interface GeoLibreCogLayerOptions {
  engine?: "maplibre-gl-raster" | "cog-tiler-wasm" | "titiler" | "auto"; // see below
  bands?: string; // "1" (single band) or "1,2,3" (RGB)
  colormap?: string; // named colormap for a single-band COG, e.g. "terrain"
  rescaleMin?: number;
  rescaleMax?: number;
  nodata?: number; // pixel value rendered transparent
  opacity?: number; // default 1
  beforeLayerId?: string;
  zoomTo?: boolean; // fit the map to the COG once loaded (default true)
}

```

```

// XYZ tiles (e.g. an imagery, topo, or DEM hillshade endpoint).
app.addTileLayer?.(
  "LINZ Aerial Imagery",
  "https://tiles.example.nz/aerial/{z}/{x}/{y}.png",
  {
    attribution: "Sourced from LINZ. CC BY 4.0",
    maxzoom: 22,
    bounds: [166.0, -47.5, 178.6, -34.0],
  }
);

// WMTS tile URL template.
app.addWmtsLayer?.(
  "LINZ Topo50",
  "https://tiles.example.nz/topo50/{z}/{x}/{y}.png"
);

// WMS – pass the request parameters; the host builds the GetMap tile URL.
app.addWmsLayer?.("LINZ Coverage", {
  url: "https://wms.example.nz/wms",
  layers: "coverage",
  transparent: true,
});

// A WMS that does not offer EPSG:3857: request a CRS it lists instead.
app.addWmsLayer?.("Cadastral parcels", {
  url: "https://wms.example.it/wms",
  layers: "parcels",
  version: "1.3.0",
  crs: "EPSG:6706",
});

// The capabilities mark the layer queryable="0": identify skips it.
app.addWmsLayer?.("Buildings", {
  url: "https://wms.example.it/wms",
  layers: "buildings",
  queryable: false,
});

// A layer found in a catalogue: keep its provenance with it.
app.addWmsLayer?.("Bathymetry", {
  url: "https://wms.example.org/wms",
  layers: "bathymetry",
  metadata: {
    catalogRecordId: "rndt:abc-123",
    catalogRecordUrl: "https://catalog.example.org/records/abc-123",
    publisher: "Example Hydrographic Office",
    license: "CC BY 4.0",
  },
});

// COG – read the GeoTIFF directly (client-side), with raster controls.
const cogId = await app.addCogLayer?.(

```

```
"LINZ DEM",
"https://cog.example.nz/dem.tif",
{ colormap: "terrain", nodata: -9999 }
);
```

`addTileLayer`, `addWmtsLayer`, `addWmsLayer`, `addWfsLayer` 는 새 레이어의 `metadata` 에 병합되는 선택적 `metadata` 객체를 받습니다. 카탈로그 레이어의 출처(레코드 id, 메타데이터 페이지 링크, 발행자, 라이선스)를 레이어와 함께 남기는 데 사용하세요. 이는 레이어의 **Metadata**(메타데이터) 대화상자에 표시되고 프로젝트와 함께 저장됩니다. 일반적인 JSON 직렬화 가능 객체여야 하며, 그렇지 않으면 호출이 예외를 던집니다(`addWfsLayer` 는 거부됨). 같은 이름의 필드가 있으면 GeoLibre 자체의 키가 우선하며, 자격 증명 이름을 가진 필드(`token`, `apiKey` 등)는 프로젝트를 공유하거나 내보낼 때 제거됩니다. Metadata 대화상자는 GeoLibre가 서비스 또는 타일 레이어의 `source` 에 보관하는 서비스 주소와 요청 필드(WMS의 경우 `url`, `layers`, `styles`, `format`, `version`, `crs`, 그리고 false일 때의 `queryable`)도 자격 증명을 제거한 상태로 표시합니다.

WFS 레이어는 GeoJSON/GML 폴백, 재투영, 데스크톱 네이티브 HTTP, 새로 고침을 포함한 호스트의 `GetFeature` 로더를 사용합니다. `addWfsLayer` 는 새 레이어 id로 해결(resolve)되며, 로드에서 실패하거나 서비스가 피처를 반환하지 않으면 거부(reject)됩니다. 저장된 프로젝트는 보통 다운로드한 컬렉션을 포함하는 대신 요청 URL을 보관하며, 다시 열 때 다시 가져옵니다. 예외: 저장 시 URL에서 자격 증명 제거되면, 비밀을 저장하지 않고도 레이어가 계속 보이도록 가져온 컬렉션을 포함하며, 정리된 URL로부터 다시 가져오지 않습니다. 선택적 `bbox`는 WGS84 `[west, south, east, north]`이며 날짜변경선을 가로지르면 안 됩니다(`west` 가 `east` 를 초과할 수 없으며, 태평양을 가로지르는 상자는 예외를 던집니다). `maxFeatures` (기본값 1,000)는 요청을 제한하고 `GetFeature` URL에 저장되므로, 새로 고침과 URL을 보관하는 프로젝트는 같은 제한을 사용합니다.

`addWfsLayer` 는 절대 HTTP(S) URL을 요구하며 **Add Data**(데이터 추가)가 사용하는 것과 같은 호스트 관리 WFS 경로를 통해 요청을 보냅니다. 데스크톱 앱은 네이티브 HTTP 클라이언트를 통해(CORS를 우회하여), 웹 빌드는 개발 프록시를 통해 가져옵니다. URL의 자격 증명은 진단 정보나 저장된 프로젝트에 절대 기록되지 않습니다. 네이티브 진단 로그는 `userinfo`와 자격 증명 쿼리 값(AWS `x-amz-*` 매개변수 포함)을 `[redacted]` 로 기록하며, 저장 시 URL의 `userinfo`와 자격 증명 매개변수를 제거하면서 가져온 컬렉션을 포함하여 비밀을 영구 저장하지 않고도 계속 보이게 합니다.

예제는 데스크톱 앱과 Vite 개발 서버가 허용하는 호스트 이름을 사용합니다. 플러그인이 선택하는 어떤 대상이든 사용자 자신의 브라우저 또는 데스크톱 네트워크 위치에서 가져오므로, URL을 신뢰할 수 없는 입력으로 취급하고 공유 프로젝트나 진단 로그에 드러나기를 원하지 않는 자격 증명 포함하지 마세요.

```
const layerId = await app.addWfsLayer?.("Roads", {
  url: "https://services.example.org/geoserver/wfs",
  typeName: "transport:roads",
  bbox: [10, 40, 12, 42],
  maxFeatures: 5000,
});
```

`options.engine` 은 렌더러를 선택합니다(GPU/deck.gl 경로는 `"maplibre-gl-raster"`, WebAssembly 타일러는 `"cog-tiler-wasm"`, TiTiler 서버는 `"titiler"`). 다른 옵션과 달리 이것은 **레이어별이 아닙니다**. 래스터 컨트롤은 자신이 관리하는 모든 래스터에 대해 하나의 엔진을 유지하므로, 엔진을 지정하면 이미 지도에 있는 래스터도

다시 렌더링됩니다. 컨트롤이 현재 사용 중인 엔진을 그대로 두려면 "auto" 를 전달하고, 생략하면 GPU 렌더러가 사용됩니다. GPU 렌더러는 메르카토르 투영을 요구하므로, 지구본에서 동작해야 하는 플러그인은 "cog-tiler-wasm" 을 요청해야 합니다.

`addWmsLayer` 는 `crs` 가 다른 CRS를 지정하지 않는 한 Web Mercator 타일을 요청합니다. `crs` 는 capabilities에 EPSG:3857이 없는 서버를 위한 것입니다. 레이어가 나열하는 것 중 하나를 고르되, 가급적 지리 좌표계(EPSG: 4326 , EPSG:4258 , EPSG:6706 , 또는 `version: "1.3.0"` 과 함께 CRS:84)를, 그렇지 않으면 EPSG:25832 와 같은 투영 EPSG:<code> 를 사용하세요. 데스크톱 앱만 해당 타일을 Web Mercator로 다시 그립니다. 웹 빌드는 여전히 Web Mercator BBOX를 보내므로 그런 레이어는 웹에서 빈 상태로 남습니다. 지원되지 않는 값이나 WMS 1.1.1과 함께 쓴 CRS:84 는 예외를 던집니다.

`addTileLayer` / `addWmtsLayer` / `addWmsLayer` 는 **미리 렌더링된 타일**(예: titiler 같은 타일러를 통해 XYZ 엔드포인트로 이미 제공되는 COG)을 기대합니다. `addCogLayer` 는 다릅니다. **GeoTIFF 자체**를 로드하여 클라이언트 측에서 렌더링하며, 래스터 패널에 밴드 선택, 리스케일, 컬러맵, nodata를 노출합니다. 비동기(파일 헤더를 가져옴)이므로 `Promise<string>` 을 반환하며 COG를 읽을 수 없으면 거부됩니다.

이 헬퍼들은 호스트 변형과의 상위 호환성을 위해 선택적으로 타입이 지정되어 있으므로 옵셔널 체이닝 (`app.addTileLayer?.(...)`)으로 호출하세요.

데스크톱(Tauri) 참고: 데스크톱 앱은 Content Security Policy를 적용합니다. `connect-src` 는 모든 `https:` 또는 `http:` 호스트를 허용하므로 플러그인의 타일 및 데이터 요청은 서버에 도달하지만, `script-src` 는 앱 자체, `blob:` URL, 그리고 버전이 고정된 짧은 CDN 경로 목록만 허용합니다. 다른 호스트에서 `<script>` 를 삽입하거나 모듈을 `import()` 하는 플러그인은 차단되므로, 그 코드를 플러그인에 번들링하세요. 번들된(퍼스트파티) 플러그인은 `apps/geolibre-desktop/src-tauri/tauri.conf.json` 의 `script-src` 에 경로를 추가할 수 있습니다([데스크톱 CSP script-src 허용 목록](#) 참고). 웹 빌드는 영향을 받지 않습니다.

Zarr 레이어

`addZarrLayer` 는 Zarr 스토어(HTTP를 통한 Zarr v2/v3)를 **GeoLibre 자체의** `@carbonplan/zarr-layer` 인스턴스를 통해 렌더링하고 그 결과를 Layers 패널에 반영합니다. `addCogLayer` 의 Zarr 버전입니다. URL에서 읽지 않는 스토어(kerchunk 기반 클라우드 NetCDF, Icechunk 저장소, 디스크의 폴더)는 이 API가 노출하지 않는 내부 `store` 옵션을 통해 같은 렌더러에 도달하므로, 플러그인이 아니라 GeoLibre 자체 패널에서 추가됩니다.

플러그인에 `@carbonplan/zarr-layer` 를 번들링하지 마세요. 두 번째 사본은 중복된 numcodecs WASM 페이로드를 포함하게 되고, `getMap().addLayer()` 로 렌더러의 레이어를 직접 추가하면 MapLibre **커스텀** 레이어가 만들어지는데, 여기에는 Style 패널이 조정할 paint 속성이 없습니다.

```
export interface GeolibreZarrLayerOptions {
  variable: string; // array to render (required)
  selector?: Record<string, number | string>; // non-spatial dims, e.g. { time: 0 }
  clim?: [number, number];
  colormap?: string | string[]; // named ramp ("viridis") or hex stops
```

```
opacity?: number;
zarrVersion?: 2 | 3;
crs?: string; // e.g. "EPSG:32633"
proj4?: string; // for a CRS with no built-in
bounds?: [number, number, number, number]; // [xMin, yMin, xMax, yMax] in the store's CRS
spatialDimensions?: { lat?: string; lon?: string };
headers?: Record<string, string>; // authenticated stores
beforeLayerId?: string;
}
```

```
// A projected national grid: crs/proj4 are forwarded to the renderer, which
// reprojects on the GPU. Without them the store is read as WGS84 and lands in
// the wrong place.
const layerId = await app.addZarrLayer?.(
  "seNorge_tmax",
  "https://example.no/senorge.zarr",
  {
    variable: "tmax",
    selector: { time: 0 },
    clim: [-30, 30],
    colormap: "viridis",
    crs: "EPSG:32633",
  }
);

// Drive a time slider without rebuilding the layer: the renderer keeps the
// chunks it already fetched. (`addZarrLayer` is optional, so guard the id.)
if (layerId) {
  await app.setZarrLayerSelector?.(layerId, { time: 12 });
}
```

`addZarrLayer`는 헤드리스입니다. Zarr 패널을 열지 않습니다(사용자는 여전히 **Add Data → Zarr Layer**(데이터 추가 → Zarr 레이어)에서 패널을 열어 컬러맵과 색상 한계를 조정할 수 있습니다). 레이어가 등록되면 새 레이어의 id로 해결되며, `variable`이 없거나 스토어를 읽을 수 없으면 거부됩니다. 이 레이어는 다른 레이어와 마찬가지로 Layers 패널에서 가시성, 불투명도, 순서 변경, 제거를 지원합니다.

`selector`는 인덱스가 아니라 좌표 값으로 슬라이스를 선택합니다. 1-12의 `month` 축에서 12월은 `{ month: 12 }`입니다. 패널의 Selector (JSON) 필드도 같은 의미이며, 이를 편집하면 이미 지도에 있는 레이어가 다시 슬라이스됩니다.

클릭하여 값 읽기 및 영역 통계

`queryZarrLayer`는 GeoJSON 지오메트리 아래의 레이어 값을 읽습니다. Identify에는 `Point`를, 구역 통계에는 `Polygon` / `MultiPolygon`을 사용합니다. `setZarrLayerSelector`의 읽기 버전으로, 레이어 id로 같은 실시간 렌더러에 도달합니다.

```
export type GeoLibreZarrQueryGeometry =
  | { type: "Point"; coordinates: [number, number] } // WGS84 lng/lat
  | { type: "Polygon"; coordinates: number[][][] }
  | { type: "MultiPolygon"; coordinates: number[][][][] };

// Dimension values to read instead of the slice on screen. A list per dimension
// (e.g. { month: [1, 7] }) nests the returned values by that dimension.
export type GeoLibreZarrQuerySelector = Record<
  string,
  | number
  | string
  | number[]
>
```

```

    | string[]
    | {
        selected: number | string | number[] | string[];
        type?: "index" | "value";
    }
>;

export interface GeolibreZarrQueryOptions {
    signal?: AbortSignal; // cancel a query the user moved past
    includeSpatialCoordinates?: boolean; // default true
}

// { [variable]: values, dimensions, coordinates }
export interface GeolibreZarrQueryResult {
    [variable: string]: unknown;
    dimensions: string[];
    coordinates: { [key: string]: (number | string)[] };
}

// Identify: read the value under a map click, for the slice on screen.
map.on("click", async (event) => {
    const { lng, lat } = event.lngLat;
    const result = await app.queryZarrLayer?.(layerId, {
        type: "Point",
        coordinates: [lng, lat],
    });
    const [value] = (result?.sst as number[]) ?? [];
    showReadout(value); // undefined outside the store's grid
});

// Zonal statistics: every pixel inside a polygon, on another time slice.
const region = await app.queryZarrLayer?.(layerId, aoi.geometry, { time: 12 });

```

렌더러는 이미 스토어의 그리드를 보유하고 있으므로 재투영과 채움 값 마스크를 직접 수행합니다. 자체 `zarrita` 포인트 읽기로 스토어를 다시 여는 대신 지도 클릭에서 얻은 WGS84 `[lng, lat]` 를 그대로 전달하세요. 반환되는 `coordinates` 는 WGS84가 아니라 스토어의 **원본** CRS(EPSC:3857이면 Web Mercator 미터, EPSG:4326이면 도, 커스텀 proj4 데이터셋이면 원본 단위)라는 점에 유의하세요.

스토어 그리드 밖의 지오메트리나 첫 청크가 아직 로드되지 않은 레이어에 대해서는 오류 대신 **빈** 값이 반환되므로, `addZarrLayer` 가 해결된 직후에 실행한 쿼리는 id가 유효하더라도 비어 있을 수 있습니다. `queryZarrLayer` 자체는 해당 id를 가진 실시간 Zarr 레이어가 없으면 `null` 로 해결되며, 중단된 쿼리는 **거부**됩니다(abort 오류). 따라서 새 클릭마다 취소한다면 호출을 보호하세요.

여기에 전달한 `selector` 는 읽기 범위만 지정합니다. 레이어는 원래 보여주던 슬라이스를 계속 렌더링하므로, 다른 시간에 대한 Identify 결과를 읽어도 지도가 흐트러지지 않습니다. 표시를 실제로 이동시키려면 `setZarrLayerSelector` 를 사용하세요.

패널은 Zarr URL 옆의 **Browse folder**(폴더 찾아보기) 버튼을 통해 디스크의 폴더에서 스토어를 열 수도 있습니다. 이 버튼은 데스크톱 앱과 File System Access API를 지원하는 브라우저(Chromium)에 나타나는데, 폴더를 읽으려면 플러그인이 제공할 수 없는 파일 시스템 API가 필요하기 때문입니다. 그 외의 환경에서는 패널이 그대로입니다. 로컬 큐브는 원격 큐브와 마찬가지로 Time Slider에 바인딩됩니다. 기록된 URL이 주소가 아닌 식별자이기 때문에 `CF units` 는 폴더에서 읽어 옵니다.

Time Slider에서 레이어 자체의 시간 차원 구동하기

Time Slider는 세 종류의 시간 레이어를 이해합니다. 두 가지는 내장되어 있습니다. 타임스탬프 속성으로 필터링되는 **벡터** 레이어와, 독이 사이클 오가는 날짜가 지정된 소스들의 **래스터 시계열**입니다. 세 번째는 시간을 **내부 차원**으로 가진 **_하나의 스토어_인 레이어**(Zarr 데이터 큐브 또는 플러그인 자체의 프레임 기반 레이어)를 위한 것으로, 타임라인이 소스가 아니라 슬라이스를 선택합니다.

이 세 번째 종류는 **temporal adapter**로 표현됩니다.

```
export interface TemporalLayerAdapter {
  getTimeValues: () => ReadonlyArray<Date | number |
string>; // the time coordinate, in index order
  setTime: (date: Date) => void | Promise<void>; // apply a date to the layer
  dimension?: string; // the axis name, default "time"
  granularity?: "hour" | "day" | "month" | "year"; // stepping unit; derived from the axis
span when omitted
  displayUnits?: ("hour" | "day" | "month" | "year")[]; // units offered by the slider's
granularity controls
}
```

스냅은 슬라이더가 담당합니다. `getTimeValues()` 를 읽고 현재 날짜에 가장 가까운 인덱스를 찾은 뒤 **해당 슬라이스 자체의 날짜**로 `setTime()` 을 호출하며, 이미 표시 중인 슬라이스에 해당하는 틱은 건너뛸니다. 따라서 월 단위로 이동하는 타임라인 아래의 일별 큐브는 틱마다가 아니라 인덱스가 바뀔 때마다 한 번의 청크 가져오기 비용만 듭니다. 핸들을 드래그하는 동안 호출은 스로틀링되며, 마지막 값은 항상 적용됩니다. 허용되는 값 형식은 `Date`, 에포크 초/밀리초, 연도만 있는 값, 날짜 문자열입니다.

`addZarrLayer` 로 추가한 Zarr 레이어는 `time` 축에 대해 **자체 어댑터를 등록**하므로(스토어 메타데이터에서 `"days since 1970-01-01"` 과 같은 CF units 를 디코딩) 여기서 호출할 필요가 없습니다. 직접 렌더링하는 커스텀 레이어에는 `registerTemporalLayer` 를 사용하세요.

```
const times = frames.map((frame) => frame.timestamp); // Date[] | number[] | string[]

const detach = app.registerTemporalLayer?.(
  layerId,
  {
    dimension: "time",
    getTimeValues: () => times,
    setTime: (date) => showFrame(nearestFrame(date)),
  },
  { bind: true }
);
```

등록하면 레이어가 **바인딩 가능**해집니다. Layers 패널의 행 메뉴에 **Bind time dimension to Time Slider**(시간 차원을 Time Slider에 바인딩)가 추가됩니다. `{ bind: true }` 를 전달하면 즉시 바인딩하고 독을 여는데, 방금 큐브를 로드한 플러그인이 보통 원하는 동작입니다. 바인딩은 `layer.metadata.timeBinding` 에 `( { kind: "selector", dimension, min, max, granularity }` 형태로) 유지되어 프로젝트 왕복 후에도 살아남습니다. 시간 값 자체는 데이터 스토어에 속하므로 유지되지 않습니다. 레이어가 다시 만들어지면 어댑터를 다시 등록하세요. 그러면 타임라인이 다시 인식합니다.

반환된 함수를 호출하거나 `app.unregisterTemporalLayer?.(layerId)` 를 호출하여 어댑터를 제거합니다. 레이어를 제거해도 마찬가지로 제거됩니다.

바인딩된 큐브는 벡터 바인딩 및 날짜가 지정된 오버레이와 타임라인을 공유합니다. 트랙은 이들 범위의 합집합에 걸리며, 가장 넓은 데이터셋이 이동 단위를 결정합니다.

독 다시 닫기. 바인딩이 연 독은 마지막 시간 레이어가 사라지면 스스로 닫히므로, 타임라인이 없는 지도 위에 남아 있지 않습니다. 바인딩된 레이어를 제거하는 것으로 충분합니다. 플러그인이 레이어는 유지하고 바인딩만 제거한다면, 어댑터 등록 해제와 함께 `layer.metadata.timeBinding` 도 지우세요. `_사용자_`가 **Plugins**(플러그인) 메뉴에서 연 독은 그대로 두며, 자체 래스터 소스나 구동할 KML `<TimeSpan>` 오버레이가 아직 있는 독도 마찬가지입니다.

다른 플러그인 활성화 및 비활성화

```
await app.activatePlugin?.("maplibre-gl-time-slider");
app.deactivatePlugin?.("maplibre-gl-time-slider");
```

`activatePlugin` 은 선택적 두 번째 인자로 대상이 활성화된 뒤 적용되는 프로젝트 상태 패치를 받습니다. 플러그인을 사용할 수 없거나, 활성화를 거부하거나, 상태를 거부하면 `false`로 해결됩니다. `deactivatePlugin` 은 그 반대 역할이며 플러그인이 결과적으로 비활성 상태가 되면 `true`를 반환합니다.

둘 다 **호출하는** 플러그인 자신을 대상으로 할 수 없습니다. 자기 자신에 대한 `activatePlugin` 은 의미가 없고, 자기 자신을 비활성화하면 아직 스택에 있는 코드가 언마운트됩니다. 이 경우 둘 다 `false`를 반환합니다.

커스텀(WebGL) 레이어와 paint 소유권

`registerExternalNativeLayer` 는 플러그인이 지도에 직접 추가한 레이어를 GeoLibre의 레이어 스토어에 반영하여 Layers 패널에 나타나고 프로젝트와 함께 유지되게 합니다.

```
export interface GeolibreExternalNativeLayerRegistration {
  id: string;
  name: string;
  type?: GeolibreLayer["type"]; // closest built-in type, e.g. "raster"
  nativeLayerIds: string[]; // the MapLibre layer id(s) you added
  source?: Record<string, unknown>;
  sourceIds?: string[];
  sourceId?: string;
  geojson?: FeatureCollection; // for vector layers
  beforeId?: string;
  opacity?: number;
  style?: Partial<LayerStyle>;
  metadata?: Record<string, unknown>;
  sourcePath?: string;
  paintMode?: "geolibre" | "plugin"; // see below
  paintBridge?: {
    // see below
    setOpacity?: (opacity: number) => void;
    setVisibility?: (visible: boolean) => void;
  };
};
```

자체 MapLibre CustomLayerInterface (자신의 픽셀을 직접 그리는 WebGL 레이어)로 렌더링하는 플러그인도 같은 방식으로 등록합니다.

```
const layer = new MyWebGLLayer({ id: "my-layer" });
app.getMap?().addLayer(layer);

app.registerExternalNativeLayer?({
  id: "my-layer",
  name: "My WebGL layer",
  type: "raster",
  nativeLayerIds: ["my-layer"],
  // The layer has no MapLibre paint properties, so GeoLibre must not offer
  // paint editors that cannot reach it.
  paintMode: "plugin",
  // Optional: keep the Opacity sliders live by forwarding them to the layer.
  paintBridge: {
    setOpacity: (opacity) => layer.setOpacity(opacity),
    setVisibility: (visible) => layer.setActive(visible),
  },
});
```

- `paintMode: "plugin"` 은 플러그인이 레이어를 그린다는 것을 Style 패널에 알립니다. 그러면 아무 효과 없이 조용히 동작하지 않는 래스터 밝기/채도/대비/색조 슬라이더 대신, 실제로 적용되는 컨트롤(아래에 삽입, 줌 범위, 브리지가 있으면 불투명도)만 표시합니다. 가시성, 순서 변경, 제거는 Layers 패널에서 계속 동작합니다. MapLibre는 커스텀 레이어에서도 `visibility` 와 줌 범위를 존중합니다.
- `paintBridge` 는 사용자가 불투명도나 가시성을 변경할 때 GeoLibre가 호출하는 세터를 제공합니다. `setOpacity` 를 제공하면 Layers 패널과 Style 패널 모두에서 Opacity 슬라이더가 유지되고, 생각하면 동작하지 않는 슬라이더를 남기는 대신 숨깁니다. 브리지를 제공하면 `paintMode: "plugin"` 이 암묵적으로 적용됩니다.
- 세터는 레이어 동기화마다가 아니라 값이 바뀔 때만 호출되며, 레이어 레코드 외부에 보관됩니다(함수는 프로젝트 파일에 직렬화할 수 없습니다). 프로젝트를 다시 로드한 뒤 브리지를 복원하려면 레이어를 다시 등록하고, `deactivate` 에서 `unregisterExternalNativeLayer(id)` 를 호출하세요.

특히 Zarr의 경우에는 위의 `addZarrLayer` 를 사용하는 것이 좋습니다. 호스트의 렌더러가 이미 패널과 통합되어 있으므로 커스텀 레이어나 브리지가 필요 없습니다.

오른쪽 사이드바 패널

플러그인은 고정 오버레이로 흉내 내는 대신, 내장 Style 패널 옆에 도킹되어 워크스페이스의 일급 구성 요소처럼 동작하는 네이티브 오른쪽 사이드바 패널을 등록할 수 있습니다. 호스트가 패널 크롬(접기 및 닫기 버튼이 있는 헤더, 접을 수 있는 레일, 크기 조절 핸들)을 렌더링하고, 플러그인은 본문만 담당합니다.

```
export const myPlugin: GeoLibrePlugin = {
  id: "my-workbench",
  name: "Workbench",
  version: "0.1.0",
  activate(app) {
    // Register once, then open. registerRightPanel returns an unregister fn.
    this._unregister = app.registerRightPanel?({
      id: "my-workbench",
      title: "Workbench",
    });
  }
};
```

```

    defaultWidth: 360,
    render(container) {
      const button = document.createElement("button");
      button.textContent = "Run analysis";
      container.appendChild(button);
      // Optional cleanup, run when the panel closes or is unregistered.
      return () => button.remove();
    },
    onOpen() {},
    onCollapse() {},
    onClose() {},
  });
  app.openRightPanel?.("my-workbench");
},
deactivate(app) {
  app.closeRightPanel?.("my-workbench");
  this._unregister?.();
},
} as GeoLibrePlugin & { _unregister?: () => void };

```

참고:

- `render(container)` 는 일반 DOM으로 채울 빈 요소와 함께 한 번 호출됩니다. 외부 플러그인은 GeoLibre 의 React 인스턴스를 공유할 수 없으므로 계약은 React 노드가 아니라 DOM입니다. 컨테이너는 접힌 동안에도 마운트된 상태로 유지되므로 DOM의 상태는 보존되며, 반환된 정리 함수는 닫기 또는 등록 해제 시 실행됩니다.
- 한 번에 하나의 플러그인 패널만 활성화됩니다. 플러그인 패널이 도킹된 쪽의 내장 패널(왼쪽은 Layers, 오른쪽은 Style)은 플러그인 패널이 그 옆에 펼쳐져 있는 동안 레일로 접히고, 플러그인 패널이 반대쪽으로 이동하거나, 자체 레일로 접히거나, 닫히면 복원됩니다.
- `openRightPanel(id)` 는 패널을 활성화하고 펼칩니다(접힌 패널도 펼칩니다). `collapseRightPanel(id)` 는 닫지 않고 레일로 접습니다. `closeRightPanel(id)` 는 워크스페이스를 해제합니다. `getActiveRightPanel()` 은 활성 id 또는 `null` 을 반환합니다.
- 패널은 지도의 flex 형제 요소이므로, 패널을 열면 지도 뷰가 줄어듭니다(지도는 남은 공간을 계속 채웁니다). 수동으로 지도 패딩을 지정할 필요가 없습니다.
- **도킹 위치:** 패널은 네 위치 중 하나(왼쪽에서 오른쪽으로)에 도킹됩니다. `left-of-layers`, `right-of-layers` (Layers와 지도 사이), `left-of-style` (지도와 Style 사이), `right-of-style` (기본값). 초기 위치를 선택하려면 등록 시 `dock` 을 설정하세요. 사용자는 패널 헤더의 두 이동 버튼(끝에서는 비활성화됨)으로 런타임에 위치를 옮길 수 있고, 플러그인은 `app.setActiveRightPanelDock?.(...)` 으로 직접 설정할 수 있습니다. 패널이 닫히거나 다른 패널이 열리면 위치는 패널에 선언된 `dock` 으로 재설정됩니다.
- **공유 레일 모드(`replace-style` / `replace-layers`):** Style(오른쪽) 또는 Layers(왼쪽) 옆의 두 번째 레일이 아니라 일급 사이드바 워크스페이스처럼 느껴지기를 원하는 워크벤치 스타일 플러그인을 위한 두 가지 비위치 독입니다. `dock: "replace-style"` (또는 `"replace-layers"`)로 등록하면 호스트가 해당 가장자리에 플러그인 패널과 내장 패널을 모두 나열하는 단일 레일을 표시합니다. 하나를 선택하면 펼쳐지고 다른 하나는 레일 항목으로 남습니다. 둘은 상호 배타적이므로 사용자가 인접한 두 레일을 보는 일은 없습니다. 내장 패널은 접힌 상태로 시작하여 워크벤치가 활성 워크스페이스로 보이며, 사용자는 언제든지 내장 패널을 펼칠 수 있습니다(그러면 워크벤치가 접힙니다). 그 외의 모든 것(크롬, 크기 조절, 접기, 닫기, 수명 주기 등)은 동일합니다.

- **런타임에 모드 전환:** 모드는 등록 시 고정되는 배타적 선택이 아닙니다. 위치 독에서는 패널 헤더에 현재 쪽의 공유 레일에 합류하는 **merge**(병합) 버튼이 표시됩니다. Layers 쪽 패널(`left-of-layers` / `right-of-layers`)은 Layers 레일에, Style 쪽 패널은 Style 레일에 합류합니다. 공유 레일에서는 패널을 같은 쪽의 이동 가능한 위치 패널(`right-of-layers` / `right-of-style`)로 다시 꺼내는 **detach**(분리) 버튼이 표시되며, 여기서 왼쪽/오른쪽 이동 버튼이 돌아옵니다. 플러그인은 `app.setActiveRightPanelDock?.` (`"replace-style" | "replace-layers" | "right-of-style" | ...`) 로 같은 전환을 수행할 수 있습니다. 공유 레일은 왼쪽/오른쪽 단계 순서의 일부가 아니며(화살표는 네 위치 독만 오갑니다), merge/detach 가 들어가고 나오는 방법입니다.
- 이 메서드들은 오른쪽 사이드바가 없는 호스트 변형과의 상위 호환성을 위해 선택적으로 타입이 지정되어 있으므로 옵셔널 체이닝(`app.registerRightPanel?.(...)`)으로 호출하세요.

툴바 메뉴

플러그인은 GeoLibre 배너에(Project / Edit / View / Plugins 옆에) 중첩 하위 메뉴와 액션 항목을 가진 자체 최상위 메뉴 버튼을 추가할 수 있습니다. `activate` 에서 메뉴를 등록하고 `deactivate` 에서 등록을 해제하세요.

```
const unregister = app.registerToolbarMenu?.({
  id: "my-plugin-menu",
  label: "Workbench",
  items: [
    {
      id: "open",
      label: "Open workbench",
      onSelect: () => app.openRightPanel?.("my-workbench"),
    },
    {
      type: "submenu",
      id: "tools",
      label: "Tools",
      items: [
        {
          id: "qa",
          label: "Data QA",
          onSelect: () => app.openFloatingPanel?.("my-qa"),
        },
      ],
    },
    { type: "separator" },
    { id: "about", label: "About", disabled: false, onSelect: () => {} },
  ],
});
```

각 항목은 **액션**(`onSelect` , `type` 을 생략했을 때의 기본값), **하위 메뉴**(중첩된 `items`), 또는 **구분선**입니다. 항목은 보통 오른쪽 패널이나 플로팅 패널을 열지만, `onSelect` 는 무엇이든 실행할 수 있습니다. 같은 `id` 를 다시 등록하면 메뉴가 교체되므로, 플러그인 상태가 바뀔 때 메뉴를 다시 구성할 수 있습니다.

외부 플러그인(zip, 매니페스트 URL, 또는 번들된 드롭인에서 로드된)의 메뉴는 Help 메뉴 뒤, 배너의 끝에 렌더링되어 서드파티 메뉴가 내장 메뉴 다음에 함께 모입니다. 내장 플러그인의 메뉴는 내장 메뉴 옆에 렌더링됩니다. 호스트가 메뉴를 소유한 플러그인에 따라 배치를 결정하므로 특별히 할 일은 없습니다.

외부 플러그인의 메뉴가 배너를 차지할지는 사용자가 결정합니다. 메뉴를 등록한 설치된 플러그인마다 **Plugins → Installed**(플러그인 → 설치됨) 항목에 **Show menu in toolbar**(툴바에 메뉴 표시) 스위치가 생깁니다. 이를 끄면 플러그인의 메뉴가 같은 항목을 가진 하위 메뉴로서 그 항목 안으로 이동합니다. 이 선택은 기기에서 플러그인별로 기억됩니다. 내장 플러그인 메뉴는 항상 배너에 남습니다. 메뉴가 한 단계 더 깊이 중첩될 수 있으므로 최상위를 짧게 유지하고, 액션이 몇 개뿐이라면 대신 [내장 메뉴에 항목 추가](#)를 고려하세요.

모든 `label` (메뉴 버튼, 하위 메뉴 트리거, 액션의 레이블)은 패널 제목과 마찬가지로 일반 문자열뿐 아니라 **게터** 함수도 받습니다.

```
app.registerToolbarMenu?({
  id: "my-plugin-menu",
  label: () =>
    app.translate?("plugin.my-plugin.menu", "Workbench") ?? "Workbench",
  items: [
    {
      id: "open",
      label: () =>
        app.translate?("plugin.my-plugin.open", "Open workbench") ??
        "Open workbench",
      onSelect: () => app.openRightPanel?("my-workbench"),
    },
  ],
});
```

호스트는 메뉴 트리를 렌더링할 때마다 모든 레이블을 다시 읽고, 언어가 바뀌면 다시 렌더링하므로, 게터를 사용하면 플러그인이 메뉴를 다시 등록하지 않아도 앱 언어를 따라갑니다. 일반 문자열은 등록 시점에 고정됩니다. 예외를 던지거나 사용할 수 없는 값을 반환하는 게터는 항목의 id 경로로 대체되고 한 번 경고하므로, 깨진 레이블 때문에 메뉴가 사라지는 일은 없습니다.

내장 메뉴에 항목 추가

대부분의 플러그인은 자체 최상위 메뉴가 필요하지 않습니다. Vector 또는 Raster 메뉴에 자신을 추가하는 QGIS 플러그인처럼, 플러그인은 대신 GeoLibre의 내장 메뉴 중 하나에 항목을 추가할 수 있습니다.

```
const unregister = app.registerMenuContribution?({
  id: "my-plugin-processing",
  menu: "processing", // "addData" | "processing" | "controls"
  items: [
    {
      id: "run",
      label: "Run analysis...",
      onSelect: () => app.openRightPanel?("my-workbench"),
    },
  ],
});
```

호스트는 항목을 항상 플러그인 이름을 딴 하위 메뉴 아래에 중첩하여 대상 메뉴 끝의 구분선 뒤에 배치하고, 플러그인 하위 메뉴를 알파벳순으로 정렬합니다. 플러그인은 내장 항목 사이에 낱개 항목을 끼워 넣거나 순서를 바꿀 수 없습니다. 같은 플러그인이 같은 메뉴에 기여한 여러 항목 묶음은 하나의 하위 메뉴를 공유하며, 그 사이에 구분선이 들어갑니다. 항목이 없는 기여는 표시되지 않습니다.

항목은 **툴바 메뉴**와 같은 형태를 사용합니다. 액션, 하위 메뉴, 구분선과 함께 선택적 아이콘 및 앱 언어를 따르는 레이블 게터를 지원합니다. `activate` 에서 등록하고 `deactivate` 에서 반환된 함수(또는 `app.unregisterMenuContribution?.(id)`)를 호출하세요. 같은 `id` 를 다시 등록하면 기여가 교체됩니다. `id` 는 툴바 메뉴 `id` 처럼 전역이므로 플러그인 `id` 를 접두사로 붙이세요. 한 플러그인의 `id` 가 다른 플러그인의 `id` 를 교체하면 호스트가 경고하며, `unregisterMenuContribution` 은 자신의 플러그인이 등록한 기여만 제거합니다. 호스트는 플러그인이 비활성화되거나 활성화에 실패할 때도 그 플러그인의 기여를 제거하므로, 디스포저 호출을 빠뜨려도 오래된 항목이 남지 않습니다. 호스트가 인식하지 못하는 `menu` (예: 더 새로운 GeoLibre에서 추가된 것)는 활성화를 실패시키는 대신 경고를 기록하고 무시됩니다. 툴바 메뉴와 마찬가지로 기여는 읽기 전용 뷰어에서, 그리고 배포가 `plugins:install` 을 허용하지 않을 때 숨겨집니다.

앱 언어 따르기

GeoLibre UI는 `react-i18next`로 번역되지만, 플러그인은 패널을 일반 DOM으로 렌더링하므로 호스트의 `React` 혹은 사용할 수 없습니다. 네 가지 메서드가 그 간극을 메웁니다.

```
// The active catalog code ("en", "zh", "pt-BR", ...).
const locale = app.getLocale?.() ?? "en";

// Resolve a key, falling back to your own English text when no catalog has it.
const title =
  app.translate?.("plugin.my-plugin.title", "Workbench") ?? "Workbench";
const label = app.translate?.("plugin.my-plugin.count", "{{n}} features", {
  n: 3,
});

// Re-render when the user switches language. Returns an unsubscribe function -
// call it from `deactivate`, or the listener keeps re-rendering DOM you no
// longer own.
const stop = app.onLocaleChange?.((next) => renderPanel(container, next));

// Ship your own translations (call once, from `activate`). Flat dotted keys per
// locale; `translate` then resolves them in that language.
app.registerTranslations?.({
  de: {
    "plugin.my-plugin.title": "Werkbank",
    "plugin.my-plugin.count": "{{n}} Objekte",
  },
  fr: {
    "plugin.my-plugin.title": "Atelier",
    "plugin.my-plugin.count": "{{n}} entités",
  },
});
```

규칙:

- **항상 자신의 영어 텍스트를 `defaultValue` 로 전달하세요.** 번역을 제공하지 않는 언어와 이 메서드들이 없는 이전 호스트에서 렌더링되는 텍스트이므로, 번역은 전제 조건이 아니라 업그레이드입니다.
- **키에 플러그인 `id`로 네임스페이스를 지정하세요** (`plugin.<your-id>.<something>`). 그래야 호스트 자체의 키와 충돌하지 않습니다. `registerTranslations` 는 이를 강제합니다. `plugin.<id>`. 아래의 문자열 값만 받아들이고 그 외의 것은 (콘솔 경고와 함께) 버립니다.

- **호스트의 카탈로그가 우선합니다.** GeoLibre에 번들된 카탈로그가 이미 정의한 키는 호스트의 텍스트를 유지하며, `registerTranslations` 는 기존 항목을 절대 덮어쓰지 않습니다. 등록은 세션 동안 유지되며, 같은 키를 다시 등록해도 아무 동작도 하지 않으므로 활성화마다가 아니라 한 번만 등록하세요.
- 복수형 키는 호스트에서와 같은 방식으로 동작합니다. `plugin.my-plugin.items_one` / `plugin.my-plugin.items_other` (그리고 러시아어나 아랍어 같은 언어에 필요한 추가 형태)를 등록하고 `params` 에 `{ count }` 를 전달하세요.
- 이 메서드들은 API의 나머지 부분과 마찬가지로 선택적으로 타입이 지정되어 있으므로 옵셔널 체이닝으로 호출하고 리터럴 풀백을 유지하세요.
- 패널 제목과 툴바 레이블이 게터를 받는 것은 바로 `app.translate?.()` 를 호출하여 최신 상태를 유지할 수 있도록 하기 위함입니다. 언어가 바뀔 때마다 다시 등록하는 대신 게터를 사용하세요. 한 번 만든 DOM(버튼, 힌트, 상태 줄)은 `onLocaleChange` 리스너에서 레이블을 다시 지정해야 합니다.

내장 플러그인(`packages/plugins`)은 `@geolibre/plugins` 에서 내보내는 두 헬퍼로 같은 계약을 따릅니다.

- `createPluginTranslator(app, pluginId)` 는 `tr(key, english, params?)` 를 반환하며, 이는 `plugin.<pluginId>.<key>` 를 해석하고 호스트에 `translate` 가 없으면 영어 풀백을 직접 보간합니다. @로 시작하는 키는 절대 키로, 호스트 키를 재사용하기 위한 것입니다(`tr("@common.cancel", "Cancel")`). API를 모듈 수준 변수에 보관하는 플러그인을 위해 `app` 은 게터일 수도 있습니다.
- `pluginDisplayTitle(app, pluginId, name)` 은 **Plugins**(플러그인) 메뉴에 표시되는 플러그인 이름인 `toolbar.plugin.<pluginId>` 를 읽는 패널 제목 게터를 반환하므로, 독 헤더와 메뉴 항목이 같은 언어로 유지됩니다.

이들의 문자열은 `registerTranslations` 가 아니라 GeoLibre에 번들된 카탈로그(`apps/geolibre-desktop/src/i18n/locales/*.json`)에 있으며, 카탈로그 테스트는 모든 로케일이 이를 포함할 것을 요구합니다(`docs/i18n.md` 참고).

자격 증명 저장

토큰이나 API 키를 기억해야 하는 플러그인은 자체 `localStorage` 키를 쓰는 대신 `app.credentials` 를 사용해야 합니다. 데스크톱 앱에서는 값이 OS 자격 증명 저장소(macOS Keychain, Windows Credential Manager, Linux Secret Service)에 보관되고, 웹 빌드, Jupyter 임베드, 모바일에서는 `localStorage` 에 보관됩니다.

```
const apiKey = app.credentials?.get("api-key") ?? ""; // "" when nothing is saved
const persisted = app.credentials?.set("api-key", value); // "" deletes
const where = app.credentials?.location(); // "keychain" | "browser", for UI copy
```

- 이름은 글자, 숫자, `_`, `-` 로 이루어진 1-64자이며, 그 외의 것은 `TypeError` 를 던집니다.
- **범위 지정:** 호스트가 플러그인 id를 추가하므로 값은 `plugin.<your-id>.<name>` 으로 저장됩니다. id를 직접 전달하지 않습니다.
- 읽기는 동기적입니다. 데스크톱 앱은 플러그인이 실행되기 전, 시작 시에 저장된 모든 값을 로드합니다.

- **set 은 값이 영구 저장되면 true** 를, 이번 세션 동안만 유지되면(`localStorage` 쓰기 실패 또는 데스크톱에서 키체인 사용 불가) `false` 를 반환합니다. 데스크톱 키체인 쓰기 실패는 앱의 자격 증명 저장소 경고도 발생시키므로 별도의 경고가 필요 없습니다.
- **이것은 저장소이지 격리가 아닙니다.** 플러그인은 앱 창에서 신뢰된 코드로 실행되므로, 플러그인은 여전히 메모리에서 또는 `fetch` 를 감싸서 다른 플러그인의 값을 읽을 수 있습니다. id 접두사는 올바르게 동작하는 플러그인들을 구분해 줄 뿐 보안 경계가 아닙니다.
- **데스크톱 자격 증명 저장소는 플러그인에서 읽을 수 없습니다.** 앱은 외부 플러그인이 로드되기 전 시작 중에 네이티브 호출 한 번으로 저장된 모든 자격 증명을 읽고, 이후 저장소는 창이 다시 로드될 때까지 추가 읽기를 거부합니다(그 읽기가 실행되지 않은 경우를 대비해 로더도 플러그인을 임포트하기 전에 읽기를 닫습니다). 저장소의 Tauri 명령을 직접 호출하는 플러그인은 저장된 토큰 대신 오류를 받습니다. 이는 플러그인이 접근할 수 있는 범위를 좁힐 뿐 플러그인을 격리하지는 않습니다. 앱이 이미 메모리에 로드한 값(Settings 토큰, 다른 플러그인의 `app.credentials` 값)은 공유 JavaScript 영역에서 여전히 접근할 수 있고, 저장소에 대한 쓰기는 제한되지 않으며, 웹, Jupyter, 모바일 빌드에서는 값이 `localStorage` 에 있습니다. 신뢰하는 플러그인만 설치하세요.
- **getProjectState 에 비밀을 넣지 마세요.** 플러그인 설정은 프로젝트 파일에 저장되어 함께 공유되거나 내보내집니다.
- **내장 플러그인도 이를 사용합니다.** Hugging Face, Mapillary, God's Eye View 토큰은 자체 `localStorage` 키에서 `app.credentials` 로 옮겨졌습니다. 호스트는 첫 실행 시 기존 평문 값을 마이그레이션하고 새 쓰기가 성공한 뒤에만 제거합니다.

세션 쿠키로 로그인

세션 쿠키로 사용자를 로그인시키는 서비스는 데스크톱 앱의 웹뷰에서 동작하지 않습니다. 페이지가 `tauri://localhost` 에서 실행되므로 서비스가 설정하는 쿠키는 서드파티 쿠키이고, WebKit(macOS, Linux)은 이를 버립니다. 로그인 요청은 200을 반환하고 다음 요청은 401을 반환합니다. 데스크톱에서는 `app.nativeFetch` 가 대신 앱의 네이티브 HTTP 클라이언트를 통해 요청을 보냅니다. `fetch` 시그니처를 가지며, 모든 메서드와 본문을 받고, CORS의 적용을 받지 않으며, 앱 세션 동안 유지되는 쿠키 저장소(jar)에 쿠키를 보관합니다.

```
const http = app.nativeFetch ?? fetch; // undefined in the browser and Jupyter builds
await http(`${server}/api/v1/auth/access-token`, {
  method: "POST",
  body: new URLSearchParams({ username: email, password }),
});
const user = await (await http(`${server}/api/v1/users/current`)).json(); // sends the cookie
```

- **쿠키는 네이티브에 남습니다.** Set-Cookie 헤더는 JavaScript에 노출되지 않으며 `credentials` 는 효과가 없습니다. 모든 요청은 해당 호스트에 대해 저장소가 보유한 쿠키를 보냅니다.
- **세션당 하나의 저장소.** 모든 플러그인이 공유하며 앱이 재시작되면 지워지므로, 플러그인은 다시 로그인할 준비가 되어 있어야 합니다.
- **제한:** 요청 및 응답 본문은 최대 64 MiB, 타임아웃은 120초이며, 링크 로컬 또는 클라우드 메타데이터 주소는 허용되지 않습니다. `AbortSignal` 로 네이티브 요청을 취소할 수 있습니다.

플로팅 패널

플로팅 패널은 호스트가 지도 위에 겹쳐 표시하는 드래그 및 닫기가 가능한 카드로, `position` 이 지정한 모서리(기본값 top-left)에서 열립니다. 도킹 가능한 오른쪽 패널(고정 위치에 도킹된 하나의 활성 패널)과 달리 여러 플로팅 패널을 동시에 열 수 있으며 지도를 줄이지 않습니다. 렌더링 계약은 오른쪽 패널과 같은 일반 DOM `render(container)` 입니다.

```
const unregister = app.registerFloatingPanel?>({
  id: "my-qa",
  title: "Data QA",
  defaultWidth: 300,
  render(container) {
    container.textContent =
      "Rendered by the plugin via registerFloatingPanel().";
    return () => {
      // optional cleanup, run on close/unregister
    };
  },
});

app.openFloatingPanel?("my-qa"); // open (or bring to front)
app.closeFloatingPanel?("my-qa"); // close
app.getOpenFloatingPanels?(); // -> string[] of open ids, stacking order
```

주요하고 지속적인 워크스페이스에는 오른쪽 패널을, 사용자가 지도 위에 배치하는 보조 도구나 대시보드에는 플로팅 패널을 사용하세요. 다른 표면과 마찬가지로 이 메서드들은 선택적으로 타입이 지정되어 있으므로 옵셔널 체이닝으로 호출하세요.

외부 플러그인

외부 플러그인 개발의 권장 시작점으로 [GeoLibre 플러그인 템플릿](#)을 사용하세요. 템플릿에는 MapLibre 컨트롤 래퍼, `plugin.json` 매니페스트, GeoLibre 플러그인 진입점, 그리고 GeoLibre Desktop이 기대하는 zip 레이아웃을 빌드하는 `package:geolibre` 스크립트가 포함되어 있습니다.

GeoLibre Desktop은 시작 시 앱 데이터의 `plugins/` 디렉터리에서 외부 플러그인을 로드합니다. 외부 플러그인은 신뢰된 코드이며, 로드 여부는 클라이언트 측 [배포 플러그인 정책](#)으로도 제어됩니다. 이 정책은 우회 가능하며 서명이나 샌드박싱을 제공하지 않습니다. 외부 플러그인은 다음과 같은 형태로 설치할 수 있습니다.

- 루트에 `plugin.json` 이 있는 `.zip` 파일.
- 루트에 `plugin.json` 이 있는 압축 해제된 디렉터리.
- HTTPS `plugin.json` 매니페스트 URL.

.zip 을 설치하는 가장 빠른 방법은 **Manage Plugins > Settings > Install from file**(플러그인 관리 > 설정 > 파일에서 설치)입니다. 패키징된 플러그인 아카이브를 선택하면 GeoLibre가 설치 전에 이를 검증합니다 (plugin.json 파싱, 매니페스트 규칙 적용, 진입점/스타일이 존재하고 크기 제한 이내인지 확인). 플러그인은 즉시 로드되고 유지되며, 같은 id를 다시 설치하면 이전 사본이 교체되고 업데이트된 버전이 다시 로드됩니다. 유지 방식은 빌드에 따라 다릅니다.

- **데스크톱**은 아카이브를 앱 데이터 plugins/ 디렉터리에 <plugin-id>.zip 으로 복사하며, 시작 시 스캔이 이를 다시 로드합니다.
- **웹**은 브라우저에서 아카이브를 압축 해제하여 번들을 IndexedDB에 저장하고 다음 방문 때 다시 재생합니다. 웹에서 설치한 플러그인은 **Install from file** 아래에 제거 컨트롤과 함께 나열됩니다(데스크톱 사본은 디스크에 있으며 거기서 관리됩니다).

Plugins 설정 섹션에서는 앱 데이터 폴더 외부의 로컬 개발 디렉터리도 추가할 수 있습니다. 구성된 각 디렉터리는 플러그인 zip이나 압축 해제된 플러그인 번들 폴더를 담을 수 있고, 그 자체가 하나의 압축 해제된 플러그인 번들일 수도 있습니다. 구성된 개발 디렉터리는 앱 데이터 plugins/ 디렉터리보다 먼저 스캔되므로, 개발 사본이 같은 ID의 설치된 외부 플러그인을 덮어쓸 수 있습니다. 내장 플러그인은 여전히 모든 외부 플러그인보다 우선합니다.

웹 앱에서는 매니페스트 URL 또는 **Install from file**(위 참고)을 사용하세요. 매니페스트 URL의 경우 GeoLibre는 매니페스트를 가져오고, 매니페스트 URL을 기준으로 entry 와 style 을 해석한 다음, 번들된 ESM 진입점을 로드합니다. 브라우저 로딩은 localhost 를 제외하고 HTTPS가 필요하며 호스트가 CORS를 허용해야 합니다. 파일에서 설치의 업로드된 zip을 브라우저에서 압축 해제하여(네트워크나 CORS 없이) IndexedDB에 유지합니다. 두 경로 모두 번들된 ESM 진입점을 같은 방식(웹 빌드의 script-src 가 허용하는 blob: import())으로 실행하므로, 외부 플러그인은 설치 방법과 관계없이 신뢰된 코드로 남습니다.

번들 플러그인 (빌드에 포함)

외부 플러그인을 GeoLibre의 일부로 배포하려면(Settings 항목도 매니페스트 URL도 없이 자동으로 로드됨), 빌드된 번들을 Vite public 디렉터리에 플러그인 id별로 폴더 하나씩 넣으세요.

```
apps/geolibre-desktop/public/plugins/example-plugin/
  plugin.json
  dist/index.js
  dist/style.css
```

이는 **매니페스트 URL이 제공하는 것과 같은 내용**입니다. 드롭인만 하면 되며, 플러그인별 소스 수정은 필요 없습니다. bundledPlugins() Vite 플러그인(apps/geolibre-desktop/vite-plugins/bundled-plugins.ts)이 빌드 및 개발 서버 시작 시 public/plugins/ 를 스캔하여 발견한 매니페스트 경로를 virtual:bundled-plugins 모듈을 통해 노출하고, usePlugins.ts 가 이를 일반 외부 플러그인 경로(fetch → blob import → register)로 로드합니다. 발견은 빌드 시점에 이루어지므로, 플러그인 폴더를 추가, 업데이트, 제거한 뒤에는 개발 서버를 재시작하거나 다시 빌드하세요.

같은 폴더가 웹과 데스크톱 빌드 **모두**에 사용됩니다. 데스크톱 앱은 동일한 프론트엔드(tauri.conf.json 의 frontendDist)를 번들링하여 tauri://localhost 에서 제공하는데, 이는 동일 출처이며 데스크톱 CSP (connect-src 'self', script-src ... blob:)에서 허용됩니다. 번들된 매니페스트 URL은 Settings에 저장

되지 않고 로드 시점에 주입됩니다. 번들 플러그인은 `allowed` 에 나열되거나 사이드로드를 허용하지 않아도 로드될 수 있지만, `blocked` 에 있는 id는 여전히 로드를 막습니다. 같은 id의 사용자/프로젝트 플러그인에 대해 플러그인 id로 중복이 제거됩니다.

비공개 플러그인은 `public/plugins/` 아래에서 git-ignore 처리하고(해당 폴더의 `.gitignore` 참고) 빌드/배포 시점에 복사해 넣어(예: CI에서 `npm run build` 전에, 또는 플러그인 저장소 자체의 설치 스크립트로) 그 코드가 GeoLibre의 히스토리에 남지 않게 해야 합니다. 발견 코드는 범용이며 커밋되어 있고, 플러그인 페이로드만 제외됩니다.

번들 드롭인의 `plugin.json` 은 추가로 `"activeByDefault": true` 를 설정하여 시작 시 플러그인을 활성화할 수 있습니다. 저장된 플러그인 상태가 여전히 우선합니다. `activePluginIds` 를 담은 로드된 프로젝트나 유지된 상태가 기본값을 덮어씁니다. 이 플래그는 번들 드롭인에 **대해서만** 적용되며, 차단된 플러그인 정책을 우회하지 않습니다.

대신 플러그인을 메인 JS 번들에 컴파일해 넣고 싶다면(`plugin.json` 도 `fetch`도 없이), 내장 플러그인으로 등록하세요([내장 플러그인 추가](#) 참고).

```
{
  "id": "example-plugin",
  "name": "Example Plugin",
  "version": "0.1.0",
  "entry": "dist/index.js",
  "description": "Optional short description",
  "style": "dist/style.css",
  "engines": ["maplibre", "cesium"]
}
```

`entry` 파일은 `GeoLibrePlugin` 을 기본 내보내기 또는 `plugin` 이라는 이름의 내보내기로 내보내야 합니다. 내보낸 플러그인의 `id`, `name`, `version` 은 `plugin.json` 과 일치해야 합니다. 이 첫 번째 로더는 zip 내부의 상대 모듈 임포트를 해석하지 않으므로 진입점은 자체 완결적인 `.js` 또는 `.mjs` 번들이어야 합니다. 선택적 `engines` 배열은 플러그인이 GeoLibre의 네 가지 지도 렌더러 중 어느 것을 지원하는지 선언합니다(`"maplibre" | "mapbox" | "cesium" | "arcgis"`, 기본값 `["maplibre"]`). 네이티브 지구본을 지원하는 플러그인은 `"cesium"` 을 추가하고, Style Spec 표면에 머무는 플러그인은 `"mapbox"` 를 추가할 수 있으며([Mapbox 렌더러 지원](#) 참고), ArcGIS 네이티브 어댑터가 있는 플러그인은 `"arcgis"` 를 추가할 수 있습니다([ArcGIS 렌더러](#) 참고). 호스트는 선택된 엔진이 이 목록에 없으면 플러그인을 일시 중단하고, 호환되는 엔진이 활성화되면 복원합니다.

외부 플러그인 진입점은 `import(URL.createObjectURL(...))` 로 실행되며, 이것이 `tauri.conf.json` 의 데스크톱 CSP가 `script-src` 에 `blob:` 을 포함하는 이유입니다. `script-src` 에서 `blob:` 을 제거하면 외부 플러그인 로딩이 깨집니다. `'unsafe-eval'` 과 결합하면 blob URL을 만들 수 있는 코드는 스크립트를 실행할 수 있다는 뜻인데, 외부 플러그인은 사용자가 설치한 신뢰된 로컬 파일이므로 허용 가능합니다.

플러그인은 호스트 문서에서 신뢰된 코드로 실행되므로 런타임 환경 맵인 `window.__GEOLIBRE_RUNTIME_ENV__` 를 읽을 수 있습니다. 데스크톱 앱에서 이 맵에는 Settings → Environment Variables에 입력된 값뿐 아니라 AI Assistant의 **OS 환경 키**(사용자 셸에서 읽은 허용 목록의 공급자 변수)도 포함됩니다. 앱 환경을 통해 접근 가능한 모든 자격 증명은 설치된 플러그인에 보인다고 간주하고, 신뢰하는 플러그인만 설치하세요.

플러그인 프로젝트 설정은 프로젝트 유출 경계에서 민감 정보로 취급됩니다. GeoLibre는 사용자가 자격 증명 유지를 명시적으로 선택한 경우에만 신뢰된 로컬 저장에 이를 보관하며, 공유, 독립형 HTML 내보내기, 임베드 스냅샷, 협업 스냅샷에서는 `plugins.settings` 객체 전체를 제거합니다. 예외는 플러그인의 레지스트리 항목이 공개 가능하다고 선언한 상태(플러그인 마켓플레이스의 `publishableSettings` 참고)로, 이는 유지되지만 여전히 자격 증명 정리됩니다. 수신자에게 필요하다면 브로커 참조와 같은 이동 가능하고 비밀이 아닌 식별자는 대신 레이어 소스나 메타데이터에 저장하세요.

매니페스트 경로는 슬래시를 사용하는 상대 zip 경로여야 하며, 선행 슬래시, 백슬래시, `..` 세그먼트가 없어야 합니다. 외부 플러그인은 내보낸 플러그인 객체에 `activeByDefault` 를 설정할 수 없으며, 매니페스트 수준의 플래그는 번들 드롭인에 대해서만 적용됩니다(위의 "번들 플러그인" 참고). 저장된 프로젝트 상태는 zip이 로드된 뒤 ID로 외부 플러그인을 다시 활성화할 수 있습니다.

선택적 `style` CSS는 플러그인에 범위가 한정되지 않고 호스트 문서에 전역으로 주입됩니다. 플러그인 작성자는 광범위한 규칙이 앱의 나머지 부분을 다시 스타일링하지 않도록 선택자의 범위를 한정할 책임이 있습니다(예: 플러그인 전용 클래스 접두사 사용). 주입된 CSS는 `url()` 참조와 `@import` 를 통해 네트워크 요청을 보낼 수도 있으므로, 플러그인 스타일시트는 외부 폰트, 이미지, 추가 시트를 로드할 수 있습니다. 플러그인 CSS를 플러그인 코드와 같은 신뢰 기대치로 다루세요.

템플릿을 사용할 때는 `geolibre-plugin/plugin.json` 과 `src/geolibre.ts` 를 함께 업데이트하여 `id`, `name`, `version` 이 동기화되도록 하세요. `npm run package:geolibre` 를 실행한 뒤, 생성된 zip을 데스크톱 앱 데이터 `plugins/` 디렉터리에 복사하거나, 로컬 개발을 위해 Settings > Plugins에 템플릿의 `geolibre-plugin/` 디렉터리를 추가하거나, `geolibre-plugin/` 디렉터리를 호스팅하고 그 `plugin.json` URL을 추가하세요.

플러그인 마켓플레이스

Settings 메뉴의 **Manage Plugins**(플러그인 관리) 항목은 **All**(전체), **Installed**(설치됨), **Not installed**(설치되지 않음), **Upgradeable**(업그레이드 가능), **Settings**(설정) 섹션이 있는 독립형 대화상자(QGIS의 플러그인 관리자를 본뜬)를 엽니다. 앞의 네 섹션은 엄선된 레지스트리 플러그인을 나열하여 사용자가 매니페스트 URL을 직접 입력하지 않고도 설치, 업데이트, 제거할 수 있게 하고, Settings 섹션은 로컬 `.zip` 에서 플러그인을 설치하고 추가 로컬 플러그인 디렉터리와 수동 매니페스트 URL을 관리합니다. 동작은 즉시 적용됩니다(설치/제거/업데이트는 실시간이며, 제거는 확인을 요청합니다). 이는 위의 매니페스트 URL 로더 위에 얹힌 얇은 계층입니다. 항목을 설치하면 플러그인 매니페스트 URL 목록에 그 매니페스트 URL이 기록되고, 기존 로더가 이를 가져와 등록합니다. 새로운 신뢰 경로는 도입하지 않습니다.

레지스트리는 JSON이며, 기본 정책의 `plugins.registryUrl`, 그다음 레거시

`VITE_GEOLIBRE_PLUGIN_REGISTRY_URL` 설정, 또는 기본적으로 `https://plugins.geolibre.app/plugin-registry.json` 의 호스팅된 레지스트리 ([opengeos/geolibre-plugins](https://plugins.geolibre.app/plugin-registry.json) 저장소, CORS가 활성화된 GitHub Pages 에 게시됨)에서 가져옵니다. 다음과 같은 항목의 배열이거나 `plugins` 배열을 가진 객체입니다.

```
{
  "version": 1,
  "plugins": [
    {
      "id": "example-plugin",
      "name": "Example Plugin",

```

```

    "version": "1.0.0",
    "description": "Optional short description",
    "author": "Example Author",
    "homepage": "https://github.com/example/example-plugin",
    "manifestUrl": "https://example.com/example-plugin/plugin.json",
    "categories": ["Example"],
    "minGeoLibreVersion": "1.0.0",
    "publishableSettings": ["search"],
    "bundleSha256": "3f5c...(64 hex characters)"
  }
]
}

```

`id`, `name`, `version`, `manifestUrl` 은 필수이고 나머지는 선택 사항입니다. 상대 `manifestUrl` 은 레지스트리 위치를 기준으로 해석되므로, 레지스트리와 함께 호스팅되는 플러그인(예: `sample/plugin.json`)은 상대 경로로 나열할 수 있습니다. `minGeoLibreVersion` 은 실행 중인 앱 버전에 대해 설치를 제한합니다.

`publishableSettings` 는 선택 사항이며 플러그인의 프로젝트 상태(`getProjectState()`)가 "Strip credentials"(자격 증명 제거)와 공유 또는 내보낸 프로젝트에서 살아남게 합니다. 기본적으로 외부 플러그인의 상태 전체는 무엇이든 담을 수 있기 때문에 그 지점에서 삭제되며 자격 증명을 포함하는 것으로 간주됩니다. 공개해도 안전한 최상위 상태 키를 나열하거나(`["search", "filters"]`), `true` 를 사용해 상태 전체를 유지하세요. 이 선언은 레지스트리 항목과 함께 검토되며 프로젝트 파일에서 읽히는 일은 없습니다. 유지되는 것도 여전히 자격 증명 이름의 필드와 자격 증명에 포함된 URL이 정리되며, 레지스트리 항목은 내장 플러그인의 목록을 넓힐 수 없습니다. 비밀은 그 키에 넣지 말고 `app.credentials` 를 사용하세요.

`bundleSha256` 은 선택 사항으로, 게시된 번들의 소문자 16진수 SHA-256이며 `plugin-integrity.ts` 의 `computePluginBundleHash` 와 같은 방식으로 계산합니다(진입점의 SHA-256, 스타일 또는 빈 문자열의 SHA-256, 그다음 두 다이제스트의 SHA-256). 호스팅된 레지스트리는 제공하는 모든 플러그인에 대해 이를 생성합니다. 항목에 해시가 있으면 마켓플레이스나 `?plugin=` 딥 링크로 설치할 때 URL이 설치되기 전에 그 해시를 고정하므로, 첫 로드는 URL이 처음 제공하는 것을 신뢰하는 대신 다운로드한 코드를 검토된 해시와 대조합니다. 불일치하면 변경된 번들과 마찬가지로 보류됩니다. Update 동작도 마찬가지로 평가 전에 일치하지 않는 다운로드를 거부합니다. `bundleSha256` 이 없는 항목은 최초 사용 시 신뢰(trust-on-first-use) 고정을 유지합니다.

레지스트리 관리자는 레지스트리 옆에 게시된 `blocklist.json` (호스팅된 레지스트리는 `https://plugins.geolibre.app/blocklist.json`, 구성된 `registryUrl` 의 경우 그 옆의 `blocklist.json`)을 통해 플러그인을 내릴 수도 있습니다. GeoLibre는 외부 플러그인을 로드하기 전에 세션당 한 번 이를 가져오고 오프라인 시작을 위해 캐시합니다. `id` 만 있는 항목은 모든 외부 소스(레지스트리, 매니페스트 URL, zip, 플러그인 디렉터리)에서 해당 플러그인의 모든 버전을 차단합니다. 설치와 로드는 배포 정책 `blocked` 와 같은 정책 게이트를 통해 거부되며, 항목의 `reason` 이 표시됩니다. `bundleSha256` 도 있는 항목은 모든 외부 소스에서 해당 번들만 차단합니다. 정확히 그 코드를 가진 URL 플러그인, zip 아카이브, 플러그인 디렉터리는 로드되거나 설치되지 않고, 그것으로의 Update도 거부되며, 모두 코드가 실행되기 전에 이루어집니다. 번들 드롭인은 절대 차단되지 않습니다. 차단 목록을 가져올 수 없으면(또는 목록이 캐시된 뒤 404를 반환하면) 캐시된 사본이 적용되며, 차단 목록을 게시한 적 없는 레지스트리는 아무것도 차단하지 않습니다.

레지스트리를 관리하고 플러그인 번들을 호스팅하려면 `sample/` 템플릿을 제공하는 opengeos/geolibre-plugins 저장소를 사용하세요.

외부에서 로드된 플러그인(레지스트리 설치, zip, 매니페스트 URL, 번들 드롭인)은 내장 항목에 섞이지 않고 Plugins 메뉴 하단의 **Plugins → Installed**(플러그인 → 설치됨) 하위 메뉴에 표시 이름의 알파벳순으로 함께 나열됩니다. 이 하위 메뉴는 외부 플러그인이 하나 이상 로드되었을 때만 나타나며 **Manage Plugins...**(플러그인 관리...) 바로가기로 끝납니다. 자체 최상위 메뉴도 등록하는 플러그인([툴바 메뉴](#) 참고)은 Installed 항목도 함께 유지합니다.

제거는 확인을 요청한 뒤 런타임에 플러그인의 등록을 해제하여(활성 지도 컨트롤을 비활성화) 다시 로드하지 않고도 Plugins 메뉴가 업데이트됩니다. 레지스트리 항목이 로드된 플러그인보다 새로운 version 을 광고하면 마켓플레이스는 매니페스트 URL을 다시 가져와 게시된 버전을 제자리에 다시 등록하는 Update 동작을 표시합니다. 새 버전은 이전 버전이 제거되기 전에 가져와 검증되므로, 업데이트에 실패해도 설치된 플러그인은 그대로 유지됩니다.

어시스턴트 도구

플러그인은 같은 동작을 패널과 AI Assistant 모두에 노출할 수 있습니다. 외부 플러그인은 @strands-agents/sdk 나 zod를 임포트하거나 번들링하지 않고, 일반 JSON Schema와 콜백으로 app.registerAssistantToolSpec 을 사용할 수 있습니다.

```
let disposeTool;

export default {
  id: "city-loader",
  name: "City loader",
  version: "1.0.0",
  activate(app) {
    // Share this function with the panel's Add button.
    async function addCities(input) {
      if (
        !input ||
        typeof input !== "object" ||
        typeof input.name !== "string"
      ) {
        throw new Error("name must be a string");
      }
      const response = await fetch(
        "https://raw.githubusercontent.com/opengeos/leafmap/master/examples/data/us_cities.geojson"
      );
      if (!response.ok) throw new Error(`Download failed: ${response.status}`);
      const data = await response.json();
      const layerId = app.addGeoJsonLayer(input.name, data);
      return { layerId, featureCount: data.features.length };
    }
    disposeTool = app.registerAssistantToolSpec?({
      name: "add_cities",
      description: "Add US cities to the map as a GeoJSON layer",
      inputSchema: {
        type: "object",
        properties: { name: { type: "string" } },
        required: ["name"],
        additionalProperties: false,
      },
      callback: addCities,
    });
  },
  deactivate() {
    disposeTool?.();
  }
};
```



```
disposeTool = undefined;
  },
};
```

JSON Schema 입력은 자동으로 검증되지 않습니다. 콜백은 동작을 수행하기 전에 모델이 제공한 입력을 검증해야 합니다. JSON 직렬화 가능한 데이터 또는 그 프로미스를 반환하세요. `undefined` 를 반환하는 콜백은 `null` 을 만듭니다. 던져진 오류는 어시스턴트 도구 오류 결과가 됩니다. `inputSchema` 를 생략하면 추가 속성이 없는 빈 객체를 기술합니다.

이미 SDK를 임포트하는 내장 플러그인은 대신 `app.registerAssistantTool(tool({ name, description, inputSchema, callback })))` 를 호출하여 자동 검증을 위해 zod 스키마와 함께 SDK의 `tool()` 팩토리를 사용할 수 있습니다. 두 진입점 모두 하나의 레지스트리를 사용하며 등록 해제 함수를 반환합니다. 외부 플러그인은 이전 호스트를 위해 이 선택적 메서드들의 존재를 확인해야 합니다.

호스트가 플러그인 소유자를 주입하며, 플러그인이 제공한 소유자 인자는 무시합니다. 모델에 보이는 이름은 `plugin_<owner-length>_<plugin-id>_<tool-name>` 입니다. 이름과 플러그인 ID는 글자, 숫자, 밑줄, 하이픈을 사용하며, 전체 이름은 64자 이내여야 합니다. 대소문자나 밑줄/하이픈 표기만 다른 이름은 충돌하면 거부됩니다. 같은 소유자에 대해 같은 이름을 등록하면 교체되며, 이전 디스포저는 교체된 것을 제거할 수 없습니다. 도구는 비활성화, 활성화 실패, 플러그인 제거 시 제거되며, 해체 후의 오래된 비동기 등록은 무시됩니다.

`activate` 에서(또는 그것이 넘겨주는 `app`에서, 비동기 포함) 등록하세요. 도구는 정확히 한 번의 활성화 동안만 존재하므로, 다른 수명 주기 콜백 (`applyProjectState`, `setMapControlPosition`, `handleUrlParameters`, `deactivate`)에 전달되는 `app`에는 두 메서드가 모두 없습니다. 이들은 비활성 플러그인에 대해서도 실행되며, 그 등록에는 어떤 정리 경로도 도달하지 못하기 때문입니다.

어시스턴트는 대화 기록을 유지하면서 다음 프롬프트 전에 도구를 새로 고칩니다. 플러그인 콜백은 패널 버튼처럼 플러그인이 작성한 코드를 실행하며, 레이어와 기타 앱 상태를 업데이트할 때는 `app API`를 사용해야 합니다.

점진적 도구 공개

활성 플러그인이 등록한 도구가 총 12개 이하인 동안에는 모든 플러그인 도구가 매 요청마다 모델에 전부 전송됩니다. 그 이상이 되면 어시스턴트는 플러그인 도구 스키마를 미리 보내는 것을 중단하므로, 설치된 플러그인마다 도구 목록이 늘어나지 않습니다. 대신 시스템 프롬프트가 `Plugin tools:` 제목 아래 플러그인별로 묶어 각 플러그인 도구를 이름과 한 줄 요약(`description` 첫 줄의 첫 문장, 최대 160자)과 함께 나열합니다. 모델은 호스트 도구 `load_plugin_tools`에 정확한 도구 이름을 전달하거나 플러그인 id를 전달하여 그 플러그인의 모든 도구를 로드함으로써 필요한 도구를 로드합니다. 로드된 도구는 같은 턴에서 호출 가능해지며 대화의 나머지 동안 로드된 상태로 유지됩니다. GeoLibre 자체의 도구는 항상 전송됩니다.

모델이 도구를 로드하기 전에 보는 것은 요약뿐이므로, 각 `description` 을 **도구가 무엇을 하고 어떤 데이터를 다루는지 말하는 문장으로 시작하세요.** 그래야 다른 플러그인이나 도메인에서 같은 형태를 가진 도구들(예:

`query_*_database`)을 스키마 없이도 구분할 수 있습니다. `registerAssistantGuidance` 로 등록된 지침은 여전히 전부 전송되며 도구 이름을 언급할 수 있습니다. 모델은 호출 전에 그 도구들을 로드합니다.

어시스턴트 지침

도구의 설명은 모델이 이미 어떤 도구를 호출할지 결정한 뒤에야 읽히며, 그 결정은 시스템 프롬프트에 의해 이루어 집니다. 플러그인의 도구에 언제 사용해야 하는지에 대한 규칙이 필요하다면(예: "get_pm25_ranking 을 직접 호출 하고 run_sql 안의 테이블 함수로는 절대 사용하지 말 것"), 그 텍스트를 지침으로 등록하세요. 호스트가 어시스턴 트의 시스템 프롬프트에 이를 덧붙입니다.

```
let disposeGuidance;

export default {
  id: "air-quality",
  name: "Air quality",
  version: "1.0.0",
  activate(app) {
    // ...register tools as above...
    disposeGuidance = app.registerAssistantGuidance?.(
      [
        "For PM2.5 questions, call plugin_11_air-quality_get_pm25_ranking directly",
        "with its own arguments. Never wrap it in run_sql or use it as a FROM clause;",
        "it is not a SQL table function.",
      ].join(" ")
    );
  },
  deactivate() {
    disposeGuidance?.();
    disposeGuidance = undefined;
  },
};
```

지침은 GeoLibre 자체 프롬프트 뒤의 `Plugin guidance`: 제목 아래에 등록 순서대로 덧붙여지며, 각 블록에는 이 를 등록한 플러그인을 나타내는 `[plugin <id>]` 레이블이 붙고, 호스트 텍스트를 교체하거나 편집하는 일은 없습 니다. 이 제목은 해당 텍스트가 플러그인 자체 도구를 언제 어떻게 호출할지만 다루며 호스트 지침은 여전히 적용된 다는 것을 모델에 알립니다. 텍스트 자체는 필터링되지 않습니다. 플러그인 코드와 마찬가지로 플러그인이 로드되면 신뢰되므로, 신뢰하는 플러그인만 설치하세요. 최대 4000자의 비어 있지 않은 문자열이어야 하며, 같은 플러그인의 동일한 텍스트는 반복되는 대신 이전 등록을 교체합니다. 도구와 마찬가지로 지침은 활성화 전용입니다. 호스트가 플 러그인 소유자를 주입하고, 플러그인이 제공한 소유자 인자는 무시하며, 비활성화, 활성화 실패, 플러그인 제거 시 텍 스트를 제거하고, 다른 수명 주기 콜백에 전달되는 app에는 이 메서드가 없습니다. 어시스턴트는 대화 기록을 유지 하면서 다음 프롬프트 전에 도구와 함께 시스템 프롬프트를 다시 구성합니다. 이전 호스트를 위해 메서드의 존재를 확인하세요.

지도 렌더러

호스트는 현재 기본 렌더러를 읽을 수 있도록 `app.getMapRenderer()` 를 노출합니다. 엔진 선언은 플러그인 매니 저가 활성화, URL 매개변수, 프로젝트 복원, 지연된 컨트롤 등록에 대해 강제하며, Plugins 메뉴와 명령 팔레트도 이 를 적용합니다. 렌더러가 바뀌면 지원되지 않는 플러그인은 일시 중단되고, 돌아올 때를 위해 저장된 설정과 활성화 상태가 유지됩니다. 호환되는 플러그인은 교체된 렌더러에 컨트롤을 다시 마운트합니다.

뷰포트 읽기

사용자가 볼 수 있는 범위로 검색을 좁히는 카탈로그나 서비스 브라우저는 `app.getMap()?.getBounds()` 가 아니라 `app.getViewBounds()` 를 통해 범위를 읽어야 합니다. `getMap()` 은 지구본에서 null이므로 두 번째 형태는 거기서 범위를 내놓지 못합니다. 그리고 "범위 없음"을 "필터 없음"으로 해석하는 플러그인은 "현재 뷰만" 체크박스가 켜진 채로 전 세계를 검색하게 되는데, 이것이 바로 `engines` 선언이 배제하려는 조용한 성공입니다.

`getViewBounds` 는 어느 엔진이 기본이든 그 엔진에서 답하며, 앱의 다른 모든 곳에서 `MapExtent` 가 그러하듯 날짜변경선 횡단(`east > 180`)을 펼쳐 줍니다.

`getViewBounds` 에는 자체적인 `null` 이 있습니다. 아직 마운트된 지도가 없거나, 지구본이 장면 모드 사이에서 변형 중이거나, 카메라가 지구 반대편을 향하고 있는 경우입니다. 이를 "필터 없음"으로 해석하지 마세요. 검색을 넓히는 것은 같은 거짓말을 두 번째 장소에서 반복하는 것입니다. ArcGIS Hub 패널이 그러하듯 검색을 거부하고 범위를 사용할 수 없다고 알려세요.

프런트엔드 테스트는 Cesium 지원을 선언한 모든 플러그인을 스캔하고, 상대 임포트를 따라가 하위 디렉터리에 나뉜 플러그인도 포함하며, `getMap()` 을 거친 범위 읽기(체이닝되었든 두 문장으로 나뉘었든)에 대해 실패합니다.

`getMap()` 이 null인지를 의도적으로 분기하는 호출에는 그 줄 자체나 바로 위에 이유와 함께 `engine-audit-allow: getMap-bounds` 주석을 붙입니다. 이 옴트아웃은 해당 호출에만 적용되므로, 같은 모듈의 다른 곳에 있는 두 번째 범위 읽기는 여전히 보고됩니다.

렌더러를 직접 구동하는 플러그인은 어느 핸들이 null이 아닌지에 따라 분기합니다. MapLibre에서는

`app.getMap()` , 지구본에서는 `app.getCesiumScene()` 입니다. Cesium 핸들은 실시간 위젯, 장면, 카메라, 시계와 함께 `@cesium/engine` 네임스페이스를 담고 있으므로, 플러그인은 엔진 자체를 임포트하지 않고도 Cesium 객체(`SunLight` , `JulianDate` , `Cartesian3`)를 생성합니다. 이것이 Cesium을 2D 부팅 경로에서 제외하는 방법입니다. `primary` 플러그는 기본 지도 영역과 그리드 창을 구분합니다. 내장 환경 플러그인은 MapLibre에서처럼 기본 지도에만 바인딩됩니다. Sun 시뮬레이션(네이티브 `SunLight` , 지구본 조명, 장면 시계), Atmospheric Effects(스카이 박스, `SkyAtmosphere` 색조/채도/밝기 변경, 배경색), Flight Simulator(내비게이션 입력을 중단한 채 매 프레임 `camera.setView()`), Clouds / Precipitation 오버레이(스토어 타일 레이어)가 이 패턴의 참조 구현입니다. 지구본에 바인딩된 플러그인은 `deactivate` 에서 자신이 변경한 장면 상태를 모두 복원해야 합니다. 렌더러 교체 시 두 엔진이 모두 다시 빌드되고 호스트가 호환되는 플러그인을 새 엔진에 대해 다시 활성화하기 때문입니다.

래스터 값 샘플링

`app.readRasterWindow(layerId, options)` 는 래스터 레이어의 한 밴드를 지리적 창에 걸쳐 작은 그리드로 다운샘플링하여 읽습니다. 뷰포트 스트레치가 사용하는 것이 바로 이것입니다. `getViewBounds()` 와 함께 사용하여 전체 장면이 아니라 실제로 화면에 있는 픽셀로부터 리스케일 범위를 계산하세요.

레이어를 사용할 수 없을 때(없거나, 컨트롤이 로드한 래스터가 아닐 때) `null` 로 해결됩니다. 래스터와 교차하지 않는 범위는 `null` 이 **아닙니다**. 그것은 빈 `values` 를 가진 읽기 결과이므로, null만 확인하지 말고 길이도 확인하세요. 요청한 `width / height` 에 가장 적합한 오버뷰에서 읽으므로, 대륙 위의 32x32 창은 샘플마다 요청 하나가 아니라 저렴한 일괄 읽기입니다.

values 는 행 우선(row-major)이며, 셀을 사용할 수 없게 되는 두 가지 경우는 서로 다릅니다. 읽을 수 없는 타일은 NaN 으로 돌아오지만, **NoData 픽셀은 센티널 값을 유지**하며, -9999 같은 센티널은 완전히 유한한 값입니다. 따라서 Number.isFinite 만으로 필터링하면 채움 값이 남아 min/max 스트레치를 그 값까지 끌어내립니다. reading.nodata 와도 비교하세요.

```
const bounds = app.getViewBounds?();
if (!bounds) return; // no map mounted, or the globe is mid-morph
const reading = await app.readRasterWindow?.(layerId, {
  bounds,
  band: 1,
  width: 32,
  height: 32,
  signal: controller.signal,
});
const values =
  reading?.values.filter(
    (value) => Number.isFinite(value) && value !== reading.nodata
  ) ?? [];
if (values.length === 0) return; // window missed the raster entirely
```

카메라 이동에 의해 구동되는 모든 것에는 signal 을 전달하고, 새 읽기가 시작될 때 이전 읽기를 중단하세요. 그래야 빠른 이동이 결과가 순서 없이 도착하는 읽기 적체를 만들지 않습니다.

Mapbox 렌더러 지원

Mapbox 렌더러는 Mapbox GL JS로 그리며, 그 런타임 API는 MapLibre가 파생되어 나온 Style Spec 표면입니다. 소스와 스타일 레이어 (addSource , addLayer , setPaintProperty , setLayoutProperty , setFilter , getSource , getLayer , getStyle , moveLayer), 이미지 (addImage , updateImage), 카메라(getBounds , getZoom , easeTo , jumpTo , fitBounds , project , unproject), 이벤트(on / off / once), DOM(getCanvas , getContainer), 피킹(queryRenderedFeatures)입니다. IControl 도 같은 계약이며, app.addMapControl 은 .maplibregl-ctrl-* 모서리 클래스를 별칭 처리하는 어댑터를 통해 MapLibre 타입의 컨트롤을 Mapbox 지도에 마운트합니다. Web Services 패널이 사용하는 도킹된 오른쪽 패널 브리지도 같은 문을 통해 마운트됩니다. 그 표면에 머무는 플러그인은 engines: ["maplibre", "mapbox"] 를 선언하고, 사용자는 두 2D 엔진 중 어느 쪽에서든 Plugins 메뉴에서 이를 켜고 끌 수 있습니다. 렌더러 교체 시 플러그인 매니저가 새 지도에 대해 이를 다시 활성화합니다.

app.getMap() 은 Mapbox에서 의도적으로 null 이므로, 그런 플러그인은 그것만으로 지도에 접근해서는 안 됩니다. 이는 위의 지구본 규칙이 방지하는 것과 같은 조용한 무동작입니다. 내장 플러그인은 getStyleMap(app) (packages/plugins/src/plugins/style-map.ts)을 통해 지도를 읽습니다. MapLibre 지도가 있으면 그것을, 없으면 MapLibre 타입을 통해 Mapbox 지도를(컨트롤을 호스팅할 때 엔진 자체가 적용하는 캐스트) 반환합니다. 외부 플러그인은 app.getMap?().() ?? app.getMapboxMap?().() 으로 같은 일을 합니다. 이 캐스트는 공유 표면에 대해서만 정직합니다. mapbox-gl 지도에는 MapLibre의 확장이 전혀 없습니다.

- addProtocol / removeProtocol — pmtiles:// , COG 및 기타 커스텀 타일 프로토콜은 로드되지 않으며, 이것이 COG 래스터 컨트롤이 MapLibre 전용으로 남는 이유입니다(Overture Maps는 대신 maplibregl-overture-maps 에 일반 .pmtiles URL을 요청하며, mapbox-gl 3.30+는 자체 타일 공급자를 통해 이를 읽습니다). setTransformRequest — Mapbox는 생성 시에만 transformRequest 를 받으므로,

GeoLens 비공개 래스터(요청마다 API 키가 주입됨)는 MapLibre 전용입니다. 커스텀

CustomLayerInterface 레이어(Mapbox에서는 capabilities.customLayers 가 false). 지형 카메라 헬퍼 (calculateCameraOptionsFromCameraLngLatAltRotation , getCenterClampedToGround):

Mapbox는 이들이 대체한 자유 카메라를 유지하므로, Flight Simulator는 거기서

setFreeCameraOptions 와 고도를 담은 MercatorCoordinate 를 통해 비행합니다. 일부 업스트림 컨트롤이 읽는 transform / _camera 내부.

- getProjection() 은 형태가 다릅니다. MapLibre에서는 { type: "globe" }, Mapbox에서는 { name: "globe" } 입니다. setProjection 은 MapLibre에서 { type } 을, Mapbox에서 이름 문자열(또는 { name })을 받습니다. 둘 다 읽으세요.
- maplibre-gl 에서 임포트한 MapLibre의 Popup 및 Marker 클래스는 mapbox-gl 지도에서 동작하지 않습니다. 그 업데이트 경로가 map._camera.transform 을 읽어 첫 이동에서 예외를 던집니다. 두 엔진 모두에서 마커가 필요한 플러그인은 Elements 패널이 그러하듯 대신 map.project 를 통해 DOM 요소를 배치합니다. 업스트림 라이브러리가 엔진 자체의 클래스를 생성해야 한다고 고집하면 app.getMapboxGl() 이 mapbox-gl 네임스페이스를 내어 줍니다. Geo Editor는 그 Marker / LngLatBounds 를 Geoman의 지도 어댑터에, Popup 을 maplibre-gl-geo-editor 의 createPopup 옵션(geo-editor-mapbox.ts)에 전달하고, Street View는 Marker 를 maplibre-gl-streetview 의 createMarker 옵션에 전달하며, Layer Swipe는 클립된 비교 창을 만드는 maplibre-gl-swipe 의 createMap 옵션에 Map 을 전달하고, GeoAgent는 maplibre-gl-geoagent 의 mapEngine 옵션에 네임스페이스 전체를 넘깁니다(geoagent-map-engine.ts). 좁힌 부분집합이 아닌 이유는 그 run_maplibre_script 도구가 실행하는 스크립트에 이를 그대로 전달하기 때문입니다. 업스트림의 패턴은 매번 같습니다. 라이브러리는 요소와 그 스타일링을 유지하고, 요소를 배치하는 엔진 클래스만 받습니다. 두 번째 Mapbox 지도를 생성하는 플러그인은 생성자 옵션에 app.getMapboxAccessToken() 도 전달해야 합니다. mapbox-gl은 토큰을 건네받지 않으면 전역 mapboxgl.accessToken 에서 읽는데, GeoLibre는 지도별로 이를 설정하므로 토큰 없이 만든 두 번째 지도는 아무것도 렌더링하지 않고 매 프레임 로그를 남깁니다.

Cesium 지원 플러그인을 스캔하는 같은 프런트엔드 감사가 Mapbox 지원을 선언한 모든 플러그인을 스캔하고, 상대 임포트를 따라가며, 같은 줄에서 getMapboxMap 으로 폴백하지 않는 app.getMap() 을 통한 읽기(의도적인 MapLibre 감지 분기에는 engine-audit-allow: getMap-mapbox 를 표시)와 위의 MapLibre 전용 멤버 사용 (런타임 엔진 검사 뒤의 호출에는 engine-audit-allow: maplibre-only)에 대해 실패합니다. 두 옵트아웃 모두 해당 호출에만 적용되며, 그 줄 또는 바로 위에 둡니다.

스토어 레이어는 두 2D 엔진에서 같은 방식으로 동작합니다. 컨트롤이 네이티브로 만들고 외부 네이티브 레이어로 스토어에 반영한 래스터 또는 WMS 레이어 (metadata.sourceId 와 nativeLayerIds 를 가진

externalNativeLayer: true , Web Services, 베이스맵, Esri Wayback, USGS LiDAR 인덱스 컨트롤이 그러하듯)는 Mapbox 엔진이 그 네이티브 id 아래에서 넘겨받습니다. 두 번 그려지지 않고, 스토어의 가시성/불투명도/제거가 컨트롤의 레이어에 적용되며, 스타일 다시 로드 시 다시 빌드됩니다. MapLibre의 레이어 동기화와 정확히 같습니다. 엔진이 source 로부터 컴파일할 수 없는 스토어 레이어(그릴 수 있는 소스가 없는

registerExternalNativeLayer 레코드, 또는 플러그인만 픽셀을 만들 수 있는 종류(time-slider , timelapse , openaerialmap-footprints))는 플러그인 소유로 간주됩니다(packages/map/src/mapbox-layers.ts 의 isMapboxPluginLayer). 엔진은 그리기를 플러그인에 맡기고, 공유 paint 빌더를 통해 스토어의 가시성과 불투명도만 플러그인의 nativeLayerIds 에 반영하며, metadata.controlOwnsPaint 를 존중합니다.

MapLibre 컨트롤이 지구본에서 얻는 것

`app.addMapControl` 은 Cesium 아래에서도 동작합니다. 지구본은 컨트롤의 DOM을 캔버스 위의 같은 네 개의 `.maplibregl-ctrl-{top,bottom}-{left,right}` 모서리 컨테이너에 마운트하므로(`index.css` 의 범위 한정 CSS가 계속 적용됨), `onAdd` 에 실제 Map 이 아니라 Cesium 장면 위의 MapLibre 형태 파사드를 넘깁니다. 이 파사드는 다음에 응답합니다.

- `getContainer`, `getCanvas`, `isStyleLoaded`, 그리고 `Evented` 메서드 (`on` / `off` / `once` / `fire`). `getContainer` 는 크기가 지정된 캔버스 부모를 반환하므로 컨트롤이 패널을 고정할 수 있습니다. 카메라 `movestart`, `move`, `moveend`, 캔버스 `resize`, 지리적 마우스 이벤트는 구독에 도달합니다. 우주 위의 포인터 이벤트는 지상 위치가 없으므로 생략됩니다. 호스트는 지구본이 파괴될 때 이 구독들을 제거합니다.
- 스토어의 지도 뷰에서 가져온 `getCenter`, `getZoom`, `getBearing`, `getPitch`, 그리고 이를 다시 써서 동작하는 `jumpTo` / `flyTo` / `easeTo`.
- 실시간 장면에서 가져온 `project`, `unproject`, `getBounds`. 좌표는 지형 표면에 투영되고, 화면 점은 지형에 대해 피킹된 뒤 타원체에 대해 피킹되며, 범위는 카메라의 뷰 사각형에서 나옵니다. 장면이 배치할 수 없는 좌표는 화면 밖 창 좌표로 읽히고, 지구본을 벗어난 화면 점은 뷰 중심으로 역투영되며, 경계가 있는 뷰 사각형이 없는 카메라는 전 세계를 보고합니다. MapLibre의 지구본 투영이 응답하는 것과 같은 형태이므로, 컨트롤이 렌더링 도중 예외를 던지는 대신 계속 실행됩니다.
- 프로젝트 베이스맵으로 라우팅되는 `setStyle(url)`.

`addSource`, `removeSource`, `addLayer`, `removeLayer`, `setPaintProperty`, `setLayoutProperty`, `getStyle` 을 포함해 Mapbox Style Spec을 통해 그리는 모든 것은 **예외를 던집니다**. 이것이 정직한 경계입니다. 자체 지도 레이어를 그리는 컨트롤에는 지구본 표현이 없으며, 조용한 무동작은 아무것도 나타나지 않는데도 성공을 보고하게 만듭니다. `addMapControl` 은 그 예외를 잡아 `false` 를 반환하므로, 이에 걸리는 컨트롤은 플러그인 활성화 화를 함께 무너뜨리는 대신 마운트에 실패합니다. 컨트롤에 그 메서드들이 필요한 플러그인은 기본값인 `engines: ["maplibre"]` 를 유지하고, 지구본이 중요하다면 `app.getCesiumScene()` 을 통한 Cesium 분기를 추가해야 합니다.

<https://geolibre.app/collaboration/>

실시간 협업(라이브 동기화 세션)

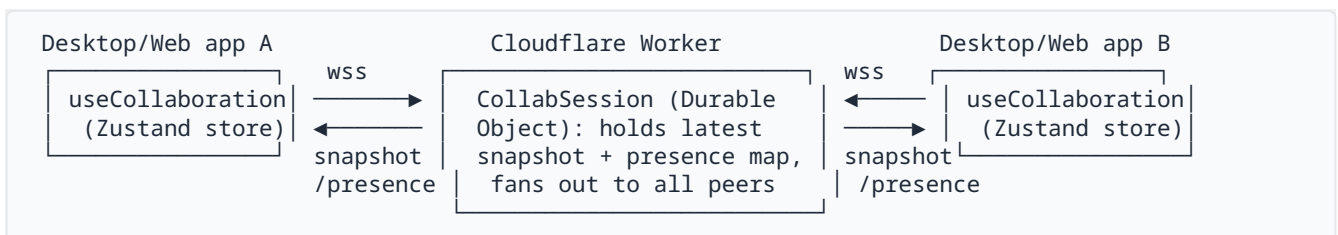
상태: **실험적 MVP**(이슈 #307). `VITE_GEOLIBRE_COLLAB_URL` 이 구성되지 않으면 비활성화됩니다.

GeoLibre의 프로젝트 공유는 기본적으로 스냅샷 기반입니다(`share.geolibre.app`에 업로드). 이 기능은 **라이브** 모드를 추가합니다. 여러 사람이 같은 세션을 열어 서로의 레이어/스타일/뷰 편집을 실시간으로 보고, 프레즌스 커서와 뷰포트 표시기를 함께 확인합니다. 교실, 워크숍, 소규모 팀을 대상으로 합니다.

동기화되는 항목

- **프로젝트 상태** — 레이어, 레이어 그룹, 스타일, 베이스맵, 그리고 지도 뷰(카메라). 프로젝트 전체 스냅샷으로 브로드캐스트됩니다.
- **프레즌스** — 각 참가자의 실시간 커서 위치와 뷰포트 사각형, 그리고 이름 + 색상. 프레즌스는 일시적이며 절대 저장되지 않습니다.
- **채팅** — 선택적으로 지도 좌표를 첨부할 수 있는 짧은 텍스트 메시지(#754). 제한된 최근 기록이 늦게 참여한 사람에게 전달되며, 프로젝트에는 절대 기록되지 않습니다.
- **권한** — 호스트가 참가자별로 지정한 보기 전용 / 편집 가능 재정의(#754). 참가자 목록에 `editOverride` 로 브로드캐스트됩니다.
- **댓글** — 지도 댓글의 추가 / 답글 / 해결 / 삭제 동작. `comment-mutation` 프레임으로 전달되어 각 피어의 스토어에 적용됩니다.

아키텍처



P2P 메시가 아니라 **하나의 중앙 집중식 릴레이**(Cloudflare Durable Object)가 있습니다. DO는 최신 프로젝트 스냅샷을 보관하여 늦게 참여한 사람이 즉시 부트스트랩될 수 있게 하고, 모든 메시지를 연결된 다른 소켓들에 팬아웃합니다.

왜 Durable Object 릴레이인가(CRDT/WebRTC가 아닌 이유)

MVP는 의도적으로 동작하는 가장 단순한 방식을 선택합니다:

- 스토어가 이미 단일 진실 공급원이며, `serializeProject / parseProject` 가 이미 검증되고 정규화된 와이어 형식을 만들어 냅니다. `useEmbedBridge` 는 이미 정확히 이것을 `postMessage` 로 브로드캐스트합니다. 협업 어댑터는 같은 패턴을 WebSocket 위에서 구현한 것입니다.
- **전체 스냅샷, 마지막 쓰기 우선(last-write-wins)** 모델은 자명하게 일관됩니다. 릴레이가 마지막으로 본 스냅샷이 이깁니다. 그것으로 끝입니다. 변경 단위 병합에는 필드별 클록이 필요하고, CRDT(Yjs/Automerger)는 상당한 클라이언트 번들과 Zustand 옆의 두 번째 진실 공급원을 추가하게 됩니다.
- 릴레이는 기존 `workers/viewer` Cloudflare 설정 위에 바로 구축됩니다.

CRDT / 동작 단위 변경 전송은 문서화된 **v2** 경로입니다(제한 사항 참조).

동기화 프로토콜

모든 프레임은 JSON이며, 각 방향은 `type` 을 기준으로 한 구별 유니온(`ClientMessage` 와 `ServerMessage`)입니다. 릴레이의 타입은 `packages/collab-core/src/protocol.ts` (`@geolibre/collab-core/protocol`, `project` 가 `unknown` 으로 타입 지정됨)에 있고, 프론트엔드는 `apps/geolibre-desktop/src/lib/collab-protocol.ts` 에 `project` 가 `GeoLibreProject` 로 타입 지정된 병렬 복사본을 유지합니다. `type` 구별자와 필드 이름을 서로 일치하게 유지하세요.

클라이언트 → 서버:

type	페이로드	비고
join	<code>displayName</code> , <code>color</code> , <code>hostToken?</code> , <code>inviteToken?</code> , <code>identityToken?</code>	연결 후 첫 번째 프레임. 릴레이가 <code>clientId</code> 를 할당합니다(<code>welcome</code> 으로 반환)
snapshot	<code>project</code> , <code>rev</code>	디바운스된 프로젝트 푸시. 공동 편집자만 가능
presence	<code>cursor?</code> , <code>view?</code>	스로틀된 커서 / 뷰포트
set-mode	<code>mode</code>	호스트 전용
set-participant-mode	<code>clientId</code> , <code>canEdit</code>	호스트 전용. 게스트 한 명을 편집 가능 / 보기 전용으로 고정합니다(#754. 참가자 키별로 저장되므로, 신원/초대 기반 참여일 때만 재연결 후에도 유지됩니다 — 모더레이션 참조)
mint-invite	<code>role</code> , <code>maxUses?</code>	호스트 전용. 공동 편집 또는 보기 전용 초대 링크를 발급합니다
revoke-invite	<code>token</code>	호스트 전용. 활성 초대 링크를 무효화합니다
set-session-config	<code>requireIdentity</code>	호스트 전용. 참여에 로그인된 신원을 요구합니다

type	페이로드	비고
set-layer-locks	lockedLayerIds	호스트 전용. 특정 레이어 ID를 읽기 전용으로 표시합니다
kick-participant	clientId, reason?	호스트 전용. 참가자의 연결을 끊습니다
block-participant	clientId, reason?	호스트 전용. 연결을 끊고 해당 참가자 키의 재참여를 차단합니다(모더레이션 아래의 주의 사항 참조)
chat	text, coordinate?	채팅 메시지. 선택적으로 지도 좌표를 첨부할 수 있습니다(#754)
comment-mutation	action	지도 댓글의 추가 / 답글 / 해결 토글 / 삭제. 릴레이가 검증하고 속도를 제한합니다

서버 → 클라이언트:

type	페이로드	비고
welcome	clientId, role, mode, participants[], snapshot \ null, presence, chat[], rev, requireIdentity, identitySupported, lockedLayerIds, invites[]	참여 시 한 번 전송. 늦게 참여한 사람의 부트스트랩
snapshot	project, origin, rev	피어의 스냅샷 팬아웃
presence	clientId, cursor?, view?	피어의 프레즌스 팬아웃
participants	participants[]	참여 / 퇴장 / 역할 / 권한 변경 시. 각 항목은 editOverride 와 identity 를 포함합니다
mode	mode	호스트가 세션 모드를 변경함
invite-created	invite	호스트가 만든 새 초대 링크의 팬아웃
invite-revoked	token	호스트가 초대 링크를 취소할 때의 팬아웃
session-config	requireIdentity	세션 신원 정책이 바뀔 때 브로드캐스트
layer-locks	lockedLayerIds	호스트가 레이어 ID를 잠그거나 잠금 해제할 때 브로드캐스트
kicked	reason	호스트에 의해 강퇴되거나 차단된 참가자에게 전송

type	페이로드	비고
chat	message	채팅 메시지의 팬아웃(발신자에게도 에코되므로 순서는 서버가 권위를 가집니다)(#754)
comment - mutation	action	피어의 댓글 변경 팬아웃
error	code, message	forbidden, too-large, bad-message, not-found, identity-required, identity-unavailable, 또는 layer-locked

에코 / 피드백 루프 방지

어댑터는 `lastAppliedContent` (직렬화된 프로젝트 문자열)를 캐시합니다. 수신한 스냅샷을 적용하기 전에 `lastAppliedContent` 를 정규화 후 문자열로 설정한 다음 `loadProject` 로 적용합니다. `loadProject` 가 트리거하는 스토어 구독은 동일한 문자열로 다시 직렬화되어 억제되므로, 원격 적용이 다시 브로드캐스트되는 일은 없습니다. 이는 `useEmbedBridge` 가 `lastPostedContent` 로 사용하는 것과 정확히 같은 기법입니다. `origin` 이 자신의 `clientId` 인 프레임도 방어적으로 무시합니다(릴레이는 이미 발신자를 제외합니다).

실행 취소와의 상호작용

원격 스냅샷은 `loadProject` 를 통해 적용되며, 이 함수는 `clearHistory()` 로 끝납니다. 이 덕분에 원격 편집이 로컬 실행 취소 스택에 들어가지 않지만, 동시에 **협업자의 편집이 내 실행 취소 기록을 지운다**는 뜻이기도 합니다. 이는 MVP에서 수용한 제한 사항이며, 기록 병합 옵션은 v2 항목입니다.

Durable Object(workers/collab)

- `POST /sessions` — 호스트가 세션을 만듭니다. 짧은 base32 코드를 생성하고, 호스트 토큰을 발급하며, `{ mode, hostToken, requireIdentity }` 를 저장하고, `{ sessionId, hostToken, mode, requireIdentity }` 를 호스트에게만 반환합니다. 허용되지 않은 오리진은 403 을, 속도 제한에 걸린 호출자는 429 를 받습니다(채팅 아래의 운영자 참고 사항 참조).
- `GET /sessions/:id/ws` — WebSocket 업그레이드. `env.COLLAB_SESSION.get(idFromName(id))` 로 라우팅됩니다.
- `GET /sessions/:id/log` / `DELETE /sessions/:id/log` — 세션 로그의 호스트 전용 다운로드 또는 삭제(`Authorization: Bearer <hostToken>`).
- `GET /health (/도 동일)` — `{ ok, service, identitySupported }`.

`CollabSession` 은 **WebSocket Hibernation API**를 사용하므로, 유휴 세션은 소켓을 열어 둔 채 메모리에서 제거됩니다. 소켓별 참가자 메타데이터는 `ws.serializeAttachment()` 로 보관됩니다(하이버네이션 후에도 유지). 영속 스토리지에는 `latestSnapshot`, 단조 증가하는 `rev`, `mode`, `hostToken`, 그리고 제한된 `chat` 로그가 저장됩니다. 프레즌스와 참가자별 권한 재정의는 인메모리 / 소켓별 첨부부에 보관됩니다. 서버 측 강제: 편집할 수 없는 게

스트(세션이 `view-only` 이거나, 호스트가 설정한 참가자별 보기 전용 재정의)가 보낸 `snapshot` 은 `error: forbidden` 과 함께 폐기됩니다. `set-mode` 와 `set-participant-mode` 는 호스트 토큰이 필요합니다. 너무 큰 스냅샷(기본값 10 MB 초과)은 `error: too-large` 로 거부됩니다. 이 상한은 수신 WebSocket 메시지에 대한 Cloudflare의 32 MiB 한도보다 낮게 설정되어 있습니다. 호스팅된 스냅샷은 SQLite 테이블의 여러 행에 청크 단위로 저장됩니다. Durable Object는 키/값 항목 하나와 단일 SQLite 문자열을 모두 2 MB로 제한하는데, 이는 로컬 파일과 외부 플러그인에서 나온 이식 가능한 GeoJSON에 필요한 크기보다 훨씬 작기 때문입니다. 빈 세션은 스토리지 알람을 통해 TTL 이후 회수되며, 이때 데이터베이스의 나머지와 함께 청크도 삭제됩니다.

프론트엔드

- `lib/collab-protocol.ts` — 메시지 타입의 프론트엔드 복사본.
- `lib/collab-client.ts` — WebSocket 전송, `resolveCollabBaseUrl()` (wss/루프백 검증, 설정되지 않으면 `null` 반환), 지수 백오프 재연결.
- `hooks/useCollaboration.ts` — 오케스트레이션: 스토어를 구독하고(공동 편집자를 위한 디바운스·중복 제거된 스냅샷 푸시), 프레즌스를 위해 `map` 의 `mousemove` (스로틀)와 `moveend` 를 읽고, 수신 프레임을 라우팅하며, 시작/참여/나가기/모드 설정 동작을 노출합니다. `resolveCollabBaseUrl()` 이 `null` 이면 아무 동작도 하지 않습니다.
- `lib/build-project-snapshot.ts` — `useEmbedBridge` 에서 끌어올린 공용 `buildProjectSnapshot()` . 브리지와 어댑터가 하나의 정의를 공유하도록 합니다.
- 스토어: 일시적인 `collaboration` 슬라이스(`packages/core`). 프로젝트 파일에서 제외되고(`projectFromStore` 가 절대 읽지 않음) 실행 취소 기록에서도 제외됩니다(`partialize` 에 절대 추가되지 않음).
- `components/layout/RemoteCursorsOverlay.tsx` — 원격 커서를 MapLibre Marker로, 뷰포트 사각형을 전용 GeoJSON 라인 레이어로 렌더링합니다.
- `components/layout/CollaborateDialog.tsx` + 플러그로 제어되는 `TopToolbar` 항목.
- `components/layout/CollaborationStatusBadge.tsx` — 참가자 명단(호스트의 참가자별 권한 토크 포함)과 채팅 서랍(#754)을 담는 캔버스 위의 상주 배지. `useCollaboration` 은 `DesktopShell` 이 소유하며 다이얼로그와 이 배지 모두에 전달되므로 하나의 소켓을 공유합니다. `participantCanEdit()` (`lib/collab-protocol.ts`)는 릴레이를 반영하는 클라이언트 UI가 사용하는 공용 유효 권한 헬퍼입니다.

신원 및 권한(MVP)

익명입니다. 호스트가 세션을 시작하고 코드/링크를 공유하면, 참여자는 표시 이름과 색상을 고릅니다. 호스트는 세션 **모드** 를 선택합니다:

- **view-only** — 게스트는 지켜보고 프레즌스를 볼 수 있지만, 스냅샷 푸시는 서버 측에서 거부됩니다.
- **co-edit** — 링크가 있는 누구나 편집할 수 있습니다.

참가자별 재정의(#754). 세션 전체 모드 외에, 호스트는 `set-participant-mode { clientId, canEdit }` 로 개별 게스트를 고정할 수 있습니다. 릴레이는 해당 소켓의 첨부에 재정의의 기록하고(하이버네이션에서 깨어나도 유지됨) `editOverride` 필드(`true / false`, 또는 세션 모드를 따르는 `null`)가 포함된 참가자 목록을 다시 브로드캐스트합니다. 클라이언트와 릴레이에서 동일하게 계산되는 유효 편집 권한은 다음과 같습니다. 호스트는 항상 편집할 수 있고, 그렇지 않으면 재정의가 우선하며, 그렇지 않으면 세션 모드가 적용됩니다. 보기 전용으로 고정된 게스트의 `snapshot` 푸시는 세션 전체 보기 전용 경로와 똑같이 `error: forbidden` 으로 거부됩니다. 재정의는 게스트의 **참가자 키**에 대해서도 저장되므로 재참여 시 다시 적용되지만, 차단과 같은 이유로(아래 모더레이션 참조) 신원 기반 및 초대 기반 참여에만 해당합니다. 익명 게스트는 새 `clientId`, 따라서 새 키를 가지고 도착하므로, 재연결 시 재정의가 세션 기본값으로 되돌아갑니다. 호스트 명단에는 게스트별 토글이 표시되고, 다른 참가자에게는 각 게스트의 현재 권한이 읽기 전용으로 보입니다.

호스트 토큰(생성자에게만 반환)은 `set-mode` 와 `set-participant-mode` 를 통제하므로, 게스트가 세션이나 다른 게스트의 권한을 올릴 수 없습니다. 코드는 추측할 수 없으며 세션은 자동으로 만료됩니다. 릴레이는 각 참가자의 `clientId` 를 서버 측에서 할당하므로(클라이언트가 보낸 값은 무시) 한 참가자가 다른 참가자의 신원을 주장할 수 없으며, `color` 는 저장/브로드캐스트 전에 16진수 값인지 검증합니다.

로그인된 신원(선택 사항)

참여자 `identityToken` 을 제시할 수 있습니다. 이는 자체 보고된 JSON이 **아닙니다**. 릴레이는 `participant.identity` 를 채우기 전에 토큰의 페이로드 세그먼트에 대한 HMAC-SHA256 서명을 검증하므로, 클라이언트가 계정을 만들어 내거나, 다른 사용자를 사칭하거나, 명단의 인증 배지를 달 수 없습니다. 형식은 다음과 같습니다.

```
<base64url(payloadJSON)>.<base64url(hmacSha256(base64url(payloadJSON)))>
```

페이로드 클레임은 `{ userId, username, provider?, exp? }` 입니다. `@geolibre/collab-core` 의 `verifyIdentityToken` 은 두 릴레이가 모두 호출하는 단일 구현이며, 그 옆의 `signIdentityToken` 은 토큰을 발급합니다.

신원은 **배포 환경이 오픈하지 않는 한 꺼져 있습니다**. 오픈하려면 발급자와 공유하는 서명 비밀 — `COLLAB_IDENTITY_SECRET` (`workers/collab`에서는 Worker 시크릿, `workers/collab-node`에서는 환경 변수) — 을 설정합니다. 비밀이 구성되지 않으면:

- 모든 `identityToken` 은 `null`로 검증되므로 모든 참여자가 익명입니다.
- `requireIdentity` 를 켤 수 없습니다. `POST /sessions` 는 `400` 으로, `set-session-config` 는 `error: identity-unavailable` 로 응답합니다. 호스트 자신의 협업자를 포함해 모든 게스트를 거부하게 될 게이트를 켜는 대신 이렇게 동작합니다.
- `GET /health` 와 모든 `welcome` 은 `identitySupported: false` 를 보고하며, 이를 통해 Collaborate 다 이얼로그는 "로그인된 계정 필요" 옵션을 아예 숨길지 결정합니다.

GeoLibre 자체에는 아직 로그인 흐름이 없으므로, 기본 배포는 비밀을 설정하지 않은 채 신원 경로 전체를 휴면 상태로 둡니다. 프로토콜과 릴레이 지원은 자체 발급자를 릴레이 앞에 두는 배포를 위해 마련되어 있습니다.

모더레이션

kick-participant 는 참가자의 연결을 끊습니다. block-participant 는 거기에 더해 **참가자 키**를 기록하여 릴레이가 다음 참여를 거부하게 합니다. 이 키는 검증된 신원이면 `user:<userId>`, 초대 기반 참여면 `invite:<token>`, 그 외에는 `anon:<clientId>` 입니다. 여기서 `clientId` 는 참여할 때마다 새로 발급됩니다.

같은 키가 저장된 참가자별 재정의도 뒷받침하므로, 둘 다 같은 주의 사항을 가집니다. 차단은 신원 기반 및 초대 기반 참여에서만 지속됩니다. **익명 게스트는 단순히 재연결하는 것으로 다시 참여할 수 있습니다.** 새 `clientId`, 따라서 새 키를 가지고 도착하기 때문입니다. 차단은 모더레이션 편의 기능이지 보안 경계가 아닙니다. 강제력 있는 차단이 필요한 세션은 초대나 로그인된 신원을 요구해야 합니다.

채팅(#754)

가벼운 세션 내 텍스트 채널입니다. `chat { text, coordinate? }` 프레임은 서버 측에서 검증되고(비어 있지 않음, 공백 제거, 2000자 제한. 좌표는 프레즌스와 같은 유한 숫자 정제기를 거침) 발신자를 포함한 **모든** 참가자에게 팬아웃되므로, 릴레이의 순서가 단일 진실 공급원이 됩니다(클라이언트는 낙관적으로 렌더링하지 않고 에코를 기준으로 렌더링합니다). 각 메시지에는 서버가 할당한 `id` 와 `ts` 가 있습니다. 릴레이는 제한된 영속 기록(최근 50개 메시지)을 보관하므로, 늦게 참여한 사람(또는 하이버네이션 이후의 welcome)은 `welcome.chat[]` 부트스트랩을 통해 최근 대화를 볼 수 있습니다. 채팅은 보기 전용 게스트를 포함해 세션의 모든 사람에게 열려 있으며, 프로젝트 편집만 통제됩니다. 메시지는 발신자의 현재 지도 중심을 클릭 가능한 좌표로 첨부할 수 있고, 수신자가 클릭하면 지도가 그 위치로 다시 중심을 맞춥니다. 채팅은 캔버스 위의 상태 배지에 있어 지도 작업 중에도 접근할 수 있으며, 일시적이고 프로젝트 파일에 절대 기록되지 않습니다.

운영자 참고: `POST /sessions` 는 요청의 `Origin` (또는 `Referer`)을 `isAllowedOrigin` 을 통해 `ALLOWED_ORIGINS` 와 대조하여 검증합니다(기본값은 호스팅 오리진(`geolibre.app`, `web.geolibre.app`, 그 레거시 별칭 `viewer.geolibre.app`, `studio.geolibre.app`, `collab.geolibre.app`), `*.geolibre-preview.pages.dev` 아래의 단일 레이블 HTTPS 배포 호스트, 루프백 호스트(`localhost`, `*.localhost`, `127.0.0.1`), 그리고 `tauri://localhost` 입니다. 중첩되거나 사용자 지정 포트를 쓰는 프리뷰 호스트와 유사 도메인은 거부되며, 공유된 `opengeos.org` GitHub Pages 프리뷰 오리진은 의도적으로 신뢰하지 않습니다). 이는 브라우저 오리진 필터링과 심층 방어를 위한 것이지(인증이나 일반적인 서버 측 접근 게이트가 아님), IP별 `checkRateLimit` (60초당 10회 요청)도 적용합니다. 응답에는 여전히 `Access-Control-Allow-Origin: *` 가 전송되므로 브라우저가 아닌 클라이언트(예: Tauri)가 CORS에 막히지 않습니다. 검증 가능한 자격 증명 요구 사항이 추가되지 않는 한, 브라우저가 아닌 클라이언트는 이 헤더를 생략해도 계속 지원됩니다. 프로덕션에서 세션을 만들 수 있는 브라우저 오리진을 제한하려면 `ALLOWED_ORIGINS` (쉼표로 구분된 목록)를 구성하세요.

기능 플래그

VITE_GEOLIBRE_COLLAB_URL 을 릴레이 기본 주소(예: `wss://collab.geolibre.app`, 또는 `wrangler dev` 용 `ws://127.0.0.1:8787`)로 설정하세요. 설정되지 않으면 혹은 아무 동작도 하지 않고 모든 협업 UI가 숨겨지므로, 프로덕션 빌드는 이 기능을 꺼진 채로 출시합니다. Tauri CSP의 `connect-src`에는 `wss` 호스트가 나열되어야 합니다(기존 `https:` 지시어는 `wss:`를 허용하지 않습니다).

Docker 이미지에서는 같은 설정을 **컨테이너 런타임**에 `-e GEOLIBRE_COLLAB_URL=...`로 지정할 수 있습니다. 엔트리포인트가 값을 검증해 `geolibre-runtime-config.js`에 기록하고, `resolveCollabBaseUrl()`은 빌드 시점 변수보다 이 값을 우선하므로, 미리 빌드된 이미지를 다시 빌드하지 않고도 자체 호스팅 릴레이로 향하게 할 수 있습니다. `wss://` (또는 루프백의 `ws://`)가 아닌 값은 협업을 조용히 꺼진 채로 두지 않고 컨테이너 부팅을 실패시킵니다. 엔트리포인트는 또한 릴레이의 오리진을 nginx CSP의 `connect-src` (`docker/nginx.conf`의 `__GEOLIBRE_COLLAB_CONNECT_SRC__`)에 치환해 넣습니다. 그 지시어에는 단독 `wss:`가 없기 때문입니다. 따라서 아래의 데스크톱 빌드와 달리 웹/Docker 경로는 수동 CSP 편집이 필요 없습니다. [Docker로 실행](#)을 참조하세요.

자체 호스팅 참고: 데스크톱 CSP는 `wss://collab.geolibre.app` (개발용 `ws://localhost/127.0.0.1` 포함)을 고정합니다. 데스크톱 빌드를 다른 릴레이로 향하게 하려면 `apps/geolibre-desktop/src-tauri/tauri.conf.json`의 `connect-src`를 수정하고 다시 빌드해야 합니다. CSP와 VITE_GEOLIBRE_COLLAB_URL 플래그는 독립적인 설정입니다. 웹 빌드는 대신 페이지의 CSP를 상속하므로 환경 변수만 있으면 됩니다.

릴레이 배포(`collab.geolibre.app`)

두 호스트가 같은 프로토콜을 구현하고 같은 권한/검증 코어를 가져옵니다:

- `workers/collab`은 호스팅 서비스에서 사용하는 Cloudflare Durable Object입니다.
- `workers/collab-node`는 자체 호스팅 가능한 Node/SQLite 릴레이입니다. 루트 `docker-compose.yml`에 `geolibre-collab`으로 포함되어 있으며, 영속 데이터베이스는 `/data/collab.sqlite`에 마운트됩니다. 필요할 때 `COLLAB_MAX_SNAPSHOT_BYTES` (릴레이 기본값 10,000,000. `compose` 파일은 1,000,000을 기본값으로 둬)와 `COLLAB_IDLE_TTL_MS` (기본값 2시간)를 구성하세요.

두 릴레이 구현 모두 `COLLAB_MAX_SNAPSHOT_BYTES`를 양의 정수 바이트 수로 받습니다. Cloudflare 릴레이에서는 Worker 변수로 설정합니다. 예를 들면 다음과 같습니다.

```
[vars]
COLLAB_MAX_SNAPSHOT_BYTES = "10000000"
```

구성한 값은 배포 플랫폼의 WebSocket 메시지 크기 제한보다 작아야 합니다.

호스팅 릴레이는 `workers/viewer` 와 같은 방식으로 Cloudflare Workers에 배포됩니다:

- **CI:** `.github/workflows/deploy-collab.yml` 은 `workers/collab/**` 을 건드리는 `main` 으로의 모든 푸시(또는 수동 `workflow_dispatch`)에서 배포합니다. 기존 `CLOUDFLARE_API_TOKEN` / `CLOUDFLARE_ACCOUNT_ID` 저장소 시크릿을 재사용하며, 토크에는 **Workers Scripts Write** 권한이 필요합니다(Cloudflare의 "Edit Cloudflare Workers" 템플릿에 포함됨). Durable Object 배포는 같은 스크립트 업로드의 일부이므로 별도의 Durable Objects 권한은 필요 없습니다.
- **수동:** `cd workers/collab && npx wrangler deploy`.

`wrangler.toml` 은 이미 `collab.geolibre.app` 사용자 지정 도메인 라우트와 SQLite Durable Object 마이그레이션을 선언하므로, 첫 배포에서 DNS, TLS, DO 클래스가 자동으로 프로비저닝됩니다. Cloudflare 대시보드에서 수동으로 할 작업이 없습니다. SQLite 기반 Durable Object는 무료 Workers 플랜에서 사용할 수 있습니다.

릴레이가 가동되면, 웹/Pages 빌드 환경에 `VITE_GEOLIBRE_COLLAB_URL=wss://collab.geolibre.app` 을 설정하여 앱이 그 릴레이를 가리키게 하세요. 이 환경 변수가 설정되기 전까지 기능은 꺼진 상태로 유지됩니다.

제한 사항 / v2

- **마지막 쓰기 우선:** 동시 공동 편집은 경쟁합니다. 마지막으로 디바운스된 스냅샷이 이기고 더 느린 편집은 덮어씁니다. 프레즌스는 사용자가 충돌을 피하는 데 도움이 됩니다.
- **페이로드 크기:** 레이어는 `FeatureCollection` 을 포함할 수 있습니다. `projectFromStore` 는 URL 기반 레이어의 중복 `geojson` 을 이미 제거하지만, 큰 인메모리/로컬 파일 레이어는 스냅샷 상한 (`COLLAB_MAX_SNAPSHOT_BYTES`: 릴레이 기본값 10 MB, Docker Compose 설정에서는 1 MB)을 초과할 수 있으며 `error: too-large` 로 거부됩니다(대신 URL로 공유하세요). v2: 차이분 / 청크 단위 레이어 동기화.
- **실행 취소:** 원격 적용은 로컬 실행 취소를 지웁니다(위 참조).
- **v2 방향:** 동작 단위 변경 또는 CRDT 전송, 원격 적용 기록 병합, 더 풍부한 권한/신원(`share.geolibre.app` 계정과 연동).

테스트

자동화:

- `npm run test:worker` 는 두 릴레이의 타입을 검사하고 Node 릴레이의 소켓 통합 스위트를 실행합니다.
- `npm run test:frontend` 는 `tests/collab-protocol.test.ts` (`set-participant-mode` / `chat` 프레임)을 포함한 프로토콜 왕복, `resolveCollabBaseUrl` 검증, 에코 억제 로직, `participantCanEdit` 유효 권한 헬퍼)와 공용 릴레이 적합성 스위트를 실행합니다.

전체 기능을 로컬에서 테스트하기

협업은 `VITE_GEOLIBRE_COLLAB_URL` 이 실행 중인 릴레이를 가리키기 전까지 꺼져 있으므로, 로컬 테스트는 두 부분으로 나뉩니다. 릴레이를 실행한 다음, 그 릴레이를 대상으로 앱을 실행합니다.

1. 한 터미널에서 **릴레이를 시작**합니다. Cloudflare 구현의 경우:

```
cd workers/collab && npx wrangler dev --port 8787 --local
# → Ready on http://localhost:8787
```

또는 자체 호스팅 가능한 Node 구현을 실행합니다:

```
npm run build -w geolibre-collab-node
npm start -w geolibre-collab-node
```

1. 다른 터미널에서 **그 릴레이를 가리키도록 앱을 시작**합니다:

```
VITE_GEOLIBRE_COLLAB_URL=ws://127.0.0.1:8787 npm run dev
# → http://localhost:5173
```

또는 반복 입력을 피하려면 `apps/geolibre-desktop/.env.local` 에 `VITE_GEOLIBRE_COLLAB_URL=ws://127.0.0.1:8787` 을 넣으세요. 변수가 설정되지 않으면 Collaborate 메뉴 항목이 숨겨진 채로 남는데, 그것이 바로 기능 플래그가 동작하는 모습입니다. (데스크톱 셀에서는 같은 변수와 함께 `npm run tauri:dev` 를 사용하세요. Tauri CSP는 이미 `ws://127.0.0.1:* / ws://localhost:*` 를 허용합니다.)

1. `http://localhost:5173` 에서 **독립적인 창 두 개를 엽니다**. 상태를 공유하지 않도록 일반 창과 시크릿 창을 함께 쓰면 좋습니다.
2. **세션을 진행**합니다:
3. 창 A: **Project → Collaborate...**(프로젝트 → 협업...)에서 이름을 입력하고 색상을 고른 뒤 **Start session**(세션 시작)을 누릅니다(Anyone can edit 선택). 세션 코드나 공유 링크를 복사합니다.
4. 창 B: 공유 링크를 직접 엽니다(Collaborate 다이얼로그가 코드가 미리 입력된 채로 자동으로 열립니다. 이름만 입력하고 **Join**(참여)을 누르면 됩니다). 또는 **Project → Collaborate...**를 열고 코드를 붙여 넣습니다.
5. 확인: B는 A의 기존 레이어를 즉시 봅니다. A에서 레이어를 추가/제거하거나, 스타일을 바꾸거나, 지도를 이동하면 약 300 ms 안에 B에 반영됩니다. 각 창은 상대방의 실시간 **커서**와 점선 **뷰포트 사각형**을 보여 줍니다. A(호스트)를 view-only로 전환하면 B의 편집이 차단됩니다.
6. 캔버스 위의 상태 배지(왼쪽 아래, 클릭하면 확장)를 통해 파트 3 & 4(#754)를 확인합니다. A(호스트)는 명단에서 B를 참가자별로 **Can edit**(편집 가능)와 **View-only**(보기 전용) 사이에서 전환할 수 있고, 어느 창에서든 **chat**(채팅) 메시지를 보낼 수 있습니다(선택적으로 현재 지도 중심을 첨부하면, 상대방이 클릭해 그 위치로 다시 중심을 맞출 수 있습니다).

릴레이 전용 스모크 테스트(UI 없음): wrangler dev 가 실행 중인 상태에서 `http://127.0.0.1:8787/sessions` 에 POST 하여 코드를 발급받은 다음, `ws://127.0.0.1:8787/sessions/<code>/ws` 로 WebSocket을 열고 `join / snapshot / presence` 프레임을 주고받습니다. 프론트엔드와 무관하게 릴레이를 확인하는 가장 빠른 방법입니다.

<https://geolibre.app/ui-profiles/>

UI 프로필과 데이터 소스 필터링

GeoLibre는 초보자를 위해 인터페이스를 단순화하거나 팀 전체의 배포를 표준화하기 위해 데이터 소스, 웹 서비스, 플러그인을 숨길 수 있습니다. 숨기기는 **비파괴적**입니다. 아무것도 제거되지 않으며, 어떤 항목이든 언제든지 다시 활성화할 수 있습니다. 프로필 환경설정은 브라우저/앱에 로컬로 저장되며, 저장된 `.geolibre.json` 프로젝트 안에 함께 담겨 이동하지 않습니다.

배포를 잠그고 싶으신가요?

프로필은 화면을 정리할 뿐이며, 사용자가 되돌릴 수 있습니다. 키오스크나 강의실 인스턴스처럼 배포가 허용되는 작업을 고정하려면 UI에 전혀 노출되지 않는 [Deployment Capabilities](#)를 참고하세요. 두 기능은 서로 독립적이며 함께 사용할 수 있습니다.

사용자를 위한 안내

온보딩

기본적으로 GeoLibre는 모든 것이 표시되는 **Advanced**(고급) 인터페이스로 시작하며, 첫 실행 환영 대화 상자를 표시하지 않습니다. 더 단순한 경험 수준은 언제든지 **Settings → Interface**(설정 → 인터페이스)에서 선택할 수 있습니다(아래 참고):

- **Beginner**(초급) — 필수 데이터 소스와 도구만 표시합니다.
- **Intermediate**(중급) — 일반적인 데이터 소스, 서비스, 플러그인을 표시합니다.
- **Advanced**(고급) — GeoLibre가 제공하는 모든 것을 표시합니다(기본값).

관리자는 기본 `deployment.json` 정책에서 인터페이스를 미리 구성하거나, 아래에 설명된 레거시 `admin-profile.json` 폴백을 사용할 수 있습니다.

Settings → Interface

인터페이스는 하나의 4단계 선택기로 제어합니다:

- **Beginner**(초급) / **Intermediate**(중급) / **Advanced**(고급)는 개발자가 큐레이션한 프리셋입니다. 하나를 선택하면 해당 레이아웃이 즉시 적용됩니다. **Advanced**는 모든 것을 드러내므로 제한 없는 전체 인터페이스입니다.
- **Custom**(사용자 지정)은 아래 항목 중 하나라도 직접 편집하는 순간 자동으로 활성화됩니다. 이는 클릭하는 버튼이 아니라 상태입니다.

Settings → Interface(설정 → 인터페이스)를 열어 다음을 수행할 수 있습니다:

- **experience level**(경험 수준)을 선택합니다. 각 항목의 복잡도에 따라 체크리스트가 채워지며 즉시 적용됩니다.
- 개별 **data sources**(데이터 소스), **plugins**(플러그인), 전체 **menus**(메뉴) (Project, Edit, View, Add Data, Processing, Controls, Plugins, Help), 그리고 Project, Edit, View, Processing, Controls, Settings, Help 메뉴 안의 항목을 체크하거나 체크 해제합니다. 항목을 편집하면 선택기가 **Custom**으로 전환됩니다.

Settings(설정) 메뉴 자체와 그 안의 Language / Layout / Interface 항목은 항상 표시되므로, 프로필 UI가 숨겨져 접근할 수 없게 되는 일은 없습니다.

관리자를 위한 안내

기본 배포 정책의 `interface` 섹션을 사용하세요. 예를 들면 다음과 같습니다:

```
{
  "version": 1,
  "interface": {
    "enabled": true,
    "level": "intermediate",
    "lock": true,
    "hiddenDataSources": ["postgres", "video"],
    "hiddenPlugins": ["maplibre-gl-geoagent"]
  }
}
```

아래의 `interface` 필드 표에서 이 섹션을 설명합니다. 이 정책은 웹/임베드용 `<base>/deployment.json`, 데스크톱 구성 디렉터리, 또는 [Docker 소스 정책](#)으로 제공됩니다.

레거시: `admin-profile.json` — 여전히 지원됨

레거시 파일은 선택된 배포 정책에 비어 있지 않은 유효한 `interface` 섹션이 없을 때 폴백으로 계속 사용됩니다. 래핑되지 않은 형식은 다음과 같습니다:

```
{
  "enabled": true,
  "level": "intermediate",
  "lock": true,
  "hiddenDataSources": ["postgres", "video"],
  "hiddenPlugins": ["maplibre-gl-geoagent"]
}
```

웹/임베드에서는 애플리케이션 루트에서 이 파일을 제공하세요. 데스크톱에서는 원래 파일이 `<app_config_dir>/admin-profile.json`에 그대로 있습니다. 표준 [Tauri 구성 경로](#)를 참고하세요. 데스크톱 파일이 존재하면 형식이 잘못되었더라도 그 파일이 우선하며, 번들된 사본으로 폴백하지 않습니다. 파일이 없거나 읽기/명령이 실패한 경우에만 웹 사본이 선택됩니다.

Interface 필드

기본 interface 객체와 레거시 `admin-profile.json` 파일은 다음 프로필 필드를 공유합니다. 레거시 파일은 이를 최상위 수준에 저장합니다.

필드	타입	의미
<code>enabled</code>	boolean	필터링 활성화 여부입니다. 기본값은 <code>true</code> 입니다.
<code>level</code>	<code>"beginner" \ "intermediate" \ "advanced"</code>	각 항목의 등급에 따라 숨김 목록을 초기화합니다. 선택 사항입니다.
<code>lock</code>	boolean	<code>true</code> 이면 사용자가 Settings에서 프로필을 변경할 수 없습니다. 다음 실행 시 잠금을 해제하려면 선택된 배포 정책에서 <code>interface.lock</code> 을 제거하거나 비우거나, 레거시 프로필에서 최상위 <code>lock</code> 을 제거하거나 비우세요.
<code>hiddenDataSources</code>	string[]	숨길 데이터 소스 id를 명시적으로 지정합니다. 존재하면 프리셋을 재정의합니다.
<code>hiddenPlugins</code>	string[]	숨길 플러그인 id를 명시적으로 지정합니다. 존재하면 프리셋을 재정의합니다.
<code>hiddenMenus</code>	string[]	숨길 최상위 메뉴 id입니다(<code>project</code> , <code>edit</code> , <code>view</code> , <code>addData</code> , <code>processing</code> , <code>controls</code> , <code>plugins</code> , <code>help</code>).
<code>hiddenMenuItems</code>	string[]	숨길 메뉴 항목 id입니다(예: <code>processing.raster</code> , <code>help.diagnostics</code> , <code>controls.minimap</code>).

데이터 소스 id는 `apps/geolibre-desktop/src/lib/ui-profile.ts` 에 있는 카탈로그 id입니다(예: `vector` , `xyz` , `mbtiles` , `postgres`). 플러그인 id는 `packages/plugins/src/plugins/*` 와 `packages/plugins/src/plugin-ids.ts` 에 정의된 안정적인 id입니다(예: `maplibre-gl-geoagent`). 메뉴 및 메뉴 항목 id는 같은 `ui-profile.ts` 에 있는 카탈로그 id입니다(`TOP_LEVEL_MENUS` , `MENU_ITEM_CATALOG`).

`level` 프리셋이 활성화되어 있으면, 시작 후 비동기적으로 로드되는 외부/번들 드롭인 플러그인도 나타나는 대로 숨김 집합에 포함되므로, 초급 프로필은 늦게 로드되는 고급 플러그인도 계속 숨깁니다.

<https://geolibre.app/deployment-capabilities/>

배포 기능(Capabilities)

배포는 자신이 제공하는 앱이 할 수 있는 일을 고정할 수 있습니다. 읽기 전용 키오스크, 교실용 인스턴스, 그리고 그 사이부터 전체 앱까지 모두 가능합니다.

기능(capability)은 의도적으로 큰 단위로 나뉩니다. 각 기능은 개별 메뉴 항목이 아니라 동작의 전체 부류("데이터를 추가할 수 있는가")를 지칭하므로, 잠긴 배포가 누군가 목록에 넣는 것을 잊은 항목 하나 때문에 뚫리는 일이 없습니다.

클라이언트 게이트는 일반적인 권한 부여 경계가 아닙니다

기능을 부여하지 않으면 해당 어포던스가 제거됩니다. 메뉴가 렌더링되지 않고, 명령 팔레트가 해당 동작을 나열하거나 실행하지 않으며, 키보드 단축키가 아무 동작도 하지 않고, 임베드 API가 명령을 거부합니다. 이러한 클라이언트 게이트는 브라우저 개발자 도구를 가진 사람을 **막지 못합니다**.

Docker 컨테이너는 또한 최종 배포 정책을 nginx에서 강제합니다. 선택된 `/sidecar/` 경로군은 `processing:run` 또는 `data:add` 를 요구하며, `/ai` 는 최종 정책이 AI를 활성화하고 승인된 프록시 환경이 구성되지 않는 한 비활성화됩니다. 정확한 경로 표와 시작 규칙은 [자체 호스팅](#)을 참조하세요. 이 강제는 **컨테이너 전용**입니다. 브라우저 WASM 엔진, 데스크톱 처리, 별도로 호스팅된 사이드카는 제한하지 않습니다.

Basic Auth 또는 실제 인증 프록시와 `GEOLIBRE_CONVERSION_ROOTS` 를 그대로 유지하세요. 컨테이너 정책 강제는 사용자 인증이 아니며, 해당 경로 밖에서는 기능이 여전히 인터페이스 측면에서의 절반만 담당합니다.

UI 프로필과는 다릅니다

[UI 프로필](#)도 메뉴와 항목을 숨기며, 둘은 독립적으로 동작합니다. 차이점은 누가 결정하느냐입니다:

	UI 프로필	배포 기능
목적	대상 사용자를 위해 혼잡함을 줄임	배포가 허용하는 것을 고정함
설정 주체	사용자, 기본 정책 <code>interface</code> , 또는 레거시 관리자 프로필	기본 런타임 정책; 레거시 런타임/빌드 환경 폴백
앱에서 되돌릴 수 있음	예, Settings → Interface(설정 → 인터페이스)에서(<code>lock</code> 이 설정되지 않은 경우)	아니요 — UI에 전혀 표시되지 않음
세분화 수준	개별 항목, 데이터 소스, 플러그인	기능 전체

둘 다 적용되는 경우 기능이 먼저 확인됩니다. 배포가 부여하지 않은 동작은 프로필이 무엇을 말하든 결코 제공되지 않습니다.

기능 목록

기능	허용 범위
<code>project:edit</code>	프로젝트 작성: New, Open, Open Recent, Import, Project History, Save, Save As, Duplicate, Save as Template, Collaborate, StoryMap; Undo/Redo(메뉴 항목 및 Ctrl/Cmd+Z, Ctrl/Cmd+Shift+Z, Ctrl+Y 단축키); Export Selection; 리뷰 코멘트 추가; 임베드 API의 <code>loadProject</code> .
<code>data:add</code>	데이터 가져오기: Add Data 메뉴 전체, 지도에 파일 드래그(브라우저 및 데스크톱), 임베드 API의 <code>addLayer</code> 및 <code>addData</code> .
<code>processing:run</code>	Processing 메뉴 전체 — Whitebox, SQL, Python, AI 어시스턴트, 지오크딩, Model Builder, 변환/벡터/래스터 도구 — 그리고 임베드 API의 <code>openTool</code> .
<code>export:data</code>	데이터나 렌더링 결과를 밖으로 내보내기: Share, Export HTML, Print, Print Layout, Offline Basemap, 임베드 API의 <code>exportImage</code> .
<code>plugins:install</code>	Plugins 메뉴, 플러그인이 등록된 톨바 메뉴, 내장 메뉴의 플러그인 항목, 플러그인 활성화 또는 비활성화, 플러그인 마켓플레이스("Manage plugins").
<code>settings:manage</code>	Settings 대화 상자와 Style Manager.

여기에 나열되지 않은 것은 모두 권한이 필요 없으며 모든 구성에서 계속 사용할 수 있습니다. 이동과 줌, View 및 Controls 메뉴, 레이어 표시 여부와 순서, 식별, 선택 도구, Help가 여기에 해당합니다.

구성 방법

런타임(권장)

`deployment.json`에 `capabilities`를 설정하거나 Docker 런타임 재정의를 사용하세요:

```
docker run --rm -p 8080:80 \
  -e GEOLIBRE_CAPABILITIES=data:add,export:data \
  ghcr.io/opengeos/geolibre:latest
```

런타임 정책 지원은 v3.3.0부터 게시된 이미지에 포함됩니다. 배포 정책을 참조하세요. 환경 변수에서 `none`은 아무 것도 부여하지 않으며, JSON에서는 `"capabilities": []`가 같은 빈 부여를 뜻합니다. 생략된 소스는 기본 전체 집합을 부여합니다.

레거시 입력 — `VITE_GEOLIBRE_CAPABILITIES`는 여전히 인식됩니다

`VITE_GEOLIBRE_CAPABILITIES`는 레거시 빌드 시점 폴백입니다:

```
VITE_GEOLIBRE_CAPABILITIES="data:add,processing:run,export:data" npm run build
```

사전 빌드된 클라이언트에서는 런타임 배포 정책이 우선하고, 그다음 `window.__GEOLIBRE_DEPLOYMENT_ENV__`, 그다음 이 빌드 시점 값이 적용됩니다. 레거시 파서는 알 수 없는 토큰을 버립니다. 인식되지 않는 토큰만 있는 비어 있지 않은 값은 아무것도 부여하지 않으며, 빈 값은 설정되지 않은 것으로 간주합니다. 이 규칙은 JSON 정책 파싱이나 Docker의 엄격한 소스 검증을 설명하는 것이 아닙니다.

```
VITE_GEOLIBRE_CAPABILITIES=none npm run build
VITE_GEOLIBRE_CAPABILITIES="export:data" npm run build
VITE_GEOLIBRE_CAPABILITIES="project:edit,data:add,processing:run,export:data" npm run build
```

위 예시는 여전히 인식되는 레거시 빌드 시점 구성입니다. Docker nginx 경로 강제는 구성하지 않습니다.

클라이언트 정책 로딩은 폴백될 수 있습니다

웹 로딩은 최대 3초 동안 기다립니다. 정책이 없거나 늦게 도착하면 런타임 환경 폴백, 그다음 레거시 빌드 환경, 그다음 기본 전체 부여를 사용합니다. 클라이언트 게이트와 브라우저 WASM은 접근 제어가 아닙니다. Docker nginx 정책 강제를 사용하거나 API를 별도로 보호하세요.

런타임 예시

선택적 기능이 전혀 없는 키오스크 또는 전시용 단말:

```
docker run --rm -p 8080:80 -e GEOLIBRE_CAPABILITIES=none ghcr.io/opengeos/geolibre:latest
```

방문자가 이미지를 내보낼 수 있도록 허용:

```
docker run --rm -p 8080:80 -e GEOLIBRE_CAPABILITIES=export:data ghcr.io/opengeos/geolibre:latest
```

프로젝트 작성, 데이터 및 처리 도구, 내보내기는 허용하되 플러그인 설치나 설정은 허용하지 않는 교실용 인스턴스:

```
docker run --rm -p 8080:80 \
  -e GEOLIBRE_CAPABILITIES=project:edit,data:add,processing:run,export:data \
  ghcr.io/opengeos/geolibre:latest
```

임베드된 지도는 빈 부여를 사용하여 `loadProject`, `addLayer`, `addData`, `openTool`, `exportImage` 같은 명령을 거부하면서도 `setView`, `highlightFeature`, 레이어 표시 여부 명령은 계속 사용할 수 있게 할 수 있습니다.

임베드 API 동작

거부된 명령은 조용히 아무 일도 하지 않는 대신 거부(reject)되므로, 호스트 페이지는 "거부됨"과 "효과 없음"을 구별할 수 있습니다:

```
await map.addData("https://example.com/data.geojson");
// Error: Missing data:add capability
```

전체 명령 목록은 [임베딩 및 공유](#)를 참조하세요. 임베드 출처 허용 목록(`GEOLIBRE_EMBED_ORIGINS`)과 기능은 서로 독립적입니다. 허용 목록은 누가 명령을 보낼 수 있는지를 결정하고, 기능은 어떤 명령이 존재하는지를 결정합니다.

관련 페이지

- [배포 정책](#) — 기본 정책과 Docker 강제
- [자체 호스팅](#) — 정확한 Docker 서버 측 경로 가드
- [UI 프로필](#) — 비파괴적 인터페이스 필터링
- [임베딩 및 공유](#) — 임베드 API와 출처 허용 목록
- [시작하기](#) — 컨테이너 구성

<https://geolibre.app/deployment-policy/>

배포 정책

`deployment.json`은 GeoLibre 배포의 클라이언트 측 설정을 기술하는 단일 버전 관리 파일입니다. 사용자가 어떤 기능(capability)을 갖는지, 어떤 인터페이스 요소가 표시되는지, 어떤 플러그인을 로드할 수 있는지, 선별된 서비스 라이브러리, 공유 엔드포인트, 브랜딩을 담습니다.

로딩

웹 및 Jupyter 빌드는 첫 렌더링 전에 앱의 기본 URL(`<base>/deployment.json`)에서 `deployment.json` 을 가져옵니다. 데스크톱은 설정 디렉터리의 파일을 먼저 읽고([데스크톱 참조](#)), 파일이 없거나 읽기에 실패한 경우에만 해당 웹 URL로 폴백합니다. 선택된 정책은 첫 렌더링 전에 결정됩니다. 선택된 파일이 없거나(404 또는 HTML 폴백 페이지), JSON이 아니거나, 알 수 없는 `version` 인 경우에는 어떤 정책도 적용되지 않습니다. 웹 파일에 접근할 수 없거나 가져오기에 3초 이상 걸리는 경우에도 정책이 적용되지 않습니다. 이러한 경우 앱은 다음 클라이언트 구성 소스를 사용합니다.

모든 빌드에는 `null` 을 담은 `deployment.json` 스텝이 포함되어 있으며, 이는 정책 없음을 의미합니다. 덕분에 정적 호스트는 404를 기록하는 대신 시작 시의 가져오기 요청에 응답합니다. 정적 호스트에 정책을 적용하려면 게시된 출력물에서 해당 파일을 교체하세요. Docker에서는 엔트리포인트가 부팅할 때마다 이 파일을 덮어씁니다. 데스크톱에서는 이 스텝이 번들된 웹 폴백이므로 설정 디렉터리의 파일이 여전히 우선합니다.

권장되는 배포 입력은 버전 관리되는 `deployment.json` 입니다. Docker에서는 엔트리포인트가 마운트된 소스를 검증하고, 비어 있지 않은 환경 변수 재정의의 필드 단위로 적용한 뒤, 부팅할 때마다 생성된 공개 정책을 기록합니다([Docker 참조](#)). 그런 다음 클라이언트는 최종 정책, `window.__GEOLIBRE_DEPLOYMENT_ENV__` , 빌드 환경의 순서로 설정을 결정합니다.

가용성: 런타임 정책 전달, 데스크톱 설정 디렉터리 로딩, Docker 강제 적용은 v3.3.0에서 처음 제공되었습니다. 이전 Docker 이미지와 데스크톱 릴리스는 `deployment.json` 을 무시하므로 v3.3.0 이상을 사용하세요([ghcr.io/opengeos/geolibre:latest](#) 이미지는 `main` 에서 다시 빌드됩니다).

이전의 `admin-profile.json` 과 `VITE_GEOLIBRE_CAPABILITIES` 입력은 **레거시이지만 여전히 지원**됩니다. 기본 정책이 사용 가능한 섹션을 지정하지 않는 경우 폴백으로 남아 있으며, 이 문서는 이들을 제거하지 않습니다.

현재 각 섹션의 역할은 다음과 같습니다.

- `capabilities` 는 앱을 제한합니다. `[]` 는 아무 기능도 부여하지 않으며, 생략하면 레거시 `VITE_GEOLIBRE_CAPABILITIES` (또는 기본값인 전체 부여)가 유효하게 유지됩니다.
- 비어 있지 않은 `interface` 객체는 레거시 `admin-profile.json` 을 통째로 대체하며, 필드는 병합되지 않습니다. 생략되거나 비어 있거나(`{}`) 유효하지 않은 `interface`는 레거시 프로필을 계속 적용 가능 상태로 둡니다.

- `plugins.registryUrl` 은 플러그인 레지스트리를 설정합니다. `allowed`, `blocked`, `sideload` 는 외부 플러그인의 로드와 설치를 제어하고, `defaultActive` 는 새 프로젝트에서의 활성화 초기값을 지정합니다.
- `services`, `sharing`, `geolens`, `branding.appName` 은 대응하는 `GEOLIBRE_*` 배포 설정을 재정의합니다. `sharing.embedOrigins: []` 는 임베드 API를 끕니다.
- `branding.welcome: false` 는 첫 실행 마법사를 표시하지 않습니다.
- `ai.enabled: true` 는 어시스턴트를 동일 출처의 `/ai` 프록시로 연결합니다. `false` 는 운영자가 구성된 AI 프록시를 제거합니다(사용자가 Settings에서 직접 입력한 공급자는 영향을 받지 않습니다). `ai.model` 은 프록시의 모델을 선택합니다.

클라이언트 파서는 관대하며 섹션 단위로 동작합니다. 알 수 없는 이름의 기능을 포함한 유효하지 않은 섹션은 경고와 함께 버려지고, 다른 유효한 섹션은 유지됩니다. 그러면 기능은 런타임 또는 빌드 환경, 혹은 기본값인 전체 부어로 폴백합니다. Docker의 소스 검증은 엄격하여, 재정의가 유효하지 않은 입력 파일을 바로잡기 전에 부팅을 실패시킵니다.

```
{"version":1,"capabilities":["data:add","export:data"],"interface":
{"enabled":true,"level":"intermediate","lock":true}}
```

기능 세부 사항과 Docker 라우트 강제 적용에 대해서는 [배포 기능과 셀프 호스팅: 컨테이너 정책 강제 적용](#)을 참조하세요.

예시

```
{
  "version": 1,
  "capabilities": ["project:edit", "data:add", "processing:run", "export:data", "plugins:install", "settings:manage"],
  "interface": {
    "enabled": true,
    "level": "intermediate",
    "lock": true,
    "hiddenDataSources": ["arcgis"],
    "hiddenPlugins": ["plugin-a"],
    "hiddenMenus": ["help"],
    "hiddenMenuItems": ["project.print"]
  },
  "plugins": {
    "registryUrl": "https://plugins.example.com/registry.json",
    "allowed": ["acme-tools"],
    "blocked": ["bad-plugin"],
    "sideload": false,
    "defaultActive": ["acme-tools"]
  },
  "services": {
    "builtins": true,
    "catalog": [
      {
        "id": "city-wms",
        "name": "City WMS",
        "kind": "wms",
        "category": "Municipal",
        "fields": { "url": "https://maps.example.com/wms", "version": "1.3.0", "opacity": 0.8, "transparent": true }
      }
    ]
  }
}
```

```

},
"sharing": {
  "shareUrl": "https://projects.example.com",
  "collabUrl": "wss://relay.example.com",
  "embedOrigins": ["https://portal.example.com"]
},
"geolens": { "url": "same-origin" },
"ai": { "enabled": true, "model": "gpt-5-mini" },
"branding": { "appName": "Acme Maps", "welcome": false }
}

```

자동 완성과 검증을 위해 편집기가 스키마를 참조하도록 하려면 "\$schema": "https://raw.githubusercontent.com/opengeos/GeoLibre/main/schema/deployment.schema.json" 을 추가하세요. GeoLibre는 \$schema 를 무시합니다.

필드 참조

version 을 제외한 모든 섹션과 모든 필드는 선택 사항입니다. 섹션이 없으면 "지정되지 않음"을 의미하며, 우선순위 체인의 다음 소스가 적용됩니다.

최상위

필드	타입	의미
version	1	정책 형식 버전입니다. 다른 버전의 문서는 무시됩니다.

capabilities

배포 기능에 정의된 기능 이름의 배열입니다: project:edit, data:add, processing:run, export:data, plugins:install, settings:manage. 생략하면 모든 기능이 부여되고, [] 는 아무것도 부여하지 않습니다.

interface

필드	타입	의미
enabled	boolean	UI 프로필 필터링의 활성화 여부입니다 (기본값 true).
level	beginner intermediate advanced	숨김 목록의 초기값을 정하는 숙련도 프리셋입니다.
lock	boolean	사용자가 Settings에서 프로필을 변경하지 못하게 합니다.
hiddenDataSources, hiddenPlugins, hiddenMenus, hiddenMenuItems	string[]	프리셋을 재정의하는 명시적 숨김 id 목록입니다.

plugins

필드	타입	의미
registryUrl	string	플러그인 마켓플레이스 레지스트리 URL로, 절대 경로이거나 앱 기준 상대 경로입니다.
allowed	string[]	로드가 허용되는 외부 플러그인 id입니다. 생략하면 모두 허용, [] 는 아무것도 허용하지 않습니다.
blocked	string[]	절대 로드되지 않는 외부 플러그인 id입니다.
sideload	boolean	매니페스트 URL, zip, 디렉터리 또는 프로젝트 파일에서의 설치를 허용합니다(기본값 true).
defaultActive	string[]	새 프로젝트에서 활성화되는 플러그인 id입니다.

services

필드	타입	의미
builtins	boolean	false 로 설정하면 내장 스타터 서비스를 숨깁니다.
catalog	object[]	선별된 항목: id, name, kind (wms, wfs, wmts, xyz, arcgis, csw), 선택적 category, 비어 있지 않은 fields (문자열, 숫자 또는 불리언 값).

sharing

필드	타입	의미
shareUrl	string	프로젝트 서버 URL(https://...; 일반 http:// 는 localhost / 127.0.0.1 에서만 허용) 또는 Share와 Gallery를 제거하는 off.
collabUrl	string	실시간 협업 릴레이(wss://...; 일반 ws:// 는 루프백에서만 허용).
embedOrigins	string[]	프레임에 삽입된 앱을 제어할 수 있는 출처(https://host), 또는 모두 허용하는 *.

geolens

필드	타입	의미
url	string	기본 GeoLens 서버(https://...; 일반 http:// 는 컨테이너 내 루프백에서만 허용), same-origin, 또는 off.

ai

필드	타입	의미
enabled	boolean	동일 출처의 AI 어시스턴트 라우트를 노출합니다(기본값 false).
model	string	기본 어시스턴트 모델 id입니다.

branding

필드	타입	의미
appName	string	툴바와 탭 제목에 표시되는 앱 이름으로, 최대 60자입니다.
welcome	boolean	false 로 설정하면 첫 실행 환영 마법사를 건너웁니다.

생략과 빈 값의 차이

capabilities 와 plugins.allowed 의 경우, 필드를 생략하는 것과 [] 를 쓰는 것은 정반대의 의미입니다. 생략은 "이 파일에서는 제한하지 않음"을 뜻하고, [] 는 "아무것도 부여/허용하지 않음"을 뜻합니다.

플러그인 우선순위

blocked 에 있는 id는 allowed 에도 있더라도 절대 로드되지 않습니다. allowed 가 있으면, 여기에 없는 비번들 외부 id는 로드되지 않습니다.

평가기는 사이드로드 권한을 먼저 확인한 다음 blocked , 그다음 allowed 를 확인합니다. public/plugins/ 아래의 번들된 드롭인은 allowed 와 sideload 에서 제외되지만 blocked 는 여전히 따릅니다. 내장 플러그인은 외부 플러그인이 아니므로 이러한 로드 제한의 적용을 받지 않습니다.

URL 플러그인의 경우 GeoLibre는 id를 알아내기 위해 plugin.json 을 가져와야 합니다. 거부된 id는 엔트리나 스타일시트를 가져오거나 코드를 임포트하는 일이 결코 없습니다. 거부는 외부 플러그인 로드 문제로 표시됩니다.

sideload: false 이면 매니페스트 URL, zip, 디렉터리 컨트롤이 사라지고 프로그래밍 방식의 설치가 거부됩니다. 프로젝트가 제공한 매니페스트 URL은 신뢰 프롬프트를 표시하지 않으며 설정에 신뢰 항목으로 추가될 수 없습니다. 이전에 설치된 URL은 설정에 남아 있지만, 현재 레지스트리의 허용된 항목이 인식하는 URL만 로드할 수 있습니다 (번들된 드롭인 포함). 해당 레지스트리를 사용할 수 없으면 설치된 URL은 안전하게 닫힌 상태로 실패하며, 번들된 드롭인은 여전히 로드할 수 있습니다. 기존에 파일로 설치된 아카이브와 추가 디렉터리는 로드할 수 없습니다. 건너뛴 각 구성 디렉터리는 내용을 읽지 않은 채 외부 플러그인 로드 문제로 보고됩니다. 설정에서 트리거되거나 강제된 각 스캔은 이전 로드를 재사용하기 전에 현재 레지스트리 포함 여부를 확인합니다. 더 이상 승인되지 않은 이전에 로드된 URL은 언로드되며, 레지스트리를 사용할 수 없는 경우에도 마찬가지입니다. 설치된 URL과 무결성 핀은 사용자가 제거할 때까지 남아 있으며, 레지스트리 승인이 복원되더라도 변경된 코드를 조용히 신뢰하지는 않습니다. 거부된 레지스트리 항목은 마켓플레이스에서 숨겨지지만, 설치된 소스 URL은 Settings에서 여전히 제거할 수 있습니다.

defaultActive 는 허용되고 로드된 외부 플러그인을 새 프로젝트에서 활성화하도록 표시합니다. 저장된 프로젝트의 활성 플러그인 목록을 재정의하지 않으며, 거부된 플러그인의 로드를 허용하지도 않습니다. URL 업데이트는 이후의 새 프로젝트를 위해 이러한 배포 기본값을 유지합니다. plugins 섹션이 없으면 기존의 로드 및 활성화 동작은 변경되지 않습니다.

플러그인 정책은 클라이언트 측 강제 적용일 뿐 서버 보안 경계가 아닙니다. 수정된 클라이언트는 이를 우회할 수 있습니다. 서명이나 샌드박싱을 제공하지 않습니다.

소스 간 우선순위

높은 순서대로 다음과 같습니다.

1. 최종 `deployment.json` 정책(컨테이너 환경 변수 재정의 포함)
2. 런타임 환경(`window.__GEOLIBRE_DEPLOYMENT_ENV__`)
3. 빌드 시점 환경

레거시 `admin-profile.json` 은 선택된 정책에 비어 있지 않은 유효한 `interface` 섹션이 없을 때에만 인터페이스에 한해 여전히 적용됩니다.

버전 관리

새 필드는 추가만 되며 `version: 1` 을 유지합니다. 다른 버전의 문서는 콘솔 경고와 함께 무시됩니다.

검증

클라이언트 파서는 관대하며 섹션별로 동작합니다.

- 유효하지 않은 필드가 하나라도 있거나 알 수 없는 키가 들어 있는 섹션은 콘솔 경고와 함께 통째로 버려집니다. 다른 섹션은 여전히 적용됩니다.
- 알 수 없는 기능 이름은 `capabilities` 섹션 전체를 버리므로, 배포는 환경 설정 또는 기본값으로 폴백합니다(아무것도 부여하지 않는 것이 아닙니다). 제한이 누락된 것처럼 보이면 콘솔을 확인하세요.
- 알 수 없는 최상위 키는 경고 하나와 함께 무시됩니다.
- JSON이 아닌 내용이나 객체가 아닌 값은 조용히 무시됩니다.

컨테이너는 엄격하게 검증합니다. 알 수 없는 키, 중복되거나 유효하지 않은 id, 공백 제거 후 충돌하는 서비스 id, 안전 정수 범위를 벗어나는 숫자 서비스 필드 값을 거부하며, 이 모두를 JSON Schema만으로는 표현할 수 없습니다.

데스크톱

`deployment.json` 을 Tauri의 `<app_config_dir>` 에 두세요(레거시 프로필 위치는 [UI 프로필 참조](#)). 표준 `org.geolibre.desktop` 애플리케이션 식별자의 경우 경로는 다음과 같습니다.

OS	정책 경로
Linux	<code>\$XDG_CONFIG_HOME/org.geolibre.desktop/deployment.json</code> , 또는 <code>XDG_CONFIG_HOME</code> 이 설정되지 않은 경우 <code>~/.config/org.geolibre.desktop/deployment.json</code>
macOS	<code>~/Library/Application Support/org.geolibre.desktop/deployment.json</code>

OS	정책 경로
Windows	%APPDATA%\org.geolibre.desktop\deployment.json (일반적으로 C:\Users\<user>\AppData\Roaming\org.geolibre.desktop\deployment.json)

이 경로들은 Tauri의 [앱 설정 디렉터리](#)를 따릅니다. 샌드박스 설치에서는 디렉터리가 샌드박스 내부로 결정될 수 있으며, 다른 애플리케이션 식별자를 사용하는 커스텀 빌드는 해당 식별자를 대신 사용합니다.

`read_deployment_policy` 명령은 원시 UTF-8 텍스트를 반환하거나, 파일이 없으면 `null` 을 반환합니다. 파서는 선행 UTF-8 BOM을 허용합니다. 선택된 정책은 기능 제한을 포함하여 앱을 다시 빌드하지 않고도 첫 렌더링 전에 적용됩니다. 파일을 변경한 뒤에는 GeoLibre를 다시 시작하세요. 변경 사항은 감시되지 않습니다.

설정 디렉터리에 파일이 존재하면 비어 있거나 형식이 잘못되었거나 지원되지 않는 버전이더라도 그 파일이 권위를 가집니다. 즉 번들 파일이나 웹 파일로 폴백하는 대신 정책 없음을 반환합니다. 설정 디렉터리 정책과 웹 정책은 결코 병합되지 않습니다. 파일이 없으면 GeoLibre는 대신 `<base>/deployment.json` 을 가져옵니다. 그 밖의 읽기 오류(권한 실패 포함)는 모든 빌드에서 콘솔 경고를 내고 같은 웹 파일로 폴백합니다. 지원되지 않는 버전 역시 모든 빌드에서 경고를 냅니다. 두 위치 모두에 파일이 없으면 기존 동작은 변경되지 않습니다.

비어 있지 않은 유효한 정책 `interface` 는 레거시 프로필을 통째로 대체합니다. 없거나 비어 있거나 유효하지 않아 버려진 `interface` 는 레거시 프로필을 계속 적용 가능 상태로 둡니다. 설정 디렉터리는 사용자가 쓸 수 있는 곳으로, 이는 데스크톱 프로비저닝이지 보안 경계나 서버 측 강제 적용이 아닙니다.

Docker

이미지는 부팅할 때마다 생성된 공개 파일을 `/usr/share/nginx/html/deployment.json` 에 기록하며, 이 파일은 `Cache-Control: no-store`, `Content-Type: application/json`, `X-Content-Type-Options: nosniff` 헤더와 함께 `/deployment.json` 으로 제공됩니다. 소스 파일은 `GEOLIBRE_DEPLOYMENT_FILE` 로 선택합니다(예: `/etc/geolibre/deployment.json`). 소스 파일은 해당 위치에 읽기 전용으로 마운트하고, 생성된 출력 위치에는 절대 마운트하지 마세요. 소스나 재정의가 없으면 이미지는 `{"version":1}` 을 기록하며, 이는 기능 제한을 추가하지 않습니다.

소스 파일은 환경 변수 재정의가 적용되기 전에 엄격하게 검증됩니다. 그런 다음 비어 있지 않은 `GEOLIBRE_*` 값이 대응하는 필드를 재정의하며, 빈 값은 설정되지 않은 것으로 간주됩니다. 유효하지 않은 입력은 부팅을 중단시키며 재정의로 바로잡을 수 없습니다.

```
docker run --rm -p 8080:80 \
  -v "$PWD/deployment.json:/etc/geolibre/deployment.json:ro" \
  -e GEOLIBRE_DEPLOYMENT_FILE=/etc/geolibre/deployment.json \
  -e GEOLIBRE_CAPABILITIES=data:add,export:data \
  ghcr.io/opengeos/geolibre:latest
```

구성 변경은 다음 컨테이너 시작 시 적용되며, 이미지를 다시 빌드할 필요는 없습니다.

변수	정책 필드
GEOLIBRE_CAPABILITIES	capabilities: 심표로 구분된 이름, 또는 아무것도 부여하지 않는 none
GEOLIBRE_SERVICES_FILE	services.catalog
GEOLIBRE_BUILTIN_SERVICES=off	services.builtins: false
GEOLIBRE_SHARE_URL	sharing.shareUrl
GEOLIBRE_COLLAB_URL	sharing.collabUrl
GEOLIBRE_EMBED_ORIGINS	sharing.embedOrigins
GEOLIBRE_GEOLENS_URL	geolens.url
GEOLIBRE_APP_NAME	branding.appName: 공백이 축약되고 60자로 잘립니다
GEOLIBRE_AI_URL / GEOLIBRE_AI_MODEL	ai.enabled: true / ai.model

부팅 로그에는 재정의마다 Deployment policy: <path> from <VAR> = <value> 줄이 하나씩 기록됩니다. 토큰은 절대 로그에 나타나지 않으며, URL의 쿼리 문자열이나 프래그먼트는 [redacted] 로 대체됩니다.

유효하지 않은 입력은 JSON 경로를 명시하는 ERROR: 줄과 함께 부팅을 중단시키므로 (예: ERROR: GEOLIBRE_DEPLOYMENT_FILE capabilities[1] must be one of: ...), nginx가 요청한 것보다 약한 정책으로 시작되는 일은 없습니다. ai.enabled: true 가 포함된 파일은 GEOLIBRE_AI_URL , GEOLIBRE_AI_PROXY_URL , GEOLIBRE_AI_PROXY_TOKEN 도 필요하며, 그렇지 않으면 부팅이 실패합니다.

GEOLIBRE_CAPABILITIES 는 컨테이너 생성 시점에 최종 정책의 클라이언트 부여 내용을 재정의합니다. nginx가 강제 적용하는 것은 생성된 정책이며, 빌드 시점의 클라이언트 부여만으로는 nginx가 구성되지 않습니다.

공개 파일이므로 비밀 정보 금지

deployment.json 은 모든 브라우저에 제공됩니다. 여기에 비밀 정보를 절대 넣지 마세요. AI 프록시 URL과 토큰, 사이드카 토큰, 신뢰할 수 있는 프록시, Basic Auth, CSP는 서버 환경 변수와 파일에 두어야 합니다.

클라이언트 측 숨김은 강제 적용이 아닙니다

인터페이스 요소를 숨기거나 제거해도 브라우저 개발자 도구를 가진 사람을 막을 수는 없습니다. Nginx 강제 적용은 번들된 Docker 라우트에만 적용되며, 데스크톱 처리, 브라우저 WASM, 별도로 노출된 사이드카, 플러그인 실행을 제한하지 않습니다. [배포 기능](#)과 [셀프 호스팅](#)을 참조하세요.

<https://geolibre.app/i18n/>

국제화(i18n)

GeoLibre의 인터페이스는 번역할 수 있습니다. UI 문자열은 컴포넌트에 하드코딩되지 않고 JSON 메시지 카탈로그에 들어 있으므로, 카탈로그를 하나 추가하는 것만으로 앱을 어떤 언어로든 제공할 수 있습니다. 이 계층은 `react-i18next` 위에 구축되어 있으며 `apps/geolibre-desktop/src/i18n/`에 있습니다.

이 작업은 **점진적**으로 진행됩니다. 사용 빈도가 높은 영역(상단 툴바와 메뉴, Settings 대화 상자)이 먼저 번역되고, 번역되지 않은 문자열은 이전될 때까지 영어로 폴백됩니다. 새 언어를 추가할 때 컴포넌트 코드를 수정할 필요는 전혀 없습니다.

동작 방식

```
apps/geolibre-desktop/src/i18n/
├── index.ts           # i18next init, catalog auto-discovery, language detection
├── languages.ts       # friendly language names + resolution helpers
├── i18next.d.ts       # types t() keys against the English catalog
└── locales/
    ├── en.json        # the baseline English catalog (source of truth)
    ├── zh.json        # Simplified Chinese
    ├── es.json        # Spanish
    ├── fr.json        # French
    ├── de.json        # German
    ├── pt.json        # Portuguese
    ├── it.json        # Italian
    ├── nl.json        # Dutch
    ├── ja.json        # Japanese
    ├── ko.json        # Korean
    ├── ru.json        # Russian
    ├── tr.json        # Turkish
    ├── id.json        # Indonesian
    ├── hi.json        # Hindi
    ├── th.json        # Thai
    ├── vi.json        # Vietnamese
    ├── az.json        # Azerbaijani
    ├── ka.json        # Georgian
    ├── ar.json        # Arabic (right-to-left)
    ├── he.json        # Hebrew (right-to-left)
    └── fa.json        # Persian (right-to-left)
```

영어 이외의 카탈로그는 기계 번역의 도움을 받아 초기 작성되었으며, 원어민의 검토와 수정을 환영합니다(PR 환영).

- **en.json 이 단일 진실 공급원입니다.** `t()` 에서 사용하는 모든 키는 여기에 존재해야 합니다. `i18next.d.ts` 가 `t()` 함수의 타입을 이 파일에 맞춰 정의하므로, 누락되거나 철자가 틀린 키는 컴파일 오류가 됩니다(`npm run typecheck`).
- **카탈로그는 자동으로 발견됩니다.** `index.ts` 가 `import.meta.glob` 을 통해 모든 `locales/*.json` 을 가져오므로, `locales/fr.json` 을 추가하기만 하면 프랑스어를 선택할 수 있습니다. 코드 변경이 필요 없습니다.

- **영어는 번들되고, 다른 로케일은 지연 로드됩니다.** `en.json` 은 앱 청크에 포함됩니다(폴백이자 타입 소스이기 때문입니다). 다른 모든 카탈로그는 별도의 청크(`i18n-locale-<code>`)로, 활성 언어가 될 때에만 임포트됩니다. 초기 로케일은 첫 렌더링 전에 대기하고(`index.ts` 의 `i18nReady`), 언어 전환(`useLanguage.ts`)은 대상 카탈로그를 로드한 뒤 적용합니다. 완전히 번역된 비영어 카탈로그의 총 크기는 수 MB에 달하므로, 부팅 시 모두 제공하는 것은 피합니다. 서비스 워커는 이러한 청크를 앱 셀 프리캐시에서 제외하고 첫 사용 시 각각을 `CacheFirst` 방식으로 캐시합니다(`vite.config.ts` 의 `maximumFileSizeToCacheInBytes` / `HEAVY_PRECACHE_IGNORES` 참조). 따라서 활성 언어는 첫 온라인 로드 이후 오프라인에서도 동작하며, 영어는 조건 없이 오프라인에서 동작합니다.
- **번들된 로케일은 `en.json` 보다 뒤쳐질 수 있습니다.** 비영어 카탈로그에서 누락된 키는 런타임에 `en` 으로 폴백되며(`fallbackLng: "en"`), 카탈로그는 각 릴리스 전에 채워집니다. **카탈로그를 완전하게 유지하기**를 참조하세요.

언어 감지

시작 시 초기 언어는 다음 우선순위 순서로 결정됩니다(`index.ts` 의 `getInitialLanguage`).

1. `?locale=` 또는 `?lang=` **쿼리 매개변수**(임베드용 — 기존의 `?theme=` / `?maponly` 매개변수와 일관됨),
2. **데스크톱 설정에 저장된 언어**(`language` 필드),
3. 브라우저의 선호 언어(`navigator.languages`),
4. 기본값 `en`.

카탈로그가 제공되는 언어만 인정되며, 그 외의 값은 다음 규칙으로 넘어갑니다. 앱 내 선택기(Settings 메뉴 → Language, 또는 Settings 대화 상자의 Language 섹션)는 언어를 즉시 변경하고 선택을 기록하여 다시 로드해도 유지되도록 합니다.

`?locale` 매개변수는 임베드가 언어를 인식하도록 합니다. 예:

```
https://geolibre.app/?maponly&locale=es
```

컴포넌트에서 번역 사용하기

```
import { useTranslation } from "react-i18next";

function Example() {
  const { t } = useTranslation();
  return <button aria-label={t("common.cancel")}>{t("common.save")}</button>;
}
```

- **보간:** `"Remove {{name}}"` 과 같은 카탈로그 값에 대해 `t("settings.env.removeAria", { name })` 을 호출합니다.
- **복수형:** `key_one` / `key_other` 를 정의하고 `t("settings.env.variablesCount", { count })` 를 호출합니다.

- **리치 텍스트(링크, 굵게):** 이름이 지정된 `components` 와 함께 `<Trans>` 컴포넌트를 사용합니다. 예를 들어 `"... at <tokenLink>share.geolibre.app/settings</tokenLink>."` 라는 카탈로그 값을 `<Trans i18nKey="..." components={{ tokenLink: <a href="..." /> }} />` 로 렌더링합니다.
- **모듈 범위 상수**는 `t()` 를 호출할 수 없습니다(스코프 내에 속이 없음). 상수에 영어 문자열 대신 안정적인 **카탈로그 키**를 저장하고, 렌더링 지점에서 `t(item.labelKey)` 로 해석하세요. `SettingsDialog.tsx` 의 `SECTION_ITEMS` 와 `toolbar/constants.ts` 의 명령/메뉴 배열을 참조하세요.

다음은 번역하지 **마세요**: 개발자 로그(`console.*`), URL, 스토어 ID, 파일 형식 이름/확장자, 영속화된 데이터 값.

새 언어 추가하기

1. `apps/geolibre-desktop/src/i18n/locales/en.json` 을 `locales/<code>.json` (예: `fr.json`, `pt-BR.json`)으로 복사하고 값을 번역합니다. 모든 키를 번역하는 것을 목표로 하세요. 빠뜨린 키는 런타임에 영어로 폴백되며, 카탈로그 테스트는 실패시키는 대신 이를 보고합니다. 영어가 해당 언어 자체의 관례인 경우 (제품 이름, 파일 형식, 단위)에만 값을 영어로 두세요.
2. 복수형 키에는 해당 언어의 CLDR 복수 범주를 사용하세요. 언어에 있는 형태만 유지합니다(예: 인도네시아어/일본어/한국어/중국어/태국어는 `_one` 을 제거하고 `_other` 만 유지하고, 러시아어는 `_few` / `_many` 를 추가하며, 아랍어는 여섯 가지 형태를 모두 가집니다). 카탈로그 테스트는 복수형을 정규화한 키를 비교하므로, 로케일이 `en` 보다 더 많거나 적은 복수 형태를 가져도 실패하지 않습니다.
3. 코드가 아직 `LANGUAGE_NAMES` (`languages.ts`)에 없다면, 선택기에 친숙한 모국어 이름이 표시되도록 항목을 추가하세요. (선택 사항 — 없으면 원시 코드로 폴백됩니다.)
4. 이것으로 끝입니다. 해당 언어는 **Settings → Language**(설정 → 언어)(그리고 Settings 대화 상자의 Language 섹션)에 나타나며 `?locale=<code>` 로도 동작합니다.

카탈로그를 완전하게 유지하기

`tests/i18n-catalogs.test.ts` (`npm run test:frontend` 의 일부)는 번들된 모든 카탈로그를 `en.json` 과 대조하여 검사합니다.

- **추가 키 없음.** `en.json` 에 없는 키는 오타이거나 잔재입니다.
- **동일한 플레이스홀더.** 각 번역은 영어 문자열의 `{{placeholders}}` 와 `<tag>` 마크업을 유지합니다.
- **누락된 키는 보고되지만 강제되지 않습니다.** 테스트는 각 로케일의 누락된 키를 로그로 남깁니다. 복수형 키는 기본형으로 비교되므로, 로케일은 `en` 의 `_one` / `_other` 쌍 대신 해당 언어에 필요한 복수 형태(`zh/ja/ko/id/th/vi`는 `_other` 만, `he`는 `_two`, `ru`는 `_few` / `_many`, `ar`는 여섯 가지 모두)를 가질 수 있습니다.

따라서 `t()` 키를 추가하거나 내장 플러그인이 `app.translate` 를 통해 해석하는 `plugin.<id>.*` 키를 추가하는 변경은 `en.json` 에만 추가하면 됩니다. 다른 카탈로그는 각 릴리스 전에 `geolibre-i18n` 감사를 통해 채워집니다. 직접 번역을 추가할 때에는 카탈로그의 순서를 바꾸지 말고 키를 추가하세요(여러 카탈로그가 `en.json` 의 순서에서 벗어나 있으며, 순서를 바꾸는 diff는 실제 변경 내용을 가립니다).

오른쪽에서 왼쪽으로 쓰는 언어

아랍어(`ar`), 히브리어(`he`), 페르시아어(`fa`)는 오른쪽에서 왼쪽으로 쓰는(RTL) 카탈로그로 제공되며, 레이아웃 방향은 추가 RTL 로케일을 덧붙일 수 있도록 구성되어 있습니다.

- **방향 레지스트리.** `languages.ts` 의 `RTL_LANGUAGES` 는 RTL 기본 서브태그(`ar` , `he` , `fa` , `ur`)를 나열합니다. `languageDirection(code)` 는 `resolveLanguage` 와 같은 방식으로 모든 태그를 해석합니다(`ar-SA` → `rtl` , 알 수 없음 → `ltr`).
- **문서 미러링.** `index.ts` 는 `il8next` 의 `languageChanged` 이벤트를 구독합니다. `init` 전에 등록되므로 초기 언어에 대해서도 발생하며, `document.documentElement.lang` 과 `.dir` 을 설정합니다. `dir="rtl"` 이면 flex 기반 셀이 자동으로 미러링됩니다. Layers는 오른쪽에, Style은 왼쪽에 위치합니다.
- **Radix 방향.** `App.tsx` 는 셀을 `DirectionProvider` (`@geolibre/ui` 에서 재내보냄)로 감싸므로, 메뉴와 하위 메뉴가 읽기 방향으로 열리고 화살표 키, 슬라이더, 탭이 미러링된 순서를 따릅니다.
- **논리적 유틸리티.** 컴포넌트는 물리적 `m1-* / left-*` 형태 대신 Tailwind의 논리적 간격/위치 유틸리티(`ms-* / me-*` , `ps-* / pe-*` , `text-start / text-end` , `border-s / border-e` , `start-* / end-*`)를 사용하므로 스타일이 문서 방향에 따라 뒤집힙니다. 새 코드에서는 논리적 형태를 사용하고, 한 요소의 같은 쪽에 물리적 유틸리티와 논리적 유틸리티를 섞어 쓰지 마세요. `tailwind-merge` 는 이들을 서로 다른 그룹으로 취급하므로 둘 다 적용됩니다. 대칭 중앙 정렬(`left-1/2 + -translate-x-1/2`), 지도에 고정된 오버레이, JS로 계산된 픽셀 오프셋은 의도적으로 물리적 형태를 유지합니다.
- **지도 크롬은 ltr을 유지합니다.** MapLibre는 물리적 CSS로 컨트롤을 고정하므로 `index.css` 가 `.maplibregl-control-container { direction: ltr }` 로 고정합니다. 지도 크롬은 모든 언어에서 동일합니다. `.maplibregl-map` 하위 트리의 나머지는 의도적으로 미러링되도록 두었으므로, 지도 컨테이너에 포털된 앱 오버레이(예: 스토리 맵 프레젠테이션)는 여전히 RTL 레이아웃을 따릅니다. 팝업의 아랍어 텍스트는 유니코드 양방향 알고리즘을 통해 여전히 올바르게 형태가 잡히며, 인터랙티브 지도의 베이스맵 레이블 셰이핑은 MapLibre의 내장 `bidirectional/셰이핑(6.11+)`이 처리합니다. 레거시 플러그인이 쿼드러 글자를 잘못 결합하기 때문에 RTL 텍스트 플러그인은 여기에 등록되지 않습니다. 내보낸 스토리 맵 HTML은 구버전 CDN `maplibre-gl`을 고정하고 있어 여전히 해당 플러그인을 로드합니다.

또 다른 RTL 언어를 추가하는 방법은 다른 언어와 동일한 절차에, 아직 등록되지 않은 경우 그 기본 서브태그를 `RTL_LANGUAGES` 에 추가하는 것입니다.

로케일을 인식하는 서식 유지하기

숫자, 날짜, 좌표 서식은 하드코딩된 서식 대신 `Intl` API(예: `new Intl.NumberFormat(undefined, ...) / Intl.DateTimeFormat`)를 통해 런타임 로케일을 계속 사용해야 합니다. 그래야 값이 사용자의 지역 관례에 맞게 렌더링됩니다.

처리 도구 메타데이터

도구 이름, 설명, 그룹 라벨, 매개변수 라벨/도움말 텍스트, 선택 옵션 라벨은 i18n에 접근할 수 없는 처리 레지스트리에 들어 있습니다. Processing 대화 상자(Vector / Network / Spatial statistics / Raster / Batch 도구, Whitebox 툴박스, Model Builder 팔레트)는 이를 `apps/geolibre-desktop/src/lib/processing-tool-i18n.ts` 를 통해 렌더링하며, 이 모듈은 `processing.toolMeta.<catalog>.<toolId>...` 를 해석하고 레지스트리 자체의 문자열로 풀백합니다. 도구 id가 레지스트리 간에 중복되기 때문에 카탈로그 이름(`vector | network | statistics | raster | whitebox`)이 모든 키의 일부입니다. 예를 들어 `reproject` 는 벡터 도구이면서 래스터 도구이기도 합니다. 그룹 라벨은 하나의 네임스페이스 `processing.toolGroup.<slug>` 를 공유하는데, 같은 라벨("Analysis")이 둘 이상의 레지스트리에 나타나기 때문입니다.

풀백이 레지스트리 문자열 자체이므로 **카탈로그 항목이 전혀 없어도 영어는 올바릅니다**. 작은 네 개의 번들 레지스트리에 대해서는 `en.json` 이 번역자가 작업할 키를 담고 있습니다. `tests/i18n-catalogs.test.ts` 는 다른 번들 로케일에서 영어 대응 항목이 없는 키를 거부합니다. 이 기준선은 생성됩니다.

```
npm run i18n:tools          # regenerate the en.json baseline from the registries
npm run i18n:tools:check    # fail if it is out of date (runs in `npm run ci`)
```

도구, 매개변수, 선택 옵션을 추가하거나 이름을 바꾼 뒤에는 이를 다시 실행하고 결과를 커밋하세요. `en.json` 의 `processing.toolMeta` 와 `processing.toolGroup` 만 다시 작성되며, 다른 로케일은 절대 건드리지 않습니다.

Whitebox 도구 이름과 매개변수 도움말 텍스트는 `processing.toolMeta.whitebox.<toolId>...` 를 사용하지만, 수백 개에 달하는 설명은 **어떤 로케일에도 번들되지 않습니다**. Whitebox가 런타임에 영어 풀백을 제공하며, 선택적 번역은 [opengeos/geolibre-language-packs](https://github.com/GeoLibre/geolibre-language-packs) 에 호스팅되는 버전 관리 언어 팩입니다. 사용자는 Settings → Language에서 활성 로케일의 팩을 다운로드하거나 호환되는 JSON 파일을 가져올 수 있습니다. 팩은 병합되기 전에 검증되며, 프로젝트 및 데스크톱 설정과 분리된 IndexedDB 데이터베이스에 저장됩니다.

v1 팩 형식은 다음 루트만 허용합니다.

- `processing.toolMeta.whitebox`
- `processing.whitebox.categories`
- `processing.whitebox.menuTool`
- `processing.whitebox.menuSubcategory`

공식 팩은 `https://languages.geolibre.app/v1/whitebox/<locale>.json` 에서 가져옵니다. 셀프 호스팅 빌드는 `VITE_LANGUAGE_PACK_BASE_URL` 을 내부 미러로 설정할 수 있습니다. 로컬 파일 가져오기는 네트워크 접근 없이도 계속 동작합니다.

Model Builder 팔레트는 이 둘을 섞어 쓰는 유일한 곳이므로 그룹 제목에 주의가 필요합니다. `groupModelTools` 는 원시 라벨을 기준으로 그룹을 묶기 때문에 제목에는 고유한 제공자가 없고, WASM 카탈로그는 래스터 레지스트리의 Terrain 과 같은 슬러그가 되는 `terrain` 카테고리(소문자)를 제공합니다. 둘 다 번역하면 동일한 라벨의 제목이 두 개 렌더링됩니다. 대신 `translateModelToolGroup` 은 그룹의 도구를 기준으로 판단합니다. 제목은 도구 중 적어도 하나가 **벡터 레지스트리**에서 온 경우에만 번역됩니다. 팔레트 제목에는 `translateToolGroup` 을 직접 쓰

지 말고 이 함수를 사용하세요. `modelProviderCatalog` 가 이제 Whitebox 도구에 대해 `null` 대신 `"whitebox"` 를 반환하므로 "자체 카탈로그가 있는가"는 더 이상 같은 검사가 아닙니다. 선택적 Whitebox 언어 팩이 설치되어 있어도 Whitebox 전용 그룹의 제목은 원시 카탈로그 텍스트로 유지됩니다.

플러그인 관련 표면

플러그인 표시 이름은 `toolbar.plugin.<pluginId>` 를 통해 해석되며, 이를 표시하는 모든 표면(Plugins 메뉴, 명령 팔레트, Settings → Interface, Manage Plugins)은 `lib/plugin-display-name.ts` 를 거치므로, 한 메뉴에서는 번역되어 보이고 다음 메뉴에서는 영어로 보이는 일이 없습니다. 카탈로그 항목이 없는 플러그인은 등록된 이름으로 폴백되며, 이는 브랜드/고유명사 플러그인(Mapillary, NASA Earthdata 등)과 호스트가 빌드 시점에 알 수 없는 외부 설치 플러그인에 적합합니다.

플러그인 **자체**의 UI는 순수 DOM이므로 `useTranslation` 을 사용할 수 없습니다. 대신 앱 API가 `getLocale()`, `onLocaleChange(listener)`, `translate(key, defaultValue, params)`, `registerTranslations(resources)` 를 제공하며, 툴바 메뉴 라벨과 패널 제목은 `getter` 함수를 받아 `translate` 를 호출하고 언어 변경 후 다시 읽을 수 있습니다. 플러그인 API 참조의 [앱 언어 따르기](#)를 참조하세요.

내장 플러그인은 문자열을 번들 카탈로그의 `plugin.<pluginId>.*` 아래에 두고 `@geolibre/plugins` 의 `createPluginTranslator(app, id)` 로 해석합니다. 이 함수는 키에 접두사를 붙이고 영어 폴백을 보간합니다. 도킹된 패널의 헤더는 `toolbar.plugin.<id>` 를 읽는 `pluginDisplayTitle(app, id, name)` 을 사용하므로, 헤더가 그 패널을 연 Plugins 메뉴 항목과 일치합니다. 일부 오래된 내장 플러그인은 대신 `usePluginLabelSync` 가 언어 변경 시마다 호출하는 `set<Plugin>Labels(...)` 내보내기를 통해 호스트로부터 라벨을 받습니다. 두 경로 모두 같은 카탈로그에 도달합니다.

플러그인이 한 번 만든 순수 DOM(버튼, 상태 줄)은 언어 변경 시 다시 렌더링되지 않습니다. GeoAgent Earth Engine의 **Sign in**(로그인) 버튼처럼 `app.onLocaleChange` 로 구독하고 텍스트를 다시 설정하세요.

플러그인 코드에서 발생하는 오류

UI가 아닌 플러그인 모듈(ArcGIS 레이어 로더, Earth Engine 로그인, 경로 기록기)에는 `t()` 가 없으며, 그 메시지는 호출자가 `error.message` 를 표시하는 곳 어디에서든 사용자에게 도달합니다. 대화 상자, 토스트, 또는 메시지를 그대로 렌더링하는 상위 컨트롤일 수 있습니다. 일반 `Error` 대신 `@geolibre/plugins` 의 `LocalizedError` 를 던지세요.

```
throw new LocalizedError(
  "arcgisService.errors.serviceRequestFailed",
  "ArcGIS service request failed with {{status}}.",
  { status: response.status },
  { cause },
);
```

이 오류는 호스트가 `setLocalizedErrorTranslator (hooks/usePlugins.ts)` 로 한 번 등록한 번역기를 통해 생성 시점에 스스로 번역되므로, 표시 지점을 바꿀 필요가 없습니다. 영어 원문은 `error.englishMessage` 에 남습니다. 서버가 제공한 텍스트도 함께 담는 오류 클래스의 경우, GeoLibre가 작성한 부분만 `localizedMessage(key,`

`english, params`) 로 만드세요. 서버 자체의 메시지(ArcGIS 오류 엔벨로프, Google OAuth 오류)는 그대로 두세요. 같은 영어 문구로 `en.json` 에 키를 추가하세요. `tests/localized-error.test.ts` 는 `packages/plugins/src` 에서 모든 호출을 스캔하여 키가 누락되었거나 영어 문구가 다르면 실패합니다.

레지스트리에서 파생되는 키

Deck.gl Layer 대화 상자는 `i18n`에 접근할 수 없는 레이어 레지스트리(`deckgl-viz/registry.ts`)의 이름을 표시합니다. `lib/deck-viz-i18n.ts` 는 레지스트리의 `id`에서 키를 파생하고

(`addData.deckViz.kinds.<kind>.label|description`, `addData.deckViz.categories.<category>`, `addData.deckViz.roles.<roleKey>`) 레지스트리의 영어로 폴백합니다. `tests/deck-viz-i18n.test.ts` 는 `en.json` 과 레지스트리가 일치하지 않으면 실패하므로, 둘을 함께 업데이트하세요.

알려진 제한 사항

일부 플러그인은 자체 UI를 렌더링하는 **서드파티 MapLibre 컨트롤**을 감쌉니다(예: Esri Wayback 플러그인의 `maplibre-gl-esri-wayback` 컨트롤로, "Only versions with local changes"와 같은 라벨이 있음). 이러한 문자열은 외부 라이브러리 내부에 있어 GeoLibre의 `react-i18next` 파이프라인을 거치지 않으므로, 선택한 언어와 관계없이 영어로 유지됩니다. 이를 번역하려면 라이브러리의 업스트림 변경(또는 포크)이 필요하며, 여기의 로케일 카탈로그에서는 할 수 없습니다.

<https://geolibre.app/python/>

Python 패키지 (Jupyter)

[pypi](#) [v3.3.0](#)
[Open in Colab](#)
[conda-forge](#) [v3.3.0](#)
[recipe](#) [geolibre](#)

GeoLibre는 Python 패키지 `geolibre` 를 제공합니다. 이 패키지는 GeoLibre 앱 전체를 `anywidget`으로 Jupyter 노트북 셀 안에 임베드하며, `leaflet` 스타일의 API를 제공합니다.

위젯은 완전한 GeoLibre 앱(메뉴, 패널, 처리 도구)을 `iframe` 안에 로드합니다. 상태는 하나의 `.geolibre.json` 프로젝트를 통해 양방향으로 동기화되므로, Python에서 추가한 데이터가 UI에 나타나고, UI에서 수행한 편집(이동, 줌, 레이어 추가)을 Python에서 다시 읽을 수 있습니다.

설치

```
pip install geolibre
```

또는 `conda-forge`에서 `conda`로 설치합니다.

```
conda install -c conda-forge geolibre
```

선택적 `extra`를 설치하면 `GeoDataFrame`과 로컬 벡터 파일을 위한 `GeoPandas/Shapely` 지원, 그리고 메모리 내 래스터를 위한 `xarray/rioxarray/rasterio/rio-tiler` 지원이 제공됩니다.

```

pip install "geolibre[all]"      # both
pip install "geolibre[vector]"  # GeoPandas/Shapely only
pip install "geolibre[raster]"  # xarray/rioxarray/rasterio/rio-tiler only

```

`[all]` 은 두 `extra`의 합집합이므로 이제 `rasterio`(그리고 함께 `GDAL`)를 설치합니다. 기존의 벡터 전용 환경을 이전처럼 가볍게 유지하려면 `[vector]` 를 사용하세요.

선택적 `[all]` `extra`는 `pip` 전용입니다. `conda`로 설치했다면 같은 환경 안에서 `pip install "geolibre[all]"` 로 추가하세요.

지도를 `Dash` 앱에 임베드하려면 `dash` `extra`를 설치하고 `from geolibre import DashMap` 을 사용하세요.

```
pip install "geolibre[dash]"
```

빠른 시작

```

from geolibre import Map

m = Map(center=(-100, 40), zoom=4)

```

```
m.add_geojson("https://example.com/data.geojson", name="Data")
m
```

GeoLibre UI 전체가 셀 안에 렌더링됩니다. 데이터를 더 추가하고 뷰를 제어해 보세요.

```
m.add_tile_layer(
    "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
    name="OpenStreetMap",
    attribution="(c) OpenStreetMap contributors",
)
m.add_cog("https://example.com/dem.tif", name="DEM", colormap="terrain")
m.add_basemap("dark")
m.set_center(-120, 47, zoom=8)
```

Google Earth Engine 레이어는 선택 사항이며 `pip install earthengine-api` 와 자격 증명(한 번의 `ee.Authenticate()`, 그다음 Google Cloud 프로젝트)이 필요합니다.

```
import ee

ee.Authenticate() # once per machine
ee.Initialize(project="your-google-cloud-project")
m.add_ee_layer(ee.Image("USGS/SRTMGL1_003"), {"min": 0, "max": 3000}, name="SRTM")
```

`add_ee_layer` 는 커널에서 Earth Engine 객체를 평가하고 결과 타일 URL을 래스터 레이어로 추가합니다 (ImageCollection은 모자이크되고, 벡터 객체는 래스터 타일로 스타일링됩니다. 벡터 객체의 경우 `vis_params` 는 이미지 키가 아니라 `color`, `fillColor`, `width`, `pointSize` 같은 `ee.FeatureCollection.style()` 키를 받습니다). 이 URL은 만료되는 Earth Engine map id에 묶여 있으므로, 저장된 프로젝트를 다시 열 때 Earth Engine 레이어를 다시 생성해야 할 수 있습니다. 결과는 일반 래스터 타일 레이어이며, 앱 자체의 Earth Engine 패널이 관리하는 라이브 레이어가 아닙니다.

`add_raster` / `add_cog` 는 커널 호스트의 로컬 GeoTIFF 경로도 받습니다. 파일은 번들된 localhost 서버가 제공하므로 앱이 읽을 수 있습니다. 이 방식은 로컬 Jupyter와 VS Code에서 바로 동작합니다. Google Colab에서는 커널 프록시가 브라우저 COG 렌더링에 필요한 바이트 범위(byte-range) 시맨틱을 보존하지 않으므로, GeoLibre는 대신 커널에서 로컬 래스터를 PNG XYZ 타일로 렌더링합니다. JupyterHub는 `jupyter-server-proxy` 가 있을 때 커널 포트를 통해 COG를 라우팅할 수 있습니다. 정적 서버 확장만 있는 배포에서는 커널 파일을 노출할 수 없으므로 그곳에서는 호스팅된 URL을 전달하세요. 제공되는 URL은 세션 범위이므로 로컬 래스터와 함께 저장된 프로젝트는 나중에 다시 열 때 해당 래스터를 복원하지 않습니다. 영구적인 프로젝트에는 호스팅된 URL을 전달하세요.

메모리 내 xarray 객체를 시각화하려면 `geolibre[raster]` 를 설치하세요. `lon` / `lat` 또는 `longitude` / `latitude` 라는 이름의 공간 차원은 기본적으로 EPSG:4326으로 간주됩니다. 그 외의 경우 CRS와 차원 이름을 명시적으로 지정하세요. Dataset의 변수는 하나를 선택하지 않는 한 밴드가 됩니다.

```
m.add_raster(data_array, name="Temperature", colormap="viridis")
m.add_raster(
    dataset,
    name="Temperature",
    array_args={"variable": "temperature", "isel": {"time": 0}},
)
```

xarray 레이어를 뒷받침하는 임시 Cloud-Optimized GeoTIFF는 위젯이 닫힐 때 제거되므로, xarray 레이어도 로컬 파일과 동일한 세션 한정 제약을 가집니다. 로컬에서는 브라우저가 그 COG를 직접 읽고, Colab에서는 Colab의 호환되지 않는 바이트 범위 프록시 동작을 피하기 위해 rio-tiler가 일반 PNG XYZ 타일로 렌더링합니다. 작업을 마치면 `m.close()` 를 호출해 즉시 제거하세요. 그렇지 않으면 Map 이 가비지 컬렉션되거나 커널이 정상 종료될 때 제거됩니다. 커널이 강제로 종료되면 파일이 시스템 임시 디렉터리에 남습니다.

스타일을 미리 계산하지 않고도 마커와 데이터 기반 심볼로지를 추가할 수 있습니다.

```
m.add_marker(-122.4, 37.8, properties={"name": "San Francisco"})
m.add_marker_cluster([(-122.4, 37.8), (-122.3, 37.9), (-122.5, 37.7)])
m.add_heatmap([(-122.4, 37.8), (-122.3, 37.9)], radius=35)
m.add_csv("cities.csv", x="longitude", y="latitude", name="Cities")
m.add_choropleth(
    "https://example.com/counties.geojson",
    column="population",
    colormap="blues",
    scheme="quantile",
)
```

범례, 컬러바, 스와이프(분할 지도) 비교를 추가합니다.

```
# The app's legend panel (Controls > Legend), built from each layer's style.
m.set_map_legend("Population", position="bottom-right")

# A built-in land-cover legend, or your own {label: color} dict.
m.add_legend(builtin="nlcd")
m.add_legend(legend_dict={"Water": "#0000ff", "Land": "#00ff00"})

# A colorbar for a continuous raster.
m.add_colorbar(colormap="terrain", vmin=0, vmax=4000, label="Elevation", units="m")

# Compare two layers (or a layer against the basemap) with a swipe slider.
before = m.add_cog("https://example.com/before.tif", name="Before")
after = m.add_cog("https://example.com/after.tif", name="After")
m.split_map(before, after)
```

양방향 동기화

프로젝트가 양방향으로 동기화되므로, UI에서 지도를 이동하거나 줌한 뒤 Python에서 라이브 상태를 다시 읽을 수 있습니다.

```
proj = m.to_project()
proj["mapView"]["center"] # reflects the live UI view
[layer["name"] for layer in proj["layers"]]
```

프로젝트를 저장하고 다시 불러올 수 있으며, 데스크톱 및 웹 앱과 완전히 호환됩니다.

```
m.save_project("my-map.geolibre.json")

m2 = Map()
m2.load_project("my-map.geolibre.json")
m2
```


`to_project()`, `save_project()`, `to_html()` 은 자격 증명(API 키, 토큰, 인증된 요청 헤더, 비밀 환경 변수 ("secret": false 로 표시되지 않은 모든 행), 지오코더 키, 자격 증명 URL 매개변수)을 제거하므로, 직렬화하거나 커밋하거나 공유하는 모든 결과물이 기본적으로 안전합니다. 자격 증명을 다시 입력하지 않고도 계속 동작해야 하는 신뢰할 수 있는 로컬 파일이라면 `to_project()` 또는 `save_project()` 에 `keep_credentials=True` 를 전달하세요.

```
m.save_project("private.geolibre.json", keep_credentials=True)
```

`project` 트레이트 자체는 자격 증명이 제거되지 않으므로, 위젯과의 라이브 양방향 동기화에서는 인증된 레이어가 계속 렌더링됩니다.

지도 옵션

```
Map(
    center=(-100, 40), # [lng, lat]
    zoom=4,
    basemap="dark", # a basemap name or a MapLibre style URL
    height="800px",
    layout="embed", # "embed" (compact UI), "full" (desktop UI), or "maponly"
    theme="light", # "light" or "dark"
)
```

인터랙티브 스크립팅

데이터를 추가하는 것 외에도, 위젯은 **실행 중인 앱을 조회하고 그에 반응**할 수 있습니다. 앱 내 [Python Console](#)과 동일한 인터페이스입니다. 이 호출들은 실행 중인 지도와 왕복 통신을 하므로, 먼저 지도를 표시한 다음(셀에서 `m` 을 표시), 이후 셀에서 조회를 실행해야 합니다.

```
m.get_center() # live [lng, lat], reflecting UI pans/zooms
m.get_bounds() # [west, south, east, north]
m.fly_to(-122.4, 37.8, zoom=10)
m.identify(-122.4, 37.8) # features at a point, like clicking the map

# Layer objects: read and mutate layers, read their features
for layer in m.layers:
    layer.opacity = 0.6
features = m.layers[0].get_features() # list of GeoJSON Feature objects

# Run a processing algorithm; result layers are added to the map
m.list_algorithms()
m.run_algorithm("buffer", {"layer": layer_id, "distance": 1000})

# Read back what the user selected or drew on the map
m.get_selected_features() # the clicked feature(s) as Feature objects
m.get_drawn_features() # features sketched with the Geo Editor
m.user_rois # the drawn ROIs as a GeoJSON FeatureCollection
m.get_drawn_features(as_gdf=True) # the same, as a GeoDataFrame (needs GeoPandas)

png = m.to_image() # PNG bytes (or m.to_image("map.png"))
m.to_html("map.html") # standalone HTML embedding the live project
```

이벤트 콜백으로 사용자 상호작용에 반응할 수 있습니다.

```
m.on_click(lambda e: print("clicked", e["lngLat"]))
m.on_selection_change(lambda e: print("selected", e))
m.on_layer_change(lambda e: print("layers", e["layerIds"]))
```

블로킹 조회

인터랙티브 조회는 앱이 응답할 때까지 커널을 블로킹합니다(자동으로 설치되는 `jupyter_ui_poll` 을 통해). 느린 호출에는 `timeout=` 을 전달하세요. 예: `m.run_algorithm(..., timeout=300)`.

API 참조

인터랙티브 조회, 이벤트 및 처리

메서드	설명
<code>get_view()</code> / <code>get_center()</code> / <code>get_bounds()</code>	라이브 카메라 / 중심 / 뷰포트 경계를 읽습니다.
<code>fly_to(lng, lat, zoom=, bearing=, pitch=, duration=)</code>	카메라를 애니메이션으로 이동합니다.
<code>fit_bounds([w, s, e, n])</code>	카메라를 경계 상자에 맞춥니다.
<code>zoom_to_bounds([w, s, e, n])</code> / <code>zoom_to_layer(layer)</code>	leafmap 스타일의 뷰 도우미입니다. 레이어는 id, 이름 또는 핸들로 지정할 수 있습니다.
<code>identify(lng, lat, layer_id=None)</code>	한 지점에서 렌더링된 피처를 조회합니다.
<code>get_features(layer_id)</code>	레이어의 피처를 <code>Feature</code> 객체로 반환합니다.
<code>get_selected_features(as_gdf=False)</code>	앱에서 선택된 피처를 <code>Feature</code> 객체(또는 <code>GeoDataFrame</code>)로 반환합니다.
<code>get_drawn_features(as_gdf=False)</code> / <code>user_rois</code>	Geo Editor로 그린 피처입니다. <code>user_rois</code> 는 <code>FeatureCollection</code> 으로 반환합니다.
<code>layers</code> / <code>get_layer(id)</code>	<code>Layer</code> 핸들입니다(상태 읽기, <code>name</code> / <code>visible</code> / <code>opacity</code> 설정, <code>set_style</code> , <code>get_features</code> , <code>zoom_to</code> , <code>remove</code>).
<code>layer_names</code> / <code>find_layer(name)</code> / <code>find_layer_index(name)</code>	표시 이름으로 레이어를 조회합니다.
<code>set_layer_visibility(layer, visible)</code> / <code>set_layer_opacity(layer, opacity)</code>	id, 이름 또는 핸들로 레이어를 변경합니다.
<code>list_algorithms()</code>	사용 가능한 처리 알고리즘(<code>id</code> , <code>parameters</code> , ...)을 나열합니다.
<code>run_algorithm(id, parameters=None, timeout=)</code>	알고리즘을 실행합니다. <code>{logs, resultLayerIds}</code> 를 반환합니다.

메서드	설명
<code>run_model_builder(graph, timeout=)</code>	직렬화된 Model Builder 그래프를 표시된 앱에서 실행합니다.
<code>list_whitebox_tools() /</code> <code>run_whitebox_tool(tool_id,</code> <code>parameters=None, timeout=)</code>	번들된 Whitebox 도구를 나열하거나 WASM을 통해 브라우저에서 실행합니다. <code>run_whitebox_tool</code> 은 <code>{logs, resultLayerIds}</code> 를 반환합니다.
<code>to_image(path=None, timeout=)</code>	지도를 PNG 바이트로 캡처하거나 <code>path</code> 에 씁니다.
<code>to_html(path=None, title=, width=,</code> <code>height=, app_url=)</code>	현재 프로젝트(자격 증명 제거됨)를 이 지도의 <code>layout</code> 과 <code>theme</code> 으로 임베드하는 독립형 HTML 페이지를 내보냅니다. HTML을 반환하거나 <code>path</code> 에 씁니다.
<code>on(event, cb) / on_click /</code> <code>on_selection_change /</code> <code>on_layer_change</code>	이벤트 콜백을 등록합니다. 구독 해제 함수를 반환합니다.
<code>request(method, params=None,</code> <code>timeout=)</code>	위 메서드들의 기반이 되는 저수준 명령 프리미티브입니다.

데이터, 뷰 및 프로젝트

메서드	설명
<code>Map(center, zoom,</code> <code>basemap=, renderer=,</code> <code>height=, layout=,</code> <code>theme=,</code> <code>server_proxy=)</code>	지도를 생성합니다.
<code>add_geojson(data,</code> <code>name=, **style)</code>	dict, 파일 경로, URL, JSON 문자열 또는 GeoDataFrame에서 GeoJSON을 추가합니다.
<code>add_gdf(gdf, name=,</code> <code>column=None,</code> <code>**style)</code>	GeoDataFrame을 추가합니다. 선택적으로 단계구분도(choropleth)로 추가할 수 있습니다.
<code>add_csv(data,</code> <code>x="longitude",</code> <code>y="latitude", name=,</code> <code>**style) /</code> <code>add_xy_data(...)</code>	CSV 경로, URL, 텍스트, DataFrame 또는 행 매핑에서 포인트를 추가합니다.
<code>add_marker(lng, lat,</code> <code>name=, properties=,</code> <code>color=, opacity=,</code> <code>radius=,</code> <code>stroke_color=,</code> <code>stroke_width=,</code> <code>shape=, size=, icon=,</code> <code>**style)</code>	단일 포인트 마커를 추가합니다(<code>properties</code> 는 클릭 시 표시됩니다).

메서드	설명
<code>add_markers(points, name=, color=, opacity=, radius=, stroke_color=, stroke_width=, shape=, size=, icon=, **style)</code>	(lng, lat) 쌍, {lng/lon/x, lat/y, ...} dict, GeoJSON 또는 GeoDataFrame에서 포인트 마커를 추가합니다.
<code>add_circle_markers(points, name=, radius=, **style)</code>	명시적인 radius 를 가진 원형 마커를 추가합니다.
<code>add_marker_cluster(points, name=, cluster_radius=, cluster_max_zoom=, **style)</code>	클러스터링된 포인트 마커를 추가합니다.
<code>add_heatmap(points, name=, radius=, intensity=, color_ramp=, weight_field=, **style)</code>	밀도 히트맵 렌더러로 포인트 데이터를 추가합니다. 선택적으로 숫자 필드로 가중치를 줄 수 있습니다.
<code>add_choropleth(data, column, name=, class_count=, colormap=, scheme=, **style)</code>	숫자 column 에서 계산된 등급 심볼로지를 가진 GeoJSON 레이어를 추가합니다.
<code>add_data(data, column=None, name=, **kwargs)</code>	데이터를 추가합니다. column 이 주어지면 단계구분도, 그렇지 않으면 일반 GeoJSON 레이어입니다 (leafmap 호환).
<code>add_vector(data, name=, render_mode=, data_format=, source_layer=, **style)</code>	URL(GeoParquet, FlatGeobuf, 압축된 Shapefile, GeoJSON, ...) 또는 로컬 파일(GeoPandas로 읽어 인라인)에서 벡터 데이터셋을 추가합니다.
<code>add_geoparquet(data, name=, **style)</code>	GeoParquet 데이터셋(URL 또는 로컬 파일)을 추가합니다.
<code>add_flatgeobuf(data, name=, **style)</code>	FlatGeobuf 데이터셋(URL 또는 로컬 파일)을 추가합니다.
<code>add_shp(data, name=, **style)</code>	Shapefile(압축된 URL 또는 로컬 .shp)을 추가합니다.

메서드	설명
<code>add_kml(data, name=, **style) / add_gpkg(data, name=, layer=None, **style)</code>	KML/KMZ 또는 GeoPackage 데이터를 추가합니다.
<code>add_polyline(polyline, name="Polyline", precision=5, unescape=False, **style)</code>	문자열 또는 문자열 목록에서 Encoded Polyline 레이어를 추가합니다(정밀도 5 또는 6).
<code>add_vector_tiles(url, name=, source_layers=, source_layer=, **style)</code>	TileJSON 엔드포인트에서 벡터 타일 레이어를 추가합니다.
<code>add_pmtiles(url, name=, tile_type=, source_layers=, **style)</code>	PMTiles 아카이브(벡터 또는 래스터)를 추가합니다.
<code>add_tile_layer(url, name=, tile_size=, attribution=, bounds=, **style)</code>	래스터 XYZ 타일 레이어를 추가합니다.
<code>add_ee_layer(ee_object, vis_params=, name=, shown=, opacity=)</code>	인증된 Google Earth Engine 객체를 래스터 타일로 추가합니다(<code>earthengine-api</code> 필요).
<code>add_wms(endpoint, layers, name=, styles=, image_format=, transparent=, tile_size=, version=, crs=, bounds=, **style)</code>	WMS 레이어(GetMap, 타일 래스터)를 추가합니다. <code>bounds</code> 는 <code>[west, south, east, north]</code> 이며 레이어로 줌하는 데 필요합니다. <code>crs</code> 의 기본값은 EPSG:3857 입니다. Web Mercator를 지원하지 않는 서버의 경우 서버가 나열하는 CRS를 전달하세요. 가급적 지리 좌표계(EPSG:4326 , EPSG:4258 , EPSG:6706 , 또는 <code>version="1.3.0"</code> 과 함께 CRS:84)를 사용하고, 그렇지 않으면 EPSG:25832 같은 투영된 EPSG:<code>를 사용하세요. 데스크톱 앱은 자체 EPSG 테이블을 이용해 이를 워핑합니다. 해당 타일을 Web Mercator로 다시 그리는 것은 데스크톱 앱뿐입니다. 웹 빌드와 <code>to_html</code> 페이지는 여전히 Web Mercator BBOX를 전송하므로 그런 레이어는 그곳에서 빈 채로 남습니다.
<code>add_wmts(url, name=, tile_size=, bounds=, **style)</code>	타일 URL 템플릿에서 WMTS 레이어를 추가합니다.

메서드	설명
<code>add_wfs(endpoint, type_name, name=, version=, output_format=, srs_name=, max_features=, **style)</code>	WFS 레이어(GetFeature GeoJSON, 가져와서 인라인)를 추가합니다.
<code>add_cog(url, name=, bands=, colormap=, rescale=, **style)</code>	Cloud Optimized GeoTIFF(URL 또는 커널 측 로컬 GeoTIFF 경로)를 추가합니다.
<code>add_raster(source, name=, url=, bands=, colormap=, rescale=, array_args=, **style)</code>	COG/GeoTIFF URL 또는 경로, 또는 xarray DataArray/Dataset을 추가합니다(xarray에는 <code>geolibre[raster]</code> 필요).
<code>add_lidar(url, name=None, **style)</code>	URL로 LAS, LAZ, COPC 또는 EPT 포인트 클라우드를 추가합니다(COPC/EPT는 세부 수준별로 스트리밍).
<code>point_cloud_annotatons()</code>	앱의 포인트 클라우드 어노테이터 가 저장한 포인트 라벨, 인스턴스 id, 사용자 정의 클래스, 3D 박스 및 3D 벡터를 읽습니다. 라벨과 인스턴스 id는 소스 URL, 그다음 노드 키와 포인트 인덱스로 키가 지정됩니다. <code>geolibre.project.apply_point_labels</code> 는 전체 파일 소스의 라벨을 <code>laspy</code> 분류에 씁니다.
<code>prelabel_point_cloud(url, input_file, tool="ground", only_unclassified=True)</code>	LiDAR 레이어의 로컬 복사본에 어노테이터의 Whitebox 사전 라벨링을 실행하고 변경된 클래스를 라벨로 저장합니다(<code>ground</code> 또는 <code>ground-vegetation</code> ; <code>geolibre[pointcloud]</code> 필요).
<code>write_labeled_point_cloud(url, input_file, output_file)</code>	저장된 라벨과 인스턴스 id를 적용한 LiDAR 레이어의 LAS/LAZ/COPC 파일 로컬 복사본을 씁니다. 메모리보다 큰 파일은 스트리밍합니다(<code>geolibre[pointcloud]</code> 필요).
<code>set_point_cloud_classes(classes)</code>	어노테이터의 사용자 정의 클래스를 정의합니다. 예: <code>[{"code": 64, "name": "Car", "color": "#e11d48"}]</code> (코드 19-255). LiDAR 레이어는 이를 해당 색상으로 그리고 범례에 이름을 표시합니다.
<code>add_3d_tiles(url=None, name=, ion_asset_id=, altitude_offset=, request_headers=, **style)</code>	3D Tiles <code>tileset.json</code> URL 또는 asset id로 Cesium Ion 타일셋을 추가합니다(3D 지구본 전용).
<code>add_cesium_ion(asset_id, name=, kind="3d-tiles", altitude_offset=, **style)</code>	id로 Cesium Ion 자산을 추가합니다. 3D Tiles 타일셋 또는 (<code>kind="imagery"</code>) 영상 레이어입니다. 앱의 Ion 토큰으로 3D 지구본에 렌더링됩니다.

메서드	설명
<code>add_czml(url=None, name=, data=, source_path=, **style)</code>	URL 또는 인라인 패킷으로 CZML(Cesium Language) 동적 3D 장면을 추가합니다. 궤도, 차량 트랙, 움직이는 모델 등입니다. 문서의 시계를 따르는 3D 지구본에 렌더링됩니다.
<code>add_cesium_kml(url=None, name=, data=, source_path=, **style)</code>	문서 스타일과 오버레이를 포함한 네이티브 KML/KMZ를 지구본에 추가합니다. URL, 인라인 XML 또는 KMZ 데이터 URL을 제공하세요. 벡터 변환에는 <code>add_kml</code> 을 사용하세요.
<code>add_video(urls, coordinates, name=, **style)</code>	지오레퍼런스된 비디오(네 개의 <code>[lng, lat]</code> 모서리)를 추가합니다.
<code>add_basemap(basemap) / set_basemap(basemap)</code>	배경 베이스맵을 설정합니다.
<code>split_map(left_layers=None, right_layers=None, orientation=, position=, control_position=)</code>	두 레이어 집합 사이에 스와이프(분할 지도) 비교 슬라이더를 추가합니다.
<code>set_map_legend(title=None, position=, group_by_layer=, visible=, collapsed=)</code>	지도 범례(Controls → Legend)를 표시합니다. 범례 행은 보이는 각 레이어의 심볼로지에서 가져옵니다. 지도당 하나이며, 다시 호출하면 갱신됩니다.
<code>add_legend(title=None, legend_dict=, labels=, colors=, builtin=, position=, shape=)</code>	<code>{label: color}</code> dict, 병렬 <code>labels / colors</code> , 또는 builtin 프리셋("nlcd", "esa_worldcover")에서 범례를 추가합니다.
<code>add_colorbar(colormap=, vmin=, vmax=, label=, units=, colors=, orientation=, position=)</code>	이름 있는 컬러맵 또는 사용자 정의 <code>colors</code> 로 연속 래스터용 컬러바를 추가합니다.
<code>add_colormap(colormap=, vmin=, vmax=, label=, **kwargs)</code>	이름 있는 컬러맵에서 컬러바를 추가합니다(<code>add_colorbar</code> 의 leafmap 스타일 별칭).
<code>set_center(lng, lat, zoom=None)</code>	지도의 중심(그리고 선택적으로 줌)을 설정합니다.

메서드	설명
<code>set_center_zoom(lng, lat, zoom=None)</code>	<code>set_center</code> 의 별칭입니다(leafmap 호환).
<code>set_zoom(zoom) /</code> <code>set_bearing(bearing) /</code> <code>set_pitch(pitch) /</code> <code>fit_project_bounds(bounds)</code>	위젯을 표시하지 않고도 카메라 변경을 저장합니다.
<code>center / zoom /</code> <code>bearing / pitch /</code> <code>basemap / name</code>	저장된 프로젝트와 카메라 상태를 읽습니다. <code>name</code> 은 쓰기 가능합니다.
<code>rename_layer(layer, name) /</code> <code>move_layer(layer, index) /</code> <code>duplicate_layer(layer, name=) /</code> <code>show_layer(layer) /</code> <code>hide_layer(layer)</code>	<code>id</code> , 이름 또는 <code>Layer</code> 핸들로 레이어를 관리합니다.
<code>layer_properties(layer) /</code> <code>column_values(layer, column) / describe()</code>	브라우저 왕복 없이 인라인 데이터를 조회하고 프로젝트를 요약합니다.
<code>remove_layer(layer_id) / clear_layers()</code>	<code>id</code> , 이름 또는 핸들로 레이어 하나를 제거하거나 모든 레이어를 제거합니다.
<code>set_popup(layer, fields=None, click=, hover=, title=, title_expression=, body_expression=, show_feature_id=, max_width=, image_height=, tooltip=, merge=False)</code>	레이어의 클릭 팝업에 무엇을 표시할지, 그리고 각 값을 어떻게 서식화할지 선택합니다.
<code>set_tooltip(layer, fields=True) /</code> <code>clear_popup(layer)</code>	호버 툴팁을 켜거나 끄거나, 팝업 설정을 삭제하고 기본 팝업을 복원합니다.

메서드	설명
<code>set_layer_metadata(layer, merge=False, title=, abstract=, keywords=, license=, attribution=, contact=, lineage=, temporal_extent=, links=) / clear_layer_metadata(layer)</code>	카탈로그용으로 레이어를 설명합니다(앱의 Metadata 대화 상자, STAC Item 내보내기 및 GeoParquet 내보내기 메타데이터). 이메일, 날짜(ISO 8601) 및 링크 URL은 검증됩니다. <code>Layer.descriptive_metadata</code> 로 다시 읽을 수 있습니다.
<code>set_layer_filter(layer, expression)</code>	불리언 MapLibre 표현식과 일치하지 않는 레이어의 피처를 숨깁니다. 예: <code>[">=", ["get", "pop"], 100000]</code> (Select by Expression → Filter layer가 저장하는 필터). <code>None</code> 은 필터를 지웁니다.
<code>set_labels(layer, field=None, expression=None, enabled=None, **options)</code>	속성 또는 텍스트 표현식으로 피처에 라벨을 붙입니다. 옵션: <code>placement</code> , <code>size</code> , <code>color</code> , <code>halo_color</code> , <code>halo_width</code> , <code>min_zoom</code> , <code>max_zoom</code> , <code>allow_overlap</code> , <code>anchor</code> , <code>offset_x</code> , <code>offset_y</code> , <code>rotation</code> , <code>max_width</code> , <code>transform</code> , <code>number_format</code> , <code>number_decimals</code> , <code>number_locale</code> , <code>dedupe</code> , 그리고 데이터 정의 옵션인 <code>size_expression</code> , <code>color_expression</code> , <code>opacity_expression</code> , <code>visibility_expression</code> , <code>priority_expression</code> . 생략한 설정은 기존 값을 유지합니다.
<code>set_plugin_state(plugin_id, state=None, position=, activate=True, allow_unknown=False, clear=False)</code>	<code>geolibre.project.PLUGIN_STATE_IDS</code> 에 있는 내장 id, 또는 <code>allow_unknown=True</code> 와 함께 외부 플러그인 id에 대해 플러그인의 저장된 프로젝트 상태(열 때 복원하는 블록)를 저장합니다. <code>state</code> 는 순수 JSON이어야 하며, 생략하면 <code>position / activate</code> 만 변경합니다. <code>clear=True</code> 는 저장된 상태를 제거하되 활성화 여부는 그대로 둡니다.
<code>set_story_map(**settings)</code>	스토리 맵의 <code>title</code> , <code>subtitle</code> , <code>byline</code> , <code>footer</code> , <code>theme</code> , <code>show_markers</code> , <code>marker_color</code> , <code>inset</code> , <code>inset_position</code> , <code>hide_chapter_nav</code> , <code>start_slide</code> , <code>end_slide</code> 를 설정합니다.
<code>add_story_chapter(title, description=, center=, zoom=, pitch=, bearing=, image=, alignment=, hidden=, map_animation=, rotate_animation=, on_enter=, on_exit=, index=)</code>	스토리 챕터를 추가합니다. 생략한 카메라 값은 저장된 뷰에서 가져옵니다. <code>on_enter / on_exit</code> 는 레이어를 페이드합니다: <code>[{"layer": "Cities", "opacity": 1, "duration": 800}]</code> .
<code>move_story_chapter(chapter, index) / remove_story_chapter(chapter)</code>	id, 제목 또는 0 기반 인덱스로 챕터의 순서를 바꾸거나 삭제합니다.

메서드	설명
<code>add_bookmark(name, center=, zoom=, pitch=, bearing=, folder=, visible_layers=)</code>	뷰를 Bookmarks 패널에 저장합니다. 생략한 카메라 값은 저장된 뷰에서 가져옵니다. <code>folder</code> 는 폴더 id 또는 이름입니다(새 이름이면 폴더를 생성). <code>visible_layers</code> 는 북마크를 열 때 표시할 레이어(id, 이름 또는 핸들)를 나열합니다.
<code>remove_bookmark(bookmark)</code>	id, 이름 또는 0 기반 인덱스로 북마크를 삭제합니다.
<code>set_identify(layer="all")</code>	클릭 시 팝업이 열리도록 Identify 도구를 활성화합니다. 레이어 하나(id, 이름 또는 핸들), 레이어 목록, 보이는 모든 레이어("all"), 또는 끄기(None)를 지정합니다.
<code>show_control(name, visible=True) / hide_control(name)</code>	툴바 패널(<code>bookmark</code> , <code>search</code> , <code>measure</code> , <code>minimap</code> , <code>print</code>) 또는 내장 지도 컨트롤(<code>navigation</code> , <code>fullscreen</code> , <code>compass</code> , <code>geolocate</code> , <code>globe</code> , <code>scale</code> , <code>attribution</code> , <code>logo</code>)을 표시하거나 숨깁니다.
<code>set_projection(projection) / projection</code>	지도를 "globe" (기본값) 또는 평면 "mercator" 지도로 그림니다. 프로젝트에 저장됩니다.
<code>to_project(keep_credentials=False)</code>	현재 프로젝트를 dict로 반환합니다. <code>keep_credentials=True</code> 가 아니면 자격 증명이 제거됩니다.
<code>load_project(src)</code>	dict, JSON 문자열 또는 <code>.geolibre.json</code> 경로에서 프로젝트를 교체합니다.
<code>save_project(path, keep_credentials=False)</code>	현재 프로젝트를 <code>.geolibre.json</code> 파일에 씁니다. <code>keep_credentials=True</code> 가 아니면 자격 증명이 제거됩니다.

스타일 키워드 인수(예: `fillColor` , `strokeColor` , `strokeWidth` , `circleRadius`)는 GeoLibre 레이어 스타일 필드에 대응합니다.

마커 심볼로지

`add_marker` 와 `add_markers` 는 일반적인 포인트 심볼로지 설정을 이름 있는 인수로 받으므로, 기반이 되는 스타일 키를 알 필요가 없습니다.

```
m.add_markers(points, color="#e11d48", radius=8, stroke_color="#ffffff", stroke_width=2)
m.add_markers(points, shape="pin", color="#e11d48", size=32)
m.add_markers(points, icon='<svg viewBox="0 0 24 24">...</svg>', size=28)
```

포인트 레이어는 두 가지 방식으로 그려집니다. 기본적으로는 `radius` 로 크기가 정해지는 MapLibre 원(circle)입니다. `shape` , `size` 또는 `icon` 을 전달하면 마커 스프라이트로 전환됩니다. `circle` , `square` , `triangle` , `diamond` , `star` , `cross` , `pin` , 또는 `custom` (`icon` , 즉 원시 SVG 마크업이나 데이터 URL이 필요) 중 하나입니다.

두 모드는 서로 다른 설정을 받으며, 이를 섞으면 조용히 무시되는 대신 오류가 발생합니다. 스프라이트 레이어는 원 레이어를 완전히 대체하고 자체 흰색 헤일로로 그리므로, `opacity`, `radius`, `stroke_color`, `stroke_width` 는 원 전용이며 `shape / size / icon` 이 함께 주어지면 거부됩니다. 스프라이트의 크기에는 `size` 를 사용하세요. 스프라이트의 `color` 는 16진수 색상이어야 합니다. 스프라이트 베이커는 다른 형식을 받지 않으며, 그렇지 않으면 아무 알림 없이 기본 파란색으로 대체되기 때문입니다.

이름 있는 인수와 그 기반 스타일 키가 둘 다 전달되면 (`add_markers(pts, radius=8, circleRadius=20)`) 원시 스타일 키가 우선합니다. 이는 저수준 탈출구입니다.

팝업과 툴팁

아무 설정도 하지 않으면 Identify가 활성화된 상태에서 피처를 클릭했을 때 레이어 이름과 보이는 모든 속성이 표시되며, 호버 툴팁은 없습니다. 스타일 오버라이드를 받는 모든 `add_*` 메서드는 이를 변경하기 위한 `popup=` 과 `tooltip=` 을 받습니다(예외는 고정된 시그니처를 가진 `add_ee_layer` 입니다). 그리고 `set_popup / set_tooltip / clear_popup` 은 이미 존재하는 레이어의 설정을 변경합니다.

```
m.add_markers(
    points,
    popup={
        "title": "name",          # heading, instead of the layer name
        "fields": [
            {"field": "name", "label": "Site"},
            {"field": "photo", "kind": "image", "label": "Photo"},
            {"field": "url", "kind": "link", "link_label": "Read more"},
            {"field": "pop", "kind": "number", "thousands": True, "suffix": " people"},
            {"field": "surveyed", "kind": "date", "date_format": "datetime"},
        ],
    },
    tooltip="name",
)

m.set_popup("Sites", ["name", "pop"], title="name") # replace the config
m.set_tooltip("Sites", ["name"])                  # add a hover tip
m.set_popup("Sites", click=False)                  # no popup on click
m.clear_popup("Sites")                             # back to the default
```

팝업은 Identify 도구가 활성화된 동안에만 열립니다. 앱에서는 레이어의 Identify 버튼이나 Layers 패널 헤더의 "Identify visible layers" 버튼이 이에 해당합니다. `set_identify` 는 Python에서 이를 활성화하며, 지도가 표시되기 전에도 실행할 수 있습니다. Identify는 레이어 하나, 레이어 목록 또는 보이는 모든 레이어를 대상으로 하며, 활성화된 동안 호버 툴팁은 일시 중지됩니다.

```
m.set_identify()          # every visible layer
m.set_identify("Sites")   # one layer
m.set_identify(["Sites", "Roads"]) # only these layers
m.set_identify(None)      # off

m.show_control("bookmark") # open the Bookmarks panel
m.show_control("search")  # open the place search box
m.hide_control("globe")   # hide the globe/flat toggle button
m.set_projection("mercator") # draw a flat map instead of a globe
```

`save_project`, `to_project`, `to_html`은 Identify 대상과 컨트롤을 프로젝트의 `interaction` 블록에 저장하므로, 저장된 파일이나 HTML 페이지가 같은 방식으로 열리며, `load_project`는 이를 다시 읽습니다.

`set_projection`은 프로젝트 환경 설정으로 저장됩니다.

`popup=`은 더 짧은 형식도 받습니다. 단일 속성 이름(`popup="name"`), 이름 목록(`popup=["name", "pop"]`), 필드 매핑 목록, 또는 클릭 팝업을 억제하는 `False`입니다. 위의 전체 매핑 형식은 `set_popup`과 동일한 키(`fields`, `click`, `hover`, `title`, `title_expression`, `body_expression`, `show_feature_id`, `max_width`, `image_height`, `tooltip`)를 받으며, 알 수 없는 키는 거부하므로 철자 오류는 조용히 아무 효과도 없는 설정이 아니라 오류가 됩니다. 이 키들은 `popup=` 안에 속합니다. `add_markers`에 직접 전달하면 스타일 키로 간주됩니다. 두 가지 크기 설정은 예외입니다. `popup_max_width=`와 `popup_image_height=`는 모든 `add_*` 메서드에서 최상위 인수로도 동작하므로, 팝업을 더 넓게 만드는 것만이 유일한 변경 사항일 때 유용합니다. `tooltip=`은 속성 이름, 이름 목록, 설정된 모든 팝업 필드를 툴팁에 넣는 `True`, 또는 끄는 `False`를 받습니다.

필드의 `kind`는 값이 어떻게 렌더링될지 결정합니다.

kind	렌더링 방식
auto (기본값)	텍스트. 단, 인라인 base64 래스터 데이터 URL은 썸네일이 됩니다.
text	<code>prefix / suffix</code> 가 적용된 텍스트.
number	지역화된 숫자. <code>decimals</code> , <code>thousands</code> , <code>prefix</code> , <code>suffix</code> .
date	지역화된 날짜. <code>date_format</code> 은 <code>date</code> , <code>datetime</code> , <code>time</code> , <code>iso</code> 또는 <code>year</code> 입니다.
link	<code>http(s)</code> 값은 <code>link_label</code> 로 라벨이 붙은 링크가 됩니다.
image	<code>http(s)</code> 값 또는 인라인 base64 래스터 데이터 URL은 썸네일이 됩니다.

팝업과 그림의 크기 조정

팝업은 기본적으로 너비 520px(그림을 포함할 때는 420px이며, 드래그해서 더 넓힐 수 있음)이고, 팝업 안의 그림은 최대 `min(50vh, 420px)` 높이로 그려집니다. `max_width`와 `image_height`는 이 둘을 CSS 픽셀 단위로 변경합니다.

```
m.add_markers(
    sites,
    shape="pin", color="#e11d48", size=32,
    popup={
        "title": "name",
        "max_width": 640,          # 288-1200; the viewport still caps it
        "image_height": 420,      # 40-1200
        "fields": [
            {"field": "photo", "kind": "image", "label": "Photo"},
            {"field": "url", "kind": "link", "link_label": "Read more"},
        ],
    },
    tooltip="name",
)

# The same two settings as top-level arguments, on any add_* method:
m.add_markers(sites, popup=["name", "photo"], popup_max_width=640, popup_image_height=420)
```

```
# Or on a layer that already exists (merge=True keeps the fields):
m.set_popup("Sites", max_width=640, image_height=420, merge=True)
```

썸네일은 종횡비를 유지하므로, 가로 사진이 추가 높이를 활용하려면 먼저 **너비**가 커져야 합니다. `image_height` 와 함께 `max_width` 도 올리세요. 앱이 렌더링하는 범위를 벗어나는 크기는 지도에서 조용히 잘리는 값이 아니라 오류가 되며, 팝업 자체의 크기와 상관없이 그림을 클릭하면 여전히 라이트박스에서 원본 크기로 열립니다.

다른 라이브러리에서 팝업을 이식하기 전에 알아두어야 할 두 가지 규칙이 있습니다.

- **속성에 들어 있는 원시 HTML은 마크업으로 렌더링되지 않으며, Markdown도 마찬가지입니다.** GeoJSON 파일에서 온 팝업 값은 신뢰할 수 없으므로 파싱되지 않고 텍스트로 기록됩니다. 그림과 링크에는 `kind="image"` 와 `kind="link"` 를 사용하고, 테이블 대신 구성된 문장을 원할 때는 `body_expression` ([MapLibre 표현식](#), JSON 텍스트 형식)을 사용하세요.
`body_expression='["concat", ["get", "name"], " - ", ["get", "county"], " County"]'`. 유일한 예외는 KML `description` 속성으로, 알려진 마크업은 정제(sanitize)되어 렌더링되므로 변환된 KML은 설명 카드를 유지합니다.
- **툴팁에는 호버용으로 표시된 필드가 필요하며, 필드를 표시하면 클릭 팝업이 좁아집니다.** 둘은 하나의 필드 목록을 공유합니다. 툴팁은 호버용으로 표시된 항목을 보여주고, 클릭 팝업은 모든 항목을 보여줍니다. 하지만 "보이는 모든 속성"으로의 폴백은 그 목록이 비어 있는 동안에만 일어납니다. 따라서

```
m.add_markers(points, tooltip="name") # click popup now shows ONLY name
```

이는 툴팁 필드를 지정하는 것이 목록을 생성하기 때문입니다. 전체 클릭 팝업을 유지하려면 클릭 시 원하는 필드도 함께 나열하고, 그중 하나를 호버용으로 표시하세요.

```
m.add_markers(points, popup=["name", "pop", "county"], tooltip="name")
```

아무것도 표시할 수 없는 툴팁은 조용히 나타나지 않는 툴팁이 아니라 오류가 됩니다. 유일하게 표시된 필드가 `image` 인 경우도 포함되는데, 이미지는 툴팁에서 제외되기 때문입니다(그 값은 URL이며, 툴팁 전체가 URL로 채워질 것입니다). 클릭 팝업은 여전히 그림을 표시합니다.

marimo에서 사용하기

`marimo`는 GeoLibre의 `anywidget`을 렌더링할 수 있지만, 그 브라우저가 GeoLibre가 번들 앱을 제공하는 임의의 127.0.0.1 포트에 접근하지 못할 수 있습니다. 증상은 `iframe에 127.0.0.1 refused to connect`가 표시되는 것입니다. 위젯을 표시하기 전에 GeoLibre의 호스팅된 앱을 가리키도록 설정하세요.

```
from geolibre import Map

m = Map(center=(-100, 40), zoom=4)
m._app_url = "https://web.geolibre.app/"
m.add_basemap("dark")
m.add_vector(
    "https://data.source.coop/giswqs/opengeos/world_cities.geojson",
    name="World cities",
```

```
)
m
```

이 방식은 일반 위젯과 동일한 프로젝트 동기화 브리지를 사용하며, 앱의 위치만 달라집니다. 셀에서 `m` 을 반환하기 전에 `_app_url` 을 설정해야 `iframe`이 첫 렌더링부터 호스팅된 URL을 사용합니다.

이 예제는 호스팅된 브라우저 앱이 원격 GeoJSON을 직접 가져오도록 `add_vector()` 를 사용합니다. 반면 `add_geojson(url)` 은 Python에서 파일을 다운로드해 인라인하는데, 데이터 호스트가 Python의 HTTP 클라이언트를 거부하면 실패할 수 있습니다.

호스팅된 앱은 커널의 임시 `localhost` 서버가 노출하는 파일에 접근할 수 없으므로, 래스터와 그 밖에 브라우저가 직접 로드하는 소스에는 호스팅된 URL을 사용하세요. GeoLibre가 Python에서 읽어 프로젝트에 인라인하는 로컬 GeoJSON, CSV 및 벡터 파일은 계속 동작합니다. `_app_url` 속성은 현재 공개 생성자 옵션이 아니라 내부 호환성 우회 수단입니다.

개인정보 보호: 위젯은 인라인된 로컬 데이터를 포함한 동기화된 프로젝트를 `window.postMessage` 를 통해 `_app_url` 의 오리진으로 전송합니다. 민감한 데이터를 다룰 때는 신뢰할 수 있는 앱 URL만 사용하거나 GeoLibre 앱을 직접 호스팅하세요.

동작 원리

휠(wheel)에는 GeoLibre 웹 빌드가 번들되어 있습니다. 임포트 시점에 패키지는 번들 앱을 제공하는 작은 `localhost` 정적 서버를 시작하고, 위젯은 그 앱을 `iframe` 안에 렌더링하며 `window.postMessage` 를 통해 프로젝트를 주고받습니다. Python에서 데이터를 추가하면 동기화된 프로젝트가 다시 작성되어 앱으로 푸시되고, UI 편집은 같은 경로로 되돌아옵니다.

환경 지원

인터랙티브 위젯은 로컬 Jupyter, VS Code, Google Colab, JupyterHub / 원격 서버, marimo에서 동작합니다.

- **로컬 Jupyter / VS Code** - 앱이 localhost에서 직접 제공됩니다.
- **Google Colab** - Colab의 내장 포트 프록시 (`google.colab.kernel.proxyPort`)를 통해 자동으로 라우팅됩니다.
- **JupyterHub**(관리형/공유 허브 포함, 런타임에 `JUPYTERHUB_SERVICE_PREFIX` 로 감지) - 프론트엔드가 두 개의 동일 오리진 경로를 탐색하고 동작하는 쪽을 사용하므로, 호스트에는 둘 중 **하나**만 있으면 됩니다.
 - `geolibre` 에 번들된 Jupyter Server 확장. 노트북 서버 자체 오리진의 `{base_url}geolibre/app/` 에 마운트됩니다. `pip install geolibre` 시 자동으로 활성화되며 `jupyter-server-proxy` 도 추가 포트도 필요하지 않으므로, 원시 포트 프록시를 차단하는 잠긴 허브에서도 동작합니다. 다만 시작 구성 드롭인에서 로드되므로 Jupyter 서버가 재시작된 후에만 등록됩니다.
 - `{base_url}proxy/{port}/` 의 `jupyter-server-proxy` . `jupyter-server-proxy` 가 설치된 곳이 라면 재시작 없이 **실행 중인** 서버에서 커널의 localhost 번들에 접근합니다.
- **기타 원격 서버**(Binder, SSH/네트워크를 통한 원격 JupyterLab) - `Map(server_proxy=True)` 를 전달하여 동일한 이중 경로 원격 방식을 사용하세요.
- **marimo** - `marimo`에서 **사용하기**에 나온 호스팅된 앱 URL을 사용하세요. Jupyter의 프록시와 서버 확장 경로는 `marimo`에서 사용할 수 없습니다.

직접 localhost 경로를 강제하려면 `Map(server_proxy=False)` 를 설정하세요. 허브에서 앱 로드에 실패하면 `jupyter-server-proxy` 를 설치하거나, `jupyter server extension list` 로 확장이 활성화되어 있는지 확인하고 (`geolibre` 를 찾으세요. 없으면 `jupyter server extension enable geolibre` 를 실행) 확장이 로드되도록 Jupyter 서버를 **재시작**하세요.

URL 가져오기

`add_geojson(url)` , `add_csv(url)` / `add_xy_data(url)` , `add_wfs()` 는 리디렉션을 따라가며 **커널**에서 URL 을 가져오므로, 노트북은 커널이 접근할 수 있는 모든 호스트에 접근할 수 있습니다. 모든 흠이 검사되며, 비공개 주소 (사설, 루프백 또는 링크 로컬 — `169.254.169.254` 같은 클라우드 메타데이터 엔드포인트 포함)로 해석되는 URL은 거부됩니다. 응답은 50 MB로 제한됩니다. 타일 및 서비스 레이어는 대신 **브라우저**가 가져오므로 여전히 로컬 서버를 가리킬 수 있습니다. 공유/멀티테넌트 커널에서는 신뢰할 수 없는 `.geolibre.json` 프로젝트를 URL을 로드하지 마세요.

MCP 서버

같은 패키지에는 노트북이나 실행 중인 앱 없이 AI 클라이언트에서 `.geolibre.json` 프로젝트를 작성하는 **MCP** 서버도 포함되어 있습니다.

```
pip install "geolibre[mcp]"
geolibre-mcp --root ~/maps
```

이 서버는 이 패키지가 사용하는 것과 동일한 빌더를 통해 프로젝트를 만들므로, 서버가 작성한 모든 것은 위젯(그리고 데스크톱 및 웹 앱)에서 변경 없이 열립니다. 도구 목록과 클라이언트 구성은 [MCP 서버](#)를 참조하세요.

소스에서 빌드하기

패키지는 [python/](#)에 있습니다. 번들 앱은 모노레포에서 다음과 같이 생성됩니다.

```
npm run build:embed      # builds the app and stages it into the wheel
python -m build          # builds the wheel
python -m twine upload dist/* # upload to PyPI
pip install -e python    # editable install for development
```

Python 코드 변경 사항은 커널 재시작 시 반영됩니다. 앱(TypeScript) 변경 사항은 `npm run build:embed`를 다시 실행하고 커널을 재시작해야 합니다.

렌더링 엔진과 혼합 창(pane) 레이아웃

```
m = Map(renderer="cesium", center=(-100, 40), zoom=4)
m.set_map_layout(1, 2, view_kinds=["cesium", "maplibre"], sync_view=True)
pane_id = m.project["secondaryMapView"][0]["id"]
m.set_renderer("cesium", pane_id=pane_id)
assert m.get_renderer() == "cesium"
```

렌더러 선택지는 `"maplibre"`, `"mapbox"`, `"cesium"`, `"arcgis"`입니다. `pane_id`를 생략하면 기본 지도를 대상으로 합니다. 그리드 크기는 1-4이며, `view_kinds`는 창마다 렌더러 하나를 담되 기본 지도가 맨 앞에 옵니다. 기존 창 ID, 카메라, 가시성 오버라이드는 레이아웃 크기를 조정해도 유지됩니다. `primaryRenderer`와 각 보조 창의 `viewKind`를 보존하려면 프로젝트를 평소처럼 저장하세요. `DashMap(renderer="cesium")`은 동일한 초기 렌더러를 선택하며, Dash 콜백은 `project`를 통해 이 필드들을 갱신할 수 있습니다.

<https://geolibre.app/r/>

R 패키지

CRAN 0.2.0 R-CMD-check passing

geolibre R 패키지는 `htmlwidgets` 를 통해 RStudio, Quarto, R Markdown, Shiny에 GeoLibre 지도 전체를 임베드합니다. 표준 `.geolibre.json` 프로젝트를 생성하므로 지도를 R과 GeoLibre 웹 또는 데스크톱 애플리케이션 사이에서 주고받을 수 있습니다.

[R 패키지 웹사이트 보기](#)[인터랙티브 예제 열기](#)[소스 둘러보기](#)

설치

CRAN에서 릴리스 버전을 설치합니다.

```
install.packages("geolibre")
```

또는 GitHub에서 개발 버전을 설치합니다.

```
install.packages("pak")
pak::pak("opengeos/geolibre-r")
```

빠른 시작

GeoJSON에서 인터랙티브 지도를 만들고 초기 카메라를 설정합니다.

```
library(geolibre)

places <- list(
  type = "FeatureCollection",
  features = list(list(
    type = "Feature",
    properties = list(name = "Washington, DC"),
    geometry = list(
      type = "Point",
      coordinates = c(-77.0369, 38.9072)
    )
  ))
)

geolibre(map_only = TRUE) |>
  add_geojson(
    places,
    name = "Places",
    style = list(fillColor = "#dc2626", circleRadius = 8)
  ) |>
  set_view(center = c(-77.0369, 38.9072), zoom = 10)
```

위젯은 RStudio Viewer와 렌더링된 HTML 문서에서 반응형으로 동작합니다. 지도는 호스팅된 GeoLibre 애플리케이션을 로드하므로, `app_url` 이 자체 호스팅 배포를 가리키지 않는 한 보려면 인터넷 연결이 필요합니다.

sf 를 이용한 공간 데이터

`add_sf()` 는 `sf` 객체를 받아 GeoJSON을 브라우저에 전달하기 전에 WGS 84로 변환합니다.

```
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

geolibre() |>
  add_sf(nc, name = "North Carolina counties")
```

`sf` 패키지는 선택 사항입니다. Linux에서는 설치에 GDAL, GEOS, PROJ, UDUNITS 같은 시스템 라이브러리가 필요할 수 있습니다.

원격 래스터

GeoLibre는 원격 Cloud Optimized GeoTIFF를 브라우저에서 직접 읽습니다. 래스터를 호스팅하는 서버는 CORS와 HTTP 범위 요청을 지원해야 합니다.

```
geolibre() |>
  add_raster(
    "https://example.org/visual.tif",
    name = "Satellite image",
    bands = c(1, 2, 3)
  )
```

프로젝트 파일

위젯 상태를 이식 가능한 GeoLibre 프로젝트로 저장하고 나중에 복원합니다.

```
map <- geolibre() |> add_geojson(places)
save_project(map, "example.geolibre.json")

restored <- geolibre(load_project("example.geolibre.json"))
```

저장된 프로젝트는 GeoLibre Web이나 GeoLibre Desktop에서도 열 수 있습니다. Shiny 애플리케이션에서는 브라우저 측 변경 사항을 `input$<outputId>_project` 를 통해 사용할 수 있습니다.

Shiny

지도를 렌더링하려면 `geolibreOutput()` 과 `renderGeolibre()` 를 사용하세요. 프록시는 출력을 다시 빌드하지 않고 라이브 위젯을 갱신합니다.

```
library(shiny)
library(geolibre)

ui <- fluidPage(
  geolibreOutput("map", height = "80vh"),
  actionButton("reset", "Reset view")
)

server <- function(input, output, session) {
  map <- reactiveVal(geolibre(map_only = TRUE))
  output$map <- renderGeolibre(map())

  observeEvent(input$reset, {
    next_map <- set_view(map(), center = c(0, 20), zoom = 2)
    map(next_map)
    update_geolibre(geolibre_proxy("map"), next_map)
  })
}

shinyApp(ui, server)
```

자체 호스팅

위젯 하나 또는 R 세션 전체에 대해 사용자 정의 GeoLibre 애플리케이션 URL을 설정합니다.

```
geolibre(app_url = "https://gis.example.org/")
options(geolibre.app_url = "https://gis.example.org/")
```

배포는 GeoLibre 웹 앱과 그 `?embed=1` 프로젝트 브리지를 노출해야 합니다. 전체 API는 [패키지 참조](#)를 참조하세요.

<https://geolibre.app/mcp/>

MCP 서버

GeoLibre는 GeoLibre 프로젝트를 작성하는 **MCP** 서버를 제공합니다. MCP 클라이언트(Claude Desktop, Claude Code 또는 그 밖의 어떤 클라이언트든)를 이 서버에 연결하면 말로 지도를 요청할 수 있습니다. 서버는 데스크톱 앱, 웹 앱, 또는 `geolibre` Jupyter 위젯에서 열 수 있는 실제 `.geolibre.json` 프로젝트를 작성하며, 이를 독립 실행형 HTML 페이지로 내보낼 수도 있습니다.

파일 도구는 **헤드리스**입니다. 브라우저도, 실행 중인 GeoLibre 인스턴스도, 번들된 웹 빌드도 필요하지 않습니다. Python 패키지가 사용하는 것과 동일한 **프로젝트 빌더**로 프로젝트 파일을 만들기 때문에, 이 도구가 작성한 프로젝트는 앱이 이미 로드하는 것과 바이트 단위로 같은 종류입니다.

`live_*` 도구는 나머지 절반입니다. 이 도구들은 데스크톱 Notebook 릴레이(**Notebook 패널**)와 통신하므로, MCP 클라이언트의 명령이 GeoLibre Desktop에 이미 열려 있는 지도를 움직입니다. 앱 실행마다 한 번 **Processing → Jupyter Notebook**(처리 → Jupyter Notebook)을 여세요. 이후에는 패널을 닫아도 됩니다. 웹과 JupyterLite에는 릴레이가 없으므로, 그곳에서는 이 도구들이 연결할 대상이 없습니다.

설치

MCP SDK는 선택적 extra입니다.

```
pip install "geolibre[mcp]"
```

실행하기

```
geolibre-mcp --root ~/maps
```

서버는 데스크톱 클라이언트가 실행하는 방식인 stdio를 통해 MCP로 통신합니다. `--root` 플래그는 반복할 수 있으며, `GEOLIBRE_MCP_ROOTS` (: 로 구분, Windows에서는 ;)가 환경 변수에서 같은 역할을 합니다. 둘 다 설정되지 않으면 워크스페이스는 현재 디렉터리입니다. `--transport` 는 HTTP로 연결하는 클라이언트를 위해 기본값 stdio 에서 `streamable-http` 또는 `sse` 로 전환합니다.

클라이언트 구성

Claude Desktop(`claude_desktop_config.json`)과 대부분의 다른 클라이언트는 같은 형태를 사용합니다.

```
{
  "mcpServers": {
    "geolibre": {
      "command": "geolibre-mcp",
      "args": ["--root", "/Users/you/maps"]
    }
  }
}
```

```
}
}
```

Claude Code의 경우:

```
claude mcp add geolibre -- geolibre-mcp --root ~/maps
```

geolibre-mcp 가 클라이언트의 PATH 에 없다면(virtualenv에 설치한 경우 흔함) 대신 인터프리터를 지정하세요.

```
{
  "mcpServers": {
    "geolibre": {
      "command": "/path/to/venv/bin/python",
      "args": ["-m", "geolibre.mcp", "--root", "/Users/you/maps"]
    }
  }
}
```

워크스페이스

모든 도구 호출의 모든 경로는 서버가 접근하기 전에 허용된 루트를 기준으로 해석되며, 이는 [사이드카](#)의 `GEOLIBRE_CONVERSION_ROOTS` 와 같은 방식입니다. 루트 밖의 경로는 거부되고, 루트 안에 있지만 밖을 가리키는 심볼릭 링크도 거부됩니다. 상대 경로는 첫 번째 루트를 기준으로 해석되므로, 클라이언트는 호스트의 레이아웃을 알지 못해도 `city.geolibre.json` 이라고 말할 수 있습니다.

쓰기에는 세 가지 보호 장치가 더 있습니다. 서버는 `.json` (프로젝트) 또는 `.html` (내보내기)로 끝나는 파일만 씁니다(이름 없는 `.json` 만 있는 경우도 거부됩니다). 호출에 `overwrite` 가 전달되지 않는 한 기존 파일의 교체를 거부하며, 기존 프로젝트를 편집하는 도구는 먼저 그 파일이 실제로 프로젝트인지 확인하므로 루트 안에 있는 관련 없는 `package.json` 이 지도로 덮어써지는 일은 없습니다.

홈 디렉터리가 아니라 지도 전용 디렉터를 지정하세요.

도구

프로젝트 수명 주기

도구	하는 일
<code>create_project</code>	이름, 중심, 줌, 베이스맵을 지정하여 새 빈 프로젝트를 작성합니다.
<code>describe_project</code>	카메라, 베이스맵, 레이어, 지도 컨트롤을 요약합니다. 인라인된 피쳐 데이터는 개수로만 보고되며 그대로 돌려 보내지 않습니다.
<code>get_point_cloud_annotations</code>	포인트 클라우드별로 앱의 주석 도구가 각 클래스로 다시 라벨링한 포인트 수와 각 인스턴스가 보유한 포인트 수, 그리고 사용자 정의 클래스, 3D 벡터, 저장된 3D 박스(상태 및 속성 포함)를 반환합니다.
<code>set_point_cloud_classes</code>	주석 도구의 사용자 정의 클래스(코드 19-255, 이름, <code>#rrggbb</code> 색상)를 정의합니다. 기존 라벨과 박스는 유지됩니다.

도구	하는 일
<code>prelabel_point_cloud</code>	LiDAR 레이어의 로컬 사본에 앱의 Whitebox 사전 라벨링 분류기를 실행하고 변경된 클래스를 그 레이어의 라벨로 저장합니다(<code>geolibre[pointcloud]</code> 필요).
<code>write_labeled_point_cloud</code>	프로젝트의 라벨과 인스턴스 id를 적용한 LiDAR 레이어 LAS/LAZ/COPC 파일의 로컬 사본을 작성합니다(<code>geolibre[pointcloud]</code> 필요).
<code>list_catalog</code>	이름이 지정된 베이스맵, 색상 램프, 범례 프리셋, <code>set_plugin_state</code> 가 허용하는 플러그인 id, 그리고 활성 워크스페이스 루트를 나열합니다.

레이어 추가

도구	용도
<code>add_geojson_layer</code>	URL, 워크스페이스 파일, 또는 리터럴 GeoJSON에서 가져와 프로젝트에 인라인되는 벡터 데이터입니다. 자체 완결적이며, <code>classify_layer</code> 가 스타일링할 수 있는 유일한 종류입니다.
<code>add_vector_layer</code>	제자리에서 읽는 대용량 원격 FlatGeobuf / GeoParquet / GeoJSON입니다.
<code>add_raster_layer</code>	밴드, 컬러맵, 리스케일 옵션이 있는 Cloud Optimized GeoTIFF입니다.
<code>add_tile_layer</code>	래스터 XYZ 타일 템플릿입니다.
<code>add_tiles_layer</code>	PMTiles 아카이브와 벡터 타일 서비스입니다.
<code>add_ogc_layer</code>	WMS 및 WMTS 엔드포인트입니다.
<code>add_lidar_layer</code>	URL로 지정한 LAS/LAZ/COPC/EPT 포인트 클라우드입니다.
<code>add_3d_tiles_layer</code>	URL 또는 Cesium Ion 자산 id로 지정한 OGC 3D Tiles 타일셋입니다.
<code>add_cesium_ion_layer</code>	id로 지정한 Cesium Ion 자산(타일셋 또는 영상)으로, 3D 지구본에서만 렌더링됩니다.
<code>add_czml_layer</code>	URL 또는 인라인 패킷으로 지정한 CZML(Cesium Language) 동적 3D 장면으로, 3D 지구본에서만 렌더링됩니다.
<code>add_cesium_kml_layer</code>	URL, 인라인 XML, 또는 KMZ 데이터 URL에서 가져온 네이티브 지구본 KML/KMZ로, 문서 스타일과 오버레이를 보존합니다.

편집

도구	하는 일
<code>update_layer</code>	이름 변경, 표시/숨김, 불투명도 설정, 순서 변경을 수행합니다.

도구	하는 일
<code>remove_layer</code>	레이어를 제거합니다.
<code>style_layer</code>	스타일 키(<code>fillColor</code> , <code>strokeWidth</code> , <code>circleRadius</code> , ...)를 병합합니다.
<code>set_layer_popup</code>	클릭 팝업에 표시할 필드와 그 라벨 및 형식, 팝업의 너비와 이미지 높이, 그리고 선택적 호버 툴팁을 지정합니다.
<code>set_layer_metadata</code>	카탈로그용으로 레이어를 기술합니다(제목, 요약, 키워드, 라이선스, 출처 표기, 연락처, 계보, 시간 범위, 링크). 앱의 Metadata 대화 상자가 편집하고 STAC Item으로 내보내는 내용입니다.
<code>classify_layer</code>	숫자 열로부터 단계 구분 코로플레스(choropleth)를 만듭니다.
<code>list_layer_properties</code>	레이어의 피쳐 속성을 샘플 값과 함께 나열합니다.
<code>set_layer_filter</code>	불리언 MapLibre 표현식과 일치하지 않는 피쳐를 숨깁니다(Select by Expression → Filter layer가 기록하는 저장된 필터). 표현식을 생략하면 필터가 해제됩니다.
<code>set_labels</code>	속성 또는 텍스트 표현식으로 피쳐에 라벨을 붙이며, 크기, 색상, 헤일로, 줌 범위, 앵커, 그리고 <code>options</code> 아래의 덜 쓰이는 설정을 지정합니다. 생략한 설정은 기존 값을 유지합니다.

레이어는 `id` 또는 표시 이름으로 지정할 수 있으므로, 클라이언트는 UUID를 추적하지 않고도 `describe_project`가 보여준 내용을 바탕으로 작업할 수 있습니다.

프레이밍과 장식

도구	하는 일
<code>set_view</code>	중심, 줌, 방위각(bearing), 기울기(pitch)를 설정하거나, <code>bbox</code> 를 전달하여 영역을 프레이밍합니다.
<code>set_basemap</code>	배경 스타일을 전환합니다.
<code>set_map_legend</code>	레이어 자체의 심볼 체계로 구성된 지도 범례를 표시합니다.
<code>add_legend</code>	프리셋, <code>{label: color}</code> 맵, 또는 쌍을 이루는 목록으로부터 범례를 추가합니다.
<code>add_colorbar</code>	연속형 데이터를 위한 컬러바를 추가합니다.
<code>add_swipe</code>	분할 지도 비교 슬라이더를 구성합니다.

플러그인 상태와 스토리 맵

도구	하는 일
<code>set_plugin_state</code>	플러그인의 저장된 프로젝트 상태(Time Slider의 타임라인처럼 열 때 복원하는 blob)를 저장합니다. 내장 id는 <code>list_catalog</code> 에서 가져오며, 외부 플러그인의 id에는 <code>allow_unknown</code> 이 필요합니다.
<code>set_story_map</code>	스토리 맵의 제목 블록, 테마, 마커, 인셋, 시작/종료 슬라이드를 설정합니다.
<code>add_story_chapter</code>	카메라, 텍스트, 이미지, 정렬, 애니메이션, 레이어 페이드를 포함한 챕터를 추가합니다. 생략한 카메라 값은 프로젝트의 저장된 뷰에서 가져옵니다.

도구	하는 일
<code>move_story_chapter</code> / <code>remove_story_chapter</code>	id, 제목, 또는 인덱스로 챕터의 순서를 바꾸거나 제거합니다.
<code>add_bookmark</code>	저장된 뷰를 Bookmarks 패널에 추가합니다. 선택적으로 폴더에 넣을 수 있고(새 폴더 이름은 폴더를 생성함) 열 때 표시할 레이어를 함께 지정할 수 있습니다. 생략한 카메라 값은 프로젝트의 저장된 뷰에서 가져옵니다.
<code>remove_bookmark</code>	id, 이름, 또는 인덱스로 북마크를 제거합니다.

이 형태들은 앱이 읽는 것(@geolibre/core 의 `parseProject`)과 일치합니다. 왕복 테스트가 이 도구들이 작성한 프로젝트를 그것으로 로드하여 아무것도 누락되거나 다시 작성되지 않았는지 확인합니다.

라이브 데스크톱 지도

도구	하는 일
<code>live_status</code>	GeoLibre Desktop 창이 수신 대기 중인지 보고합니다.
<code>live_list_layers</code>	열린 지도의 레이어를 나열합니다.
<code>live_fly_to</code> / <code>live_fit_bounds</code> / <code>live_zoom_to_layer</code>	카메라를 이동합니다.
<code>live_set_basemap</code>	베이스맵을 전환합니다.
<code>live_add_geojson</code> / <code>live_remove_layer</code>	레이어를 추가하거나 제거합니다.
<code>live_set_visibility</code> / <code>live_set_opacity</code> / <code>live_set_style</code>	레이어를 변경합니다.
<code>live_list_algorithms</code>	앱의 처리 알고리즘과 매개변수를 나열하며, 선택적으로 <code>query</code> 로 필터링합니다.
<code>live_run_algorithm</code>	열린 지도에서 알고리즘을 실행하고, 로그와 추가된 결과 레이어의 id 를 반환합니다.

처리는 라이브 전용입니다. 알고리즘 카탈로그와 이를 실행하는 엔진(Turf.js, DuckDB-WASM)은 앱 안에 있으므로, 파일 도구가 실행할 수 있는 것은 없습니다. 릴레이는 결과를 약 5초 동안 기다립니다. 더 오래 걸리는 실행은 앱에서 계속 진행되며, 도구는 결과 레이어 id 대신 제시간에 끝나지 않았다고 보고하므로, 다시 실행하기 전에 `live_list_layers` 를 확인하세요.

내보내기

`export_html` 은 호스팅된 GeoLibre 뷰어를 임베드하고 그 안에 프로젝트를 주입하는 독립 실행형 페이지를 작성하므로, 받는 사람은 아무것도 설치할 필요가 없습니다. 내보내는 과정에서 프로젝트의 자격 증명은 제거됩니다. 로컬 파일을 가리키는 레이어는 다른 사람에게는 로드되지 않으므로, 공유 가능한 내보내기에는 호스팅된 URL을 사용하세요.

app_url 은 신뢰 경계입니다

export_html 의 선택적 app_url 은 내보낸 페이지가 임베드하는 뷰어를 지정하며, 페이지는 정확히 그 출처로 프로젝트를 전송합니다(http / https URL이어야 합니다). 셀프 호스팅 배포를 고정할 수 있도록 존재하는 옵션입니다.

이를 외관상의 설정이 아니라 목적지로 취급하세요. 내보낸 파일을 여는 사람은 누구든 프로젝트의 내용(인라인된 GeoJSON, 레이어 URL, 카메라)을 그 출처에 넘겨주게 됩니다. 자격 증명은 이미 제거되어 있으므로 키 유출은 아니지만, 프로젝트의 나머지 부분은 여전히 전송됩니다.

여기서 호출자가 모델이라는 점이 중요합니다. 모델은 자신이 읽은 콘텐츠에 따라 행동하고 있을 수 있습니다. 내보낸 페이지가 사용자가 선택하지 않은 곳을 가리킨다면 다시 한 번 살펴볼 가치가 있습니다. 의도한 app_url 만 수락하세요.

참고 사항과 제한

- **bbox 를 사용한 set_view 는 근사치입니다.** 저장된 프로젝트는 중심과 zoom 을 저장하고, 앱은 저장된 bbox에 맞추는 대신 로드 시 그 값을 그대로 적용합니다. 따라서 서버는 가정된 지도 패널 크기를 사용하여 박스를 직접 카메라로 변환하며, 앱 자체의 "레이어로 줌" 기능이 선택할 값에서 대략 반 줌 레벨 이내에 도달합니다. 정확한 프레이밍이 필요하면 center 와 zoom 을 전달하세요.
- **인라인 GeoJSON은 50 MB로 제한되며,** 서버가 읽는 프로젝트 파일은 256 MB로 제한됩니다. 이를 넘으면 add_vector_layer 나 타일 소스를 사용하세요.
- **원격 가져오기는 검사됩니다:** 호스트가 사설, 루프백, 또는 링크 로컬 주소로 해석되는 URL은 첫 요청뿐 아니라 모든 리디렉션 홉에서도 거부되므로, 조작된 URL이 클라우드 메타데이터 엔드포인트에 도달할 수 없습니다.
- **파일 도구는 프로젝트를 작성합니다. live_* 도구는 열린 데스크톱 지도를 움직입니다.** 라이브 편집은 앱에서 저장하기 전까지 세션 안에만 있습니다. 릴레이는 루프백 전용이고 리디렉션을 거부하므로 Jupyter 토큰은 머신 안에 머뭅니다. 같은 스크립팅 표면이 Python 위젯과 임베드 API를 뒷받침하며, 호스트 페이지나 노트북이 MCP 없이 지도를 제어하는 방법은 여전히 그것들입니다.

에이전트에서 서버 제어하기

GeoLibre는 외부 AI 에이전트에게 이 도구들을 잘 사용하는 법을 가르치는 [에이전트 스킬](#)을 제공합니다. 어떤 데이터에 어떤 add_*_layer 도구가 맞는지, 어떤 순서로 호출해야 하는지, 위의 제한 사항은 무엇인지 등을 다룹니다. 저장소의 [skills/geolibre/](#) 에서 설치하세요.

내부 구조

도구들은 프로젝트 dict에 대한 연산(레이어 추가/제거/스타일 변경, 카메라 이동, 지도 컨트롤 구성)을 담은 위젯 독립 모듈인 `geolibre.authoring` 의 얇은 래퍼입니다. `geolibre.Map` 도 같은 모듈에 위임하므로, 노트북 위젯과 MCP 서버가 프로젝트를 만드는 방식에서 서로 어긋날 수 없습니다.

렌더러와 패널 작성

프로젝트를 지구본에서 열려면 `set_renderer(path, "cesium")` 을 사용하세요. `set_map_layout(path, 1, 2, view_kinds=["cesium", "maplibre"])` 는 혼합 그리드를 만들고 보조 패널들을 ID와 함께 `secondaryMapViewes` 로 반환합니다. 그중 하나를 `set_renderer` 에 `pane_id` 로 전달하면 해당 패널만 변경할 수 있습니다. 카메라 도구는 계속해서 경도/위도와 공유되는 줌, 방위각, 기울기 관례를 사용합니다. 허용되는 렌더러 이름은 `maplibre`, `mapbox`, `cesium`, `arcgis` 입니다.

<https://geolibre.app/agent-skill/>

에이전트 스킬

GeoLibre는 **에이전트 스킬**을 함께 제공합니다. 이는 외부 AI 코딩 에이전트(Claude Code, Claude Desktop, 또는 스킬을 로드하는 모든 런타임)가 사용자를 대신해 GeoLibre 지도를 만드는 방법을 가르치는 `SKILL.md` 파일입니다.

저장소의 `skills/geolibre/`에 있습니다.

용도

GeoLibre에는 이미 앱 내 **AI Assistant**가 있어서, 실행 중인 앱 안에서 데이터와 대화하고 앱 자체의 스토어를 통해 변경 사항을 적용합니다. 스킬은 그 반대 방향을 다룹니다. 즉, GeoLibre **바깥**에서 — 터미널, 편집기, 또는 노트북에서 — 작업하는 에이전트가 GeoLibre 지도를 결과물로 만들어 내도록 합니다.

	AI Assistant	에이전트 스킬
실행 위치	앱 안	앱 바깥의 에이전트 안
작업 대상	지금 보고 있는 라이브 지도	디스크의 <code>.geolibre.json</code> 파일
필요 사항	구성된 AI 제공자	스킬을 로드하는 에이전트
출력	Ctrl/Cmd + Z로 되돌릴 수 있는 편집	프로젝트 파일 및/또는 독립 실행형 HTML 페이지

둘은 함께 쓸 수 있습니다. 에이전트가 프로젝트를 작성하면, 그 프로젝트를 GeoLibre에서 열어 Assistant로 작업을 이어갈 수 있습니다. MCP 서버의 `live_*` 도구는 세 번째 경로로, Notebook 릴레이를 통해 Desktop에 이미 열려 있는 지도를 움직입니다. **MCP 서버**를 참조하세요.

가르치는 내용

- **어떤 진입점을 사용할지** — 채팅 세션에는 **MCP 서버**, 노트북과 스크립트에는 **Python 패키지**, 아무것도 설치되지 않았을 때는 직접 작성하는 **프로젝트 JSON**, 페이지 안의 라이브 인스턴스에는 **임베드 API**.
- **작성 워크플로** — 프로젝트 생성, 레이어 추가, 카메라 구도 설정, 스타일 지정 또는 분류, 범례나 컬러바 추가, 공유 가능한 HTML 페이지 내보내기.
- **틀리기 쉬운 규칙들** — `classify_layer`에는 인라인 GeoJSON이 필요하다는 점, 로컬 파일 경로는 수신자에게 전달되지 않는다는 점, `bbox`는 카메라로 근사적으로 변환된다는 점, 베이스맵과 컬러 램프 이름은 추측이 아니라 `list_catalog`에서 가져와야 한다는 점, 그리고 `export_html`의 `app_url`은 신뢰 경계라는 점.

설치

```
git clone --depth 1 https://github.com/opengeos/GeoLibre /tmp/geolibre
mkdir -p ~/.claude/skills
cp -r /tmp/geolibre/skills/geolibre ~/.claude/skills/
```

하나의 저장소에만 적용하려면 `~/.claude/skills/` 대신 프로젝트 안의 `.claude/skills/` 를 사용하세요. 이후 새 에이전트 세션을 시작하면, 요청이 일치할 때 스킬이 스스로 로드됩니다.

가장 빠른 경로를 원한다면, 스킬이 선호하는 MCP 서버를 설치하세요:

```
pip install "geolibre[mcp]"
claude mcp add geolibre -- geolibre-mcp --root ~/maps
```

MCP 서버 없이도 스킬은 동작합니다. 이 경우 Python API와 프로젝트 JSON 직접 작성으로 대체합니다.

다른 에이전트 런타임

`SKILL.md` 는 YAML 프론트매터가 있는 일반 Markdown이므로, 스킬을 지원하는 대부분의 에이전트 런타임이 그대로 로드할 수 있습니다. 사용 중인 런타임에 스킬 로더가 없다면 파일을 시스템 프롬프트나 `AGENTS.md` 에 붙여 넣으세요. 이 파일은 단독으로도 읽히도록 작성되어 있으며, `references/` 파일들은 작업에 필요할 때만 읽힙니다.

<https://geolibre.app/notebook/>

Notebook 패널

GeoLibre는 크기 조절과 접기가 가능한 패널에서 **지도 옆에** Jupyter 노트북을 실행할 수 있습니다(**Processing → Jupyter Notebook**(처리 → Jupyter 노트북)에서 엽니다). 웹과 데스크톱 빌드 모두에서 동작하며, 같은 `<iframe>` 패널 뒤에 서로 다른 Python 런타임이 있습니다.

- **웹 빌드 → JupyterLite.** Pyodide(WebAssembly) 커널 위에서 전적으로 브라우저 안에서 실행되는 완전한 Jupyter UI입니다. 서버가 없습니다. 추가 패키지는 셀에서 `%pip install geopandas`로 필요할 때 설치합니다.
- **데스크톱 빌드 → 실제 JupyterLab 서버.** 앱은 uv가 관리하는 Python 환경(FastAPI 사이드카와 동일한 메커니즘)에서 `127.0.0.1:8766`에 토큰 인증된 `jupyter lab`을 실행하고 이를 임베드합니다. 이를 통해 네이티브 지리공간 스택(geopandas, rasterio, GDAL, ...)을 갖춘 완전한 CPython을 사용할 수 있습니다. Mac App Store 빌드는 예외입니다. 서버를 생성할 수 없으므로 웹 빌드처럼 JupyterLite 사이트를 임베드합니다.

이는 `geolibre Python 패키지`와는 다릅니다. 그 패키지는 반대로 GeoLibre 앱 전체를 노트북 셀 안에 임베드합니다.

셀에서 지도 스크립팅하기

경량 `geolibre` 클라이언트가 노트북 커널에 미리 로드되어 있습니다. 웹에서는 JupyterLite의 파일시스템에 번들되어 있어 `pip install`이 필요 없고, 데스크톱에서는 커널의 임포트 경로에 배치됩니다. 그냥 임포트하면 됩니다.

```
import geolibre

m = geolibre.connect() # or geolibre.Map()
m.fly_to(-122.4, 37.8, zoom=11) # animate the live map in the left pane
layer_id = m.add_geojson(
    gdf,
    name="My layer",
    fillColor="#facc15",
    strokeColor="#d97706",
) # GeoDataFrame, dict, or JSON string; returns an id on desktop
m.get_layer(layer_id)
m.list_layers()
m.fit_bounds([-123, 37, -122, 38])
m.set_basemap("https://.../style.json")
```

대부분의 변경 호출은 **발사 후 망각(fire-and-forget)** 방식입니다. 데스크톱에서 `add_geojson`은 릴레이의 상관된 요청/응답 경로를 사용하여 새 레이어 id를 반환합니다. 같은 경로로 `list_layers()`도 호출되는데, 이는 라이브 레이어마다 `id`, `name`, `type`, `visible`, `opacity`를 담은 dict 하나를 반환하며, `get_layer(layer_id)`는 일치하는 레이어 하나를 반환하거나 `ValueError`를 발생시킵니다. 이 id는 `set_visibility`, `set_opacity`, `set_style`, `remove_layer`, `zoom_to_layer`에 직접 전달할 수 있습니다.

모든 명령은 같은 Jupyter 서버에 연결된 모든 GeoLibre 창으로 전파됩니다. `add_geojson`, `list_layers`, `get_layer` 의 경우 첫 번째 상관된 응답이 노트북에 반환되고 이후 응답은 무시됩니다. 각 창이 자체 지도 상태를 유지하므로, 반환된 레이어 id를 이후의 읽기 호출에 연결할 때는 연결된 창 하나만 사용하세요. 이 동작은 하나의 Jupyter 서버에 여러 GeoLibre 창을 연결한 경우에만 드러납니다.

`add_geojson` 은 두 가지 상황에서 id 대신 `None` 을 반환하며, 어느 경우에도 완전히 실패하지는 않습니다.

- **아무것도 명령을 받지 못한 경우**(연결된 창이 없거나 릴레이 자체에 접근할 수 없음). 다른 모든 변경 호출과 같이 동작하여, 레이어가 `GeoLibreNotConnectedWarning` 과 함께 디스플레이 전송 경로로 나갑니다.
- **창이 명령을 받았지만 5초 안에 응답하지 않은 경우**. 큰 `FeatureCollection` 에서 발생할 수 있습니다. `GeoLibreTimeoutWarning` 이 발생하며, 레이어는 여전히 추가되는 중이므로 의도적으로 다시 보내지 않습니다(두 번 추가될 것이기 때문입니다). `list_layers()` 로 찾으세요.

읽기 호출은 두 상황 모두에서 반환할 것이 없으므로 대신 `GeoLibreNotConnectedError` / `GeoLibreTimeoutError` (둘 다 `RuntimeError`)를 발생시킵니다.

동기식 읽기는 데스크톱 전용입니다. JupyterLite는 브라우저 `postMessage` 를 사용하므로, 그 Python 호출을 블로킹하면 결과를 전달해야 하는 브라우저 이벤트 루프도 블로킹됩니다. 정식 클라이언트 소스: `backend/geolibre_server/notebook_client.py`.

외부 클라이언트(VS Code 등)에서 지도 제어하기

데스크톱 앱의 JupyterLab 서버는 토큰 인증을 사용하는 일반 Jupyter 서버이므로, 어떤 Jupyter 클라이언트든 연결하여 자신의 편집기를 그대로 사용할 수 있으며 위의 지도 명령도 계속 동작합니다. Notebook 패널을 한 번 열고(이것이 서버를 시작합니다), 패널 헤더의 **링크 버튼**으로 연결 URL을 복사하여 클라이언트에 붙여넣으세요(VS Code에서는 Jupyter: Specify Jupyter Server for Connections → Existing).

이것이 가능한 이유는 명령이 노트북이 표시되는 방식에 의존하지 않고 Jupyter 서버의 **릴레이**를 통해 이동하기 때문입니다.

- `backend/geolibre_server/geolibre_server/jupyter_relay.py` 는 `POST .../geolibre/relay/command`, `.../geolibre/relay/socket` WebSocket, `GET .../geolibre/relay/status` 를 노출하는 Jupyter Server 확장(`jupyter_server_config.py` 에서 활성화)입니다. 로드 시 `GEOLIBRE_RELAY_URL` 과 `GEOLIBRE_RELAY_TOKEN` 을 서버 환경에 게시하며, **커널이 이를 상속**합니다. 그래서 `import geolibre` 가 아무 설정 없이 지도를 찾을 수 있습니다.
- 앱은 수명 내내 그 소켓을 구독하며(`useJupyterRelay`) 백오프로 재연결하고, 각 명령을 동일한 `createScriptingHandlers` 인터페이스에 대해 실행합니다.
- `useNotebookBridge` 는 **임베드된** 패널과 웹(JupyterLite)을 위한 `postMessage` 경로로 남아 있습니다. 그 곳에서는 노트북 페이지가 실제로 앱의 `iframe`입니다.

POST는 몇 개의 앱 창이 명령을 받았는지 응답하므로, 연결이 끊긴 세션은 조용히 아무것도 하지 않는 대신 보고됩니다.

```
geolibre.is_connected() # False -> no GeoLibre window is listening
```

명령을 전달할 수 없을 때 클라이언트는 여러분의 코드 줄을 가리키는 `GeoLibreNotConnectedWarning` 을 발생시킵니다. 다음과 같이 오류로 승격할 수 있습니다.

```
import warnings, geolibre
warnings.simplefilter("error", geolibre.GeoLibreNotConnectedWarning)
```

릴레이는 데스크톱 서버에만 있습니다. 웹(JupyterLite)에는 외부 클라이언트를 연결할 서버가 없습니다.

테마

웹 빌드에서는 노트북이 앱의 라이트/다크 테마를 실시간으로 따릅니다(다시 로드 없음). JupyterLite는 `exposeAppInBrowser` 로 빌드되어 (`apps/geolibre-desktop/jupyterlite/jupyter-lite.json` 참조) 앱 객체를 `window.jupyterapp` 에 두는데, JupyterLite iframe이 동일 오리진이므로 `useNotebookThemeSync` 가 앱 테마가 바뀔 때마다 내장 `apputils:change-theme` 명령을 실행합니다. **데스크톱 빌드**에서는 JupyterLab 서버가 다른 오리진이므로 이 교차 오리진 호출은 아무 효과가 없고 노트북은 자체 테마를 유지합니다(그곳에서 동기화하려면 작은 JupyterLab 확장이 필요하며, 이는 향후 개선 사항입니다).

웹 JupyterLite 사이트 빌드하기

웹 빌드는 `apps/geolibre-desktop/public/jupyterlite/` 아래에 자체 호스팅 JupyterLite 사이트를 임베드합니다. 이는 커밋되지 **않으며**(약 70 MB) `jupyter lite` CLI가 필요한 최선 노력(best-effort) 프리빌드 단계에서 생성됩니다.

```
pip install -r apps/geolibre-desktop/jupyterlite/requirements.txt
npm run build:jupyterlite
```

`npm run dev` 와 `npm run build` 모두 이를 자동으로 실행합니다.

- `npm run dev` 는 `predev` 혹(`--if-missing`)을 통해 첫 실행 시 **한 번** 빌드하고, 이후 실행에서는 즉시 시작됩니다.
- `npm run build` 는 변경된 클라이언트/구성이 반영되도록 항상(`prebuild` 를 통해) 다시 빌드합니다.

둘 다 `jupyter lite` 가 설치되어 있지 않으면 **경고와 함께 건너뛰므로** Node만 있는 빌드도 여전히 성공합니다. 그 결과가 무엇인지 알아두세요. 패널은 "빌드되지 않음" 메시지를 표시하지 않습니다. 패널의 iframe은 `/jupyterlite/lab/index.html` 을 요청하는데, 알 수 없는 경로에 `index.html` 로 응답하는 모든 것(Tauri의 자산 리졸버, 또는 `try_files $uri /index.html` 같은 SPA 폴백이 있는 정적 호스트)은 패널에 대신 **GeoLibre의 두 번째 사본**을 건네줍니다(GeoLibre#1851, GeoLibre#1658). 위의 의존성을 설치하고 다시 빌드하세요.

따라서 두 가지 빌드는 CLI 누락을 건너뛸 수 있는 것이 아니라 치명적인 것으로 취급합니다. 사이트를 직접 제공하므로 성능 저하 모드로 동작할 수 없기 때문입니다. Mac App Store 빌드와 `GEOLIBRE_JUPYTERLITE_REQUIRED=1`을 설정하는 모든 빌드(Docker 이미지가 그렇습니다)입니다. Mac App Store 빌드는 JupyterLab 서버를 생성할 수 없으므로 그 Notebook 패널은 대신 JupyterLite 사이트를 임베드합니다. 그 밖의 모든 데스크톱(Tauri) 개발 및 빌드 경로는 이를 완전히 건너뛰므로(실제 JupyterLab 서버를 사용), 정적 사이트가 설치 프로그램을 부풀리는 일은 없습니다.

Docker 이미지는 사이트를 빌드하고 제공하며, `docker/nginx.conf` 에서 `/jupyterlite/` 에 자체 블록을 부여합니다.

- 자체 CSP. JupyterLab은 인라인 `<script>` 에서 부트스트랩하는데 앱의 정책은 이를 금지하므로, 앱 정책 하에서는 사이트가 HTML을 로드한 뒤 부팅되지 않습니다.
- SPA 폴백 없음(`try_files $uri $uri/ =404`). 따라서 이 이미지에서는 위의 실패가 복제된 앱이 아니라 일반 **404**로 드러납니다. 복제본은 Tauri와 SPA 폴백 호스트가 만들어내는 결과로 남아 있습니다.
- 콘텐츠 해시된 자산에 대한 장기 불변 캐시 정책. 그렇지 않으면 접두사 일치가 이를 빼앗아 갔을 것입니다.

빌드 구성은 `apps/geolibre-desktop/jupyterlite/` 에 있습니다 (`jupyter_lite_config.json`, `jupyterlite.json`, 빌드용 `requirements.txt`). 빌드는 커널 클라이언트와 시작용 `Welcome.ipynb` 를 `backend/geolibre_server/` 아래의 정식 사본(`notebook_client.py`, `notebook_examples/Welcome.ipynb`)에서 `files/`로 스테이징합니다. 생성된 디렉터리는 PWA 프리캐시에서 제외됩니다 (`apps/geolibre-desktop/vite.config.ts`의 `pwaPlugin` 참조).

데스크톱 서버

- 백엔드 extra: `backend/geolibre_server/pyproject.toml`의 `notebook` (`jupyterlab`, `jupyter-server`). 사이트카의 환경을 절대 건드리지 않도록 자체 `uv` 프로젝트 환경에 동기화됩니다.
- 런처 / 수명 주기: `apps/geolibre-desktop/src-tauri/src/lib.rs`의 `start_jupyter_server / stop_jupyter_server` Tauri 명령(사이드카 런처를 반영)과 `apps/geolibre-desktop/src/lib/jupyter.ts`의 TS 래퍼.
- 프레이밍: `backend/geolibre_server/jupyter_server_config.py`는 앱이 서버를 임베드할 수 있도록 `Content-Security-Policy: frame-ancestors`를 Tauri 웹뷰 / 루프백 오리진으로 완화합니다. Tauri CSP(`tauri.conf.json`)는 루프백 오리진을 `frame-src / child-src`에 추가합니다.
- 지도 명령 릴레이: 같은 구성이 `geolibre_server.jupyter_relay` 서버 확장을 활성화합니다(위 참조). 그 WebSocket은 앱 자체의 오리진만 받아들이고 모든 엔드포인트는 서버의 실행별 토큰을 요구하므로, 명령은 그곳에서 이미 커널 실행 권한을 가진 것에서만 올 수 있습니다.

<https://geolibre.app/roadmap/>

GeoLibre 로드맵

v0.1: 지도 뷰어와 GeoJSON

- [x] Tauri + React + MapLibre 셸
- [x] GeoJSON 불러오기, 레이어 패널, 스타일 패널
- [x] 속성 테이블(기본)
- [x] 로컬 알고리즘을 갖춘 처리(Processing) UI
- [x] 플러그인 인터페이스 + 샘플 플러그인

v0.2: 프로젝트 영속성

- [x] `.geolibre.json` 저장/열기
- [x] 세션 내 최근 프로젝트 추적
- [x] 속성 테이블에서 피쳐 강조 표시
- [x] 선택한 피쳐로 줌 이동 옵션
- [x] 최근 프로젝트 UI 및 영속성

v0.3: 클라우드 네이티브 포맷

- [x] DuckDB-WASM을 통한 GeoParquet 가져오기
- [x] DuckDB-WASM 및 URL 기반 Components 플러그인 패널을 통한 FlatGeobuf 가져오기
- [x] Components 플러그인을 통한 PMTiles
- [x] COG 및 GeoTIFF 래스터 렌더링
- [x] GeoJSON 및 소스 경계를 인식하는 레이어 유형에 대한 레이어로 줌 이동

v0.4: DuckDB Spatial

- [x] DuckDB-WASM 통합
- [x] `INSTALL spatial / LOAD spatial`
- [x] Shapefile, KMZ/KML, GeoPackage, GeoParquet, FlatGeobuf, GML 및 관련 벡터 가져오기 경로

v0.5: 고급 Add Data 및 플러그인 기반 레이어

- [x] XYZ, WMS, 벡터 파일, GeoJSON URL, 벡터 타일, 래스터 타일 템플릿, COG 및 GeoTIFF 래스터, MBTiles, ArcGIS 레이어를 위한 **Add Data**(데이터 추가) 대화 상자

- [x] FlatGeobuf, PMTiles, Zarr, LiDAR, Gaussian splat 패널을 갖춘 MapLibre Components 플러그인
- [x] Tauri 명령을 통한 데스크톱 MBTiles 메타데이터 및 타일 읽기
- [x] Plugins(플러그인) 메뉴의 플러그인 컨트롤 위치 제어
- [x] GeoLibre가 관리하는 레이어에 대한 레이어 컨트롤 통합

v0.6: 프로젝트 접근, 웹 임베드, 확장된 통합

- [x] 데스크톱 파일 최근 항목과 URL 기반 웹 최근 항목을 갖춘 영구 최근 프로젝트
- [x] Open Project from File(파일에서 프로젝트 열기)과 Open Project from URL(URL에서 프로젝트 열기) 흐름 분리
- [x] 간결한 레이아웃, 아이콘 전용 톨바, 숨겨진 패널을 위한 브라우저 데모 쿼리 옵션
- [x] 데스크톱 Martin 서버 통합을 통한 PostgreSQL 레이어 워크플로
- [x] 카탈로그 기반 래스터 레이어 추가를 위한 STAC 검색 워크플로
- [x] Esri Wayback, GeoAgent, GeoEditor, Street View, Swipe 플러그인 통합

v0.7: Add Data 확장, 식별, 설정, 처리

- [x] URL 또는 로컬 파일에서 GPX 불러오기, 웨이포인트·트랙·루트 레이어 선택 가능
- [x] 경도와 위도 필드를 사용하여 URL 또는 로컬 파일에서 구분 텍스트 불러오기
- [x] **Add Data**(데이터 추가) 대화 상자를 통한 WFS GetFeature 불러오기
- [x] 강화된 팝업 처리를 갖춘 WMS GetFeatureInfo 식별 지원
- [x] 관리형 Python 사이드카 기반 Whitebox 툴박스
- [x] 인라인 속성 편집, 테이블 가로 스크롤, 스크롤 가능한 식별 팝업
- [x] 지도 환경 설정과 런타임 환경 변수를 위한 Settings(설정) 대화 상자
- [x] 프로젝트 파일에 플러그인 상태 영속화
- [x] 기본 GeoJSON 샘플 URL 및 더 큰 식별 팝업
- [x] 로컬 래스터 파일 불러오기 수정
- [x] 대용량 파일 pre-commit 가드

v0.8: 뷰어, 데스크톱 패키징, 플러그인, 동적 레이어

- [x] `web.geolibre.app` 에서 제공되는 Cloudflare Worker 뷰어
- [x] 브라우저 데모 링크를 프로덕션 뷰어로 업데이트
- [x] GPX 드래그 앤 드롭을 이름이 지정된 웨이포인트·트랙·루트 레이어로 분리
- [x] 불러올 때 벡터 레이어를 EPSG:4326으로 재투영
- [x] 데스크톱 About(정보) 대화 상자 업데이트 확인
- [x] 앱 데이터 plugins 디렉터리에서 외부 플러그인 zip 동적 로드
- [x] 비보안 컨텍스트에서 `crypto.randomUUID` 의 안전한 폴백
- [x] `plugin.json` 을 사용한 외부 플러그인 매니페스트 지원

- [x] `maplibre-gl-3d-tiles` 를 통한 3D Tiles 레이어 지원
- [x] 프로젝트를 다시 열 때 3D Tiles 복원
- [x] GeoParquet 패널 DuckDB 시작 수정
- [x] MSIX 데스크톱 패키징 및 더 깔끔한 빌드 출력
- [x] 로컬 디렉터리에서 등록되는 외부 네이티브 GeoJSON 레이어
- [x] 외부 네이티브 레이어로 등록되는 래스터 베이스맵
- [x] GeoJSON 레이어에 렌더링되는 텍스트 마커 라벨
- [x] WFS 및 GeoJSON URL 레이어의 수동·자동 새로 고침
- [x] 다중 DuckDB SQL 쿼리 결과 레이어
- [x] 데스크톱 진단 패널 및 개선된 진단/상태 표시줄 대비
- [x] Colorbar(컬러바), Legend(범례), HTML 패널을 위한 톨바 토글

v0.9: 데이터 통합, 처리, 메뉴 재구성

- [x] 불러온 레이어, 로컬 파일, 원격 URL을 대상으로 DuckDB Spatial SQL을 실행하는 SQL Workspace(SQL 워크스페이스), 샘플 쿼리, 쿼리 기록, 결과를 지도에 추가하거나 내보내기 지원
- [x] STAC 데이터를 탐색하고 불러오는 Planetary Computer 패널
- [x] 데이터셋을 탐색하고 불러오는 Earth Engine 패널
- [x] Overture 데이터 테마를 불러오는 Overture Maps 플러그인
- [x] `maplibre-gl-time-slider` 기반으로 시계열 래스터 및 벡터 데이터를 애니메이션하는 Time Slider(타임 슬라이더) 플러그인
- [x] 네 가지 연방 데이터 플러그인을 갖춘 Web Services(웹 서비스) 메뉴
- [x] `maplibre-gl-raster` 플러그인 기반 Add Raster Layer(래스터 레이어 추가)
- [x] `maplibre-gl-vector` 플러그인 기반 Add Vector Layer(벡터 레이어 추가)
- [x] DuckDB 레이어에 대한 식별, 선택, 속성 테이블 지원
- [x] Processing(처리) 아래의 Conversion(변환) 메뉴: Vector to GeoParquet/FlatGeobuf/PMTiles, CSV to GeoParquet, Raster to COG를 경로 허용 목록을 갖춘 강화된 변환 사이드카로 지원
- [x] Processing 아래의 Vector(벡터) 메뉴: 일반적인 지오메트리 도구(버퍼, 중심점, 볼록 껍질, 디졸브, 경계 상자, 단순화, 클립, 교차, 차집합, 합집합)를 Turf.js로 클라이언트 측에서 실행하며, 선택적 GeoPandas 사이드카 엔진 제공
- [x] Processing 아래의 Raster(래스터) 메뉴: 일반적인 래스터 도구(음영기복, 경사, 경사향, 재투영, 리샘플링, 범위로 클립, 마스크 레이어로 클립, 폴리곤화, 등고선)를 rasterio 사이드카로 지원, 경로 입력과 경로 출력
- [x] 벡터 및 GeoTIFF/COG 래스터 파일을 지도에 드래그 앤 드롭하여 레이어로 추가
- [x] 선택한 입력 디렉터리를 대상으로 실행되는 Whitebox 배치 도구
- [x] Measure(측정), Bookmark(북마크), Minimap(미니맵), View State(뷰 상태) 도구를 갖춘 Controls(컨트롤) 메뉴
- [x] `PrintControl` 기반 Print(인쇄) 메뉴
- [x] New(새로 만들기), Open(열기), Save(저장), Save As(다른 이름으로 저장)를 통합한 Project(프로젝트) 메뉴

- [x] 패널별 표시 토글을 갖춘 Layout(레이아웃) 설정
- [x] 스택에서 레이어 위치를 지정하는 Insert before(앞에 삽입) 드롭다운
- [x] 컴포넌트 패널 영속화 및 새 프로젝트 시 컨트롤 초기화
- [x] 플러그인이 URL 쿼리 매개변수를 선언하고 처리할 수 있음
- [x] 크롬 없는 지도 임베드를 위한 `maponly` 쿼리 매개변수
- [x] 임베드의 초기 라이트/다크 테마를 설정하는 `theme` 쿼리 매개변수
- [x] 브라우저 앱을 위한 Docker 지원
- [x] 오프라인 spatial 확장 로드를 위한 `VITE_DUCKDB_SPATIAL_EXTENSION_PATH`

v1.0: 처리 파이프라인, 외부 플러그인 시스템, 안정적인 프로토타입

- [x] GDAL / Rasterio / GeoPandas 파이프라인
- [x] 버퍼, 재투영, GeoJSON 내보내기 처리 도구
- [x] 확장된 WhiteboxTools 범위
- [x] 외부 플러그인 패키지 배포 워크플로
- [x] 플러그인 마켓플레이스 / 레지스트리 설계([플러그인 마켓플레이스](#)와 [레지스트리](#) 참조)
- [x] 플러그인 마켓플레이스 MVP: 큐레이션된 레지스트리와 탐색·설치 UI
- [x] 플러그인 업데이트(제자리 재가져오기) 및 확인을 거친 제거
- [x] 개인 API 토큰을 사용하여 `share.geolibre.app`에 업로드하는 Project(프로젝트) 메뉴의 Share(공유) 동작
- [x] Jupyter 노트북용 Python 패키지(`geolibre`): leafmap 스타일 API(`add_geojson`, `add_tile_layer`, `add_cog`)와 양방향 `.geolibre.json` 프로젝트 동기화를 갖춘 `anywidget`으로 전체 앱을 임베드
- [x] 성능 튜닝 및 테스트 스위트
- [x] 크로스 플랫폼 설치 프로그램
- [x] 문서 및 튜토리얼

v1.1: 벡터 스타일링, 속성 테이블 관리, 대기 효과

- [x] Pyodide를 통한 Vector 도구용 인브라우저 GeoPandas 엔진(서버 없음, 선택적 사이드카와 동일한 결과)
- [x] 호스트 `deck.gl`을 `app.getDeckGL()` 로 외부 플러그인에 노출하여, 플러그인이 자체 복사본을 번들하는 대신 공유 인스턴스에 렌더링
- [x] Add Vector Layer(`maplibre-gl-vector`) 레이어에 대한 스타일 패널 지원, 컨트롤의 네이티브 레이어에 적용되는 단일·범주형·등급형·표현식 심볼로지 포함
- [x] 레이어 패널에서 이름을 더블클릭하거나 레이어 동작 메뉴에서 레이어 이름 변경
- [x] 레이어 동작 메뉴에 Open attribute table(속성 테이블 열기) 및 Export(내보내기) 동작 추가
- [x] 수동 및 자동(시간 기반) 새로 고침을 Add Vector Layer URL 레이어로 확장
- [x] 속성 테이블 열 관리: 필드 이름 변경, 삭제, 숨기기/표시, 순서 변경, 프로젝트와 함께 영속화

- [x] Atmospheric Effects(대기 효과) 플러그인: 심우주 배경, 시차 별 배경, 혜성, 낮은 줌에서의 지구 대기 후광(Controls 메뉴에서 토클)
- [x] 문서의 conda-forge 설치 안내 및 비디오 튜토리얼
- [x] MIT 라이선스
- [x] 브라우저 빌드에서 DuckDB-WASM이 번들을 불러올 수 있도록 `cdn.jsdelivr.net` CSP 허용

v1.2: 새 데이터 소스, 속성 분석, 라우팅, 플랫폼 다듬기

- [x] osmix로 브라우저에서 파싱되는 OpenStreetMap PBF 파일 불러오기
- [x] kerchunk 참조를 통해 불러오는 Cloud-Optimized NetCDF/HDF 레이어
- [x] 사용자 지정 요청 헤더를 통한 인증된 3D Tiles 타일셋
- [x] 지리참조된 비디오 오버레이 레이어
- [x] 업로드한 파일과 원격 URL로 deck.gl 오버레이를 구성하는 Deck.gl Layer 빌더
- [x] DuckDB Spatial SQL Workspace와 나란히 제공되는 PGLite 기반 인브라우저 PostgreSQL SQL 엔진
- [x] 속성 테이블의 필드 추가 및 필드 계산기 도구
- [x] 속성 테이블 Charts(차트) 패널(히스토그램, 산점도, 막대, 선, 상자)
- [x] Vector 도구에 공간 조인 추가
- [x] Select by value(값으로 선택) 및 Select by location(위치로 선택) 도구
- [x] 육각형 그리드를 만들고 포인트를 H3 셀에 비닝하는 H3 도구
- [x] 포인트 히트맵 렌더러와 클러스터링, Add Vector Layer 포인트 레이어 포함
- [x] `maplibre-gl-directions` 를 통한 대화형 라우팅용 Directions(길찾기) 플러그인, 활성화 전 1회 개인 정보 안내 표시
- [x] 레이어 및 스타일 작업의 실행 취소/다시 실행
- [x] 명령 팔레트(`Ctrl / Cmd + K`) 및 `?` 치트 시트가 있는 전역 키보드 단축키
- [x] PNG 및 PDF 내보내기를 갖춘 인쇄 레이아웃 작성기
- [x] 설치 가능하고 오프라인에서 동작하는 프로그레시브 웹 앱(PWA) 빌드
- [x] 추출된 문자열 카탈로그와 `?locale / ?lang` 임베드 언어 매개변수를 갖춘 국제화 프레임워크(`react-i18next`)
- [x] 주요 화면 전반의 axe 검사를 통한 접근성 점검
- [x] 앱, 섹션, 플러그인 React 오류 경계
- [x] Playwright 엔드투엔드 스모크 테스트 및 CI 작업
- [x] 더 많은 Add Data 레이어 유형을 다루는 확장된 Python Map API
- [x] Jupyter 휠과 데스크톱 바이너리를 줄이기 위한 CDN 로드 PGLite/PostGIS

v1.3: 분석 심화, 실시간 협업, 스토리 맵, 스크립팅, AI 어시스턴트

- [x] Processing(처리) 아래의 Spatial Statistics(공간 통계) 툴박스
- [x] Vector 도구: Smooth(평활화), Regular grid(정규 그리드), Voronoi/Delaunay
- [x] IDW / 크리깅 보간(포인트 레이어 → 연속 래스터 표면)

- [x] 일치하는 키로 테이블의 필드를 조인하는 속성(테이블) 조인 벡터 도구
- [x] 래스터 분석 도구: 구역 통계, 래스터 계산기, 재분류, 모자이크, 초점 통계
- [x] Python 사이드카를 사용할 수 없을 때 브라우저에서 실행되는 클라이언트 측 래스터 처리 폴백
- [x] 래스터 스타일링을 위한 분류 기반 단일 밴드 의사 색상 및 RGB 밴드 조합
- [x] 처리 도구의 배치 실행 및 모델/파이프라인 체이닝
- [x] 네트워크 분석: 등시선, 서비스 영역, 출발지-목적지 비용 행렬
- [x] 레이어 패널의 접을 수 있는 레이어 그룹/폴더
- [x] 좌표에 배치되는 glTF/GLB 3D 모델 레이어
- [x] 대용량 로컬 벡터 레이어를 위한 클라이언트 측 벡터 타일링
- [x] 매우 큰 벡터 파일을 불러오기 전 경고
- [x] SQL 쿼리에서 공개 S3, GCS, Azure 클라우드 스토리지 URL의 투명한 재작성
- [x] Shapefile 및 GeoPackage 내보내기
- [x] 추가 SQL Workspace 엔진으로서의 Apache Sedona
- [x] 다중 제공자 지오코딩 추상화를 갖춘 배치 지오코딩 및 역지오코딩 도구
- [x] Cloudflare Worker 기반 실시간 다중 사용자 협업(MVP)
- [x] 스크롤 기반 스토리 맵 빌더, 프레젠테이션, 독립 실행형 HTML 내보내기
- [x] 인쇄 레이아웃을 위한 사용자 편집 가능 범례
- [x] 속성 테이블의 필드 통계 요약 패널
- [x] [segment-geospatial](#)(SamGeo)과 Meta의 SAM 3를 통한 AI Segmentation(AI 분할) 툴박스, 사이드카를 거쳐 별도의 `samgeo-api` 모델 서버로 프록시
- [x] 자연어 요청을 감사 가능하고 실행 취소 가능한 GeoLibre 작업으로 바꾸는 자연어 GIS 어시스턴트 (Strands 에이전트)
- [x] Python 자동화 API 및 앱 내 Python Console(Python 콘솔)
- [x] Python 패키지: 로컬 래스터, 마커/클러스터, 단계구분도 API; `split_map`, `add_legend`, `add_colorbar` 헬퍼; 선택/그린 피처의 타입 지정 읽기; `to_html` 내보내기
- [x] macOS용 Homebrew Cask 패키징
- [x] Tauri v2 모바일을 통해 동일한 코드베이스에서 만든 네이티브 Android 앱, 서명된 ABI별 릴리스 APK(~40 MB)를 빌드하는 CI 워크플로 포함 — [Android](#) 참조
- [x] Android에서 데스크톱 프로세스 도구(Whitebox, Raster, Conversion, AI Segmentation, PostgreSQL/Martin)를 숨겨 실행할 수 없는 것이 표시되지 않도록 하는 `isMobile()` 기능 게이팅
- [x] 반응형 터치 친화적 모바일 레이아웃: 휴대폰에서 Layers/Style 패널이 슬라이드오버 시트로 지도 위에 오버레이되고, 포인터 이벤트(터치) 패널 크기 조절, 툴바가 시스템 상태 표시줄을 피하도록 하는 안전 영역 인셋
- [x] 현재 지도 뷰의 베이스맵 타일을 서비스 워커 캐시에 미리 캐시하는 Download Offline Area(오프라인 영역 다운로드) 도구
- [x] CDN 로드 Pyodide 및 PGLite/PostGIS 엔진의 서비스 워커 캐싱으로 최초 사용 후 브라우저 SQL과 Python이 오프라인에서도 계속 동작

v1.4: 지도 옆의 Jupyter, 분광 지수, 지리참조, 현장 수집

- [x] 지도 옆에 도킹되는 크기 조절·접기 가능한 Notebook(노트북) 패널: 웹 빌드는 자체 호스팅 JupyterLite 사이트(인브라우저 Pyodide 커널)를 임베드하고 데스크톱 빌드는 uv로 관리되는 JupyterLab 서버를 실행하며, 둘 다 실행 가능한 Welcome 투어가 미리 포함됩니다. 노트북 셀은 자동 로드되는 `geolibre` 클라이언트를 통해 공유 스크립팅 브리지로 라이브 지도를 제어하고, JupyterLite 테마는 앱 테마를 따릅니다 — [노트북 패널 참조](#)
- [x] Processing → Raster 아래의 Spectral Index(분광 지수) 툴박스(NDVI, GNDVI, NDWI, NDMI, NDBI, NBR, EVI, SAVI), Sentinel-2, Landsat 8-9, NAIP, 사용자 지정 밴드 레이아웃과 반사율 스케일 조절 기능 제공, `geotiff.js`로 클라이언트 측에서 또는 기존 래스터 계산기를 통해 `rasterio` 사이드카에서 계산
- [x] Raster Georeferencer(래스터 지리참조기)(Processing → Georeferencing): 지상 기준점을 사용하여 지리참조되지 않은 이미지를 지도에 고정하며, 최소제곱 아핀 피팅과 GCP별 및 RMS 잔차를 사용하고, 프로젝트에 영속화되며 오프라인에서도 동작하는 모서리 고정 오버레이로 추가
- [x] 레이어별 사용자 지정 양식(텍스트/숫자/날짜/선택 필드와 선택적 인라인 사진)으로 포인트·라인·폴리곤 관측을 수집하는 Field Collection(현장 수집) 도구(Controls 메뉴), 기기 GPS 또는 지도 탭으로 배치하며 플로팅 빠른 열기 컨트롤 제공; 수집 결과는 태그가 지정된 GeoJSON 레이어에 기록되어 속성 테이블, 내보내기, 오프라인 사용으로 이어짐
- [x] 기존 런타임 환경 시스템을 통한 `VITE_PYODIDE_INDEX_URL` 및 `VITE_DUCKDB_SPATIAL_EXTENSION_PATH`의 런타임 재정의로, 에어갭 또는 기업 배포 환경에서 앱을 다시 빌드하지 않고 Pyodide와 DuckDB Spatial 확장을 내부 미러로 지정 가능

v1.5: 대시보드, 인브라우저 Whitebox, 지도 탐색 기록, 서명된 macOS 설치 프로그램

- [x] 불러온 레이어를 한눈에 요약하는 차트 위젯 Dashboard(대시보드) 패널, 구성 가능한 차트와 지도 옆에 도킹되는 접을 수 있는 레이아웃 제공
- [x] Whitebox 툴박스가 이제 래스터 I/O를 갖춘 WebAssembly 런타임을 통해 브라우저에서 완전히 실행되므로, 그 도구들(과 이제 같은 툴박스에 표시되는 GeoLibre 자체 WASM 래스터 도구)이 Python 사이드카 없이 동작
- [x] 인쇄 레이아웃 작성기에 명시적 지도 축척 입력, 편집 가능한 제목과 바닥글이 있는 제목 블록, 페이지 크기 컨트롤, 사용자 지정 인쇄 범위가 추가되고, 프로덕션 품질 PNG 및 PDF 지도 내보내기를 위한 더 안정적인 미리보기 렌더링 제공
- [x] Time Slider가 지도에 이미 있는 기존 벡터 레이어를 타임라인에 바인딩하여 데이터를 다시 가져오지 않고 시계열 속성을 애니메이션 가능
- [x] 뷰포트 기록 탐색(이전 지도 범위를 앞뒤로 이동), 회전 표시기가 있는 피치 및 방위 재설정 컨트롤, 모호한 나침반 대신 뚜렷한 북쪽 화살표, 지도 경계가 제한될 때의 잠금 표시기, 다듬어진 기본 오른쪽 상단 지도 컨트롤 세트를 갖춘 새 View(보기) 메뉴
- [x] 웹 서비스 레이어를 위한 저장된 서비스 라이브러리로, 자주 사용하는 WMS, WFS, XYZ, ArcGIS 엔드포인트를 한 번 저장해 두고 매 세션마다 다시 입력하는 대신 클릭 한 번으로 다시 추가 가능

- [x] Add Data 대화 상자가 이제 완전히 국제화되었으며 좌표 데이터에 심표 소수 구분 기호와 드래그 앤 드롭된 CSV 파일을 허용
- [x] Identify 아이콘에서 직접 COG 픽셀 값 검사, 역순 및 사용자 지정 래스터 색상 램프, 전체 컬러맵 목록, 레이어 동작 메뉴에서 다시 열 수 있는 스타일링 패널
- [x] 표시되는 메뉴, 패널, 데이터 소스를 조정하는 사용자 지정 가능 UI 프로파일로, 배포 환경이 사용자에게 앱의 집중된 하위 집합을 제공 가능([UI 프로파일](#) 참조)
- [x] 북마크가 이제 카메라와 함께 활성 레이어를 저장하며, 선택적 내보내기, 크기 조절 및 순서 변경 가능한 패널, 다른 이름으로 저장 이름 프롬프트 제공
- [x] 순서가 지정된 웨이포인트 집합을 지나는 경로를 계산하는 Processing 아래의 Sequential route(순차 경로)(길찾기) 네트워크 도구
- [x] New project(새 프로젝트) 대화 상자의 Protomaps 베이스맵, 여러 래스터 베이스맵 중첩 지원, 레이어 컨트롤의 베이스맵 요소 표시 프리셋
- [x] Atmospheric Effects 아래의 Spinning Globe(회전 지구본) 패널과 지구본 뷰를 위한 사용자 지정 가능 대기 후광 및 심우주 색상
- [x] Notebook 패널이 지도와 워크스페이스를 50/50으로 분할하고 더 넓은 공간을 위해 Style 패널을 자동으로 접을 수 있으며, 속성 테이블에 넓은 테이블에서 필드를 찾고 토글하는 열 탐색기 추가
- [x] USGS LiDAR 플러그인이 USGS 3DEP 포인트 클라우드 데이터 탐색과 불러오기를 위해 이전 LiDAR Viewer를 대체하고, 가져온 KML 및 KMZ 레이어가 내장 심볼로지를 따르며, 벡터 스트로크를 축척 비례 미터 단위로 지정 가능
- [x] macOS 데스크톱 설치 프로그램이 이제 Apple Developer ID 인증서로 서명되고 Apple의 공증을 받아 Gatekeeper 우회 없이 열리며, 공식 Homebrew cask에 GeoLibre를 제출하는 생성기 제공([다운로드](#) 참조)

v1.6: 다중 지도 레이아웃, 고급 심볼로지와 라벨링, 플러그인 zip 설치

- [x] 워크스페이스를 카메라 이동이 동기화된 지도 뷰 그리드로 분할하는 다중 지도 그리드로, 베이스맵, 레이어, 시간 단계를 나란히 비교 가능
- [x] 규칙 기반 렌더러(필터 기반 스타일 규칙), 비례 심볼, 채우기 패턴, 더 풍부한 포인트 및 폴리곤 스타일링을 위한 내장 마커 라이브러리를 갖춘 고급 벡터 심볼로지
- [x] 배치 및 스타일링 컨트롤로 벡터 피처에 어떤 속성으로든 라벨을 붙이는 라벨 엔진
- [x] 기존 매니페스트 URL 및 번들 드롭인 경로와 함께 데스크톱과 웹 모두에서 업로드한 zip으로 외부 플러그인 설치, Manage Plugins(플러그인 관리) 목록은 이제 알파벳순 정렬
- [x] 이동, 시공간, 셀 커버리지 분석을 위한 Processing → Vector 아래의 새 벡터 분석 도구
- [x] 모든 Add Data 대화 상자에서 표준화된 샘플 데이터 드롭다운으로, 업스트림 기반 패널마다 바로 불러올 수 있는 예제 데이터셋 제공
- [x] 패널을 벗어나지 않고 위치를 지오코딩하는 Layers(레이어) 패널 바닥글의 Search places(장소 검색) 상자
- [x] 라이트와 다크를 넘어선 강조 색상 구성표로, UI 테마를 선택한 강조 색상으로 물들일 수 있음
- [x] 레이어 패널에서 핵심 베이스맵을 더블클릭하여 교체

- [x] 스토리 맵 프레젠테이션에 Reset(초기화) 버튼이 추가되고 발표 중에는 더 깔끔한 전체 화면 스토리를 위해 사이드 패널을 자동으로 접음
- [x] 더 명확한 최초 구성을 위해 웹사이트와 로컬 토큰 단계를 분리한 점진적 Share 설정
- [x] Python 사이드카가 실행되지 않거나 시작에 실패할 때 안내하는 Whitebox 툴박스과 AI Segmentation의 대화형 사이드카 도움말 배너
- [x] 시작 시 업데이트 확인, 업데이트 환경 설정, 더 명확한 상태 색상을 갖춘 안내형 업데이트 워크플로
- [x] Time Slider가 지정되지 않은 종료 날짜를 현재 날짜로 기본 설정
- [x] OpenGeos.GeoLibre 로서의 Windows Package Manager(winget) 패키징으로, winget을 통해 앱을 설치하고 업데이트 가능
- [x] [Microsoft Store](#) 등록으로, Windows 사용자가 서명된 자동 업데이트 빌드를 Store에서 직접 설치 가능 ([다운로드 참조](#))

v1.7: 플러그인 UI 호스트 API, 색상 램프 미리보기, 범주별 탐색 Whitebox 도구

- [x] 플러그인이 오버레이를 흉내 내는 대신 내장 Style 패널 옆에 도킹되는 일급 오른쪽 사이드바 패널, 툴바 메뉴, 플로팅 패널을 등록할 수 있게 하는 플러그인 UI 호스트 API, 외부 플러그인 툴바 메뉴는 이제 Help(도움말) 메뉴 뒤에 배치([플러그인 API 참조](#))
- [x] 벡터 Style 패널과 래스터 Color ramp(색상 램프) 선택기 모두의 색상 램프 그라데이션 견본으로, 단순한 이름 목록에서 고르는 대신 선택하는 동안 각 컬러맵의 그라데이션을 인라인(트리거와 모든 옵션 옆)으로 확인 가능
- [x] ArcGIS 스타일 컨트롤(앵커, X/Y 오프셋, 회전, 줄바꿈 너비, 대소문자), Duplicate labels(중복 라벨) 옵션, 같은 좌표에 쌓인 포인트를 하나의 중복 제거된 라벨로 합치는 고유/연결 모드를 갖춘 더 풍부한 벡터 라벨링
- [x] Whitebox 툴박스를 이제 Processing 메뉴에서 범주별로 직접 탐색 가능하며, 중첩된 하위 범주 하위 메뉴, 자체 소제목 아래 묶인 GeoLibre 자체 WASM 도구, 제한된 환경을 위한 오프라인 번들 도구 카탈로그, 미리 선택되어 스크롤된 상태로 대화 상자를 여는 도구 제공
- [x] 캔버스 위의 협업 세션 상태 배치와 명단(맥동하는 라이브 점, 연결된 참가자 수, 참가와 퇴장을 알리는 확장 가능한 클라이언트 목록), 그리고 호스트가 지도로 돌아갈 비파괴적 방법을 제공하는 명확한 "Go to map and collaborate"(지도로 이동하여 협업) 버튼
- [x] 임베드에서는 시작 마법사가 표시되지 않음: 프로젝트 딥 링크(?url=)는 온보딩을 자동으로 건너뛰고, 새 ?welcome=0 매개변수로 모든 임베드가 선택 해제 가능

v1.8: 카메라 투어, 라이브 스토리 맵, 독립 실행형 HTML 내보내기, 지도 주석

- [x] Controls 메뉴에서 바로 애니메이션 카메라 투어를 비디오로 녹화하며, 더 명확한 키프레임 레이아웃, 키프레임별 재캡처, 2단계 저장, 전체 투어 설정을 JSON 파일로 저장하고 다시 불러오는 기능 제공

- [x] Story Map(스토리 맵) 플러그인이 이제 별도의 편집기 대신 라이브 지도에서 직접 챕터를 구성하고, 완성된 스토리의 인쇄 가능한 PDF 유인물을 생성
- [x] 서버 없이 오프라인으로 실행되는 단일 독립 실행형 대화형 HTML 파일로 프로젝트 내보내기, 그리고 공유 프로젝트를 탐색하고 클릭 한 번으로 여는 프로젝트 갤러리
- [x] 지도에 직접 텍스트, 화살표, 강조 표시를 그리는 지도 주석 레이어, 프로젝트와 함께 영속화
- [x] Gridlines(격자선) 오버레이(Graticule에서 이름 변경)가 지도 전체에 가장자리 라벨이 있는 좌표 격자를 그림
- [x] EXIF GPS에서 지오태그 사진을 포인트 레이어로 가져오기, 내장 좌표가 없는 사진은 수동 배치 및 드래그로 위치 지정
- [x] URL 또는 로컬 파일에서 불러오는 Add Data의 GeoRSS 피드 지원
- [x] 베이스맵 라이브러리에 EOX Sentinel-2 cloudless 위성 영상과 Openbasiskaart 추가 (Openbasiskaart는 basemap-control 0.7.0을 통해)
- [x] 캔버스를 벗어나지 않고 좌표를 읽고 빠른 동작에 접근하는 지도의 오른쪽 클릭 컨텍스트 메뉴
- [x] 실시간 협업 세션을 위한 참가자별 권한과 앱 내 채팅 패널
- [x] 더 깔끔하게 묶인 저장된 뷰를 위해 북마크를 폴더로 정리
- [x] AI 백엔드를 선택하고 구성하는 기능별 제공자 드롭다운을 갖춘 Settings의 전용 AI Providers(AI 제공자) 섹션
- [x] 플러그인 API가 이제 네이티브 래스터 및 타일 레이어를 등록할 수 있으며, 플러그인 패널을 위한 공유 레일(스타일 대체) 오른쪽 패널 도킹 모드 제공([플러그인 API](#) 참조)
- [x] Time Slider가 래스터 스택의 픽셀 시계열 차트를 그려, 샘플링한 픽셀의 값을 타임라인에 걸쳐 플롯
- [x] Colorbar 패널에 쌓기 방향 컨트롤, 크기 조절 가능한 패널, 여러 컬러바의 쌓기 순서 수정 추가
- [x] 활성 상호작용 모드가 계속 표시되도록 하는 Directions 및 Reverse Geocode(역지오코딩) 도구의 영구 모드 배너
- [x] 렌더링된 지도를 다른 앱에 바로 붙여넣기 위해 Print Layout(인쇄 레이아웃) 작성기에 Copy to Clipboard(클립보드에 복사) 추가
- [x] 도-십진분(DDM) 입력을 지원하는 더 명확한 Set view(뷰 설정) 좌표 입력 워크플로
- [x] 래스터 페인트 컨트롤에 그레이스케일 토클, 초기화 동작, 숫자 입력이 추가되고, 레이어 줌 표시 컨트롤을 설명하는 정보 아이콘 제공
- [x] 레이어 컨트롤의 인라인 숫자 불투명도 입력과 Background(배경) 레이어의 고정 이름 안내
- [x] 설치 없이 데스크톱 앱을 실행할 수 있는 Windows 포터블 zip 빌드

v1.9: CAD 가져오기, 더 똑똑한 서비스 검색, 도킹된 SQL 워크스페이스

- [x] CAD 도면(DXF/DWG)을 레이어로 추가, 불러올 도면 레이어를 고르는 레이어 선택기와 지오메트리를 지도에 올바르게 배치하는 CRS 선택기 제공
- [x] WMS 및 WFS 패널이 이제 서비스의 GetCapabilities 문서를 읽어 사용 가능한 레이어와 피처 유형을 나열하므로, 레이어 이름을 직접 입력하는 대신 채워진 드롭다운에서 선택
- [x] 기존의 대상별 변환기와 함께, 파일 확장자에 따라 지원되는 모든 벡터 포맷 간에 변환하는 범용 Vector to Vector 변환 도구

- [x] SQL Workspace가 (플로팅 창이 아닌) 지도 옆의 크기 조절 가능한 패널로 도킹되고, 테이블, 열, SQL 키워드에 대한 편집기 자동 완성 추가
- [x] 카메라 투어에 더 세밀한 속도 조절을 위한 키프레임별 유지 및 전환 지속 시간 컨트롤과 이름이 지정된 투어 설정을 저장하고 다시 불러오는 기능 추가
- [x] Story Map 플러그인에 여정 숨기기 토크, 인쇄 가능한 유인물의 부제 및 작성자 필드, 더 세련된 프레젠테이션을 위한 전용 시작 및 마무리 슬라이드 추가
- [x] 색상 선택기의 투명 채우기 및 윤곽선 옵션으로, 선택기를 벗어나지 않고 채우거나 스트로크 없이 피처를 스타일링 가능
- [x] 플러그인이 이제 maplibre-gl-raster 스택과 투명 컨트롤을 사용할 수 있어 외부 플러그인이 렌더링하고 구성할 수 있는 범위 확장([플러그인 API](#) 참조)
- [x] 팔레트 래스터의 내장 색상 테이블에서 범례가 자동으로 채워져, 수동 입력 없이 지도 색상과 일치
- [x] 프로젝트 홈과 소스에 빠르게 접근할 수 있도록 Help 메뉴에 Website(웹사이트) 및 GitHub 링크 추가

v1.10: I3S 씬 레이어, OGC API 벡터 타일, 로컬 NetCDF/HDF 불러오기

- [x] ArcGIS I3S 씬 레이어(Integrated Mesh 및 3D Object 레이어)를 데이터 소스로 추가, I3SLoader가 있는 Tile3DLayer를 통해 공유 deck.gl 인스턴스에서 3D로 스트리밍 및 렌더링
- [x] Add Data가 이제 OGC API - Tiles 벡터 타일 서비스를 원격 소스로 지원하므로, 표준 기반 벡터 타일 엔드포인트를 XYZ 및 ArcGIS 벡터 타일과 함께 추가 가능
- [x] 로컬 HDF5 및 NetCDF-4 파일을 디스크에서 직접 불러와, NetCDF/HDF 워크플로를 Cloud-Optimized 참조를 넘어 이미 컴퓨터에 있는 파일로 확장
- [x] GeoEditor 플러그인이 지도 뷰에 현재 표시된 벡터 피처를 편집기로 가져올 수 있어, 소스 데이터를 다시 가져오지 않고 보고 있는 것을 바로 편집 시작 가능

v2.0: 3D 지구본, 행성 지도 제작, 심볼로지 교환, 편집 가능한 소스 레이어

- [x] 어떤 지도 창이든 2D 지도와 카메라가 동기화되고 레이어 스택을 반영하는 **CesiumJS 3D 지구본** 뷰로 전환하여, 평면 워크스페이스 옆에 진정한 사실적 지구본을 추가(Cesium Ion 토큰은 선택 사항이며, 지형과 함께 래스터 형태가 없는 베이스맵의 폴백으로 Ion World Imagery를 추가)
- [x] OpenPlanetaryMap 베이스맵 세트(화성과 달)와 Web Mercator로 재투영된 수성, 금성, 갈릴레이 위성, 타이탄, 명왕성, 카론용 USGS Astrogeology 베이스맵을 사용한 행성 지도 제작, 반지름이 거리/면적/축척을 결정하는 프로젝트별 타원체, Layers 패널의 행성 전환기, 확장된 EOX Maps 카탈로그와 다크 모드를 인식하는 베이스맵 테마
- [x] 벡터 레이어 심볼로지를 OGC SLD, QGIS QML, Mapbox GL 스타일 JSON으로 가져오기 및 내보내기하여, 스타일이 GeoLibre, QGIS, Mapbox/MapLibre 생태계 사이를 왕복 가능
- [x] 벡터 레이어를 편집하고 변경 사항을 소스에 다시 기록, GeoPackage 및 GeoJSON 파일과 PostGIS 데이터베이스 테이블 지원
- [x] 라이브 구름 및 강수 레이더 오버레이(RainViewer)가 있는 새 Weather(날씨) 메뉴, Controls 메뉴의 Clouds(구름) 오버레이, 사실적인 조명을 위한 Google Earth 스타일 태양 위치 시뮬레이션

- [x] 더 풍부한 KML/KMZ 지원: GroundOverlay 이미지를 지도 오버레이로 렌더링(시간 태그가 있으면 Time Slider로 애니메이션)하고 내장 Collada(.dae) 3D 모델 표시
- [x] Z 좌표를 가진 벡터 레이어를 지면에 평평하게 펴는 대신 진정한 3D로 렌더링
- [x] 브라우저에서 직접 COG, WMS, XYZ 경계 상자 하위 집합을 추출하고, 임의의 HTTP COG에서 정규화 차 이 지수를 생성
- [x] 새 내장 플러그인: Mapillary 커버리지 및 거리 수준 이미지 뷰어, Historical Imagery(과거 영상) 패널, Elevation Profile(고도 단면) 도구, 그리고 Gridlines 오버레이용 UTM 동거/북거 격자 모드
- [x] Field Calculator(필드 계산기)가 지오메트리 길이와 면적을 계산할 수 있고, 속성 테이블이 Ctrl 및 Shift 클릭 다중 행 선택을 지원
- [x] Google Earth 스타일 카메라 초기화 키보드 단축키, View in Google Maps(Google Maps에서 보기) 및 View in Google Earth(Google Earth에서 보기) 동작, 수직 과장을 설정하는 더블클릭 지형 컨트롤
- [x] 좌표가 없는 구분 텍스트(CSV)를 독립 실행형 속성 테이블로 가져오기
- [x] 13개 로케일 카탈로그 모두 완성, 남아 있던 하드코딩된 패널 및 대화 상자 문자열을 번역 시스템으로 이전
- [x] AI 어시스턴트가 OS 환경 변수에서 제공자 API 키를 읽을 수 있고, 데스크톱 진단 네트워크 로그가 이제 네이티브 Tauri HTTP 요청을 캡처하고 실패한 `fetch()` 오류를 분류

v2.1: 데이터 소스 Browser 패널, 경로 애니메이션, 인브라우저 객체 탐지, 지도 녹화

- [x] 한 곳에서 데이터를 탐색하고 추가하는 QGIS 스타일 **Browser 패널**(브라우저 패널)(Data Source Manager): 지도 Services(서비스)와 Recent(최근) 항목 탐색, PostGIS 데이터베이스에 연결하여 스키마와 테이블 탐색, 로컬 파일 드릴다운, Favorites(즐거찾기) 저장 및 다시 열기, 서비스 종류별 New connection(새 연결) 추가, 키보드로 전체 트리 탐색
- [x] **경로 애니메이션**: 임의의 라인 레이어를 따라 마커를 애니메이션하고, 카메라 컨트롤로 3D에서 트랙을 따라가며, 전체 애니메이션을 MP4 비디오로 내보내기
- [x] 서버나 Python 없이 웹뷰에서 ONNX/YOLO 모델을 직접 실행하는 **인브라우저 객체 탐지**
- [x] 브라우저에서 바로 지도 캔버스(또는 그린 경계 상자)를 비디오 파일로 녹화
- [x] 지오태그 사진을 위한 진정한 네이티브 해상도 뷰어로, 전체 디테일 영상이 지도에서 선명하게 유지
- [x] Wikipedia 지식 카드: 지도에서 장소를 클릭하여 Wikipedia 요약과 정보 카드 표시
- [x] 9개 천체를 추가한 USGS 행성 베이스맵으로 행성 지도 제작 카탈로그 확장
- [x] 공개 항공 영상을 검색하는 새 OpenAerialMap 플러그인과 적절한 출처 표시가 있는 GEBCO 해양 수심 WMS 샘플
- [x] source.coop 공개 데이터를 탐색하는 새 Source Cooperative 플러그인: 카탈로그 검색, 제품 파일 탐색, GeoParquet, PMTiles, COG, GeoJSON, FlatGeobuf를 지도에 바로 추가(소스에서 스트리밍)하거나 다운로드
- [x] Whitebox 툴박스가 이제 플로팅 패널이며, Processing 하위 집합 도구가 현재 지도 뷰에서 경계 상자를 채울 수 있음
- [x] 축척 막대가 미터법, 야드파운드법, 해리 단위를 전환

- [x] 번들 플러그인 드롭인이 매니페스트에서 `activeByDefault` 를 설정할 수 있고, 배포 환경이 시작 대화 상자를 선택 해제할 수 있으며, 앱이 이제 기본적으로 Advanced(고급) 인터페이스를 사용하고 첫 실행 시 시작 대화 상자를 건너뛴다
- [x] 웹(Docker) 컨테이너를 위한 선택적 HTTP Basic Auth

v2.2: 스타일링 개편, 표현식 기반 필드와 라벨, 인쇄 아틀라스, 브라우저 네이티브 변환

- [x] 규칙별 심볼 속성, 축척 종속 표시, 중첩 규칙, 규칙별 토글을 갖춘 **규칙 기반 렌더러**로, 단일 레이어가 스타일링 규칙의 전체 계층을 가질 수 있고 `else` 규칙이 꺼져 있으면 어떤 규칙에도 일치하지 않는 항목을 숨김
- [x] 재사용 가능한 심볼, 색상 램프, 라벨 프리셋을 개인 라이브러리에 저장하고 프로젝트 간에 적용하는 **Style Manager**(스타일 관리자)
- [x] 반전 폴리곤 마스크, 화살표 및 마커 라인, 지오메트리 생성기(속성 기반 중심점 크기와 버퍼 거리 포함)를 다루는 심볼로지 팩과 마커 아이콘의 데이터 기반 비례 크기 조절
- [x] **다이어그램 심볼로지**: 피처의 속성에서 바로 파이, 도넛, 막대 차트를 피처 위에 그리기
- [x] 표현식 기반 라벨 속성, 배치 우선순위, 라벨 그리기 방식에 대한 완전한 제어를 갖춘 **데이터 정의 라벨링 엔진**
- [x] 필터, 라벨, 스타일링, 선택에 연결된 공유 **Expression Builder**(표현식 빌더) — 함수 참조, 검색 가능한 필드 목록, 실시간 미리보기, 재사용 가능한 변수
- [x] **가상 필드**: 기반 데이터가 변경되면 갱신되는 표현식 기반 계산 결과 단일 밴드 범주형 래스터를 위한 Raster Attribute Table(래스터 속성 테이블)
- [x] 구조화된 데이터 입력을 위한 편집 위젯, 유효성 검사 제약 조건, 조건부 필드 표시를 갖춘 **속성 양식 디자이너**
- [x] 레이어 속성에서 구성되는 **영구 속성 조인**으로, 테이블이 레이어의 피처를 보강하고 세션 간에 유지
- [x] **Select by Expression**(표현식으로 선택) 및 **Select by Location**(위치로 선택), 데이터가 갱신되면 다시 실행되는 DuckDB SQL 기반 라이브 쿼리 레이어
- [x] Print Layout의 **아틀라스 / 지도 시리즈** — 피처당 한 페이지를 생성하거나 강이나 트레일 같은 라인을 따라 균일한 페이지 시리즈를 생성 — 그리고 속성 테이블과 차트 블록을 페이지에 배치
- [x] COG, FlatGeobuf, Shapefile, GeoPackage의 브라우저 네이티브 포맷 변환과 백그라운드 워커에서 실행되는 Vector to PMTiles, Python 사이드카 불필요
- [x] **라이브 GPS 추적**: 움직이는 위치 마커, 기록된 트랙 로그, GPS 피드에서 바로 새 피처 디지털라이징
- [x] 유효성 검사, 지오메트리 수정, 토폴로지 규칙 검사를 위한 데이터 품질 도구로, 문제를 일으키기 전에 잘못된 지오메트리를 찾아 복구
- [x] 모든 도구 실행을 나열하고, 클릭 한 번으로 다시 실행하며, 동등한 Python 코드를 복사하는 **Processing History**(처리 기록) 패널
- [x] Timelapse(타임랩스) 모드가 이제 제공자 선택기와 범례를 갖추고 EOX Sentinel-2 cloudless 연간 베이스맵과 NASA GIBS 제공자(Landsat/WELD 및 MODIS 토지 피복)를 애니메이션
- [x] 데스크톱에 네이티브 HTTP를 위한 OS 신뢰 저장소 및 mTLS 클라이언트 인증서 지원, 디스크의 로컬 파일이 변경되면 자동 다시 불러오기, Esri File Geodatabase(`.gdb`) 레이어 지원 추가

- [x] Plugins > Web Services 아래의 새 Natural Earth 및 Source Cooperative 데이터 브라우저, Source Cooperative의 대용량 GeoParquet 열기 또는 스트리밍 포함
- [x] Measure 도구의 지형 인식 3D 측정, 비디오 녹화의 선택적 제목/출처 캡션과 지도 위 패널 캡처(HTML, 범례, 컬러바), 완전한 아랍어 오른쪽에서 왼쪽 지원과 함께 새 조지아어 로케일

v2.3: 지도 위 Legend 패널, iOS와 Google Play, GeoLens 카탈로그, 시공간 핫스팟

- [x] 표시된 레이어의 심볼로지에서 스스로 파생되는 지도 위의 자동 생성 **Legend 패널**(범례 패널) — 등급형, 범주형, 규칙 기반, 표현식 스타일링의 클래스별 행, 히트맵과 연속 래스터 컬러맵의 그라데이션 막대, 비례 심볼 크기 램프, 다이어그램 필드, Raster Attribute Table에서 읽은 토지 피복 라벨 — 항목 이름 변경, 숨기기, 순서 변경, 색상 사전에서 섹션 추가, 모서리 선택, 섹션 접기, 패널 크기 조절, 렌더링된 범례를 JSON으로 내보내기를 위한 편집 모드 제공, 모두 프로젝트와 함께 저장되고 Print Layout 범례와 공유
- [x] **Layers 패널의 심볼로지 견본**: 모든 행에 레이어 자체 스타일링에서 색을 가져온 점, 선, 사각형, 이미지 클리프가 표시되고 레이어가 숨겨지면 흐려지므로, 긴 레이어 스택을 한눈에 읽을 수 있음
- [x] 자체 호스팅 **GeoLens** 서버에 연결하고, 카탈로그를 검색하고, 데이터셋을 서명된 벡터 타일, OGC API Features GeoJSON, 또는 서버 렌더링 래스터 타일로 추가하는 새 **GeoLens 카탈로그 브라우저** 플러그인, 자동 타일 토큰 갱신과 각 데이터셋 페이지로 돌아가는 Metadata(메타데이터) 링크 제공
- [x] 처음부터 끝까지 스캐폴딩된 **iOS 지원** — Tauri iOS 구성, 위치 권한, 서명 및 TestFlight CI 워크플로, 게시 가이드. [iOS](#) 참조
- [x] **Android 빌드가 Google Play 준비 완료**: 영구적인 `org.geolibre.app` 패키지 id, API 레벨 36, CI에서 검증된 16 KB 페이지 크기 정렬, 아키텍처별 사이드로드 APK와 함께 게시되는 서명된 유니버설 App Bundle. [Android](#) 참조
- [x] **Emerging Hot Spot Analysis**(신흥 핫스팟 분석): 타임스탬프가 있는 포인트를 시공간 큐브로 집계하고, 시간 슬라이스별로 Getis-Ord Gi*를 실행하며, 모든 셀을 새로운, 강화, 지속, 감소, 산발, 진동, 과거 핫스팟 또는 콜드스팟으로 분류, 전적으로 브라우저에서 수행
- [x] Time Slider가 **모자이크 소스** — 날짜별 다수의 COG로 이루어진 MosaicJSON 또는 STAC 컬렉션 — 를 GPU 또는 WASM 엔진을 통해 전체 공간 모자이크로 렌더링하여 애니메이션하며, WASM 엔진은 메르카토르 뿐 아니라 지구본 투영에서도 동작
- [x] 레이어 메뉴에서 레이어 간 **레이어 스타일 복사 및 붙여넣기**로, 일련의 벡터나 래스터를 하나하나 다시 스타일링하지 않고 일관된 모양으로 통일
- [x] **여러 개의 이름이 지정된 AI 프로필**: 여러 제공자, 모델, 자격 증명 조합을 정의하고, 하나를 기본값으로 표시하며, 어시스턴트 패널에서 전환
- [x] `?tool=` URL 매개변수로 **모든 Whitebox 도구에 딥 링크**, Processing 대화 상자를 미리 선택된 상태로 열고 나머지 쿼리 매개변수에서 도구 양식을 미리 채우며, 변경한 설정으로 공유 가능한 링크를 만드는 Copy link(링크 복사) 버튼 제공
- [x] Add Data에 **구분 텍스트용 소스 CRS 필드**가 추가되어 투영된 동거 및 복거 열이 있는 CSV가 올바른 위치에 놓이고, **다중 레이어 GeoPackage용 레이어 선택기**로 피쳐 테이블이 가득한 컨테이너에서 요청한 것만 추가

- [x] 레이어 메타데이터 대화 상자가 GeoTIFF 헤더에서 읽은 래스터의 실제 지리참조 — CRS 및 EPSG 코드, CRS 단위의 픽셀 크기와 범위, 데이터 유형, nodata, 압축, 타일링, 오버뷰 — 를 크기 조절 가능한 대화 상자에 보고
- [x] 장소 검색 상자에 좌표를 십진도, DMS, 또는 DDM으로 입력하여 지오코드 왕복 없이 바로 이동
- [x] 새 Random extract(무작위 추출) 벡터 도구, 활성 레이어가 이제 프로젝트와 함께 영속화, Terrain(지형) 컨트롤을 켜면 3D 기복이 즉시 활성화, Logos(로고) 하위 메뉴가 MapLibre 로고와 함께 Maptoolkit 브랜딩 추가

v2.4: STAC 및 Earthdata 카탈로그, 자유 비행 카메라, 타일과 데이터 큐브용 타임라인, 라이브 임베드 API

- [x] STAC Index에서 공개 SpatioTemporal Asset Catalog를 발견하고, 정적 카탈로그와 STAC API 모두에 연결하며, 컬렉션의 항목을 검색하고, 시각화 가능한 모든 자산 — COG, GeoJSON 등 — 을 지도에 바로 추가하는 새 **STAC 카탈로그 브라우저** 플러그인으로, STAC 작업을 위해 더 이상 앱을 떠나 자산 URL을 손으로 복사할 필요가 없음
- [x] NASA의 EOSDIS ArcGIS 포털을 검색하고 그 영상, 지도, 피쳐 서비스와 게시된 웹 맵을 일급 레이어로 렌더링하는 Plugins > Web Services 아래의 새 **Earthdata GIS 브라우저**, 빈 타일을 반환하던 모자이크 픽셀 크기 제한을 자동으로 처리
- [x] Hub를 탐색하는 새 **Hugging Face** 패널: 데이터셋을 검색하거나 계정 이름을 지정하고, 저장소 폴더를 탐색하며, 각 포맷을 이미 처리하는 리더를 통해 벡터 및 래스터 파일을 지도에 추가 — 그리고 반대 방향으로, GeoLibre 안에서 데이터셋 저장소를 만들고 레이어를 업로드
- [x] **비행 시뮬레이터**: 목적지를 선언하고 스크립트된 flyTo를 지켜보는 대신, 키보드로 지형과 3D 레이어 위를 조종하는 연속적인 대화형 자유 비행 카메라
- [x] **Time Slider가 이제 타일 데이터를 애니메이션**: 벡터 타일, PMTiles, MBTiles를 타임라인에 바인딩하고 전체 범위에 걸쳐 애니메이션하며, 타임스탬프 필드는 데이터의 로컬 복사본 없이 라이브 타일 샘플에서 감지
- [x] Time Slider가 또한 범용 시간 어댑터를 통해 레이어의 **자체 내부 시간 차원**을 구동하므로, Zarr 스토어 같은 데이터 큐브가 고유한 시간 컨트롤을 갖는 대신 공유 타임라인에 참여하고, 바인딩은 프로젝트와 함께 저장
- [x] **Zarr에 진정한 Add Data 경로 제공**: 원격 스토어나 디스크의 폴더를 위한 Add Zarr Layer(Zarr 레이어 추가) 대화 상자, 원시 인덱스 대신 스토어의 실제 좌표 값을 제공하는 변수 및 차원 선택기, 이미 지도에 있는 레이어에 적용되는 Selector(선택자) 필드
- [x] WFS의 JSON 네이티브 후속인 **OGC API - Features** 컬렉션을 가지고 있는 어떤 URL — 랜딩 페이지, /collections, 전체 items URL — 에서든 벡터 레이어로 추가 가능
- [x] GeoLibre를 프레임에 담는 호스트 페이지를 위한 버전 관리된 **postMessage API**: 런타임에 프로젝트 불러오기, 카메라 이동, 피쳐 강조 표시, 처리 도구 열기, 그리고 ready, projectLoaded, selectionChanged, viewChanged, toolCompleted, serverFileWritten 이벤트 반환. 배포 환경이 신뢰할 수 있는 출처를 지정하지 않으면 꺼짐. **임베딩과 공유** 참조
- [x] **앱 외부의 Jupyter 클라이언트가 지도를 제어 가능**: VS Code의 Jupyter 확장, jupyter console, nbclient를 데스크톱 앱의 Jupyter 서버에 연결하면 이제 fly_to, add_geojson 등이 동작하는데, 전송이 노트북이 표시되는 방식이 아니라 커널의 서버를 따르기 때문

- [x] 선택한 해상도로 현재 뷰 위에 H3 격자를 렌더링하고, 셀을 식별하여 인덱스, 부모, 자식, 이웃, 중심을 검사하며, 격자나 선택 항목을 GeoJSON 또는 CSV로 내보내는 새 **H3 육각 격자 플러그인** — 그리고 장소 검색 상자에 **H3 셀 인덱스**를 16진수 문자열이나 64비트 정수로 입력하여 해당 셀을 프레임에 담고 윤곽선 표시
- [x] **지오태그 사진의 객체 탐지**: 지도 영상뿐 아니라 가져온 사진 레이어에 대해서도 인브라우저 ONNX/YOLO 모델 실행
- [x] AI 어시스턴트에 **패널 자체의 모델 선택기**, 모든 제공자에 대해 갱신된 모델 목록, 제공자 변경 후에도 유지되는 자격 증명, 화살표 키로 이전 프롬프트 불러오기 추가
- [x] **Dashboard 패널의 지표(KPI) 타일**: 개수, 합계, 평균, 최소, 최대, 중앙값 집계와 사용자 지정 접두사 및 접미사를 갖춘 큰 숫자 위젯, 기존 차트 위젯과 함께 제공
- [x] **GeoLens 데이터셋 제자리 편집**: GeoJSON으로 추가된 데이터셋을 GeoEditor로 다시 그리거나 속성 테이블에서 다시 입력할 수 있고, 플러그인의 Edits(편집) 섹션이 변경 사항을 보여주며 GeoLens 데이터셋에 다시 기록 — 추가, 이동, 삭제된 피처는 피처별 POST / PUT / PATCH / DELETE 호출이 되고, GeoLens가 할당한 행 id는 레이어에 다시 기록됨. 비공개 GeoLens 래스터도 이제 렌더링되며, API 키는 정확히 그 타일 URL에만 첨부
- [x] 플러그인이 addZarrLayer를 호출하여 자체 번들 대신 GeoLibre가 이미 제공하는 Zarr 렌더러를 사용하고, 사용자 지정 레이어에 자체 페인트 속성을 유지하여 Style 패널의 슬라이더가 실제로 동작하며, 자체 레이어 그룹을 노출 가능
- [x] GeoLibre가 **Google Play**에 등록되어, Android 사용자가 APK를 사이드로드하는 대신 한 번의 탭으로 서명된 자동 업데이트 빌드를 설치 가능([다운로드](#) 참조)
- [x] GeoLibre Desktop이 **Mac App Store**에 등록되어, Store를 통해 설치하고 업데이트하는 동일 앱의 샌드박스 빌드 제공. 그 편의를 위해 Python 사이드카 엔진, martin을 통한 서버 기반 PostgreSQL/PostGIS 레이어, 로컬 Jupyter 서버, Earth Engine 로그인, 외부 플러그인 설치를 포기하므로, Homebrew 및 DMG 빌드가 전체 기능 경로로 유지([다운로드](#) 참조)
- [x] Linux **ApplImage 릴리스가 이제 업데이트 정보와 .zsync를 포함**하므로, ApplImageUpdate, ApplImageLauncher, AppManager, AM이 앱을 업데이트하고 전체 다운로드 대신 변경된 블록만 전송 가능
- [x] 조지아어를 포함한 15개 비영어 로케일 모두 100% 커버리지로 복귀

v2.5: QGIS 및 ArcGIS Pro 프로젝트 가져오기, 이산 전역 격자, 초분광 큐브, 자체 호스팅 가능한 서버

- [x] **QGIS 프로젝트 가져오기**: .qgs 또는 .qgz를 열면 레이어, 중첩 그룹, 그룹 표시 여부, 레이어 순서, 스타일링, 저장된 지도 뷰가 GeoLibre에 재구성되며, 전부 아니면 전무의 실패 대신 건너뛰어야 했던 항목의 레이어별 보고서 제공. [프로젝트](#) 참조
- [x] **ArcGIS Pro 프로젝트 가져오기**: .aprx 및 .mapx 파일을 ArcPy 없이 CIM JSON에서 직접 읽어, 첫 번째 2D 지도의 범위, 로컬 벡터 및 GeoTIFF 레이어, 중첩 그룹, 표시 여부, 단순 심볼, 필드 라벨, 벡터 타일 포털 항목, 캐시된 지도 서비스를 복원. [프로젝트](#) 참조
- [x] **이산 전역 격자 시스템**이 Plugins > DGGS 아래의 세 가지 새 플러그인 — **A5, DGGRID, DGGAL** — 으로 도입되어, 선택하거나 자동으로 정한 해상도로 현재 뷰 위에 격자를 렌더링하고, 셀을 식별하여 id, 부모, 자식, 이웃, 중심을 읽으며, 격자나 선택 항목을 레이어로 지도에 추가하거나 내보내기

- [x] 이에 맞는 **DGGS 처리 도구**: 범위를 셀로 채우는 DGGS Generator, 포인트 레이어를 셀에 집계하는 DGGS Binning, 전체 자식 집합을 부모로 합치는 DGGS Compact
- [x] **초분광 NetCDF 및 HDF 격자가 단일 회색 밴드가 아닌 읽을 수 있는 데이터가 됨**: 로컬 격자는 Style 패널이 사용하는 것과 같은 카탈로그에서 브라우저 내 컬러맵이 적용되고, 추가 시 카메라가 레이어에 맞춰지며, 큐브는 파장으로 선택한 RGB 밴드 조합을 가짐
- [x] **3D 이미지 큐브 뷰**가 초분광 장면을 드래그 가능한 슬라이스 절단면이 있는 여섯 외부 면으로 렌더링하여 볼륨을 볼륨으로 읽을 수 있음 — 윈도우 및 스트라이드 읽기로 전체 EMIT 반사율 변수를 브라우저가 감당할 수 있는 범위 내로 유지
- [x] **클릭으로 분광 프로파일**: NetCDF 레이어의 픽셀을 식별하여 값을 읽고, 밴드 축을 따라 파장 대비 스펙트럼을 차트로 그리며, 최대 여섯 개의 샘플링 포인트를 비교 — 각 포인트는 차트 색상의 번호가 매겨진 점으로 지도에 표시 — PNG 또는 CSV로 내보내는 플로팅 드래그 가능 창 제공
- [x] **공유, 계정, 라이브 협업을 이제 자체 호스팅 가능**: 프로젝트 및 ID 계약이 문서화되었고, FastAPI 참조 구현과 하나의 세션 코어를 공유하는 순수 Node 협업 릴레이가 제공되며, 단일 적합성 스위치가 두 릴레이에 대해 실행되어 권한 동작이 어긋날 수 없음. [서버 API](#) 참조
- [x] 게시된 웹 이미지를 다시 빌드하기 위해 이미지를 포크하는 대신 `GEOLIBRE_SHARE_URL` 과 `GEOLIBRE_COLLAB_URL` 을 통해 **런타임에 해당 서버를 가리키도록** 할 수 있으며, `off` 는 Share와 Project Gallery(프로젝트 갤러리)를 완전히 제거하고 잘못된 값은 공개 호스팅 서비스에 조용히 프로젝트를 업로드 하는 대신 부팅을 거부
- [x] **추가 시 스마트 스타일링**: 새 벡터 레이어마다 정성적 팔레트에서 다음 미사용 색상을 가져오고 지배적인 지오메트리에 따라 크기가 정해지므로, Style 패널을 열기 전에도 새로 불러온 스타크를 읽을 수 있으며, 닫을 수 있는 **Style suggestions**(스타일 제안) 스트립이 레이어 자체 속성에서 파생된 최대 세 가지 원클릭 렌더러 제공
- [x] **벡터 타일 레이어를 타일 속성으로 분류 가능**: PMTiles, MBTiles, 원격 벡터 타일은 로컬 피처가 없으므로, Style 패널이 이제 뷰포트에 이미 불러온 것을 샘플링하여 필드 목록과 분류기 값을 채우고, 3D 돌출은 실제 높이 속성을 기본값으로 사용
- [x] 지도의 오른쪽 클릭 메뉴와 레이어 동작 메뉴의 **빠른 분석**: 클릭한 지점을 세 가지 거리로 버퍼하거나 그 지점에서 운전 및 도보 등시선을 생성하고, 전체 레이어를 버퍼, 중심점, 볼록 껍질, 경계 상자 처리하며, 기본값이 채워지고 모든 실행이 Processing History에 기록되어 다시 실행하거나 Python으로 복사 가능
- [x] 지도의 주석 — 텍스트, 화살표, 사각형·타원·자유형 강조, 핀 마커, 스티커 메모, 배치된 이미지 — 을 나열하는 **Elements 패널**(요소 패널)로, 캔버스에서 찾아다니는 대신 목록에서 각 항목을 관리하며, 배치된 이미지는 선택적으로 범위에 고정하여 뷰와 함께 크기 조절. [주석과 Elements 패널](#) 참조
- [x] **Layer Library**(레이어 라이브러리): 소스, 스타일, 라벨, 필터, 조인, 가상 필드, 속성 양식이 완전히 구성된 레이어를 저장한 뒤, Browser 패널의 My Data(내 데이터) 섹션에서 클릭 한 번으로 이후 어떤 프로젝트에든 다시 추가하고, 라이브러리를 팀과 공유할 JSON 번들로 내보내기
- [x] **프로젝트 복제, 템플릿, 사본 열기**: 프로젝트를 복제하고, 데이터는 제거하면서 베이스맵, 그룹, 스타일, 범례, 위젯, 레이어아웃을 유지하는 재사용 가능한 템플릿으로 저장하며, 갤러리의 어떤 항목이든 사본으로 열기
- [x] **라이브 레이어 연결이 프로젝트와 함께 영속화**: WFS, GeoJSON URL, Add Vector Layer URL 레이어의 새로 고침 주기, 마지막 동기화 시각, 마지막 오류, 실패 시 정책이 `connection` 레코드로 저장되므로, 다시 연 프로젝트가 일정에 따라 계속 새로 고침되고 Layers 패널이 각 라이브 레이어의 상태를 표시

- [x] Plugins > Web Services 아래의 **ArcGIS Hub, Socrata, CKAN**(Humanitarian Data Exchange)용 **공개 데이터 카탈로그 브라우저**: 키워드로 공개 데이터셋 카탈로그를 검색하고, ArcGIS Hub 검색을 현재 지도 영역으로 제한하며, 결과를 페이지 단위로 넘기고, 데이터셋을 지도에 추가하거나 다운로드
- [x] **가져올 때 CSV 지오코딩**: Delimited Text(구분 텍스트) 패널에 선택한 열을 연결하고 프로젝트의 제공자를 통해 각 행을 지오코딩하는 Addresses(주소) 모드가 추가되며, 일치하지 않는 행은 조용히 버려지는 대신 null 지오메트리 피쳐로 유지되어 속성 테이블에서 보이고 수정 가능
- [x] API 키 없이 스페인 공식 주소 데이터를 대상으로 정방향 및 역방향 지오코딩을 수행하는 **CartoCiudad(IGN España) 지오코딩 제공자**
- [x] 스타일이 적용된 레이어를 Google Earth로 다시 내보낼 수 있는 **KML 및 KMZ 레이어 내보내기**, 그리고 전체를 한꺼번에 불러오는 대신 NetworkLink 기반 타일 피라미드를 지도에 제공하는 **KML Super-Overlay 가져오기**
- [x] 범주형 필드를 칩 집합으로 바꾸고 같은 레이어에 바인딩된 다른 모든 위젯을 단일 또는 다중 선택 모드로 교차 필터링하는 **Dashboard 선택기 위젯**
- [x] **GPS Tracking이 Web Serial 또는 Web Bluetooth로 외부 NMEA 수신기를 읽음**, 보드 레이트 선택기와 라이브 문장 및 측위 카운터 제공, 기기 자체 위치 정보와 함께 사용. [GPS 추적](#) 참조
- [x] Processing 대화 상자가 사용자에게 대신 변환하라고 요구하지 않음: WGS84 벡터 레이어의 거리 매개변수는 도구가 실제로 읽는 도 단위로 변환하는 **미터법 단위 선택기**를 갖고, EPSG 매개변수는 Geographic과 Projected로 묶인 **검색 가능한 CRS 카탈로그**를 가지며, Download OSM Vector의 네 가지 경계 숫자는 Use map extent(지도 범위 사용)와 Draw on map(지도에 그리기)이 있는 하나의 **Area of interest**(관심 영역) 컨트롤이 됨
- [x] **GeoLibre Desktop이 Mac App Store에 등록**, Store를 통해 설치하고 업데이트하는 동일 앱의 샌드박스 빌드. [다운로드](#) 참조
- [x] **GeoLibre가 App Store에 등록**, Tauri v2 모바일을 통해 동일 코드베이스에서 빌드되어 App Store 심사를 통과한 네이티브 iPhone 및 iPad 앱으로, iOS 사용자가 한 번의 탭으로 설치하고 업데이트. [iOS](#) 참조
- [x] geolibre Python 패키지에 **leafmap 동등 헬퍼** 추가: 로컬 래스터, 마커 및 클러스터, 단계구분도 레이어, `split_map`, `add_legend`, `add_colorbar`, 선택 및 그린 피쳐의 타입 지정 읽기, `to_html` 내보내기. [Python 패키지](#) 참조
- [x] **태국어**가 제공 로케일에 합류하여 카탈로그가 영어 외 16개 언어로 확대
- [x] 벡터 내보내기가 속성 테이블에서 숨긴 필드를 존중하고, 플러그인이 `queryZarrLayer` 로 네이티브 Zarr 레이어의 픽셀을 읽을 수 있으며, 레이어가 전체 선택 단위로 그룹에 이동하고, 인쇄 범위를 터치로 그릴 수 있음

v2.6: 데이터 및 스타일 딥 링크, AI 클라이언트용 MCP 서버, COG 기반 지형, 자체 디자인 시스템

- [x] **딥 링크가 데이터 자체를 전달 가능**: `?data=` 는 프로젝트 파일을 먼저 게시하지 않고도 호스팅된 GeoJSON, GeoParquet, COG, PMTiles, FeatureCollection을 반환하는 REST 엔드포인트, GeoJSON 파일의 ZIP을 열고, `?style=` 은 그 옆에 벡터 또는 래스터 심볼로지를 적용하며, 간결한 GeoLibre 스타일 내보내기로 이미 지도에 있는 레이어에서 해당 스타일 파일을 쉽게 생성. [임베딩과 공유](#) 참조

- [x] **ArcGIS MapServer 및 ImageServer가 Add Data에 합류:** 렌더링된 이미지를 제공하는 두 서비스 유형이 일반 래스터 레이어로 불러와지므로, 불투명도, Style 패널의 밝기·대비·채도 컨트롤, 순서 변경, 프로젝트 저장 및 다시 불러오기가 모두 동작하며, 표준 Web Mercator 체계로 구축된 경우 서비스 자체 융합 캐시에서, 그렇지 않으면 `/export` 또는 `/exportImage` 에서 타일을 가져옴. 하위 레이어는 id를 입력하는 대신 서비스 자체 목록에서 탐색하고 선택하며, ImageServer 렌더링 규칙은 필드 하나 거리에 있음
- [x] **Cloud Optimized GeoTIFF가 3D 지형을 구동 가능:** 지형은 전역 AWS Terrarium 타일에 고정되어 있어 고해상도 로컬 고도가 음영기록을 형성할 수 없었습니다. 이제 Terrain(지형) 설정이 HTTP 또는 로컬 파일의 단일 밴드 EPSG:3857 또는 EPSG:4326 COG를 허용하며, 범위 단위로 읽어 사용자 지정 MapLibre 프로토콜을 통해 Terrarium 타일로 인코딩
- [x] **중국부터 시작하는 지역 베이스맵:** GeoLibre의 기본값은 중국 본토 외부에 호스팅되어 그 안에 존재하지 않으므로, 그곳에서는 선택기가 불러올 수 있는 것을 전혀 제공하지 못했습니다. New Project와 Change Basemap(베이스맵 변경) 패널 모두의 접힌 **Regional**(지역) 섹션이 키 없는 중국 베이스맵 다섯 가지(高德地图, 高德卫星, 高德混合, 腾讯地图, 腾讯深色)를 추가하고, Basemaps 컨트롤 플러그인에는 같은 제공자와 Tianditu가 추가되며, 이 섹션은 코어의 테이블로 구동되므로 두 번째 지역은 또 다른 대화 상자 변경이 아닌 항목 하나로 추가 가능
- [x] **선택한 베이스맵이 다시 돌아올 때의 베이스맵:** 빈 시작 워크스페이스는 마지막으로 선택한 베이스맵에서 시작하는 반면, 프로젝트 자체 베이스맵은 권위를 유지하고 행성체는 타원체 동작을 유지
- [x] **협업 스냅샷의 이식성:** 스냅샷이 협업자가 읽을 수 없는 로컬 파일 참조와 컨트롤 관리 벡터 데이터를 유지했기 때문에 게스트가 세션에 참여하면 빈 지도를 볼 수 있었습니다. 이제 공유 레이어가 피처를 포함하고, 호스트가 편집을 기다리는 대신 첫 스냅샷을 시도하며, 게스트가 호스트의 뷰포트에 도착하고, 세션 Copy(복사) 버튼이 참여 가능한 URL을 생성. [협업 참조](#)
- [x] **지도 댓글의 탐색 가능화:** 지도의 핀을 클릭하면 해당 카드가 드러나고 강조 표시되며 스크롤되고, 패널은 발견하기 쉽도록 Style 레일에 접힌 상태로 제공되며, `c` 는 명령 팔레트에서 새 댓글을 배치하고, 댓글은 프로젝트와 함께 저장. [검토 댓글 참조](#)
- [x] **프로젝트 작성을 위한 MCP 서버** `geolibre-mcp`: AI 클라이언트가 브라우저도 실행 중인 앱도 없이 실제 `.geolibre.json` 파일을 작성하게 하는 헤드리스 stdio 서버로, Python 패키지가 이미 사용하는 것과 같은 빌더로 구성하므로 작성한 모든 것이 데스크톱 앱, 웹 앱, Jupyter 위젯에서 변경 없이 열림. 모든 읽기와 쓰기는 `--root` 또는 `GEOLIBRE_MCP_ROOTS` 로 지정한 루트로 제한. [MCP 서버 참조](#)
- [x] **Python 패키지가 브라우저 왕복 없이 레이어를 관리하고 카메라를 이동:** 레이어 순서 변경, 복제, 이름 변경, 제거, 속성 값 읽기, 지도 프레임링을 `Map` 과 `Layer` 의 순수 프로젝트 변경으로 수행하며, 위젯을 표시하지 않는 스크립트를 위해 작성 헬퍼를 내보내고, 노트북 셀이 출력할 수 있는 모든 것에서 자격 증명을 제거. [Python 패키지 참조](#)
- [x] **Jupyter 위젯에서 Whitebox 도구 실행:** `list_whitebox_tools` 와 `run_whitebox_tool` 이 카메라와 클라이언트 측 알고리즘이 이미 사용하는 같은 채널로 번들 WASM 카탈로그에 접근하여, `Layer` 핸들을 레이어 id로 해석하고 벡터 및 래스터 출력을 모두 지도에 다시 추가하므로, 지형 및 래스터 작업을 위해 더 이상 노트북을 떠나 UI로 갈 필요가 없음
- [x] **토폴로지 폴리곤 디지털라이징으로,** 이웃에 맞대어 그린 폴리곤이 슬리버나 겹침을 남기는 대신 그 변을 공유

- [x] **Print Layout에 교차 페이지 필터 추가로**, 속성 테이블 블록이 페이지 안에 완전히 들어오는 피쳐만이 아니라 페이지에 닿는 모든 피쳐를 나열할 수 있으며, 날짜 변경선을 가로지르거나 원점에서 여러 세계 복사본 만큼 이동한 뷰에서도 올바르게 동작
- [x] **스토리 맵 PDF 유인물에 클릭 가능한 위치 마커 추가**: 각 챕터 페이지에 챕터 카메라의 중심 좌표에 끝이 놓인 핀과 그 좌표를 Google Maps에서 여는 PDF 링크가 스토리의 마커 색상으로 그려져, 인쇄된 유인물의 독자가 그 장소로 찾아갈 수 있음
- [x] **기성 shadcn을 대체하는 앱 자체의 디자인 시스템**: Windows의 Segoe UI, Linux의 Roboto, macOS의 SF 대신 어디서나 인터페이스가 같게 읽히도록 자체 호스팅한 IBM Plex Sans와 IBM Plex Mono; 패널, 대화 상자, 메뉴가 모두 거의 동일한 색상 하나를 공유하던 것을 대체하는 진정한 다크 모드 고도 사다리; 다크 모드에서 사실상 보이지 않던 것을 대체하는 테마 인식 그림자; 그리고 열한 개의 어긋난 복사본을 대체하는 지도 위에 떠 있는 크롬을 위한 단일 map-glass 처리
- [x] **임베드가 지도를 앞세울 수 있음**: `panels=collapsed` 레이아웃은 Layers 및 Style 아이콘 레일에 접근할 수 있게 유지하면서 두 패널을 접힌 상태로 시작하고, 상단 툴바를 완전히 숨겨 집중된 뷰어를 만들 수 있어, 패널 열림과 패널 없음 사이의 중간 지점 제공. [임베딩과 공유](#) 참조
- [x] 개별 로그인에 필요한 호스팅 배포를 위한 **선택적 Clerk 접근 게이트**, 선택적 대기 목록 화면 포함, 필요할 때 로드되며 기본 PWA 프리캐시에서 제외. 게이트는 클라이언트가 제어하는 쿼리 매개변수가 아니라 빌드 대상에 의해 결정되며, 공개, 네이티브, 임베드 빌드는 변경 없음. [시작하기](#) 참조
- [x] 신뢰할 수 없는 서드파티 호스트에서 로드할 수 없는 배포를 위한 **외부 CDN 없는 빌드**: `GEOLIBRE_NO_EXTERNAL_CDN=1` 은 GeoLibre가 제어하는 CDN 참조를 제거하고, PGLite 및 CereusDB 엔진을 제거하는 대신 빌드에 벤더링하며, 원격 호스트가 진정으로 필요한 소수의 기능이 실행 끝에서 실패하는 대신 미리 알리도록 함. [자체 호스팅](#) 참조
- [x] **외부 플러그인이 레이어를 읽기 전용으로 쿼리 가능**, 소스를 직접 다시 가져와 파싱하는 대신 호스트를 통해 레이어가 보유한 피쳐를 읽음. [플러그인 API](#) 참조
- [x] **페르시아어와 베트남어가 제공 로케일에 합류**하여 카탈로그가 영어 외 18개 언어로 확대, 페르시아어는 Vazirmatn으로 조판되고 그 폰트의 아랍 문자 하위 집합만 포함되어 다른 로케일이 비용을 치르지 않음
- [x] 레이어를 선택해도 더 이상 지도 위로 Style 패널이 튀어나오지 않고, 대신 레이어 메뉴의 명시적 **Open Style panel**(스타일 패널 열기) 항목에서 펼쳐짐

v2.7: 비주얼 Model Builder, 모든 종류의 STAC 자산, Chrome 데이터 열기 확장, 그리기로 선택

- [x] **처리 워크플로를 캔버스에 그림**, ArcGIS Model Builder 스타일: 도구를 노드로 배치하고, 출력을 다음 도구의 입력에 연결하며, 그래프를 하나의 작업으로 다시 실행되는 모델로 저장하고, 무엇이든 실행하기 전에 전체 그래프를 검증(순환, 누락된 연결, 지어낸 매개변수, 숫자 자리의 문자열)
- [x] **AI 어시스턴트가 평이한 영어 설명으로 모델을 작성**하고 실행 전 검토를 위해 캔버스에 열며, 캔버스 자체와 같은 팔레트를 사용하므로 함몰 채우기에서 유량 누적, 하천 추출로 이어지는 래스터 체인이 유효한 모델이 됨. 저장되지 않은 캔버스 작업이나 아직 진행 중인 실행을 대체하기 전에 확인을 요청. [AI 어시스턴트](#) 참조
- [x] **모델이 실행 가능한 Python 스크립트로 복사**되므로, 포인트 앤 클릭으로 만든 워크플로가 캔버스에서 멈추지 않고 Notebook 패널이나 JupyterLite에서 이어짐

- [x] **STAC 브라우저가 앱이 그릴 수 있는 모든 자산 유형을 추가:** PMTiles 아카이브, GeoParquet 테이블, 변수 선택기가 있는 Zarr 스토어가 이미 불러오던 COG에 합류하고, Icechunk 저장소는 자체 매니페스트를 통해 읽히며 저장소만이 아니라 브랜치로 키가 지정되고, Azure 호스팅 자산은 추가 시 서명되어 비공개 Planetary Computer 컨테이너가 공개 컨테이너처럼 열림
- [x] **정적 STAC 카탈로그를 검색뿐 아니라 트리로 탐색 가능:** 컬렉션과 하위 컬렉션을 따라가고, 폴더를 열어 읽으며, 선택한 항목에서 검색을 시작하고, 해당 컬렉션의 자체 범위로 이동
- [x] **지도에 그려서 피쳐 선택:** QGIS 스타일 클릭, 사각형, 폴리곤, 자유형, 반경 제스처, Shift와 Alt로 결과를 기존 선택과 결합, 옆에 놓인 Clear selection(선택 해제), 그래서 몇 개의 피쳐를 고르는 것이 표현식으로 설명하는 대신 가리키는 일이 됨
- [x] Chrome Web Store의 **Chrome 확장 Open data in GeoLibre**(GeoLibre에서 데이터 열기)가 보고 있는 페이지의 데이터셋 링크와 지도 서비스(WMS, WMTS, WFS, OGC API - Features, ArcGIS Feature Services, XYZ/TMS, 벡터 타일, 임베드된 지도와 원시 응답 안에서 감지된 서비스 포함)를 수집하고 선택한 것들을 하나의 GeoLibre 지도에 함께 열기
- [x] **Apache Iceberg 테이블을 벡터 레이어로 불러오기,** DuckDB의 iceberg 및 spatial 확장을 통해 테이블의 메타데이터 위치나 연결된 REST 카탈로그에서 브라우저 내 읽기, 스캔 전에 매니페스트에서 테이블의 실제 행 수를 보고하고, 스키마에서 지오메트리 열의 CRS를 가져오며, 선택적 SQL 상자가 렌더링할 행을 결정
- [x] **클라이언트 측 스위트에 없던 세 가지 벡터 도구:** 속성 스키마를 합치고 각 피쳐의 소스 레이어를 기록할 수 있는 다중 레이어 매개변수 선택기를 갖춘 Merge layers(레이어 병합); 모든 정점을 부분 및 정점 인덱스를 담은 포인트로 바꾸는 Extract vertices(정점 추출); 라인과 폴리곤 경계를 고정 간격으로 측지선적으로 따라가는 Points along geometry(지오메트리를 따라 포인트)
- [x] **인코딩된 폴리라인이 일급 포맷이 됨:** Google 및 OSRM 정밀도 5와 Valhalla 및 Mapbox 정밀도 6을 위한 코덱, Add Data의 대화형 미리보기 및 좌표 검사기, 사용자 지정 구분 기호를 사용한 배치 파싱, 처리 도구, 두 정밀도 중 하나로 레이어 내보내기, Python 패키지의 대응 지원
- [x] **공유 프로젝트 및 협업 세션의 활동 로그:** 프로젝트 소유자가 공유 프로젝트를 누가 열고 편집했는지 읽을 수 있고, 세션 호스트가 세션 로그를 다운로드할 수 있으며, 항목 수와 저장 바이트 모두 제한되고 URL 쿼리가 아닌 bearer 토큰으로 읽음. [서버 API](#) 참조
- [x] **벡터 레이어의 자동 편집자 추적:** 피쳐가 추가되고 변경될 때 created-by, created-at, edited-by, edited-at 필드가 스스로 채워지며, 레이어별로 구성되고 프로젝트와 함께 저장. [프로젝트 포맷](#) 참조
- [x] **건물 매싱 작성:** 바닥 면적을 스케치하고 높이를 부여하면 돌출된 매스를 얻으며, 매싱 피쳐가 생기고 사라짐에 따라 돌출 스타일이 자동으로 관리되고 일반 폴리곤 스케치는 평평하게 유지
- [x] **Elevation Profile 플러그인이 선택한 라인 피쳐의 단면을 그림으로,** 이미 지도에 있는 경로를 손으로 다시 그리지 않고 차트화
- [x] **플러그인과 처리 도구의 번역 가능화:** 플러그인 표시 이름이 한 곳에서 해석되므로 플러그인이 Plugins 메뉴에서는 번역되고 명령 팔레트에서는 영어로 읽히는 일이 더 이상 없으며, 모든 레지스트리 도구 이름, 설명, 그룹 라벨, 매개변수, 선택 옵션이 번역자가 작업할 수 있도록 영어 카탈로그에 생성. [국제화](#) 참조
- [x] **지오메트리 열이 둘 이상인 PostGIS 테이블이 더 이상 추측이 아님:** 등록된 모든 지오메트리 열이 Add Data에 제공되고, PostGIS 카탈로그에 대해 검증되며, 읽기, 저장, 새로 고침을 거쳐 유지

- [x] **공유 링크가 프레젠테이션 설정을 전달 가능:** `settingsUrl=` 이 첫 렌더링 전에 언어, 레이아웃, 강조 테마, UI 프로필을 불러오며, 링크가 자격 증명, 플러그인, 로컬 경로를 주입할 수 없도록 프레젠테이션 필드로 제한되고, 사용자가 저장한 것을 대체하는 대신 해당 페이지에서만 지속. [임베딩과 공유](#) 참조
- [x] **지오메트리 생성기의 속성 기반화:** 파생된 중심점이 비례 심볼로서 필드에 따라 크기가 조절되고, 각 피처의 버퍼가 입력한 숫자 하나 대신 열에서 거리를 가져오며, QGIS의 데이터 정의 재정의에 따름
- [x] **Excel 워크북을 X 및 Y 열에서 포인트 레이어로 가져오기,** 워크시트 선택기 포함
- [x] **Layers 패널 검색 상자가 지도 자체 데이터를 검색:** 입력한 쿼리가 지오코더와 함께 불러온 벡터 레이어의 속성과 대조되고, 로컬에서 실행되어 네트워크가 응답하기 전에 데이터 일치 항목이 나타나며, 장소 결과 위에 레이어별로 묶이고 정확 일치, 접두사, 포함 순으로 순위가 매겨지며, 피처를 선택하면 해당 레이어에서 선택되고 속성 테이블의 강조 표시와 함께 이동
- [x] **적합성 및 지수 분석을 위한 복합 점수 빌드:** 둘 이상의 숫자 필드를 정규화(최소-최대, z-점수, 순위, 분위수)하고, 가중치를 부여하며, 가중 산술 평균 또는 기하 평균으로 결합하고, 레이어가 즉시 스타일링되는 하나의 지수(기본 0-100, 요청 시 0-1)를 기록하며, 값이 없는 피처의 처리 방식은 조용한 기본값이 아닌 명시적 선택
- [x] 모든 레이어 카드의 Style 패널 버튼, 선택한 피처로 범위가 제한된 필드 통계, COPC로 클립 및 다운로드 되는 USGS LiDAR, 서비스 자체 목록에서 탐색하는 ArcGIS ImageServer 래스터 함수, 여러 URL을 한 번에 허용하는 `?data=`

v2.8: 팝업 디자이너, 빠른 필터, 레이어별 권한, npm에 게시된 패키지

- [x] **클릭과 호버 시 뷰어가 보는 것을 작성자가 결정:** Style 패널의 레이어별 Popup(팝업) 섹션이 Identify 팝업에 표시할 필드, 순서, 라벨, 숫자·날짜·링크·이미지 서식, 제목, 호버 툴팁이 포인터를 따라갈지 여부를 선택하므로, 공유 지도, `layout=viewer` 임베드, 스토리 맵 챕터가 독자에게 조인 흔적, 편집자 추적 열, 내부 id를 더 이상 보여주지 않음. [프로젝트 포맷](#) 참조
- [x] **빠른 필터가 표현식 대신 컨트롤로 레이어에 답함:** 필드를 고르면 GeoLibre가 값을 프로파일링하여 맞는 컨트롤을 제공 — 범주형 필드에는 값별 개수가 있는 체크박스, 숫자 필드에는 입력 가능한 경계가 있는 범위 슬라이더, 타임스탬프에는 날짜 범위, 그 외에는 텍스트 일치 — 컴파일된 표현식이 아닌 컨트롤 상태가 저장되므로 필터를 언제든지 다시 열어 변경 가능. 필터링된 레이어는 팔레기 아이콘으로 표시되고 한 번의 동작으로 해제되며, 컨트롤은 읽기 전용 뷰어 레이아웃에서도 렌더링되고, 호스트 페이지의 `setFilter`, Time Slider의 창, 사용자 자신의 필터가 이제 서로를 덮어쓰는 대신 함께 레이어를 줍힘
- [x] **레이어별 기능:** 레이어를 읽기 전용으로 표시하거나 피처 생성, 갱신, 삭제로 제한할 수 있으며 UI와 백엔드 모두 이를 적용하므로, 공유 프로젝트의 참조 레이어를 공유받은 사람들이 편집할 수 없음
- [x] **레이어별 지도 제작 혼합 모드:** 음영기복 위에 색상이나 영상을 겹칠 때 두 레이어를 결합하는 대신 희석시키는 불투명도 감소 대신 Multiply(곱하기)를 사용. MapLibre는 레이어별 혼합 API를 노출하지 않으므로, 렌더 루프를 감싸 한 레이어가 그려지는 동안 GPU 혼합 상태를 설정하여 같은 캔버스의 올바른 z 위치에서 혼합하며, 채우기와 선은 하나의 표면으로 합성하고 라벨은 제외하여 지명이 읽기 쉽게 유지
- [x] **@geolibre/core 와 @geolibre/map 이 npm에 게시되어,** 다른 애플리케이션이 모노레포를 벤더링하지 않고 GeoLibre의 스토어, 프로젝트 스키마, MapLibre 레이어 동기화 엔진 위에 구축 가능. [유지 관리](#) 참조

- [x] 앱 외부에서 GeoLibre를 제어하기 위한 에이전트 스킴: `skills/geolibre/` 는 SKILL.md와 MCP 도구 표면, Python API, 손으로 작성한 프로젝트 JSON, 베이스맵/램프/포맷 카탈로그에 대한 점진적 공개 참조로 구성되어, 외부 에이전트가 어떤 진입점에 어떤 순서로 접근해야 하는지 알 수 있음. 그 복사본은 실제 레지스트리에 대한 테스트로 보호. [에이전트 스킴](#) 참조
- [x] 대화형 SamGeo 분할, 텍스트, 포인트, 상자, 자동 모드를 갖춘 도킹 가능한 SAM3 워크플로우로, 결과를 WGS84로 재투영하여 생성된 폴리곤이 영상이 있는 곳에 놓임
- [x] Python 패키지가 Earth Engine과 xarray를 지원: `add_ee_layer` 가 인증된 Earth Engine 이미지, 컬렉션, 피쳐 컬렉션, 피쳐를 geemap 스타일로 복원 가능한 타일 레이어로 바꾸고, `DataArray` 또는 `Dataset` 은 브라우저에서 직접 렌더링되는 세션 범위 COG로 구체화. [Python API](#) 참조
- [x] 모든 제공 로케일이 다시 완성, 세 번에 걸쳐: Whitebox 툴박스의 1066개 도구 이름, 45개 하위 범주 라벨, 잠금 해제된 도구 매개변수가 카탈로그에 생성되어 중국어 간체로 번역되고, SamGeo 패널, Whitebox 작업 문자열, 언어 팩 관리자가 18개 로케일 전체에 채워졌으며, 이후의 점검으로 모든 비영어 카탈로그에서 어긋났던 136개 키를 마감. [국제화](#) 참조
- [x] 타일 아카이브가 소스 레이어의 그룹으로 열림으로, 다중 레이어 PMTiles 파일이 하나의 불투명한 행 대신 이름이 지정되고 개별적으로 스타일링 가능한 레이어들의 폴더 하나로 들어옴
- [x] 베이스맵이 나열될 뿐 아니라 미리보기됨: 각 래스터 항목은 z=2 타일을 표시하고 각 키 없는 스타일 항목은 배경색을 표시한 뒤 오프스크린 MapLibre 스냅샷을 표시하므로, 적용 전에 베이스맵을 구별 가능
- [x] Settings의 구성 가능한 시작 지도 뷰로, 배포 환경이나 개인 설치가 세계 전체가 아닌 사용자가 작업하는 곳에서 열림
- [x] GeoEditor 스케치를 프로젝트 레이어로 내보내기, 스타일, 불투명도, 표시 여부를 가지고 가므로, 그린 작업이 다시 그리거나 다시 가져오지 않고 편집기를 떠남
- [x] KML 폴더 구조가 가져오기 후에도 유지: Folder 안에 있는 플레이스마크는 폴더당 하나의 레이어가 되고, 나머지는 파일별로 병합된 채 유지되며, 가져오기가 결합된 범위에 프레임을 맞춤
- [x] 릴리스 다운로드가 GitHub Pages로 미러링되고 `web.geolibre.app` 이 그곳으로 이동, Model Builder 패널이 지도 캔버스 전체로 최대화, 주식 툴바가 접힘, 호스팅 사이트 분석이 빌드 시 옵트인이 됨(데스크톱, Jupyter, Docker, 포크 빌드에는 절대 포함되지 않음 — [개인정보 보호](#) 참조), 그리고 문서에 [비디오 튜토리얼](#) 페이지와 실행 중인 앱에 대해 검증한 사용자 가이드 추가

v2.9: 토큰 없는 3D 지구본, 키오스크 및 교실 배포, 도킹된 서비스 패널, 키 없는 전역 고도

- [x] 3D 지구본에 더 이상 Cesium Ion 토큰이 필요 없으며, 마침내 프로젝트 자체 베이스맵을 그립니다. 이창은 토큰이 구성되지 않으면 스스로 숨겨지고 베이스맵을 완전히 무시했으므로, 지구본과 함께 저장된 프로젝트가 평면 지도로 다시 열리고 다크 테마 프로젝트가 구석에 밝은 지구본을 유지했습니다. 행성 및 지역 베이스맵은 래스터 타일 세트로 넘어가고, GeoLibre의 벡터 스타일은 톤이 맞는 키 없는 래스터 베이스맵에 매핑되며, 빈 베이스맵은 맨 타원체를 그립니다. 이제 토큰은 관문이 아닌 업그레이드로, Cesium World Terrain과 Ion 세계 영상을 추가

- [x] **두 창이 마침내 축척에 합의:** 지구본 카메라가 타원체가 아닌 지형 표면을 조준하므로, 고지대 위의 뷰가 2D 쌍보다 두 배 이상 가깝게 렌더링되는 일이 멈춤. 지구본은 또한 `cesium` 배럴 대신 맨 `CesiumWidget` 을 구축하고 `@cesium/engine` 을 직접 가져와, 다시 끄기만 하던 열 개의 위젯과 함께 달려오던 Knockout 의존성을 제거하고, 이제 처음으로 자동화된 테스트 커버리지를 가짐
- [x] **키오스크 및 교실 배포가 앱을 잠글 수 있음:** `@geolibre/core` 의 기능 모델이 빌드가 허용하는 작업을 정의하고, UI, 명령 팔레트, 키보드 단축키 레이어, 임베드 API가 모두 이를 적용하므로, 거부된 동작에는 메뉴, 단축키, 호스트 페이지 어디서든 접근 불가. 거부는 컨트롤을 조용히 비활성화하는 대신 인라인으로 스스로를 설명하고, v2.8의 레이어별 권한이 이제 하나의 모델 아래에 놓임. 처리, 사이드카 도구, 프로젝트 편집, 데이터 로드, 플러그인, 설정이 각각 독립적으로 게이트됨
- [x] **계정 없는 고도 데이터:** 키 없는 전역 DEM 다운로드가 공개 AWS Terrain Tiles를 현재 지도 뷰나 지도에 그린 경계 상자에 맞춰 모자이크하고 클립하며, Processing > GeoLibre Toolbox > Raster에서 바로 실행되고, 얻어야 할 API 키가 없으며, 도구가 닫히면 진행 중인 다운로드가 취소됨
- [x] **Web Services 플러그인이 다른 모든 것처럼 도킹됩니다.** 처음 다섯 개가 MapLibre 컨트롤 수명 주기와 레이어 동기화를 유지한 채 공유 도킹 가능 패널 호스트로 이동하여, 카탈로그 브라우저가 지도 위에 떠 있는 대신 Layers 및 Style 패널 옆에서 크기 조절, 접기, 나란히 배치됨
- [x] Plugins > Web Services 아래의 **두 가지 새 공개 데이터 카탈로그:** 공개 카탈로그가 미리 선택된 STAC 브라우저를 통해 Planet의 재난 릴리스를 여는 **Planet Open Data**, 그리고 이벤트와 단계로 필터링하고 경계 상자를 그리며 자체 COG 렌더링 엔진(GPU, WebAssembly, TiTiler)을 고르는 사전·사후 이벤트 영상용 STAC 탐색기 **Vantor Open Data**. 패널 상태를 공유하는 카탈로그 플러그인은 이제 상호 배타적이며, 전환 실패 시 밀려난 플러그인을 복원
- [x] **Identify가 표시된 모든 레이어에 한 번에 답함**, 레이어를 먼저 고르고 다시 클릭하게 하는 대신 표시된 모든 쿼리 가능 레이어의 벡터 피처와 래스터 픽셀 값을 하나의 그룹화된 확장 가능 팝업으로 모음
- [x] **주석 레이어의 완전한 편집 가능화:** Elements 패널이 레이어가 보유한 항목을 나열하고, 주석이 이동하면 이미지 오버레이 모서리가 동기화된 상태를 유지하며, 표시되는 스타일 컨트롤이 선택한 요소에 적용되는 것들임
- [x] **GeoParquet의 geo 메타데이터를 전부 읽음** — `version`, `primary_column`, `encoding`, `geometry_types`, `bbox`, `covering`, `edges` — Parquet 2.0 파일이 지오메트리 열의 GEOMETRY/GEOGRAPHY 논리 유형에만 기록하는 CRS와 함께, 이는 GeoLibre가 이전에 전혀 볼 수 없던 것. 3D `bbox` 는 이제 `y`max를 고도로 착각하는 대신 최솟값 우선으로 읽히고, 지오메트리 없는 경도/위도 테이블이 인식됨
- [x] **GeoLibre가 Plotly Dash 앱에 임베드**, 새 `DashMap` 컴포넌트를 통해 Jupyter 위젯처럼 준비 핸드셰이크 후 마운트된 `iframe`으로 프로젝트 변경 사항을 전달. [Python API](#) 참조
- [x] **프로젝트가 파일처럼 열림:** 데스크톱 파일 관리자에서 `.geolibre.json` 을 더블클릭하면 파일 연결을 통해 열리고(macOS에서는 네이티브 프로젝트 이벤트), 지도에 하나를 드롭하면 프로젝트가 전환되며, 저장되지 않은 작업 프롬프트를 유지하고 잘못된 파일을 실패한 레이어가 아닌 프로젝트 오류로 보고
- [x] **Background 컨트롤이 하는 일을 설명하고**, Blank(빈) 배경이 스타일 다시 불러오기 후에도 유지되며 기본적으로 테마를 따르는 사용자 지정 색상을 가지되, 자체 색상을 가진 행성 및 지역 스타일에는 영향을 주지 않음
- [x] Shift 및 Shift+Alt가 다시 클릭 선택에 적용되어, 모든 클릭이 선택을 대체하는 대신 여러 피처에 걸쳐 추가와 교차가 동작; 원격 압축 Shapefile이 웹 빌드에서 불러와짐; identify, Vantor, Cesium 기능이 도입된 후 18개 제공 로케일 모두 다시 완성. [국제화](#) 참조

v3.0: 일급 렌더링 엔진으로서의 Cesium, 지구본 위의 동적 3D 씬, ArcGIS 쓰기 저장, 편집 가능한 SVG 인쇄 레이아웃

- [x] **Cesium이 사이트 창이 아닌 기본 렌더링 엔진이 될 수 있음:** View → Rendering engine(렌더링 엔진) 이 주 지도 영역을 MapLibre 2D와 Cesium 3D 사이에서 전환하고 프로젝트와 함께 저장되므로, 3D 우선 워크플로가 원치 않는 분할 레이아웃을 만들고 보지도 않는 MapLibre 지도를 유지하는 일이 없음. 이 전환 뒤에서 앱 전체가 이제 렌더러 중립적인 **MapEngine** 인터페이스를 통해 지도를 구동하며 **CesiumEngine** 이 두 번째 구현이고, 메뉴, 도구, 플러그인은 조용히 아무것도 하지 않는 대신 활성 엔진이 실제로 할 수 있는 것에 따라 게이트됨. [아키텍처](#) 참조
- [x] **지구본이 자체 크롬을 가짐:** Reset view(뷰 초기화), 전체 화면, 3D, 2D, Columbus 뷰를 위한 씬 모드 선택기(평면 모드는 프로젝트 지도에 맞춰 Web Mercator), Cesium Ion 영상과 키 없는 Esri 및 OpenStreetMap 스타일을 갖춘 네이티브 베이스맵 및 지형 갤러리, 공유 상태 표시줄의 커서 좌표와 고도, 2D 대응물처럼 모서리 간에 이동하는 컨트롤
- [x] **벡터 레이어가 지도에서처럼 지구본에서도 같은 모양과 동작:** 2D 지도가 사용하는 것과 같은 페인트 표현식에서 컴파일된 피쳐별 범주형, 등급형, 규칙 기반, 표현식, 비례, 마커 심볼로지; 필드 및 표현식 라벨; 대용량 포인트 레이어를 위한 포인트 클러스터링과 배치 렌더링; 모든 것을 지면에 고정하는 대신 폴리곤 돌출과 진정한 Z 고도; 레이어 필터, Time Slider 시계, 스토리 맵 불투명도 페이드; 그리고 같은 팝업 디자인을 통한 클릭 팝업, 호버 툴팁, identify, 강조 표시
- [x] **거의 모든 레이어 종류가 지구본에 도달:** 하나의 프로토콜 브리지를 통한 COG, 래스터 PMTiles, 로컬 MBTiles, 네이티브 타일 페처; 스타일이 정확히 일치하도록 숨겨진 MapLibre 렌더러를 통해 드레이프되는 벡터 타일, 벡터 PMTiles, 벡터 MBTiles; ArcGIS MapServer, WMTS, 이미지 오버레이; 그리고 Cesium이 네이티브로 불러오는 I3S 씬 레이어, LiDAR 포인트 클라우드, Gaussian splat 타일셋
- [x] **동등성을 넘어선 Cesium 네이티브 콘텐츠:** 자산 id로 지정하는 Cesium Ion 타일셋과 영상(Cesium OSM Buildings, Google Photorealistic 3D Tiles, Asset Depot 샘플 빠른 선택 포함), **Cesium3DTileStyle** 로의 변환기를 통해 Style 패널에서 분류되는 3D Tiles, 지구본 시계와 동기화된 궤도, 궤적, 이동 엔티티를 위한 시간 동적 **CZML** 씬, 문서 스타일링, 오버레이, 네트워크 링크를 유지하는 네이티브 **KML/KMZ**. [선택적 3D 지구본 자격 증명](#) 참조
- [x] **도구와 플러그인이 지구본까지 따라옴:** 범위 그리기와 모델 핀 드래그, 스크린샷, 인쇄 레이아웃, 지형과 타일이 안정될 때까지 기다리는 비디오 및 카메라 투어 녹화, 활성 Cesium 지형에서 샘플링한 Elevation Profile, Sun(태양) 시뮬레이션, Atmospheric Effects, Flight Simulator(비행 시뮬레이터), Clouds, Precipitation(강수) 플러그인의 네이티브 지구본 분기. 플러그인은 지원하는 엔진을 선언하고, 렌더러를 전환하면 지원되지 않는 플러그인은 설정을 유지한 채 일시 중단. [플러그인 API](#) 참조
- [x] **ArcGIS Feature Service에 대한 편집이 서비스로 다시 전달됨:** Layer actions(레이어 동작) → Save edits to ArcGIS service(ArcGIS 서비스에 편집 저장)가 **applyEdits** 를 통해 추가, 속성 및 지오메트리 변경, 삭제를 제출하고, 서비스의 기능과 필드에 대해 검증하며, 부분 실패와 서버 할당 id를 조정하고, 결과를 확인할 수 없는 쓰기는 재시도를 거부
- [x] **편집 가능한 SVG 인쇄 레이아웃:** 단일 페이지 Print Layout이 SVG로 보내지므로, 제목, 범례, 컬러바, 축척 막대, 북쪽 화살표 및 기타 레이아웃 요소가 최종 손질을 위해 Inkscape나 Illustrator에서 실제 벡터로 열림

- [x] **래스터가 화면에 있는 것에 맞춰 스트레치**: 분류되지 않은 래스터가 최소/최대, 5~95 백분위수, 평균 ± 2 표준편차로 현재 뷰포트에서 표시 범위를 다시 계산하고, Style 패널을 닫은 채 이동하는 동안에도 계속 그렇게 함. 이산 래스터 클래스도 자체 색상 및 불투명도 컨트롤을 얻었고, 불러온 모든 COG 레이어를 3D 지형 소스로 선택 가능
- [x] **Select by Expression이 레이어에서 선택하는 것뿐 아니라 필터링도 가능**: 검증된 표현식이 두 엔진 모두와 스타일 내보내기에서 시간, 임베드, 빠른, 규칙 필터와 결합되는 영구적이고 프로젝트에 저장되는 레이어 필터가 됨. [프로젝트 포맷](#) 참조
- [x] **데이터를 찾고 접근하는 새 방법**: 공개 Portolan Registry를 여는 **Portolan** 카탈로그 브라우저, WMS, WFS, ArcGIS 결과를 미리 채워진 Add Data 양식으로 넘기는 OGC **CSW** 카탈로그 검색, 유선 추적, 수문 위치 지정, 유역 분석, 네트워크를 따라 관측소, 댐, 수질 측정 지점으로 탐색하는 **USGS NLDI** 패널([USGS NLDI](#) 참조), 프로젝트에 포함되는 로컬 GLB 및 glTF 모델 가져오기, 스크롤하면 더 불러오는 프로젝트 갤러리
- [x] **Dimensions(치수) 플러그인**이 끝점이 벡터 정점에 스냅되고 그 정점이 편집될 때 연결 상태를 유지하는 CAD 스타일 선형 및 각도 치수선을 그림
- [x] **포인트 히트맵이 색상 램프와 속성 가중치를 가짐**, 저장된 스타일, 스타일 내보내기, 자동 범례를 거쳐 유지
- [x] **링크와 자동화**: 좌표 URL(`?lat=&lon=&zoom=` 및 간결한 `?12/40.7/-74.0`)과 Android `geo:` 인텐트가 특정 장소에서 GeoLibre를 열고, `loading=true` 매개변수가 헤드리스 캡처를 위한 스크린샷 준비 상태를 노출하며([임베딩과 공유](#) 참조), 외부 플러그인이 순수 JSON Schema로 AI Assistant 도구를 등록하고, Python 패키지가 팝업, 툴팁, 마커 심볼로지, WMS/WMTS 경계를 작성([Python API](#) 참조)
- [x] Field Collection이 관측당 여러 사진을 받고, 숫자 라벨이 천 단위 구분 기호로 서식화되며, 버퍼 도구가 안쪽으로 또는 경계를 가로질러 실행되고, 저장된 스타일 프리셋의 이름을 바꿀 수 있으며, Share 대화 상자가 로컬 파일 레이어가 공유 지도에서 누락될 것임을 미리 경고

v3.1: 렌더링 엔진에 합류한 Mapbox와 ArcGIS, 라이브 God's Eye View 지구본, 공유를 위한 OAuth 로그인, 음성 구동 AI Assistant

- [x] **네 가지 상호 교환 가능한 렌더링 엔진**: MapLibre와 Cesium에 더해 View $\rightarrow$ Rendering engine이 이제 **Mapbox GL JS**와 **ArcGIS Maps SDK for JavaScript**(CDN에서 필요 시 로드)를 제공하고, 앱이 실행 간에 마지막 엔진을 기억. 레이어 컨트롤, 기본 지도 컨트롤, 지도 영역 오버레이, GPS, 검토 댓글, 지리참조, 샘플 마커가 모든 엔진에 마운트. [렌더링 엔진](#) 참조
- [x] **Mapbox 렌더러의 동등성 달성**: PMTiles, MBTiles, LiDAR, 3D Tiles, Zarr, Deck.gl 레이어, 3D 모델, DuckDB 레이어가 모두 렌더링되고; 마커 아이콘, 채우기 패턴, 반전 채우기, 선 장식, 지오메트리 생성기, 히트맵 및 클러스터 렌더러, 라벨 중복 제거, 데이터 정의 라벨 재정의가 이식되며; Identify가 호버 팁과 사진 팝업을 갖춘 WMS, 픽셀, DuckDB 레이어를 다루고; Plugins 메뉴, Print Layout 아틀라스, Layer Swipe, Street View, Overture Maps, GeoAgent, Geo Editor, Flight Simulator의 자유 카메라가 모두 그 위에서 실행. [Mapbox 렌더러](#) 참조

- [x] **ArcGIS가 2D 지도와 3D 씬에서 렌더링**: 네이티브 ArcGIS 서비스와 PMTiles, MBTiles, Zarr, NetCDF 격자; 지형과 돌출, 씬 줌 및 경계 제한, 해리 축척 막대를 갖춘 3D 씬; 벡터 타일의 아이콘, 패턴, 라벨; 반전 채우기, 지오메트리 생성기, 선 장식, 라벨 재정의; 래스터 효과와 혼합 모드; identify 팝업, 호버 맵 팁, 지오태그 사진 팝업, 선택, 스토리 맵 페이드; 그리고 Print Layout 아틀라스. 데이터 플러그인 컨트롤이 ArcGIS 엔진에 적응. [ArcGIS 렌더러](#) 참조
- [x] **God's Eye View 플러그인이 Cesium 지구본을 라이브 상황실로 바꿈**: 라이브 CZML로서의 USGS 지진과 CelesTrak 위성 궤도, 키 없는 NASA FIRMS 활성 화재, Entur 및 기타 제공자의 대중교통, Austin, New South Wales, Ontario, DriveBC의 공개 CCTV 스냅샷과 교통 카메라, TomTom 라이브 교통 흐름, AIS 선박, 그리고 필요한 피드를 위한 플러그인 내 API 키 입력. [플러그인](#) 참조
- [x] **공유 및 게시를 위한 OAuth 로그인**: 자체 호스팅 가능한 서버 API에 S256 PKCE를 사용한 OAuth Authorization Code 흐름과 범위가 지정되고 만료되는 개인 토큰이 추가되고; 웹 빌드가 프로젝트 Share와 Gallery를 위해 로그인하며, 데스크톱 앱에 세션 관리를 갖춘 OAuth 로그인 추가. 공유는 접근 역할(보기, 댓글, 편집), 링크 만료, 비밀번호 보호, 취소를 가짐. [서버 API](#) 참조
- [x] **더 빠르고 더 유능한 AI Assistant**: 음성 명령 모드, TypeSafe를 통한 간단한 지도 명령의 1초 미만 빠른 경로, **OpenRouter** 제공자, Google, Anthropic, OpenAI, Bedrock의 라이브 모델 검색, 적용된 구간이 다시 보고되는 `apply_symbolology`의 명시적 클래스 구간, 열두 개를 넘으면 필요 시 로드되는 플러그인 도구, 플러그인이 기여하는 시스템 프롬프트 안내, 정확한 도구 이름을 먼저 순위 매기는 Whitebox 카탈로그에 대한 의미 검색. [AI Assistant](#) 참조
- [x] **새 데이터 플러그인**: 모델 신뢰도로 색상이 지정된 **Fields of the World** 농경지 경계, AlphaEarth와 Tessera 같은 사전 계산된 파운데이션 모델 임베딩을 탐색하고 다운로드하는 **Satellite Embeddings**, 프랑스 국가 포인트 클라우드를 위한 **IGN LiDAR HD**, OSM 다운로드 웹 서비스
- [x] **Cesium의 지속적 성장**: Zarr 레이어가 zarr-cesium을 통해 3D 지구본에 렌더링되고, DXF 및 DWG 도면이 3D에서 Z 값을 유지하며, 베이스맵 선택기가 Cesium 및 Mapbox 베이스맵을 제공하고, Ion 빠른 선택에 Google Photorealistic 3D Tiles와 Asset Depot 샘플 추가
- [x] **더 많은 진입 방법**: `?stac=` 링크가 STAC 카탈로그나 API 컬렉션을 열고, `?data=`가 LiDAR 포인트 클라우드를 열며, OGC API - Features 양식이 지도 뷰에서 경계 상자를 채우고, 래스터 TileJSON 메타데이터가 XYZ 소스를 통해 가져와지며, 데스크톱 앱이 EPSG:3857을 제공하지 않는 서버에서 WMS 타일을 그림. 런타임 구성 배포 서비스 라이브러리로 호스팅 빌드가 자체 큐레이션 서비스를 제공 가능. [데이터 추가](#) 참조
- [x] **스타일링 및 레이어 개선**: 연속 래스터 색상 램프가 불투명도 클래스를 가지고, 팝업에 구성 가능한 너비와 이미지 높이가 있으며, 레이어 그룹의 내용을 A에서 Z 또는 Z에서 A로 정렬하고, 레이어 메뉴와 Style 패널에서 스타일을 붙여넣을 수 있으며, Buffer 도구가 결과를 디폴트 가능
- [x] **개발자와 노트북을 위해**: GeoLibreAppAPI의 `getLayers / onLayersChanged`, 플러그인 API의 `moveLayerGroupToGroup`과 `listLayerGroups`, 기억되어 자동 연결되는 GeoLens 연결, Identify를 준비하고 컨트롤을 토글하며 투영을 설정하는 Python API. [플러그인 API](#) 및 [Python API](#) 참조
- [x] **히브리어가 제공 로케일에 합류**하여 두 번째 오른쪽에서 왼쪽 언어가 되고, 누락된 351개 키를 채워 모든 카탈로그가 다시 완성. [국제화](#) 참조

v3.2: 포인트 클라우드 어노테이션, 비공개 S3 버킷, OS 키체인에 저장되는 자격 증명, 미국 모든 주의 GIS 포털

- [x] **새 Point Cloud Annotation 플러그인으로 지도에서 LiDAR 포인트에 라벨 지정**: 브러시나 폴리곤으로 포인트를 선택하고, 숫자 단축키와 클래스 잠금으로 ASPRS 또는 사용자 정의 클래스를 할당하며, 포인트를 인스턴스로 묶고, SAM 마스크나 건물 윤곽 같은 폴리곤 레이어를 클라우드 위로 올리고, 브라우저에서 실행되는 Whitebox LiDAR 분류기로 사전 라벨링. 방향성 3D 박스는 상단, 측면, 정면 뷰와 검토 상태, 속성을 갖추고 있으며, 3D 폴리라인, 폴리곤, 키포인트도 함께 제공. 스트리밍 COPC는 한 영역을 전체 상세도로 불러올 수 있고, 라벨은 프로젝트와 함께 저장되며, 결과는 라벨이 지정된 LAS/LAZ 재작성, NumPy 배열, 또는 GeoJSON과 Segments.ai 큐보이드로 내보내기 가능. [포인트 클라우드 어노테이션](#) 참조
- [x] **노트북과 에이전트에서의 포인트 클라우드 라벨링**: `add_lidar`, 어노테이션 리더, 클래스 스키마, 인스턴스, 전체 파일 재작성을 수반하는 헤드리스 Whitebox 사전 라벨링을 Python 패키지와 MCP 도구로 사용 가능. [Python API](#) 및 [MCP 서버](#) 참조
- [x] **비공개 S3 버킷이 공개 버킷처럼 열림**: S3 연결이 액세스 키, 명명된 프로필, IAM Identity Center(SSO) 로그인, IAM 역할을 받으며, 탐색 가능한 S3 Browser와 UI에서 마스킹되는 SQL 자격 증명 제공. [클라우드 스토리지](#) 참조
- [x] **데스크톱에서는 비밀 정보가 운영 체제의 자격 증명 저장소에 보관됨**: 프로젝트 파일에 저장되는 API 키, 토큰, 자격 증명이 일반 설정 대신 macOS Keychain, Windows Credential Manager, Linux Secret Service로 가고, 플러그인은 새 `app.credentials` API를 통해 같은 저장소에 접근. [플러그인 API](#) 참조
- [x] **미국의 GIS 포털을 하나의 메뉴에**: 새로운 US Federal GIS(Census Bureau와 NOAA부터 USGS와 FEMA까지 24개 기관), US State GIS(50개 주 전체와 DC), US Local GIS(약 150개 도시, 카운티, 지역 기관), Tennessee GIS 웹 서비스, 그리고 HUB Ocean의 공개 해양 데이터셋을 위한 Ocean Data Platform 플러그인. [웹 서비스](#) 참조
- [x] **모든 투영의 WMS**: 데스크톱 Add Data(데이터 추가) 대화 상자에서 WMS 레이어의 CRS를 선택할 수 있고, 투영 CRS만 제공하는 서버는 즉석에서 Web Mercator로 재투영되며, `addWmsLayer` 가 선택적 `crs` 를 받음. GeoJSON 출력이 없는 WFS 서버는 GML을 통해 불러오고, ArcGIS MapServer 하위 레이어는 Browser 패널에서 탐색할 수 있는 별도 레이어로 분리됨. [데이터 추가](#) 참조
- [x] **레이어 이름으로 스타일 재사용**: Project → Export → Export Layer Styles(레이어 스타일 내보내기)가 모든 레이어의 스타일을 레이어 이름을 키로 하는 하나의 JSON 파일에 기록하고, Import Layer Styles(레이어 스타일 가져오기)가 일치하는 레이어를 한 번의 실행 취소 단계로 다시 스타일링하며, Startup 설정이 레이어가 추가될 때마다 일치하는 레이어에 스타일 파일을 자동 적용. 플러그인은 같은 일을 하는 `importLayerStyle` 을 얻음. [스타일링](#) 참조
- [x] **레이어 구성**: 지도 위 레이어 컨트롤이 레이어 그룹을 표시하고, KML/KMZ 파일의 레이어는 파일 이름을 딴 그룹 아래에 묶이며, Add Vector Layer(벡터 레이어 추가) 레이어는 전체 스타일링과 편집을 위해 네이티브 GeoJSON 레이어로 변환됨
- [x] **배포와 공유를 위해**: 관대한 파서를 갖춘 `deployment.json` v1 스키마가 하나의 배포 정책 파일을 위한 토대를 마련하고, 배포가 툴바와 탭 제목의 앱 이름을 바꿀 수 있으며, `?plugin=` 딥 링크가 내장 플러그인을 활성화하고, 프로젝트 소유자가 삭제 보호를 유지한 채 공유 프로젝트의 소유권을 이전 가능. [배포 정책](#) 참조

- [x] **곳곳의 추가 개선:** MapLibre 컨트롤 플러그인이 ArcGIS 렌더러에서 실행되고, LiDAR 오버레이 그래픽이 오버레이 레이어로 그려지며, Satellite Embeddings 플러그인이 Tessera v1.1을 시각화하고, Python Map 이 identify와 컨트롤 상태를 저장하며, **아제르바이잔어**가 20개 카탈로그를 모두 완성한 채 제공 로케일에 합류. [국제화](#) 참조

v3.3: 가시선, MGRS 격자와 좌표, SQL Server, 비교 및 복원이 가능한 프로젝트 이력, 그리고 도킹하고 검증하며 메뉴를 확장하는 플러그인 생태계 (현재)

- [x] **가시선 분석:** 지도에서 두 점을 선택하면 지형 위에서 두 점 사이의 대화형 가시선을 확인 가능
- [x] **MGRS, USNG, 임의 EPSG 좌표:** MGRS/USNG 또는 임의 EPSG 코드로 된 좌표 표시, 격자 참조 검색, MGRS/USNG 격자 오버레이, Set View(뷰 설정)의 격자 참조. [지도 도구](#) 참조
- [x] **데스크톱의 SQL Server와 Azure SQL:** Add Data(데이터 추가) 소스, 사이드카의 편집 가능한 데이터 소스, 편집 내용 저장, 테이블을 위한 Browser 패널. [데이터 추가](#) 참조
- [x] **프로젝트 스냅샷 비교와 단일 레이어 복원:** 프로젝트 이력이 스냅샷 간에 무엇이 바뀌었는지 보여주고, 프로젝트 전체 대신 레이어 하나만 되돌릴 수 있게 하며, 대용량 프로젝트에서 자동 저장이 일시 중지되면 알려 줌. [프로젝트](#) 참조
- [x] **STAC Item 내보내기를 갖춘 편집 가능한 레이어 메타데이터:** 레이어의 메타데이터를 편집하고, STAC Item으로 내보내며, 서비스 레이어의 소스를 Metadata 대화 상자에서 확인하고, 플러그인이 메타데이터를 추가하도록 허용
- [x] **Identify에서 바로 편집:** Identify 결과의 Edit geometry(지오메트리 편집)와 Edit attributes(속성 편집), 필드로 읽어 들이는 WMS GetFeatureInfo HTML 테이블, 질의 불가능한 WMS 레이어 건너뛰기, 오류로 표시되는 WMS 예외
- [x] **3D 돌출 레이어를 glTF, OBJ, STL 모델로 내보내기**하여 3D 모델링 및 프린팅 도구에서 사용
- [x] **래스터 표시 스트레치를 위한 대화형 히스토그램**으로 최소값과 최대값을 조정하는 동안 분포를 확인 가능
- [x] **S3 프로젝트:** S3 Browser에서 프로젝트를 바로 열 수 있고, S3 Browser는 프로젝트를 불러오는 동안에도 열린 채 유지되며 시작 시 열리도록 설정 가능. [클라우드 스토리지](#) 참조
- [x] **더 차별하고 명확한 인터페이스:** Report issue(문제 보고) 동작을 갖춘 통합 토스트 알림, Cesium 및 ArcGIS 엔진의 레이어 실패와 타일 상태 토스트, 개별 도구까지 도달하는 명령 팔레트, 색각 미리보기, New Project(새 프로젝트)의 스타터 프로젝트
- [x] **도킹하고 검증하는 플러그인 생태계:** Historical Imagery, Timelapse, Overture, Basemaps, GeoAgent, Layer Swipe, Elevation Profile, USGS LiDAR 패널이 사이드 패널에 도킹되고 다크 테마를 따르며; 레지스트리 설치가 게시된 번들 해시에 대해 검증되고 레지스트리 차단 목록을 존중하며; 플러그인이 내장 툴바 메뉴에 항목을 추가하고, 자격 증명 제거를 거쳐도 프로젝트 상태를 유지하며, 호스트의 proj4 및 WFS API를 사용하고, 데스크톱에서 쿠키 저장소를 갖춘 네이티브 fetch를 얻으며; ?plugin= 링크가 신뢰 확인 프롬프트 뒤에서 레지스트리 플러그인을 열고; 외부 플러그인은 Plugins → Installed(설치됨) 아래에 묶임. 새로운 USGS 3DEP 고도(DEM) 플러그인이 카탈로그에 합류. [플러그인 API](#) 참조
- [x] **Python과 AI 에이전트를 위해:** geolibre-mcp 가 열려 있는 데스크톱 지도를 구동하고, 필터, 라벨, 플러그인 상태, 스토리 맵을 작성하며, 라이브 처리를 실행 가능. [MCP 서버](#) 및 [Python API](#) 참조

- [x] **배포와 서버를 위해:** deployment.json이 첫 화면 그리기 전에 그리고 데스크톱 구성 디렉터리에서 로드되고, Docker 이미지가 부팅 시 검증된 deployment.json을 생성하며 사이드카, AI, 플러그인 정책을 강제하고, 서버 API가 링크 역할, 만료, 비밀번호를 갖춘 공유, 조직별 OIDC 연합과 신뢰 헤더 로그인, 조직별 보안 정책, SCIM 2.0 프로비저닝을 얻음. [배포 정책](#) 및 [서버 API](#) 참조
- [x] **완성된 카탈로그와 줌 환경 설정:** 모든 로케일 카탈로그가 완성되고 플러그인이 자체 번역을 제공할 수 있으며, Map Preferences(지도 환경 설정) 옵션으로 새로 추가된 레이어로의 줌을 끌 수 있음. [국제화](#) 참조

플러그인 마켓플레이스와 레지스트리 (설계)

이 절은 v1.0 의 "플러그인 마켓플레이스 / 레지스트리" 항목에 대한 설계를 정리합니다. 기존 외부 플러그인 기반, plugin.json 매니페스트 계약, HTTPS 매니페스트 URL 로딩, 데스크톱 앱 데이터 plugins/ 스캔, 번들된 public/plugins/ 드롭인 메커니즘 위에 구축되며, "외부 플러그인 패키지 배포 워크플로" 항목과도 관련이 있습니다.

목표

사용자가 매니페스트 URL을 직접 입력하지 않고도 큐레이션된 레지스트리에서 신뢰할 수 있는 외부 플러그인을 발견하고, 설치하고, 업데이트하고, 제거할 수 있게 합니다. 데스크톱과 웹 빌드 모두에서 동작하면서, 플러그인을 신뢰할 수 있는 코드로 취급하는 기존 신뢰 모델은 그대로 유지합니다.

레지스트리

- 정적 JSON으로 게시되는 큐레이션된 버전 관리 인덱스(예를 들어 `geolibre.app`에 호스팅되는 `registry.json`, 또는 제출물을 모아 둔 GitHub 저장소에서 생성). MVP에는 라이브 백엔드가 필요하지 않습니다.
- 각 항목은 `id`, `name`, `version`, `description`, `author`, `homepage`, `manifestUrl`, `categories`, `minGeoLibreVersion`, 그리고 선택적 `screenshots`를 담습니다.
- 인덱스는 HTTPS로 가져와 캐시되며, 항목들은 기존 로더가 이미 이해하는 것과 같은 `plugin.json` 매니페스트를 가리킵니다.

탐색 및 설치 UI

- QGIS를 본떠 만든 독립형 Manage Plugins(플러그인 관리) 대화 상자(Settings 메뉴 > Manage Plugins)에 All(전체) / Installed(설치됨) / Not installed(설치 안 됨) / Upgradeable(업그레이드 가능) / Settings(설정) 섹션이 있습니다. 네 개의 탐색 섹션은 검색과 함께 레지스트리 항목을 나열하고 항목별 설치, 설치됨, 업데이트 상태를 보여주며, Settings 섹션은 플러그인 소스를 관리합니다.
- 설치: 현재의 외부 플러그인 로더를 재사용합니다. 항목의 `manifestUrl`을 해석하고, 검증한 다음, 플러그인을 등록합니다.
- 데스크톱: 번들을 앱 데이터 `plugins/<id>/` 디렉터리에 다운로드하여 유지되게 하고, 기존 스캔을 통해 시작 시 로드되도록 합니다.

- 웹: 항목의 `manifestUrl` 을 데스크톱 설정에(그리고 공유 프로젝트의 경우 프로젝트의 `plugins.manifestUrls` 에) 기록하여 다음에 열 때 로드되도록 합니다.
- 제거는 기록된 매니페스트 URL을 삭제하고 런타임에 플러그인을 등록 해제하므로(활성 컨트롤이 있으면 해체), 재시작 없이 변경 사항이 적용됩니다. (MVP는 번들을 다운로드하는 대신 매니페스트 URL을 기록합니다. 위의 데스크톱 번들 다운로드 경로는 이후의 개선 사항입니다.)

업데이트와 버전 관리

- 설치된 `version` 을 레지스트리 항목과 비교하여 업데이트 가능 상태를 표시하고, 번들을 다시 가져오는 원클릭 업데이트를 제공합니다.
- `minGeoLibreVersion` 을 존중하여 호환되지 않는 플러그인은 설치하지 않고 표시만 합니다.

신뢰와 보안

- 레지스트리는 허용 목록입니다. 큐레이션된 항목만 설치 대상으로 제공됩니다.
- HTTPS 전용 매니페스트(기존 `isAllowedPluginManifestUrl` 규칙).
- 플러그인 항목은 신뢰할 수 있는 코드로 실행되므로(데스크톱 CSP는 설계상 `blob:` 스크립트 실행을 허용) 설치 시 명시적인 사용자 동의를 받습니다.
- 큐레이션된 레지스트리와 명시적 설치 동의가 주된 통제 수단입니다.

번들 플러그인과의 관계

- 번들된 `public/plugins/<id>/` 드롭인은 여전히 빌드 안에 자사 또는 비공개 플러그인을 담아 배포하는 무설정 방법입니다. 마켓플레이스는 발견 가능하고 사용자가 설치하는 서드파티 플러그인을 다루며, 둘은 상호 보완적이고 같은 `plugin.json` 계약과 로더를 공유합니다.

단계

1. 큐레이션된 정적 레지스트리와 매니페스트 URL을 통한 탐색 및 설치(현재 로더를 재사용하고 매니페스트 URL을 설정에 기록). **완료.**
2. 버전 확인, 업데이트 및 제거 흐름.
3. 서드파티 작성자를 위한 제출 워크플로.

구현 (1단계)

MVP는 데스크톱 앱에 포함되어 출시되며, 같은 프론트엔드가 웹 빌드도 제공하므로 양쪽 모두에서 동작합니다.

- `apps/geolibre-desktop/src/lib/plugin-registry.ts` 가 레지스트리(`{ "plugins": [...] }`)를 가져와 정규화하고, 각 항목의 `manifestUrl` 을 레지스트리 위치를 기준으로 해석합니다. 레지스트리 URL은 `VITE_GEOLIBRE_PLUGIN_REGISTRY_URL` 이거나, 기본적으로 `https://plugins.geolibre.app/plugin-registry.json` 에 호스팅된 레지스트리입니다.

- `apps/geolibre-desktop/src/components/layout/ManagePluginsDialog.tsx` 는 All(전체) / Installed(설치됨) / Not installed(설치 안 됨) / Upgradeable(업그레이드 가능) / Settings(설정) 섹션을 갖춘 독립형 대화 상자(Settings 메뉴 > Manage Plugins)로, 검색, 설치, 제거 전 확인 단계, 더 새로운 버전이 게시되었을 때의 Update(업데이트) 동작, `minGeoLibreVersion` 호환성 검사, 인라인 오류 처리를 제공합니다. Settings 섹션은 추가 로컬 디렉터리와 수동 매니페스트 URL을 관리합니다. 모든 동작은 즉시(라이브) 적용됩니다.
- 설치하는 항목의 매니페스트 URL을 플러그인 매니페스트 URL 목록에 기록하므로, 기존 외부 플러그인 로더가 이를 가져와 등록합니다. 새로운 신뢰 경로는 도입되지 않습니다. 제거는 (확인 후) 런타임에 플러그인을 등록 해제하며 — 활성 지도 컨트롤이 있으면 해체하여 — 다시 불러오지 않아도 Plugins 메뉴가 갱신됩니다. 업데이트는 매니페스트 URL을 다시 가져와 게시된 버전을 제자리에서 다시 등록하는데, 기존 버전을 해체하기 전에 새 버전을 먼저 가져오므로 업데이트가 실패해도 설치된 플러그인은 그대로 남습니다.
- 레지스트리와 플러그인 번들은 [opengeos/geolibre-plugins](https://opengeos.github.io/geolibre-plugins) 저장소에 있으며 GitHub Pages를 통해 `plugins.geolibre.app` 에 게시됩니다. 이 저장소는 `sample/` 템플릿을 제공하고, 메인테이너가 그곳에 큐레이션된 항목을 추가합니다.

<https://geolibre.app/contributing/>

기여하기

GeoLibre 개선에 관심을 가져 주셔서 감사합니다. 이 가이드는 개발 환경 설정 방법, 프로젝트 구조, 로컬 품질 게이트, 그리고 풀 리퀘스트 워크플로를 다룹니다. 오타 수정부터 새 처리 도구나 플러그인 추가까지, 규모에 관계없이 모든 기여를 환영합니다.

참여함으로써 상호 존중하고 건설적인 태도로 소통하는 데 동의하는 것으로 간주합니다.

기여 방법

- **버그 신고 또는 기능 요청**은 [이슈](#)를 열어 주세요. 재현 단계, 기대한 결과, 실제로 일어난 일을 포함하고, 사용 중인 OS와 웹 빌드에서 겪었는지 데스크톱 빌드에서 겪었는지도 함께 적어 주세요.
- **문서 개선**은 [docs/](#) (이 사이트) 아래에서 할 수 있습니다.
- **버그 수정 또는 기능 구현**은 앱이나 패키지 중 하나에서 할 수 있습니다.
- **플러그인 작성**은 외부 플러그인 API를 사용합니다(아래 [플러그인](#) 참조).

큰 변경을 계획하고 있다면, 풀 리퀘스트에 시간을 들이기 전에 먼저 이슈를 열어 접근 방식에 대해 합의할 수 있도록 해 주세요.

사전 요구 사항

- **Node.js** 22 이상 (`.nvmrc` 가 메이저 버전을 고정하므로 `nvm use` 로 자동 선택됩니다)
- Tauri 데스크톱 빌드를 위한 **Rust** 툴체인([rustup](#)) (`rust-toolchain.toml` 이 CI가 사용하는 stable 채널을 선택합니다)
- Linux 전용: `webkit2gtk` 및 `libayatana-appindicator` ([Tauri 사전 요구 사항](#) 참조)
- [pre-commit](#)(아래의 커밋 후)을 위한 **Python** 3.10 이상, 그리고 백엔드 작업을 한다면 변환 사이드카와 그 테스트를 위해서도 필요합니다

설정

저장소를 포크한 다음 포크를 클론하세요. `YOUR_GITHUB_USERNAME` 을 본인의 GitHub 사용자 이름으로 바꾸세요:

```
git clone https://github.com/YOUR_GITHUB_USERNAME/GeoLibre.git
cd GeoLibre
npm install
```

GeoLibre는 npm 워크스페이스 모노레포이므로 루트에서 한 번의 `npm install` 로 모든 패키지가 연결됩니다. npm을 사용하세요. 저장소는 `package-lock.json` 을 추적합니다.

로컬에서 실행하기

웹 빌드(브라우저의 지도):

```
npm run dev
```

<http://localhost:5173>을 여세요.

데스크톱 빌드(Tauri, 파일 시스템 대화 상자, 로컬 MBTiles, 로컬 래스터 읽기에 필요):

```
npm run tauri:dev
```

저장소 구조

```
apps/geolibre-desktop    # Tauri + React app (shell, composition, Tauri I/O)
packages/core            # Domain types, Zustand store, project format
packages/map             # MapLibre integration and layer sync
packages/ui              # Tailwind + shadcn/ui primitives
packages/plugins         # Plugin API and built-in plugins
packages/processing      # Client-side algorithm registry
packages/embed           # Typed iframe embed client (@geolibre/embed)
packages/collab-core     # Shared collaboration code for the collab workers
workers/                # viewer, collab, collab-node, tiles and ai-proxy workers
backend/geolibre_server  # Optional FastAPI conversion sidecar (Python)
backend/geolibre_server_api # Reference projects and identity API (Python)
python/                 # The geolibre Python package (Jupyter widget, MCP server)
e2e/                    # Playwright end-to-end tests
tests/                  # Frontend unit and component tests
docs/                   # This documentation site (Zensical, mkdocs.yml)
```

이들이 어떻게 맞물리는지는 [아키텍처](#) 참조 문서를, 저장된 프로젝트 스키마는 [프로젝트 형식](#) 참조 문서를 참고하세요.

개발 워크플로

1. `main` 에서 기능 브랜치를 만드세요. 절대 `main` 에 직접 커밋하지 마세요.

```
git switch -c feat/short-description
```

1. 변경 범위를 좁게 유지하면서 작업하세요. 새로운 패턴을 도입하기보다 주변 코드의 스타일을 따르세요.
2. **품질 검사**를 실행하고 통과하는지 확인하세요.
3. 명확한 메시지로 커밋하세요. 히스토리는 [Conventional Commits](#) 스타일의 접두사를 따릅니다. 예:


```
feat: , fix: , docs: , refactor: , chore: .
```
4. 브랜치를 푸시하고 `main` 을 대상으로 풀 리퀘스트를 여세요. 풀 리퀘스트 템플릿은 무엇을 왜 변경했는지, 어떻게 테스트했는지, 어떤 이슈와 관련되는지를 묻습니다.

풀 리퀘스트는 병합 전에 리뷰됩니다. 자동 리뷰어가 인라인 댓글을 남길 수 있습니다. 이를 반영하거나, 제안이 적용되지 않는 이유를 설명하세요.

품질 검사

작업하는 동안 빠른 TypeScript 단위 테스트를 실행하세요:

```
npm run test:frontend
```

Vite 빌드 없이 타입 검사만 하려면 `npm run typecheck:fast` 를 실행하세요(앱 전체에 대해 `tsc -b` 를 실행하며, 증분 캐시 외에는 출력을 쓰지 않습니다. 먼저 작은 `@geolibre/embed` 패키지를 빌드하는데, 앱이 그 `dist/` 타입을 임포트하기 때문입니다). `npm run typecheck` 는 전체 프로덕션 빌드의 별칭입니다.

풀 리퀘스트를 열기 전에, 변경한 파일에 대해 pre-commit 혹은 실행한 다음 로컬 품질 게이트를 실행하세요:

```
pre-commit run --files path/to/changed.ts path/to/other.tsx # list each file you changed
npm run ci:web # frontend-only changes: no Rust or Python needed
npm run ci # the full gate that CI runs
```

pre-commit은 `--all-files` 가 아니라 `--files` 로 본인 파일에만 한정하세요. `--all-files` 는 저장소의 모든 파일을 다시 포맷하고 다시 검사하므로 느리고, 손대지 않은 파일까지 변경할 수 있습니다. 로컬 `npm-build` 혹은 여전히 호출마다 전체 프로덕션 빌드를 한 번 실행합니다. 이미 빌드했다면 명령 앞에 `SKIP=npm-build` 를 붙이세요.

`npm run ci:web` 은 웹 앱과 그 패키지만 건드리는 변경을 위한 빠른 게이트입니다: 린트, 공유 의존성, i18n 카탈로그, 갤러리 및 플러그인 문서 검사, `typecheck:fast`, [테스트 타입 검사](#), 그리고 프론트엔드 단위 테스트를 실행합니다. Node만 있으면 됩니다.

`npm run ci` 는 지속적 통합을 그대로 반영하는 전체 게이트를 다음 순서로 실행합니다:

단계	명령	범위
린트	<code>npm run lint</code>	<code>apps/</code> , <code>packages/</code> , <code>workers/</code> , <code>tests/</code> 에 대한 ESLint
공유 의존성 검사	<code>npm run check:shared-deps</code>	앱과 <code>packages/plugins</code> 양쪽에 선언된 의존성이 같은 범위를 갖는지 확인(공유 의존성 범위 참조)
i18n 카탈로그 검사	<code>npm run i18n:tools:check</code>	<code>en.json</code> 의 처리 도구 문자열이 도구 레지스트리와 일치하는지 확인(<code>npm run i18n:tools</code> 로 재생성)
갤러리 검사	<code>npm run gallery:check</code>	<code>docs/gallery.md</code> 와 <code>docs/demos.md</code> 티저가 <code>scripts/demo-gallery.json</code> 과 일치하는지 확인(<code>npm run gallery</code> 로 재생성)
플러그인 문서 검사	<code>npm run plugins:docs:check</code>	<code>docs/user-guide/plugins.md</code> 의 내장 플러그인 표가 플러그인 레지스트리와 일치하는지 확인(<code>npm run plugins:docs</code> 로 재생성)
빌드	<code>npm run build</code>	TypeScript 컴파일(<code>tsc -b</code>) 및 Vite 빌드

단계	명령	범위
프론트엔드 테스트	<code>npm run test:frontend:coverage</code>	<code>tests/</code> 아래의 단위 테스트, 커버리지 하한 으로 게이트
미테스트 모듈	<code>npm run check:untested-modules</code>	어떤 테스트도 로드하지 않는 500줄 초과 소스 파일, 기준선 래치 으로 관리
워커 검사	<code>npm run test:worker</code>	다섯 워커(<code>viewer</code> , <code>collab</code> , <code>collab-node</code> , <code>tiles</code> , <code>ai-proxy</code>) 전부를 타입 검사하고 <code>collab-node</code> 테스트를 실행
테스트 타입 검사	<code>npm run typecheck:tests</code>	<code>tests/</code> 에 대한 <code>tsc</code> , 타입 오류 래치 으로 게이트
백엔드 테스트	<code>npm run test:backend:coverage</code>	Python 사이드카용 <code>pytest</code> , 커버리지 하한 으로 게이트
Docker 테스트	<code>npm run test:docker</code>	<code>docker/tests</code> 아래 컨테이너의 배포 및 사이드카 정책 스크립트용 <code>pytest</code>
Rust 검사	<code>npm run check:rust</code>	Tauri 셀용 <code>cargo check</code>

표의 중간 부분은 두 스크립트로 묶여 있으며 각각 단독으로도 실행할 수 있습니다: `npm run ci:frontend` (i18n 검사부터 워커 검사까지, Node만 필요)와 `npm run ci:backend` (백엔드 및 Docker 테스트, 스크립트 실행을 위해 Python과 npm CLI가 필요하지만 `npm install`은 필요 없음). CI는 `.github/workflows/ci.yml`에서 린트, `ci:frontend`, `ci:backend`, `check:rust`를 순차가 아니라 별도의 병렬 작업으로 실행하며, `check:shared-deps`와 `typecheck:tests`는 "Lint and type check" 작업에서 린트 옆에서 실행됩니다. 따라서 `npm run ci`에 단계를 추가한다면 이 그룹 중 하나에 추가하면 CI가 이를 자동으로 반영합니다.

풀 리퀘스트에서는 린트와 "Build and test"가 항상 실행됩니다. 나머지 작업(E2E core, 백엔드, Rust, 컨테이너, 그리고 데이터베이스 및 OIDC 통합 작업)은 PR이 해당 작업에 영향을 주는 경로를 변경할 때만 실행됩니다. `ci.yml` 상단의 `changes` 작업이 이 경로 필터를 담고 있습니다. 단일 **CI passed** 작업이 이들 모두를 기다렸다가 어느 하나라도 실패하면 실패하므로, 지켜봐야 할 검사는 이것 하나이며 브랜치 보호가 요구하는 검사도 이것입니다. 새 CI 작업은 반드시 그 `needs` 목록에 추가해야 하며, 그렇지 않으면 그 작업의 실패가 병합을 막지 못합니다.

손댄 영역의 툴체인만 있으면 됩니다. 문서만 또는 프론트엔드만 변경하는 경우 Rust나 Python이 필요하지 않지만 (`npm run ci:web`을 사용), 전체 `npm run ci` 게이트는 필요로 합니다. 백엔드 단계는 사이드카의 `test` 추가 기능이 필요합니다 (`pip install -e "backend/geolibre_server[test]"`). 이것이 없으면 벡터, 래스터, SQL, ML 테스트가 스스로 건너뛩니다.

컴포넌트 테스트

React 패널은 같은 `node --test` 러너에서, [happy-dom](#) DOM과 [Testing Library](#)를 사용하여, 뒤에 지도 없이 테스트됩니다. `tests/component-*.test.ts` 파일들이 따라 할 예시입니다. 컴포넌트 테스트는 먼저 `tests/helpers/dom.ts`를 임포트한 다음 동적 `import()`로 컴포넌트를 로드합니다:

```
import { fireEvent, render, screen, useAppStore } from "../helpers/dom";
import { createElement } from "react";

const { LayerPanel } = await import("../apps/geolibre-desktop/src/components/panels/
LayerPanel");
```

컴포넌트는 반드시 동적으로 임포트해야 합니다. 헬퍼는 Node가 Vite처럼 컴포넌트를 로드할 수 있게 하는 로더 혹은 등록합니다: CSS와 `?url / ?raw` 임포트는 스텝이 되고, `virtual:` 모듈과 `import.meta.env` 는 대체물을 받으며, `.tsx` 는 자동 JSX 런타임으로 컴파일됩니다. 혹은 등록된 이후에 로드되는 모듈에만 적용되는데, 정적 임포트는 어떤 모듈 코드가 실행되기도 전에 로드됩니다. 헬퍼는 또한 DOM을 설치하고 앱 자체의 `i18next`를 `en.json`으로 초기화하므로, 쿼리는 실제 영어 문자열을 사용합니다. 헬퍼의 `render` 는 요소를 앱의 `i18n`, 툴팁, 방향 프로바이더로 감쌉니다.

스토어를 통해 상태를 설정하고(`useAppStore.setState(...)`), 사용자가 보는 것이나 스토어에 들어가는 것에 대해 단언하세요. 각 테스트 후 헬퍼는 모든 것을 언마운트하고 앱 및 데스크톱 설정 스토어, `localStorage`, `fetch`, 그리고 모든 `stubLayout()` 을 초기화합니다. 그 밖의 규칙:

- `mapControllerRef` 에는 `{ current: null }` 을 전달하세요. 어떤 동작을 위해 살아 있는 지도가 필요한 패널은 그 로직이 `lib/` 모듈에 속해야 한다는 신호이며, 거기서는 일반 단위 테스트로 커버할 수 있습니다.
- `render` 이후에 수행하는 스토어 쓰기는 `act(...)` 로 감싸서 단언하기 전에 React가 다시 렌더링하도록 하세요.
- `fetch` 는 기본적으로 거부됩니다. `mockFetch(...)` 로 응답을 제공하세요.
- 속성 테이블 같은 가상화 목록에는 `stubLayout()` 을 호출하세요. `happy-dom`은 레이아웃을 수행하지 않으므로, 이것이 없으면 모든 요소의 높이가 `0px`이 되어 목록이 행을 렌더링하지 않습니다.
- DOM 노드가 아니라 일반 값(`textContent`, `.checked`, `.value`, 스토어 상태)에 대해 단언하세요. 노드에 대한 `assert.equal` 이 실패하면 Node가 `happy-dom` 트리 전체를 포맷하는데, 이것은 멈춘 것처럼 보입니다. 대신 `queryAll...().length` 로 쿼리 결과 개수를 세세요.

이 테스트들은 다른 테스트와 마찬가지로 프론트엔드 [커버리지 하한](#)에 포함되므로, 큰 패널에 대한 첫 테스트는 그 패널을 보고서에 추가합니다.

엔드투엔드 스모크 테스트

`npm run test:e2e` 는 빌드된 웹 앱을 대상으로 `e2e/` 의 Playwright 스위트를 실행합니다(빌드하고, `vite preview` 로 서빙하고, 헤드리스 Chromium을 구동합니다). 브라우저는 `npx playwright install chromium` 으로 한 번 설치하세요.

스위트는 두 Playwright 프로젝트로 나뉘며, 둘이 함께 `e2e/` 를 분할합니다. 일반 `npm run test:e2e` 는 여전히 모든 스펙을 정확히 한 번씩 실행합니다:

프로젝트	명령	다루는 내용	실행 시점
core	<code>npm run test:e2e:core</code>	앱이 부팅되고 지도를 렌더링하는지, 그리고 공유 UI 표면: 레이어 패널, 속성 테이블, 대화 상자, 드래그 앤 드롭, 테마, RTL, 접근성, PWA 셀, 프로젝트 저장 및 재열기, 플러그인 디링크 문서.	모든 푸시, 그리고 웹 빌드가 읽는 경로를 변경하는 모든 PR에서 <code>ci.yml</code> 의 <code>E2E core</code> (Playwright) 작업으로 실행.
core-engine	(<code>npm run test:e2e:core</code> 가 실행)	대체 엔진(Cesium, ArcGIS)마다 한 번의 스모크 패스: 해당 엔진으로 전환하고, GeoJSON을 추가하고, 피처를 식별합니다. 소프트웨어 렌더링 3D 뷰가 CPU를 포화시키므로 <code>core</code> 다음에 워커 하나로 실행됩니다. ArcGIS 패스는 <code>js.arcgis.com</code> 에서 SDK를 로드합니다.	모든 푸시, 그리고 웹 빌드가 읽는 경로를 변경하는 모든 PR에서 <code>ci.yml</code> 의 <code>E2E core</code> (Playwright) 작업으로 실행.
features	<code>npm run test:e2e:features</code>	기능별 통합: Mapbox/Cesium 엔진, STAC, 내보내기, 스토리 맵, 썸 그래프, 플러그인 설치.	매일 밤 및 <code>e2e-full.yml</code> 을 통한 수동 실행, 4개로 샤딩 — 또는 <code>full-e2e</code> 라벨이 붙은 PR에서.

이 분할은 소요 시간에 따른 결정이지 가치에 대한 판단이 아닙니다: 전체 스위트는 CI 러너에서 직렬로 약 36분이 걸리고 그중 기능 스펙이 약 27분을 차지하여, 다른 모든 작업이 10분 이내에 끝나는 동안 이 작업이 CI의 유일한 임계 경로가 되었습니다. 대부분의 기능 스펙은 실제로 출시되었던 특정 회귀를 막기 위한 것이므로 여전히 실행되지만, 모든 커밋마다 실행되지는 않습니다.

모든 사용자에게 문제가 될 경우에만 새 스펙을 `core`에 추가하세요. 목록은 `playwright.config.ts`의 `CORE_SPECS`에 있으며, 거기에 이름이 없는 것은 자동으로 `features`에 속합니다. 야간 스위트가 다루는 영역을 건드리고 있다면, 병합 전에 그 검사를 받을 수 있도록 PR에 `full-e2e` 라벨을 붙이세요.

두 작업 모두 실패 시 Playwright 보고서를 아티팩트로 업로드합니다.

각 PR의 Cloudflare 프리뷰도 배포가 완료되면 한 번 스모크 테스트됩니다: `Preview smoke` (Cloudflare) 커밋 상태는 배포된 `/demo/` 앱이 지도 로딩을 끝내지 못하거나, 콘솔 오류를 기록하거나, 자체 호스트에서 4xx/5xx를 받으면 실패합니다. 이는 `/`에서 실행되는 `vite preview`로는 잡을 수 없는 호스트 및 하위 경로 문제를 잡아냅니다. 어떤 배포에 대해서든 `PREVIEW_URL=<app URL> npm run test:e2e:preview`로 실행할 수 있습니다. `core` 스위트는 로컬 빌드에 대해 같은 클린 콘솔 검사를 실행합니다.

로컬에서는 스위트가 빌드된 앱을 4173 포트로 서빙하며, 이미 그 포트에서 대기 중인 서버가 있으면 재사용합니다. 다른 포트를 사용하려면 `E2E_PORT`를 설정하세요. 예를 들어 두 번째 체크아웃이 이미 4173에서 자체 빌드를 서빙하고 있을 때입니다.

코딩 규칙

- 자동 포매팅은 pre-commit hooks를 통해 매 커밋마다 실행됩니다. [pre-commit.ci](#)는 모든 PR(포크 포함)에서 같은 hooks를 실행하고 `style: auto-format` 수정을 PR 브랜치에 자동 커밋하므로, 로컬 hooks가 놓친 것은 자동으로 수정됩니다. main으로의 푸시에서는 실행되지 않으므로, 자동 포맷된 커밋이 리뷰를 우회하는 일은 없습니다. 도구는 언어별로 나뉩니다:
- **Python + Jupyter 노트북**(`.py`, `.ipynb`) — [ruff](#)가 포맷하고 린트합니다(`ruff.toml`, 줄 길이 100, 규칙 `F/I/W/E`).
- **JS/TS/JSON/CSS/YAML/TOML** — [oxfmt](#) (npm, NAPI 바인딩)가 포맷합니다(`.oxfmttrc.json`, 2칸 들여 쓰기, 큰따옴표, 세미콜론, 너비 100). `oxfmt`는 `package.json` 키도 관례적 순서(예: `scripts`가 `dependencies` 앞)로 정렬하므로, 그 파일을 편집할 때는 공백 변경뿐 아니라 키 순서 변경도 예상하세요. `.oxfmttrc.json`의 `ignorePatterns`는 다른 도구에 속하거나 빌드마다 변하는 경로에서 `oxfmt`를 제외합니다: Jupyter 노트북과 Python 소스(`ruff` 담당), 잠금 파일(`package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`), 생성/벤더 번들(`dist/`, `build/`, `target/`, `node_modules/`, 축소된 파일, `.d.ts`), Vite 플러그인 출력(`*.generated.js`, `*.generated.py`, `cesium/`, `jupyterlite/`), 그리고 `apps/geolibre-desktop/public/` 아래의 기계 생성 앱 자산(Cesium 자산, MapLibre 스타일링 아틀라스, `oxfmt`를 거치면 약 64% 커지는 Whitebox 도구 카탈로그 스냅샷). 의도적으로 잘못된 형식의 `e2e/fixtures/malformed.geojson` 픽스처도 제외됩니다.
- **ESLint**는 React Hooks 규칙 위반에서 실패하고, 떠다니는 프로미스, `any`, 누락된 Hook 의존성, 물리적 Tailwind 방향 클래스(`m1-*`, `left-*`, `text-right`; [국제화](#) 참조)에 대해 경고합니다. 경고는 [라잇](#)으로 관리되므로, 경고를 하나 추가하는 변경은 린트에 실패합니다. `npm run build` 타입 검사도 실행됩니다.
- **노트북 출력은 매 커밋마다 저장소 전체에서 제거됩니다.** `strip-notebook-outputs` 혹은 커밋에 포함된 노트북만이 아니라 추적되는 모든 `.ipynb`에 대해 `nbstripout`을 실행하므로, 로컬에서 실행된 출력이나 종의 무관한 커밋에 딸려 들어갈 수 없습니다. `nbstripout`은 제자리에서 다시 쓰기 때문에, 다음에 무언가를 커밋할 때 아직 작업 중인 노트북의 출력도 지워집니다. 노트북을 다시 실행해 출력을 되살리거나, 추적되지 않는 임시 사본을 보관하세요. 어떤 `.ipynb`도 git에 출력을 담고 있어서는 안 됩니다.
- 모든 텍스트 파일은 `.gitattributes` (`* text=auto eol=lf`)를 통해 LF로 정규화되므로 플랫폼 간에 줄 끝이 일관됩니다. `core.autocrlf`는 재정의됩니다.
- 나머지 pre-commit 혹은 LF 줄 끝(`mixed-line-ending --fix=lf`)과 기본 공백(파일 끝 및 후행 공백 수정기)을 강제합니다. 수동으로 포맷할 일은 거의 없습니다. 한 번 커밋하고, hooks가 다시 쓰게 한 다음, 수정된 파일을 `git add` 하고 다시 커밋하세요. 클론 후 `pre-commit install`로 hooks를 설치하세요.
- `node_modules`의 파일을 편집하지 마세요. 서드파티 MapLibre 컨트롤에 앱 전용 스타일링이 필요하다면, 해당 컨트롤의 클래스로 한정된 범위 지정 오버라이드를 `apps/geolibre-desktop/src/index.css`에 추가하세요.
- 변경은 그것이 속한 패키지 안으로 한정하고, `packages/ui`의 공유 프리미티브와 `packages/core`의 헬퍼를 재사용하는 쪽을 택하세요.

문서

사이트는 같은 팀이 만든 MkDocs Material의 후속작인 **Zensical**로 빌드됩니다. 기존 `mkdocs.yml` 설정을 그대로 읽습니다. 변경 사항을 로컬에서 미리 보려면:

```
python -m pip install -r requirements-docs.txt
zensical serve
```

<http://localhost:8000>을 여세요. 새 페이지를 추가하면 `mkdocs.yml`의 `nav`에 추가하세요. CI에서는 사이트가 `zensical build --strict`로 빌드되므로, 깨진 링크와 내비게이션에서 빠진 페이지는 빌드를 실패시킵니다. 다른 문서 페이지로는 상대 경로(예: `architecture.md`)로 링크하고, `docs/` 외부의 파일로는 전체 GitHub URL로 링크하세요.

이미지, 데모, 샘플 데이터

스크린샷, GIF, 동영상, 샘플 데이터셋을 이 저장소에 커밋하지 마세요. 바이너리는 Git 히스토리에서 결코 완전히 삭제되지 않으므로, 그 하나하나가 CI를 포함한 모든 사람의 클론을 영구적으로 키웁니다.

대신 [opengeos/geolibre-assets](https://github.com/GeoLibre/geolibre-assets)에 넣으세요. 단순한 정적 호스트이므로 어떤 파일 형식이든 동작합니다. 그 저장소의 `main` 브랜치에 푸시된 것은 무엇이든 GitHub Pages가 단일 호스트 이름 <https://assets.geolibre.app> 아래 대응하는 경로에 보통 1분 안에 게시합니다:

저장소 경로	게시된 URL
<code>images/my-panel.webp</code>	<code>https://assets.geolibre.app/images/my-panel.webp</code>
<code>demos/my-demo.gif</code>	<code>https://assets.geolibre.app/demos/my-demo.gif</code>
<code>data/sample.parquet</code>	<code>https://assets.geolibre.app/data/sample.parquet</code>

그런 다음 Markdown에서 게시된 URL을 참조하세요:

```
![Raster style panel](https://assets.geolibre.app/images/raster-style-panel.webp)
```

이것이 위의 "`docs/` 외부의 파일로는 전체 GitHub URL로 링크" 규칙의 유일한 예외입니다: `geolibre-assets`의 자산은 GitHub blob이나 raw URL이 아니라 게시된 `assets.geolibre.app` URL로 참조합니다.

알아 두면 좋은 몇 가지:

- 정지 이미지에는 `images/`, 애니메이션과 화면 녹화에는 `demos/`, 샘플 데이터셋에는 `data/`, 지도 리소스에는 `styles/` 또는 `fonts/`를 사용하세요.
- 정지 이미지에는 **WebP** 또는 AVIF를, 데이터에는 PMTiles 또는 GeoParquet를 우선하세요. 이 사이트의 스크린샷은 WebP입니다.
- 개별 파일은 GitHub의 엄격한 제한인 100 MB 미만으로 유지하세요.
- 게시된 경로는 사실상 영구적입니다. 파일 이름을 바꾸거나 삭제하면 이미 그것을 가리키는 모든 페이지가 깨지므로, 이름은 한 번에 정하세요.

- 자산은 관대한 CORS로 제공되므로 앱이 교차 출처로 가져올 수 있습니다.

샘플 데이터셋은 여전히 GeoLibre가 운영하는 다른 호스트인 `data.geolibre.app` 에서도 제공됩니다. 그 URL들은 계속 동작하며 다시 쓸 필요가 없습니다. `geolibre-assets` 는 단지 폴 리퀘스트를 열 수 있는 쪽이므로, 새 자산은 그곳으로 보내세요.

이 저장소에 속하는 유일한 이미지는 `docs/assets/` 아래의 사이트 크롬입니다. 현재는 `mkdocs.yml` 이 로고와 파비콘에 사용하는 아이콘뿐입니다. 그 디렉터리에 추가하지 마세요. 아무것도 참조하지 않는 것이 거기 있다면, 그것은 선례가 아니라 잔재입니다.

플러그인과 확장

GeoLibre는 zip, 로컬 디렉터리, 또는 호스팅된 매니페스트 URL에서 로드되는 외부 플러그인을 지원합니다. 플러그인을 만들려면 [GeoLibre 플러그인 템플릿](#)에서 시작하고 [플러그인 API](#) 계약을 따르세요. 플러그인을 배포하기 위해 GeoLibre 자체를 포크할 필요는 없습니다.

백엔드 사이드카

선택적인 [FastAPI 사이드카](#)는 서버 측 변환 도구를 구동합니다. `conversion` 추가 기능은 변환 런타임(DuckDB, rio-cogeo 등)을 설치하고 `dev` 추가 기능은 테스트 러너를 설치하므로, 작업할 때는 둘 다 설치하세요:

```
cd backend/geolibre_server
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -e ".[conversion,dev]"
geolibre-server
```

변환 기능 없는 기본 실행(`pip install -e .` 과 일반 `uvicorn` 실행)은 [사이드카 README](#)를 참조하세요.

테스트는 저장소 루트에서 `npm run test:backend` 로, 또는 백엔드 디렉터리에서 `python -m pytest` 로 실행하세요.

후원

코드만이 기여하는 유일한 방법은 아닙니다. 여러분이나 여러분의 조직이 GeoLibre에 의존하고 있다면, 재정적 지원은 개발, 호스팅, 앱 스토어 배포를 계속 이어가게 합니다. [GitHub Sponsors](#)와 [Buy Me a Coffee](#)에 대해서는 [후원자 되기](#)를 참조하세요.

라이선스

GeoLibre는 [MIT 라이선스](#)로 배포됩니다. 기여함으로써 여러분의 기여물이 같은 조건으로 라이선스된다는 데 동의하는 것입니다.

<https://geolibre.app/maintenance/>

유지보수 노트

이 페이지는 **조용히** 깨지기 쉬운 저장소 규칙들을 모아 둔 곳입니다. 업스트림이 내보내지 않는 상수를 손으로 미리링한 것, 커버리지 및 감사 게이트, 게시 레이아웃, 생성 파일 등이 여기에 해당합니다. `CLAUDE.md` 는 이 내용을 다시 적는 대신 이 페이지로 링크합니다.

이제 막 환경을 설정하는 중이라면 **기여하기**를 먼저 읽으세요.

수동 확인이 필요한 의존성 범프

여러 기능이 업스트림 패키지가 내보내지 않는 값이나 DOM 계약에 의존하므로, GeoLibre는 이를 손으로 미리링합니다. 드리프트가 생겨도 보통 **빌드 오류는 나지 않고** 기능이 그냥 동작을 멈춥니다. 아래 패키지 중 하나를 범프한 뒤에는(**Dependabot PR 포함**) 나열된 확인을 수행하고 프론트엔드 스위트를 실행하세요.

`tests/upstream-contracts.test.ts` 는 아래 각 미러가 의존하는 정확한 형태(게시된 파일의 클래스 이름, 실제 인스턴스의 비공개 필드, 상수, 메서드 본문의 일부)를 **설치된** 패키지에서 확인하며, 실패 메시지는 읽어야 할 이 페이지의 섹션을 알려 줍니다. 자체 실제 패키지 테스트가 따로 있는 미러(각 섹션에 이름이 적힘)는 거기서 반복하지 않습니다. 임포트 시 DOM이나 WebGL을 건드리는 패키지는 임포트하는 대신 게시된 파일의 텍스트에 대해 확인합니다. 거기서 테스트가 통과하는 것이 확인의 전부는 아닙니다: 형태로는 드러나지 않는 의미(순서, 픽셀, 네트워크 동작)는 여전히 나열된 수동 단계가 필요합니다. 미러를 추가할 때는 그 계약도 거기에 추가하세요.

패치된 패키지(`patches/`)

`postinstall` 은 `patches/` 의 `patch-package` 패치를 적용하며, 각 패치 파일은 그것이 만들어진 정확한 버전을 이름에 담고 있습니다. 패치된 패키지를 그 버전 너머로 옮기는 범프는 CI에서 `npm install` 을 실패시킵니다. 새 버전에 대해 패치를 다시 생성하고(업스트림이 버그를 고쳤다면 패치를 제거), 패치 파일 이름을 바꾸고, 같은 PR에서 `package-lock.json` 을 갱신하세요.

`@carbonplan/zarr-layer` 는 `apps/geolibre-desktop` 과 `packages/plugins` 양쪽에서 정확한 버전(^ 없음)으로 선언되어 있습니다. 0.10.0까지는 패치를 가지고 있었는데, 그 버전에서 같은 non-throwing 유니폼 조회가 업스트림에 포함되었습니다 (이것이 없으면 Mesa GPU에서 줌 12 이상일 때 레이어가 사라졌습니다). 범프하기 전에 `dist/index.js` 의 `createShaderProgram` 이 여전히 `shift_x`, `shift_y`, `u_worldX0ffset` 을 `mustGetUniformLocation` 이 아니라 `gl.getUniformLocation` 으로 조회하는지 확인하세요(`tests/upstream-contracts.test.ts` 가 이를 검사합니다).

geolibre-wasm (`packages/processing/package.json`)

- **Processing 메뉴 카탈로그.** `ProcessingMenu.tsx` 는 체크인된 자동 생성 카탈로그 `apps/geolibre-desktop/src/lib/whitebox-menu-catalog.ts` (직접 편집 금지)로부터 렌더링합니다. `node scripts/gen-whitebox-menu-catalog.mjs` 를 실행하고 결과를 커밋하세요. 그렇지 않으면 새로 추가되거나 이름이

바뀐 WASM 도구가 조용히 메뉴에서 빠집니다. Processing_대화 상자_는 도구를 동적으로 나열하므로, 누락은 메뉴에서만 드러납니다. Whitebox 번역은 `opengeos/geolibre-language-packs` 의 선택적 외부 팩이며, GeoLibre의 번들 로케일 JSON에 생성되는 항목이 아닙니다.

- `MAX_VECTOR_PMTILES_ZOOM` (`packages/processing/src/wasm-convert.ts`)은 `vector_to_pmtiles` 가 허용하는 가장 깊은 줌(18 — 그 이상이면 도구가 `validation error: max_zoom must be <= 18` 로 종료)을 미러링합니다. 이 상한은 WASM 바이너리 안에 있으며 내보내지지 않습니다. 드리프트가 생기면 브라우저의 Vector to PMTiles가 타일러가 이제 허용하는 줌을 거부하거나, 사용자가 기다린 뒤에 거부될 줌을 받아들이게 됩니다. 이것은 사이드카의 상한이 **아닙니다**: `freestiler`는 24를 허용하며 (`ConversionDialog.tsx` 의 `MAX_PMTILES_ZOOM`, `backend/geolibre_server/geolibre_server/app/conversion.py` 를 미러링), 대화 상자는 실행될 엔진에 맞춰 검증합니다. 미러가 드리프트하면 `tests/wasm-convert.test.ts` 가 실패합니다.
- `DISTANCE_SEGMENTS / NON_DISTANCE_NAMES` (`apps/geolibre-desktop/src/lib/whitebox-distance-params.ts`)는 매개변수_이름_으로 어떤 Whitebox 매개변수가 지상 거리인지, 그래서 Processing 대화 상자의 미터 단위 선택기를 받을지 결정합니다(GeoLibre#1540). 세그먼트는 일반적 이름 (`tolerance`, `radius`, `length`, `resolution`)이므로, 길이가 아닌데도 일치하는 이름을 가진 도구가 있을 수 있습니다. `corridor_tolerance` 는 0-1 범위의 비율입니다. 지금은 선택기가 모든 데이터셋 입력이 벡터 레이어인 도구로 한정되어 있고, 충돌하는 이름들이 우연히 영상/LiDAR 도구에 있기 때문에 안전할 뿐이며, 이는 우연이지 보장이 아닙니다. 새 카탈로그에서 규칙에 일치하는 `double` 중 설명이 비율, 비, 각도, 가중치로 읽히는 것을 찾아 `NON_DISTANCE_NAMES` 에 추가하세요. 하나라도 놓치면 그 도구의 필드는 미터 단위를 제공하고 무차원 숫자를 거리인 것처럼 조용히 변환합니다.

maplibre-gl

- `GLOBE_CONTROL_TOGGLE_SELECTOR` (`packages/map/src/globe-control-toggle.ts`)는 MapLibre 자체 `GlobeControl` 이 토크 버튼에 붙이는 클래스 이름 — `maplibregl-ctrl-globe` 와 `maplibregl-ctrl-globe-enabled`, 투영이 바뀔 때마다 교체됨 — 을 미러링합니다. `MapCanvas` 는 `projectiontransition` 이벤트가 아니라 그 버튼에 대한 **클릭**으로부터 투영 변경을 저장하는데, 스타일 초기화와 프로젝트 정합도 그 이벤트를 발생시켜 오래된 이벤트가 방금 로드된 프로젝트의 투영을 덮어쓰기 때문입니다. `tests/globe-control-toggle.test.ts` 는 실제 `GlobeControl` 을 만들어 미러가 더 이상 일치하지 않으면 실패합니다.
- **레이어별 블렌드 모드** (`packages/map/src/layer-blend-modes.ts`)는 세 개의 내보내지지 않는 `maplibre-gl` 내부를 감쌉니다. MapLibre는 모든 레이어를 하나의 캔버스에 렌더링하고 레이어별 블렌드 API를 제공하지 않기 때문입니다(업스트림 초안: `maplibre/maplibre-gl-js#8073`). 래퍼는 `Painter.prototype.renderLayer` (한 레이어의 드로우를 감쌘), `Painter.prototype.useProgram` (레이어 불투명도 합성 드로우를 그것에 공급하는 드로우들과 구별), `Context.prototype.setColorMode` (모든 드로우가 GL 블렌드 상태를 결정하는 단일 지점)입니다. 채우기와 선 레이어는 추가로 `style-mapper` 가 `fill-layer-opacity / line-layer-opacity` 를 1 바로 아래로 고정하여 MapLibre 6의 `render-to-texture` 합성을 선택하게 하므로, 레이어가 겹치는 폴리곤마다가 아니라 **전체로서** 블렌드됩니다. `installLayerBlendModes` 는 모든 접합부를 기능 감지하여 지도를 망가뜨리는 대신 기능을 비활성화(Style 패널 컨트롤 숨김)하므로, 드리프트는 조용히 실패합니다. 그래서 `tests/layer-blend-modes.test.ts` 가 접합부를 단언하고 `e2e/blend-modes.spec.ts` 가 실제 픽셀을 단언합니다. 범프 시 둘 다 실행하세요.

목록을 확장하기 전에 [블렌드 모드 추가하기](#)를 참조하세요.

- `DEFAULT_MARKER_OFFSET_Y` (`apps/geolibre-desktop/src/components/storymap/storymap-engine.ts`)는 `maplibregl.Marker` 가 기본 핀에 적용하는 `-14 px` 수직 오프셋을 미러링합니다. `createStoryMapMarker` (`apps/geolibre-desktop/src/lib/field-collection-map.ts` 의 `FieldCollection` 캡처 마커에서도 재사용)는 모든 엔진(Mapbox, Cesium, ArcGIS, MapLibre 모두)에서 그 핀을 손으로 배치하므로, 범프가 기본 핀의 앵커나 오프셋을 바꾸면 스토리 마커가 아무 오류 없이 좌표에서 어긋납니다. `tests/upstream-contracts.test.ts` 는 게시된 번들에서 기본 오프셋을 읽어 그것이 바뀌면 실패합니다. 핀의 앵커가 바뀌었다면 MapLibre가 아닌 렌더러에서도 스토리를 재생해 보세요.

@deck.gl/mapbox 및 @deck.gl/maplibre

`bridgeArcgisDeckControl` (`packages/plugins/src/plugins/arcgis-deck/control-adapter.ts`)은 각 오버레이의 비공개 `_props` 필드를 읽어, ArcGIS가 MapLibre 컨트롤을 마운트하지 않고도 초기 레이아웃을 네이티브 `ArcgisDeckOverlay` 로 옮길 수 있게 합니다. 캐스트가 업스트림 변경을 TypeScript로부터 숨깁니다. 어느 `deck.gl` 패키지든 범프한 뒤에는 `tests/arcgis-control-adapters.test.ts` 를 실행하고 오버레이가 여전히 초기 `DeckProps` 객체를 담은 `_props` 를 노출하는지 확인하세요.

@loaders.gl/tiles (@deck.gl/geo-layers 를 통해) — 타일 캐시 상한

`applyThreeDTilesTilesetMemoryLimit` (`packages/plugins/src/plugins/arcgis-i3s-tiles.ts`)은 `Tileset3D` 가 로드 옵션에서 결코 복사하지 않는 `maximumMemoryUsage` 필드에 대해 타일 캐시를 정리하는 문제를 우회합니다. 그래서 상한이 32 MB에 머물고 카메라가 움직일 때마다 방금 그린 타일이 쫓겨납니다(이슈 #2560). 같은 PR에서 `memoryAdjustedScreenSpaceError` 를 껐는데, 그 캐시가 차면 세부 수준을 아래로 래치하기 때문입니다. `deck.gl`이나 `loaders.gl`을 범프한 뒤에는 `tests/arcgis-i3s-tiles.test.ts` 를 실행하세요. 실제 `Tileset3D` 케이스는 필드 이름이 바뀌거나 더 이상 32 MB에서 시작하지 않으면 실패합니다. 업스트림이 이 옵션을 존중하기 시작하면 헬퍼는 제거할 수 있습니다.

@maplibre/maplibre-gl-style-spec

`propertySpecFor` (`packages/core/src/expressions.ts`)는 `createExpression` 이 기대 결과 타입 강제 (Expression Builder의 필터 → 불리언 / 색상 검사)에 사용하는 **내보내지지 않는**

`StylePropertySpecification` 형태를 만들어 냅니다. 캐스트가 계약 변경을 컴파일러로부터 숨기므로 프론트엔드 스위트를 실행하세요. `tests/expressions.test.ts` 의 "enforces an expected result type" 테스트는 형태가 더 이상 존중되지 않으면 실패합니다.

`cssColor` (`packages/map/src/feature-style.ts`)는 컴파일된 색상 표현식이 평가되어 나오는 `Color` 객체의 `toString()` 을 읽고 `rgba()` 또는 `#` 접두사를 받아들여 CSS로 바꿉니다. 그 형식은 스펙의 `Color` 가 우연히 그렇게 출력되는 것이지 문서화된 계약이 아닙니다. 범프가 이를 바꾸면 지구본은 실패하는 대신 분류된 모든 피처를 조용히 단색 폴백 색으로 칠합니다. `tests/cesium-feature-style.test.ts` 의 "classifies by a categorical field" 테스트는 실제로 컴파일된 표현식을 거치므로, 범프 후 프론트엔드 스위트를 실행하고 지구본에서 분류된 레이어를 다시 확인하세요.

`SPEC_DEFAULT_COLOR` (`packages/map/src/mapbox-style-import.ts`)는 `fill-color`, `line-color`, `circle-color`에 대한 스펙의 `default` — 셋 모두 `#000000` — 를 미러링하며, 이는 색을 지정하지 않은 스택 클래스 레이어가 임포트될 때의 색입니다. `@geolibre/map`은 게시되는 패키지이고 스펙에 의존하지 않기 때문에 (`packages/core`만 의존) 스펙에서 읽지 않고 하드코딩되어 있습니다. 프론트엔드 스위트는 양방향을 보호합니다: `tests/mapbox-style-import.test.ts`의 테스트는 스펙이 여전히 `SPEC_DEFAULT_COLOR`를 말하는지 단언하고, 그 옆의 동작 테스트는 임포트된 무색 클래스가 그 색으로 렌더링되는지 단언하므로, 어느 쪽이 바뀌어도 실패합니다.

같은 파일의 `COLOR_OVERRIDING_PAINT`는 그 조회의 나머지 절반을 미러링합니다: 피처를 스스로 그리는 페인트 속성들이며, 그래서 색상 기본값이 적용되지 않습니다. 스펙은 이를 `line-color`의 `requires: [{ "!": "line-pattern" }]`로 인코딩하며, 같은 테스트가 이를 단언합니다. `fill-pattern`과 `line-gradient`도 같은 이유로 나열되어 있지만 스펙은 이를 인코딩하지 않으므로, 테스트로 보호할 수 있는 미러는 `line-pattern` 항목뿐입니다. 나머지 둘은 MapLibre가 그것들을 렌더링하는 방식에 기대고 있습니다. 이들 중 하나라도 사용하는 클래스는 검정으로 임포트되지 않고 스택을 거부합니다.

Web Services 컨트롤 패키지(`packages/plugins/package.json`)

- **도킹 패널 DOM 브리지.** `dockable-map-control.ts`는 `maplibre-gl-fema-wms`, `maplibre-gl-nasa-earthdata`, `maplibre-gl-enviroatlas`, `maplibre-gl-national-map`을 컨트롤 생명주기를 직접 호출하고, `onAdd()`가 지도 컨테이너에 덧붙이는 패널 요소를 GeoLibre의 네이티브 오른쪽 패널 호스트로 옮기는 방식으로 적응시킵니다. Vantor는 대신 래퍼를 반환하므로 브리지는 그 `.vantor-panel` 자손을 선택합니다. `index.css`의 범위 지정 CSS는 각 패키지의 패널, 헤더, 토글, 닫기, 크기 조절 핸들 클래스 이름을 미러링합니다. `tests/upstream-contracts.test.ts`는 패키지별로 이들을 나열하고, 패키지가 그중 하나를 더 이상 렌더링하지 않으면(또는 `index.css`가 더 이상 스타일링하지 않으면) 실패합니다. 이 패키지들 중 하나라도 범프한 뒤에는 마이그레이션된 모든 Web Services 플러그인을 활성화하고, 카탈로그가 렌더링되는지, GeoLibre 도크의 크기를 조절해도 내용이 유지되는지, 지도 컨테이너 아래에 벤더 패널이 남아 있는지 확인하세요.

maplibre-gl-components (`packages/plugins/package.json`)

- `MAP_PANEL_SELECTOR` (`apps/geolibre-desktop/src/components/layout/RecordVideoDialog.tsx`)는 **렌더링된** 컨트롤 클래스 이름 — `maplibre-gl-html-control`, `maplibre-gl-legend`, `maplibre-gl-colorbar` — 을 미러링하여, Record Video의 "Include map panels" 옵션이 지도 위 오버레이들을 녹화에 래스터화할 수 있게 합니다. 이들은 표시 요소이며, 의도적으로 `*-gui-control` 작성 편집기는 **아닙니다**. 클래스가 드리프트하면 옵션이 그 패널을 영상에 굽는 일을 조용히 멈추거나(또는 체크박스가 아예 나타나지 않거나) 빌드 오류는 나지 않습니다. 대신 `tests/upstream-contracts.test.ts`가 실패합니다.
- **PMTiles 컨트롤의 레이어 id** (`pmtilesControlLayerId` / `pmtilesIdsForSourceLayers` / `pmtilesIdNamesSourceLayer`, `packages/map/src/pmtiles-layer.ts`, `layer-sync.ts`와 `packages/plugins/src/plugins/components/pmtiles.ts`에서 읽음)는 `PMTilesLayerControl`에 관한 내보내지지 않은 사실을 미러링합니다: 컨트롤은 MapLibre 레이어 이름을 **원시** 소스 레이어 이름으로부터 `${sourceId}-${name}-${kind}`로 짓는 반면, `pmtilesVectorLayerId`는 이를 퍼센트 인코딩합니다.

인코딩이 필요 없는 모든 이름에 대해 둘은 일치하므로, 컨트롤 자체의 id를 가진 스토어 레이어 — 통째로 유지된 아카이브나, 자기 소스 레이어를 가리키는 id를 유지하는 분할된 부분 — 는 이름에 /, 공백, 또는 비 ASCII가 들어가기 전까지는 인코딩 체계만으로도 인식됩니다. 그 시점이 되면 `layer-sync` 는 소스 레이어에 네이티브 레이어가 없다고 판단하고 컨트롤의 것 위에 **두 번째** fill/line/circle 트리오를 추가합니다: 두 번 그려지고, 패널에 응답하는 것은 컨트롤의 사본뿐입니다. 따라서 두 체계 모두 매칭하며, 스토어가 실제로 보유한 소스 레이어를 가리키는 id만 유지합니다.

사용자가 **체크한** 것은 의도적으로 그 id들로부터 추론하지 않습니다: `selectedSourceLayers` 는 모든 핸들러에 전달되는 내보내진 `PMTilesLayerControlState` 의 문서화된 필드이므로, `pmtilesLayerOptions` 가 이를 읽고 컴파일러가 검사합니다. 오래된 선택에 대한 규칙과, 컨트롤이 패널 닫기를 가로질러 재사용하는 아카이브 id에 대한 규칙은 그 함수와 `addPMTilesArchive` 에 적혀 있습니다. 재사용된 id는 GeoLibre가 복구할 수 없는 유일한 경우입니다: 두 아카이브가 하나의 MapLibre 소스를 가리키게 되어, 먼저 동기화된 쪽이 차지하고 다른 쪽은 아무것도 그리지 않으므로, `addPMTilesArchive` 는 아닌 척하는 대신 경고합니다. 반면 레이어를 완전히 넘겨받는 아카이브는 대체한 것의 이름, 폴더, 스타일링을 유지하면서 올바르게 그려집니다. `tests/pmtiles-control-contract.test.ts` 는 **실제** 컨트롤을 실제 아카이브에 대해, 실제 `layeradd` 핸들러를 거쳐 스토어까지 구동하며, id 체계가 바뀌거나 선택이 핸들러에 도달하지 않으면 실패합니다.

- **MeasureControl의** `_panel` 과 `_sourceId` (`packages/plugins/src/plugins/terrain-measure.ts`, `measurePanelElement` 와 `measureSourceId` 를 통해)는 컨트롤만 아는 비공개 멤버입니다: 패널은 GeoLibre의 Terrain (3D)/heading 섹션과 크기 조절 스타일링을 담고, 소스 id는 `lidar-measure-mirror.ts` 가 LiDAR 포인트 클라우드 위에 다시 그릴 지오메트리를 찾는 방법입니다(#2533). 두 리더 모두 이름이 바뀌면 경고하고 아무것도 하지 않는 쪽으로 폴백합니다. `tests/upstream-contracts.test.ts` 는 실제 `MeasureControl` 을 만들어 어느 필드든 사라지면 실패합니다. 범프 후에도 Terrain 섹션이 나타나는지, 측정된 선이 포인트 클라우드 안에서 보이는지 확인하세요.
- **측정 페인트**(`MEASURE_LINE_COLOR` / `MEASURE_LINE_WIDTH` / `MEASURE_FILL_COLOR` , `packages/plugins/src/plugins/lidar-measure-mirror.ts`)는 기본값에 맡기지 않고 컨트롤에 명시적으로 전달됩니다. `deck.gl` 미러가 같은 지오메트리를 같은 색으로 다시 칠해야 하기 때문입니다. 한 형식을 바꾸면 그 옆의 RGBA 쌍도 바꾸세요. `tests/lidar-measure-mirror.test.ts` 가 둘이 일치하는지 단언합니다.

maplibre-gl-basemap-control (`packages/plugins/package.json`)

`BASEMAP_PANEL_SELECTOR` / `BASEMAP_ROW_SELECTOR` / `BASEMAP_ROW_ID_ATTR` (`packages/plugins/src/plugins/basemap-thumbnails.ts`)는 컨트롤이 렌더링하는 DOM — `.basemap-control-panel` , `.basemap-control-result` , `data-basemap-id` — 을 미러링하며, Basemaps 패널의 썸네일은 이를 통해 행을 찾고 각 행을 카탈로그 항목에 다시 연결합니다. 그 패키지는 `BasemapControl` / `BasemapDefinition` 만 내보내므로, 클래스 이름이 바뀌어도 빌드 시점에는 아무것도 실패하지 않습니다: 쿼리가 더 이상 일치하지 않고 썸네일이 조용히 나타나지 않게 됩니다. `tests/basemap-thumbnails.test.ts` 는 실제 컨트롤을 만들어 렌더링된 패널을 미러에 대해 단언합니다.

같은 파일의 `hasUnresolvedPlaceholder` 는 그 패키지의 자격 증명 플레이스홀더 (`{api-key}` , `{aws-region}`)를 미러링하는 대신, 자신이 치환하는 타일 토큰의 **여집합**을 의도적으로 매칭합니다. 그래서 새 공급자의 플레이스홀더는 문자 그대로 가져와지는 대신 건너뛰어집니다. 플레이스홀더 이름을 열거하지 말고 그대로 유지하세요.

maplibre-gl-vector (packages/plugins/package.json)

`MAX_VECTOR_BYTES` (`packages/plugins/src/plugins/remote-file-formats.ts`)는 그 패키지의 **내부, 내보내지지 않은** 상수 `MAX_REMOTE_FILE_BYTES` (2 GiB — DuckDB-WASM은 원격 파일 크기를 32비트로 보관)를 미리 링합니다. 임포트할 수 없으므로, 그 패키지의 `src/lib/utils/remote.ts` 를 다시 확인하고 값이 바뀌었으면 미리를 갱신하세요. 드리프트가 생기면 원격 탐색 패널(Source Cooperative, Hugging Face)이 엔진이 이제 열 수 있는 GeoParquet를 조용히 차단하거나, 반드시 실패할 Add를 제공합니다. 상수만 갱신하면 됩니다: 사용자에게 표시되는 제한은 그 값에서 렌더링되며 문구에 적혀 있지 않습니다. `tests/upstream-contracts.test.ts` 는 게시된 체크에서 상수를 읽어 비교합니다.

`remote-file-formats.ts` 는 이 규칙과 그 패널들이 공유하는 다른 형식/리더/크기 규칙의 **단일** 거처입니다. 패널 별 사본은 이 검사를 놓치므로, 새 탐색 패널은 복제하지 말고 그 모듈에 대해 추가하세요(`source-coop-api.ts` 는 호환성을 위해 자체 이름으로 다시 내보냅니다).

채택(`vector-layer-sync.ts` 의 `adoptVectorControlLayers` , #2715)은 컨트롤의 작은 GeoJSON 모드 레이어를 GeoLibre에 넘기며, 패키지가 약속하지 않는 세 가지에 기댁니다. `getLayerGeoJSON` 은 `renderMode: "geojson"` , `ingestMode: "table"` 레이어에 대해 전체 컬렉션을 반환해야 합니다. `removeLayer` 는 지도 소스 뿐 아니라 레이어의 DuckDB 테이블도 제거해야 하며, 그렇지 않으면 채택이 데이터 사본을 두 개 유지하게 됩니다. 그리고 `KML_ICON_PROPERTY` 는 컨트롤의 KML/KMZ 아이콘 레이어가 필터링하는 피쳐 속성 (`__geolibre_kml_icon_url`)을 미리링합니다. 이를 가진 레이어는 컨트롤에 남으므로, 업스트림에서 이름이 바뀌면 KML 레이어가 채택되어 아이콘을 잃게 됩니다. 범프 시 세 가지 모두 다시 확인하세요(계약 테스트는 `KML_ICON_PROPERTY` 만 다룹니다).

maplibre-gl-lidar (packages/plugins/package.json) — 컴파일러가 절반만 검사

공간 효과 엔진은 MapLibre 캔버스를 `z-index: 4` 로 올려 별 배경 캔버스가 아래에 놓일 수 있게 합니다. 그 패키지는 포인트 클라우드를 **오버레이된** `deck.gl` 캔버스에 렌더링하며, 이를 캔버스 컨테이너 안, 지도 캔버스 바로 뒤에 `DECK_CANVAS_CLASS` (`maplibre-gl-lidar-canvas`) 태그를 붙여 둡니다. `effectsOverlayCss()` (`packages/plugins/src/plugins/maplibre-effects.ts`)는 그 래퍼에 캔버스 자체의 `z-index`를 넘깁니다. 이 규칙을 빼면 래퍼가 올려진 캔버스 아래로 떨어져 포인트 클라우드가 완전히 숨겨지고, 업스트림에서 재부호화를 빼면 래퍼가 다시 Measure/Colorbar/Legend/HTML/Bookmark 패널을 덮게 됩니다(#2530).

`lidar-measure-mirror.ts` 는 Measure 도구의 선/폴리곤을 같은 오버레이 (`LidarControl.getDeckOverlay()`)에 `depthTest: false` 로 그립니다. 이는 플러그인 자체의 단면선이 포인트 위에 놓이기 위해 쓰는 기법입니다. `addLayer(id, layer, { overlay: true })` (0.21+)로 추가되어 나중에 스트리밍되어 들어오는 체크를 포함한 모든 포인트 클라우드 체크 다음에 그려지며, 포인트 클라우드 주석기의 강조, 박스, 벡터 레이어도 같은 플래그를 사용합니다. 컴파일러가 검사하지 않는 것이 하나 있습니다: 지오메트리는 MapLibre/Mapbox `geojson` 소스의 `_data` 필드에서 읽는데, 어느 라이브러리도 공개 리더를 노출하지 않기 때문입니다. 이를 잃으면 미러만 잃을 뿐입니다. 측정된 선이 다시 클라우드 안에 숨겨지는데, 이것이

2533이었습니다. tests/upstream-contracts.test.ts 는 두 형태를 실제

라이브러리(실제 MapLibre GeoJSONSource, mapbox-gl 번들의 setData)에 대해 검사합니다.

클래스는 패키지의 DECK_CANVAS_CLASS 를 임포트가 아니라 손으로 유지하는 사본입니다: maplibre-gl-lidar 는 자체 지연 청크로 빌드되며, 이 문자열 하나를 임포트하는 것만으로도 그 청크가 시작 경로에 끌려옵니다. tests/effects-settings.test.ts 는 기대 셀렉터를 패키지 내보내기로부터 만들므로, 업스트림에서 이름이 바뀌면 npm run test:frontend 가 실패합니다. 배치는 컴파일러에 보이지 않으므로, e2e/lidar-canvas-stacking.spec.ts 가 실제 컨트롤을 마운트하고 결과 DOM 순서와 z-index를 단언합니다. 범프 시 실행하세요 (npx playwright test e2e/lidar-canvas-stacking.spec.ts --project=features).

포인트 클라우드 주석기의 Full detail in view는 loadRegion / clearPinnedRegion (0.20+)을 호출하며, 업스트림 플래그 없이 클라우드가 자격이 있는지 결정합니다: 스트리밍 COPC는 소스에 .copc. 가 들어 있고 nodeRanges 가 "file" 이 아니라 옥트리 키를 담은 것입니다(point-cloud-annotation/index.ts 의 canLoadFullDetail). 저장된 라벨도 고정된 영역의 노드가 옥트리 키를 유지한다고 가정하므로, 노드 키나 고정 의미를 바꾸는 범프는 e2e/point-cloud-annotation.spec.ts ("loads the view at full detail")를 다시 실행해야 합니다.

주석기의 라벨 인코딩(델타 varint 포인트 인덱스, 그 다음 클래스 바이트 또는 varint 인스턴스 id, 원시 DEFLATE, base64; label-store.ts)은 컴파일러가 연결할 수 없는 세 곳에서 디코딩됩니다: geolibre.project (Python 리더), geolibre.pointcloud (사전 라벨 및 라벨된 파일 재작성), 그리고 backend/geolibre_server/geolibre_server/app/pointcloud.py 의 내장 작업 스크립트 (/pointcloud/apply-labels, 변환 런타임에서 실행되며 Python 패키지를 임포트할 수 없음). 이들의 테스트는 앱에서 인코딩한 픽스처(인스턴스 id의 경우 Y2BkWcP0ahHLfyDgBwA=)를 공유하므로, 형식 변경은 세 곳과 픽스처를 함께 갱신해야 합니다. 이들이 계층 구조로부터 재구성하는 COPC 노드 순서(청크 오프셋으로 정렬된 데이터 노드)는 maplibre-gl-lidar의 nodeRanges 인덱스가 노드 안에서 가리키는 순서와 일치해야 합니다.

?data= LiDAR 딥링크는 컴파일러가 볼 수 없는 두 가지에 더 기댑니다. isStreamedLidarUrl (apps/geolibre-desktop/src/lib/data-url.ts)은 LidarControl.loadPointCloud 맨 위의 라우팅(/ept.json 접미사 또는 URL 어디에든 있는 .copc. 는 스트리밍, 그 밖의 것은 통째로 다운로드)을 복사하여, 통째로 다운로드하는 경우에 만 크기 검사를 받게 합니다. 업스트림이 그 라우팅을 바꾸면 사본을 갱신하세요. 그렇지 않으면 스트리밍 파일이 불필요한 크기 검사를 받고 다운로드 파일은 검사를 건너뛰니다. addLidarLayerFromUrl 은 또한 loadPointCloud 가 resolve되기 전에 load 가 발생하고 스토어 레이어가 추가되는 데 의존하며, 그렇지 않으면 throw합니다. tests/lidar-url-layer.test.ts 는 둘의 GeoLibre 쪽을 고정하고, tests/upstream-contracts.test.ts 는 loadPointCloud 가 그 문자열들로 라우팅하기를 멈추면 실패합니다. 범프 시 loadPointCloud 를 다시 읽으세요.

포인트 클라우드 주석기(packages/plugins/src/plugins/point-cloud-annotation/)는 패키지가 0.18에서 추가한 포인트 편집 API(getPointCloudData, refreshPointColors, getRenderSettings, pauseStreaming / resumeStreaming, isStreamingLoading, DeckOverlay.getViewports)를 통해 컨트롤이

로드한 포인트를 편집하며, 모두 `lidar-access.ts` 에서 호출됩니다. 컴파일러가 검사하지 않는 것이 두 가지 있습니다. 편집은 `getPointCloudData` 가 반환하는 배열에 쓰는데, 이는 그 배열들이 레이어가 렌더링하는 버퍼(스트리밍 클라우드의 경우 로더 버퍼의 뷰)라는 데 의존합니다. 저장된 라벨은 `nodeRanges ( (node key, index - start) )`로 키가 지정되므로, 로더가 노드의 포인트를 정렬하는 방식이 바뀌면 라벨이 조용히 잘못된 포인트에 다시 적용됩니다. `classes.ts` 의 `ASPRS_CLASSES` 는 패키지의 내보내지지 않은 `CLASSIFICATION_COLORS` 를 손으로 미러링합니다. `tests/point-cloud-annotation.test.ts` 는 `ColorSchemeProcessor` 를 통해 실제 색을 읽어와 드리프트 시 실패합니다. 범프 시 `npx playwright test e2e/point-cloud-annotation.spec.ts --project=features` 를 실행하세요.

maplibre-gl-layer-control (packages/map/package.json) — 비공개 내부

`packages/map/src/layer-control-host.ts` 는 지도 위 레이어 컨트롤을 공개 옵션과 어댑터 메서드(0.18.0에서 추가된 레이어 그룹 메서드 포함)를 통해 구동하지만, 컴파일러가 검사할 수 없는 비공개 멤버에도 손을 뻗습니다. 그 파일의 `LayerControlInternalState` 가 이들을 나열합니다:

- `panel` 과 `.layer-control-item` / `.layer-control-checkbox` / `.layer-control-opacity` / `.layer-control-name` DOM. 스토어의 가시성, 불투명도, 이름을 이미 구축된 패널에 제자리에 미러링하기 위한 것입니다.
- `state.layerStates` (같은 이유)와 `state.collapsed`. 재구축 전에 읽어, 열려 있던 컨트롤이 열린 상태로 다시 마운트되게 합니다.
- `basemapLayerIds`. 가져올 수 있는 베이스맵 스타일 URL이 없을 때 시드됩니다.
- `buildLayerItems()`. 루트 베이스맵 레이어가 없는 스타일에 Background 행을 추가하고, 스토어가 거부한 재정렬 후 행을 다시 그리기 위해 호출됩니다.

업스트림이 이 중 하나라도 이름을 바꾸면 패널이 오류 없이 스토어를 따르지 않게(또는 재구축이 패널을 닫게) 됩니다. `tests/upstream-contracts.test.ts` 는 실제 `LayerControl` 을 만들어 위의 모든 멤버와 행 클래스 이름을 검사합니다. 재구축이 이들을 어떻게 사용하는지는 검사할 수 없으므로, 범프 시 앱에서 컨트롤을 직접 구동하세요: 그룹화된 레이어를 토글하고, 그룹 안에서 재정렬하고, 그룹을 숨기세요. 수정은 공개 API로서 업스트림에 속합니다. 이 패키지는 우리 것입니다 (`opengeos/maplibre-gl-layer-control`).

maplibre-gl-splat (packages/plugins/package.json) — 비공개 내부

`packages/plugins/src/plugins/components/splatting.ts` 는 컴파일러가 검사할 수 없는 `GaussianSplatControl` 의 비공개 필드에 손을 뻗습니다:

- `reserveSplattingIds` 는 `_layerCounter` / `_modelCounter` 인스턴스 필드를 접근자로 대체합니다. 업스트림은 각 자산의 이름을 `splat-${this._layerCounter++}` / `model-${this._modelCounter++}` 로 짓고, 새 컨트롤은 0에서 다시 시작하므로, 접근자가 없으면 새로 로드한 것이 저장된 레이어의 id를 가져가 덮어쓸 수 있습니다. 저장된 레이어를 자체 id로 복원하는 것도 이들을 거칩니다.

- `recordSplattingPlacements` 는 인스턴스의 `loadSplat / loadModel` 을 감싸고 `_splatLayers / _modelLayers` (자산별 경도/위도/고도), `_state.rotation / _state.scale`, `_options.defaultModelRotation` 을 읽어, 스토어 레이어가 복원에 필요한 배치를 담도록 합니다. 복원은 실행 중에 `_options.flyTo` 를 끕니다.

업스트림이 그 필드들의 이름을 바꾸거나 id를 할당하는 방식을 바꾸면, id 예약과 배치 복원이 오류 없이 동작을 멈춥니다. `tests/splatting-restore.test.ts` 는 같은 형태의 가짜를 구동하므로 이를 잡아내지 못합니다.

`tests/upstream-contracts.test.ts` 는 실제 컨트롤을 만들어 각 필드와 `splat-${counter} / model-${counter}` 명명, 그리고 `loadSplat / loadModel` 의 배치 기록을 검사합니다. 더 나은 방법은 id 옵션과 자산별 배치 getter를 업스트림에 올리고 패칭을 제거하는 것입니다.

@geoman-io/maplibre-geoman-free (packages/plugins/package.json) — change 모드 내부

Geoman의 change 모드는 오른쪽 클릭으로 정점을 제거하지만, `LineString`, `Polygon`, `MultiPolygon`에 대해서만 그렇습니다. `packages/plugins/src/plugins/maplibre-geo-editor.ts` 의 `installMultiLineVertexRemoval` 은 `MultiLineString`을 추가합니다(토론 #2750). 이는 `geoman.actionInstances` 에 훅을 걸고 `edit__change` 액션의 `cutVertex` 를 감싸며, `featureData / markerData` 페이로드를 읽고 `fireFeatureUpdatedEvent` 를 호출합니다. 이 중 어느 것도 컴파일러가 검사하지 않습니다. 여기서는 `patch-package` 도 도움이 되지 않습니다: 앱은 루트가 아니라 `packages/plugins` 와 `apps/geolibre-desktop` 아래의 중첩 사본을 로드합니다.

범프가 액션 키나 그 멤버들의 이름을 바꾸면, 래퍼가 조용히 적용을 멈추고 `MultiLineString` 정점은 다시 `EditChange.cutVertex: feature not updated` 를 로그에 남기게 됩니다. `tests/upstream-contracts.test.ts` 는 두 중첩 사본의 `dist/maplibre-geoman.es.js` 에서 훅 지점을 검사합니다(텍스트만: Geoman은 살아 있는 지도 없이는 빌드할 수 없습니다). 범프 시 Edit 모드에서 `MultiLineString` 정점을 오른쪽 클릭해 보세요. 업스트림이 `MultiLineString` 지원을 추가하면 래퍼를 제거하세요.

zarr-cesium (packages/map/package.json) — 비공개 내부

`packages/map/src/cesium-zarr-imagery.ts` 는 지구본에 Zarr 레이어를 그리며, `zarr-cesium`이 API로 내보내지 않는 두 가지에 의존합니다:

- `ZarrLayerProvider` 는 이름 있는 `matplotlib` 컬러맵만 존중하므로(문서화된 `colorScale` 옵션은 0.3.3에서 전달되지 않음), GeoLibre는 레이어의 램프를 프로바이더의 비공개 `source.colorScale.colors` 에 쓰고 `source.updateColormapTexture()` 를 호출합니다. 이들이 옮겨지면 지구본은 기본 램프를 그리고 경고 하나를 로그에 남깁니다.
- `ZARR_DIMENSION_ALIASES` 는 `zarr-maps-tiling`의 `DIMENSION_ALIASES_DEFAULT` (`time / elevation` 아래로 분류하는 이름들)를 미러링합니다. 드리프트가 생기면 선택기가 잘못된 키로 가서 Time Slider가 아무것도 넘기지 못합니다.

`tests/cesium-zarr-imagery.test.ts` 는 실제 프로바이더를 생성하고 실제 별칭 테이블을 임포트하므로, 범프 시 둘 다 CI에서 실패합니다. 실행하고, 업스트림이 이제 `colorScale` 을 전달하여 비공개 쓰기를 제거할 수 있는지 확인하세요. `zarr-cesium`은 `cesium` 래퍼도 임포트하는데, `apps/geolibre-desktop/vite.config.ts` 가 이를 `@cesium/engine` 으로 별칭합니다. 범프가 `cesium` 임포트를 더 추가하더라도 그 별칭을 유지하세요. 그렇지 않으면 지구본이 두 번째 엔진을 로드합니다.

maplibre-gl-planetary-computer (packages/plugins/package.json) — 비공개 STAC 클라이언트

Planetary Computer STAC API는 `GET /collections` 에 CORS 헤더를 보내지 않고 `JSON POST /search` 에 필요한 프리플라이트를 거부하므로, 라이브러리 자체의 `STACClient` 는 브라우저와 Tauri 웹뷰에서 "Failed to fetch"로 실패합니다. `packages/plugins/src/plugins/planetary-computer-stac.ts` 는 이를 서브클래스합니다: 검색은 `GET /search` 로 나가고, 컬렉션 목록은 실시간 목록을 읽을 수 없을 때 번들된 `planetary-computer-collections.json` 으로 폴백합니다. 서브클래스는 비공개 `fetch / abortController` 필드를 재사용하고, `maplibre-planetary-computer.ts` 는 컨트롤이 추가되기 전에 이를 컨트롤의 비공개 `_stacClient` 필드에 바꿔 넣습니다(컬렉션은 `onAdd` 에서 로드됨).

업스트림이 그 필드들의 이름을 바꾸거나, 생성자에서 컬렉션을 로드하거나, 새 POST 요청을 추가하면, 패널은 컴파일러 오류 없이 다시 깨집니다. `tests/upstream-contracts.test.ts` 는 실제 클라이언트와 컨트롤에서 비공개 필드, 생성자가 아무것도 로드하지 않는다는 것, 그리고 검색이 아닌 모든 읽기가 GET이라는 것을 검사합니다. 범프 시 이것과 `tests/planetary-computer-stac.test.ts` 를 실행하세요. 새 컬렉션을 반영하려면 `node scripts/gen-planetary-computer-collections.mjs` 로 번들 목록을 다시 생성하세요. 더 나은 방법은 GET 검색과 클라이언트/fetch 옵션을 업스트림에 올리고 교체를 제거하는 것입니다.

maplibre-gl-raster — 스트레치 및 감마 곡선

`buildContinuousColormapRgba` (`packages/plugins/src/plugins/raster-symbolology.ts`)는 렌더 파이프라인의 값 조정을 **손으로**, 역으로 미러링합니다. 값 범위 불투명도는 알파를 주입된 256폭 컬러맵 텍스처에 칠하므로, 텍스처 열을 데이터 값으로 되돌려 매핑해야 합니다. 그런데 렌더러는 그 텍스처를 리스케일, 스트레치 곡선, 감마 이후에 샘플링합니다. 따라서 미러는 각 곡선의 **역함수**를 렌더러의 반대 순서로 적용합니다. 감마를 먼저, 그 다음 스트레치입니다:

- 감마 — 렌더러 `pow(x, 1 / max(gamma, 0.0001))` , 미러 `pow(t, max(gamma, 0.0001))`
- `sqrt` 스트레치 — 렌더러 `sqrt(x)` , 미러 `t * t`
- `log` 스트레치 — 렌더러 `log(1 + 99x) / log(1 + 99)` , 미러 `(100^t - 1) / 99`

이 중 어느 것도 내보내지지 않습니다: 곡선은 패키지의 `pushAdjustments` / 셰이더 모듈에 있고, 강도 상수(99)는 하드코딩된 `prop`입니다. 업스트림이 파이프라인 순서를 바꾸거나, 곡선을 바꾸거나, `log` 강도를 다시 조정해도 빌드는 실패하지 않습니다. 불투명도 임계값이 사용자가 입력한 값에서 어긋날 뿐이며, 그것도 기본값이 아닌 스트레치나 감마에서만 그렇습니다. 범프 시 그 패키지의 `pushAdjustments` 에서 정방향 곡선과 그 순서를 다시 읽고(히스토그

램 눈금에 쓰이는 자체 `inverseStretch` 헬퍼는 위 두 스트레치 역함수의 두 번째 사본), `tests/raster-symbolology.test.ts` 를 실행하세요. `tests/upstream-contracts.test.ts` 는 게시된 청크에서 `pushAdjustments` 와 세 세이더 곡선을 읽어, 순서, 곡선, 강도가 바뀌면 실패합니다.

maplibre-gl-raster — 컴파일러가 검사

`GeoLibreCogRenderEngine` (`packages/plugins/src/types.ts`)은 그 패키지가 내보내는 `RenderEngine` 유니온(`maplibre-gl-raster` | `cog-tiler-wasm` | `titiler`)을 미러링합니다. 임포트가 아니라 손으로 작성된 이유는 `types.ts` 가 공개 플러그인 API 표면이고, 거기서 임포트하면 그 패키지의 타입이 모든 외부 플러그인의 필수 의존성이 되기 때문입니다. 위의 미러들과 달리 이것은 **컴파일러가 검사**합니다:

`packages/plugins/src/plugins/maplibre-raster.ts` 의 `CogRenderEngineMirrorIsExact` 가 실제 임포트된 타입에 대해 양방향 할당 가능성을 단언하므로, 엔진 식별자가 바뀌거나 빠지면 `npm run typecheck` 가 실패합니다. 범프 시 빌드를 돌리는 것 외에 추가로 할 일은 없습니다.

maplibre-gl-raster — 시작 경로에서 떼어 두기 위해 복사한 선택기 데이터

`apps/geolibre-desktop/src/lib/raster-picker-mirror.ts` 는 패키지의 `COLORMAP_OPTIONS`, `NORMALIZED_DIFFERENCE_INDICES`, `CUSTOM_NORMALIZED_DIFFERENCE`, `indexById`, `guessBandForRole` 을 손으로 유지하는 사본입니다. Style 패널의 컬러맵과 분광 지수 선택기는 렌더링되자마자 이들이 필요하고, 패키지는 하나의 공유 청크로 배포되므로, 상수 하나만 임포트해도 약 0.35 MB 라이브러리 전체가 시작 경로에 올라갑니다. GeoLibre가 패키지에서 가져오는 나머지는 모두 비동기이며 대신 동적으로 임포트합니다. 새 값 임포트도 그렇게 유지하세요. `tests/raster-picker-mirror.test.ts` 는 모든 값과 두 헬퍼를 패키지 내보내기와 비교하므로, 업스트림에서 컬러맵이 추가되거나 이름이 바뀌면 `npm run test:frontend` 가 실패합니다. 그럴 때는 패키지로부터 사본을 다시 생성하세요.

tauri-plugin-persisted-scope — 비공개 디스크 형식

`PersistedScopeState` (`apps/geolibre-desktop/src-tauri/src/lib.rs`)는 플러그인의 비공개 `bincode Scope` 구조체를 미러링하여, 플러그인이 시작 시 동기적으로 재생하기 전에 GeoLibre가 레거시 사진별 권한을 제거할 수 있게 합니다. `tauri-plugin-persisted-scope` 범프 시 업스트림 구조체의 필드 순서와 타입을 이 미러와 비교하고 Rust 스코프 정리 테스트를 실행하세요. `Bincode`는 필드를 위치로 인코딩하므로 업스트림 레이아웃 변경은 컴파일러가 검사하지 않습니다. 이 미러는 Rust 쪽이므로 `tests/upstream-contracts.test.ts` 가 다루지 않습니다.

`bincode` 의존성 자체는 **1.x** 계열에 고정되어 있고 `Dependabot`은 메이저 범프를 건너뛰도록 설정되어 있습니다 (`.github/dependabot.yml`): 플러그인이 `bincode 1`로 파일을 쓰므로 GeoLibre는 같은 와이어 형식으로 디코딩하고 다시 인코딩해야 합니다. `tauri-plugin-persisted-scope` 가 옮겨갈 때만 옮기세요. (`bincode 3.0.0`은 추가로 의도적으로 빌드할 수 없는 릴리스입니다. 소스 전체가 `compile_error!("https://xkcd.com/2347/")` 입니다.)

@tauri-apps/plugin-http — API가 아닌 두 가지 업스트림 동작

`createNativeSidecarFetch ( apps/geolibre-desktop/src/lib/sidecar-fetch.ts )`는 Windows 사이드카 트래픽을 플러그인의 네이티브 `fetch`로 라우팅하고, 문서화된 계약이 아니라 `request`의 구현에서 효과가 나오는 두 옵션으로 이를 강화합니다. 옵션 `_이름_`은 컴파일러가 검사합니다. `NativeFetchInit`는 `typeof import("@tauri-apps/plugin-http").fetch`에서 파생되므로 옵션 이름이 바뀌거나 빠지면 `npm run typecheck`가 실패합니다. 그러나 의미는 검사되지 않으며, 둘 다 조용히 실패합니다:

- `maxRedirects: 0`은 `request::redirect::Policy::none()`에 대응합니다 (`tauri-plugin-http/src/commands.rs`). 이것이 없으면 네이티브 클라이언트가 리다이렉트를 따라가고, `request`는 호스트 간에 `Authorization / Cookie`만 제거하므로, 실행별 `X-GeoLibre-Token`이 3xx가 가리키는 어디로든 재전송됩니다.
- `proxy: { all: { url, noProxy: "*" } }`는 사이드카가 루프백에 직접 도달하는 방법입니다. 플러그인에는 "프록시 비활성화" 스위치가 없지만, 어떤 `ClientBuilder::proxy` 호출이든 `auto_sys_proxy = false`를 설정하고 (`request/src/async_impl/client.rs`), `NoProxy::from_string("*")`는 모든 호스트에 일치하므로 (`hyper-util/src/client/proxy/matcher.rs`), 제공된 프록시도 결코 가로채지 않습니다. 이것이 중요한 이유는 `request`의 `system-proxy` 기능이 해결된 그래프에 실제로 들어 있고 (`cargo tree -e features -i request`로 확인; 플러그인의 기본 `macos-system-configuration` 기능이 끌림), `hyper-util`의 Windows 리더는 레지스트리 `ProxyOverride` 목록을 그대로 복사하기 때문입니다. Windows가 "로컬 주소에 프록시 서버 사용 안 함"에 대해 쓰는 `<local>` 토큰을 결코 확장하지 않습니다. 이를 빼면 회사 프록시를 쓰는 Windows 컴퓨터가 사이드카 요청 본문과 토큰을 프록시로 보냅니다.

`tests/sidecar-fetch.test.ts`는 어댑터가 전달하는 옵션을 고정하여 여기서의 부주의한 편집은 잡지만 Rust 쪽은 실행할 수 없습니다. 플러그인 범프 시 위의 소스에 대해 두 동작을 다시 확인하세요. 실패 양상은 토큰 유출과 프록시 사용자에게만 도달 불가능한 사이드카인데, 어느 쪽도 CI나 프록시 없는 개발 컴퓨터에서는 드러나지 않습니다.

cesium / @cesium/engine / @cesium/widgets — URL로 가져오는 런타임 자산

3D 지구본 창은 코드를 `@cesium/engine`에서 로드하지만 런타임 자산은 `cesium` 래퍼에서 로드합니다. 둘 다 의존성이며 함께 움직여야 합니다: `cesium@1.x`는 대응하는 `@cesium/engine`을 고정하고, 래퍼에서 스테이징되는 미리 빌드된 `Workers/Assets`는 이를 실행하는 엔진과 일치해야 합니다. `@cesium/widgets`는 같은 시계 위의 세 번째 의존성으로 — 자체적으로 `@cesium/engine`에 의존 — 지구본의 홈, 씬 모드, 전체 화면 버튼을 제공합니다 (`packages/map/src/cesium-widget-controls.ts`).

코드는 `cesium` 배럴을 거치지 않고 엔진과 위젯을 직접 임포트합니다. 배럴은 둘 다 다시 내보내어 트리 셰이킹을 무력화하기 때문입니다. 창이 기본 `CesiumWidget`만 만들어도 베이스 레이어 선택기, 지오코더, 정보 상자, `Knockout`이 모두 자연 청크에 들어갔습니다. `import("cesium")`으로 되돌리면 빌드 오류도 테스트 실패도 없이 약 340 KB가 다시 추가됩니다.

범프 후 다섯 가지 모두 확인하세요. 어느 것도 빌드를 실패시키지 않습니다:

- **자산 복사.** `vite-plugins/copy-cesium-assets.ts` 는 설치된 버전을 키로 `cesium/Build/Cesium` 에서 `Assets` , `ThirdParty` , `Widgets` , `Workers` 를 `public/cesium/` 으로 복사합니다. 사본은 gitignore되므로 오래된 사본은 자동으로 갱신됩니다. 업스트림에서 디렉터리 이름이 바뀌거나 제거되면 크게 실패합니다. `cpSync` 가 `buildStart` 에서 `ENOENT` 를 throw하므로 개발 서버와 빌드 둘 다 멈춥니다. 조용한 경우는 그 반대입니다: 업스트림에서 런타임 디렉터리가 추가되면 단지 `RUNTIME_DIRS` 에 없을 뿐이라 복사되지 않고, 지구본이 그것을 찾을 때 404로만 드러납니다.
- **CESIUM_BASE_URL .** `CesiumCanvas.tsx` 는 이를 하드코딩된 `/cesium` 이 아니라 앱의 `BASE_URL` 에서 파생하므로, 하위 경로 배포(`/demo/` 빌드)에서도 여전히 해석됩니다. Cesium은 임포트 시점에 이를 읽습니다. `Workers`가 404이면 렌더 루프가 오류 경계 없이 죽습니다.
- **스타일시트.** `CesiumCanvas.tsx` 는 32 KB짜리 `Widgets/widgets.css` 대신 네 경로를 손으로 링크합니다 (`CESIUM_CSS_PATHS`): 캔버스 크기, 크레딧 컨테이너, 오류 패널을 위한 `Widgets/CesiumWidget/CesiumWidget.css` ; 두 툴바 위젯이 기반으로 삼는 `.cesium-button` 크롬을 위한 `Widgets/shared.css` ; 그리고 그 두 위젯을 위한 `Widgets/SceneModePicker/SceneModePicker.css` 와 `Widgets/FullscreenButton/FullscreenButton.css` . 범프 후 하나하나 확인하세요. 업스트림이 옮긴 경로로의 `<link>` 는 조용히 404가 되고, 실패는 어떤 단언도 잡지 못하는 방식으로 외관상에만 드러납니다: 지구본은 여전히 마운트되지만 캔버스가 창을 채우지 않거나, 툴바 버튼이 크기를 잃고 0x0으로 렌더링됩니다. (홈 버튼은 자체 스타일시트가 없습니다. `shared.css` 만 있으면 됩니다.)
- **위젯 클래스와 뷰 모델.** `cesium-widget-controls.ts` 는 `HomeButton` , `SceneModePicker` , `FullscreenButton` 을 직접 만들고 `viewModel.tooltip` / `tooltip2D` / `tooltip3D` / `tooltipColumbusView` 를 통해 툴팁을 씁니다. 그런 다음 `index.css` 가 Cesium 자체의 클래스 이름 (`.cesium-button` , `.cesium-toolbar-button` , `.cesium-sceneModePicker-wrapper`)을 통해 이들을 다시 스타일링합니다. 옴저버블 이름이 바뀌면 영어 기본값이 그대로 남고, 클래스 이름이 바뀌면 GeoLibre 툴바에 Cesium의 짙은 파란색 크롬이 남습니다. 어느 쪽도 빌드를 실패시키지 않습니다. 툴팁 단언은 컨트롤 마운트 및 정렬 검사와 함께 `e2e/cesium-primary-renderer.spec.ts` 에 있습니다. 전체 화면 버튼이 취약한 쪽입니다: 그 툴팁은 읽기 전용 `computed`이므로, 번역된 문자열은 `fullscreenchange` 리스너에서 요소에 쓰이며 DOM 리스너가 등록 순서로 발생하기 때문에만 살아남습니다. 범프가 위젯이 자체 `title`을 갱신하는 방식을 바꾸면(마이크로태스크로 배치되거나 다른 대상에 바인딩되면) 라벨이 첫 토클 이후에 영어로 되 돌아갑니다. 그래서 스펙은 마운트 시 한 번 확인하는 대신 전체 화면을 토글하고 `title`을 다시 읽습니다.

썸 모드 수명: 2D/3D/Columbus 선택기는 현재 `CesiumWidget` 만 제어합니다. MapLibre로 전환했다가 돌아오거나 프로젝트를 다시 여는 것을 포함해 위젯을 다시 만들면 3D로 시작합니다. 프로젝트는 렌더링 엔진과 카메라를 유지하지만 Cesium 썸 모드는 직렬화하지 않습니다. 썸 모드 유지는 툴바 통합의 범위 밖입니다. 추가하려면 명시적인 프로젝트/스토어 필드와 각 Cesium 창에 대한 복원이 필요합니다.

- **PWA 글롭.** `vite.config.ts` 의 `**/cesium-*` / `**/Cesium-*` 는 청크를 앱 셀 프리캐시에서 제외하고 대신 `CacheFirst`로 캐시합니다. 그 패턴을 벗어나게 이름이 바뀐 청크는 프리캐시되어 첫 로드 수 메가바이트를 추가합니다.

`e2e/cesium-globe.spec.ts` 는 실제 엔진을 키 없이 마운트하고 지구본을 드래그하므로, 깨진 `CESIUM_BASE_URL` 이나 죽은 청크를 잡아냅니다. `e2e/cesium-primary-renderer.spec.ts` 는 툴바를 추가합니다: 세 버튼이 마운트되고, 번역된 툴팁을 가지며, 오른쪽 가장자리를 공유하는지 단언하여, 이름이 바뀐 뷰 모델 옵저버블과 스타일시트 누락의 0x0 버튼 경우를 잡아냅니다. `tests/upstream-contracts.test.ts` 는 정적 절반을 추가합니다: `Build/Cesium` 의 디렉터리는 `RUNTIME_DIRS` 와 같아야 하고, 모든 `CESIUM_CSS_PATHS` 파일이 존재하며 `index.css` 가 다시 스타일링하는 각 `.cesium-*` 클래스를 정의해야 하고, 뷰 모델은 여전히 툴팁 옵저버블을 가져야 합니다. 이 중 어느 것도 번들 크기 회귀는 잡지 못합니다. 그것은 빌드 출력에서 확인하세요.

ArcGIS Maps SDK for JavaScript — Esri CDN에서 로드

ArcGIS 렌더러(`docs/arcgis-renderer.md`)는 **npm 의존성이 없습니다**. `packages/map/src/arcgis-sdk.ts` 는 런타임에 `https://js.arcgis.com/<ARCGIS_SDK_VERSION>/@arcgis/core/...` 에서 SDK의 ES 모듈을 임포트하고 사용하는 표면을 손으로 타입 지정하므로, 여기서는 컴파일러가 Esri의 선언에 대해 아무것도 검사하지 않습니다. 따라서 `ARCgis_SDK_VERSION` 범프는 Dependabot 이벤트가 아니라 수동 확인입니다:

- **모듈 경로와 기본 내보내기.** `arcgis-sdk.ts` 의 `SDK_MODULES` 는 엔진이 로드하는 모든 모듈을 나열합니다. `tests/arcgis-renderer.test.ts` 는 가짜에 대한 조립만 검사합니다. 실제 CDN을 탐색하고(각 URL에 `curl -sI` 가 200을 반환) 브라우저에서 창을 마운트하세요. 옮겨진 모듈은 로드 전체를 거부하고 창은 오류 배너를 표시합니다. `packages/plugins/src/plugins/arcgis-deck/overlay.ts` 의 deck 어댑터 지연 임포트도 확인하세요: `layers/Layer`, `views/2d/layers/BaseLayerViewGL2D`, `views/3d/webgl/RenderNode`. 버전 범프 후 2D MapView와 로컬 SceneView 모두에서 deck.gl 레이어를 마운트하세요. 그 임포트들은 엔진의 모듈 레지스트리 밖에 있습니다.
- **레거시 위젯.** `widgets/Zoom`, `Compass`, `ScaleBar`, `Fullscreen`, `Locate` 가 내장 컨트롤을 뒷받침합니다. Esri는 4.32에서 웹 컴포넌트를 위해 이들을 사용 중단했고, 5.x에서도 각각 콘솔 경고와 함께 여전히 배포합니다. 이들을 제거하는 릴리스는 `setBuiltInControlVisible` 을 깨뜨립니다. 대체물은 `@arcgis/map-components` CDN 빌드입니다. 어트리뷰션은 이미 5.x 경로를 사용합니다: `view.attributionVisible` 이 켜져 있는 동안 뷰가 그리므로, 사용 중단된 `Attribution` 위젯은 로드되지 않습니다.
- **Compass 위젯의** `viewModel.reset`. 엔진은 MapLibre의 나침반처럼 클릭이 방위뿐 아니라 피치도 수평으로 맞추도록 이를 덮어씹습니다. 그 멤버는 타입 지정된 표면의 일부가 아니며, `viewModel` 이 없으면 조용히 건너뛰므로, 이를 재구성하는 릴리스는 나침반을 조용히 방위 전용으로 되돌립니다. 범프 후 씬을 기울이고 나침반을 클릭하세요: 방위와 기울기 모두 0으로 돌아와야 합니다.
- **ESM CDN 안내.** SDK는 로드 시 "Only use ES modules from ArcGIS CDN for testing"을 로그에 남깁니다. Esri가 문서화한 프로젝트 경로로는 npm 빌드인데, 이 렌더러는 의도적으로 이를 피합니다(이슈 #2421의 크기 논거 참조). ESM CDN이 언젠가 철회되면 AMD CDN(`<script src="https://js.arcgis.com/<version>/">`)이 지원되는 대안입니다.
- **히트 테스트 속성.** `GeoJSONLayer` 그래픽은 `outFields: ["*"]` 가 설정되지 않으면 렌더러가 읽는 필드만 가져옵니다. 식별은 컴파일러의 `gl__id` 필드가 도착하는 데 의존합니다. `e2e/arcgis-renderer.spec.ts` (옵트인, `ARCgis_API_KEY`)가 그 검사입니다.

- **CSP와 캐싱.** <https://js.arcgis.com/> 은 `tauri.conf.json` 과 `docker/nginx.conf` 의 `script-src` 와 `font-src` 에 허용 목록으로 들어 있고, `vite.config.ts` 의 `geolibre-arcgis-sdk` 서비스 워커 규칙으로 캐시됩니다. 버전이 모든 URL에 들어 있으므로 범프는 새 캐시 항목을 만듭니다.

데스크톱 CSP `script-src` 허용 목록

데스크톱 CSP(`apps/geolibre-desktop/src-tauri/tauri.conf.json`)는 <https://cdn.jsdelivr.net/npm/> 전체를 허용하지 않습니다. 앱이 jsDelivr에서 실행하는 패키지마다 버전 고정 경로 하나를 나열합니다. 이는 CDN의 다른 패키지나 버전의 스크립트를 차단하지만, 허용된 경로에서 무엇이 제공되는지는 검증하지 않으므로 고정된 경로의 손상된 릴리스는 여전히 실행되며, 요청이 리다이렉트되면 CSP는 경로 검사를 멈춥니다. 각 버전은 다른 무언가가 소유하므로, 범프는 CSP가 옛 URL을 유지하는 동안 앱이 요청하는 URL을 옮길 수 있습니다. 그러면 패키징된 앱은 `tauri dev` 가 결코 보여 주지 않는 CSP 차단에 부딪힙니다. `tauri dev` 는 CSP를 적용하지 않기 때문입니다. `tests/tauri-csp.test.ts` 는 모든 경로를 소유자로부터 다시 파생하여 불일치 시 실패합니다:

script-src 경로	소유자(이것을 범프한 뒤 CSP 갱신)	로드하는 곳
<code>pyodide/v<ver>/full/</code>	<code>pyodide-config.ts</code> 의 <code>PYODIDE_VERSION</code>	Pyodide의 <code>pyodide.asm.js import()</code> (Python Console)
<code>npm/@electric-sql/pglite@<ver>/ , npm/@electric-sql/pglite-postgis@<ver>/</code>	잠금 파일	<code>pglite-loader.cdn.ts</code> (SQL Workspace → PostGIS)
<code>npm/onnxruntime-web@<ver>/dist/</code>	<code>packages/processing/src/ort.ts</code> 의 <code>ORT_VERSION</code>	onnxruntime의 wasm 글루 <code>import()</code> (객체 탐지, SAM)
<code>npm/@duckdb/duckdb-wasm@<ver>/dist/</code>	잠금 파일(<code>apps/geolibre-desktop</code>)	<code>maplibre-gl-components</code> 의 DuckDB 변환기: <code>getJsDelivrBundles()</code> 의 워커 URL에 <code>importScripts()</code> 를 호출하는 blob 워커
<code>npm/@duckdb/duckdb-wasm@1.31.0/ , npm/sql.js@1.13.0/dist/ , npm/geojson-vt@4.0.2/ , npm/vt-pbf@3.1.3/</code>	<code>maplibre-gl-vector</code> 에 하드코딩된 URL	Add Data → Vector Layer(DuckDB, GeoPackage 패치, MVT 폴백)
<code>npm/apache-arrow@... , npm/tslib@... , npm/flatbuffers@... , npm/pbf@... , npm/ieee754@... , npm/@mapbox/point-geometry@... , npm/@mapbox/vector-tile@...</code>	위 <code>maplibre-gl-vector</code> URL들의 <code>jsDelivr /+esm</code> 빌드	그 번들 안의 <code>import</code> 문

Blob 워커는 페이지의 CSP를 상속하므로, 그 안의 `importScripts()` 는 `script-src` 에 대해 검사됩니다. 앱 자체 파일에서 시작된 워커(`pyodide-worker.js` , DuckDB 및 Whitebox 워커)는 CSP를 전혀 갖지 않습니다: Tauri 는 HTML 응답에만 정책 헤더를 붙입니다. Wasm, 훔, PGLite 데이터, CereusDB/gdal3.js 바이너리는 `fetch` 되므로 `connect-src` 가 이들을 다룹니다.

`maplibre-gl-vector` 가 CDN URL을 바꾸면 테스트의 `MAPLIBRE_GL_VECTOR_URLS` 단언이 실패합니다. 각 새 버전들의 임포트를 나열하여 전이적 `/+esm` 경로를 다시 파생하세요. 예를 들어 `curl -s https://cdn.jsdelivr.net/npm/vt-pbf@3.1.3/+esm | grep -o '/npm/[^"]*'` 를 실행하고 각 결과로 재귀한 다음, `ESM_TRANSITIVE_PATHS` 와 `CSP`를 함께 갱신하세요.

웹 빌드의 `CSP`(`docker/nginx.conf` 의 앱 `location /`)는 정확히 같은 고정 경로를 나열하며, 두 목록이 다르면 테스트가 실패하므로 두 파일을 같은 변경에서 갱신하세요. 거기서도 앱 자체 워커 파일은 `CSP`를 갖지 않습니다: `nginx`는 `.js` 파일을 정적 자산 `location`에서 제공하며, 그곳은 `CSP`를 보내지 않습니다. `JupyterLite` `location`은 `jsDelivr`의 `pyodide/` 만 고정 없이 허용합니다: 사이트의 `Pyodide`는 이미지를 빌드할 때 `pip`이 해결하는 `jupyterlite-pyodide-kernel` 릴리스에서 옵니다.

자체 호스팅 `VITE_PYODIDE_INDEX_URL` 미러는 어느 빌드에서도 `CSP` 변경이 필요 없습니다: `pyodide-console.ts` 는 미러의 진입 스크립트를 가져와 `blob: URL`에서 실행하고, 벡터 도구 워커는 `CSP` 아래에 있지 않습니다. 다른 호스트에서 스크립트를 로드하는 재빌드 앱은 두 파일 모두의 `script-src` 에 그 호스트의 경로를 추가해야 합니다.

'unsafe-eval' 은 유지됩니다. `maplibre-gl-vector` 는 모듈 범위에서 `new Function("url", "return import(url)")` 로 CDN 로더를 만들고, 그 모듈은 시작 그래프에 있습니다. `'unsafe-eval'` 이 없으면 패키징된 앱은 부팅 중 `EvalError` 를 throw하고 빈 창을 표시합니다(Linux WebKitGTK 빌드에서 확인). 다른 기능들도 런타임에 문자열을 컴파일합니다: 속성 필드 계산기(`attribute-expression.ts`), 래스터 계산기(`raster-client.ts`), AI Assistant의 JavaScript 도구, Earth Engine 코드 편집기, Emscripten `embind` 클루(`maplibre-gl-lidar`, `maplibre-gl-splat`), 그리고 `ndarray`, `Ajv`, `Mapillary`의 필터 컴파일러. `'wasm-unsafe-eval'` 만으로는 이 중 어느 것에도 충분하지 않습니다. `'unsafe-eval'` 을 빼는 일은 업스트림 `maplibre-gl-vector` 변경에서 시작하며, 그 다음 이 소비자들 각각에 대체물이 필요합니다.

`connect-src` 는 `https: 와 http: 를 유지합니다`. 사용자는 임의의 호스트에서 타일, COG, 벡터, 서비스 URL(XYZ, WMS/WMTS, ArcGIS, STAC, PMTiles)을 추가하며, 그중 일부는 LAN의 일반 HTTP 서버이므로 고정된 호스트 목록으로는 이들을 다룰 수 없습니다. `img-src` 도 같은 이유로 `https: 를 유지합니다`.

httpx (backend/geolibre_server_api/pyproject.toml) — 비공개 전송 풀

프로젝트 서버가 내부 주소로 향하는 ID 공급자 요청으로부터 보호하는 기능(`geolibre_server_api/egress.py`)은 `httpcore` 네트워크 백엔드가 필요한데, `httpx.HTTPTransport` 는 이를 받지 않습니다. 그래서 `GuardedTransport` 는 비공개 `HTTPTransport._pool` 을 `GuardedBackend` 위에 만든 `httpcore.ConnectionPool` 로 대체합니다. `_pool` 의 이름이 바뀌면 전송 구축이 시작 시 `RuntimeError` 를 발생시킵니다. `httpx`가 `_pool` 은 유지하되 요청을 그것을 통해 보내지 않게 되면, 가드는 오류 없이 우회됩니다. 범프 후 `HTTPTransport.handle_request` 가 여전히 `self._pool` 을 통해 요청을 보내는지 확인하고, `python -m pytest backend/geolibre_server_api/tests/test_egress.py` 를 실행하세요 (`test_identity_provider_client_refuses_loopback` 은 전송을 통해 실제 연결을 만듭니다).

공유 의존성 범위

`apps/geolibre-desktop` 과 `packages/plugins` 는 같은 업스트림 패키지를 다수 선언합니다. 범프가 둘 중 한 곳에서만 범위를 옮기면 npm은 더 이상 사본 하나를 호이스팅할 수 없어 불일치하는 워크스페이스 아래에 두 번째 사본을 중첩합니다: 앱은 두 버전을 번들하고, 위의 계약 테스트는 한 사본을 검사하는 동안 다른 사본이 실행됩니다. Dependabot은 매니페스트마다 PR 하나를 열므로 실수로 병합하기 쉽습니다. `npm run check:shared-deps` (`scripts/check-shared-deps.mjs`, CI의 "Lint and type check" 작업에서 실행)는 두 매니페스트 모두에, 어느 의존성 섹션에든 선언된 패키지가 서로 다른 범위를 가지면 실패합니다. 둘에 같은 범위를 주고 같은 PR에서 `package-lock.json` 을 갱신하여 고치세요.

하나의 공유 three

앱과 three 기반 플러그인들(`maplibre-gl-3d-tiles`, `maplibre-gl-splat`, `maplibre-gl-components`, `deck.gl` 메시 레이어)은 three 사본 하나를 공유합니다. 두 가지가 이를 유지합니다:

- 업스트림 opengeos 패키지들은 three를 ES `dist` 에서 제외하고 피어로 선언합니다(`maplibre-gl-splat` `>= 0.2.9`, `maplibre-gl-3d-tiles` `>= 0.5.11`, `maplibre-gl-components` `>= 0.31.2`). 더 오래된 릴리스는 자체 three를 다시 인라인합니다.
- 루트 `package.json` 의 `overrides` 는 three 를 앱의 범위로 고정합니다. `@dvt3d/maplibre-three-plugin` 이 `three: ^0.178.0 (0.178.x만)`을 선언하여 그렇지 않으면 두 번째의 더 오래된 사본을 끌어오기 때문입니다. `mapillary-js` 도 오래된 three를 선언하지만, 그 `dist` 는 자체 사본을 번들하고 결코 임포트하지 않으므로 거기서는 오버라이드가 무해합니다. Mapillary 뷰어를 열면 여전히 그 번들된 0.134 사본이 로드되는데, 이것이 마지막으로 남은 "Multiple instances of Three.js" 경고입니다.

three 를 범프할 때는 오버라이드를 앱의 범위와 함께 옮긴 다음 `npm ls three` 를 실행하세요: 버전 하나만 나열되어야 합니다. npm이 오래된 중첩 사본을 유지한다면 [의존성 업데이트와 감사 허용 목록](#)의 잠금 파일 정리를 따르세요. Spark의 `dist` 는 정렬 워커의 소스 문자열 안에 three를 내장합니다. 그 사본은 Web Worker에서 실행되며 결코 두 번째 메인 스레드 인스턴스로 등록되지 않습니다.

블렌드 모드 추가하기

브라우저에서 확인하지 않고 블렌드 모드를 추가하지 마세요. MapLibre의 블렌드 상태는 알파 채널도 포함하며, 블렌드된 레이어를 뷰포트를 채우는 하나의 쿼드로 합성하므로, 소스 알파가 0일 때 "대상을 그대로 둬"로 환원되지 않는 모드는 지도 전체를 다시 칠합니다. 그것이 `darken` (`MIN` 방정식이 베이스맵 전체를 투명한 검정으로 지움)과 `subtract` (역 뺄셈이 캔버스를 `dstA - srcA` 로 남겨 레이어를 통해 페이지가 비침)을 탈락시킨 이유입니다. 출시된 목록은 `@geolibre/core` 의 `BLEND_MODES` 이며, 단위 테스트의 블렌드 시뮬레이터와 e2e 스펙 모두 그 제외를 고정합니다.

스타일 스펙에서 `*-layer-opacity` 를 가진 것은 `fill` 과 `line` 뿐이므로, 이 둘만 **레이어 전체로** 블렌드됩니다. `circle` 과 `fill-extrusion` 은 심볼별로 블렌드되며 화면에서 심볼이 겹치는 곳이 눈에 띄게 두 번 어두워집니다 (Multiply에서 측정: 겹침 부분 `rgb(23, 77, 220)` 대 단일 심볼 `rgb(76, 136, 222)`). 이는 업스트림의 제한으

로 **레이어 관리**에 문서화되어 있습니다. "has a layer-level composite for fill and line only" 테스트는 범프가 누락된 속성 중 하나를 추가하면 실패하며, 그때는 `COMPOSITE_LAYER_TYPES` 와 `style-mapper` 를 함께 확장하고 주의 사항을 제거하세요.

Style 패널 컨트롤(`StylePanel.tsx` 의 `blendModeControl` , 각 말단 분기에서 렌더링됨)은 `!pluginOwnsPaint && !controlRendersLayer` 로 게이트됩니다: 블렌딩은 **GeoLibre 자체가 칠하는** 레이어에만 도달하므로, 컨트롤이 렌더링하거나 칠하는 것(3D Tiles, Gaussian splat, LiDAR, COG 래스터 컨트롤, 그리고 `customLayerType` 과 `controlOwnsPaint` 를 모두 설정하는 Add Vector Layer)은 제외됩니다. `layer-sync`는 그것들에 결코 `fillPaint / linePaint` 를 적용하지 않으므로, 합성을 선택하게 하는 `*-layer-opacity` 가 결코 도달하지 않고 거기의 Blend 메뉴는 조용히 아무것도 하지 않을 것입니다. `docs/user-guide/layers.md` 와 `tests/layer-blend-modes.test.ts` ("the layer kinds the Blend control is offered for")를 그 게이트와 일치하게 유지하세요. 테스트의 모의 객체는 실제 컨트롤이 메타데이터를 만드는 방식으로 만드세요. 그렇지 않으면 결코 발생하지 않는 형태에 대해 통과합니다.

커버리지 하한

`:coverage` 테스트 변형은 같은 스위트를 실행하고 커버리지 요약을 출력합니다. CI는 이를 실행하므로 모든 빌드가 커버리지를 보고합니다. 이들은 **하한으로 게이트됩니다**: `test:frontend:coverage` 는 라인 78% / 분기 78% / 함수 63% 미만에서 실패하고, `test:backend:coverage` 는 55% 미만에서 실패합니다(`--cov-fail-under`). 하한은 현재 수치보다 몇 포인트 아래에 **레딧**으로 자리합니다. 회귀는 CI를 실패시키고, 커버리지가 하한을 넉넉히 넘어서면 하한을 올려 이득을 고정하세요.

프론트엔드 보고서는 테스트가 실제로 임포트하는 파일만 세므로, 테스트가 없는 모듈은 0%가 아니라 아예 나타나지 않습니다. 바로 이 부분이 문제가 됩니다: 테스트되지 않은 큰 모듈에 대한 첫 테스트를 작성하면 커버리지 **회귀**로 읽힙니다. 그 모듈과 그것이 임포트하는 모든 것이 한꺼번에 분모에 들어가기 때문입니다. GeoLibre#1784는 `usePlugins.ts` 를 임포트하는 테스트를 추가하여 내장 플러그인 레지스트리 전체, 39개 파일을 끌어들였고, 함수 커버리지를 72.90% → 60.36%로 떨어뜨려 `main` 을 빨강게 만들었습니다. 해결책은 하한을 낮추는 것이 아니라 리프 모듈에 대해 테스트하는 것입니다(GeoLibre#1888은 `lib/plugin-layer-queries.ts` 를 추출했습니다. `@geolibre/plugins` 의 `geo-editor-geometry.ts` 도 같은 패턴입니다). 커버리지 하락이 코드가 나빠졌다는 뜻이라고 가정하기 전에 새 테스트가 전이적으로 무엇을 임포트하는지 확인하세요. 아래의 **미테스트 모듈 레딧**은 어떤 테스트도 로드하지 않는 큰 모듈들이 시야 밖에 머무르지 않게 합니다.

`test:frontend:coverage` 는 `node --test` 를 직접 호출하지 않고 `scripts/coverage-check.mjs` 를 통해 실행됩니다. Node는 여전히 세 하한 모두를 강제합니다. 래퍼는 모든 테스트가 통과하면서 **라인** 커버리지만 부족할 때 한 번만 다시 측정합니다. 라인 커버리지는 CI에서 비결정적이며(GeoLibre#1889: 바이트 단위로 동일한 소스에 대한 두 실행이 81.82%와 76.47%를 보고했고, 444개 파일 중 114개가 라인에서 다르고 분기나 함수에서는 하나도 다르지 않았음), Node 22나 26 어느 쪽에서도 로컬에서 재현되지 않습니다. 분기 및 함수 부족과 모든 테스트 실패는 재시도 없이 즉시 실패하므로, 실제 회귀는 여전히 빠르게 실패합니다. `classify()` 는 내보내져 `tests/coverage-check.test.ts` 가 다룹니다. 재시도 정책은 하한을 느슨하게 하는 대신 거기서 바꾸세요. 재시도가 정기적으로 발생하기 시작하면 완화책을 넓히는 대신 측정을 고치세요.

백엔드 커버리지 실행(그리고 `:coverage` 변형을 호출하는 `npm run ci`)에는 백엔드 `dev` 추가 기능의 `pytest-cov`가 필요합니다. 전체 백엔드 스위트를 실행하려면 `test` 추가 기능을 설치하세요. 선택적 엔진(`geopandas/rasterio/sedona/httpx`)이 없으면 벡터/래스터/SQL/ML 테스트가 스스로 건너뛰어 CI는 녹색이지만 속 빈 상태가 됩니다: `pip install -e "backend/geolibre_server[test]"`.

미테스트 모듈 래치

보고서는 임포트된 파일만 세기 때문에, 어떤 테스트도 로드하지 않는 큰 모듈은 하단에 보이지 않습니다. `npm run check:untested-modules` (`scripts/check-untested-modules.mjs`)가 이를 보이게 합니다. 이는 `test:frontend:coverage`가 `coverage/frontend.lcov`에 쓰는 lcov 보고서(스위트가 직접 또는 전이적으로 로드한 모든 파일을 나열)를 읽어 `apps/*/src`와 `packages/*/src` 아래의 모든 소스 파일과 비교합니다. 선언 파일, `index.*` 배럴, 로케일 카탈로그, 생성 파일(`AUTO-GENERATED`, `@generated`, 또는 `DO NOT EDIT` 헤더, 그리고 Whitebox 메뉴 카탈로그)은 세지 않습니다. `ci:frontend`에서 커버리지 단계 바로 뒤에 실행되므로 CI의 "Build and test" 작업과 `npm run ci`에서 실행됩니다. `ci:web`은 커버리지를 측정하지 않으므로 건너뛴다. 로컬에서는 `npm run test:frontend:coverage` 후에 실행하세요.

500줄을 넘는 테스트되지 않은 모든 소스 파일은 `scripts/untested-modules-baseline.json`에 줄 수와 함께 나열되어야 하며, 다음의 경우 검사가 실패합니다:

- 어떤 테스트도 로드하지 않는 500줄 초과 새 파일이 나타남. 그것을 임포트하는 테스트나, 거기서 추출한 리프 모듈을 추가하세요(리프를 테스트하는 것이 커버리지 하한을 안정적으로 유지하는 이유는 위 참조). 이름이 바뀐 기존 파일은 새 파일로 나타납니다: 항목을 새 경로로 옮기세요.
- 기존 파일이 기록된 줄 수보다 50줄 넘게 커졌는데 여전히 로드하는 테스트가 없음. 테스트를 추가하거나 새 코드를 테스트된 모듈에 넣으세요.

기준선은 줄어들기만 합니다. 항목이 테스트를 얻거나, 500줄 이하로 줄거나, 삭제되면 검사가 힌트를 출력합니다.

`npm run check:untested-modules -- --prune`은 그 항목들을 제거하고 줄어든 항목의 수치를 낮추며, 결코 추가하지 않습니다. 이득을 유지하도록 같은 PR에서 하세요. `--write-baseline`은 파일을 처음부터 다시 쓰며 부트스트랩 용도로만 존재합니다. 새 미테스트 모듈을 흡수하는 데 사용하지 마세요.

같은 명령은 게이트 없이 `backend/geolibre_server/tests`에 `test_<module>.py` 또는 `test_<module>_*.py`가 없는 백엔드 사이드카 모듈도 출력합니다. 일부는 다른 테스트 파일을 통해 실행되므로(라우트 테스트가 `app/main.py`를 구동), 이 목록은 테스트되지 않은 코드의 수가 아니라 확인하라는 신호로 다루세요.

린트 경고 래치

`npm run lint`는 현재 경고 수로 설정된 `--max-warnings`(루트 `package.json`에 있음)를 전달하므로, 린트 경고는 커버리지 하한처럼 동작합니다: 수는 줄어들기만 할 수 있습니다. 경고는 `react-hooks/exhaustive-deps`, `@typescript-eslint/no-explicit-any`, 타입 인식 `@typescript-eslint/no-floating-promises`(앱, 패키지, 워커 `src/`에 한정, 각 파일에 가장 가까운 `tsconfig.json`에 대해 검사), `local/no-physical-`

tailwind (eslint-rules/no-physical-tailwind.mjs, 국제화의 오른쪽에서 왼쪽 규칙), 그리고 앱과 패키지 .tsx 에 대한 두 eslint-plugin-jsx-a11y 규칙 (control-has-associated-label, no-static-element-interactions)에서 나옵니다.

eslint-plugin-jsx-a11y 6.10은 ESLint 9까지를 피어로 선언합니다. ESLint 10에서 실행되며, 루트 package.json 의 해당 overrides 항목이 피어를 설치된 ESLint로 가리킵니다. 플러그인이 ESLint 10 지원을 게시하면 그 오버라이드를 제거하세요. 미래의 ESLint가 플러그인을 깨뜨리면 npm run lint 가 크게 실패합니다. --legacy-peer-deps 에 손을 뻗는 대신 ESLint를 고정하거나 두 규칙을 비활성화하세요.

- **PR이 경고를 추가함:** 고치세요. 의도적인 발사 후 망각(fire-and-forget)인 떠다니는 프로미스는 호출 앞에 void 를 붙이고 자체 거부를 처리하는지 확인하세요. 물리적으로 유지해야 하는 클래스(예: 지도에 고정된 오버레이)에는 // eslint-disable-next-line local/no-physical-tailwind -- <why> 를 추가하세요. 한도를 올리지 마세요.
- **PR이 경고를 고침:** 같은 PR에서 ESLint가 출력하는 새 총계로 한도를 낮춰 이득을 유지하세요.
- **PR이 새 규칙을 켜:** 한도가 올라가는 유일한 경우입니다. 현재 코드에 대한 새 규칙의 수만큼 정확히 올리고, PR에서 그렇게 말하고, 그 경고들은 나머지처럼 시간을 두고 고치세요.

테스트 타입 검사 래치

프론트엔드 테스트는 node --import tsx --test 로 실행되는데, tsx는 타입을 검사하지 않고 제거하므로, 대신 하는 인터페이스에서 드리프트한 테스트 가짜(이름이 바뀐 속성, 새 필수 멤버, 바뀐 인자)도 여전히 실행되고 여전히 통과합니다. npm run typecheck:tests 는 그 틈을 메웁니다: tests/ 의 세 프로젝트에 대해 tsc --noEmit 을 실행하고 테스트 파일의 오류 수가 --max-errors (루트 package.json 에 있음)를 넘으면 실패합니다. CI의 "Lint and type check" 작업과 npm run ci / ci:web 에서 실행됩니다.

- tests/tsconfig.json 은 모든 테스트를 DOM lib와 Node, React, Vite 타입, 그리고 앱과 패키지가 컴파일에 쓰는 앰비언트 .d.ts 파일에 대해 검사합니다. allowJs 는 테스트가 다루는 scripts/*.mjs 를 추론된 타입으로 임포트할 수 있게 합니다.
- tests/tsconfig.workers.json 은 workers/tiles 나 workers/viewer 를 임포트하는 테스트를 @cloudflare/workers-types 에 대해 검사하고, tests/tsconfig.ai-proxy.json 은 workers/ai-proxy 를 임포트하는 테스트를 그 워커의 생성된 worker-configuration.d.ts 에 대해 검사합니다. Workers 런타임 타입은 DOM 전역을 다시 선언하므로 이 테스트들은 메인 프로그램을 공유할 수 없습니다. 워커 소스를 임포트하는 새 테스트는 메인 프로젝트의 exclude 와 워커 프로젝트 하나의 목록에 들어갑니다. 제외된 테스트가 어느 워커 프로젝트에도 없으면 스크립트가 실패합니다.

tests/ 의 오류만 셉니다. 테스트의 임포트는 제품 소스를 프로그램에 끌어들이고, 그 소스는 자체 워크스페이스의 tsconfig로 검사됩니다. 테스트 설정 아래에서 보고되는 오류(예: Workers 타입 아래의 DOM을 쓰는 @geolibre/core 코드)는 정보용으로 출력되며 세지 않습니다.

한도는 [린트 경고 래치](#)처럼 동작합니다:

- **PR이 테스트에 타입 오류를 추가함:** 고치세요. 가짜는 실제 타입에 맞춰 만드세요: 부분 `LayerStyle` 에는 `DEFAULT_LAYER_STYLE` 을 스프레드하고, 레이어에는 `tests/helpers/layer-fixtures.ts` 를 쓰고, 큰 인터페이스(`MapLibre Map`, `Cesium 뷰어`)의 의도적으로 부분적인 가짜는 매 호출마다가 아니라 작은 타입 지정 헬퍼에서 `unknown` 을 통해 한 번만 캐스트하세요. 한도를 올리지 마세요.
- **PR이 타입 오류를 고침:** 같은 PR에서 스크립트가 출력하는 새 총계로 한도를 낮추세요.

세어지는 모든 오류를 나열하려면 `node scripts/check-test-typecheck.mjs --max-errors 0 --verbose` 를, tsc 자체 출력을 보려면 `npx tsc --noEmit -p tests/tsconfig.json` 을 실행하세요.

부트 번들 예산

`npm run build` 는 앱 진입점이 정적으로 임포트하는 JS(셀이 마운트되기 전에 `index.html` 이 `modulepreload` 하는 것)가 `apps/geolibre-desktop/boot-budget.json` 의 어느 한 예산을 넘거나 `Cesium` 을 포함하면 실패합니다. 검사는 `apps/geolibre-desktop/vite.config.ts` 의 `bootBundleBudgetPlugin` 입니다.

예산	한도	설정 당시 측정값
<code>rawBytes</code>	3 MiB, 축소 후	~2.2 MB
<code>gzipBytes</code>	900 KiB, gzip 레벨 9	~680 kB

Gzip은 브라우저가 다운로드하는 양에 더 가까운 대리 지표이고, raw는 브라우저가 파싱하는 양입니다. 대부분의 회귀는 둘 다 걸리는데, 지연 패키지 전체(`Cesium`, `DuckDB`, `maplibre-gl-*` 플러그인)를 부트 경로로 끌어오는 청크 그룹화 변경에서 오기 때문입니다. 오류는 가장 큰 부트 청크를 나열합니다. 예산을 올리는 대신 `CODE_SPLITTING_GROUPS / manualChunks` 에서 그룹화를 고치세요. 새 즉시 로드 코드가 부트에 속한다는 것을 확인한 뒤에만 올리고, PR에서 그렇게 말하세요.

`npm run bundle:report` (빌드 후)는 `bundle-report/bundle-report.md` 와 `.json` 을 씁니다: `dist/assets` 의 모든 JS/WASM 자산과 raw 및 gzip 크기, 빌드된 `index.html` 에서 읽은 부트 집합을 두 예산에 대조한 것, 그리고 각 최상위 `dist` 항목의 크기입니다. CI의 빌드 작업은 이를 `bundle-report` 아티팩트로 업로드하고 Markdown을 작업 요약에 덧붙이므로, 재빌드 없이 두 실행을 비교할 수 있습니다.

워커는 메인 빌드와 청크를 공유하지 않습니다

Vite는 각 `new Worker(new URL(...))` 를 별도 빌드로 번들하므로, 메인 스레드에 같은 모듈이 있어도 워커가 임포트하는 것은 다시 출력됩니다. 그 때문에 한때 `h5wasm(libhdf5, 약 4.8 MB)`이 로컬 NetCDF 리더용으로 한 번, `netcdf-remote.worker.ts` 안에 한 번, 두 번 배포되었습니다. 이제 메인 빌드는 `h5wasm`을 명시적 청크로 출력하고 워커는 URL로 이를 임포트하며(`apps/geolibre-desktop/vite-plugins/shared-h5wasm.ts`), 정확히 하나의 `hdf5_h1-*.js` 외에 다른 것이 출력되면 빌드가 실패합니다. 큰 라이브러리를 임포트하는 새 워커는 같은 방식을 따르거나, 중복을 알고서 받아들여야 합니다. 번들 보고서는 두 번째 사본을 쉽게 발견하게 해 줍니다.

의존성 업데이트와 감사 허용 목록

의존성은 두 가지 방식으로 감시됩니다: **Dependabot**(`.github/dependabot.yml`)이 npm, pip(백엔드 + python/), cargo, Actions에 대해 그룹화된 주간 업데이트 PR을 열고, CI는 npm run audit:ci (차단형, "Lint and type check" 작업에서)와 해결된 백엔드 환경에 대한 비차단형 pip-audit ("Backend tests" 작업에서)를 실행합니다.

audit:ci 는 `scripts/audit-check.mjs` 로, `npm audit --omit=dev` 를 감싼 얇은 래퍼이며 `ALLOWLIST` 에 나열된 것을 제외한 모든 high/critical 권고에서 여전히 실패합니다. 이 래퍼가 존재하는 이유는 일반 `npm audit` 이 개별 발견 하나를 수용할 수 없어서, 패치할 수 없는 전이적 권고 하나가 업스트림이 수정을 배포할 때까지 모든 PR 을 빨강게 만들기 때문입니다. 관리되지 않는 리프 패키지라면 그 시점은 영영 오지 않을 수도 있습니다. **업그레이드 할 패치 버전이 없고** 취약한 코드가 **GeoLibre 런타임 경로에서 도달 불가능할** 때만 권고를 허용 목록에 넣고, 항목에 두 가지 모두에 대한 이유를 적으세요. 업그레이드 가능한 것은 대신 업그레이드합니다. 오래된 항목은 실패하는 대신 경고를 출력하는데, 권고 데이터베이스는 실시간 서비스이고 일시적 누락이 무관한 PR을 빨강게 만들어서는 안 되기 때문입니다.

수정이 범위 지정 `overrides` 항목(예: `@loaders.gl/compression` → `fflate`)일 때, npm 12는 이를 받아들이고 `npm ls` 는 오버라이드의 범위에 대해 검사합니다. 하지만 잠금 파일은 여전히 패키지 자체가 선언한 의존성 (`fflate: 0.7.4`)을 나열하고 npm은 옛 중첩 사본을 유지하므로, `npm ls` 는 이를 `invalid` 로 보고합니다. 그 패키지의 중첩 `node_modules/.../<pkg>` 항목을 `package-lock.json` (및 `node_modules`)에서 삭제한 다음 `npm install` 을 다시 실행하여 새로 해결되게 하세요. 커밋 전에 `npm ls <pkg>` 로 확인하세요.

오버라이드를 만족하는 사본을 이미 설치하는 다른 패키지가 없으면 npm은 하나를 가져오지 않습니다. 패키지를 트리에서 빼 버리고 `npm audit` 은 취약점 0개를 보고하는데, 이는 수정처럼 보이지만 수정이 아닙니다. 그래서 `texture-compressor` → `image-size@^2.0.4` 오버라이드에는 손으로 쓴 잠금 항목이 필요했습니다: `npm view <pkg>@<version> --json` 에서 가져온 `version` , `resolved` , `integrity` , `license` , `bin` , `engines` (npm 12 는 그 출력을 배열로 감쌉니다). 커밋 전에 새 `npm ci` 를 `npm ls <pkg>` 로 확인하세요. 그 오버라이드는 GHSA-w3rx-r6r6-pgpr와 GHSA-5p2g-fcmc-qvqq를 패치하지만, `image-size 2.x`는 경로가 아니라 버퍼를 받으므로 `texture-compressor` 자체 CLI는 더 이상 이미지 크기를 읽을 수 없습니다. GeoLibre는 그것을 결코 실행하지 않습니다: Node 전용 인코더인 `@loaders.gl/textures` 의 `encodeImageURLToCompressedTextureURL` 만 `npm` 을 통해 이를 생성합니다. `loaders.gl`이 `texture-compressor` 의존을 멈추면 오버라이드를 제거하세요(4.5.x는 이미 피어로 만듭니다).

@geolibre/core 와 @geolibre/map 게시

둘 다 `@geolibre/embed` 와 함께 각 GitHub Release마다 `.github/workflows/publish-packages.yml` 에 의해 npm에 게시됩니다. 체크인된 `main / types / exports` 는 TypeScript 소스를 가리키는데, 모노레포가 그렇게 소비하기 때문입니다: `Vite` , `tsc` , `tsx` 모두 패키지 자체의 `exports` 를 통해 `./src/index.ts` 를 해석하므로 `npm run dev` 와 `node --import tsx --test tests/<name>.test.ts` 에 빌드 단계가 필요 없습니다.

npm 타르볼은 대신 `dist` 를 배포하며, npm은 그 분리를 자체적으로 표현할 수 없습니다: pnpm 및 Yarn과 달리 `publishConfig` 아래에 중첩된 진입 필드를 의도적으로 무시하므로(npm/cli#7586), `dist` 진입점을 거기에만 적은 매니페스트는 `src` 를 결코 받지 못하는 소비자에게 `./src/index.ts` 를 게시합니다. 따라서 게시 진입점은 `publishConfig` 아래에 있고, `scripts/prepare-npm-package.mjs` 가 `npm publish` 직전에 이를 끌어올립니다(그리고 `"*" @geolibre/core` 의존성을 릴리스 버전으로 고정합니다).

그 최상위 필드를 `dist` 로 가리키면 `@geolibre/map` 하위 경로를 임포트하는 모든 프론트엔드 테스트가 `ERR_MODULE_NOT_FOUND` 로 실패합니다. `dist` 는 gitignore되고 스위트 전에 아무것도 그것을 빌드하지 않기 때문입니다. `tests/prepare-npm-package.test.ts` 는 양쪽 절반을 보호하며, 게시되는 각 경로가 패키지 자체의 `tsdown` 진입점이 실제로 출력하는 것인지도 포함합니다 (`--format esm --dts` 는 `.d.ts` 가 아니라 `<entry>.mjs` 와 `<entry>.d.mts` 를 씁니다). 릴리스 워크플로는 두 패키지를 빌드하기는 하지만, 녹색 빌드는 번들이 쓰였다는 것만 증명하지 매니페스트가 게시하는 모든 경로가 그중 하나를 가리킨다는 것은 증명하지 않으므로, 그 밖에는 아무것도 그 드리프트를 알아채지 못합니다.

번들된 사이드카 잠금 파일

`backend/geolibre_server/uv.lock` 은 커밋됩니다(루트 `.gitignore` 는 다른 모든 곳의 `uv.lock` 을 무시하고 이 경로 하나에 대해서만 부정합니다). 그 프로젝트는 데스크톱 설치 프로그램에 번들되어 `src-tauri/src/lib.rs` 에서 `uv run --frozen --project <resource dir>` 로 실행됩니다. 사용자가 쓸 수 없는 디렉터리입니다(`C:\Program Files\...`, `/usr/lib/GeoLibre Desktop/...`). 잠금 없이 배포하면 `uv` 가 해결한 다음 거기에 `uv.lock` 을 쓰려 하고, "Permission denied"로 실패하며 2로 종료합니다. 이는 사용자에게 "Jupyter server exited before it was ready (exit code: 2)"로 도달하고 원인은 보이지 않습니다.

따라서 그 `pyproject.toml` 의 의존성에 대한 모든 편집은 갱신된 잠금(`uv lock --project backend/geolibre_server`)과 함께 들어가야 합니다. CI의 "Check the bundled sidecar lockfile is in sync" 단계(`uv lock --check`)는 둘이 어긋나면 실패합니다.

자격 증명은 재배포 가능한 빌드에 절대 들어가서는 안 됩니다

우리가 다른 사람에게 건네는 모든 것 — 무엇보다 Jupyter 휠 — 은 우리의 자격 증명을 담고 있어서는 안 됩니다. 이 빌드의 세 가지 속성이 이를 잘못하기 쉽게 만들며, 아래의 각 가드는 그중 하나를 막습니다.

1. `apps/geolibre-desktop/vite.config.ts` 는 맨 셸 변수를 `VITE_` 이름으로 연결합니다. `GOOGLE_MAPS_API_KEY` → `VITE_GOOGLE_MAPS_API_KEY`, `MAPBOX_TOKEN` 과 `CESIUM_TOKEN` 도 마찬가지입니다. 로컬 테스트에는 편리하지만, 빌드 머신의 셸이 빌드 입력이 된다는 뜻이기도 합니다.
2. 그래프의 무언가가 `import.meta.env` 를 객체 전체로 읽습니다. Vite는 그런 읽기가 어떤 키를 원하는지 알 수 없으므로 키별 치환을 멈추고 `env` 레코드 전체 — 빌드 머신의 모든 `VITE_` 변수 — 를 그 읽기가 도달하는 모든 청크에 인라인합니다. 우리 것은 `packages/core/src/runtime-env.ts` 였습니다. 고칠 수 없는 것은 `@clerk/shared` 의 `getEnvVariable.mjs` 로, 계산된 이름으로 `import.meta.env[name]` 을 수행하므로 인라인된 레코드가 `ClerkGate-*.js` 청크에 들어갑니다.

3. `python/hatch_build.py` 는 `static/app` 이 이미 존재하고 `GEOLIBRE_FORCE_JS_BUILD` 가 설정되지 않았으면 JS 빌드를 건너뛵니다. 따라서 로컬 `python -m build` 는 이전 `npm run build:embed` 가 스테이징해 둔 것을 JavaScript를 전혀 실행하지 않고 패키징합니다.

현재의 규칙

- `vite.config.ts` 의 `BUILD_ENV_KEYS` 는 번들에 도달해도 되는 `VITE_` 이름의 허용 목록입니다. `pruneBuildEnv()` 는 Vite가 읽기 전에 다른 모든 `VITE_` 변수를 `process.env` 에서 삭제합니다. 새 빌드 시점 변수를 추가한다는 것은 여기에 추가한다는 뜻입니다. 그렇지 않으면 조용히 `undefined`로 해석됩니다.
- `CREDENTIAL_ENV_KEYS` 는 빌드를 실행한 사람으로 인증하고 그 사람에게 과금되는 부분 집합입니다. 재배포 가능한 빌드에서는 이들이 `""` 로 비워집니다(삭제가 아니라 비움이므로 `.env` 파일이 다시 끼워 넣을 수 없음). `GEOLIBRE_EMBED=1` (Jupyter 휠) 또는 `GEOLIBRE_STRIP_CREDENTIALS=1` 일 때 빌드는 재배포 가능합니다. 웹 배포는 재배포 가능하지 **않습니다**: 우리 자신의 리퍼러 제한 키를 쓰는 우리 자신의 사이트이므로 키를 유지합니다.
- 설계상 공개인 식별자는 모든 빌드에 남습니다: Clerk publishable 키, Auth0 클라이언트 ID와 도메인, GEE OAuth 클라이언트 ID, GA 측정 ID. `publish-python.yml` 은 의도적으로 GEE 클라이언트 ID를 휠에 주입합니다.
- `import.meta.env` 를 직접 읽는 대신 `@geolibre/core` 의 `getBuildEnvironment()` 를 우선하세요. 객체 전체 읽기는 도달하는 모든 청크에 대해 원인 (2)를 다시 열며, 타입 시스템의 어떤 것도 이를 알려 주지 않습니다.

제거된 각 자격 증명은 `getRuntimeEnvironment()` 를 통해 해석되며, 이는 Settings → Environment variables 의 `window.__GEOLIBRE_RUNTIME_ENV__` 를 덧씌웁니다. 따라서 휠 사용자는 자신의 토큰을 제공하고, 영향을 받는 표면은 문서화된 대로 성능이 저하됩니다: Mapbox는 베이스맵 API 키 보기에서 입력을 요청하고, 3D 지구본은 지형과 Ion 영상을 잃으며(창 자체는 여전히 동작), Protomaps 베이스맵은 숨겨집니다.

스캔

`scripts/scan-credentials.mjs` 는 **출력**을 검증합니다. 손으로 실행한 빌드는 위의 모든 규칙을 만족하고도 여전히 잘못될 수 있고, 트리에 떨어뜨린 서드파티 자산은 이미 키를 안에 담은 채 도착할 수 있습니다. 어느 쪽도 설정에서 보이지 않습니다. 두 곳에서 실행됩니다:

- `scripts/build-embed.mjs` , `dist-embed` 를 Python 패키지에 스테이징하기 전.
- `python/hatch_build.py` , 휠이나 `sdist`를 패키징하기 전. 위의 오래된 자산 경로 (3)을 포함하며, 이는 JS 쪽 가드로는 다룰 수 없습니다. 이쪽은 Node가 필요 없습니다.

둘 다 `scripts/credential-patterns.json` 을 읽으므로 JS와 Python 스캐너는 드리프트할 수 없습니다. 이 파일은 `sdist`에 강제 포함되어 `sdist` → 휠 빌드도 게이트됩니다. `tests/credential-scan.test.ts` 가 이를 다룹니다.

빌드된 디렉터리를 손으로 스캔하려면:

```
node scripts/scan-credentials.mjs apps/geolibre-desktop/dist-embed
```


의존성 범프 후 스캔이 발동할 때

`credential-patterns.json`의 `allowedValueHashes`는 패턴에 일치하지만 설계상 공개인 값들의 SHA-256을 담습니다. 현재는 CesiumJS의 내장 기본 Ion 토큰으로, `cesium` 안에 배포되어 모든 빌드에 나타납니다. Cesium 업그레이드는 그 토큰을 바꾸므로 가드가 새 토큰에서 발동할 것입니다.

그것은 의도된 동작입니다. 무언가를 하기 전에 페이로드를 디코딩하세요: CesiumJS의 것은 `sub: "CesiumJS"`와 `iss: "https://api.cesium.com"`을 가집니다. 벤더 자체의 토큰임을 확인한 뒤에만 해시를 교체하세요. 디코딩하지 않은 발견을 잠재우려고 해시를 추가하지 마세요. 이 목록의 요점은 짧고 모든 항목이 확인되었다는 것입니다.

`apps/geolibre-desktop/public/plugins/<id>/` 아래의 번들된 플러그인 드롭인도 스캔됩니다. 이들은 여기에 커밋되지 않는 서드파티 빌드 산출물이므로, 그 안의 하드코딩된 키는 허용 목록에 넣는 것이 아니라 그 플러그인 자체 저장소에서 고칩니다. 그런 드롭인을 제거해도 그것이 있을 때 빌드된 `dist/`는 정리되지 않는다는 점에 유의하세요: Vite는 `public/`을 출력에 복사하고 빌드가 실제로 실행될 때만 그 출력을 비웁니다. 재빌드하거나 오래된 디렉터리를 삭제하세요.

생성 파일과 파일 간 동기화

- **오픈 데이터 갤러리.** `gallery.md`와 `demos.md`의 "Open data showcase" 티저(`<!-- demo-gallery-teaser -->` 마커 사이)는 `scripts/demo-gallery.json`에서 생성됩니다: 테마, 추천 데모, 그리고 데모별 항목 하나(슬러그, 제목, 테마 태그, 캡션, 데이터 크레딧). JSON을 편집하고 `npm run gallery`를 실행하고 두 파일을 커밋하세요. `npm run gallery:check` (`npm run ci`의 일부)는 드리프트 시 실패합니다. 각 항목의 스크린샷은 이미 `https://assets.geolibre.app/images/<slug>.webp` (`opengeos/geolibre-assets`)에 있어야 하고, `theme`는 `share.geolibre.app` 프로젝트의 태그와 일치해야 합니다.
- **내장 플러그인 참조.** `user-guide/plugins.md`의 표(`<!-- plugin-reference:start/end -->` 마커 사이)는 `scripts/gen-plugin-reference.mjs`가 생성합니다. 플러그인 목록과 순서는 `usePlugins.ts`의 `BUILT_IN_PLUGINS`에서, 각 플러그인의 `id`와 `name`은 `packages/plugins/src`의 정의에서, 표시 이름은 `en.json`의 `toolbar.plugin`에서, 메뉴 위치는 `PluginsMenu.tsx`와 `WEB_SERVICE_PLUGIN_IDS / DGGs_PLUGIN_IDS` 그룹에서, `?plugin=` 링크 이름은 `plugin-deep-link.ts` 규칙에서 읽습니다. 한 줄 설명(그리고 `PluginsMenu`가 숨기는 플러그인의 `menu`)만 `scripts/plugin-reference.json`에 손으로 작성됩니다. 내장 플러그인을 등록, 이름 변경, 재그룹화한 뒤에는 항목을 추가하고 `npm run plugins:docs`를 실행하고 두 파일을 커밋하세요. `tests/plugin-reference-docs.test.ts`와 `npm run plugins:docs:check` (`npm run ci`의 일부)는 드리프트 시 실패하며, 생성기 자체는 설명 없는 등록된 플러그인에서 실패합니다.
- **처리 도구 메타데이터.** 이름, 설명, 그룹 라벨, 매개변수 라벨/도움말, 선택 옵션은 `i18n`에 접근할 수 없는 레지스트리에 있으므로, 대화 상자는 `apps/geolibre-desktop/src/lib/processing-tool-i18n.ts`를 통해 이를 해석하고 레지스트리 자체의 영어 문자열로 폴백합니다. 네 개의 작은 번들 레지스트리에 대해서는 `en.json`의 `processing.toolMeta / processing.toolGroup` 하위 트리가 번역자가 작업하는 **생성된** 기준입니다. 그 도구, 매개변수, 선택 옵션 중 하나를 추가하거나 이름을 바꾼 뒤에는 `npm run i18n:tools`를 실행하고 결과를 커밋하세요. CI는 드리프트 시 실패합니다. Whitebox의 훨씬 큰 메타데이터는 의도적으

로 모든 번들 로케일에서 빠져 있으며, `languages.geolibre.app` (또는 로컬 파일 가져오기)의 선택적이고 검증된 팩에서 옵니다. `processing.toolMeta.whitebox`, `processing.whitebox.categories`, `menuTool`, `menuSubcategory` 를 번들 로케일에 다시 추가하지 마세요.

- **에이전트 스킬.** `skills/geolibre/` 는 사용자 대상 에이전트 스킬입니다. `SKILL.md` 와 `references/` 로 구성되며, 외부 AI 에이전트에게 `geolibre-mcp`, Python 패키지, 또는 손으로 쓴 JSON을 통해 `.geolibre.json` 프로젝트를 작성하는 법을 가르칩니다. GeoLibre 자체를 작업하는 기여자를 위한 것이 아닙니다. 다른 곳에 있는 것들을 다시 서술합니다: MCP 도구 표면(`python/src/geolibre/mcp/server.py`), 베이스맵/색상 램프/범례 카탈로그(`python/src/geolibre/basemaps.py`, `color_ramp.py`, `legends.py`), 프로젝트 스키마([프로젝트 형식](#)), 그리고 임베드 매개변수([임베딩](#)). `python/tests/test_agent_skill.py` 는 기계적으로 확인할 수 있는 부분을 보호합니다: 등록된 모든 MCP 도구가 도구 참조에 나타나야 하고, 스킬이 이름 대는 모든 도구와 Map 메서드가 존재해야 하며, 베이스맵, 색상 램프, 범례 프리셋, 레이어 타입, 프런트매터, 참조 파일 목록이 소스와 일치해야 합니다. 산문은 확인할 수 없으므로, 바뀐 크기 상한, 한도, 동작상 주의 사항은 여전히 손으로 옮겨야 합니다. 같은 PR에서 스킬을 갱신하세요. 오래된 주의 사항은 에이전트를 더 이상 동작하지 않는 경로로 보내며, 어디서도 실패가 나지 않습니다.

<https://geolibre.app/citation/>

인용 방법

연구, 교육, 소프트웨어 또는 기타 작업에 GeoLibre를 사용하신다면 인용해 주세요. 인용은 다른 사람들이 프로젝트를 찾고, 기여자를 인정하며, 무료 오픈 소스 지리공간 소프트웨어의 지속적인 개발을 지원하는 데 도움이 됩니다.

DOI 10.5281/zenodo.20785400

GeoLibre는 [Zenodo](#)에 보관되며, Zenodo는 모든 릴리스에 대해 디지털 객체 식별자(DOI)를 발급합니다. 위의 DOI는 **컨셉트 DOI**로, 항상 최신 버전으로 연결됩니다. 특정 버전을 인용하려면 [Zenodo 레코드](#)를 열고 해당 릴리스에 표시된 DOI를 복사하세요.

권장 인용

Wu, Q. (2026). GeoLibre: A lightweight, cloud-native GIS platform for visualizing, exploring, and analyzing geospatial data. Zenodo. <https://doi.org/10.5281/zenodo.20785400>

BibTeX

```
@software{wu_geolibre,
  author   = {Wu, Qiusheng},
  title    =
  {{GeoLibre: A lightweight, cloud-native GIS platform for visualizing, exploring, and analyzing
  geospatial data}},
  year     = {2026},
  publisher = {Zenodo},
  doi      = {10.5281/zenodo.20785400},
  url      = {https://doi.org/10.5281/zenodo.20785400}
}
```

APA

Wu, Q. (2026). GeoLibre: A lightweight, cloud-native GIS platform for visualizing, exploring, and analyzing geospatial data [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.20785400>

인용 메타데이터 파일

저장소에는 `CITATION.cff` 파일이 포함되어 있으므로, GitHub의 **"Cite this repository"**(이 저장소 인용하기) 버튼([저장소 페이지](#)의 사이드바에 있음)을 사용하여 바로 쓸 수 있는 APA 또는 BibTeX 항목을 복사할 수도 있습니다. Zotero와 같은 참고 문헌 관리 도구와 `cffconvert` 같은 도구도 같은 파일을 읽을 수 있습니다.

의존성 인용

GeoLibre는 많은 오픈 소스 프로젝트 위에 구축되어 있습니다. 작업이 특정 구성 요소에 크게 의존한다면 해당 프로젝트도 함께 인용하는 것을 고려해 주세요. GeoLibre를 가능하게 하는 프로젝트의 전체 목록은 [감사의 글](#) 페이지를 참조하세요.

<https://geolibre.app/acknowledgements/>

감사의 글

GeoLibre는 오픈 소스 지리공간 및 웹 커뮤니티라는 거인의 어깨 위에 서 있습니다. 이 페이지는 GeoLibre를 가능하게 한 프로젝트, 조직, 개인의 노고를 기립니다. 혹시 실수로 누락된 분이 있다면 바로잡을 수 있도록 [이슈를 열어](#) 주세요.

오픈 소스 프로젝트

GeoLibre는 거의 전적으로 자유 오픈 소스 소프트웨어 위에 구축되었습니다. 다음 프로젝트(그리고 이들이 의존하는 수많은 전이적 종속성)의 관리자와 기여자 여러분께 감사드립니다.

지도 작성 및 렌더링

- [MapLibre GL JS](#) — 핵심 지도 렌더링 엔진.
- [CesiumJS](#) — 3D 지구본 렌더링 엔진.
- [deck.gl](#) 및 [vis.gl](#) / Open Visualization 툴킷 — 래스터, 포인트 클라우드, 3D 오버레이.
- [Three.js](#) — 지구본 및 대기 효과에 사용되는 3D 렌더링.
- [PMTiles](#) 및 [Protomaps](#) — 단일 파일 타일 아카이브.
- [OpenFreeMap](#) 및 [CARTO](#) 베이스맵, 그리고 그 기반 데이터를 제공하는 더 넓은 [OpenStreetMap](#) 커뮤니티.
- [Overture Maps Foundation](#) — 오픈 지도 데이터.

데이터, 포맷, 분석

- [DuckDB](#) 및 [Spatial](#) 확장이 포함된 [DuckDB-WASM](#) — 브라우저 내 공간 SQL 및 벡터 변환.
- [Apache Sedona](#) 및 [PGLite](#)(PostGIS 포함) — 추가 브라우저 내 SQL 엔진.
- [Turf.js](#) — 클라이언트 측 벡터 지오메트리 도구.
- [GDAL/OGR](#), [GeoTIFF.js](#), [proj4js](#), [shpjs](#), [FlatGeobuf](#) — 지리공간 데이터 포맷 및 채투영.
- [GeoParquet](#) 및 [Apache Arrow](#) / Parquet 생태계.
- [STAC](#) — SpatioTemporal Asset Catalog 사양 및 도구.
- [Zarr](#) 및 [CarbonPlan](#) — 클라우드 네이티브 다차원 배열 접근.
- [Icechunk](#) 및 [icechunk-js](#) — 브라우저에서 읽는 Zarr용 트랜잭션 스토리지.
- [H3](#) — 육각형 계층적 공간 인덱싱.

Python 사이드카 및 노트북

- [FastAPI](#), [rasterio](#), [GeoPandas](#), [Shapely](#), [pyproj](#) — 선택적 Python 사이드카.
- [WhiteboxTools](#) — Whitebox 지오프로세싱 툴박스.
- [segment-geospatial \(SamGeo\)](#) 및 Meta AI의 [Segment Anything](#) — AI 분할.

- [Jupyter](#), [JupyterLite](#), [Pyodide](#), [anywidget](#) — 임베디드 노트북 경험.
- [leafmap](#) — [geolibre](#) Python 패키지의 API 스타일과 영감.

앱 프레임워크 및 UI

- [Tauri](#) — 크로스 플랫폼 데스크톱 및 모바일 셸.
- [React](#), [Vite](#), [TypeScript](#).
- [Zustand](#) — 애플리케이션 상태 관리.
- [Radix UI](#), [shadcn/ui](#), [Tailwind CSS](#), [Lucide](#) — UI 프리미티브 및 아이콘.
- [react-i18next](#) — 국제화.

Atmospheric Effects(대기 효과) 플러그인(심우주 배경, 시차 별발, 혜성, 지구본 대기 헤일로)은 [Leonel Dias](#)의 글 [Globe atmosphere, halo, and comets](#)에서 기법과 시각 디자인을 가져왔습니다 — 계층화된 Canvas 2D 접근 방식, 헤일로 그라디언트와 "screen" 블렌드, 기울기(pitch) 아래에서도 헤일로를 정렬 상태로 유지하는 림(limb) 샘플링, 그리고 별발/혜성 매개변수가 그것입니다.

커뮤니티 기여자

GeoLibre는 직접 써 보고, 버그를 보고하고, 기능을 요청하고, 다른 사람의 시작을 돕는 분들에 의해 만들어집니다. 이슈를 열고, 토론을 시작하고, 자신의 커뮤니티에 GeoLibre를 소개해 주신 모든 분께 감사드립니다.

- [Ryanphoenix](#) — 이슈 보고, 피드백, 개선을 포함하여 프로젝트에 많은 소중한 기여를 해 주셨습니다.

베타 테스터

- [René van der Velde](#), 네덜란드 — 숙련된 QGIS 실무자로서, GeoLibre를 접근하기 쉬운 입문~중급 GIS 옵션으로 적극 알려 주셨고, 초기 테스트 기간 동안 사려 깊고 상세한 버그 보고와 기능 요청으로 기여해 주셨습니다.

참여하기

여기에 이름을 올리고 싶으신가요? 잘 정리된 버그 보고서를 제출하는 것부터 플러그인을 작성하는 것까지, 크고 작은 모든 기여를 환영합니다. 시작하려면 [기여 가이드](#)를 참조하세요.

프로젝트를 재정적으로 후원하고 싶다면 [후원자 되기](#)를 참조하세요.

<https://geolibre.app/privacy/>

개인정보 처리방침

최종 업데이트: 2026년 10월 6일

GeoLibre Desktop("GeoLibre", "앱")은 OpenGeos 커뮤니티가 개발한 오픈 소스 데스크톱 GIS 애플리케이션입니다. 이 방침은 앱이 사용자의 데이터를 어떻게 처리하는지, 그리고 저희 웹사이트(geolibre.app 및 web.geolibre.app)가 사용자의 데이터를 어떻게 처리하는지 설명합니다.

요약

GeoLibre는 사용자의 기기에서 로컬로 실행됩니다. 계정이 필요하지 않으며, 앱 자체는 분석 데이터, 원격 측정 데이터, 사용 데이터를 수집하지 않습니다. 사용자의 지리공간 데이터와 프로젝트는 사용자가 직접 공유하거나 내보내지 않는 한 기기에 남아 있습니다. 플러그인 레지스트리에서 플러그인을 설치하면, 레지스트리는 앱이 이미 보내는 요청을 바탕으로 플러그인 사용을 익명으로 집계합니다. 아래의 [플러그인 레지스트리의 플러그인](#)을 참조하세요.

저희 웹사이트는 앱과 별개입니다. geolibre.app과 web.geolibre.app은 아래 [웹사이트 분석](#)에 설명된 대로 Google Analytics로 페이지 방문을 측정합니다.

앱 사용에는 [서비스 약관](#)도 적용됩니다.

로컬에서 처리되는 데이터

GeoLibre에서 열고, 만들고, 분석하는 파일과 프로젝트(벡터 및 래스터 데이터셋, 프로젝트 파일 등)는 사용자의 컴퓨터에서 로컬로 처리됩니다. GeoLibre는 이를 개발자나 어떤 서버에도 업로드하지 않습니다. 선택적 보조 프로세스(사용자 컴퓨터의 127.0.0.1에서 실행되는 Python 구성 요소)가 일부 지오프로세싱을 전적으로 기기 내에서 수행합니다.

제3자 서비스로 전송되는 데이터(선택적 기능)

특정 선택적 기능을 사용하면 GeoLibre는 해당 제3자 서비스에 직접 요청을 보냅니다. 이러한 서비스는 사용자 기기의 IP 주소와 요청 내용을 받으며, 각자의 개인정보 처리방침이 적용됩니다:

- **베이스맵 / 지도 타일:** 현재 지도 뷰를 사용하여 베이스맵 제공업체(예: OpenFreeMap 및 CARTO)에 타일을 요청합니다.
- **검색 / 지오코딩:** 검색한 지명이나 주소가 구성된 지오코딩 제공업체로 전송됩니다.
- **AI 어시스턴트:** 사용하는 경우, 입력한 프롬프트와 함께 프로젝트에 현재 로드된 레이어에 관한 메타데이터(레이어 이름 및 속성 필드 이름)가 구성된 AI/LLM 제공업체로 전송됩니다.
- **클라우드 데이터 카탈로그:** Microsoft Planetary Computer나 Google Earth Engine 같은 서비스에 연결하면 사용자의 쿼리가 해당 서비스로 전송됩니다.

- **실시간 협업:** 공유 세션에 참여하면 사용자의 프로젝트 데이터(로컬에서 로드한 파일의 GeoJSON 포함)가 OpenGeos 프로젝트가 운영하는 릴레이 서버(현재 Cloudflare에 호스팅됨)를 거쳐 다른 참가자와 공유됩니다. 릴레이는 나중에 참여하는 사람이 세션을 로드할 수 있도록 최신 프로젝트 스냅샷을 보관하며, 세션이 끝나면 이를 삭제합니다.
- **프로젝트 공유:** Project → Share(프로젝트 → 공유)를 사용하면 프로젝트 파일이 OpenGeos 프로젝트가 운영하는 share.geolibre.app에 업로드됩니다. 로컬 파일에서 추가한 벡터 레이어는 업로드되는 파일에 포함되며, 그 밖의 로컬 파일은 업로드되지 않습니다.

GeoLibre는 이러한 제3자 서비스를 통제하지 않습니다. 각 서비스가 데이터를 어떻게 처리하는지는 해당 개인정보 처리방침을 확인하세요.

플러그인 레지스트리의 플러그인

Settings → Manage Plugins(설정 → 플러그인 관리)는 OpenGeos 프로젝트가 운영하는(현재 GitHub Pages와 Cloudflare에 호스팅됨) <https://plugins.geolibre.app>의 GeoLibre 플러그인 레지스트리에 있는 플러그인을 나열합니다. 이 대화 상자를 열면 레지스트리를 다운로드하며, GeoLibre가 시작될 때마다 레지스트리의 차단 목록(안전상의 이유로 내려진 플러그인 목록)과 레지스트리에서 설치한 플러그인의 파일을 다운로드합니다. 다른 웹 서버와 마찬가지로 레지스트리는 사용자의 IP 주소와 어떤 파일을 요청했는지 볼 수 있습니다.

레지스트리는 이러한 요청을 이용해 각 플러그인이 얼마나 자주 사용되는지를 플러그인 카탈로그에 공개 카운트로 표시합니다. 기록하는 것은 다음 이상이 아닙니다:

- 플러그인별 일일 실행 횟수;
- 주간 고유 사용자 수를 세기 위해, 플러그인과 사용자 IP 주소(IPv6의 경우 /64 네트워크)를 매주 바뀌는 비밀 키로 해시한 값. 이 해시는 해당 비밀 키 없이는 IP로 되돌릴 수 없고, 주 또는 플러그인 간에 대조할 수 없으며, 주가 끝나면 삭제됩니다. 주간 합계만 보관됩니다.

IP 주소, 계정, 그 밖에 사용자나 프로젝트에 관한 어떤 정보도 저장되지 않습니다. 앱은 이를 위해 추가로 아무것도 보내지 않습니다. 카운트는 앱이 이미 수행하는 플러그인 다운로드에서만 산출됩니다. 자체 플러그인 레지스트리를 설정한 배포(`deployment.json`의 `plugins.registryUrl`)는 해당 레지스트리가 가리키는 플러그인 파일에 대해서만 plugins.geolibre.app에 접속합니다. 이러한 다운로드도 다른 것과 똑같이 집계되므로, 집계를 전혀 원하지 않는 배포는 플러그인 파일을 직접 호스팅해야 합니다. 자세한 내용: <https://plugins.geolibre.app/registry/#usage-statistics>.

웹사이트 분석

이 섹션은 설치된 앱이 아니라 저희 웹사이트에 관한 내용입니다.

문서 사이트(<https://geolibre.app>)와 호스팅된 브라우저 앱 (<https://web.geolibre.app> 및 <https://geolibre.app/demo/>의 데모)은 Google Analytics 4를 사용하여 페이지 방문 수를 세고 어떤 페이지가 사용되는지 확인합니다. Google은 사용자의 IP 주소, 현재 페이지, 참조 페이지, 일반적인 기기/브라우저 정보를 받으며, 사용자의 브라우저에 자체 쿠키나 유사한 식별자를 설정합니다. 여기에는 [Google의 개인정보 처리방침](#)이 적용됩니다.

분석 도구가 결코 보지 못하는 것은 사용자가 작업하는 지리공간 데이터입니다. 레이어, 파일, 프로젝트 내용, 쿼리, 좌표는 브라우저에서 처리되며 Google로 전송되지 않습니다. 보고되는 페이지 주소는 사이트와 경로(예: <https://web.geolibre.app/>)로 잘라내고 쿼리 문자열은 제거합니다. GeoLibre 링크의 매개변수에는 프로젝트 URL, 인라인 데이터셋, 협업 세션 식별자가 포함될 수 있기 때문입니다.

콘텐츠 차단기나 브라우저의 추적 방지 기능으로 www.googletagmanager.com 을 차단하면 수집을 거부할 수 있습니다. 분석이 차단되어도 GeoLibre Web은 정상적으로 작동합니다.

분석은 저희가 호스팅하는 사이트에서만 실행됩니다. 데스크톱 앱, Jupyter 위젯, Docker 이미지는 결코 이를 로드하거나 실행하지 않습니다. 측정 ID는 빌드 시점에 제공되며 해당 빌드에는 존재하지 않으므로, Google에 아무것도 요청되지 않고 아무것도 보고되지 않습니다. 직접 빌드하여 호스팅하는 사본도 마찬가지이며, 다만 빌드 시 의도적으로 자신의 측정 ID를 제공하는 경우에는 그 분석이 저희가 아닌 사용자의 것이 됩니다 ([자체 호스팅](#) 참조).

개인 정보

GeoLibre는 계정 생성을 요구하지 않으며 이름, 이메일 주소 또는 그와 유사한 식별 정보를 수집하지 않습니다. 위에 설명한 선택적 기능의 네트워크 요청에는 사용자 기기의 IP 주소가 필연적으로 포함되며, 이를 수신하는 서비스로 로그로 남길 수 있습니다.

아동

GeoLibre는 만 16세 이상의 사용자를 대상으로 하는 전문 GIS 도구입니다. 아동을 대상으로 하지 않으며, 저희는 아동의 개인 데이터를 고의로 수집하지 않습니다.

사용자의 선택

GeoLibre는 데이터를 로컬에 저장하므로 사용자가 데이터를 통제하며 언제든지 프로젝트와 파일을 삭제할 수 있습니다. 제3자 서비스로 데이터를 보내지 않으려면 위에 나열된 선택적 온라인 기능을 사용하지 마세요.

이 방침의 변경

저희는 이 방침을 수시로 업데이트할 수 있습니다. 최신 버전은 항상 <https://geolibre.app/privacy/>에서 확인할 수 있습니다.

문의

질문은 <https://github.com/opengeos/GeoLibre/issues>에서 프로젝트에 문의할 수 있습니다.

<https://geolibre.app/terms/>

서비스 약관

최종 업데이트: 2026년 7월 10일

GeoLibre("앱")는 OpenGeos 커뮤니티가 개발한 무료 오픈 소스 GIS 애플리케이션입니다. 데스크톱, 웹, 임베디드 빌드를 포함하여 GeoLibre를 다운로드, 설치 또는 사용함으로써 귀하는 본 서비스 약관("약관")에 동의하게 됩니다. 동의하지 않는 경우 앱을 사용하지 마세요.

서비스

GeoLibre를 사용하면 지리공간 데이터를 보고, 만들고, 분석하고, 공유할 수 있습니다. 앱은 귀하의 기기에서 로컬로 실행되며 계정이 필요하지 않습니다. 일부 선택적 기능은 제3자 서비스(예: 베이스맵 제공업체, 지오코딩 서비스, AI/LLM 제공업체, Google Earth Engine과 같은 클라우드 데이터 카탈로그)에 연결합니다. 이러한 기능은 [개인정보 처리방침](#)에 설명되어 있습니다.

라이선스

GeoLibre는 프로젝트 저장소 <https://github.com/opengeos/GeoLibre/blob/main/LICENSE>에 게시된 MIT 라이선스에 따라 배포되는 오픈 소스 소프트웨어입니다. 소프트웨어의 사용, 수정 및 재배포에는 해당 라이선스가 적용됩니다. 본 약관은 소프트웨어 라이선스를 보완하며 대체하지 않습니다.

허용되는 사용

귀하는 GeoLibre를 합법적이고 책임 있게 사용하는 데 동의합니다. 특히 다음 행위를 하지 않는 데 동의합니다:

- 적용되는 법률이나 규정을 위반하거나 타인의 권리를 침해하기 위해 앱을 사용하는 행위;
- 사용 권한이 없는 데이터에 접근하거나 이를 저장 또는 배포하기 위해 앱을 사용하는 행위;
- 앱이 연결하는 제3자 서비스를 오용하거나 과부하를 일으키거나 무단 접근을 시도하는 행위, 또는 해당 서비스 자체의 약관을 위반하는 방식으로 그 서비스를 사용하는 행위; 또는
- 앱이나 앱이 연결하는 서비스의 보안 또는 인증 메커니즘을 제거, 비활성화 또는 우회하는 행위.

Google Earth Engine 및 기타 제3자 서비스

GeoLibre의 선택적 기능을 통해 귀하는 **Google Earth Engine**을 포함한 제3자 서비스에 자신의 자격 증명으로 인증하고 접근할 수 있습니다.

- 제3자 서비스에 연결할 때 귀하는 해당 제공업체의 **귀하 자신의 계정**으로 연결하며, 귀하의 사용에는 본 약관에 더하여 **해당 제공업체의 서비스 약관 및 개인정보 처리방침**이 적용됩니다. Google Earth Engine의 경우 여기에는 **Earth Engine 서비스 약관**과 **Google API 서비스 약관**이 포함됩니다.
- GeoLibre는 귀하가 시작한 Earth Engine 요청을 수행하는 데 필요한 액세스 토큰을 귀하를 대신하여, 귀하의 동의하에 얻기 위해서만 Google OAuth를 사용합니다. GeoLibre는 귀하의 데이터를 판매하지 않으며, Google 사용자 데이터를 광고 목적이나 귀하가 요청한 기능을 제공하는 것 이외의 어떤 목적으로도 사용하지 않습니다. 무엇이 어디로 전송되는지에 대한 자세한 내용은 **개인정보 처리방침**을 참조하세요.
- 귀하는 앱을 통해 제3자 서비스를 사용하는 데 필요한 권리와 할당량을 보유하고 있는지 확인할 책임이 있으며, 접근하는 데이터에 대한 해당 서비스의 사용 제한과 라이선스 조건을 준수할 책임이 있습니다.

GeoLibre는 이러한 제3자 서비스를 통제하지 않으며, 그 가용성, 콘텐츠 또는 행위에 대해 책임지지 않습니다.

귀하의 콘텐츠

GeoLibre는 귀하가 앱으로 열거나 만든 데이터, 파일 또는 프로젝트에 대한 소유권을 주장하지 않습니다. 귀하는 귀하의 콘텐츠에 대한 모든 권리를 보유합니다. 대부분의 처리는 귀하의 기기에서 로컬로 이루어집니다. 실시간 협업이나 제3자 통합과 같은 온라인 기능을 활성화하면, **개인정보 처리방침**에 설명된 대로 해당 데이터가 그 서비스로 전송될 수 있습니다. 귀하는 콘텐츠를 백업하고 이를 합법적으로 사용할 책임이 있습니다.

보증의 부인

GeoLibre는 상품성, 특정 목적에의 적합성, 정확성 및 비침해에 대한 보증을 포함하되 이에 국한되지 않는, 명시적이든 묵시적이든 **어떠한 종류의 보증도 없이 "있는 그대로" 그리고 "이용 가능한 상태로"** 제공됩니다. 개발자는 앱이 중단 없이, 오류 없이, 또는 유해한 구성 요소 없이 작동할 것임을, 혹은 앱이 생성하는 분석이나 출력이 정확하거나 특정 목적에 적합할 것임을 보증하지 않습니다. 귀하는 자신의 책임하에 앱을 사용하고 그 출력에 의존합니다.

책임의 제한

법률이 허용하는 최대 범위 내에서, OpenGeos 커뮤니티, 프로젝트 관리자 또는 기여자는 그러한 손해의 가능성을 통보받은 경우라 하더라도, 귀하의 GeoLibre 사용(또는 사용 불능)으로 인해 또는 이와 관련하여 발생하는 간접적, 부수적, 특별, 결과적 또는 징벌적 손해, 혹은 데이터, 이익 또는 영업권의 손실에 대해 어떠한 경우에도 책임지지 않습니다.

앱 및 본 약관의 변경

GeoLibre는 활발히 개발 중이며, 기능이 언제든지 추가, 변경 또는 제거될 수 있습니다. 저희는 또한 본 약관을 수시로 업데이트할 수 있습니다. 최신 버전은 항상 <https://geolibre.app/terms/>에서 확인할 수 있습니다. 변경 사항이 발효된 후에도 앱을 계속 사용하면 개정된 약관에 동의한 것으로 간주됩니다.

문의

본 약관에 관한 질문은 <https://github.com/opengeos/GeoLibre/issues>에서 프로젝트에 문의할 수 있습니다.